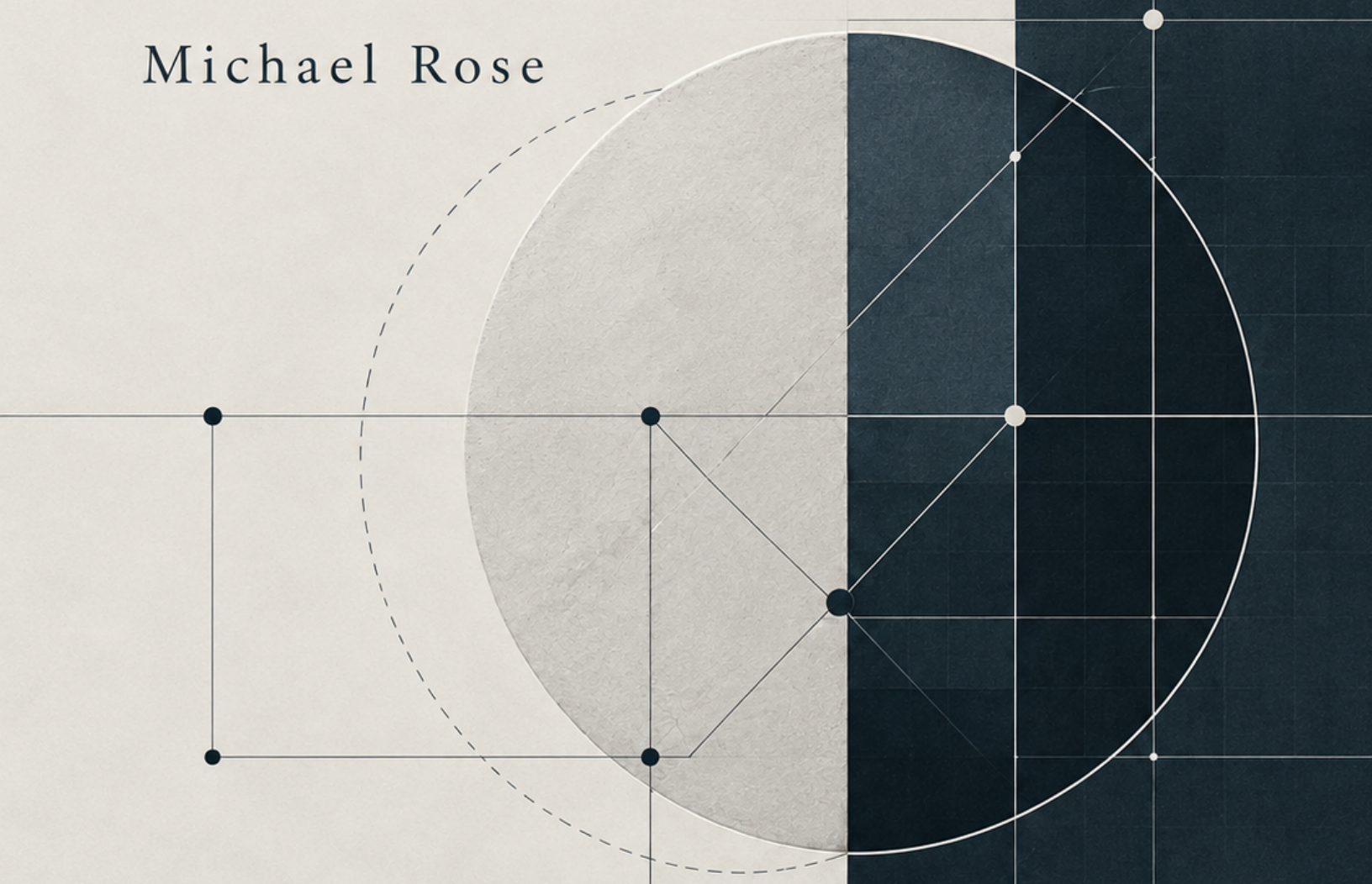
Gesso
Unified Design

Canonical architecture, assurance model, and reference application

Michael Rose



2026-09-22

Part I — Foundations

Canonical document identity and provenance

This document specifies the intended canonical r4 Gesso unified design. It is an end-state specification of the required architecture and behavior, not a claim that the current implementation already satisfies those requirements.

R4 is assembled from twelve ordered parts. The complete reviewed assembly supersedes canonical r3. During sectional delivery, an individual part specifies only its stated scope; it must not be presented as a complete replacement for the entire r3 specification. The revision remains marked `r4-draft` until all parts and their cross-references have been reconciled.

Historical source plans remain provenance. They do not carry independent normative revision identities inside this specification.

The principal source lineage is:

Source	Revision / role
Gesso Choreography Design	v4.5; foundational choreography semantics
Gesso Provable Hardening Plan	v7.376; compositional rejection and proof direction
Gesso/HumanHelp Biff 2 Migration Plan	native Biff 2 substrate and lifecycle integration
Gesso/HumanHelp Preflight Recovery Plan	v662; no-silent-downgrade and production-path closure
Gesso Unified Design	canonical r2; prior consolidation
Gesso Unified Design	canonical r3; direct textual predecessor
Gesso Unified Design Revision and Validation Plan	plan revision 2; r3 review discipline

Canonical r4 preserves those commitments except where this document explicitly clarifies, reconciles, or supersedes wording. A change in implementation does not by itself change the requirement.

Revision scope

R4 preserves the semantic commitments of r3 while making their construction, runtime, acceptance, and assurance boundaries more uniform. Its requirements remain owned by the sections that define them; this summary is a navigation aid.

The revision emphasizes:

- owner-relative admission for asynchronous events, including shared events;
- scope-preserving acquisition requirements and precisely stated resource bounds;
- independent required-capability acceptance alongside assembly closure and production-path enforcement;
- participant-relative command uncertainty after local retirement or lost delivery;
- explicit HumanHelp adoption policy and testable product outcomes;
- production/formal correspondence classified by the evidence actually supplied;
- a single normative home for each obligation, with examples that preserve it;
- connected reference realizations and controlled defects on production paths.

Unresolved product choices remain explicit. A completion claim that depends on one of those choices is unavailable until the choice and its acceptance conditions are specified. Generic Gesso semantics do not silently choose HumanHelp business policy.

Document organization and sectional assembly

The document has the following logical parts. The two-digit delivery order is also the concatenation order.

Order	Part	Scope
01	I — Foundations	Purpose, ownership direction, normative interpretation, correctness scopes, assurance, evidence vocabulary, construction principles
02	II — Semantic model	Identity, time, resource instances, trust, knowledge, provenance, observation evidence, agreement
03	III — Choreography language and projection	Choreography, operation types, Malli, branch knowledge, projection, long logical workflows
04	IV — Authoritative operations and progression	Model integration, FX, atomic transitions, delivery debt, XTDB basis and scoped progression, command/execution identity
05	V — Provisional state and executable presentation	Optimism, continuity, views, portable execution, ExecutablePlan and diagnostic sidecars
06	VI — Browser realization and recovery	Adapter refinement, owner-relative event admission, acquisition coordination, stale plans, managed effects, absence, failure and convergence
07	VII — Whole-application construction	ApplicationAssembly, closure, artifacts, routes, capabilities, settlement/acquisition, static and dynamic preflight, Biff lifecycle, diagnostics
08	VIII — HumanHelp proving application	Production model ownership, example integration, lifecycle, creation, adoption, board behavior, product decisions and independent acceptance
09	IX — Compositional certification	Semantic interfaces, typed rules, certificate hierarchy, dependency identity, proof kernel, mutation corpus, HumanHelp proof pilot
10	X — Formal correspondence and system boundaries	Assurance targets, theorem families, formalization, production correspondence, TCB, validation, multi-node semantics
11	XI — Release and completion	Incremental builds, release evidence, non-goals, progress measures and the four completion scopes
12	XII — Implementation order and final invariants	Ordered stages, dependency relationships, cross-stage acceptance and final unified principle

Only Part I carries the document-wide Org title, date, options, and revision metadata. Subsequent parts begin with their own top-level part heading and use subordinate headings within that part. Concatenation introduces no additional document headers, include directives, or independent normative documents.

Internal links may refer forward to sections in later parts. Such references are resolved against the complete assembly. Every referenced anchor must occur exactly once in that assembly. Historical Part references are replaced by the corresponding r4 section links or current part names during revision.

Cross-cutting requirement references

The following links are a navigation aid, not a second semantic registry. The linked sections are the authoritative homes of the corresponding requirements.

Requirement / decision	Authoritative section
Assurance classification	Assurance model
Semantic category and admission	Semantic category, representation, and admission
Trusted-code boundary	Trusted application code and accidental misuse
Observation evidence	Observation claims, receipts, and authority
Progression scope	Progression requirements are scoped
Advisory discharge	Advisory obligations and authoritative acquisition
Participant-relative command uncertainty	Participant-relative command uncertainty
Provisional resolution	Provisional resolution and local retirement
Scoped authoritative installation	Scoped authoritative installation
Application assurance claims	Three separate application assurance claims
Assembly construction phases	Assembly construction and admitted generations
Production-path enforcement	Production-path enforcement evidence
HumanHelp selective adoption	Selective board adoption
HumanHelp Refresh coverage	Refresh coverage and concurrent activity
HumanHelp remote lifecycle policy	Remote lifecycle changes

Purpose

This document specifies the intended end-state design of Gesso choreography, proof, execution, application assembly, and its use by HumanHelp.

The goal is not merely to provide a convenient syntax for distributed state machines. Gesso should provide a useful correctness boundary for distributed application operations. It should make coherent programs straightforward to express, malformed programs difficult to assemble, and contradictions visible as early and as locally as the available information permits.

The central design target is:

> A Gesso application may still express a coherent but domain-wrong business rule, but it should be difficult to assemble a structurally contradictory distributed program that appears to work.

Gesso therefore distinguishes two very different failure classes.

A **well-formed but domain-wrong** program may express the wrong policy while remaining internally coherent. For example, an application may validly encode a rule that an Organization chose incorrectly. Gesso cannot infer product intent merely from syntax.

A **malformed** program contradicts its own declared semantic structure. Examples include:

- a role consuming a fact it cannot know;
- a browser affordance naming an operation that has no projected realization;
- an operation requiring authority that no trusted server path can establish;
- a provisional trajectory with no permitted resolution or recovery path;
- a projected plan whose required semantic dependency no longer matches the certified dependency used to produce it;
- a lifecycle operation silently degrading to a weaker ordinary request because required semantic metadata is absent;
- a proof or diagnostic artifact being accepted as runtime authority;
- a stale callback mutating a newer execution merely because both refer to the same logical DOM target.

Those are framework-recognizable contradictions and should be rejected as early and descriptively as feasible.

The distinction is easier to see in a small but nontrivial application slice. Suppose an expense system models submitted expenses and a current approval policy. The following code is illustrative rather than a frozen public API, but it shows the amount of semantic structure Gesso should be able to reason about without requiring the application author to reproduce framework internals.

```
(defschema Money
  [:decimal
   {:semantic/type :money
    :semantic/currency :usd}])

(defschema Expense
  [:map
   [:expense/id uuid?]
   [:expense/employee-id uuid?]
   [:expense/department keyword?]
   [:expense/amount Money]
   [:expense/status
    [:enum :draft
           :submitted
           :approved
           :rejected
           :needs-finance]]])

(defschema ApprovalPolicy
  [:map
   [:policy/department keyword?]
   [:policy/manager-limit Money]
   [:policy/finance-threshold Money]])

(defrule manager-may-approve?
  {:inputs {:expense Expense
            :policy ApprovalPolicy}
   :returns :boolean}
  [:and
   [:= [:expense/status :expense] :submitted]
   [:= [:expense/department :expense]
       [:policy/department :policy]]
   [:< [:expense/amount :expense]
       [:policy/manager-limit :policy]]])

(defrule needs-finance?
  {:inputs {:expense Expense
            :policy ApprovalPolicy}
   :returns :boolean}
  [:and
   [:= [:expense/status :expense] :submitted]
   [:≥ [:expense/amount :expense]
       [:policy/finance-threshold :policy]]])

;; Suppose this is what the Organization actually intends.
(def accounting-policy
  {:policy/department :accounting
   :policy/manager-limit 500M
   :policy/finance-threshold 500M})
```

Now imagine that the application author accidentally changes only the policy data:

```
(def accounting-policy
  {:policy/department :accounting
   :policy/manager-limit 5000M
   :policy/finance-threshold 5000M})
```

The program may still be perfectly coherent. `5000M` is a valid monetary value. The rule is well typed. The operation may project correctly. The authoritative transaction may apply the rule exactly as declared. Gesso has no basis for inventing the unstated product intention that the limit should have been `500M`.

Contrast that with a malformed rule whose own semantic structure is contradictory:

```
(defrule malformed-manager-rule
  {:inputs {:expense Expense
            :policy ApprovalPolicy}
   :returns :boolean}
  [:and
   ;; A status enum is not an ordered monetary value.
   [:< [:expense/status :expense]
       [:policy/manager-limit :policy]]

   ;; A money value is not a status variant.
   [:= [:expense/amount :expense]
       :submitted]])
```

A checked rule language has enough information to reject that construction before it becomes a choreography or reaches a model operation. A diagnostic could identify both the location and the expected semantic categories:

```
{:error/type :gesso.rule/type-error
 :rule       'malformed-manager-rule
 :at         [:and 0]
 :operator   :<
 :left       {:type [:enum :draft :submitted :approved
                     :rejected :needs-finance]
              :source [:expense/status :expense]}
 :right      {:type :money
              :source [:policy/manager-limit :policy]}
 :required   {:operator :<
              :operands [:ordered :ordered]}}

{:error/type :gesso.rule/type-error
 :rule       'malformed-manager-rule
 :at         [:and 1]
 :operator   :=
 :left       {:type :money
              :source [:expense/amount :expense]}
 :right      {:type [:enum-value :submitted]}
 :required   {:operator :=
              :operands [:same-semantic-category
                         :same-semantic-category]}}
```

The difference is the foundational one this document relies on everywhere else. The first application can be wrong because humans chose the wrong coherent policy. The second is malformed because its own declarations do not compose. Later examples apply the same distinction to knowledge, authority, browser ownership, application assembly, and proof evidence.

The choreography system should provide the following broad properties:

- one global choreography describes an allowed distributed interaction;
- the global choreography is verified before execution;
- the choreography is projected into actor-local machines;
- local machines execute independently in Clojure and ClojureScript;
- actors acquire justified knowledge through declared provenance paths such as inputs, communication, authoritative observation, deterministic derivation, and explicit trusted assertion;
- authoritative model transitions remain atomic at their model transaction boundaries;
- clients may hold explicit provisional state without confusing it with authoritative state;
- actors may temporarily observe different frontiers of an evolving world;
- where convergence is required, the protocol provides a path toward justified knowledge of the relevant authoritative facts;
- durable application facts normally rendezvous through model authority rather than requiring direct client-to-client delivery;
- browser-local concerns such as continuity remain implementation concerns rather than becoming distributed truth;
- sleeping, reconnecting, stale, or incompatible browser projections have a defined recovery path rather than being assumed compatible forever;
- browser actors are treated as adversarial participants that may replace or modify every byte of client JavaScript and protocol state;
- authoritative security never depends on an honest browser projection;
- portable choreography and browser-adapter semantics are explicit transition systems suitable for machine-checked reasoning;
- the production browser adapter uses the same semantic transition model that is checked or proved rather than independently reimplementing a paper model;
- HTMX remains responsible for the ordinary declarative browser lifecycle it already owns;
- Gesso-owned browser effects remain behind a narrow explicit boundary;
- verification and projection produce one canonical executable artifact plus separate optional proof and diagnostic artifacts;
- proof or diagnostic artifacts cannot participate in runtime semantic decisions;
- whole-application assembly checks the correspondence between the semantic operation, projected plan, browser artifact, transport, trusted server capability, authoritative operation, settlement/progression, and permitted authoritative reacquisition;
- claimed safety properties are backed by machine-checked derivations, exhaustive semantic checking over justified abstractions, or refinement arguments rather than by test coverage alone;
- tests remain essential for trusted assumptions, implementation correspondence, real browsers, external libraries, operational behavior, and application policy.

The design should make downstream application code easier to reason about even when Gesso itself becomes substantially more sophisticated.

Correctness, explicit boundaries, debuggability, strong guarantees, and clear repair paths are more important than minimizing implementation size.

Design priority and dependency direction

Gesso is organized around a strict distinction between semantic truth, distributed coordination, physical realization, and evidence about that realization.

At the highest level:

```
domain truth
    |
    v
public authoritative model operations
    |
    v
global choreography
    |
    v
verified role projections
    |
    v
canonical executable plans
    |
    v
runtime adapters / transport / browser realization
```

Evidence flows alongside this path:

```
semantic declaration
    |
    v
normalized semantic interface
    |
    v
checked obligations / derivations
    |
    v
certificate / proof sidecar / application explanation
```

Evidence may justify acceptance of an executable artifact, but evidence does not become an executable authority channel.

That distinction separates **admission evidence** from **runtime authority**.

A certificate, derivation, proof sidecar, exact artifact digest, preflight report, or other assurance artifact may establish that an `ExecutablePlan` was constructed from the declared semantics, depends on the expected semantic interfaces, satisfies the checks claimed for it, and is therefore eligible to run. Once admitted, however, that evidence does not grant the running program new principals, permissions, authoritative facts, capabilities, branch discriminators, or current authorization.

For example, a certificate may establish:

> This Claim plan can reach the production Request claim operation only through a path that requires the declared claim-authority premise.

The certificate does not establish:

> This particular HTTP request is currently made by principal P and P is currently authorized to claim Request R.

Those concrete premises still come from trusted authentication, current model state, runtime authorization, and other declared runtime boundaries.

A fuller HumanHelp Claim slice makes that separation visible in code. The application may define a checked rule and semantic operation approximately like this:

```
(defschema ClaimInput
  [:map
   [:request/id uuid?]])

(defrule claimable?
  {:inputs {:request Request
```

```
           :membership Membership
           :organization Organization}
   :returns :boolean}
  [:and
   [:= [:request/status :request] :open]
   [:= [:membership/status :membership] :active]
   [:= [:membership/organization-id :membership]
       [:request/organization-id :request]]
   [organization-allows?
    :organization
    :membership
    :request]])

(defoperation claim-request
  {:operation/id :request/claim

   ;; This describes the semantic participant position. It is not a browser
   ;; assertion that the current user is a helper.
   :actor :helper

   :input ClaimInput

   :requires
   [[:knows? :helper [:request/id]]
    [:runtime-premise :authenticated-principal]
    [:runtime-premise :current-membership]
    [:runtime-premise :current-request]
    [:runtime-premise :current-organization]
    [:rule claimable?
     {:request :current-request
      :membership :current-membership
      :organization :current-organization}]]

   :authority
   [:request/claim-authority
    {:principal :authenticated-principal
     :request   :current-request}]

   :effect
   [:model/invoke :request/claim
    {:request-id [:request/id :current-request]
     :principal  :authenticated-principal
     ;; Allocated once by the admitted command boundary, then preserved
     ;; through retries and reconciliation. Browser-carried values are
     ;; validated and bound; their bytes alone grant no authority.
     :command-id :gesso/command-id}]

   :settlement
   {:accepted
    {:learns [[:request/status :claimed]
              [:request/helper :authenticated-principal]]
     :reacquire [:request/card :request/id]}

    :rejected
    {:learns [:claim/rejected]
     :reacquire [:request/card :request/id]}

    :unknown
    {:learns [:claim/outcome-unknown]
```

```
     :recover [:request/reconcile-claim
               {:request-id :request/id
                :command-id :gesso/command-id}]}}})
```

Verification can establish a structural proposition about that declaration. A certificate or checked report might contain evidence resembling:

```
{:certificate/class :gesso/choreography
 :semantic-object   :request/claim
 :guarantees
 [{:proposition
   [:plan/reaching-model-invoke-requires
    :request/claim
    #{:authenticated-principal
      :current-membership
      :current-request
      :current-organization
      :claimable-rule-guard}]
   :evidence :kernel-checked
   :scope :admitted-semantic-plan}

  {:proposition [:rule/well-typed :claimable?]
   :evidence :kernel-checked}

  {:proposition
   [:projection/does-not-create
    :request/claim-authority]
   :evidence :kernel-checked}

  {:proposition
   [:settlement/covered
    #{:accepted :rejected :unknown}]
   :evidence :exact-finite-checked}]}
```

That is useful evidence about the **plan**, provided the checker actually proves the stated reachability obligation. A rule checker can validate a rule's type and semantics; it cannot certify that the rule holds for a future invocation's current Request and Membership. Nor does a plan certificate alone establish that an independently authored production handler implements the checked guard. The server binding and model transaction require their own admission and runtime-enforcement evidence. This certificate cannot answer whether Alice may claim Request R now.

A trusted invocation boundary must bind the concrete principal and pass the semantic command to the public authoritative model operation. The operation owns current domain authorization and the decision about a repeated command:

```clojure
(defn invoke-claim!
  [{:keys [session now] :as ctx}
   {:request/keys [id] :gesso/keys [command-id]}]

  ;; Authentication establishes the principal for this concrete request;
  ;; the admitted command binding validates the command id and arguments.
  (let [principal
        (auth/current-principal! session)]

    ;; The model checks transaction-current facts and recognizes compatible
    ;; repetitions before interpreting the now-claimed Request as a new claim.
    (request-model/claim!
      (assoc ctx
        :biff.fx/now now
        :principal principal)
      {:request-id id
       :helper-id (:user/id principal)
       :command-id command-id})))
```

Why can a Claim be retried without claiming the Request twice?

Before the first possible send, the admitted command boundary assigns a stable `:gesso/command-id` to the Claim command. It binds that identity to the operation, its arguments, and the authenticated principal, and preserves the identity across retries. The browser may carry the identifier back to the server. Choosing or presenting its bytes does not authenticate the browser, authorize the Claim, or prove that the command committed.

The public `request-model/claim!` operation handles an existing command record before treating an invocation as a new Claim. It checks that the recorded operation, arguments, and principal match the admitted command. It also checks whether the current principal may learn the recorded result. A compatible retry returns that permitted result without applying the transition again. Thus, a retry of a successful Claim may return success even though the Request is now claimed. Reusing the identifier for a different command is rejected, and the existence of a record must not disclose another principal's result.

For a new command, Claim must check the current Request state, the helper's active Membership, the Organization policy, and the Request/Organization association as part of the atomic transition. The ownership change and successful command-result record commit together. If a required guard fails, the model returns a typed rejection and permits authoritative card reacquisition under the declared contract.

Separate pre-reads cannot provide this guarantee. A relevant fact may change between a pre-read and commit. Conversely, checking only the Request's current status before consulting the command record could reject a retry because its first attempt already succeeded. The record establishes whether this command was incorporated; the atomic guards decide whether a new command can commit.

Part IV distinguishes the model outcomes `:committed`, `:rejected`, `:already-incorporated`, and `:commit-unknown`. This example presents `:accepted`, `:rejected`, or `:unknown` to the participant. The assembly must check the explicit mapping:

Model outcome	Presentation settlement
`:committed`	`:accepted` when the successful result is delivered
`:rejected`	`:rejected` when the typed rejection is delivered
`:already-incorporated`	The permitted recorded result; a recorded successful Claim maps to `:accepted` without another transition
`:commit-unknown`	`:unknown` until authorized command-outcome evidence resolves it

A delivery failure can leave the participant uncertain even if the transaction committed. Reading a Request that is currently claimed may settle its displayed state, but that observation alone does not establish which command claimed it.

The exact declaration, certificate representation, and handler API are illustrative. A checked certificate can justify admitting the Claim plan when its obligations are discharged. Separate production and transaction evidence must show that the running realization enforces command binding, permitted result disclosure, current authority, and the atomic guards. A certificate cannot manufacture the authenticated principal, active Membership, current Request state, Organization policy, or present authorization.

Sending that certificate back from the browser would add no authority. The browser may carry a reference to an admitted operation or plan; the trusted runtime boundaries must still establish the current facts and permissions they own.

Therefore:

```
proof / certificate / preflight evidence
        |
        v
artifact admitted for execution

artifact executes
        |
        v
runtime semantics + current trusted premises
        |
        v
concrete authority decision
```

must not collapse into:

```
certificate says "valid"
        |
        v
grant authority / accept fact / choose branch
```

This applies more broadly than authorization. Assurance evidence must not secretly decide which role the current user has, which principal is authenticated, whether a current authoritative fact is true, whether a stale basis is acceptable, which settlement is canonical, or what transaction should commit.

A separately declared runtime credential, observation receipt, or other evidence protocol may supply a runtime premise only through its own trusted validation contract, with its subject, scope, applicability, and authorization meaning made explicit. That is a distinct runtime input category. It does not turn a proof sidecar, build report, or matching artifact hash into authority.

Proof and diagnostic sidecars are therefore runtime-nonsemantic. Running the same `ExecutablePlan` with or without a matching read-only sidecar must produce the same semantic transitions and abstract effects, modulo diagnostics. Evidence explains or justifies why an executable was accepted; it does not become a bearer capability or an alternate semantic control path.

The dependency direction is intentional:

- application business meaning belongs in application models;
- choreography coordinates use of public model semantics;
- projection derives local behavior from the global choreography;
- runtime code executes projected behavior;
- proof/checking artifacts explain why construction was accepted;
- physical browser, network, database, and process mechanisms realize declared effects under explicit assumptions.

A lower layer must not acquire higher-level meaning merely because doing so would make an implementation convenient. E.G.

- a DOM node does not become a semantic Request merely because it has an id;
- a web node does not become Request authority merely because it handles the request;
- a browser-supplied principal id does not become authenticated identity because it has the expected shape;
- a map with `:verified? true` does not become a verified choreography;
- a matching hash does not establish the truth of the proposition represented by the hashed artifact;
- a Live invalidation does not establish the changed authoritative fact;
- a rendered button does not confer authorization to execute its operation.

Normative language

This design uses the following terms deliberately.

- **MUST** or **must** identifies a correctness requirement of the intended design.
- **MUST NOT** or **must not** identifies behavior that would violate the intended correctness boundary.
- **SHOULD** or **should** identifies the ordinary design path. Deviations require a concrete reason and must preserve the relevant invariants.
- **MAY** or **may** identifies an implementation freedom that does not change the semantic contract.
- **proved** is reserved for mechanically established properties under named assumptions.
- **verified** may also describe a mechanically checked closed construction whose result is not claimed as a theorem over an explicitly formalized semantic relation. The result classification must remain precise.
- **trusted** does not mean ”probably correct.” It means the proposition is outside the proof currently being claimed and is accepted as an explicit assumption.

A green test suite never silently upgrades a tested property into a proved property.

A successful schema validation never silently upgrades a value into trusted authority.

A valid certificate identity never silently upgrades the certificate's claims into truth unless an appropriate checker has accepted the certificate under the premises and evidence classification attached to those claims.

Interpretation of examples and implementation freedom

Examples illustrate the enclosing contract. Their concrete syntax is illustrative unless the owning section explicitly defines it as the required public or wire representation. Their stated required behavior must still obey the normative invariants.

An abbreviated diagram does not waive an authorization, admission, scope, ownership, or outcome-attribution condition merely because the diagram omits that step. Worked examples must identify consequential policy choices instead of silently selecting behavior that the specification leaves open.

The implementation may choose a representation or mechanism where the design allows alternatives. It must state which supported contract was chosen and provide evidence appropriate to the resulting claim. Implementation freedom does not permit silent weakening of a required capability or assurance class.

A contradiction between normative passages is a specification defect to be reconciled at the owning section. It is not permission to select whichever passage makes an implementation easier to accept.

A declared recovery path establishes available modeled behavior. It does not by itself establish that the environment will cause that path to execute or that every execution terminates. Liveness claims retain their named premises.

Correctness scopes

The system intentionally combines several correctness scopes. No one scope subsumes the others.

Domain correctness

Application models define domain meaning.

Examples include:

- whether a Request may be claimed;
- whether a Membership is active;
- whether an Organization permits an operation;
- what a ”reconciled” claim outcome means;
- which facts must be protected atomically at commit time.

Gesso may check that a rule expression is well typed, that a declared operation contract composes with its dependencies, and that the distributed execution respects that contract. Gesso does not infer whether the Organization chose the right business policy.

Choreographic correctness

Choreo defines and checks distributed coordination around public semantic operations.

It reasons about:

- participating roles;
- principal and authority requirements;
- knowledge;
- provenance;
- communication;
- authoritative observation;
- branch discriminators;
- provisional state;
- command and execution identity;
- settlement;
- resource ownership;
- convergence;
- failure and recovery;
- projection into role-local behavior.

Runtime correctness

Runtime code realizes the checked semantics.

It must preserve:

- semantic identities;
- execution-instance identity and declared resource-instance ownership;
- stale-work rejection;
- correlation;
- progression requirements;
- resource lifetimes;
- provisional-versus-authoritative distinctions;
- permitted machine transitions;
- transport and server realization.

Whole-application closure

Individually valid components can still form an impossible application.

The assembled application therefore has its own closure boundary. A rendered semantic affordance must have a coherent path through the entire application:

```
rendered affordance
    |
    v
semantic operation
    |
    v
valid choreography trajectory
    |
    v
projected local plan
    |
    v
exact current browser artifact
    |
    v
declared transport
    |
    v
trusted server capability
    |
    v
public authoritative model operation
    |
    v
typed settlement / authoritative progression
    |
    v
permitted authoritative reacquisition
    |
    v
renderable authoritative successor
```

Later parts of this design specify that closure in detail. Its inclusion here establishes an important foundational rule: local correctness does not imply whole-application correctness.

Three distinct application assurance claims

Whole-application assurance separates three claims that are related but not interchangeable.

Claim	What it establishes	What it does not establish
Declared closure	Required relationships among the application elements within the declared assembly scope are checked	That every intended product capability was declared
Production enforcement	Actual supported execution passes through the boundaries that make the declared checks effective	That every possible host-language escape hatch is impossible
Behavioral completeness	Required user capabilities and authoritative outcomes are present and usable	That their internal assembly is structurally closed unless closure is checked separately

A self-consistent assembly can omit intended functionality. Conversely, a behaviorally successful path can exist while bypassing a checker that the design claims is mandatory. The three claims therefore reinforce one another without collapsing into one another.

Assembly closure is always relative to declared scope. Gesso cannot infer an application author's missing business intention merely because the remaining graph is closed. Required product behavior therefore needs an acceptance basis independent of the assembly's own declaration set. That acceptance basis should state user capabilities and authoritative outcomes rather than duplicate route strings, operation registries, projected-plan identities, or other realization metadata already owned by the assembly.

Production enforcement is a separate obligation. A checker that correctly rejects an isolated malformed fixture provides no application guarantee when the real application path can bypass that checker. Strong claims about an assembled application must therefore identify the production boundary that makes the relevant check effective.

Inside trusted host-language code, deliberate escape hatches may remain possible. For example, trusted application code that intentionally obtains a lower-level transaction capability may bypass the model/Gesso path. The ordinary assurance goal inside that trusted boundary is misuse prevention: supported application construction should make accidental semantic bypass difficult, explicit, and diagnosable. A deliberate departure from that supported path exits the corresponding assurance claim unless separately checked.

These distinctions are foundational for later HumanHelp acceptance:

```
assembly closed
    ≠
production path demonstrably enforced
    ≠
all required HumanHelp capabilities present
```

One Claim implementation can be used to see all three application-level claims without inventing three unrelated examples.

Suppose the application assembly describes the supported production pieces as data:

```
(def humanhelp-assembly
  {:assembly/id :humanhelp/web

   ;; Each referenced identity is backed by a source-owned declaration;
   ;; these abbreviated entries show the edges needed by this slice.
   :choreographies
   {:request/claim {:operation :request/claim}}

   :plans
   {:plan/request-claim {:from :request/claim
                         :artifact :artifact/gesso-live}}

   :operations
   {:request/claim
    {:choreography :request/claim
     :plan         :plan/request-claim
     :artifact     :artifact/gesso-live
     :transport    :transport/trusted-post
```

```
    :route        :route/request-claim
    :capability   :server/request-claim
    :model-op     :request/claim
    :settlement   :request/claim-settlement
    :reacquire    :request/card}}

  :transports
  {:transport/trusted-post {:kind :http/post
                            :route :route/request-claim}}

  :routes
  {:route/request-claim {:transport :transport/trusted-post
                         :capability :server/request-claim}}

  :views
  {:request/card
   {:affordances
    {:claim
     {:operation :request/claim
      :when      :request/claim-visible?}}}}

  :server-capabilities
  {:server/request-claim
   {:route    :route/request-claim
    :model-op :request/claim}}

  :model-operations
  {:request/claim {:public-entry :request-model/claim!}}

  :settlements
  {:request/claim-settlement
   {:outcomes #{:accepted :rejected :unknown}
    :reacquire {:accepted :request/card
                :rejected :request/card}
    :recover {:unknown :request/reconcile-claim}}}

  :acquisitions
  {:request/card {:result :request/card}}

  :recovery-capabilities
  {:request/reconcile-claim
   {:by :gesso/command-id
    :model-op :request/claim}}

  :artifacts
  {:artifact/gesso-live
   {:contains-plans #{:plan/request-claim}}}})
```

A closure checker can establish relationships **inside that declaration**:

```
(assembly/check humanhelp-assembly)
;; ⇒
{:result :accepted
 :evidence :assembly-verified
 :closed-edges
 [[:request/card :claim :request/claim]
  [:request/claim :choreography :request/claim]
  [:request/claim :plan/request-claim]
  [:plan/request-claim :artifact/gesso-live]
  [:request/claim :transport/trusted-post]
  [:transport/trusted-post :route/request-claim]
  [:request/claim :route/request-claim]
  [:route/request-claim :server/request-claim]
  [:server/request-claim :request/claim]
  [:request/claim :request/claim-settlement]
  [:request/claim-settlement :request/card]
  [:request/claim-settlement :request/reconcile-claim]]}
```

The checker must collect these entries from the actual admitted choreography, projected plan, artifact bytes, route table, server binding, model operation, settlement, and acquisition/recovery declarations. Copying plausible entries into a disconnected map would show only that the map agrees with itself. The accepted report is illustrative of a checker that resolves every referenced source and validates the recorded edges.

That still does not establish production enforcement. The actual startup and request path must make the admitted assembly unavoidable for the claimed supported path. Conceptually, production wiring might look like:

```
(def humanhelp-module
  {:biff.core/id :humanhelp/web

   :biff.core/start
   (fn [ctx]
     ;; Failure prevents the semantic web application from starting.
     (let [admitted
           (assembly/admit!
             {:assembly humanhelp-assembly
              :artifact (browser/current-artifact ctx)
              :routes (server/current-route-table ctx)})]

       {:admitted-assembly admitted
        :handler
        (→ (routes/build admitted)
           (wrap-session)
           (wrap-current-assembly admitted))})})

(defn claim-handler
  [ctx request]
  ;; Route lookup comes from the admitted assembly. The handler does not
  ;; reconstruct an operation id from arbitrary request fields.
  (let [binding
        (assembly/require-route-binding!
          (:admitted-assembly ctx)
          :route/request-claim)]
    (server/invoke-bound-operation!
      ctx
      binding
      request)))
```

Now the relevant malformed production fixture is not merely passed to an isolated checker. It is sent through the same startup boundary:

```
(deftest stale-artifact-prevents-startup
  (is
    (thrown-with-msg?
      clojure.lang.ExceptionInfo
      #"browser artifact does not realize admitted plan"
      (start-humanhelp!
        (assoc test-ctx
          :browser-artifact
          artifact-without-request-claim-plan)))))
```

That supplies evidence for production enforcement of this boundary.

Behavioral completeness still needs an independent statement of what the user must actually be able to do:

```
(def required-humanhelp-capabilities-excerpt
  [{:capability :request/create
    :eligible-scenario :authenticated-customer
    :required-outcome :request/created-and-visible-to-creator}

   {:capability :request/claim
    :eligible-scenario
    :authenticated-active-helper-sees-open-request
    :required-outcome
    :request/claimed-by-helper}

   {:capability :board/refresh
    :eligible-scenario
    :helper-has-outstanding-board-advisory
    :required-outcome
    :authorized-current-board-adopted}])
```

These three entries are an excerpt for the Claim illustration, not HumanHelp's complete required-capability basis. The complete independent acceptance basis and its operational milestone scope are specified in Part VIII's required capabilities.

If `:request/claim` is removed from both the view and the assembly, the remaining assembly may still be perfectly closed. The independent capability list is what prevents that self-consistent omission from being called complete.

Conversely, a hand-written `POST /claim` route may make a browser acceptance test appear green while bypassing `assembly/admit!` and the declared Claim operation. Behavior happened, but production enforcement of the claimed Gesso boundary was not established.

The point of keeping all three claims separate is therefore practical rather than terminological: each one catches a different way a plausible application can ”work” while failing the assurance statement the project intends to make.

Operational correctness

Formal semantics do not eliminate operational concerns.

Real-browser tests, fault injection, property tests, load tests, lifecycle tests, and application acceptance remain necessary for:

- browser primitive assumptions;
- HTMX and EventSource/SSE behavior;
- compiler/runtime implementation correspondence;
- scheduler and timer behavior;
- memory and CPU characteristics;
- dependency outages and reconnect behavior;
- application-specific semantics;
- human-visible interaction behavior.

Required capabilities are independent of exposed capabilities

A milestone's behavioral acceptance must name the required user capabilities and authoritative outcomes independently of the set currently exposed by the implementation. Tests derived only from existing affordances cannot establish that an intended affordance was not omitted.

For each required capability, acceptance must establish an eligible scenario in which the intended interaction is usable and reaches its required outcome. Relevant ineligible scenarios must establish the specified unavailability, rejection, or recovery behavior. Marking every occurrence unavailable does not satisfy a requirement to provide a usable capability.

An explicitly narrower milestone may defer a capability. Its scope must say so; its completion must not be reported as completion of a broader milestone that requires that capability. This specification distinguishes such deferral from unavailability caused by current production model policy.

The independent acceptance basis owns user capabilities and outcomes. It must not become a second registry of route strings, generated plan identities, transport metadata, or model implementation already owned by the assembly.

Assurance model

Gesso uses several kinds of evidence, and the design must not collapse them under a single word such as `verified`, `safe`, or `proved`.

The governing question for every assurance claim is:

> What proposition is being claimed, over what object, under what assumptions, and what evidence justifies accepting it?

The assurance model distinguishes:

1. properties established mechanically from explicit semantics or checked derivations;
2. premises enforced dynamically when a concrete execution meets the external world;
3. assumptions that remain outside the mechanically discharged core;
4. whole-application closure properties checked over assembled artifacts;
5. liveness claims whose validity depends on named environmental assumptions;
6. test evidence over trusted implementation boundaries and concrete systems.

These classifications are related, but they are not interchangeable.

For example:

```
proved semantic property
≠
exact finite-state check
≠
closed application assembly
≠
runtime-enforced premise
≠
trusted assumption
≠
green integration test
```

A complete assurance argument may require several of them simultaneously.

A store checkout provides a useful worked example because one ordinary button crosses nearly every assurance class without any one class subsuming the rest.

Suppose the semantic operation is declared approximately as:

```
(defoperation place-order
  {:operation/id :checkout/place-order

   :actor :customer

   :input
   [:map
    [:cart/id uuid?]
    [:checkout/command-id uuid?]]

   :requires
   [[:runtime-premise :authenticated-customer]
    [:authoritative :current-cart]
    [:rule :cart/checkout-eligible? :current-cart]]

   :provisional
   {:view :checkout/submitting
    :owned-by :checkout/command-id}

   :effect
   [:model/invoke :order/place
    {:cart-id [:cart/id :current-cart]
     :customer :authenticated-customer
     :command-id :checkout/command-id}]

   :settlement
   {:accepted
    {:learns [:order/placed]
     :reacquire [:order/by-command :checkout/command-id]}

    :rejected
    {:learns [:checkout/rejected]
     :reacquire [:cart/current :cart/id]}

    :unknown
    {:learns [:checkout/outcome-unknown]
     :recover [:order/reconcile-command
               :checkout/command-id]}}})
```

A release report for one concrete assembly might preserve evidence instead of flattening it:

```
{:subject
 {:assembly :store/web
  :operation :checkout/place-order
  :plan :plan/checkout-place-order
  :artifact-digest "sha256:..."}

 :claims
 [{:id :checkout/rule-well-typed
   :proposition
   [:rule/well-typed :cart/checkout-eligible?]
   :evidence
   {:class :kernel-checked
    :certificate :cert/cart-checkout-rule}}

  {:id :checkout/principal-bound-at-runtime
   :proposition
   [:runtime/binds-current-principal
    :checkout/place-order]
   :evidence
```

```
  {:class :runtime-enforced
   :boundary :server/authenticated-checkout-handler}}

 {:id :checkout/payment-provider-contract
  :proposition
  [:provider/acceptance-means
   :payment-authorized]
  :evidence
  {:class :trusted
   :owner :external/payment-provider
   :assumption :payment/provider-contract-v4}}

 {:id :checkout/application-path-closed
  :proposition
  [:application/closed-path
   :checkout/place-order]
  :evidence
  {:class :assembly-verified
   :report :assembly/store-web-A12}}

 {:id :checkout/browser-realization-tested
  :proposition
  [:implementation-corresponds-in-scenarios
   :checkout/place-order
   #{:accepted :rejected :unknown}]
  :evidence
  {:class :tested
   :suite :chromium/checkout-lifecycle}}

 {:id :checkout/retry-eventually-recovers
  :proposition
  [:eventually
   [:checkout/settled-or-reconciled]]
  :evidence
  {:class :conditional
   :premises
   #{:eventual-provider-availability
     :eventual-authoritative-read
     :continued-participant-execution}}]}
```

The report is deliberately heterogeneous. Consider what happens if each claim is misread as another:

```
;; Wrong: a theorem about the checked rule does not authenticate this request.
(when (certificate/valid? :cert/cart-checkout-rule)
  (treat-request-as-authenticated! request))

;; Wrong: a green Chromium test does not prove the provider contract.
(when (tests/green? :chromium/checkout-lifecycle)
  (assume-provider-cannot-lie!))

;; Wrong: assembly closure does not imply eventual provider availability.
(when (assembly/closed? :store/web)
  (claim-unconditional-termination!))

;; Wrong: the trusted provider premise does not prove the browser adapter.
(when (trusted? :payment/provider-contract-v4)
  (claim-browser-correspondence-proved!))
```

The correct composition keeps the propositions separate. A downstream release claim may depend on all of them, but the resulting explanation should still be able to say exactly which part is mechanically checked, which is dynamically enforced, which remains trusted, which is assembly evidence, which is tested, and which is conditional on future environmental behavior.

That is what the assurance vocabulary is for. It is not a taxonomy applied after the fact; it prevents evidence from silently acquiring a stronger meaning as it moves through the application.

The classification belongs to the **claim and its evidence**, not merely to the source file, namespace, subsystem, or tool that produced it. A single component may therefore participate in several assurance classes.

For example, the browser adapter may have:

- machine-checked transition invariants;
- runtime-enforced event normalization and plan-identity checks;
- trusted assumptions about DOM/browser primitives;
- Chromium integration tests providing tested evidence of implementation correspondence;
- conditional liveness claims requiring eventual event delivery.

The root assurance report must preserve these distinctions rather than reducing them to a single boolean such as `verified?`.

Nominal or otherwise admitted semantic categories participate in this model but do not add a new assurance class by themselves. For example, an admitted `VerifiedChoreography` value may be the product emitted after verification, but its assurance comes from the verifier/checker and its stated premises, not from the runtime type name. Likewise, an `AuthoritativeObservation` representation is authoritative only when the trusted acquisition path establishes that provenance.

This distinction prevents the implementation convenience of nominal typing from being mistaken for proof:

```
schema / representation validation
    -> representation has permitted shape

semantic-category admission
    -> supported code may treat value as category C

checked judgment / certificate
    -> stated semantic proposition follows under named premises
```

Those stages may be represented by different types, by checked constructors, or by other existing mechanisms. Their assurance meaning remains distinct even if an implementation combines some of the physical representations.

Machine-checked semantic properties

A machine-checked semantic property is established mechanically from an explicit semantic model, checked derivation, exhaustive semantic state space, proof kernel, model checker, theorem prover, or another precisely identified formal mechanism.

The important requirement is not the name of the tool. The important requirement is that the claimed proposition and the relationship between the checked object and the production semantic object are explicit.

Candidate framework properties include:

- well-typed checked rule expressions;
- knowledge entailment;
- provenance entailment;
- authority derivation and non-escalation;
- message payload closure and knowledge-transfer constraints;
- behavior-sensitive branch knowledge;
- projection preservation for the supported language subset;
- declared resource invariants;
- progression algebra invariants;
- browser-adapter transition invariants;
- retired execution/resource isolation;
- semantic dependency compatibility;
- executable-plan provenance;
- certificate dependency consistency.

The individual phrases above name different judgments rather than one generic notion of ”verification”:

- **Well-typed checked rule expressions.** A rule expression is checked against the closed semantic rule language. Referenced fields must exist on the declared semantic type; operators must be applied to compatible operand types; enum values must belong to the declared variant set; optional values must be refined before operations requiring presence; relationship predicates must use valid endpoint types; and closed case analysis must be exhaustive where required. This establishes semantic well-formedness of the rule. It does not establish that the product organization chose the desired rule. A coherent but undesirable policy remains representable.
- **Knowledge entailment.** The checker asks whether a role is justified in knowing a semantic fact at the point where that fact is consumed. Relevant sources include declared entry knowledge, communication, authoritative observation, checked deterministic derivation, and explicit trusted assertion. Knowledge does not arise merely because a value happens to exist in a host-language map, another role knows the fact, the browser physically received related data, or an undeclared field survived serialization.
- **Provenance entailment.** The checker asks whether the claimed source or epistemic strength of a fact follows from the path through which the fact was obtained. A browser-supplied value is not an authoritative observation merely because it has the same shape or value. A fact learned by communication does not automatically become an independent authoritative observation. Any strengthening of provenance must be an explicit semantic rule rather than an inference from physical message origin.
- **Authority derivation and non-escalation.** Every authority or capability consumed by a semantic step must arise from an explicit trusted/runtime premise or an allowed checked derivation. Untrusted input, mere knowledge, provenance labels supplied by the browser, possession of a rendered affordance, possession of a URL, or successful schema validation cannot silently manufacture stronger authority. Principal, resource, scope, operation, and authority identity remain part of the judgment, so authority over one resource cannot be reused for another merely because their representations have the same shape.

  The framework-level claim is not ”Alice is currently authorized to claim Request R.” That concrete fact normally depends on runtime authentication and current model state. The machine-checkable claim is closer to:

  > No admitted semantic execution may consume claim authority unless the required authority premise is present through a declared trusted/runtime source or permitted derivation, and no weaker or untrusted fact can silently manufacture stronger authority.

Conceptually:

```
browser says:
    role = helper
        |
        X
        |
        v
    helper authority
```

is invalid, while a chain such as:

```
trusted authentication
    establishes principal P
        |
authoritative Membership observation
    establishes active helper relationship
        |
authoritative Request / Organization facts
    +
model policy
        |
        v
claim operation is currently admissible for P over R
```

may be valid when every edge is declared and discharged at the appropriate assurance boundary.

- **Message payload closure and knowledge-transfer constraints.** A semantic message defines the information flow permitted by one choreography communication boundary. By default its semantic payload is closed: only fields declared by the message contract participate in the semantic message. Physical transport metadata may accompany the message, but it does not silently become choreography knowledge.

  The checker should establish that every semantic payload field is declared, the sender has justified knowledge of every semantic fact it communicates, receipt introduces only the semantic facts declared by the message contract, provenance introduced by receipt follows the declared communication rule, and undeclared transport or implementation fields cannot become branch discriminators, authority premises, or arbitrary additions to choreography context.

  Conceptually:

  ```
  receiver knowledge after receive
      =
  receiver knowledge before receive
      +
  declared semantic consequences of message
  ```

  rather than:

  ```
  receiver knowledge after receive
      =
  receiver context merged with arbitrary incoming map
  ```

  An explicitly extensible payload may be supported where required, but the semantic status of extension data must remain explicit. For example, extension data may be transported opaquely without automatically creating first-class choreography facts. Payload closure is therefore distinct from ordinary Malli shape validation: Malli may establish that a value has the expected structure, while choreography checking establishes which information that communication is semantically allowed to transfer.

- **Behavior-sensitive branch knowledge.** A role must know enough to distinguish branches before its observable behavior diverges between those branches. If a role behaves identically in both alternatives, it need not learn a choice solely because the condition occurs in the global choreography. If its behavior differs, the verifier must justify knowledge of the discriminator through an allowed path. This prevents both under-checking and needless communication introduced only to mirror global syntax.

- **Projection preservation.** Projection relates the accepted global choreography to generated role-local machines. Executions of those projected machines must remain within behavior admitted by the global choreography for the supported language subset. Projection must not invent knowledge or authority, erase communication required for justified behavior, lose

required resource constraints, admit forbidden branch behavior, change terminal/failure meaning, or introduce executions absent from the accepted global semantics. The exact preservation/refinement theorem must state the supported language subset and observable behavior relation.

- **Declared resource invariants.** Where choreography semantics include resources, the checker may establish that a resource is used only after acquisition, exclusive ownership is not duplicated, transfers occur only through declared rules, release occurs according to the declared lifecycle, terminal paths do not retain resources that must be released, and stale or retired executions cannot retain semantic ownership. Physical cleanup debt does not confer continuing semantic ownership.
- **Progression algebra invariants.** Progression requirements form a semantic consistency algebra rather than an arbitrary collection of tokens. Canonicalization should be deterministic; composition must preserve every required authoritative constraint; weaker frontiers must not masquerade as satisfying stronger requirements; unrelated requirements must compose without destructive interference; contradictory or malformed requirements must be rejected; and normalization must not silently discard semantically relevant coordinates. Concrete XTDB tokens remain runtime values even when the algebra governing their requirements is checked independently.
- **Browser-adapter transition invariants.** The browser adapter is modeled as an explicit transition relation such as:

  ```
  AdapterStep
    (ExecutablePlan,
     AdapterState,
     NormalizedEvent)
        ->
    (AdapterState',
     AbstractEffects)
  ```

  Candidate invariants include execution-instance isolation, resource-instance ownership, prevention of obsolete presentation being restored after justified authoritative supersession within the applicable observation scope, separation of provisional and authoritative state, immediate semantic retirement of obsolete work, scope-preserving acquisition requirements for queued refreshes, and terminal ownership/cleanup conditions. The proof applies to the abstract transition semantics; separate implementation-correspondence evidence is required for the browser shell that normalizes concrete events and performs abstract effects.
- **Retired execution/resource isolation.** Once an execution instance or resource instance loses its relevant semantic ownership, a callback, timer, network result, settlement, continuation event, or delayed effect cannot use that retired ownership to resume execution or mutate a replacement owner's state. Admission identifies the event's declared owner and concrete resource instance; matching a reused logical target name is insufficient. Physical cleanup may occur later, and new logical ownership need not wait for every obsolete physical operation to disappear. Independently useful authoritative or command-outcome evidence may enter only through its separately declared admission path; it does not reactivate the retired owner.
- **Semantic dependency compatibility.** A certified downstream artifact may depend only on upstream semantic interfaces whose guarantees satisfy the assumptions recorded when the downstream artifact was checked. A same-shaped value, namespace, or source path is insufficient. A relevant upstream semantic change must invalidate stale downstream certification rather than silently compose.
- **Executable-plan provenance.** An accepted `ExecutablePlan` must remain traceable to the semantic inputs from which it was produced, including the accepted choreography, relevant semantic dependencies, projection target, and exact executable representation identity. Executable identity is not itself a proof; it binds the executable object to the checking history and semantic inputs that produced it.
- **Certificate dependency consistency.** A certificate may claim only guarantees justified by its checked derivation, certified dependencies, explicit assumptions/assertions, and the checker or proof-kernel rules applicable to its certificate class. Certificates may not self-certify, depend on undeclared semantic inputs, silently accept stale dependencies, form unsupported circular justification, or claim stronger conclusions than their derivations establish.

”Machine checked” is deliberately broader than ”formally proved.” Useful result classifications include:

- **Kernel-checked or theorem-proved:** a proposition is accepted by a proof kernel or theorem prover from explicit premises and checked derivations.
- **Exact finite-state checked:** a finite semantic state space is exhaustively explored and the property holds for every state/transition in that exact model. This is stronger than sampled testing but is not automatically a theorem about an unbounded concrete system.
- **Abstraction-based machine proof:** an unbounded concrete claim may be justified through a finite or symbolic abstraction only when a stated abstraction/refinement argument shows that every semantically relevant concrete behavior is represented soundly by the abstraction.
- **Proof-producing elaboration:** a richer compiler or elaborator constructs derivations that are accepted by a smaller checker/kernel. The elaborator does not become trusted merely because it emits proof-looking data; a buggy elaborator must be unable to convince the checker of an invalid conclusion.

Where the design calls a property **proved**, the proof claim must identify:

- the exact proposition;
- the formal or executable semantic model over which it is established;
- the assumptions/premises;
- the supported language subset;
- the observable/refinement relation where one is involved;
- any abstraction used to reduce an unbounded concrete domain;
- why that abstraction is sound for the claimed property;
- whether the result is bounded or unbounded;
- the checker, kernel, model checker, theorem prover, or proof mechanism;
- the identity/version of the checked semantic object where relevant;
- the relationship between the checked object and the production object;
- any implementation-correspondence assumption remaining outside the proof.

Passing generated examples is not by itself a proof.

Property-based testing is not by itself a proof.

Exhaustively checking a finite concrete truncation is not by itself an unbounded proof.

A proof over an abstraction may support an unbounded claim only when the abstraction/refinement argument justifies mapping all semantically relevant concrete cases into that abstraction.

Bounding a count of requests, executions, or queue slots does not by itself bound the information retained inside them or establish a finite semantic state space. The claim must identify the state dimensions being bounded, the retained data that remains unbounded if any, and the abstraction used for the property.

Conformance vectors generated from a formal model retain their actual evidence class. Executing every generated vector is not proof that those vectors cover all production inputs or derivations. Rule coverage alone is not a general production/formal correspondence theorem. A stronger claim requires a stated exhaustiveness or sound correspondence argument for its claimed domain.

The unqualified word `proved` must never be used merely to mean:

- unit tested;
- property tested;
- preflight accepted;
- schema valid;
- assembly closed;
- observed to work in Chromium.

Those may all be important evidence. They are different evidence.

Runtime-enforced premises

Some semantic or proof premises can be discharged only when a particular execution meets concrete external state.

Examples include:

- Malli value shape at external boundaries;
- authenticated principal binding;
- operation-registry lookup;
- route and capability correspondence;
- execution-instance identity and relevant resource-instance identity;
- command/correlation identity;
- projected-plan compatibility;
- browser-artifact identity;
- progression token/header decoding;
- current server-side authorization;
- authoritative basis interpretation;
- protected read/subscription authorization;
- concrete effect results;
- transaction outcomes;
- current model/entity existence;
- current membership or relationship state.

A runtime-enforced premise is best understood as:

```
static semantic result:
    if premise P holds,
    transition T is admissible

concrete runtime:
    check P for this execution

if P succeeds:
    execute within proved/checked boundary

if P fails:
    reject or follow declared recovery semantics
```

Runtime enforcement therefore does not weaken the static result. It connects that result to the concrete execution.

For example, a static semantic result may establish:

> `:request/claim` may execute only with an authenticated principal possessing a currently valid claim-authority premise.

The runtime system must still establish which principal this HTTP request actually represents and whether the model currently authorizes that principal to perform the operation.

These are not second-class guarantees. They are explicit dynamic boundaries whose successful enforcement keeps concrete execution inside the premises of the semantic result.

If a runtime premise fails, execution must reject, reacquire authority, restart under a current plan, reauthorize, or follow another explicitly declared semantic recovery path. It must not silently weaken the operation.

In particular:

```
required Choreo premise fails
    X
    |
    +--> silently perform ordinary weaker HTMX action
```

is forbidden.

Trusted assumptions

Some propositions remain outside the currently mechanically discharged core.

Examples include:

- authentication middleware establishes the named principal correctly;
- XTDB supplies the documented transaction and consistency semantics on which Gesso relies;
- an application model operation satisfies a semantic contract not yet discharged by a stronger model certificate;
- HTMX follows the documented lifecycle subset Gesso depends on;
- browser primitives implement the operations assumed by the browser effect shell;
- Clojure, ClojureScript, JavaScript, JVM, and compiler/runtime behavior preserve the implementation semantics assumed by the checker;
- cryptographic/hash primitives used for artifact identity or authenticated evidence satisfy their required engineering assumptions;
- operating-system/network primitives behave according to the explicitly assumed contract.

A trusted assumption is part of the trusted computing base for every proposition that depends upon it.

An assumption is not a failure.

An invisible assumption is a failure of the assurance boundary.

Assumptions must therefore be named, classified, attached to the claims that rely on them, visible in explanation/root assurance output, and narrowed or discharged when stronger evidence becomes available.

Useful classifications include:

- `:authentication`;
- `:trusted-model-contract`;
- `:runtime-primitive`;
- `:external-library`;
- `:external-system`;
- `:compiler-runtime`;
- `:environment-liveness`.

Application assertions

Application assertions remain distinct from framework/trusted implementation assumptions.

An application may need to state semantic facts that Gesso cannot derive. For example:

```
"this external identity provider is the authority for employee status"

"this role mapping is organizational policy"

"this third-party callback means payment has been accepted"
```

Such statements may be necessary premises. They must not be represented as though Gesso proved them.

An `:application-asserted` proposition therefore remains an explicit leaf in the assurance graph until a stronger certificate or runtime enforcement discharges it.

This distinction is useful because:

```
framework trusts browser primitive X
≠
application asserts business fact Y
```

Both are assumptions, but they have different owners and repair paths.

Assembly-verified properties

Some important propositions concern whether a finite collection of individually valid artifacts forms one closed executable application. These are naturally checked as graph/assembly properties.

Examples include:

- every rendered semantic affordance names a declared semantic operation;
- every rendered semantic operation has a projected executable plan;
- every plan has a corresponding current browser artifact;
- every semantic command has a declared physical transport;
- every transport resolves to a trusted server route/capability;
- every trusted capability resolves to the declared authoritative operation;
- required execution capabilities are supplied;
- settlement/progression obligations have declared realization paths;
- every managed fragment has initial authoritative content or an allowed acquisition path;
- every operation successor can be reacquired through an authorized authoritative path;
- no ordinary anonymous transport occupies a route reserved for a semantic operation;
- generated browser artifacts correspond exactly to the current assembly;
- startup lifecycle ordering makes required preflight unavoidable.

Assembly verification is mechanical, but it is useful to classify the result separately from a semantic theorem. It establishes closure over a concrete application graph.

For example:

```
choreography theorem:
    projected Claim respects knowledge requirements

assembly verification:
    this application's Claim button actually resolves to that plan,
    current browser artifact,
    route,
    server operation,
    settlement,
    and reacquisition path
```

Both can be true or false independently.

Assembly checks should therefore be classified with precise language such as:

- `:assembly-verified`;
- `:preflight-closed`;
- `:artifact-current`;

rather than being upgraded automatically to `proved`.

A future proof system may discharge some assembly propositions more formally. Until then, the distinction remains explicit.

Conditional liveness

Liveness is not an independent evidence mechanism. It is a qualification on propositions whose truth depends on future behavior of the environment.

A statement such as:

> Every provisional claim eventually reaches a resolving outcome.

is meaningful only relative to premises such as:

- required events are eventually delivered;
- authoritative dependencies remain reachable;
- required authoritative rereads eventually complete;
- participants continue executing;
- relevant clocks/timers eventually fire;
- retry policy continues where required;
- the application can recover from an incompatible stale projection;
- application-defined conflicts do not remain permanently unresolved;
- required external systems eventually respond where the claim depends upon them.

The claim should therefore have a form conceptually like:

```
assuming:
    eventual event delivery
    eventual authoritative availability
    continued participant execution

prove/check:
    every admitted provisional state eventually reaches
    a resolving semantic state
```

The assumptions are part of the proposition. Removing them changes the proposition.

The design must distinguish:

- **reachability:** there exists a path to a terminal/resolving state;
- **deadlock freedom:** no admitted nonterminal state is permanently stuck under the modeled transition relation;
- **fairness-dependent progress:** enabled progress eventually occurs under a fairness assumption;
- **environment-dependent convergence:** the system converges if specified external events eventually occur;
- **unconditional termination:** every execution terminates without such assumptions.

These are materially different claims.

The design must not call ”there exists a path to terminal state” an unconditional termination proof, nor hide network recovery, browser resumption, user action, or external-system availability inside the word `eventually`.

Tested evidence

Tests remain a required assurance layer even where stronger semantic checking exists.

Testing is particularly important for:

- implementation correspondence between semantic core and runtime shell;
- real JVM/Node/Chromium execution;
- HTMX/SSE lifecycle behavior;
- XTDB integration;
- browser primitive assumptions;
- HTTP/session/authentication integration;
- generated-artifact loading;
- fault injection;
- race schedules;
- reconnect behavior;
- load/resource behavior;
- multi-client HumanHelp behavior;
- multi-node realization once it exists.

Useful test classes include unit tests, property/generated tests, differential tests, mutation tests, integration tests, adversarial tests, browser acceptance tests, and load/fault tests.

Tests may provide very strong practical evidence. They must nevertheless remain classified as `tested` unless the particular procedure genuinely exhausts the explicitly claimed semantic domain and is classified more precisely, for example `exact-finite-checked`.

Thus:

```
100,000 generated executions passed
```

does not mean:

```
all executions satisfy the theorem
```

unless a separate argument establishes that the tested set is exhaustive for the claimed semantic domain.

Composition of assurance claims

The most important property of the assurance model is that evidence composes without losing its classification.

A downstream artifact should be able to say conceptually:

```
Claim ExecutablePlan

depends on:

    rule well-typed
        evidence: kernel-checked

    helper knows request-id
        evidence: machine-checked derivation

    claim authority available
        evidence:
            semantic requirement machine-checked
            concrete current authorization runtime-enforced

    Request claim model contract
        evidence: trusted-model-contract

    projection preserves supported semantics
        evidence: exact-finite-checked under abstraction A

    browser realization complete
        evidence: assembly-verified

    browser shell follows adapter contract
        evidence:
            trusted runtime primitives
            Chromium integration tested
```

The resulting assurance claim must not erase those differences and simply say ”Claim is verified.”

A stronger downstream result cannot silently upgrade a weaker premise.

For example:

```
trusted model contract
    +
proved choreography rule
```

does not become:

```
proved model implementation
```

The conclusion remains conditional on the trusted model contract.

Likewise:

```
assembly verified
    +
browser test green
```

does not establish a projection theorem.

Each evidence class retains its identity through composition.

The root report should therefore expose what was proved, kernel-checked, exact-finite checked, assembly-verified, runtime-enforced, trusted, application-asserted, conditional on liveness assumptions, and supported by testing.

Failure and repair interpretation

The assurance classification should tell the developer where repair belongs.

Typical relationships include:

```
rule typing failure
    -> repair rule/domain declaration

knowledge entailment failure
    -> add justified information path or change behavior

authority derivation failure
    -> repair authority premise/policy boundary

message payload closure failure
    -> repair message contract or remove undeclared semantic transfer

projection-preservation failure
    -> repair choreography/projector/semantics

assembly closure failure
    -> repair application wiring

runtime authorization failure
    -> current concrete operation is not admissible

trusted assumption violated
    -> trusted boundary or assumption definition is wrong

Chromium integration failure with semantic theorem still valid
    -> implementation-correspondence/trusted shell defect

liveness premise absent
    -> liveness conclusion is not currently available
```

This is why precise assurance terminology is not cosmetic. It makes a failed claim point toward the layer that owns the repair.

Assurance-model invariant

The governing invariant is:

> Every meaningful Gesso guarantee identifies the proposition being claimed, the evidence supporting it, the premises on which it depends, and the boundary between mechanically discharged facts and facts that remain runtime-enforced, trusted, asserted, conditional, or tested.
>
> No stronger assurance classification may be inferred merely because several weaker forms of evidence happen to agree.

The intended result is not that every statement becomes a formal theorem.

The intended result is that Gesso can state, mechanically and honestly, **why** each accepted artifact or application is believed to be valid, what remains outside that claim, and exactly where construction or execution must stop when one of its premises fails.

Semantic evidence vocabulary

The following vocabulary is shared by the later compositional hardening and whole-application proof architecture.

It is defined here so later layers do not need to redefine what their evidence means.

Identity and instance terminology

An identity names a particular semantic object or concrete instance. Its meaning must be explicit wherever admission, ownership, compatibility, or evidence binding depends on it. The canonical distinctions include:

- **Command identity:** the logical intent, which may survive multiple attempts.
- **Execution identity:** one concrete semantic execution of a choreography or operation; its relation to a command is explicitly declared.
- **Resource-instance identity:** one concrete timer, subscription, transport request, continuity capture, or other managed resource, distinct from its reusable logical resource/target name.
- **Transport-request identity:** one transport attempt, distinct from the domain object being acted upon and from the command it may carry.
- **Admitted assembly generation:** the immutable application realization admitted together, including its routes, providers, prepared operations, and artifacts.
- **Compatibility identity/version:** the declared protocol or plan compatibility class, distinct from exact executable-artifact identity and from deployment.

A deployment may host multiple admitted assembly generations during a supported transition. A deployment identifier, assembly generation, execution identity, and resource-instance identity are not interchangeable coordinates.

Use `generation` only when its owning subject and purpose are explicit, such as `assembly generation` or a declared `resource-instance generation counter`. A counter is one possible representation; opaque identity is sufficient where only equality and non-aliasing are needed. Do not imply an ordering, universal counter, or shared lifetime merely by calling a value a generation. A later reference may abbreviate a locally established subject only when it remains unambiguous.

For browser transport examples, `:transport-request-id` names the transport attempt. Domain Request arguments such as HumanHelp's `:request-id` retain their domain meaning. Different transport attempts must not be conflated because they carry the same command or concern the same domain Request.

Operation identity is **preserved** through realization: it cannot be silently erased or substituted. Reserve **monotonic** for a stated ordering or information- growth relation. In particular, preventing retired work from restoring obsolete presentation does not imply that all observations form one totally ordered history or that every valid domain transition increases a numeric value.

Executable semantic core, proof kernel, and dedicated checkers

The **executable semantic core** defines the production transition meaning for its supported semantic subset. It consumes the applicable executable rules, admitted state, and inputs/events, and yields semantic state and declared effects. A portable machine step or browser adapter transition function is an executable semantic core within its stated scope. There need not be one universal step function for all layers.

The **proof kernel** checks derivations against explicit inference rules and premises. It accepts or rejects the claimed judgment; it does not perform the application's runtime transitions or supply runtime authorization. It is separate from the richer elaborator that proposes terms and evidence. It must not trust the elaborator's conclusion merely because the elaborator supplied it.

A **dedicated semantic checker** establishes a particular judgment or property, such as exact finite-state exploration or structural application closure, through its declared checking mechanism. It need not be implemented as part of the proof kernel. Its result retains the evidence class and trust premises of that mechanism.

These components may share canonical semantic definitions. Sharing definitions neither merges their responsibilities nor establishes implementation soundness. The relationship between the checked subject and the production object remains an explicit correspondence obligation. Canonical terminology calls the runtime component the executable semantic core, reserving `proof kernel` for derivation checking; external proof-assistant kernels must be identified as external.

These definitions fix terminology and boundaries. They do not select a namespace layout or specify the later judgment calculus by implication.

Raw declaration

A **raw declaration** is user, application, or framework data that has not yet crossed the semantic checker appropriate to its role.

Examples include:

- a raw Malli or domain schema;
- a raw business-rule expression;
- a raw operation contract;
- a raw choreography;
- a raw authority declaration;
- a raw derivation;
- a raw dependency reference;
- a raw application assembly declaration.

Raw data is permitted at construction boundaries.

Raw data must not masquerade as a certified semantic artifact merely because it has the same map shape as one.

Admitted semantic value

An **admitted semantic value** is a value that has crossed the construction, verification, authoritative-acquisition, or other checking boundary required for the semantic category in which downstream code will consume it.

Admission answers a question such as:

> May this value be treated by the supported program as an instance of semantic category C at this boundary?

Admission does not automatically answer every stronger question about the value. In particular:

- admission into a nominal category is not automatically a proof of every proposition named by that category;
- admission does not create current authorization unless authorization is the proposition established by the trusted runtime boundary;
- admission does not make a client-carried value authoritative merely because it was once authoritative before serialization;
- an admitted artifact identity binds the relevant object to its construction history but does not establish semantic truth by identity alone.

A nominal type, opaque wrapper, private constructor, checked constructor, or registry-resolved object may help represent admission. The requirement is that unsupported construction or same-shaped raw data cannot silently receive the stronger semantic treatment through the normal API.

Semantic interface

A **semantic interface** is the normalized meaning exported by one checked component to its dependents.

It contains only propositions downstream reasoning is permitted to rely on.

Depending on the component, this may include:

- declared value types;
- variants;
- cardinality;
- optionality/refinement rules;
- relationships;
- operation input/output facts;
- preconditions;
- postconditions;
- authority requirements;
- provenance requirements;
- effects and resources;
- named assumptions;
- named guarantees;
- compatibility constraints.

Implementation details irrelevant to downstream reasoning should not become semantic commitments merely because they are convenient to serialize.

Assumption

An **assumption** is a named proposition accepted from outside the proof currently being established.

Assumptions are explicit leaves in the proof or explanation graph.

They are not inferred from trust in a namespace name, source location, familiar implementation, passing test, or matching hash.

Guarantee

A **guarantee** is a proposition established by the current checked component relative to its certified dependencies and named assumptions.

Obligation

An **obligation** is a proposition that must be established before a particular checked result or certificate may be issued.

An unresolved obligation must remain visible.

It must not be replaced by a default success value or silently omitted from the result.

Derivation

A **derivation** is machine-readable evidence describing how a guarantee follows from certified premises and inference rules.

Derivations are useful for:

- mechanical checking;
- diagnostics;
- explanation;
- repair;
- trusted-computing-base reduction when an independent checker is introduced.

A derivation is evidence, not runtime authority.

Certificate

A **certificate** is a closed evidence artifact stating enough information for an appropriate checker to decide the claims it represents.

A certificate may include:

- the semantic object checked;
- its semantic identity;
- certified dependency identities;
- assumptions;
- guarantees;
- proof obligations;
- derivations or checked references to derivations;
- applicable semantic-checker or proof-kernel version;
- result classification.

A certificate is not a boolean.

A certificate is not accepted because an ordinary Clojure map contains a key such as `:verified? true`.

A certificate is accepted only through the checker boundary appropriate to that certificate class.

Semantic identity

A **semantic identity** is a deterministic identity of normalized semantic meaning.

It is deliberately distinct from:

- source-file checksum;
- file path;
- namespace name;
- build timestamp;
- runtime locator;
- physical artifact location.

Whitespace, comments, local helper names, and semantically irrelevant implementation refactors should not necessarily invalidate downstream evidence.

Changes to relied-upon semantic commitments must.

Examples include changes to:

- value types;
- rule contracts;
- authority contracts;
- operation postconditions;
- provenance requirements;
- compatibility rules;
- resource semantics.

Semantic identity identifies normalized semantic material under the declared normalization and identity contract. A matching untrusted identifier is not self-authenticating evidence that the corresponding material was checked.

Semantic identity by itself does not prove the truth of that material. Evidence about a particular generated artifact or implementation must also remain bound to the concrete object to which that evidence applies. Semantic equivalence does not automatically transfer test results or physical artifact correspondence from an earlier build to a different one.

Proof DAG

A **proof DAG** is the dependency graph from a root checked artifact through all certified semantic dependencies and named assumptions.

A realistic root may eventually depend on:

```
schema certificates
    |
domain certificates
    |
relationship certificates
    |
business-rule certificates
    |
model-operation certificates
    |
choreography certificate
    |
projection certificate
    |
executable-plan certificate
    |
whole-application closure evidence
```

A later stage may rely only on propositions actually exported by its certified dependencies or named as assumptions.

No stage receives credit for a predecessor's intended meaning when that meaning was never certified.

The certificate dependency DAG must be acyclic. Recursive or fixed-point semantics may be supported only through a deliberately formalized rule with checked premises; introducing a dependency cycle does not supply that justification.

This requirement is distinct from ordinary recursive computation. In particular, a cycle among Clojure namespace dependencies is a load-time dependency error, not a mechanism for expressing logical recursion. Any recursive reasoning admitted by the certification system must therefore be represented explicitly in its semantics rather than encoded as a cyclic namespace or certificate dependency.

Construction principles

The following principles apply throughout the design.

They are not implementation-history advice. They are properties of the intended finished system.

State semantic intent once

The ordinary coherent formulation should state semantic intent in one canonical place.

If Gesso already knows:

- operation identity;
- participating role;
- projected browser plan;
- command schema;
- transport requirement;
- expected settlement;
- progression requirement;

then views, routes, browser registries, and server handlers should not all independently restate those facts unless an independent physical choice genuinely exists.

Prefer:

```
one semantic declaration
    |
    +--> derived browser metadata
    |
    +--> derived or checked transport binding
    |
    +--> derived diagnostics
    |
    +--> checked application closure edges
```

over:

```
operation declaration
+ unrelated button declaration
+ unrelated plan-name declaration
+ unrelated transport declaration
+ route operation string
+ bootstrap registry entry
```

When independently authored values must agree, correspondence must be checked explicitly.

A Claim card shows what ”state semantic intent once” should mean for ordinary application code. A plausible public authoring surface could let the operation own the semantic declaration and let presentation bind to that identity:

```
(defoperation request-claim
  {:operation/id :request/claim
   :actor :helper

   :input
   [:map
    [:request/id uuid?]]

   :requires
   [[:knows? :helper [:request/id]]
    [:runtime-premise :current-claim-authority]]

   :effect
   [:model/invoke :request/claim
    {:request-id :request/id
     :principal :authenticated-principal}]

   :provisional
   {:view :request/claiming}

   :settlement
   {:accepted {:reacquire [:request/card :request/id]}
    :rejected {:reacquire [:request/card :request/id]}
    :unknown  {:recover [:request/reconcile-claim :request/id]}}})

(defview request-card
  [{:request/keys [id status title] :as request}]
  [:article.request
   [:h3 title]
   [:p (name status)]

   (when (= status :open)
     (gesso/action
       {:operation :request/claim
        :args {:request/id id}}
       "Claim"))])
```

From those semantic declarations, Gesso may derive or validate lower-level realization data:

```
(project :request/claim)
;; ⇒
{:plan/id :plan/request-claim
 :operation :request/claim
 :role :helper
 :command-schema
 [:map
  [:request/id uuid?]
  [:gesso/command-id uuid?]]
 :transport-requirement :trusted-semantic-post
 :settlement
 {:accepted :request/card
  :rejected :request/card
  :unknown :request/reconcile-claim}}

(realize humanhelp-assembly
  :request/claim)
;; ⇒
{:browser/binding
 {:operation :request/claim
  :plan :plan/request-claim}

 :transport/binding
 {:kind :trusted-semantic-post
  :route :route/request-claim}

 :server/binding
 {:capability :server/request-claim
  :model-op :request/claim}}
```

The exact projection and realization representations are open design choices. What matters is that the view author did not separately type:

```
;; This is the kind of manually synchronized authoring Gesso should avoid.

[:button
 {:hx-post "/requests/claim"
  :data-operation "claim-request"
  :data-plan "humanhelp-claim-plan-v7"
  :data-command-param "__gesso_live_optimistic_command"
  :data-server-handler "claim-handler"
  :data-model-op "request/claim"}
 "Claim"]
```

That second version contains several strings that all purport to describe one semantic intent. Each value can be locally plausible while the collection is globally contradictory. A typo can easily produce a path that appears to render correctly and even sends an HTTP request while no longer corresponding to the declared choreography.

The desirable API therefore pushes complexity inward. Application code states the semantic operation and domain arguments; Gesso derives what it owns and checks correspondence where an independent physical choice genuinely remains. The illustrative `gesso/action` form is not an API commitment. It is a usability constraint on whatever public API eventually replaces it.

Derive rather than synchronize

Duplication creates programs whose pieces are individually plausible but mutually contradictory.

Whenever a fact can be derived safely from a stronger semantic source, derive it.

Do not create a second source of truth merely to make one layer easier to implement.

Fail at the earliest legitimate boundary

A contradiction should fail at the earliest layer that possesses enough information to establish it.

Examples of definition/construction-time failures include:

- malformed choreography;
- unknown role;
- impossible local transition;
- invalid message declaration;
- inconsistent operation metadata;
- invalid certified dependency reference.

Examples of build/application-preflight failures include:

- declared semantic affordance with no projected executable plan;
- stale generated artifact;
- missing transport;
- missing trusted route/capability;
- missing server execution capability;
- managed fragment with no permitted initial acquisition;
- authoritative successor with no permitted reacquisition path.

An affordance introduced only while constructing a concrete view is checked before that rendered surface is serialized or exposed. Static assembly preflight checks the affordances and operation bindings available from its declarations; it cannot claim to have inspected future arbitrary view data.

The same HumanHelp Claim slice can fail legitimately at three different times, depending on when enough information exists.

A static construction defect can be rejected before the application starts:

```
(def broken-assembly
  {:operations
   {:request/claim
    {:plan :plan/request-claim
     :route :route/request-claim}}

   ;; Accidentally generated from an older application assembly.
   :artifact
   {:plans #{:plan/request-create
             :plan/request-refresh}}})

(assembly/admit! broken-assembly)
;; throws:
{:error/type :assembly/missing-browser-plan
 :operation :request/claim
 :required-plan :plan/request-claim
 :artifact-plans #{:plan/request-create
                   :plan/request-refresh}}
```

Nothing about a concrete Request or principal is needed to establish that contradiction, so runtime is too late.

A rendered-surface defect may depend on concrete view construction. Suppose a generic Request-card renderer allows applications to choose actions, but one particular rendering asks for `:request/reassign` without supplying the required target relationship:

```
(render
  request-card
  {:request request
   :actions
   [{:operation :request/reassign
     :args
     {:request/id (:request/id request)}
     ;; required :helper/id is absent
     }]})

;; render preflight rejects before serialization:
{:error/type :view/operation-arguments-incomplete
 :view :request/card
 :operation :request/reassign
 :missing #{:helper/id}
 :available #{:request/id}}
```

The assembly may contain a perfectly valid Reassign operation. The defect exists only in this attempted concrete rendering, so render-time preflight is the earliest legitimate boundary.

Finally, current authorization may change after a completely valid render:

```
;; T1: Alice is currently an active helper.
(html
  (request-card
    {:request/id R
     :request/status :open}))
;; ⇒ contains a valid bound :request/claim affordance

;; T2: Alice's Membership is withdrawn.

;; T3: Alice clicks the still-visible affordance.
(invoke-claim!
  current-server-context
  {:request/id R :gesso/command-id C42})
;; ⇒ {:outcome :rejected :reason :membership-inactive}
```

Static preflight should not have rejected the application merely because a helper can later lose authorization, and rendering should not claim that the authorization can never change. The runtime boundary owns that dynamic fact and must reject the concrete invocation using current authority.

The three failures therefore form one continuous construction story:

```
known from declarations
    -> fail during assembly admission

known only for a concrete rendered surface
    -> fail before serialization / exposure

known only from current external/model state
    -> fail at trusted runtime invocation
```

”Fail early” means the earliest **legitimate** boundary, not the earliest point at which the framework could guess.

Runtime-only conditions and required recovery

Some facts relevant to an execution can be established only at runtime. Examples include:

- the current authenticated principal is not authorized for a concrete Request;
- the Request changed after it was rendered;
- a transaction guard fails;
- an external provider is unavailable;
- a browser or another required participant disconnects;
- an expected environment event has not occurred within an application-relevant lease, deadline, or interval.

Do not attempt to statically prove genuinely dynamic domain facts merely to increase preflight coverage.

Runtime-only establishment does not place these conditions outside the application's correctness model. The distinction is between knowing before execution whether a condition will occur and knowing before execution that the application has a coherent response if it does occur.

Where a declared operation or workflow can encounter a runtime condition that invalidates its premises, removes a participant required for progress, or would otherwise strand the workflow, the application must define a permitted path forward. Depending on the application, that path may involve rejection, authoritative reacquisition, retry, compensation, timeout, retirement, reassignment, cancellation, explicit degradation, or another declared successor state.

Static checking may therefore establish a conditional property such as:

```
if runtime condition F is established,
the admitted application has a declared path
from F to a valid recovery, successor,
or terminal state
```

without claiming to establish statically whether F will occur in a particular execution.

A Request changing after rendering is one example. Gesso cannot establish beforehand that the Request will remain unchanged until the user acts. It can require that an operation whose rendered premises have become stale is checked against current authority and model state and that failure of those premises leads through declared rejection, reacquisition, or recovery semantics rather than allowing execution to proceed from obsolete state.

A seat-reservation operation shows what that requirement can look like as a small executable contract rather than a prose-only promise.

```
(defoperation reserve-seat
  {:operation/id :flight/reserve-seat

   :actor :traveler

   :input
   [:map
    [:flight/id uuid?]
    [:seat/id uuid?]]

   ;; This observation justifies what the traveler saw. It does not reserve the
   ;; seat or guarantee that the observation remains current until invocation.
   :entry
   [[:authoritative-observation
     :seat/availability
     {:scope [:flight/id :seat/id]}]]

   :requires
   [[:runtime-premise :authenticated-traveler]
    [:runtime-premise :current-seat-state]]

   :effect
   [:model/invoke :seat/reserve
    {:flight-id :flight/id
     :seat-id :seat/id
     :traveler :authenticated-traveler
     :command-id :gesso/command-id}]

   :settlement
   {:accepted
    {:learns [:seat/reserved-by-self]
     :reacquire [:flight/seat-map :flight/id]}

    :rejected
    {:reason-set #{:seat/already-reserved
                   :seat/no-longer-offered
                   :traveler/not-eligible}
     :reacquire [:flight/seat-map :flight/id]}

    :unknown
    {:recover
     [:seat/reconcile-reservation
      {:flight-id :flight/id
       :seat-id :seat/id
       :command-id :gesso/command-id}]}}})
```

The admitted command boundary supplies `:gesso/command-id` once, before the first possible send. The authoritative model operation uses the identity for deduplication and reconciliation, and guards eligibility and seat availability **inside the same transaction** that records the reservation. Pre-reading seat state may provide a prompt rejection but cannot be the commit guard:

```clojure
(defn reserve-seat!
  [{:keys [db principal now]} {:keys [flight-id seat-id command-id]}]

  (seat-model/transact-reservation!
    {:db db :now now}
    {:flight-id flight-id
     :seat-id seat-id
     :traveler-id (:user/id principal)
     :command-id command-id

     ;; All of these are evaluated against transaction-current state;
     ;; the winning commit preserves them, or returns a typed rejection.
     :commit-guards
     [[:traveler/eligible? (:user/id principal) flight-id]
      [:seat/offered? flight-id seat-id]
      [:seat/unreserved? flight-id seat-id]]

     ;; The scoped identity binds principal, operation, flight, and seat.
     ;; A repeat returns the recorded outcome, never a second reservation.
     :deduplicate-by
     [:flight/reserve-seat (:user/id principal) command-id]
     :bind-arguments
     {:flight-id flight-id :seat-id seat-id}}))
```

The transaction API above is schematic: the implementation must prove that its chosen database transaction or compare-and-set mechanism rejects a conflicting commit, that changes to any guard-owned eligibility record cannot slip between the guard and the write, and that command deduplication is atomic with the reservation. A failed guard maps to `:seat/already-reserved`, `:seat/no-longer-offered`, or `:traveler/not-eligible` based on current authoritative evidence. When delivery or commit status cannot be established, the client retains `:unknown` and reconciles by the bound command identity; it must not infer rejection from a timeout.

A valid browser interaction may therefore begin from an authoritative observation at `T1` and still receive a typed rejection because Bob reserved the seat at `T2`. That rejection does not mean the earlier observation was fraudulent; it means the dynamic premise changed before the authoritative operation committed.

Preflight can check that `:rejected` is represented and that its reacquisition path exists. It cannot decide in advance which traveler wins the race. The example therefore makes the static/runtime boundary operational: dynamic uncertainty remains dynamic, while the permitted response to that uncertainty is still part of the checked application design.

Loss of a participant is another example. A physical network disconnect does not necessarily establish that the participant is semantically unavailable; temporary network failure, browser suspension, or reconnect may be ordinary execution conditions. The application therefore owns the policy that determines when loss of participation becomes semantically relevant. That policy may use a lease, heartbeat, deadline, authoritative presence state, explicit withdrawal, or another declared mechanism.

Once a participant is established to be unavailable, however, a workflow whose progress depends on that participant must not simply remain indefinitely dependent on an actor that can no longer satisfy its role. HumanHelp may decide that the affected Request becomes stale, becomes reassignable, returns to an available pool, remains assigned for a bounded lease, is cancelled, or follows another coherent application-defined path. Gesso does not choose among those policies, but absence of any declared handling is not itself an acceptable policy when the condition can leave the admitted workflow permanently non-serviceable.

The same distinction applies to transaction conflicts and external dependencies. A transaction guard may fail only at commit time, and an external service may become unavailable only during execution. Those facts need not be statically knowable. Their possible effect on an admitted workflow still requires declared semantics whenever failure can invalidate the current trajectory or prevent required progress.

Some workflows may deliberately rely on an environmental event eventually occurring rather than define a timeout or alternate path. In that case the corresponding liveness claim is explicitly conditional on that environmental assumption. Omitting the failure or non-participation path must not silently transform a conditional progress claim into an unconditional completion guarantee.

When a required premise is known not to hold, the dependent liveness conclusion is unavailable. A conditional eventual-recovery assumption must not be used to claim progress for a participant already established to be permanently unable to satisfy the required role. Safety obligations continue to apply while progress is unavailable.

Local timeout, retirement, cancellation of a browser resource, and physical disconnect must not invent an authoritative command outcome. The command's status remains participant-relative until the required outcome evidence is available. Application recovery and current authoritative eligibility remain model-owned even if a retired or disconnected browser later resumes.

The governing rule is:

> Dynamic facts need not be statically knowable, but dynamically possible conditions that materially affect correctness must have declared semantics sufficient to preserve the application's stated safety and liveness claims.

Fail closed rather than silently downgrade semantics

A semantically managed operation must never become an ordinary weaker operation merely because required semantic metadata is absent.

Forbidden examples include:

```
missing authoritative basis
    -> emit naked semantic hx-post anyway

missing browser plan
    -> guess a route and send anyway

unknown browser-artifact correspondence
    -> start runtime anyway

missing optimistic command
    -> perform ordinary mutation anyway
```

If a semantically managed realization cannot be constructed, the correct behavior is a precise failure or an explicitly modeled unavailable operation. The unavailable state must preserve operation identity and the reason its realization is unavailable. It does not satisfy a milestone that independently requires the operation to be usable in an eligible scenario.

This rule does not make ordinary HTMX illegal. Ordinary non-semantic HTMX remains valid where the application intentionally requested an ordinary HTMX interaction.

The prohibition is against silent loss of previously declared semantics.

Production boundaries carry the promised checks

A construction check contributes to an application's assurance only when the supported production path actually invokes it before the corresponding action becomes possible. An isolated checker test does not establish that relationship.

For a claimed production-enforced obligation, evidence must identify the boundary that admits the relevant object or action and demonstrate rejection of a representative controlled defect through that boundary. The corresponding valid fixture must remain usable, so a checker that rejects everything cannot satisfy the requirement.

The detailed startup, handler, generated-artifact, and rendered-surface boundaries belong to the whole-application construction part. Their assurance classification follows the three application assurance claims.

Keep authoring vocabulary small and canonical

The ordinary path should have one obvious representation.

If five public APIs express the same semantic operation binding, humans and LLMs gain four extra ways to build inconsistent applications.

Prefer:

```
explicit data
+ strong derivation
+ strong validation
+ strong explanation
```

over:

```
implicit magic
+ duplicated configuration
+ weak runtime discovery
```

Escape hatches may exist for unusual requirements, but they must be explicit and must not weaken unrelated invariants.

The same principle applies to semantic-category distinctions. Stronger internal admission should reduce downstream ambiguity rather than impose a manual wrapping tax. Where Gesso can mechanically derive or construct an admitted value from declarations it already owns, ordinary application authors should not be required to construct nominal wrappers, certificate maps, transitive proof objects, or authority tokens by hand.

The desirable developer experience is therefore asymmetric:

```
correct ordinary path
    -> little additional ceremony

semantic-category mistake
    -> early precise diagnostic

manual crossing of a strong semantic boundary
    -> deliberate, explicit API
```

Errors are part of the public correctness surface

Important failures should have stable structured representations.

A useful error should answer, where applicable:

1. What failed?
2. Which semantic object or operation was involved?
3. Where was the obligation introduced?
4. Which semantic/proof/application edge failed?
5. What was required?
6. What was actually available?
7. Which known alternatives are legal?
8. Which assurance class was expected?
9. Which assumption, if any, would be required to proceed?

Fuzzy suggestions may help diagnostics but must not silently alter security-sensitive execution.

Proof/check state is inspectable

A developer or LLM should not need to reconstruct application closure by mentally executing dozens of namespaces.

The finished system should be able to expose stable summaries of:

- semantic operation registry;
- role and authority declarations;
- projected plans;
- executable artifact identity;
- view affordances;
- transport bindings;
- trusted server capabilities;
- settlement/progression bindings;
- authoritative acquisition paths;
- open obligations;
- named assumptions;
- result classification for each guarantee.

Inspection is diagnostic.

Inspection does not create a new semantic input to runtime.

Stronger proof machinery must pay for itself

Every increase in proof complexity should answer:

> Which concrete malformed application, unsafe behavior, or required application behavior becomes impossible or immediately recognizable because of this addition?

If the answer is only theoretical elegance, the default is to defer the added formal machinery.

The proved language should grow vertically from real correctness requirements rather than horizontally toward maximal expressiveness.

Part II — Semantic model

This part defines the semantic distinctions shared by choreography, authoritative operations, browser realization, and application assembly. It uses the assurance and admission vocabulary established in Part I. The definitions do not require a universal persisted fact graph, a new runtime class for every concept, or a single global revision coordinate.

Values and evolving values

A value may be a singular immutable value:

```
42
:open
{:user/id user-id}
```

A value may also be understood as evolving across a time, revision, causal, or transformation-step dimension.

For example:

```
foo = :bar at T1
foo = :baz at T2
```

or:

```
foo = :bar at step S1
foo = :baz at step S2
```

The observations themselves may remain immutable facts.

The evolving value is the relationship among those observations and the rules used to select or interpret one observation relative to a particular point of view.

”Current value” is therefore not an eternal mutable fact.

It is a derived view relative to some combination of:

- valid/effective time;
- causal or transformation relation;
- authoritative basis;
- authoritative progression;
- actor knowledge frontier;
- application policy.

For a HumanHelp Request:

```
F1: Request R was unclaimed at valid time T1.
F2: Request R was claimed by user U at valid time T2.
```

Both statements can remain true under the model's declared valid-time and correction semantics. The historical fact that an observation was made is distinct from a claim that its interpretation of the world can never be corrected. Recording an earlier observation does not make an erroneous domain assertion eternally true.

The Request's current state depends on the point from which the Request is evaluated and on later facts that supersede, refine, reconcile, or otherwise relate to the earlier history.

The framework must not confuse:

```
fact existed
fact became authoritative
fact became visible at a database frontier
actor learned fact
actor derived another fact
actor displayed representation of fact
```

These are distinct events and relationships.

Conceptual fact graph versus required persistence

The semantic model may reason in terms of facts and relations among facts without requiring Gesso to persist a universal fact DAG.

The conceptual graph is useful because it allows the design to reason about:

- provenance;
- supersession;
- derivation;
- authority;
- agreement;
- causality;
- command/result relations;
- provisional versus authoritative state.

Actual persistence remains owned by application models and their chosen authoritative systems.

For HumanHelp, XTDB and public model semantics remain the primary persistence and authority boundary.

Gesso must not introduce a duplicate universal fact store merely because the proof vocabulary can describe facts.

Immutability and correction

The statement that observations are immutable does not prohibit an application from correcting its model of the world.

A later authoritative fact may state that an earlier interpretation was wrong, superseded, withdrawn, or effective over a different valid-time interval.

What must remain explicit is the relationship.

The system should avoid interpreting absence from a current projection as evidence that an earlier observation never occurred. This semantic distinction does not require indefinite storage of every observation, prohibit legitimate data deletion, or introduce an audit-history retention policy. Persistence and retention remain model-owned.

This distinction becomes important for:

- reconciliation;
- idempotency;
- audit;
- stale-command interpretation;
- provisional settlement;
- authoritative reread;
- domain-specific corrections.

Semantic value versus representation

A semantic value is not identical to a wire encoding, DOM representation, XTDB object, or Clojure map merely because one of those represents it.

Representations may change while semantic identity remains stable.

Conversely, two values with identical serialized shape may have different semantic meaning because their:

- provenance differs;
- authority differs;
- basis differs;
- principal differs;
- operation context differs;
- plan/protocol compatibility identity differs.

Shape equality alone is insufficient when the semantic judgment depends on distinctions not established by that shape. Ordinary immutable values may still use ordinary value equality when the enclosing contract makes it sufficient. Gesso must not mistake matching schemas or encodings for equality of authority, provenance, subject, observation scope, or admission status.

Representation, semantic category, and admission

Representation validity, semantic category, semantic admission, and proof are separate questions.

A Clojure map, record, scalar, wire value, DOM node, or other representation may carry a semantic value without the representation itself establishing the stronger status in which later code intends to use it.

The design therefore distinguishes conceptually:

```
representation has the required shape
        ≠
value belongs to the required semantic category
        ≠
value has crossed the admission/checking boundary for that category
        ≠
the strongest propositions associated with that value have been proved
```

This distinction matters most where two values may have the same or nearly the same representation while differing materially in provenance, authority, lifecycle, or assurance status. Examples include:

- `ObservedBasisClaim` versus an authoritative basis established by trusted acquisition;
- `CommandId` versus `ExecutionId`;
- provisional versus authoritative projections;
- raw versus checked choreography declarations;
- projected plans versus admitted executable plans;
- raw certificate data versus checker-accepted certificate evidence;
- authenticated observation evidence versus an operation capability;
- historical observation evidence versus current authorization.

The supported Gesso APIs must not silently treat one such category as another merely because the underlying values have equal map shape, equal scalar representation, matching discriminator keys, or compatible Malli schemas.

This is an admission rule rather than a requirement to turn every semantic concept into a new runtime class. Permitted realizations include, where they fit the existing implementation:

- nominal types or records;
- opaque wrappers;
- smart or checked constructors;
- values reachable only through a verifier, elaborator, authoritative acquisition path, or registry lookup;
- tagged representations whose tags are validated by the appropriate trusted boundary;
- existing Gesso representations that already make the distinction impossible or immediately rejectable.

The design requires the semantic distinction and its enforcement at the supported boundary. It does not prescribe a nominal-type rewrite when an existing mechanism already provides the required protection.

Nominality is especially useful as misuse prevention inside cooperating trusted application code. For example, if `CommandId` and `ExecutionId` are both physically UUIDs, a supported API should not make accidental interchange look ordinary merely because both values satisfy a UUID schema. Similarly, a raw map containing the fields of a checked certificate must not become accepted certificate evidence solely by carrying a discriminator such as `:certificate/type` or `:verified? true`.

A boundary must retain or establish the information required to distinguish the categories it promises to check. If two untagged UUIDs reach a boundary with no trustworthy category or correlation information, UUID shape validation cannot reconstruct whether one originated as a command or an execution.

The supported construction path must therefore preserve the needed distinction through checked construction, typed or opaque values, declared field bindings, trusted correlation, or another mechanism that actually enforces the contract. Merely assigning different variable names to interchangeable unchecked inputs does not establish that protection. Framework-owned construction should derive these bindings wherever it already possesses the required information.

Nominal or opaque representation alone is not a security proof. A value called `VerifiedChoreography` is trustworthy only to the extent that its construction or admission path actually establishes the claims downstream code relies on. Likewise, a Clojure record constructor is not treated as a cryptographic or process-isolation boundary.

Crossing an untrusted serialization boundary also strips any purely in-process admission status. Conceptually:

```
server-side admitted value
        |
        | encode / transmit
        v
untrusted bytes or client data
        |
        | decode + verify / re-establish trusted premises
        v
server-side admitted value, if justified
```

Deserializing data into the same nominal-looking representation must not by itself restore authority, trusted provenance, certification, or admission.

Ordinary downstream application authoring should not pay unnecessary ceremony for these distinctions. Gesso should preferentially construct admitted values behind verification, elaboration, acquisition, projection, model, and assembly boundaries. Routine application code should not be required to manually wrap every Request id, title, status, or other ordinary domain value merely to satisfy framework ceremony.

The practical acceptance criterion is:

> Where two same-shaped values have materially different semantic meanings, accidental substitution through the supported API is either structurally unavailable or rejected at the earliest legitimate boundary with a diagnostic that identifies the expected semantic category, the supplied category, and the missing derivation or admission step. If the supplied category cannot be established, the diagnostic identifies it as unknown or unadmitted rather than inventing a trustworthy origin from an untrusted tag.

An expense-approval path shows what that criterion means in application code. The following records are only one possible realization; the design does not require Gesso to turn every semantic distinction into a record. They are useful here because they make the admission boundary visible.

```
(defschema EmployeeValue
  [:map
   [:employee/id uuid?]
   [:employee/department keyword?]
   [:employee/risk-band
    [:enum :ordinary :enhanced-review]]])
```

```
(defschema ExpenseValue
  [:map
   [:expense/id uuid?]
   [:expense/employee-id uuid?]
   [:expense/department keyword?]
   [:expense/amount decimal?]
   [:expense/status
    [:enum :draft :submitted :approved :rejected]]])

(defrecord ProposedEmployee
  [value])

(defrecord ObservedEmployee
  [value
   authority
   basis
   projection
   observation-id])

(defrecord AdmittedEmployee
  [value
   authority
   basis
   projection
   principal
   admission-id])

(defn decode-employee-form
  [params]
  (let [value
        {:employee/id
         (parse-uuid (:employee-id params))

         :employee/department
         (keyword (:department params))

         :employee/risk-band
         (keyword (:risk-band params))}]

    ;; Malli establishes representation shape here.
    (m/assert EmployeeValue value)

    ;; The browser supplied the data, so shape success does not promote it
    ;; into authoritative employee knowledge.
    (→ProposedEmployee value)))

(defn observe-employee!
  [{:keys [db principal authorization employee-admission-registry]}
   employee-id]
  (let [{:keys [value basis]}
        (employee-model/read-authorized-current!
         db
         {:principal principal
          :authorization authorization
          :purpose :expense/approval
          :employee-id employee-id
          :projection :employee/approval-facts})]
```

```
    (m/assert EmployeeValue value)

    ;; This value has crossed the model-owned authoritative acquisition
    ;; boundary, so it carries provenance that the form value does not.
    (let [authority [:authority :employee employee-id]
          projection :employee/approval-facts
          observation-id
          (admission/issue-observation!
            employee-admission-registry
            {:principal (:user/id principal)
             :value value
             :authority authority
             :basis basis
             :projection projection})]
      (→ObservedEmployee
        value authority basis projection observation-id))))

(defn admit-employee-for-approval!
  [{:keys [principal authorization employee-admission-registry]}
   observed]

  (when-not (instance? ObservedEmployee observed)
    (throw
      (ex-info
        "Approval requires an authoritative Employee observation."
        {:error/type :semantic/wrong-category
         :expected :employee/observed
         :supplied
         (cond
           (instance? ProposedEmployee observed)
           :employee/proposed

           (map? observed)
           :representation/raw-map

           :else
           :semantic/unknown)})))

  ;; The record class is only a local category hint. This checks a
  ;; server-owned issuance binding before admitting its contents.
  (admission/require-observation!
    employee-admission-registry
    (:observation-id observed)
    {:principal (:user/id principal)
     :value (:value observed)
     :authority (:authority observed)
     :basis (:basis observed)
     :projection (:projection observed)})

  (when-not
    (authorization/can-observe-employee-for-approval?
      authorization
      principal
      (:authority observed))
    (throw
      (ex-info
        "Principal is not authorized to use this Employee observation."
        {:error/type :semantic/admission-denied
         :principal (:user/id principal)
```

```
         :authority (:authority observed)
         :projection (:projection observed)})))

  (let [admission-id
        (admission/issue!
          employee-admission-registry
          {:purpose :expense/approval
           :principal (:user/id principal)
           :value (:value observed)
           :authority (:authority observed)
           :basis (:basis observed)
           :projection (:projection observed)})]
    (→AdmittedEmployee
      (:value observed)
      (:authority observed)
      (:basis observed)
      (:projection observed)
      principal
      admission-id)))

(defn approval-context!
  [{:keys [principal] :as ctx}
   expense-id]

  (let [expense
        (expense-model/require-authorized-current!
          ctx
          expense-id
          :expense/approval)

        observed-employee
        (observe-employee!
          ctx
          (:expense/employee-id expense))

        admitted-employee
        (admit-employee-for-approval!
          ctx
          observed-employee)]

    {:expense expense
     :employee admitted-employee
     :principal principal}))
```

Now compare three values that contain the same nested Clojure map:

```
(def browser-proposal
  (decode-employee-form
    {:employee-id "98a47a10-0ab8-4a8a-8f61-d4bd598e7729"
     :department "engineering"
     :risk-band "ordinary"}))

(def authoritative-observation
  (observe-employee!
    ctx
    #uuid "98a47a10-0ab8-4a8a-8f61-d4bd598e7729"))

(def admitted-observation
  (admit-employee-for-approval!
    ctx
    authoritative-observation))

(= (:value browser-proposal)
   (:value authoritative-observation)
   (:value admitted-observation))
;; ⇒ true
```

Ordinary value equality is doing its job. The records contain equal domain data. The semantic categories answer a different question: what may this value justify at this boundary?

A downstream approval rule might intentionally accept only the admitted observation:

```
(defn manager-may-approve?
  [{:keys [employee-admission-registry] :as ctx}
   {:keys [expense employee principal]}]

  (when-not
    (and
      (instance? AdmittedEmployee employee)
      (admission/issued-for?
        employee-admission-registry
        (:admission-id employee)
        {:purpose :expense/approval
         :principal (:user/id principal)
         :value (:value employee)
         :authority (:authority employee)
         :basis (:basis employee)
         :projection (:projection employee)}))
    (throw
      (ex-info
        "Approval rule requires an admitted Employee observation."
        {:error/type :semantic/unadmitted-input
         :expected :employee/admitted-for-approval})))

  (and
    (= :submitted
       (:expense/status expense))

    (= (:expense/department expense)
       (get-in employee [:value :employee/department]))

    (= :ordinary
       (get-in employee [:value :employee/risk-band]))

    (= (:user/id principal)
       (:user/id (:principal employee)))))
```

The tempting substitution is exactly the one Gesso should make difficult:

```
;; Same inner map. Wrong semantic category.

(manager-may-approve?
  ctx
  {:expense expense
   :employee browser-proposal
   :principal principal})

;; ⇒ rejected before the rule is allowed to consume it
;; {:error/type :semantic/unadmitted-input
;;  :expected :employee/admitted-for-approval}
```

Serialization makes the distinction sharper. Suppose an admitted observation is encoded into HTML or EDN and later returned by the browser:

```
(def wire-value
  (pr-str
    {:employee (:value admitted-observation)
     :basis (:basis admitted-observation)
     :authority (:authority admitted-observation)
     :category :employee/admitted-for-approval}))

(def returned
  (edn/read-string wire-value))

;; This is still client-carried data. The browser was able to alter every field.
;; Recreating a similarly named record must not restore the old admission.
(→AdmittedEmployee
  (:employee returned)
  (:authority returned)
  (:basis returned)
  :employee/approval-facts
  principal
  (:admission-id returned))
```

A supported server API should instead either reacquire the authoritative Employee or validate a separately defined server-verifiable receipt whose contract actually establishes the premise required by the operation.

Even the reconstructed `AdmittedEmployee` above fails `admission/issued-for?` unless trusted server state contains a matching, still applicable admission bound to the actual principal, value, authority, basis, projection, and purpose. The same check on `ObservedEmployee` prevents a forged observation record from being laundered into admission. The wire example does not carry a server admission. A browser cannot create one by choosing record tags or an `:admission-id`; the registry is trusted server context and is never populated from decoded browser data. Realizing that registry across web nodes requires a declared shared or server-verifiable mechanism; a process-local cache alone cannot support a cross-node claim. The authorized read must enforce access **before** disclosing Employee data. Finally, an approval mutation still rechecks the expense and Employee policy against mutually compatible, transaction-current facts. This example's admitted observation supports the rule's input discipline, but does not by itself make an earlier pre-read safe for a later commit.

The lesson is not that records are magical security objects. They are not. The lesson is that shape, semantic category, admission, and proof can coincide in simple cases but must remain separable when later correctness depends on the difference. A good public API should preserve that distinction while hiding most of the ceremony from ordinary downstream code.

Business rules and the auditable semantic surface

A correctness-relevant business rule should not exist only as an incidental condition buried inside arbitrary implementation code.

For Gesso's purposes, a **business rule** is a named semantic proposition, relation, precondition, postcondition, transition restriction, authorization condition, disclosure condition, or other application judgment whose truth or applicability affects a correctness-relevant operation.

Possible application rules include the following. This list illustrates the kind of policy that belongs on the surface; it does not assert that each rule has already been adopted by HumanHelp:

- a Request may be claimed only while it is open;
- the claimant must have an active Membership in the Request's Organization;
- a requester may not claim their own Request;
- a completed Request is terminal;
- a protected observation may be disclosed only to an authorized principal;
- a command may be incorporated only under the authority and argument binding established for that command.

The semantic rule is distinct from any one Clojure expression that happens to implement or test it. A condition such as:

```
(= :active (:membership/status membership))
```

may participate in the realization of a rule such as `:membership/active-for-request-operation`. By itself, however, this expression establishes only that the particular `membership` value supplied to it has status `:active`. It does not establish whose Membership it is, whether it belongs to the Request's Organization, whether the authenticated principal is the claimant, or whether the Membership is still active when Claim commits. Those bindings and timing requirements belong in the rule's meaning. The expression is one implementation step; the named rule is the semantic proposition whose meaning should remain stable across refactoring, generated realization, formal translation, diagnostics, and audit views.

A Gesso application therefore has an **auditable semantic surface**: the compact set of named rules, operations, authorities, lifecycle transitions, invariants, knowledge/agreement requirements, and explicit trusted premises that determine its correctness-relevant behavior.

That surface should normally be substantially smaller than the application's full implementation. A large application may contain tens or hundreds of thousands of lines concerned with rendering, formatting, queries, adapters, integration plumbing, logging, caching, and incidental computation while its business semantics consist of a much smaller collection of rules and operations. Gesso should preserve that compression rather than require an auditor to rediscover policy by rereading the entire implementation.

The requirement is not that every conditional expression become a business rule. Conditions concerned only with presentation mechanics, formatting, local optimization, or other behavior outside the claimed semantic boundary need not be promoted merely because they are written with `if`, `when`, or a predicate call.

A condition is a strong candidate for the semantic surface when it controls or materially contributes to matters such as:

- authorization or eligibility;
- protected disclosure or observation;
- ownership or responsibility;
- lifecycle admission or transition;
- authoritative mutation;
- transfer of money, value, quota, inventory, or another conserved resource;
- command acceptance, incorporation, settlement, or retry meaning;
- establishment or use of authoritative knowledge;
- a domain invariant or cross-operation consistency rule.

The application may represent such rules using ordinary Clojure data, pure functions, checked constructors, existing model contracts, thin macro sugar, or another representation that preserves their semantic identity. This design does not require a second policy language or a macro-heavy pseudo-language. What matters is that the supported Gesso path can identify the rule and the semantic judgment it contributes, rather than knowing only that some opaque branch occurred in host code.

A rule representation may conceptually carry information such as:

```
{:rule/id :request/claim-requires-active-membership

 ;; Meaning:
 ;;   The authenticated helper must have an active Membership in the
 ;;   Organization of the Request being claimed when Claim commits.
 ;;
 ;; Rationale:
 ;;   Claiming transfers operational responsibility for the Request.
 ;;
 ;; Correctness significance:
 ;;   Principal binding, Request/Organization association, Membership
 ;;   ownership, and active status must be mutually compatible and
 ;;   protected or re-established at the authoritative commit boundary.
 ;;   A stale browser or server pre-read is not sufficient.

 :proposition
 [:and
  [:= [:membership/user-id]
      [:authenticated-principal/user-id]]
  [:= [:membership/organization-id]
      [:request/organization-id]]
  [:= [:membership/status] :active]]

 :subjects
 {:principal :trusted-authenticated-principal
  :request :current-authoritative-request
  :membership :current-authoritative-membership}

 :required-at :request/claim-authoritative-commit

 :applies-to
 #{:request/claim}}
```

The exact representation above is illustrative; its `:subjects` and `:required-at` entries express obligations, not evidence that current facts were acquired or that a transaction guard was implemented. The three field tests must refer to the **same** Membership, the trusted current principal, and the Request selected by this Claim. Part IV determines how an authoritative realization protects those facts and their relationship at commit. The semantic requirements are that the rule has stable identity, that its checked meaning is available to the parts of Gesso that rely on it, and that enough correspondence is retained for later assembly, diagnostics, certification, and formal artifacts to refer back to the same rule. An opaque Clojure predicate with the same name does not make that proposition mechanically established.

Business rules should also be generously explained in ordinary source. Where practical, the declaration or function that defines a rule should be surrounded by comments explaining at least:

- what the rule means in domain terms;
- why the rule exists;
- which subtle correctness boundary matters to its enforcement;
- any intentionally surprising exception or asymmetry.

Comments improve human and model review, but comments alone cannot be premises of a mechanical assurance claim because ordinary Clojure reading/compilation need not preserve them. Prose that must survive elaboration into generated semantic inventories, diagnostics, or assurance artifacts must therefore be retained as checked metadata, documentation strings, source metadata, or another explicit representation. The machine-checked semantic core remains distinct from its explanatory prose.

This produces two complementary views of the same rule:

```
machine-checkable semantic core
    : request/claim requires a current active Membership held by the
      authenticated helper in this Request's Organization at commit

human-auditable explanation
    : what "active" means here, why it matters, and why the check must
      participate in the authoritative transition rather than rely on an
      earlier browser observation
```

Neither view should silently substitute for the other. Explanatory prose does not prove the rule, and a terse machine predicate is not sufficient by itself for an application intended to remain auditable by humans.

When a correctness-relevant decision necessarily depends on application code whose semantics Gesso cannot inspect or derive, that dependency should still have an explicit semantic name and boundary. It may be represented as an opaque or trusted premise rather than being allowed to disappear into an unnamed implementation branch. Later certification and whole-application assembly determine what assurance may be claimed from such a premise; this part requires only that the semantic dependency itself remain visible.

The semantic-surface acceptance criterion is:

> A reviewer should be able to inspect the correctness-relevant business policy of a Gesso application as a compact set of named semantic rules and explicit premises without reconstructing that policy from unrelated implementation code. Refactoring may change the physical Clojure realization without erasing or silently changing the identity of the rule being realized. Any correctness-relevant dependency that cannot be represented or derived by the supported semantic path remains explicit rather than being mistaken for an assured rule merely because opaque host code returned a truthy value.

Facts, time, and authoritative progression

The design distinguishes at least the following dimensions.

Valid/effective time

Valid or effective time describes when a domain fact applies in the modeled world.

Examples:

- Membership became active on one date;
- a Request was open during one interval;
- Organization policy changed at a domain-effective time;
- a correction applies retroactively to an earlier period.

Valid time belongs to domain meaning.

Domain or causal relation

Domain or causal relations describe how semantic facts relate.

Examples include:

- based on;
- produced by;
- superseded by;
- reconciled by;
- rejected by;
- confirmed by;
- corrected by;
- derived from.

These relations are not necessarily database transaction order.

Authoritative database progression

Authoritative database progression describes the consistency/basis frontier at which authoritative facts are visible.

For HumanHelp's primary persistent authority, this is supplied through XTDB semantics.

An authoritative progression value may advance because unrelated data changed.

Therefore:

```
database advanced
≠
this Request changed
```

Actor knowledge frontier

An actor knowledge frontier describes what authoritative information a particular runtime actor has acquired through its admitted observation paths. It is scoped by the facts, authority, projection, and relevant temporal context of those observations. It need not be one token covering all actor knowledge.

For the supported browser implementation, the frontier records semantic acquisition and incorporation by that implementation. It is not a server proof that an adversarial browser or the human using it retained or understood the information.

Two browser actors may observe the same authoritative database at different frontiers.

The same principal may therefore have two browser tabs with different knowledge.

The database's current state and an actor's current knowledge are distinct.

Physical delivery time

Physical delivery time may matter operationally without becoming semantic authority.

An SSE notification may arrive after another message.

A callback may fire after an execution has been retired.

A browser may wake after a deployment.

The fact that an event arrived later in wall-clock time does not by itself make the event semantically newer or authorized to mutate current state.

Do not collapse dimensions into one revision integer

The framework must not collapse:

```
valid time
domain fact identity/version
causal relation
XTDB progression
actor knowledge frontier
command identity
execution identity
artifact compatibility identity
```

into one global monotonically increasing application revision.

A global revision number would erase distinctions that matter to correctness.

For example, XTDB may advance while Request R is unchanged.

Likewise, Request R may have meaningful valid-time history independent of when a database transaction was recorded.

A browser may hold knowledge through frontier X17 while authority has progressed to X23 without any relevant Request fact changing.

That is ordinary distributed execution, not necessarily staleness requiring rejection.

Progression as provenance

An observed authoritative basis records something approximately like:

> The representation from which this actor formed its command was justified by an authoritative read consistent through this frontier.

It does not mean:

- the entire database is still at that frontier;
- every fact the command depends on is unchanged;
- the client owns authority over that frontier;
- the client can prove merely by sending the token that it really performed the observation;
- the framework should reject every command formed from an older frontier.

The authoritative-operations and progression part defines the exact command, scope, and XTDB semantics, including scoped progression requirements.

At the ontology level, the important point is that authoritative progression is one provenance coordinate among several and is not a universal domain revision.

Roles, principals, actors, hosts, and authorities

These concepts are distinct and must remain distinct throughout declaration, projection, runtime execution, proof, diagnostics, and application assembly.

Role

A **role** is a static position in a choreography.

Examples:

```
:helper
:request-authority
:observer
```

A role describes a position in the distributed protocol.

A role is not automatically:

- a user;
- a browser tab;
- a process;
- a web server;
- an authorization grant.

A principal or actor may instantiate a role only under the rules of the particular operation and trusted runtime boundary.

Principal

A **principal** is the identity or entity whose authority is being exercised.

Examples:

```
[:user user-id]
[:service :humanhelp]
```

A principal is a semantic/security identity.

A principal is not identical to a browser actor.

One principal may have:

- multiple browser tabs;
- multiple devices;
- multiple protocol executions;
- retries through multiple hosts.

Those actors may hold different knowledge while acting for the same principal.

Actor

An **actor** is a concrete participant carrying a local projected machine, execution state, or knowledge state.

Examples:

```
[:client client-id]
[:service-instance logical-service-actor-id]
```

An actor gives semantic identity to a participant beyond one physical callback or request.

The exact runtime realization may be short-lived.

An actor identity does not require a permanently resident process.

Host

A **host** is the physical runtime that happens to execute a step.

Examples:

```
[:web-node node-1]
[:web-node node-3]
```

Hosts are replaceable physical realizations.

A retry may run on another host without changing:

- semantic command;
- principal;
- authority;
- logical operation;
- relevant actor identity, where the protocol preserves it.

A host does not acquire semantic authority merely because it executes authority code.

Authority

An **authority** is the semantic locus entitled to establish some class of authoritative fact.

An authority is named by an authority identifier such as:

```
[:authority :request]
[:authority :request request-id]
```

The identifier does not itself contain an authoritative fact, nor does it name a particular database record or server process. It names a locus in the application's semantic model. Application rules, together with Gesso's execution rules, determine which facts and transitions that locus is entitled to establish and under which premises.

For example:

```
[:authority :request R]
```

may govern authoritative facts concerning Request `R`, such as its current lifecycle state, ownership, or the incorporation of a command affecting it. The concrete facts may be represented by one or several XTDB documents and may depend on other authoritative observations, such as Membership or Organization state. Those storage objects help realize the authority; they are not themselves its semantic identity.

This distinction permits the physical representation to change without changing what the application means by:

```
[:authority :request R]
```

For example, the same authority could be realized using a different document layout, a different database, or a different web node while continuing to govern the same class of semantic facts.

Authority is therefore distinct from both **storage location** and **execution location**.

A physical web node may temporarily realize an authoritative operation without becoming the authority itself.

For a Request operation:

```
semantic authority
    [:authority :request R]

first host
    [:web-node A]

retry host
    [:web-node C]
```

Node `A` and node `C` may each exercise the same semantic authority while executing different attempts of the operation. Process-local identity or process-local memory does not determine the authority.

The authority survives host replacement.

Consequently, any fact that must survive retries, failover, or migration cannot derive its semantic force merely from having been remembered by the same process. It must instead be recoverable from authoritative state, from evidence validly issued on behalf of the semantic authority, or from another mechanism whose semantics are explicitly admitted by the application and Gesso.

Server is not a universal semantic identity

”Server” is normally an implementation location or a role category, not one unique actor identity.

Multiple web nodes may realize the same logical authority while holding different process-local state.

The design therefore avoids proofs or runtime assumptions that rely on:

```
same server process
= same semantic authority
```

or:

```
different server process
= different semantic authority
```

Neither equivalence is generally valid.

Relationship among role, principal, actor, host, and authority

A useful conceptual example is:

```
role
    :helper

principal
    [:user U]

actor
    [:client C]

host of the browser-side actor
    user's browser/device

authoritative role
    :request-authority

semantic authority
    [:authority :request R]

host currently realizing authority
    [:web-node A]
```

A later retry may change the final line while leaving the semantic operation otherwise intact.

Verification and diagnostics should identify which category an identity belongs to instead of overloading one generic ”id” field wherever practical.

Authentication establishes the principal binding. The operation's trusted admission and model policy establish whether that principal may instantiate a particular role for a particular subject. Actor and host identity provide correlation or realization information; neither substitutes for those decisions. Changing a browser tab, transport connection, or physical host must not silently carry permissions into a different principal or authority scope.

A collaborative editor makes the categories easier to follow when several of them change independently.

Suppose Alice is currently allowed to edit document D. One browser tab opens a projected editor actor and later reconnects through another web node:

```
(def editor-session
  {:semantic/role
   :editor

   :principal
   [:user #uuid "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa"]

   :actor
   [:client #uuid "c1000000-0000-0000-0000-000000000001"]

   :authority
   [:authority
    :document
    #uuid "dddddddd-dddd-dddd-dddd-dddddddddddd"]

   :operation
   :document/save

   :command
   #uuid "ca110000-0000-0000-0000-000000000077"

   :execution
   #uuid "e0000000-0000-0000-0000-000000000019"

   :transport-request-id
   #uuid "70000000-0000-0000-0000-000000000204"

   :host
   [:web-node :A]})
```

The first transport attempt disappears before Alice receives a response. The operation's retry contract permits another transport attempt through node C:

```
(def retry-session
  (→ editor-session
      (assoc
        :transport-request-id
        #uuid "70000000-0000-0000-0000-000000000205"

        :host
        [:web-node :C])))

(select-keys editor-session
  [:principal :actor :authority :operation :command :execution])
;; ⇒
;; same values

(select-keys retry-session
  [:principal :actor :authority :operation :command :execution])
;; ⇒
;; same values

(select-keys editor-session
  [:transport-request-id :host])
;; ⇒
;; http attempt 204 on node A

(select-keys retry-session
  [:transport-request-id :host])
;; ⇒
;; http attempt 205 on node C
```

The host and transport attempt changed. That does not create a new principal, document authority, semantic command, or necessarily a new protocol execution.

Now Alice opens a second browser tab:

```
(def second-tab
  (→ retry-session
     (assoc
       :actor
       [:client #uuid "c2000000-0000-0000-0000-000000000002"]

       :execution
       #uuid "e0000000-0000-0000-0000-000000000020"

       :transport-request-id
       #uuid "70000000-0000-0000-0000-000000000206")))

(= (:principal retry-session)
   (:principal second-tab))
;; ⇒ true

(= (:actor retry-session)
   (:actor second-tab))
;; ⇒ false

(= (:authority retry-session)
   (:authority second-tab))
;; ⇒ true
```

The two actors may now hold different local knowledge even though they act for the same principal and document authority.

Finally, the Organization revokes Alice's editing permission:

```
(defn admit-editor-operation!
  [{:keys [authorization]} session]
  (let [principal (:principal session)
        authority (:authority session)]

    (when-not
      (authorization/currently-allows?
        authorization
        {:principal principal
         :role :editor
         :authority authority
         :operation (:operation session)})
      (throw
        (ex-info
          "Principal may no longer instantiate the editor role."
          {:error/type :authorization/role-admission-denied
           :principal principal
           :role :editor
           :authority authority
           :operation (:operation session)})))

    session))

(admit-editor-operation!
  revoked-ctx
  retry-session)
;; ⇒ rejected

(admit-editor-operation!
  revoked-ctx
  second-tab)
;; ⇒ rejected
```

Neither browser actor had to disappear, and neither host had to change. The dynamic authorization relation changed while the principal, actors, authority, and physical hosts remained perfectly intelligible.

This is why Gesso benefits from category-specific identity in diagnostics. A generic message such as ”id mismatch” or ”server changed” would obscure the actual question: which identity changed, which one stayed stable, and which semantic correspondence does the current boundary require?

Logical identity and physical realization

Logical identity must not be inferred from incidental physical identity.

The following identities are distinct:

```
role identity
principal identity
actor identity
authority identity

semantic operation identity
semantic command identity
protocol execution identity

global choreography identity
projected-plan compatibility identity
exact executable artifact identity

logical fragment/target identity
logical resource/continuity-slot identity
concrete effect/resource-instance identity
concrete continuity-capture identity

transport-request identity (one transport attempt)
physical DOM-node identity
web-node/host identity

admitted assembly generation identity
deployment identity
```

Some applications may not need every category in every operation.

The architecture must nevertheless avoid conflating categories merely because one implementation currently uses the same value for two of them. Resource- instance identity is the general category; a transport-request identity or continuity-capture identity may be a specialized instance of it. Listing these categories does not require multiple independent tokens for the same instance. The representation must preserve whichever distinctions the admission contract actually uses.

A checkout operation puts most of those identities into one compact lifecycle. The following structure is intentionally verbose so the independence is visible:

```
(def checkout-attempt
  {:operation/id
   :checkout/place-order

   :command/id
   #uuid "11111111-1111-1111-1111-111111111177"

   :execution/id
   #uuid "22222222-2222-2222-2222-222222222219"

   :plan/id
   :plan/checkout-place-order-v3

   :artifact/id
   "sha256:browser-artifact-A12"

   :assembly/generation
   :assembly-A12

   :deployment/id
   :deploy-2026-09-21-1

   :cart/id
   #uuid "33333333-3333-3333-3333-333333333333"

   :order/id
   nil

   :transport
   {:transport-request-id
    #uuid "44444444-4444-4444-4444-444444444204"

    ;; Fetch callback ownership is distinct from HTTP attempt correlation.
    :resource-instance/id
    #uuid "55555555-5555-5555-5555-555555555051"

    :host
    [:web-node :A]}})
```

Suppose the request connection disappears before the browser knows whether the command committed. A permitted retry might preserve the semantic command while replacing the transport attempt and its resource instance:

```
(def retry
  (→ checkout-attempt
     (assoc-in
       [:transport :transport-request-id]
       #uuid "44444444-4444-4444-4444-444444444205")
     (assoc-in
       [:transport :resource-instance/id]
       #uuid "55555555-5555-5555-5555-555555555052")
     (assoc-in
       [:transport :host]
       [:web-node :C])))

(assert
  (= (:command/id checkout-attempt)
     (:command/id retry)))

(assert
  (= (:execution/id checkout-attempt)
     (:execution/id retry)))

(assert
  (not=
    (get-in checkout-attempt [:transport :transport-request-id])
    (get-in retry [:transport :transport-request-id])))

(assert
  (not=
    (get-in checkout-attempt [:transport :resource-instance/id])
    (get-in retry [:transport :resource-instance/id])))
```

A different recovery strategy might retire the original execution and begin a new execution that still tries to reconcile the same command:

```
(def reconciliation-execution
  (→ retry
     (assoc
       :execution/id
       #uuid "22222222-2222-2222-2222-222222222220")
     (assoc-in
       [:transport :transport-request-id]
       #uuid "44444444-4444-4444-4444-444444444206")
     (assoc-in
       [:transport :resource-instance/id]
       #uuid "55555555-5555-5555-5555-555555555053")))

(assert
  (= (:command/id retry)
     (:command/id reconciliation-execution)))

(assert
  (not=
    (:execution/id retry)
    (:execution/id reconciliation-execution)))
```

If an authoritative read later finds the created Order, a domain identity enters the picture without replacing any of the earlier operation identities:

```
(def reconciled
  (assoc reconciliation-execution
    :order/id
    #uuid "66666666-6666-6666-6666-666666666483"))

{:command (:command/id reconciled)
 :execution (:execution/id reconciled)
 :order (:order/id reconciled)}
;; ⇒
;; three different semantic coordinates
```

And a deployment may change the admitted assembly or artifact compatibility without saying anything about whether command 177 committed:

```
(def after-deploy
  (→ reconciled
     (assoc
       :assembly/generation :assembly-A13
       :artifact/id "sha256:browser-artifact-A13"
       :deployment/id :deploy-2026-09-21-2)))

;; This may force a new compatible executable realization.
;; It does not retroactively create, cancel, or settle the command.
```

The point is not that every runtime map must literally contain every field. The point is that a design using one generic `generation` or one generic `request-id` could not faithfully answer all of these questions:

- is this the same semantic intent?
- is this the same protocol trajectory?
- is this the same transport attempt?
- does this callback belong to the current physical resource?
- is this browser artifact compatible with the admitted assembly?
- which Order, if any, was authoritatively created?

The identities exist because those questions have different answers.

Semantic command identity

A semantic command identifies an attempted semantic operation.

It may survive:

- network retry;
- host replacement;
- a new protocol execution;
- another transport attempt;

where operation semantics permit that retry.

The command identity and uncertainty contract in the authoritative-operations part defines the relevant retry and attribution conditions. Possessing the same command identifier alone does not establish that a retry is permitted or that the command already committed.

Protocol execution identity

A protocol execution identifies one realization of a distributed trajectory.

Two executions may relate to the same semantic command without being the same execution.

Their resources and callbacks must not be interchangeable merely because the semantic command matches.

Plan compatibility identity

A projected plan may remain in a browser across deployment.

Compatibility identity answers whether that suspended or retried projection is safe to continue under the current server/runtime contract.

Compatibility does not necessarily require exact artifact equality. It must nevertheless follow an explicit supported compatibility rule. A runtime must not infer compatibility from matching names, approximate version ordering, or the absence of a visible error.

Exact executable artifact identity

Diagnostic and proof correspondence may require a stronger identity than runtime compatibility.

The exact executable artifact identity answers:

> Does this diagnostic or proof metadata describe these exact emitted runtime coordinates and transitions?

A plan/protocol compatibility identity and an exact executable digest therefore serve different purposes. Evidence tied to exact emitted coordinates must not be attached to a merely compatible artifact as though exact correspondence held. Conversely, a compatible continuation need not be rejected solely because diagnostic-only material changed, if the declared compatibility contract permits it and any exact correspondence claim remains accurate.

Admitted assembly generation and deployment identity

An admitted assembly generation identifies the immutable application realization accepted for use under the construction contract in Part VII. Its bindings include the relevant routes, prepared server operations, providers, rendering rules, executable plans, and artifact correspondence.

Deployment identity identifies a physical rollout or placement, if the implementation needs that distinction. One deployment may host more than one admitted assembly generation during a supported transition. Moving an admitted realization between hosts does not, by itself, change its semantic identity or establish a new compatibility relation.

Neither identity substitutes for command, execution, or resource-instance identity. Two concurrent executions using the same assembly generation still have separate ownership. Replacing a resource inside one execution need not change that execution's plan or the application's assembly generation.

Transport-request identity

A transport-request identity identifies one transport attempt. In browser/runtime examples its canonical field name is `:transport-request-id`. A new retry attempt has its own transport identity even when the command identity is retained under the operation's retry contract. The execution-to-attempt relationship is declared by the supported realization rather than inferred from shared field values.

HumanHelp's domain `:request-id` identifies a Request being acted upon. It does not identify an HTTP request. Domain arguments and route parameters retain that meaning; they must not be renamed as transport identities.

Event correlation identifies a candidate owner/attempt. Possessing a matching transport-request identifier alone does not establish semantic admission, current ownership, authorization, or command outcome.

Logical target identity

A logical UI target may survive replacement of its physical DOM node.

For example, a Request card may remain the same logical fragment after HTMX replaces the element representing it.

The runtime must not use physical-node identity as the sole semantic target identity when representation replacement is expected.

Resource instance identity

A replaceable physical or runtime resource must be distinguishable from prior instances of the same logical resource when events from those prior instances could still arrive.

For example:

```
logical slot S
    |
resource instance R1

replace R1
    |
resource instance R2

late callback from R1
    |
    X  must not mutate state belonging to R2
```

Resource instance identity answers a narrow question:

> Does this event belong to the currently admitted instance of this replaceable resource?

A resource-instance generation counter, scoped epoch, opaque token, UUID, object identity, or another suitable mechanism may represent resource-instance identity. The representation must provide the properties required by the boundary; additional ordering or structure is unnecessary unless the contract uses it.

A resource-instance generation counter is appropriate when the contract uses monotonic ordering between successive instances within that resource scope. A distinct opaque identity is sufficient when the only required question is whether an event belongs to the current instance.

The distinction must remain valid for as long as callbacks from an earlier instance can still reach the boundary. Reusing a token, resetting a counter, or wrapping a resource-instance counter must not make a retired instance indistinguishable from its replacement. A reset is safe only when a separate scope or lifecycle boundary prevents old events from being admitted as current.

Resource instance identity must not be used as a compressed representation of independent semantic facts.

An event may be inadmissible because it belongs to a retired resource instance, but it may also be inadmissible because its command, execution, plan, artifact, authoritative basis, ownership, principal, authorization context, or another required semantic correspondence no longer matches the current operation.

Those distinctions remain separate even when several of them happen to change at the same time.

For example:

```
continuity slot S
    |
resource instance R7
    |
execution X12
    |
authoritative basis B40

physical resource replaced

continuity slot S
    |
resource instance R8
    |
execution X12
    |
authoritative basis B40

late callback carrying R7
    |
    X  rejected because the resource instance is stale
```

In this case the execution and authoritative basis may still correspond. The callback is rejected specifically because it belongs to the replaced physical resource instance.

Conversely:

```
continuity slot S
    |
resource instance R8
    |
execution X12
    |
authoritative basis B40

authoritative state advances

continuity slot S
    |
resource instance R8
    |
execution X12
    |
authoritative basis B41
```

The resource instance has not changed, but an operation that requires basis B41 must not treat possession of R8 as evidence that B40 remains admissible.

This separation prevents a resource-instance counter, epoch, or opaque identity from becoming a universal proxy for semantic freshness. A numerical comparison is meaningful only within the declared counter scope and ordering contract; identities from different resource or execution scopes cannot be ordered merely because both happen to be integers.

A browser Refresh race shows how narrow that identity can be in executable code. Assume a logical board target remains `:request-board` while HTMX or the Gesso adapter replaces the physical fetch/resource used to refresh it.

The runtime might hold state resembling:

```
(def initial-board-runtime
  {:target
   :request-board

   :execution/id
   #uuid "70000000-0000-0000-0000-000000000012"

   :current-resource
   {:resource-instance/id
    #uuid "71000000-0000-0000-0000-000000000007"

    :kind :authoritative-acquisition

    :scope
    {:projection :request/board
     :organization-id
     #uuid "72000000-0000-0000-0000-000000000001"}

    :status :in-flight}

   :adopted-basis :B40})
```

Before Refresh A completes, the component is reconstructed and starts Refresh B:

```
(def after-replacement
  (→ initial-board-runtime
     (assoc
       :current-resource
       {:resource-instance/id
        #uuid "71000000-0000-0000-0000-000000000008"

        :kind :authoritative-acquisition

        :scope
        {:projection :request/board
         :organization-id
         #uuid "72000000-0000-0000-0000-000000000001"}

        :status :in-flight})))

(def refresh-a-result
  {:event/class :acquisition-complete
   :target :request-board
   :execution/id
   #uuid "70000000-0000-0000-0000-000000000012"
   :resource-instance/id
   #uuid "71000000-0000-0000-0000-000000000007"
   :basis :B41
   :html "<div>older acquisition</div>"})

(def refresh-b-result
  {:event/class :acquisition-complete
   :target :request-board
   :execution/id
   #uuid "70000000-0000-0000-0000-000000000012"
   :resource-instance/id
   #uuid "71000000-0000-0000-0000-000000000008"
   :basis :B42
   :html "<div>current acquisition</div>"})
```

A narrow owner check can reject A for exactly the reason that matters, but it must not call that narrow result semantic admission:

```
(defn current-resource?
  [runtime event]
  (let [resource (get runtime :current-resource)]
    (and
      (= :in-flight (:status resource))
      (some? (:target runtime))
      (some? (:resource-instance/id resource))
      (= (:resource-instance/id resource)
         (:resource-instance/id event))
      (= (:target runtime) (:target event)))))

(defn same-execution?
  [runtime event]
  (and
    (some? (:execution/id runtime))
    (= (:execution/id runtime)
       (:execution/id event))))

(defn correlate-acquisition-owner
  [runtime event]
  (cond
    (not (same-execution? runtime event))
    {:result :owner-rejected
     :reason :wrong-execution}

    (not (current-resource? runtime event))
    {:result :owner-rejected
     :reason :retired-resource-instance}

    :else
    {:result :owner-correlated
     :event event}))

(correlate-acquisition-owner
  after-replacement
  refresh-a-result)
;; ⇒
;; {:result :owner-rejected
;;  :reason :retired-resource-instance}

(correlate-acquisition-owner
  after-replacement
  refresh-b-result)
;; ⇒
;; {:result :owner-correlated
;;  :event ...}
```

That result proves only the narrow ownership/correlation fact needed at this stage. Full semantic admission remains a distinct boundary. Conceptually it can consume the correlated completion only after the rest of the acquisition contract has also been established:

```
(defn admit-acquisition-completion
  [trusted-context runtime event]
  (let [{:keys [result] :as owner}
        (correlate-acquisition-owner runtime event)]
    (if (not= result :owner-correlated)
      owner
      (acquisition-admission/admit!
        {:trusted-context trusted-context
         :runtime runtime
         :event event

         ;; These names stand for independently checked premises, not
         ;; declarations supplied by the event itself.
         :require
         #{:authorized-principal
           :expected-acquisition-plan
           :trusted-provenance
           :declared-observation-scope
           :compatible-progression
           :current-resource-owner
         :permitted-installation-right}}))))
```

The `:require` set is selected by the admitted acquisition contract, not by event fields or arbitrary callers. Source authenticity, observation scope, progression, and installation rights remain checks in that boundary. Owner correlation alone cannot make received HTML safe to install.

Notice what the owner guard does **not** infer. It does not claim that B42 is universally newer than every basis in the application. It does not prove that the returned HTML is authorized for some other user. It does not establish that the command which caused an invalidation succeeded. It establishes only that this completion belongs to the currently selected resource instance and execution. The later admission boundary establishes the additional premises required before downstream code may treat the completion as an admitted authoritative acquisition result.

A different event class can have a different semantic owner. For example, a shared advisory is not execution-private:

```
(def shared-advisory
  {:event/class :board-advisory
   :owner
   {:subscription/id
    #uuid "73000000-0000-0000-0000-000000000031"

    :source-instance/id
    #uuid "74000000-0000-0000-0000-000000000031"

    :scope
    {:organization-id
     #uuid "72000000-0000-0000-0000-000000000001"
     :projection :request/board}}

   :requirement
   {:kind :authority-may-have-advanced
    :scope :request/board}})

(def advisory-runtime
  {:board-subscription
   {:subscription/id
    #uuid "73000000-0000-0000-0000-000000000031"

    :source-instance/id
    #uuid "74000000-0000-0000-0000-000000000031"

    :scope
```

```
    {:organization-id
     #uuid "72000000-0000-0000-0000-000000000001"
     :projection :request/board}

    :status :open}})

(defn admit-board-advisory
  [trusted-context runtime event]
  (let [subscription (:board-subscription runtime)
        owner (:owner event)]
    (if-not
      (and
        (= :open (:status subscription))
        (some? (:subscription/id subscription))
        (some? (:source-instance/id subscription))
        (= (:subscription/id subscription) (:subscription/id owner))
        (= (:source-instance/id subscription)
           (:source-instance/id owner))
        (= (:scope subscription) (:scope owner)))
      {:result :owner-rejected
       :reason :wrong-subscription-or-source}
      (advisory-admission/admit!
        {:trusted-context trusted-context
         :runtime runtime
         :event event
         ;; Fixed by the admitted shared-event contract, not by the event.
         :require
         #{:declared-board-advisory-class
           :admitted-source-provenance
           :valid-scoped-requirement
           :permitted-shared-transition}}))))
```

The subscription check is only owner correlation. Shared semantic admission also checks source provenance and the permitted advisory transition. A client-supplied `:owner` or `:event/class` does not by itself prove either.

Assigning that advisory to whichever command execution happens to be current would be a category error. Conversely, marking a private callback as `:shared? true` in untrusted event data must not bypass the execution/resource checks above.

This is the executable consequence of owner-relative admission: first determine which declared semantic owner governs the event class, then apply the correspondence checks that owner's transition actually needs.

Required correspondence checks are semantic gates

At every runtime boundary, an event must satisfy the particular identity and correspondence conditions required by that boundary.

Depending on the operation, those conditions may include:

- current resource instance identity;
- command identity;
- execution identity;
- plan or artifact identity;
- authoritative basis;
- ownership;
- authenticated principal;
- current authorization;
- continuity or provisional ownership state.

A boundary need not check every category. It must check the categories on which its correctness actually depends.

These checks are not merely debug assertions. They determine whether a runtime event is semantically admissible.

The design therefore does not require one universal generation value. It requires independent semantic distinctions to remain independently representable and requires each runtime event to satisfy the correspondence conditions appropriate to the operation it is attempting to affect.

Admission follows the declared semantic owner

Every asynchronous event capable of influencing Gesso semantic state passes the common semantic admission boundary. The boundary first resolves the event's declared class and semantic owner, then applies the correspondences required by that class.

Conceptually:

```
normalized event
    -> resolve declared event class and owner
    -> validate owner-specific correlation and applicability
    -> validate the permitted transition
    -> admit transition or classify rejection/recovery
```

The owner is not universally a command execution.

Event class	Relevant owner and checks
Execution-private callback	Admitted execution, current resource instance, ownership, correlation, and permitted execution state
Shared fragment advisory	Declared subscription or fragment coordination scope, admitted source/correlation, current source instance where required, and valid advisory requirement
Authoritative acquisition completion	Admitted acquisition owner, request/resource correlation, returned observation scope and progression, and applicable installation rules
Continuity completion	Declared continuity owner or slot, current resource instance, and restoration rights still held for the target

This table identifies distinctions rather than prescribing one public event record or a closed enumeration of all future events. Each supported event class must define its own required correspondence contract.

An idle board can therefore admit a legitimate shared advisory without an unrelated command execution. A shared event must not be assigned to whichever execution happens to be current. An execution-private callback must not become shared merely because its execution was retired.

The declaration of an event as shared is part of the checked contract. An untrusted field such as `:shared? true` cannot select weaker admission rules. Shared admission permits only the declared shared transition; it does not restore a retired execution's private ownership or authorize installation of domain content carried by an advisory.

The browser-realization part defines the concrete normalization, transition, and effect contracts implementing this boundary. All such paths obey this owner-relative rule; there is no alternate callback path that may mutate semantic state without admission.

Trust boundaries and adversarial browser actors

A browser is an untrusted participant.

This statement is stronger than saying browser input should be validated.

The user controls the machine and may modify:

```
gesso-live.js
projected choreography data
local machine state
DOM
hidden fields
command envelopes
role names
principal ids
observed basis values
provenance labels
provisional/authoritative markers
available buttons
bound capabilities
diagnostic/proof sidecars
runtime correlation fields
```

Therefore no browser-originated value may confer trusted authority merely because it has the expected representation.

What the browser may propose

A browser may propose values such as:

- a semantic operation;
- operation arguments;
- a semantic command identity;
- an observed-basis claim used as protocol context;
- correlation information whose validity is checked independently;
- provisional local state affecting only its own representation;
- a projected-plan identity whose compatibility is checked by trusted code;
- transport data required to correlate a request with a server-side operation.

Proposal is not authority.

What the browser may not establish by assertion

The browser may not establish by assertion:

- authenticated principal identity;
- entitlement to a choreography role;
- model authority;
- authorization to operate on a protected target;
- authoritative provenance;
- an authoritative fact;
- a privileged server-machine state;
- permission to read protected data;
- permission to subscribe to protected Live state;
- the truth of a proof certificate;
- that a stale execution is current;
- that a provisional projection is authoritative;
- that an observed-basis claim proves prior observation.

Trusted server code must derive or check those propositions through trusted boundaries.

Trusted authoritative execution

For each protected authoritative operation, the supported trusted path must establish the following obligations. Their exact physical ordering may depend on the operation, but no protected action may occur before its required premises have been established:

1. establish authenticated principal identity from trusted server context;
2. resolve the requested semantic operation through the trusted current application/operation registry;
3. establish the permitted server-side trajectory rather than accepting a client-supplied privileged machine state;
4. re-evaluate current authorization and relevant model policy;
5. read the authoritative facts required by the public model operation;
6. invoke the public model operation that owns the authoritative transition;
7. produce only the settlement/progression outcomes admitted by the declared operation contract.

An authorization decision or relevant precondition established during preparation must be protected or re-established at the authoritative transition boundary wherever an intervening change could invalidate the operation. The atomic-operation contract defines those guards; browser freshness does not supply them.

The server must never treat a client-supplied machine continuation such as:

```
{:next-state :request-authority/commit-admin-operation}
```

as evidence that the client is entitled to reach that state.

Instead the browser requests a semantic operation, conceptually:

```
{:command-id C42
 :operation :request/claim
 :arguments {:request-id R}
 :observed-basis X17}
```

and trusted server semantics decide what operation and continuation are permitted.

Security boundary

A malicious browser can always lie to itself.

It may render false information, suppress buttons, manufacture buttons, change labels, or locally pretend an operation succeeded.

Preventing a user from corrupting their own local representation is not the security target.

The security target is:

> Compromising or replacing a browser projection may corrupt that browser's local representation and protocol participation, but must not increase the authority of its authenticated principal, disclose data unavailable to that principal, or produce an authoritative model transition unavailable to that principal.

This is a verification target, not merely an application coding convention.

Read and subscription authorization

Hiding content in a view is not access control.

Not rendering a button is not access control.

Not advertising a Live interest is not access control.

Every protected authoritative observation and subscription must be authorized from trusted server context.

An opaque or signed token may protect integrity, correlation, or accidental mutation.

Possession of such a token is not automatically authorization unless an explicit capability design says that possession has that semantic meaning and its applicability and validity conditions are satisfied.

The protected-observation and subscription contracts must state when authorization is checked and what later permission changes imply for subsequent disclosure. Initial subscription admission must not silently become an indefinite permission grant. The protocol may use revalidation, bounded grants, revocation, or another explicit policy consistent with the claimed protection. No such policy can make an adversarial browser forget data already disclosed.

Proof artifacts are untrusted in the browser

A proof sidecar or diagnostic certificate shipped to a browser cannot become a security capability.

An adversarial browser may fabricate, modify, remove, or replace it.

Authoritative security therefore depends on trusted server-side state and checked protocol boundaries, not the browser's possession of proof metadata.

This follows the general rule:

```
evidence about why code was accepted
≠
runtime authority to perform operation
```

Trusted host code and accidental-misuse prevention

The adversarial-browser boundary must not be confused with a claim that Gesso sandboxes arbitrary Clojure code executing inside a trusted application process.

Code that deliberately obtains a lower-level transaction capability, database handle, model implementation namespace, or another authority-bearing resource may be able to bypass the supported Gesso/model path. If such code can mutate the same authoritative state directly, it belongs to the trusted computing base for every guarantee that depends on those model invariants unless a stronger isolation boundary prevents the bypass.

Within that trusted host-language boundary, Gesso is primarily concerned with preventing **accidental semantic misuse** by cooperating application code. The normal path should make incorrect substitutions, missing derivations, stale artifacts, undeclared authority paths, and accidental bypasses difficult to express and easy to diagnose.

The intended progression is:

```
ordinary supported application code
    -> narrow semantic/model interfaces
    -> checked construction and runtime premises
    -> authoritative model operation

accidental mismatch
    -> early specific rejection

deliberate lower-level bypass by trusted host code
    -> explicit exit from the corresponding Gesso assurance claim
```

Architectural restrictions may reduce that trusted surface further. For example, application assembly, dependency checks, narrow capabilities, or namespace discipline may make ordinary view/handler code unable to reach a transaction primitive through the supported construction path. Later whole-application assurance specifies what such enforcement establishes.

These mechanisms are valuable even though a sufficiently determined trusted application author can rewrite the program or deliberately acquire a lower-level handle. The target is not to make trusted code physically incapable of cheating. The target is to make the coherent path obvious, accidental misuse hard, and deliberate escape from the claimed boundary conspicuous.

Knowledge

Knowledge is tracked with respect to actors or projected roles.

A fact or value may become known through several distinct provenance paths.

At minimum the semantic model recognizes:

- `:input`;
- `:communicated`;
- `:authoritative`;
- `:derived`;
- `:asserted`.

These labels describe justification, not merely transport. Their appearance in an incoming map does not establish the corresponding provenance. A knowledge judgment includes the subject and scope of the fact and the premises on which the permitted use depends; a provenance label alone is not the whole judgment.

The important verifier question is not:

> Which packet delivered this map key?

It is:

> Before this actor performs behavior that depends on fact F, what justifies that the actor knows F?

Input knowledge

Input knowledge is justified knowledge supplied at operation entry.

Conceptually:

```
F in declared inputs(A)
-----------------------
A knows F
provenance: input
```

Input knowledge is not equivalent to arbitrary function arguments.

The operation contract must say which incoming facts are accepted as justified entry knowledge and under what assumptions.

For example, a trusted server may supply an authenticated principal as input knowledge to a server projection.

A browser cannot create the same trusted input merely by serializing an identical principal id.

Communicated knowledge

Communication may establish knowledge constructively.

Conceptually:

```
A knows X
A sends declared X to B
B receives declared X
-----------------------
B knows X
provenance: communicated
```

An admitted choreography must justify the sender's knowledge of the value it transmits. At an adversarial transport boundary, a message's receipt does not prove that the sender satisfied that semantic premise; trusted validation or the explicitly stated assumption must supply the justification being claimed.

The receiver gains only the values declared by the message contract.

Arbitrary extra map keys must not silently become choreography context.

Communication establishes the fact that the declared value was communicated in that execution; it does not automatically establish that the value is authoritative domain truth.

Authoritative knowledge

An actor may acquire knowledge by observing a representation justified by authority.

Conceptually:

```
authority establishes authoritative F
B observes an authoritative projection containing F
at a basis sufficient to justify that projection
------------------------------------------------
B knows authoritative F at that observation frontier
provenance: authoritative
```

The physical representation may be:

- structured data;
- server-rendered HTML;
- another checked projection;
- an application-specific authoritative representation.

The representation format does not weaken the provenance if the trusted path from authority to observation is valid.

Derived knowledge

A role may derive a fact from other facts it already knows through a declared deterministic derivation.

Conceptually:

```
B knows X
B knows Y
D requires X and Y
D deterministically produces F under its declared semantic contract
-------------------------------
B knows F
provenance: derived
```

A derivation must declare enough information for the verifier/checker to know which inputs justify the result.

The system must not assume arbitrary separately executed Clojure functions produce equal results across actors unless that relationship is justified by the declared semantics. Any clock, randomness, external read, locale, or other environmental input affecting the result must be supplied as a declared input or remain an explicit assumption; a function name does not establish determinism.

Derivation preserves the premises on which the result depends. Deriving a value from a client assertion does not silently make the result authoritative. A derived result can support a stronger claim only when the declared derivation rule and its premises justify that claim.

Asserted knowledge

An explicit assertion supplies knowledge the verifier does not derive.

This is a legitimate escape hatch for facts whose truth Gesso cannot establish.

The assertion must remain visible as a trusted premise.

It must not be silently promoted to:

- `:derived`;
- `:communicated`;
- `:authoritative`.

The application should be able to explain which behavior depends on the assertion.

Knowledge use

Knowledge determines whether a role may, depending on the operation:

- transmit a value;
- consume a value;
- use a value as an FX input;
- correlate a message;
- choose a branch;
- rely on a discriminator when its projected behavior diverges;
- derive another value;
- construct provisional state;
- invoke an operation whose entry contract requires the fact;
- claim agreement with another role;
- establish a checked rule precondition.

A role must not consume facts it cannot justifiably know.

Knowledge is semantic, not incidental context

The presence of a key in a runtime map is not evidence of knowledge.

For example:

```
{:user/id U
 :request/status :open}
```

does not answer:

- who knows those values;
- why they know them;
- whether they are authoritative;
- at what basis they were observed;
- whether they may be transmitted;
- whether they may justify a branch;
- whether they are merely local provisional data.

The semantic machinery must not infer provenance from incidental map membership.

Knowledge and privacy

Knowledge is scoped.

Correctness does not require every actor to learn every fact.

Privacy, authorization, role relevance, and protocol structure may deliberately prevent knowledge propagation.

A proof that ”eventually everybody knows everything” would be wrong for many applications.

The useful property is:

> Every actor knows enough, with sufficient justification, for the behavior it is permitted to perform.

Knowledge derivations as inspectable evidence

Where knowledge is machine checked, the checker should retain enough derivation information to explain why a fact is known.

A boolean such as:

```
{:knows? true}
```

is insufficient as the long-term proof product.

The derivation should be able to expose leaves such as:

- input;
- communication;
- authoritative observation;
- deterministic derivation;
- explicit assertion.

These derivations belong to verifier/checker artifacts.

They do not need to be shipped as production browser state.

Authoritative observation as a knowledge path

Authoritative observation is a first-class semantic provenance path even when it is not represented as a first-class portable machine opcode.

This distinction is important for ordinary web applications.

A HumanHelp browser may learn facts such as:

```
Request R is approved
Request R is claimed
Membership U is inactive
Organization O is closed
```

by rereading an authoritative representation after Live indicates that relevant authority may have changed.

That knowledge is not weaker merely because it arrived as rendered HTML rather than as a dedicated choreography participant message.

The authority remains in the model/database.

The HTML is one representation from which the actor learns the fact.

Advisory invalidation versus authoritative observation

The normal causal relationship, when the application policy permits the acquisition, is:

```
authoritative mutation
        |
        v
model / XTDB
        |
        v
Live invalidation
        |
        v
application-permitted acquisition trigger
        |
        v
authorized authoritative reread
        |
        v
actor acquires newer authoritative knowledge
```

A Live invalidation means approximately:

> Relevant authority may have changed; a sufficiently current authoritative observation may be required for later behavior or offered for explicit adoption.

It does not mean:

> This particular domain fact is now true.

Therefore invalidation is advisory knowledge about possible change, not authoritative knowledge of the changed fact.

Receipt of the advisory does not by itself choose when the application adopts new content. Acquisition may follow an explicit user action, a declared automatic policy, or another admitted trigger. Any convergence claim that relies on such a trigger must name it among its liveness premises.

Observation scope and selective incorporation

An authoritative observation establishes only the facts and coverage justified by its acquisition contract. Its semantic context includes the authority, subject or projection scope, relevant basis/progression, and applicable temporal or query meaning. These need not all be repeated as fields when they are unambiguously supplied by the admitted contract.

A later observation of one Request does not establish acquisition of all board membership at the same database progression. An actor may hold individually justified observations from different acquisitions without possessing a single consistent snapshot of their union.

Consequently, installation of new authoritative content must preserve each observation's justified scope. Claims spanning multiple observations require the compatibility or consistent-acquisition premise appropriate to that claim. The mere existence of separate authoritative observations is insufficient.

The authoritative-operations and progression part owns the detailed scope and progression contract and advisory discharge rules. Browser installation follows the scoped installation contract. HumanHelp's selective-adoption policy is a use of those generic contracts.

Observed basis is protocol context, not proof of prior observation

A naked browser-supplied `:observed-basis` value is a client claim about the context from which the browser believes it is acting. It is not, by itself, proof that the browser actually received or incorporated an authoritative observation at that basis.

For ordinary Gesso commands this is intentional. The basis is useful protocol context for consistency, reconciliation, provenance, stale-command handling, and authoritative reacquisition even when the browser is adversarial.

For example, a browser may say:

```
{:request-id R
 :observed-basis B}
```

Here `R` is the domain Request identity, not a transport-request identity. The server may safely use `B` to understand the command's claimed starting context without treating `B` as permission:

```
client says it acted from B
        |
        v
server compares B with current authority
        |
        +-- compatible/current enough -> continue under normal model guards
        |
        +-- stale/incompatible -> reconcile, reject, or reacquire
```

Current authenticated principal binding, model authorization, authoritative state, and transaction guards remain decisive. A client may lie about `B` without thereby gaining authority.

The trust requirement changes if an application gives **prior observation itself** security significance. Examples include:

- an operation is allowed only after the principal was previously issued a particular authoritative representation;
- a continuation is valid only when it follows a specific server-issued observation;
- an acknowledgement must refer to an exact previously delivered semantic version;
- an application distinguishes ”client claims to have seen X” from ”the server can verify that it issued X to this principal/session.”

In that case a forgeable client field is insufficient. The application needs an explicit server-verifiable mechanism such as:

- a server-issued capability;
- a MAC-authenticated observation receipt;
- trusted server-side session state;
- a signed server attestation;
- a nonce or continuation identifier bound to a prior response;
- another mechanism whose trust semantics are stated explicitly.

The framework must never silently promote:

`ClientClaim<ObservedBasis>`

into:

`VerifiedObservationEvidence`

merely because the value has the expected shape or happens to equal a real basis.

A Message Authentication Code (MAC), for example HMAC-SHA-256, is one possible way to let an adversarial browser carry integrity-protected server evidence, under the named cryptographic, key-management, encoding, and verification assumptions. This example specifies the trust relationship rather than a complete cryptographic implementation. The server authenticates a canonical payload using a secret key known only to trusted infrastructure:

```
receipt-payload
    =
{:principal-id P
 :authority [:request R]
 :projection :request/public-row
 :basis B
 :purpose :prior-observation
 :issued-at I
 :expires-at E}

mac
    =
HMAC(server-secret, canonical-encoding(receipt-payload))
```

The browser may read, store, copy, and return the payload and MAC. Under the declared integrity assumptions it cannot feasibly change `P`, `R`, `B`, `purpose`, or another authenticated field and produce accepted replacement evidence. The MAC does not encrypt the payload; receipt contents must therefore be suitable for disclosure to the recipient.

On receipt, the server recomputes and verifies the MAC and then checks the context in which the receipt is being used:

```
untrusted client-carried bytes
        |
verify MAC
        |
verify principal / authority / purpose / expiry
        |
        v
server-authenticated prior-observation evidence
```

The MAC authenticates the statement represented by the payload; it does not make an unjustified statement true. If the server authenticates a bad or overstated proposition, the MAC faithfully preserves that mistake.

In particular, a receipt normally proves something like:

> Trusted server infrastructure issued an attestation for principal/session P about the declared semantic observation of authority A at basis B, under the receipt's stated issuance contract.

The issuance contract must identify the event actually attested: for example, construction of an authorized response or release of a response to a transport. Issuance is not proof of successful delivery, browser installation, retention, or human acknowledgement. Returning the receipt proves possession of the authenticated bytes, not those stronger events by itself.

It does not by itself prove:

> The browser installed the observation, or the human read, understood, or consciously incorporated it.

If human acknowledgement is semantically relevant, that is a stronger protocol and needs its own explicit proposition and evidence.

Observation evidence must also remain distinct from an operation capability. A valid authenticated receipt concerning an old authoritative observation may remain historically valid even after current authority changes:

```
valid observation receipt
≠
current authorization
```

For example, another helper may have claimed Request R after the receipt was issued. The receipt can still prove that an earlier observation was issued, while current authoritative reread and model guards correctly reject a new claim.

Gesso should therefore distinguish at least these concepts:

```
ObservedBasisClaim
    client supplied
    forgeable
    useful for consistency/reconciliation

AuthenticatedObservationReceipt
    server authenticated
    evidence about a prior server-issued observation

CurrentAuthoritativeObservation
    trusted acquisition satisfying its declared scope and consistency contract
    current relative to that acquisition, not a promise of perpetual freshness

OperationCapability
    explicitly scoped permission under declared conditions

CurrentAuthorization
    current trusted decision that a concrete principal may perform the
    operation against current authoritative state
```

None should implicitly coerce into another. In particular, historical observation evidence and possession of an operation capability do not silently become current authorization unless the declared capability semantics explicitly make possession sufficient and all of that capability's current validity conditions are satisfied.

Gesso may emit compact authenticated observation receipts alongside authoritative acquisitions when a supported protocol benefits from evidence of prior issuance. Routine issuance is an implementation option, not a baseline requirement justified solely by the cost of computing a MAC. Its key lifecycle, payload disclosure, expiry, and replay contract must be explicit. The authenticated object should be the **semantic observation** or a compact digest/identity of it, not arbitrary HTTP response bytes.

For example:

```
{:receipt/version 1
 :principal-id P
 :authority [:request R]
 :projection :request/card
 :basis B
 :semantic-digest D
 :issued-at I
 :expires-at E}
```

could be MACed as one canonical receipt.

Even if such receipts become common, ordinary `:observed-basis` should remain a distinct weaker protocol concept. Requiring cryptographic evidence whenever a client reports its starting basis would unnecessarily turn useful consistency context into a security credential.

Replay semantics also remain explicit. A MAC makes a token difficult to forge or alter; it does not make the token single-use. A reusable observation receipt may legitimately be replayable because it attests to a historical fact. A one-shot capability must enforce its declared consumption or replay policy. Expiry limits validity duration but does not by itself prevent reuse within that duration. A nonce distinguishes issuances but does not by itself record consumption. A consumed-token record or another trusted mechanism may enforce single use; command identity and idempotency may instead enforce a permitted duplicate-use outcome. Those are different contracts and must not be presented as interchangeable guarantees.

The governing rule is:

> When prior observation becomes a premise of authorization or another security-sensitive conclusion, the mechanism establishing that premise must cross an explicit server-verifiable trust boundary. Client assertion remains client assertion until that boundary is crossed.

Observation need not imply a long-lived waiting opcode

Many authoritative observations form entry knowledge for a later short choreography.

For example:

```
browser renders authoritative Request at X17
    |
execution ends

...later...

Live advisory arrives
    |
application policy or explicit user action permits acquisition
    |
browser performs authorized authoritative reread at X24
    |
new short choreography begins with knowledge from X24
```

The portable choreography language therefore need not acquire a universal `:observe` or durable wait operation merely to model ordinary web rereads.

If an application genuinely needs one semantic execution to suspend on an authoritative condition and later resume, that facility should be designed deliberately with explicit persistence, compatibility, ownership, and recovery semantics.

It must not appear accidentally as a consequence of normal Live behavior.

Observation and representation independence

Authoritative observation is semantic.

Its correctness must not depend on one specific presentation format.

A browser may acquire justified knowledge from an authoritative HTML projection when the server-rendered representation and its acquisition path establish the required facts.

Another application may use structured data.

The important proof obligations concern:

- authority;
- authorization;
- basis/progression;
- projection semantics;
- correspondence between representation and declared fact.

They do not concern whether the bytes happened to be HTML or EDN unless the representation mechanism itself introduces a relevant assumption.

Agreement

Agreement is stronger than two actors having values with the same shape.

Agreement means the protocol can justify that the actors share the relevant semantic value or fact under the declared relation, scope, and frontier.

This is a judgment about those actors' relevant observations or values at the specified points. It does not automatically mean that either actor knows the other actor received the value, that both know that both know it, or that they continue to share the same current value after later changes. Protocols needing acknowledgement or stronger knowledge about receipt must declare that additional obligation.

If A communicates X to B:

```
A knows X
A communicates declared X to B
B receives declared X
--------------------------------
A's sent observation and B's received observation agree on declared X
provenance: communicated, subject to the message contract's premises
```

Agreement can arise through multiple provenance paths.

At minimum:

- `:communicated`;
- `:authoritative`;
- `:derived`;
- `:asserted`.

Communicated agreement

Communication can establish agreement on the communicated value.

It does not imply agreement on every fact either actor knows.

Message contracts therefore matter.

Closed payloads make the agreement boundary explicit.

Authoritative agreement

Two actors may independently observe the same authoritative fact and obtain justified agreement without receiving one identical network packet.

For example:

```
authority establishes F

actor A
  reads F through web node 1
  at compatible authoritative frontier

actor B
  reads F through web node 3
  at compatible authoritative frontier

=> A and B may have authoritative agreement on F
```

The hosts differ.

The packets differ.

The semantic authority and relevant fact agree. The required agreement is on the scoped proposition being used, not necessarily on the exact read timestamp or complete database snapshot. Equal progression tokens alone do not establish agreement when the subjects or projection meanings differ.

Derived agreement

Agreement may propagate through a declared deterministic derivation when the inputs and derivation justify doing so.

The system must not infer derived agreement merely because two actors happen to run source code with the same function name.

The derivation relation must be part of the checked semantic contract.

Asserted agreement

Application code may explicitly assert agreement that Gesso cannot derive.

Such assertions are trusted leaves.

They must remain visible in diagnostics and proof reports.

They should not be used casually to suppress useful obligations.

Shape equality is not agreement

Two values can validate against the same Malli schema without being agreed semantic facts.

For example:

```
{:request/status :open}
```

in two browser contexts does not establish:

- same Request identity;
- same authoritative basis;
- same provenance;
- same valid time;
- same semantic observation;
- same authority.

Malli answers shape questions.

Agreement answers semantic relationship questions.

Agreement is scoped

Actors need not agree on all application state.

Agreement may be scoped to:

- a Request;
- a fact;
- a command;
- an operation input;
- a branch discriminator;
- an authoritative frontier;
- another explicitly modeled semantic object.

Privacy and irrelevance are legitimate reasons not to establish agreement.

Agreement frontiers

For evolving values, actors may agree only through a common frontier for the relevant facts and relation. A frontier is not necessarily one globally ordered position for all actor knowledge.

For a single modeled chain whose prefix observations are retained and justify the claimed agreement, for example:

```
F1 -> F2 -> F3 -> F4

Actor A knows through F4.
Actor B knows through F3.

A and B agree through F3.
```

The application universe may change continually.

Complete instantaneous agreement among all relevant actors may never occur.

That is not itself a correctness failure.

Correctness instead requires:

- established historical facts remain immutable under the model's semantics;
- relevant divergence is explicit rather than silently erased;
- actors possess enough justified knowledge for the behavior they perform;
- where convergence is required, the protocol provides a valid path toward a newer common authoritative frontier;
- under quiescence and the choreography's stated liveness assumptions, required agreement debt can fall to zero.

Frontier is relation, not necessarily arithmetic

A frontier need not be represented by an integer.

The browser must not invent arithmetic ordering over opaque progression values unless the authoritative system explicitly provides a supported ordering operation.

The relevant semantic questions are relational:

- is this observation at least as strong as the requirement that triggered it?
- are two authoritative observations compatible for the fact whose agreement is being claimed?
- has this actor incorporated the authoritative change it was required to observe?
- may this local projection continue under the current compatibility contract?

The concrete XTDB and scope-coverage mechanisms are specified in the authoritative-operations and progression part. An ordering relation must not be assumed to provide a join, a greatest common frontier, or complete observation coverage unless the supported contract actually establishes those properties.

Different actors may have different valid frontiers

For example:

```
Authority:
    F1 -> F2 -> F3 -> F4 -> F5

Actor A:
    knows through F5

Actor B:
    knows through F3

Actor C:
    knows only a privacy-scoped subset through F4
```

There may be no meaningful statement that all three actors share one universal application frontier.

Agreement and knowledge are scoped to the relevant facts and authorities.

Advisory knowledge and adopted knowledge

An actor may learn that authority advanced without immediately adopting the new authoritative projection.

This distinction is important for application behaviors such as a multi-user board:

```
other client learns:
    "new relevant Request authority is available"

but does not yet replace:
    currently adopted board projection

explicit refresh:
    acquires and adopts newer authoritative projection
```

The advisory requirements and adopted authoritative observations are therefore distinct pieces of actor state. The word frontier is shorthand for the scoped knowledge relation; it does not require either state to be represented by one maximum scalar token.

A correct application may intentionally preserve that distinction.

HumanHelp's board can represent that distinction explicitly instead of overloading one ”current revision” slot.

Suppose Alice creates Request R3 while Bob is viewing an adopted board containing only R1 and R2. A browser-side semantic state might look like:

```
(def bob-board
  {:actor
   [:client
    #uuid "81000000-0000-0000-0000-000000000002"]

   :principal
   [:user
    #uuid "82000000-0000-0000-0000-000000000002"]

   :projection
   {:projection :request/board
    :organization-id
    #uuid "83000000-0000-0000-0000-000000000001"}

   :adopted
   {:requests
    [#:request{:id
               #uuid "84000000-0000-0000-0000-000000000001"
               :status :open
               :title "R1"}

     #:request{:id
               #uuid "84000000-0000-0000-0000-000000000002"
               :status :open
               :title "R2"}]

    :observation
    {:authority [:authority :request-board]
     :scope
     {:organization-id
      #uuid "83000000-0000-0000-0000-000000000001"
      :projection :request/board}
     :basis :X17}}

   :advisory
   {:requirements []}})
```

Alice's create operation commits R3 authoritatively. Bob receives an advisory whose semantic content is deliberately weaker than the Request itself:

```
(def create-advisory
  {:event/class :board-advisory

   :owner
   {:subscription/id
    #uuid "85000000-0000-0000-0000-000000000001"
    :source-instance/id
    #uuid "86000000-0000-0000-0000-000000000001"
    :scope
    {:organization-id
     #uuid "83000000-0000-0000-0000-000000000001"
     :projection :request/board}}

   :scope
   {:organization-id
    #uuid "83000000-0000-0000-0000-000000000001"
    :projection :request/board}

   :requirement
   {:kind :newer-authority-available
```

```
    :after :X17}})

(def current-board-subscription
  {:board-subscription
   {:subscription/id
    #uuid "85000000-0000-0000-0000-000000000001"
    :source-instance/id
    #uuid "86000000-0000-0000-0000-000000000001"
    :scope (:projection bob-board)
    :status :open}})

(defn record-advisory
  [board admitted-advisory]
  ;; This is an internal semantic transition. The common shared-event
  ;; boundary has already validated source, owner, scope, and requirement.
  (advisory-admission/require-admitted-for!
    admitted-advisory
    (:projection board))
  (update-in board
    [:advisory :requirements]
    conj
    (:requirement admitted-advisory)))

(def admitted-create-advisory
  (advisory-admission/admit!
    {:trusted-context trusted-board-context
     :runtime current-board-subscription
     :event create-advisory
     :require #{:declared-board-advisory-class
                :admitted-source-provenance
                :valid-scoped-requirement
                :permitted-shared-transition}}))

(def bob-after-advisory
  (record-advisory
    bob-board
    admitted-create-advisory))

(get-in bob-after-advisory [:adopted :requests])
;; ⇒
;; still R1 and R2

(get-in bob-after-advisory [:advisory :requirements])
;; ⇒
;; [{:kind :newer-authority-available
;;   :after :X17}]
```

Bob has learned something real: relevant authority may have advanced beyond what his adopted board covers. He has **not** thereby learned R3's title, status, requester, visibility, or even that one particular Request must appear in his authorized projection.

The Refresh action performs a new authoritative acquisition:

```
(defn refresh-board!
  [{:keys [principal] :as ctx} board-state]
  (let [started @board-state
        scope (:projection started)
        acquisition
        (request-model/read-authorized-board
          ctx
          {:principal principal
           :organization-id (:organization-id scope)
           :projection (:projection scope)})]

    ;; Installation happens against the latest state, including advisories
    ;; received during the read. The admitted installer checks current owner,
    ;; trusted provenance, scope, and progression before replacing the board.
    (swap! board-state
      (fn [current]
        (board-admission/install-acquisition
          current
          {:acquisition acquisition
           :expected-scope scope
           :principal principal})))
    @board-state))

(def bob-board-state
  (atom bob-after-advisory))

(def bob-after-refresh
  (refresh-board!
    current-context
    bob-board-state))
```

The installer is a pure state transition, since `swap!` may call it again after a concurrent state change; it cannot send network requests or emit irreversible effects inside that function. It discharges only advisory requirements that the new acquisition actually covers under Part IV's scope and progression rules. It preserves newer or unrelated requirements and rejects a stale result if another acquisition already installed a stronger board or the owner changed. Evaluating that rule against the captured `started` value would erase an advisory received while the read was in flight. The principal comes from trusted context; a client field cannot choose whose board is read or installed.

If Bob is authorized to see R3, the new projection may contain it. If policy or privacy excludes R3, Refresh can still be a valid authoritative acquisition without leaking it.

Now compare Alice and Bob:

```
(def alice-knowledge
  {:actor [:client :alice-tab]
   :request R3
   :fact [:request/status :open]
   :provenance :authoritative
   :basis :X18})

(def bob-before-refresh
  {:actor [:client :bob-tab]
   :advisory
   [:authority-may-have-advanced
    {:after :X17}]
   :adopted-basis :X17})

(def bob-after-refresh-knowledge
  {:actor [:client :bob-tab]
   :request R3
   :fact [:request/status :open]
   :provenance :authoritative
   :basis :X19})
```

Before Refresh, Alice and Bob do not have authoritative agreement on R3's current board fact merely because Bob received an SSE event. After Refresh they may have scoped authoritative agreement on the fact that R3 is open and belongs in Bob's authorized projection, even though they acquired it through different requests and potentially different web nodes.

Agreement remains narrower than total-state equality. Alice may also know private requester details that Bob is not authorized to receive:

```
(def authority-state
  {:request/id R3
   :request/status :open
   :request/title "Need help moving a cabinet"
   :request/requester-id Alice
   :request/requester-phone "+1-555-0100"})

(def alice-projection
  {:request/id R3
   :request/status :open
   :request/title "Need help moving a cabinet"
   :request/requester-phone "+1-555-0100"})

(def bob-projection
  {:request/id R3
   :request/status :open
   :request/title "Need help moving a cabinet"})

;; Alice and Bob may agree on:
[:request/id R3]
[:request/status :open]

;; They need not, and must not, be forced to agree on:
[:request/requester-phone "+1-555-0100"]
```

The example makes the semantic model's use of ”frontier” deliberately non-global. Bob can owe adoption work for the board projection while already holding newer authoritative knowledge about some unrelated Request or account setting. Likewise, privacy may make full convergence impossible and undesirable. The meaningful question is always: agreement about which proposition, for which authority and scope, under which justified observations?

Convergence does not require replay of every intermediate change

A browser offline from X17 through X900 need not replay every invalidation between those points.

If the application semantics permit it, the actor may converge by acquiring a current authoritative observation at X900 or later.

The correctness obligation is not ”deliver every historical notification.”

It is:

> Provide a valid path by which the actor can acquire sufficiently current authoritative knowledge for the behavior it needs to perform.

Agreement debt

Where the choreography requires actors to converge, the difference between their relevant justified knowledge may be understood as agreement debt.

The design does not require one universal numeric debt metric.

The useful property is qualitative:

- divergence is represented;
- required convergence paths exist;
- stale or provisional actors do not pretend agreement already exists;
- under stated liveness assumptions and quiescence, the required divergence can be eliminated.

The existence of a recovery path is distinct from its eventual execution. Where adoption requires a user to activate Refresh, required eventual adoption is conditional on that action or another permitted acquisition trigger. Where the product permits indefinite deferral, remaining advisory information is not by itself a violation of an unconditional convergence guarantee; such a guarantee was never established for that deferred adoption.

Privacy constrains convergence

Agreement is always scoped by authorization and relevance.

The system must not satisfy a convergence property by leaking protected facts to actors that are not entitled to observe them.

A correct convergence path may therefore involve:

- one actor learning the full authoritative fact;
- another actor learning only an authorized projection;
- both actors agreeing on a smaller shared semantic proposition.

For example:

```
authority knows:
    full Request details + private requester metadata

helper A may know:
    Request is available for claim

observer B may know:
    Request count increased

A and B need not agree on private requester metadata.
```

Foundational invariant

The foundational agreement invariant is:

> A Gesso execution never treats mere co-presence, shape equality, physical delivery, or coincidental local computation as semantic agreement. Agreement must be justified by a declared communication, compatible authoritative observation, checked derivation, or explicit trusted assertion, and any claim over evolving facts is relative to a meaningful common frontier.

Part III — Choreography and projection

This part defines choreography as a global semantic specification, the operation contracts it exposes, and the obligations for deriving role-local behavior. It uses the evidence classifications of Part I and the identity, admission, knowledge, and authority model of Part II. Concrete model/FX integration follows in Part IV; browser realization and whole-application construction follow in Parts VI and VII.

What a choreography is

A choreography is a global description of a distributed state machine involving N roles.

The global choreography is projected into N role-local machines. At runtime, concrete actors instantiate those roles and execute the local machines independently.

There does not need to be a second centralized global machine executing beside them. The global choreography is the specification from which local machines are verified and projected.

Under the declared entry, runtime, and environmental premises, the interaction of admitted projected machines must refine behavior allowed by the global choreography. The refinement claim must identify its supported language subset, observable behaviors, and treatment of internal or physical steps.

This claim about cooperating projected machines does not assume that an adversarial browser follows its projection. Trusted-side admission and model enforcement must still preserve the protected authority boundary for arbitrary client inputs, as specified in the trust model.

A choreography may describe:

- local computation;
- communication;
- authoritative establishment and observation requirements;
- environmental events;
- branching;
- authoritative operations;
- provisional projections;
- resource acquisition and release;
- failure paths;
- timeout paths;
- settlement and reconciliation;
- convergence requirements.

A choreography does not imply that all participants change state atomically. Distributed actors may temporarily hold different knowledge.

The important distinction is between:

```
authoritative state
provisional state
actor-local knowledge
```

and not a false claim that the entire distributed system changes simultaneously.

A choreography also does not imply one persistent runtime machine per connected client or one persistent server process per conceptual workflow.

The global description may be realized by short-lived projections that interact through durable authoritative facts.

A substantial expense-approval slice shows what ”global description” means here. The following syntax is illustrative, but the separation among declared roles, authoritative observation, branching, communication, and authoritative effects is the intended semantic pressure.

First, assume the application already owns ordinary domain schemas and public model operations:

```
(defschema Expense
  [:map {:closed true}
   [:expense/id uuid?]
   [:expense/employee-id uuid?]
   [:expense/department keyword?]
   [:expense/amount decimal?]
   [:expense/status
    [:enum :draft
           :submitted
           :awaiting-finance
           :approved
           :rejected]]])

(defschema ApprovalPolicy
  [:map {:closed true}
   [:policy/department keyword?]
   [:policy/manager-limit decimal?]
   [:policy/finance-required-at decimal?]])

(defrule needs-finance?
  {:inputs {:expense Expense
            :policy ApprovalPolicy}
   :returns :boolean}
  [:≥
   [:expense/amount :expense]
   [:policy/finance-required-at :policy]])

(defoperation submit-expense
  {:operation/id :expense/submit
   :actor :employee
   :effect [:model/invoke :expense/submit]})

(defoperation manager-approve
  {:operation/id :expense/manager-approve
   :actor :manager
   :effect
   [:model/invoke :expense/approve-as-manager]})

(defoperation escalate-to-finance
  {:operation/id :expense/escalate-to-finance
   :actor :manager
   :effect
   [:model/invoke :expense/mark-awaiting-finance]})

(defoperation finance-approve
  {:operation/id :expense/finance-approve
   :actor :finance
   :effect
   [:model/invoke :expense/approve-as-finance]})
```

One global choreography can then describe how those operations relate:

```
(defchoreo expense-approval
  {:roles
   {:employee
    {:principal :user
     :actor :client}

    :manager
    {:principal :user
```

```
   :actor :client}

  :finance
  {:principal :user
   :actor :client}

  :expense-authority
  {:authority :expense}}

 :inputs
 {:expense-id
  {:schema :uuid
   :known-by #{:employee}
   :provenance :input}}}

(employee
  ;; Only a confirmed authoritative submission permits this message.
  ;; Rejected or unknown submissions follow their declared settlement and
  ;; reconciliation paths, not the manager continuation.
  (invoke expense-authority
    submit-expense
    {:expense-id expense-id})

  (send manager
    [:expense/submitted
     {:expense-id expense-id}]))

(manager
  ;; The manager is not allowed to branch merely because the employee sent
  ;; an Expense id. The branch depends on current authoritative facts.
  (observe expense-authority
    [:expense/current expense-id]
    → expense)

  (observe expense-authority
    [:expense/current-policy
     (:expense/department expense)]
    → policy)

  (branch
    (needs-finance?
      {:expense expense
       :policy policy})

    (then
      (invoke expense-authority
        escalate-to-finance
        {:expense-id expense-id})

      (send finance
        [:expense/finance-review-required
         {:expense-id expense-id}]))

    (else
      (invoke expense-authority
        manager-approve
        {:expense-id expense-id}))))

(finance
```

```
    ;; This local behavior exists only on the selected Finance branch.
    (on [:expense/finance-review-required
         {:keys [expense-id]}]

      (observe expense-authority
        [:expense/current expense-id]
        → expense)

      (invoke expense-authority
        finance-approve
        {:expense-id expense-id})))
```

The global form is not intended to run forever as one omniscient process. It is a specification from which local behavior can be derived.

This first choreography assumes a short supported execution in which the employee's confirmed submission and the manager/Finance responses have a declared delivery and progress contract. The multi-day realization later in this part uses durable model state and separate executions instead. In either realization, model operations recheck current status, principal eligibility, and applicable policy at their own guarded commit. A policy or status change after the manager's observations yields a declared rejection/reacquisition or retry path; it does not justify sending a Finance message for an uncommitted escalation. Finance's own operation also guards `:awaiting-finance` and current Finance authority. The observation and projection examples do not replace those model obligations.

A manager projection might contain only what the manager needs:

```
(defprojected expense-approval-manager
  {:role :manager
   :from expense-approval}

  (state :waiting-for-submission
    (on-message
      [:expense/submitted
       {:keys [expense-id]}]

      (goto :load-authority
        {:expense-id expense-id})))

  (state :load-authority
    (observe
      [:expense/current :expense-id]
      → :expense)

    (observe
      [:expense/current-policy
       [:expense/department :expense]]
      → :policy)

    (goto :choose-path))

  (state :choose-path
    (if
      (needs-finance?
        {:expense :expense
         :policy :policy})

      (then
        (invoke
          :expense/escalate-to-finance
          {:expense-id :expense-id})

        (send
          :finance
          [:expense/finance-review-required
           {:expense-id :expense-id}])

        (terminate :escalated))

      (else
        (invoke
          :expense/manager-approve
          {:expense-id :expense-id})

        (terminate :approved-by-manager)))))
```

The Finance projection is smaller because Finance has no behavior until the selected branch communicates the review requirement:

```
(defprojected expense-approval-finance
  {:role :finance
   :from expense-approval}

  (state :waiting-for-review
    (on-message
      [:expense/finance-review-required
       {:keys [expense-id]}]

      (goto :load-expense
        {:expense-id expense-id})))

  (state :load-expense
    (observe
      [:expense/current :expense-id]
      → :expense)

    (goto :approve))

  (state :approve
    (invoke
      :expense/finance-approve
      {:expense-id :expense-id})

    (terminate :approved-by-finance)))
```

The employee projection can be smaller still:

```
(defprojected expense-approval-employee
  {:role :employee
   :from expense-approval}

  (state :ready
    (invoke
      :expense/submit
      {:expense-id :expense-id})

    (emit
      [:expense/submitted
       {:expense-id :expense-id}])

    (terminate :submitted)))
```

Several design points are visible at once.

The manager projection contains the branch because the manager's behavior actually differs. Finance does not receive the branch discriminator in advance; it receives only the message that justifies its local participation. The employee does not learn the internal manager-versus-Finance choice because this particular choreography gives the employee no divergent local behavior after submission.

The projections are therefore smaller than the global specification without being independent handwritten workflows. Their legal traces remain related to the global choreography, and any knowledge used locally has to come from the entry contract, communication, authoritative observation, deterministic derivation, or another declared provenance path.

Choreography is specification, not a hidden coordinator

The global choreography is the semantic specification of allowed distributed behavior. It is not a requirement that Gesso run one omniscient global state machine alongside every projected role.

The implementation must therefore preserve two complementary truths:

- the global description is authoritative for what behaviors are admitted;
- projected actors execute locally and independently using only the knowledge, resources, and authority available to them.

A runtime architecture that requires every local step to consult a central choreography interpreter solely to rediscover projected control flow would defeat an important purpose of projection. This does not prohibit a declared authoritative operation from consulting its centralized model authority or checking a trusted runtime premise. It would turn the global specification into a distributed lockstep coordinator rather than a source from which local programs are derived and checked.

Conversely, independently executing local machines do not become semantically independent programs. Their combined traces remain constrained by the global description.

The intended relationship is:

```
global choreography
        |
        | verify / analyze / project
        v
role-local executable plans
        |
        | execute independently
        v
distributed concrete behavior
        |
        | must refine
        v
behaviors admitted by the global choreography
```

The absence of a central runtime machine is therefore not a weakening of the global specification. It is the point of a successful projection.

Semantic behavior versus physical realization

A choreography describes semantic behavior. Physical realization is permitted to vary when that variation does not change the semantic trace.

For example, the semantic statement:

```
:request-authority establishes the result of :request/claim
```

does not require:

```
one fixed Aleph process
one fixed HTTP connection
one fixed DOM node
one fixed browser tab
one fixed database connection
```

The authority may be realized through different physical resources at different times. A retry may reach another host. A fragment may be replaced by a new DOM node. A browser may reconnect. None of these physical substitutions creates a new semantic authority or silently changes the meaning of the operation.

This end-state design therefore requires every layer that crosses a replaceable physical boundary to preserve the logical identities needed by the semantic layer rather than reconstruct them from physical coincidence.

Detailed multi-node routing belongs to the later multi-node realization part of this design. The semantic requirement established here is only the stronger and more durable one:

> A host may realize a role or authority for an execution, but physical hosting does not create semantic authority.

Choreography may be partial with respect to the whole application

One choreography need not describe an application's entire lifetime.

A choreography may describe one distributed semantic operation or one coherent short-lived protocol. Durable model facts may connect many such protocols over hours, days, deployments, or browser lifetimes.

This means that the following are both legitimate:

```
one choreography
    claim Request R
    -> authoritative outcome
    -> settlement
    -> terminate
```

and:

```
choreography E1
    create approval request
    -> commit :approval/pending
    -> terminate

... later ...

choreography E2
    approve request
    -> commit :approval/approved
    -> terminate
```

The durable relationship between E1 and E2 lives in authoritative model state unless the application explicitly adopts a durable workflow facility.

This separation prevents Gesso from smuggling a universal workflow engine into ordinary Choreo merely because some business processes span human time.

Choreography validity and application realizability are distinct questions

A choreography can be internally valid while the surrounding application fails to realize a required use of it. An intentionally unexposed operation is not automatically an assembly defect: required exposure follows the declared realization scope and independent product acceptance.

For example, a verified operation may still be unusable because:

- no view binds it;
- no projected browser plan is packaged;
- the packaged artifact is stale;
- no transport binding reaches the trusted server;
- the server has no corresponding execution capability;
- no authoritative acquisition path can observe the successor state.

Missing required realization edges are whole-application assembly failures. Omitting an intended user capability can instead leave the remaining assembly closed while failing behavioral completeness. Those distinctions follow the three application assurance claims in Part I. Neither failure changes the meaning of choreography validity itself.

The distinction is important:

```
valid choreography
    ≠
complete application realization
```

and equally:

```
HTTP route exists
    ≠
valid choreography
```

The unified design therefore preserves Choreo as a strong semantic layer while also requiring whole-application checks of the joins between semantic and physical/application layers, production enforcement of those checks, and independent capability acceptance. Closure results retain their actual assurance classification rather than automatically becoming semantic theorems.

Construction must fail when the contradiction is already knowable

Choreography construction and verification must reject known contradictions at the earliest supported boundary possessing enough information to establish them. Unsupported constructs must be identified explicitly; they must not receive a successful result by omission from the checker.

Definition-time failures include, for example:

- unknown roles;
- malformed transitions;
- impossible branch references;
- invalid message declarations;
- inconsistent operation metadata;
- use of an undeclared authority;
- a local behavior that requires a fact that the role can never know.

These failures should not be deferred to browser execution merely because a runtime implementation could eventually notice them.

This does not mean Gesso should pretend to know dynamic domain facts in advance. A Request becoming claimed by another user after rendering remains a runtime/domain event. Static strictness applies only where the contradiction is already present in the declared semantic structure.

Canonical authoring principle

Application authors should state semantic intent once whenever the system can derive the rest.

A choreography should not require synchronized copies of the same semantic fact in:

- operation metadata;
- view metadata;
- browser-plan names;
- transport declarations;
- route strings;
- server registries;
- bootstrap registries.

Where an independent physical choice genuinely exists, it may be declared separately and checked for correspondence. Otherwise the preferred direction is:

```
one semantic declaration
    |
    +--> derived local plan
    +--> derived binding metadata
    +--> derived/validated transport requirements
    +--> derived diagnostics
```

This principle does not collapse all layers into one macro. It reduces duplicated sources of semantic truth while preserving explicit architectural boundaries.

HumanHelp's Claim action gives a useful target for that authoring principle because the same semantic intent must eventually appear in a view, browser plan, transport, trusted handler, model operation, settlement, and authoritative reacquisition path.

A declaration-first surface might begin with one semantic operation:

```
(defoperation request-claim
  {:operation/id :request/claim

   :roles
   {:helper
    {:principal :user
     :actor :client}

    :request-authority
    {:authority :request}}

   :input
   [:map {:closed true}
    [:request/id uuid?]]

   :entry
   [[:known-by :helper [:request/id]]
    [:provenance [:request/id] :input]]

   :runtime-premises
   [[:authenticated-principal :helper]
    [:current-membership :helper]
    [:current-request :request/id]
    [:current-organization
     [:request/organization-id :current-request]]
    [:current-claim-authority
     {:principal :authenticated-principal
      :request :current-request}]]

   :provisional
   {:role :helper
    :projection :request/claiming}

   :effect
   [:invoke :request-authority
    :request/claim
    {:request-id :request/id
     :principal :authenticated-principal
     :command-id :gesso/command-id}]

   :settlement
   {:accepted
    {:reacquire
     [:request/card :request/id]}

    :rejected
    {:reacquire
     [:request/card :request/id]}

    :unknown
    {:recover
     [:request/reconcile-claim
      {:request-id :request/id
       :command-id :gesso/command-id}]}}})
```

The view should be able to refer to that semantic operation without duplicating its lower-level names:

```
(defview request-card
  [{:request/keys [id title status] :as request}]
  [:article.request-card
   {:data-request-id id}

   [:h3 title]
   [:p.status (name status)]

   (when (= status :open)
     (gesso/action
       {:operation :request/claim
        :args {:request/id id}}
       "Claim"))])
```

Projection may elaborate the operation into a helper-local executable plan:

```
(project-operation request-claim
  {:role :helper})
;; ⇒
{:plan/id :plan/request-claim

 :operation/id :request/claim

 :entry
 {:known
  #{[:request/id]}

  :runtime-required
  #{:authenticated-principal
    :current-membership
    :current-request
    :current-organization
    :current-claim-authority}}

 :command
 {:schema
  [:map
   [:request/id uuid?]
   [:gesso/command-id uuid?]]

  :transport
  :trusted-semantic-post}

 :provisional
 {:projection :request/claiming
  :owned-by :gesso/command-id}

 :settlement
 {:accepted
  {:action :authoritative-reacquire
   :projection :request/card}

  :rejected
  {:action :authoritative-reacquire
   :projection :request/card}

  :unknown
  {:action :reconcile-command
   :operation :request/reconcile-claim}}}
```

The assembly can then choose a physical route and trusted capability while checking that they realize the semantic operation:

```
(bind-operation
  humanhelp-assembly
  {:operation :request/claim

   :view
   {:projection :request/card
    :affordance :claim}

   :browser
   {:plan :plan/request-claim}

   :transport
   {:kind :http/post
    :route :route/request-claim}

   :server
   {:capability :server/request-claim}

   :model
   {:operation :request/claim}})
```

The trusted handler still establishes dynamic premises rather than trusting the browser's view of the world:

```
(defn request-claim-handler
  [{:keys [session] :as ctx} request]

  (let [binding
        (assembly/require-current-binding!
          (:admitted-assembly ctx)
          :route/request-claim)

        command
        (server/decode-bound-command!
          binding
          request)

        principal
        (auth/current-principal!
          session)]

    (server/invoke-bound-model-operation!
      binding
      (assoc ctx
        :principal principal)
      {:request-id (:request/id command)
       :helper-id (:user/id principal)
       :command-id (:gesso/command-id command)})))
```

The admitted decoder binds the command identity, operation, arguments, and authenticated principal. The public Request model operation establishes transaction-current Request, Membership, and Organization premises and checks the claim rule in the same guarded transition that records the result. It first recognizes a compatible already-incorporated command under the declared idempotency and disclosure contract. A separate handler pre-read could race with the commit and could reject a retry after a successful claim because the Request is no longer open. Part IV owns the concrete model transaction; the handler above illustrates the required authority handoff, not the transaction implementation.

The syntax in all four blocks is provisional. The design pressure is not.

The application author should not have to synchronize:

```
;; Avoid this style of duplicated semantic naming.

{:button-operation "claim"
 :button-plan "claim-v9"
 :hx-post "/requests/claim"
 :wire-command "__gesso_live_optimistic_command"
 :handler-var 'humanhelp.routes/claim
 :server-capability "claim-handler"
 :model-op "request/claim"
 :settlement-fragment "request-card"}
```

when those values are all consequences of one declared semantic operation. Where the physical route or capability name genuinely remains independently authored, assembly construction should check its correspondence rather than hoping the strings stay synchronized.

This example also makes choreography validity and whole-application realization visibly different. The `request-claim` choreography can be valid while the `:request/card` view omits the affordance, while the current browser artifact omits `:plan/request-claim`, or while the route is not bound into the current assembly. Those are application-realization failures, not reasons to weaken the operation contract.

Choreographic operation types

A choreographic operation type is richer than an ordinary function type.

An ordinary function type is roughly:

```
A -> B
```

A choreographic operation type describes an entry into a distributed protocol and the valid continuations from that entry.

It may include:

- ordinary value schemas;
- evolving values and their basis;
- roles participating in the operation;
- principal and authority requirements;
- facts known by each role on entry;
- provenance justifying those facts;
- facts on which roles agree;
- resources required by operations;
- facts and values produced;
- possible outcomes;
- provisional execution permissions;
- authority transitions;
- continuation requirements;
- compatibility/version requirements where a suspended projection may outlive a deployment.

The operation type should **not** duplicate the complete choreography by manually listing every branch already represented by the choreography.

The choreography is the source of truth for control flow.

The type declaration supplies external semantic commitments that cannot or should not be inferred, while analysis derives as much as possible from the actual choreography.

For example, the declaration may state:

```
{::choreo/roles
 {:helper
  {:principal :user
   :actor :client}

  :request-authority
  {:authority :request}}

 ::choreo/inputs
 {:request-id
  {:schema :uuid
   :known-by #{:helper}
   :provenance :input}

  :observed-basis
  {:schema ObservedBasisClaim
   :known-by #{:helper}
   :provenance :input}}

 ::choreo/authority
 {:request-authority #{:request/claim}}

 ::choreo/optimism
 {:role :helper
  :projection :request/claim}}
```

This example declares role requirements, input knowledge, and the intended optimistic realization. It does not grant claim authority to whoever sends a matching role label. Trusted server admission must establish the concrete principal, subject, and operation premises. The helper's input knowledge of an `ObservedBasisClaim` is knowledge of the claimed token, not evidence that the helper acquired the observation it names.

The names `ObservedBasisClaim` and `AuthoritativeBasis` are conceptual semantic categories. They do not require two unrelated physical token formats, nor do they require a particular Clojure nominal-type implementation. The browser may carry the same underlying XTDB token bytes that trusted code later compares or uses to construct a consistency requirement, while the semantic status of those values remains different.

XTDB-specific representation belongs in the XTDB/Live consistency adapter rather than being permanently baked into the generic Choreo type vocabulary.

The verifier derives knowledge changes, branch requirements, and continuation structure from the choreography body.

The exact Clojure syntax remains tentative. These semantics are the intended contract.

Operation identity is semantic, not transport identity

A choreographic operation has a stable semantic identity that is distinct from the mechanism used to invoke it.

For example:

```
:request/claim
```

is not synonymous with:

```
POST /requests/123/claim
```

nor with:

```
button id="claim-request"
```

nor with:

```
browser plan index 17
```

The transport, view, and executable-plan coordinates are realizations of the semantic operation. They may change across refactors or deployments without changing the operation's meaning, provided compatibility rules permit the change.

This distinction is required so that application closure can later verify correspondence instead of mistaking physical equality for semantic identity.

A stable operation name is also distinct from the normalized semantic identity of its contract. Keeping `:request/claim` unchanged while changing a relied-upon authority rule, input requirement, outcome, or postcondition does not preserve dependent validity merely because the name stayed the same. Conversely, a transport refactor need not change the semantic contract if its checked realization and compatibility obligations remain satisfied.

Semantic categories at operation boundaries

Operation APIs must preserve distinctions whose accidental substitution would violate a semantic invariant even when the underlying Clojure representations are identical or structurally compatible.

Representative distinctions include:

- `ObservedBasisClaim` versus trusted `AuthoritativeBasis`;
- semantic `CommandId` versus runtime `ExecutionId`;
- provisional versus authoritative observations;
- raw declarations versus checker-admitted declarations;
- raw certificate data versus checker-accepted certificate evidence.

The requirement is semantic non-interchangeability through the supported API. The design does not mandate that every such category be implemented as a `defrecord`, Java class, wrapper object, or other nominal runtime type. Permitted realizations include:

- nominal or opaque host-language values;
- smart constructors with inaccessible or conventionally private construction;
- checker-produced admitted values;
- explicit tagged data whose use is guarded by the appropriate admission path;
- an existing Gesso representation that already prevents the accidental substitution.

The choice should be the smallest mechanism that makes the wrong ordinary call difficult and the resulting diagnostic precise. A representation technique is not valuable merely because it looks more typed.

Ordinary downstream application code should normally author domain values and semantic declarations, not manually wrap every UUID, basis token, command, or projection in framework ceremony. Gesso should perform category establishment at elaboration, checking, authoritative acquisition, model, or runtime boundaries wherever it can derive the distinction itself.

For example, a useful public failure is conceptually:

```
boundary:               trusted claim preparation requiring an acquired basis
requires:               AuthoritativeBasis for the declared scope
received:               unadmitted ObservedBasisClaim
missing step:           trusted authoritative acquisition / admission
```

rather than accepting the same-shaped value and relying on a comment to explain why the caller should not have done so. The diagnostic applies at the boundary that requires the acquired basis. A command intake may legitimately accept an `ObservedBasisClaim` as weaker context; it must not demand that the browser manufacture server-trusted authority before submitting a command.

The category and admission rules are defined in Part II. When the supplied category cannot be established, diagnostics report an unknown or unadmitted value rather than trusting a client-supplied category label.

A nominal host-language value also does not retain trusted status merely because it was serialized and reconstructed. Crossing an adversarial wire boundary returns the value to an untrusted representation until the relevant verifier, authenticator, decoder, or trusted acquisition path re-establishes the stronger semantic category.

This is primarily an accidental-misuse boundary inside cooperating application code. It does not claim that deliberately hostile trusted host code is unable to bypass Gesso, import lower-level implementation namespaces, or directly use a database capability it already possesses.

Operation contracts export a semantic interface

An operation type should expose a normalized semantic interface containing only facts that downstream reasoning is allowed to rely on.

Depending on the operation, that interface may include:

- input value types;
- required entry knowledge;
- required provenance;
- role participation;
- principal/authority constraints;
- model operation identity;
- possible semantic outcomes;
- values/facts produced;
- authority changes;
- resource requirements;
- provisional permissions;
- settlement requirements;
- continuation compatibility;
- named assumptions and guarantees.

It should not include arbitrary implementation details merely because those details are easy to serialize.

For example, the operation contract may establish:

```
operation:
    :request/claim

requires:
    helper knows Request identity
    for the declared optimistic realization:
        helper has acquired the required scoped authoritative Request projection
    trusted server binds authenticated principal
    request authority may invoke public request.core/claim

outcomes:
    :confirmed
    :reconciled
    :rejected
    :already-incorporated
```

The outcome names above describe authoritative/model semantic dispositions. They do not imply that every participant can immediately establish which disposition occurred. Participant-relative uncertainty and command-specific reconciliation are specified in the command/failure layer that follows this section. The supported operation must define which dispositions actually exist, their payloads, and the predicates or model outcomes that justify them. This illustrative list does not require every model to implement all four outcomes. An unknown participant-relative outcome is not silently converted into any of these authoritative dispositions.

The semantic interface does not encode incidental realization details such as:

```
specific Ring handler var
specific Reitit route index
specific DOM node id
specific XTDB Java class
specific browser bundle offset
```

Those belong to later realization layers unless independently required by runtime semantics.

External commitments versus derived structure

Operation declarations should state only semantic commitments that cannot or should not be safely inferred.

The verifier/projector should derive:

- reachable continuations;
- behavior-sensitive branch obligations;
- knowledge propagation caused by declared communication;
- terminal paths;
- local plan structure;
- resource obligations that follow from the body.

It should not require the author to restate those facts manually in metadata.

Redundant declarations are dangerous because each copy may remain individually plausible while disagreeing with the others.

The preferred rule is:

> Declare external semantic commitments; derive internal control-flow facts from the choreography body.

Availability is not authorization

An operation may be present in a model or choreography registry without being currently executable by a particular principal.

Likewise, a prepared or bound operation may be rendered as an affordance without becoming durable authorization.

These distinctions must remain explicit:

```
operation exists
    ≠
operation is valid in this choreography state

operation is valid in this choreography state
    ≠
current authenticated principal is authorized now

view renders operation capability
    ≠
browser possesses authority
```

The trusted side re-establishes dynamic authorization at execution and protects the relevant model premises at the authoritative transition boundary where intervening changes matter. A rendered affordance, a successful earlier check, and a valid operation contract do not reserve future eligibility.

This separation lets Gesso reason strongly about operation identity and distributed protocol structure without pretending to statically prove current business facts that can change between render and commit.

Malformed operation type versus wrong business policy

The unified design distinguishes structural/type malformedness from product correctness.

If a declaration says that `:request/status` is an enum and a rule attempts:

```
(< request.status 10)
```

the expression is malformed relative to its semantic types and should be rejected.

If the application states:

```
(= request.status :open)
```

when product policy actually intended another valid status test, the expression may be perfectly well formed while encoding the wrong policy.

Gesso should make the former difficult or impossible to certify. It cannot infer the latter merely from syntax.

This distinction becomes more explicit in the later compositional-proof part, where typed business-rule expressions and certified semantic interfaces are defined in detail. It is already a language-design constraint here because Choreo must not confuse ”type checks” with ”is the policy humans intended.”

A valid rule may still be inadmissible at a distributed program point

Intrinsic rule validity is not enough.

For example:

```
rule:
    (= request.private-review :approved)

intrinsic type:
    Boolean
    VALID

browser use before browser knows request.private-review:
    INVALID AT THIS PROGRAM POINT
```

A distributed use of a rule therefore creates obligations beyond ordinary type checking, including:

- the role knows every required fact;
- required provenance is available;
- trust or authority requirements are satisfied;
- optional values have been refined before use;
- authoritative-basis requirements are met where the rule depends on authoritative observation.

This is one of the central reasons Choreo needs a semantic type/knowledge layer richer than ordinary function typing.

Operation declarations should remain inspectable data

The ordinary authoring surface may eventually include macros or helpers, but the semantic representation after elaboration should remain explicit enough to:

- inspect;
- normalize;
- verify;
- project;
- fingerprint semantically;
- explain on failure;
- compare against dependent declarations.

Convenient syntax must not create an opaque second semantics.

Normalization must preserve the information needed to identify the owning declaration and explain failed obligations without making source coordinates part of semantic identity when they are diagnostic-only. The compiler or elaborator must not synthesize an undeclared trusted assertion simply to make a knowledge, authority, or continuation obligation pass.

The desired direction is:

```
pleasant author syntax
        |
        v
explicit normalized semantic data
        |
        +--> verification
        +--> projection
        +--> diagnostics
        +--> later certification
```

Malli as the value-schema substrate

Malli should provide the ordinary value-shape layer underneath choreographic types.

A choreographic value may conceptually carry:

```
{:schema ClaimCommand
 :known-by #{:helper :request-authority}
 :agreement #{#{:helper :request-authority}}
 :provenance :communicated}
```

or:

```
{:schema RequestProjection
 :known-by #{:helper}
 :provenance :authoritative
 :basis AuthoritativeBasis}
```

Malli is responsible for questions such as:

- is this a UUID?
- does this map contain the required keys?
- is this settlement one of the allowed values?
- does this intermediate machine context have the promised shape?

Malli is not responsible for questions such as:

- did this value come from a trusted actor rather than merely having a valid shape?
- may this helper claim this Request?
- is this Organization currently open?
- does this principal have authority?
- is this stale basis still semantically compatible with the operation?
- did communication actually establish knowledge?
- did an authoritative observation justify the actor's knowledge?
- does this branch satisfy the global choreography?

Those remain domain, FX, Choreo, or authority/progression concerns.

Runtime validators should be compiled and reused for the admitted schema and registry definitions whose identities they reference. If those bindings belong to an admitted assembly generation, validator use must remain coherent with that assembly generation. A relevant schema or registry change requires an appropriately recompiled or independently validated compatible validator; an old compiled validator must not silently enforce an obsolete contract. Validator correspondence does not depend on a browser execution identity or resource-instance counter.

The default design should validate important runtime boundaries in production, including:

- externally supplied choreography inputs;
- message payloads;
- authoritative observation/projection envelopes where they cross a runtime boundary;
- FX input/output boundaries;
- projected-machine state transitions;
- settlements;
- provisional/canonical protocol payloads.

The successful path should use compiled boolean validators. Rich Malli explanations should be constructed only on failure.

The default should be to keep these validations enabled in production and optimize only when measurement shows a real hot path. Measurement alone does not justify removing enforcement of a required premise. An optimization must preserve the contract, for example by avoiding redundant checks after a trusted admission boundary while retaining checks where untrusted or dynamic values can enter.

Shape validation is necessary but not semantic certification

Successful runtime validation establishes conformance to the particular schema under the validator and predicate contracts being relied upon. It is classified as runtime enforcement, not automatically as a formal theorem. A custom predicate's name does not establish an authority or provenance claim that its implementation never checks.

It does not prove:

- that a browser supplied the value honestly;
- that the value has authoritative provenance;
- that the current principal may use it;
- that another actor knows the same value;
- that a model operation may be invoked;
- that a choreography branch is globally legal.

This is not a limitation to work around. It is a deliberate separation of correctness scopes.

The unified design therefore treats ordinary value schemas as one layer in a larger semantic interface:

```
value shape
    |
    v
semantic type / refinement
    |
    v
knowledge + provenance
    |
    v
authority + operation contract
    |
    v
choreographic admissibility
```

A success at one layer cannot be silently promoted into a stronger proposition owned by a later layer.

The same rule applies when a stronger semantic category happens to use a nominal host-language representation. Malli may validate the fields inside an `AuthoritativeBasis` representation, for example, but shape validation does not establish that the value came through the trusted authoritative acquisition path. Conversely, an admitted value may still use ordinary immutable Clojure data internally. What matters is the checked construction/admission relation, not whether the physical carrier is a record or a map.

A discriminator such as `:type :verified` is therefore useful only as data unless the supported API also establishes why callers are entitled to rely on that category.

Closed value boundaries are the default

Where data crosses a semantic boundary, the declared schema should normally describe a closed payload.

Closedness is especially important for:

- client commands;
- choreography messages;
- provisional-state envelopes;
- settlement envelopes;
- authoritative projection metadata;
- FX inputs and outputs.

An open map is not harmless merely because consumers ignore unknown keys. Unexpected fields can become an accidental covert semantic channel when later code begins to read them.

When an application genuinely needs an open payload, that must be explicit and its semantic treatment visible to verification and diagnostics. Transporting opaque extension data need not grant it first-class knowledge status. If a program wishes to branch on an extension field or consume it as authority, correlation, or an operation input, that use requires a declared contract and the appropriate admission.

Semantic payload closure is distinct from rejecting every byte of physical transport metadata. The decoder may separate declared payload from transport metadata through an explicit contract. It must reject undeclared semantic fields at a closed boundary or keep explicitly permitted nonsemantic data outside the semantic context; it must not silently merge it into that context.

Runtime validation preserves static premises

Static reasoning depends on runtime values continuing to satisfy the contracts under which the reasoning was performed.

Consequently, runtime checks are not ”extra tests” after proof. They are enforcement of proof premises where untrusted or dynamic values enter.

Representative boundaries include:

```
browser bytes
    -> command schema validation
    -> trusted operation lookup

external read result
    -> projection-envelope validation
    -> trusted acquisition / scope / provenance checks
    -> justified knowledge establishment

FX implementation result
    -> declared output validation
    -> next semantic step

settlement payload
    -> settlement schema validation
    -> declared source / command / owner correlation checks
    -> admitted browser transition
```

If a runtime premise fails, execution must reject or recover according to the declared protocol. It must not continue on the theory that static verification made runtime validation unnecessary.

Validators should be reusable and diagnostic failure should be rich

The successful path should use precompiled validators where possible.

The failure path should preserve enough information to explain:

- which semantic value failed;
- which schema or refinement was expected;
- where the value entered;
- which operation or transition depended on it;
- whether failure concerns shape, provenance, authority, or another layer.

A raw Malli explanation may be part of this evidence, but downstream error reporting should not force users to reconstruct the semantic context from a low-level schema mismatch alone.

Schema changes participate in semantic compatibility

A schema is not merely runtime validation configuration when another semantic artifact relies upon its meaning.

Changes such as:

- removing an enum member;
- changing required versus optional;
- changing cardinality;
- changing a field's nominal semantic type;
- changing a closed variant set;

may invalidate dependent operation, rule, or choreography artifacts.

Later proof/certificate work therefore assigns semantic identities to normalized interfaces rather than source files. These operational schema requirements do not depend on completion of the later certificate machinery. The language/model boundary must nevertheless expose those semantic dependencies instead of hiding them inside arbitrary host-language code.

Projection and branch knowledge

Projection creates a local machine for each role.

Projection must preserve whatever mechanism is required to justify local behavior.

Sometimes that mechanism is communication.

Sometimes the actor already has justified entry knowledge.

Sometimes the relevant fact is learned from authority outside the current execution and enters a later choreography as authoritative input knowledge.

The projector/verifier must therefore reason in terms of knowledge requirements, not merely message presence.

A role may branch only when it knows enough information to choose the branch.

The verifier should use behavior-sensitive choice analysis:

- if a role behaves differently between two branches, it must know the discriminator or an equivalent fact before its behavior diverges;
- the required knowledge may be justified through communication, authoritative observation, input knowledge, declared derivation, or explicit assertion;
- if its behavior is equivalent across the branches, it does not need to learn the choice merely because it is syntactically mentioned.

This is intentionally more precise than a simple ”every role mentioned in either branch must know the condition” rule.

For example:

```
authority chooses approved/rejected

helper behavior differs afterward
        |
helper must know enough to distinguish the continuations
```

That requirement does not imply a dedicated authority-to-helper choreography message if the helper can instead obtain the required approval fact through a declared, authorized authoritative observation that satisfies the same branch obligation.

The substitution must preserve subject, scope, provenance, and the relationship to the decision being continued. Reading a currently approved Request does not necessarily establish which earlier command was approved or which historical branch was selected. If later model changes can erase or supersede the needed fact, the protocol needs a suitable retained outcome, receipt, communication, or different continuation contract. The projector must not invent that history or treat an arbitrary later read as equivalent evidence.

Messages should be closed by default.

A message contract declares the fields that may cross the boundary. Arbitrary extra fields should not be merged into choreography context.

An explicit `:open-payload? true` may exist for cases that genuinely require an open payload, but closed contracts are the default.

Within the admitted choreography semantics, a sender must have justified knowledge of every semantic fact it transmits. Receipt from an adversarial participant still requires the trusted-boundary treatment defined in the knowledge model.

Receiving a message should only introduce the declared communicated values into the receiving context.

Projection must not erase communication that is genuinely required to justify knowledge.

Conversely, projection should not manufacture communication merely because the global choreography contains a choice when the target role's behavior does not depend on it or its knowledge is established through another valid path.

Projection is a semantic compilation boundary

Projection is not a convenience transformation that happens to produce smaller maps.

It is the boundary at which the global specification is turned into the local information and actions available to a role.

A correct projector must therefore preserve every fact required to justify local behavior, including when relevant:

- message schemas;
- correlation identity;
- entry knowledge;
- knowledge-producing communication;
- authoritative-observation requirements;
- branch discriminators;
- provenance requirements;
- resource ownership;
- failure/terminal structure;
- executable guards required by local semantics.

At the same time, it should omit global facts that the role neither knows nor needs.

The target is neither ”copy the whole global machine to everyone” nor ”erase everything inconvenient.” It is a justified local program.

Projection cannot create knowledge

A projector may rearrange or encode information justified by the global choreography. It may not manufacture a fact merely because the local machine would be easier to implement if the fact were present.

Bad:

```
global:
    only authority knows :approved?

project browser:
    include :approved? in browser context anyway
```

Good:

```
global:
    authority communicates approved outcome
or
    browser performs authorized authoritative observation

project browser:
    local branch becomes possible after that knowledge path
```

The same principle applies to provenance. Projected data cannot silently upgrade:

```
asserted -> derived
communicated -> authoritative
client supplied -> trusted principal identity
```

Projection preserves or weakens claims according to explicit semantics; it does not strengthen them by serialization.

Behavior-sensitive choice is the central branch rule

A role needs to know a choice only when its observable local behavior depends on that choice. The required knowledge must exist before the first differing behavior, not merely become available eventually.

This avoids both extremes:

- projecting too little and leaving a role unable to decide its next behavior;
- overcommunicating every global discriminator to every mentioned role.

The analysis should compare continuations semantically rather than merely by source syntax.

If two global branches project to equivalent local behavior for role R, R need not learn the branch solely because R appears somewhere inside both branches.

If they diverge, the projector/verifier must establish a valid knowledge path before the divergence.

A richer expense example shows why ”behavior-sensitive” is the important word.

Suppose the global choreography has three relevant continuations after the manager observes the Expense and current policy:

```
(defchoreo expense-decision
  {:roles
   {:manager {:principal :user}
    :finance {:principal :user}
    :notifier {:principal [:service :notifications]}
    :expense-authority {:authority :expense}}
   :inputs
   {:expense-id
    {:schema :uuid
     :known-by #{:manager}
     :provenance :input}}}

  (manager
    (observe expense-authority
      [:expense/current expense-id]
      → expense)

    (observe expense-authority
      [:expense/current-policy
       (:expense/department expense)]
      → policy)

    (branch
      (needs-finance?
        {:expense expense
         :policy policy})

      (then
        (invoke expense-authority
          :expense/mark-awaiting-finance
          {:expense-id expense-id})

        (send finance
          [:expense/finance-review-required
           {:expense-id expense-id}])

        (send notifier
          [:expense/review-recorded
           {:expense-id expense-id}]))

      (else
        (invoke expense-authority
          :expense/approve-as-manager
          {:expense-id expense-id})

        (send notifier
          [:expense/review-recorded
           {:expense-id expense-id}])))))
```

The notifier performs the same semantic action in both branches with the same payload. It does not need the manager's private threshold or the boolean `needs-finance?` merely because the global source contains a branch around the notification. The common message means that a manager review was recorded; it does not assert the Expense is still `:submitted` after one branch changed it to `:awaiting-finance` and the other to `:approved`. If a notification must report current status, it needs an authorized authoritative read or a branch-dependent communicated outcome, with the corresponding knowledge path.

A projected notifier machine can therefore be independent of that discriminator:

```
(defprojected expense-decision-notifier
  {:role :notifier}

  (state :waiting
    (on-message
      [:expense/review-recorded
       {:keys [expense-id]}]

      (notify-user!
        {:expense-id expense-id
         :event :review-recorded})

      (terminate :sent))))
```

Finance is different. Its behavior exists only in one continuation, so it needs a knowledge-producing path before it can act. The selected branch communicates exactly such a path:

```
(defprojected expense-decision-finance
  {:role :finance}

  (state :waiting
    (on-message
      [:expense/finance-review-required
       {:keys [expense-id]}]

      (goto :load-authority
        {:expense-id expense-id})))

  (state :load-authority
    (observe
      [:expense/current :expense-id]
      → :expense)

    (goto :review))

  (state :review
    (invoke
      :expense/finance-approve
      {:expense-id :expense-id})

    (terminate :approved)))
```

Now consider a tempting but invalid projection:

```
(defprojected invalid-finance-projection
  {:role :finance}

  (state :waiting
    ;; No message was received.
    ;; No authoritative observation established this discriminator.
    ;; It simply appeared because the projector knew the global branch.
    (if :needs-finance?
      (goto :review)
      (terminate :not-needed)))

  (state :review
    (invoke
      :expense/finance-approve
      {:expense-id :expense-id})))
```

The global verifier knows `:needs-finance?`. Finance does not. The projector cannot smuggle the fact into Finance merely because it would make the generated machine easier.

An alternative global design could legitimately omit the direct manager-to- Finance message if Finance can learn the required condition through authority. For example, the manager could commit durable `:awaiting-finance` state and Finance could later enter a new short choreography through an authorized queue observation:

```
(defchoreo finance-review-from-authority
  {:roles
   {:finance {:principal :user}
    :expense-authority {:authority :expense}}}

  (finance
    (observe expense-authority
      [:expense/pending-finance-queue
       {:principal :authenticated-principal}]
      → pending)

    (for-each [expense pending]
      (invoke expense-authority
        :expense/finance-approve
        {:expense-id (:expense/id expense)}))))
```

That version has a different knowledge path, not weaker semantics. Finance knows the relevant fact because the authoritative queue observation establishes it.

The two legal designs expose a general principle:

```
global branch fact
    |
    +--> communicated to the role that needs it
    |
    `--> established by an equivalent authorized authoritative observation

but never:

global branch fact
    |
    `--> silently copied into a local projection
```

Behavior-sensitive projection therefore controls both **how much** knowledge is distributed and **when** it must exist. A role whose local behavior is equivalent across branches does not need the discriminator. A role whose behavior diverges needs an admissible knowledge path before the first divergent action.

1. Supported equivalence and conservative rejection

   Behavior-sensitive checking must define the local equivalence relation used by its supported language subset. The observables

include whichever messages, effects, authoritative invocations, resource actions, and terminal or failure outcomes the semantic contract distinguishes. Relevant ordering and timing constraints must not disappear merely because the final values happen to match.

Permitted internal steps may be abstracted only under an explicit semantic relation. The checker is not required to decide equivalence of arbitrary Clojure functions or unrestricted programs.

When the supported analysis cannot establish equivalence or a sufficient knowledge path, it must reject or report an unresolved obligation at the appropriate admission boundary. It must not silently assume equivalence, disclose a private discriminator, or introduce an undeclared message to make projection succeed. An explicit supported communication path or visible trusted assumption may be authored where permitted by the relevant contract.

2. Trace preservation and progress are separate obligations

   Excluding forbidden local traces is a safety claim. It does not by itself prove that a projected program can perform every required interaction, avoid deadlock, or eventually complete. A projection that admits no useful behavior must not satisfy the required operation merely because its empty behavior is safe.

   Each claimed projection result must state the global/local behavior relation and the entry and environmental premises under which it applies. Required realizability, terminal behavior, and progress obligations must be discharged with the appropriate evidence. Liveness remains conditional on its named delivery, availability, fairness, or application-action assumptions.

   These obligations supplement, rather than replace, the independent application capability acceptance defined in Part I.

Message payload closure protects knowledge boundaries

Closed messages are not only a schema preference. They are a knowledge-flow boundary.

If a message declares:

```
{:request-id UUID
 :outcome ClaimOutcome}
```

Here `:request-id` identifies the domain Request involved in the Claim outcome; it is not `:transport-request-id`, which identifies a transport attempt. Receiving this message may establish only the declared communicated facts.

The receiver does not acquire arbitrary sender context, and the implementation must not merge unknown payload keys into machine context by convenience.

An explicitly open payload must follow the extension-data contract from the value-schema boundary. Openness alone does not grant undeclared fields semantic knowledge status or permission to influence a branch. Any weaker guarantee actually selected remains visible in the result.

Projection preserves necessary communication but does not worship messages

Communication is one way to establish knowledge, not the definition of knowledge.

A projector must preserve a message when that message is what justifies another role's behavior.

It must not insert a redundant direct message merely because one actor's choice eventually affects another actor if the other actor can instead obtain the necessary fact by:

- justified entry knowledge;
- authoritative observation;
- deterministic derivation;
- another declared knowledge path.

This matters deeply for Gesso's web architecture because durable business facts often rendezvous naturally through model authority and reread rather than long-lived direct participant messaging.

HumanHelp Request publication and a collaborative typing indicator provide a useful contrast because they look superficially similar in a browser while having different semantic recovery requirements.

For a newly created HumanHelp Request, the durable fact already belongs to Request authority:

```
(defoperation create-request
  {:operation/id :request/create
   :actor :customer

   :effect
   [:model/invoke :request/create
    {:customer :authenticated-principal
     :request :request/draft}]

   :settlement
   {:accepted
    {:creator-reacquire
     [:request/card :created-request-id]

     :remote-advisory
     {:scope
      [:request/board :organization-id]

      :requirement
      :newer-relevant-authority-available}}}})
```

After the authoritative commit, other clients need not receive the entire Request directly from the creator's execution. A shared advisory can merely record that newer relevant authority may be available:

```
(defn handle-board-advisory
  [trusted-context board-runtime event]
  (let [admitted
        (advisory-admission/admit!
          {:trusted-context trusted-context
           :runtime board-runtime
           :event event
           ;; Fixed by the admitted shared-event contract; event data
           ;; cannot weaken these checks.
           :require #{:current-subscription-owner
                      :current-source-instance
                      :admitted-source-provenance
                      :valid-scoped-requirement
                      :permitted-shared-transition}})]
    (advisory-admission/require-admitted! admitted)
    (update
      board-runtime
      :advisory-requirements
      conj
      (:requirement admitted))))
```

The transition above runs only after admission succeeds; a failed admission rejects the event without changing state. Matching a subscription id alone does not establish source provenance, valid scope, or permission to record the requirement.

Bob may continue displaying his adopted board until the product's explicit Refresh/adoption policy permits reacquisition:

```
(defn refresh-request-board!
  [{:keys [principal] :as ctx}
   board-state]

  (let [started @board-state
        scope (:projection started)
        acquisition
        (request-model/read-board
          ctx
          {:principal principal
           :organization-id
           (:organization-id scope)})]

    ;; Install against current state. A shared advisory can arrive while
    ;; read-board is in flight; the installer must preserve it unless this
    ;; acquisition proves that it covers the advisory's scope and minimum.
    (swap! board-state
      (fn [current]
        (board-admission/install-acquisition
          current
          {:acquisition acquisition
           :expected-scope scope
           :principal principal})))
    @board-state))
```

`board-admission/install-acquisition` is a pure transition because `swap!` may retry it. It checks current acquisition owner, scope, trusted provenance, authorization, and progression against the latest runtime state. A stale completion cannot overwrite a newer board, and a concurrent advisory cannot be silently erased by refreshing from an earlier snapshot.

The advisory does not need to carry the new Request title, requester, lifecycle state, or every intermediate board mutation. The durable authoritative model is the rendezvous point.

Now compare a collaborative editor's typing indicator:

```
(defmessage typing-indicator
  {:message/id :document/typing

   :from :editor
   :to :peer-editor

   ;; The adversarial wire payload describes ephemeral intent only.  It does
   ;; not get to assert who the sender is.
   :payload
   [:map {:closed true}
    [:document/id uuid?]
    [:typing? boolean?]]

   ;; Delivery of this message establishes sender knowledge only after the
   ;; trusted communication boundary authenticates the connection and checks
   ;; that this principal may participate in this document.
   :sender-binding
   {:principal-source :authenticated-channel
    :actor-source     :admitted-channel-actor
    :authorize        :document/current-editor
    :bind-principal-as :principal/id
    :bind-actor-as     :actor/id}

   :delivery
   {:durability :ephemeral
    :ordering :best-effort
    :replay :none}
```

```
   :semantics
   {:expires-after-ms 3000}})

(defn admit-typing-message
  [{:keys [authenticated-principal
           authenticated-actor
           current-document-membership
           decoded-message]}]
  (let [{:keys [document/id typing?]} decoded-message]
    (m/assert TypingPayload decoded-message)
    (when-not (= id (:document/id current-document-membership))
      (throw (ex-info "typing message is for an unauthorized document"
                      {:document/id id})))
    (when-not (= authenticated-principal
                 (:principal/id current-document-membership))
      (throw (ex-info "typing sender is not the authenticated editor"
                      {:document/id id})))
    (when-not
      (and (= :active (:membership/status current-document-membership))
           (some? authenticated-actor))
      (throw (ex-info "typing sender has no active admitted actor"
                      {:document/id id})))

    ;; This admitted value is produced by the trusted boundary.  The actor
    ;; identity comes from the channel's admitted actor binding, distinct from
    ;; its principal. The receiver's document subscription must also admit
    ;; this scope before delivery to a peer machine.
    {:message/id :document/typing
     :document/id id
     :typing? typing?
     :actor/id authenticated-actor
     :principal/id authenticated-principal
     :provenance
     {:actor/id :admitted-channel-actor
      :principal/id :authenticated-channel
      :document/id :validated-payload
      :membership :current-authority}}))
```

`TypingPayload` denotes the declared closed message schema above; a concrete elaborator may generate its validator. The membership check must be backed by current trusted authority, not a field decoded from the client. The admitted message is routed only to peer subscriptions currently permitted to receive typing indicators for this document.

A projected peer machine may legitimately react to the received message itself:

```
(defprojected peer-typing-projection
  {:role :peer-editor}

  (state :idle
    ;; on-admitted-message means the trusted communication boundary above has
    ;; already established sender identity, authorization, payload shape, and
    ;; the provenance that justifies local knowledge of :actor/id.
    (on-admitted-message
      [:document/typing
       {:keys [document/id actor/id typing?]}]

      (if typing?
        (then
          (effect
            [:ui/show-typing-indicator
             {:document/id id
```

```
             :actor/id actor/id}])

        (acquire-resource
          {:kind :timer
           :purpose :typing-expiry
           :after-ms 3000}
          → expiry-timer)

        (goto :showing
          {:document/id id
           :actor/id actor/id
           :timer expiry-timer}))

      (terminate :not-typing))))

(state :showing
  ;; The admitted router selects this indicator by document and actor.
  (on-admitted-message
    [:document/typing
     {:keys [document/id actor/id typing?]}]

    (if typing?
      (then
        (release-resource :timer)
        (acquire-resource
          {:kind :timer
           :purpose :typing-expiry
           :after-ms 3000}
          → replacement-timer)
        (goto :showing
          {:document/id id
           :actor/id actor/id
           :timer replacement-timer}))

      (then
        (release-resource :timer)
        (effect
          [:ui/hide-typing-indicator
           {:document/id id
            :actor/id actor/id}])
        (terminate :stopped))))

  (on-resource
    :timer-expired
    {:resource :timer
     :instance :current-timer-instance}

    (effect
      [:ui/hide-typing-indicator
       {:document/id :document/id
        :actor/id :actor/id}])

    (terminate :expired))))
```

The peer machine is keyed by document and admitted actor. A `typing? false` event closes only that actor's current indicator. A later `typing? true` event replaces the expiry timer; a timeout from the retired timer cannot hide the replacement because resource-instance ownership is checked before the timeout transition. Ephemeral messages need no durable replay, but their local timers still need that owner check.

If Alice never receives Bob's typing message, an authoritative reread two minutes later cannot reconstruct the semantic event in

any useful sense. There may be no durable `:typing=true` fact to query, and replaying it after expiry would be wrong.

The two cases therefore require different projection behavior:

```
{:request/create
 {:durable-truth :request-authority
  :remote-coordination :advisory
  :recovery :authoritative-reread}}

{:document/typing
 {:durable-truth nil
  :remote-coordination :direct-ephemeral-message
  :recovery :none
  :expiry :local-resource}}
```

A projector that inserts direct participant messages for every durable model change over-specifies delivery and recovery. A projector that replaces every message with ”just reread later” destroys genuinely ephemeral semantics.

Communication is therefore neither sacred nor suspicious. It is one declared knowledge path among several, and the correct projection preserves whichever path the application's semantics actually require.

Local executable guards belong in the executable projection

Proof derivations and global proof trees do not belong in the production browser.

However, a local predicate or guard whose result is required to execute the projected semantics is runtime-semantic data and must remain in the executable projection.

The distinction is:

```
"this callback may resume execution E only while its declared resource
 instance I retains the required ownership for E"
    -> executable admission guard if needed at runtime

"here is the proof tree explaining why that guard is sufficient"
    -> diagnostic/proof artifact
```

The guard identifies the execution and resource instance required by this callback's admission contract; it is not a universal generation comparison. Shared fragment or subscription events use their own declared ownership checks. Passing an ownership guard does not by itself establish the knowledge, provenance, or authority required for the resumed branch.

The executable semantic core evaluates runtime guards as part of the admitted transition semantics. A proof kernel or dedicated semantic checker establishes its stated judgment about those semantics; it does not replace the runtime admission check or supply missing runtime facts.

The production machine must never depend on an optional proof sidecar to decide a transition.

Projection failure should identify the missing justification

When projection cannot produce a legal local machine, diagnostics should explain the actual semantic deficit.

Useful failure information includes:

- role;
- global state/transition;
- local behaviors that diverge;
- required discriminator;
- facts currently known;
- available provenance;
- missing knowledge path;
- candidate communication or authoritative-observation paths when derivable.

A generic ”projection failed” error throws away precisely the information that the verifier already computed.

Host-language callbacks may not bypass projected semantics

A local runtime may invoke host-language functions to realize declared local effects, but arbitrary callbacks must not become an alternate transition path.

In particular:

```
projected machine
    -> declared local effect
    -> host implementation
    -> normalized completion/failure
    -> projected machine
```

is compatible with the design.

But:

```
host callback
    -> mutate choreography state directly
```

reopens the proof and knowledge boundary.

The common boundary is owner-relative event admission in Part II. Part VI develops its browser implementation. An execution-private effect completion must not bypass that boundary, and a declared shared event must use its actual fragment or subscription owner rather than an invented execution.

Long logical workflows and short runtime executions

A choreography may describe a logical workflow that spans human-scale time.

That does not imply that one Aleph node should retain a suspended server-side machine in process memory for hours or days.

For example, a business flow may conceptually involve:

```
request approval
        |
wait
        |
approval granted
        |
continue
```

A robust HumanHelp realization may instead be:

```
short choreography E1
    |
commit authoritative :approval/pending
    |
E1 terminates

...minutes or hours...

approver observes pending authority
    |
short choreography E2
    |
commit authoritative :approval/approved
    |
E2 terminates

requestor later observes approved authority
    |
short choreography E3 if further action is needed
```

The durable wait lives in model state.

Subject to the authoritative store's declared durability and recovery assumptions, the durable workflow fact can survive:

- web-node failure;
- deployment;
- load-balancer rerouting;
- client disconnect;
- browser suspension;
- long offline periods.

The initial Gesso choreography runtime therefore does not promise durable persistence of arbitrary suspended server-local executions.

If a future application genuinely needs durable workflow-machine continuations, that should be an explicit opt-in capability with its own persistence, ownership, versioning, and recovery semantics.

It must not silently become a requirement of ordinary choreography.

Durable waiting belongs in authoritative state by default

Human-scale waiting is normally represented as durable domain state rather than a suspended in-memory choreography execution.

This has several consequences.

First, subject to those persistence and compatibility assumptions, the durable workflow meaning can survive:

- deployment;
- process loss;
- browser suspension;
- load-balancer rerouting;
- reconnect;
- long periods without any participant online.

Second, later actors can acquire the relevant facts by authoritative observation rather than requiring the original process to resume.

Third, the framework avoids making one runtime's memory an accidental source of business truth.

A later choreography is not automatically the same execution

When durable state connects two periods of activity, the later protocol may be a new execution with a new execution identity.

For example:

```
E1:
    submit approval request
    -> command C1
    -> commit :pending
    -> terminate

E2:
    approver later acts
    -> command C2
    -> commit :approved
    -> terminate
```

The business process is continuous in domain meaning, but E1 and E2 need not be one persistent protocol execution. E2 must establish its own authenticated principal, current model eligibility, required knowledge, and applicable plan. The existence of E1's durable result does not carry E1's transient authority or resource ownership into E2.

This distinction avoids conflating:

- semantic command identity;
- protocol execution identity;
- durable domain identity;
- workflow/business identity.

Where an application truly requires a durable suspended execution, that feature must define its own:

- persistence;
- ownership;
- versioning;
- migration;
- replay;
- resource cleanup;
- liveness;
- recovery semantics.

It is not silently inherited from ordinary Choreo.

The expense example can be extended across several days without adding a suspended process.

On Monday, the employee submits Expense E. A short choreography validates the submission and commits durable pending state:

```
(defchoreo submit-expense
  {:roles
   {:employee {:principal :user}
    :expense-authority {:authority :expense}}
   :inputs
   {:expense-id
    {:schema :uuid
     :known-by #{:employee}
     :provenance :input}}}

  (employee
    (invoke expense-authority
      :expense/submit
      {:expense-id expense-id
       :employee :authenticated-principal
       :command-id :gesso/command-id}
      → submit-outcome)

    ;; A later current-state read might already show a further transition.
    ;; This branch is justified by the scoped outcome of this command.
    (require-outcome
      submit-outcome
      :committed-or-compatible-already-incorporated
      :otherwise [:rejected :unknown])

    (terminate :submitted)))
```

The public model operation owns the durable transition:

```clojure
(defn submit-expense!
  [{:keys [db principal now]}
   {:keys [expense-id command-id]}]

  (expense-model/transact-submission!
    db
    {:expense-id expense-id
     :employee-id (:user/id principal)
     :command-id command-id
     :submitted-at now

     ;; The model enforces these against transaction-current facts, in the
     ;; same atomic decision that writes :submitted and the command outcome.
     :commit-guards
     [[:expense/owned-by expense-id (:user/id principal)]
      [:expense/status-is expense-id :draft]]

     :retry-binding
     {:operation :expense/submit
      :principal (:user/id principal)
      :expense-id expense-id}}))
```

The transaction form is schematic. The selected store mechanism must make the ownership and status guards effective at commit, reject concurrent conflicting changes, and bind a recorded result to the command identity atomically with the transition. On a compatible retry the model checks disclosure rights and returns the already-incorporated result without requiring the Expense to remain `:draft`. A rejected guard and an unknown commit follow different settlement paths; neither authorizes the employee's confirmed-submission continuation. The Thursday and Friday operations likewise guard current status, actor authorization, and any policy premise they rely on at their own commits.

Execution E1 can now terminate. There is no correctness benefit in keeping the employee's server process resident until a manager responds.

On Thursday, a manager opens an approval queue and authoritatively observes the pending Expense:

```
(defchoreo review-submitted-expense
  {:roles
   {:manager {:principal :user}
    :expense-authority {:authority :expense}}}

  (manager
    (observe expense-authority
      [:expense/pending-for-manager
       {:principal :authenticated-principal}]
      → pending-expenses)

    (choose expense-id
      :from pending-expenses)

    (observe expense-authority
      [:expense/current expense-id]
      → expense)

    (observe expense-authority
      [:expense/current-policy
       (:expense/department expense)]
      → policy)

    (branch
      (needs-finance?
        {:expense expense
         :policy policy})

      (then
        (invoke expense-authority
          :expense/mark-awaiting-finance
          {:expense-id expense-id})

        (terminate :awaiting-finance))

      (else
        (invoke expense-authority
          :expense/approve-as-manager
          {:expense-id expense-id})

        (terminate :approved)))))
```

This is a new execution E2 with its own current principal, observations, execution identity, resource ownership, and compatible plan.

Friday's Finance review can be yet another execution:

```
(defchoreo review-finance-expense
  {:roles
   {:finance {:principal :user}
    :expense-authority {:authority :expense}}}

  (finance
    (observe expense-authority
      [:expense/pending-for-finance
       {:principal :authenticated-principal}]
      → queue)

    (choose expense-id
      :from queue)

    (observe expense-authority
      [:expense/current expense-id]
      → expense)

    (invoke expense-authority
      :expense/approve-as-finance
      {:expense-id expense-id})

    (terminate :approved)))
```

The domain history links the executions:

```
{:expense/id E
 :expense/status-history
 [{:status :draft
   :effective-at monday-08-00}

  {:status :submitted
   :effective-at monday-09-12
   :caused-by {:command C1
               :operation :expense/submit}}

  {:status :awaiting-finance
   :effective-at thursday-14-20
   :caused-by {:command C2
               :operation :expense/mark-awaiting-finance}}

  {:status :approved
   :effective-at friday-10-03
   :caused-by {:command C3
               :operation :expense/approve-as-finance}}]}
```

The runtime identities need not form one persistent chain:

```
{:monday
 {:execution E1
  :command C1
  :principal employee-P
  :plan :plan/submit-expense-A12}}

{:thursday
 {:execution E2
  :command C2
  :principal manager-M
  :plan :plan/review-expense-A13}}

{:friday
 {:execution E3
  :command C3
  :principal finance-F
  :plan :plan/finance-review-A13}}
```

The business process is continuous because authoritative Expense state is continuous under the model's semantics. The protocol executions are separate because each period of activity has its own current participants, authorization, knowledge, resources, and executable compatibility.

A future durable-workflow facility could choose to preserve suspended execution state across days, but that would be an additional semantic facility with explicit persistence and migration rules. Ordinary Choreo does not need to pretend that Monday's process remained asleep somewhere in order for Friday's approval to be part of the same business story.

Durable model state is the normal rendezvous point

For long workflows, authoritative state provides a stable rendezvous point between actors who were never concurrently connected.

A later actor may know what happened because authority establishes:

```
:approval/pending
:approval/approved
:request/claimed
:request/completed
```

not because it receives a packet from the actor who caused that state.

This follows Part II's authoritative observation model: observation is a first-class knowledge path.

Durable pending state is not itself a progress guarantee. The application must provide the declared opportunity for an eligible actor to discover and act on it, or state the environmental premises on which progress depends. Where loss of a required participant can strand the workflow, the runtime-condition and recovery requirements from Part I apply. Any lease, reassignment, cancellation, or other successor policy remains model-owned.

Deployment compatibility remains explicit

A durable business fact may outlive the executable plan that originally produced it.

That is normal.

What cannot be assumed is that a suspended **runtime execution** from an older deployment remains compatible forever.

The later stale-plan/recovery design therefore treats durable model truth and runtime-plan compatibility as separate dimensions.

A deployment may invalidate continuation of an old projection while leaving the authoritative business state entirely valid. If a deployment also changes the meaning or representation of persisted domain state, the model needs its own compatible interpretation or migration. Restarting a current browser plan does not perform that domain migration by implication.

Part IV — Authoritative operations and progression

This part defines the public model boundary, FX realization, atomic transitions, scoped authoritative acquisition, and semantic command identity. It separates authoritative reality from propagation status and participant knowledge. The concrete browser coordinator follows in Part VI; whole-application capability and acquisition closure follow in Part VII.

Model integration

HumanHelp models remain the source of domain truth.

For a model such as Request:

```
model/request/
  domain.clj
  schema.clj
  graph.clj
  fx.clj
  core.clj
  choreo.clj
```

The responsibilities are:

- `domain.clj` – Request semantic rules and pure transitions;
- `schema.clj` – Request persistence/value schemas;
- `graph.clj` – Request graph declarations;
- `fx.clj` – authoritative effect planning and atomic persistence realization;
- `core.clj` – the narrow public semantic interface of the Request model;
- `choreo.clj` – higher-level distributed programs built exclusively from public model interfaces.

The dependency direction is an architectural commitment:

```
domain / schema / graph / fx
             |
             v
          core
             ^
             |
          choreo
```

`request.choreo` sits **above** `request.core`. It is not an implementation detail behind `request.core` and `request.core` does not facade it.

A model choreography is therefore a higher-level distributed program written against public model semantics.

`request.choreo` must not bypass the public model boundary and reach into Request domain, graph, schema, or FX internals.

For example:

```
(ns net.humanhelp.site.model.request.choreo
  (:require
   [gesso.choreo.core :as choreo]
   [net.humanhelp.site.model.request.core :as request]))
```

and an authoritative choreography step calls something conceptually like:

```
(choreo/at :request-authority
  (request/claim ...))
```

not:

```
(request.fx/claim ...)
```

Cross-model dependencies follow the same rule. If a Request choreography needs Membership or Organization semantics, it may require their public `core` namespaces, never their internals:

```
request.choreo
   |
   +--> request.core
   +--> membership.core
   +--> organization.core
```

The choreography may know facts that the semantic model API does not need to know, including:

- roles;
- runtime actors;
- principals;
- communication;
- distributed knowledge;
- provenance;
- agreement;
- command identity;
- observed authoritative basis;
- provisional execution;
- settlement;
- timeout/retry behavior;
- convergence;
- projection compatibility/version.

Those concerns must not flow downward into the model merely because the model has a choreography namespace. Some information, such as a semantic command identity or an expected domain version, may independently belong in the public model contract when durable deduplication or domain preconditions require it. That does not make browser execution state or settlement transport a model concern. The placement test below determines the owner of the meaning.

A useful placement test is:

> Would this rule still matter if the public model operation were called synchronously from one JVM with no browser, network, projection, or second actor?

If yes, it normally belongs in the model semantics below `core`.

If no because it concerns communication, distributed knowledge, optimism, settlement, retry, projection compatibility, or convergence, it is a candidate for `choreo.clj`.

A choreography involving one model's semantic operation but multiple runtime actors may live in that model's `choreo.clj`.

A choreography whose meaning genuinely spans several peer models should live in a higher-level flow/choreography namespace rather than turning one model into an aggregation point.

For example, a future onboarding workflow that coordinates Invitation, Membership, Organization, and external provisioning should not be forced into `invitation.choreo` merely because Invitation happens to be the first step.

Model namespaces export semantic interfaces, not implementation privilege

The public `core` namespace is the semantic API boundary of a model.

A higher layer may rely on what that interface declares, but not on arbitrary knowledge of the implementation beneath it.

For a Request model, a public operation contract may expose:

- semantic operation identity;
- typed inputs;
- typed outcomes;
- preconditions represented as public semantic commitments;
- authority requirements;
- atomicity guarantees;
- model-defined failure classes;
- stable facts/projections that higher layers may consume.

It should not require Choreo to know:

- XTDB transaction forms;
- internal graph query structure;
- implementation-only helper vars;
- private schema storage details;
- private FX staging data.

This is the boundary that allows the authoritative model to remain usable without Choreo and allows Choreo to coordinate the model without becoming its implementation.

Within one trusted Clojure process, a sufficiently determined application author may still import lower-level namespaces, obtain a transaction handle, or write to the database outside these interfaces. Gesso does not claim to sandbox such trusted host code. The design goal inside that boundary is instead that normal cooperating application code receives narrow semantic interfaces, accidental bypass is difficult, and deliberate departure from the supported path is conspicuous and outside the assurance claim that depended on the boundary.

Where practical, ordinary web/view/choreography code should not need a raw authoritative mutation capability merely to perform a declared model operation. This reduces accidental invariant bypass without pretending that Clojure records, namespace privacy, or nominal wrappers constitute a security boundary against the application author.

Semantic dependency is not necessarily namespace load order

The conceptual dependency:

```
schema/domain/relationships/rules
        |
        v
FX / model operation contract
        |
        v
core
        |
        v
choreo
```

describes which semantic commitments may justify later reasoning.

It does not require the final source tree to mirror this diagram mechanically, nor does it mean every namespace participates in the later proof system.

What matters is that a dependent semantic layer cannot claim a guarantee that its dependencies neither establish nor explicitly assume.

Business rules remain model semantics

A rule belongs below `core` when its meaning would still matter in a synchronous single-JVM call with no browser or distributed protocol.

Examples include:

- whether a Request may be claimed;
- whether Membership is active;
- whether an Organization permits an operation;
- what state transition follows from a valid command;
- what facts must remain true at authoritative commit.

Choreo may rely on the public result of those rules, but must not duplicate them merely because a browser operation depends on them.

This prevents two dangerous architectures:

```
model says one thing
view/choreo reimplements "close enough" version
```

and:

```
core is only a facade
real business semantics secretly live in choreo
```

Neither is compatible with the intended end state.

Well-formed business rules and correct business policy are different

The model layer should eventually make malformed rule expressions rejectable where their semantic types are known.

For example, if `request.status` is a closed enum, an ordered numeric comparison against it is malformed and should not become a valid certified rule.

This stronger rule-language machinery is specified later in the compositional proof part.

The architectural boundary established here is simpler:

- domain/model code owns business meaning;
- the semantic interface exposes only checked/declared commitments;
- Choreo consumes those commitments;
- Choreo does not decide whether product management chose the right policy.

Cross-model semantics do not imply cross-model leakage

A Request operation may depend on public facts from Membership and Organization.

That does not authorize `request.choreo` or `request.core` to reach through those models into their internal FX/schema/graph implementation.

The dependency remains through public semantic interfaces.

Where several model facts must participate in one atomic transition, the authoritative operation may compose the necessary model-level transaction work behind the public operation contract. API boundaries do not require separate transactions.

The atomic authoritative transition contract below specifies this distinction in detail.

Higher-level cross-model choreography should be named at the right level

A distributed program whose meaning genuinely spans several peer models should live in a higher-level flow/choreography namespace.

For example:

```
onboarding.choreo
    |
    +--> invitation.core
    +--> membership.core
    +--> organization.core
    +--> provisioning.core
```

is preferable to making `invitation.choreo` own the entire workflow merely because Invitation happens to be the first model touched.

The namespace location should reflect semantic ownership, not incidental chronology.

HumanHelp is a reference consumer, not a source of generic Gesso semantics

HumanHelp may reveal missing generic abstractions.

When it does, the design test is:

> Is this missing concept intrinsic to distributed semantic correctness for Gesso applications generally, or is it HumanHelp business policy?

Generic examples include:

- operation capability;
- knowledge/provenance requirement;
- transport requirement;
- server capability;
- acquisition requirement;
- progression contract;
- application-closure edge.

HumanHelp-specific examples include:

- who may claim a particular Request;
- Organization policy;
- Membership semantics;
- Request lifecycle business rules.

The former may justify additions to Gesso. The latter remain in HumanHelp models.

Relationship between core, FX, and choreography

FX and Choreo do not merge into one execution mechanism.

They operate at different correctness scopes.

FX is responsible for realizing an authoritative model transition, usually within one atomic transaction boundary.

Choreo is responsible for distributed coordination around use of that public authoritative operation.

Conceptually:

```
request.choreo/claim
        |
        | command, knowledge, observed basis, provisional state
        v
   authoritative step
        |
        v
request.core/claim
        |
        v
request.fx
        |
        | atomic XTDB transaction
        v
 authoritative result
        |
        v
request.choreo continues or terminates
        |
        | settlement / Live propagation / later authoritative observation
        v
actors learn result
```

There is one semantic operation identity, for example `:request/claim`, but the public semantic operation and its higher-level distributed realization are not the same function and need not have the same type.

Roughly:

```
authoritative semantic operation

ClaimCommand x AuthoritativeWorld
        ->
ClaimResult


distributed choreographic operation

HelperKnowledge
        ->
Confirmed
| Reconciled
| Rejected
| LocallyRetiredWithOutcomeUnknown
| AnotherDeclaredProtocolOutcome
```

These alternatives are illustrative, not a required public enum. An execution failure and an authoritative rejection are different outcomes; the former may leave the command's disposition unknown to the participant.

The choreography should not invent a second, competing meaning for `:request/claim`. Its authoritative step delegates to `request.core/claim`.

Likewise, `request.core` must not require or facade `request.choreo` merely to present a single namespace to callers. The layering is intentionally public:

request.core: public Request semantics

request.choreo: public distributed programs built from Request semantics

Views and higher-level application code may consume prepared or bound choreographic capabilities without weakening this dependency direction.

Choreo may rely on an authoritative operation's declared contract – input and result schemas, authority, atomicity, or other public metadata – but it must not understand or reproduce the model's XTDB writes.

This preserves an important independence property: `request.core/claim` remains meaningful and testable from a REPL, batch job, test, server endpoint, or other non-choreographic caller.

FX realizes model semantics; it does not define distributed truth

FX may perform multi-step computation, query authoritative state, prepare transaction data, validate commit-time conditions, and publish an authoritative result.

Those steps belong to realization of a model operation.

Choreo observes the public semantic boundary:

```
authoritative operation invoked
        |
        v
declared semantic outcome
```

rather than treating every internal FX step as a distributed choreography state.

This preserves a small and meaningful distributed protocol while allowing the authoritative implementation to remain sophisticated.

One operation identity may span several implementation layers

The same semantic operation identity may be referenced by:

- model `core`;
- Choreo;
- UI operation capability;
- server operation registry;
- transport binding;
- diagnostics.

That does not mean all those layers are the same object.

For example:

```
:request/claim
    |
    +--> request.core/claim
    |
    +--> request.choreo/claim
    |
    +--> bound UI capability
    |
    +--> trusted server realization
```

Each layer adds a different kind of structure around one semantic operation.

Later whole-application preflight must verify that independently realized edges actually correspond. It must not assume correspondence merely because names look similar.

No semantic backchannel from Choreo into model internals

The following dependency is forbidden:

```
request.choreo
    -> request.fx/private-prepare
    -> request.graph/private-query
```

even if it appears convenient.

Such a path lets distributed coordination depend on implementation facts that the public model interface does not promise.

It also makes later proof/certification misleading because the apparent public operation contract no longer captures the real dependencies.

The authoritative operation may internally use those facilities. Choreo may not.

Core remains meaningful without Choreo

A public model operation must remain useful in contexts such as:

- REPL invocation;
- tests;
- batch jobs;
- administrative tools;
- server APIs that do not need a distributed choreography;
- future application flows using a different higher-level protocol.

This is a deliberate independence property, not merely a convenience for testing.

If `request.core/claim` only makes sense when called through `request.choreo/claim`, business semantics have leaked upward.

Independent use does not mean bypassing required authority. A REPL, batch job, or administrative caller must supply the trusted premises required by the same public model contract. Removing the browser protocol must not remove current authorization, transaction guards, or invariant enforcement.

Choreo adds distributed obligations that core should not own

Conversely, `request.core` should not need to understand:

- browser execution-instance identity and resource ownership;
- communication topology;
- projected branch knowledge;
- provisional DOM state;
- settlement transport;
- reconnect;
- continuity;
- stale projected-plan recovery.

Those are higher-level distributed/runtime concerns.

The two layers meet at an explicit authoritative operation contract.

Public operation metadata must be sufficient but minimal

Choreo may need public metadata describing an authoritative operation, such as:

- semantic identity;
- typed command/input;
- typed outcome/result;
- required authority;
- atomicity class;
- declared externally visible effects;
- model-defined reconciliation outcomes;
- stable facts that may be learned from the result.

This metadata should be rich enough for distributed reasoning without exposing the operation's persistence implementation.

A useful test is:

> Could a correct alternative implementation of this model operation satisfy the same public semantic contract without Choreo changing?
>
> If the answer is no only because Choreo depends on private implementation details, the boundary is wrong.

Model and choreography boundary invariant

At the end of this language/model layer, the unified design requires the following relationship:

> The choreography is the global specification of distributed control flow. Projection produces role-local behavior justified by each role's knowledge. Model `core` operations remain the public owners of authoritative business semantics, FX realizes those operations below that boundary, and Choreo coordinates their distributed use above it. No layer gains stronger semantic facts merely by crossing a representation, namespace, transport, or runtime boundary.

HumanHelp Claim can make that dependency direction concrete without requiring Choreo to know how Request persistence is implemented. A plausible public model contract might look like this:

```
(ns net.humanhelp.site.model.request.core
  (:require
   [net.humanhelp.site.model.request.domain :as domain]
   [net.humanhelp.site.model.request.fx :as request.fx]))

;; This is semantic command material supplied by the caller.  The authenticated
;; helper is deliberately absent: browser or transport data does not get to
;; assert the principal that will own the authoritative transition.
(def ClaimCommand
  [:map {:closed true}
   [:request-id uuid?]
   [:command-id uuid?]])

(def ClaimOutcome
  [:multi {:dispatch :outcome}
   [:committed
    [:map
     [:outcome [:= :committed]]
     [:request Request]
     [:command-id uuid?]]]

   [:rejected
    [:map
     [:outcome [:= :rejected]]
     [:reason
      [:enum
       :request-not-open
       :membership-inactive
       :membership-not-for-principal
```

```
       :wrong-organization
       :organization-disallows-claim
       :authoritative-precondition-changed]]
     ;; A principal may not be entitled to see the current Request on denial.
     [:request {:optional true} [:maybe Request]]
     [:command-id uuid?]]]

   [:already-incorporated
    [:map
     [:outcome [:= :already-incorporated]]
     [:request Request]
     [:command-id uuid?]]]

   [:commit-unknown
    [:map
     [:outcome [:= :commit-unknown]]
     [:command-id uuid?]]]])

(def claim-contract
  {:operation/id :request/claim

   :input ClaimCommand

   :outcome ClaimOutcome

   :authority
   [:authority :request]

   :atomicity
   :one-authoritative-transition

   :requires
   #{:authenticated-principal
     :current-request
     :current-membership
     :current-organization}

   :effects
   {:authoritative
    #{:request/claim-transition}

    :after-commit
    #{:request/live-invalidation
      :request/settlement}}})

(defn claim
  "Public Request semantic operation.

   The command carries caller intent and semantic command identity.  Trusted
   runtime context supplies the authenticated principal and the authoritative
   observations required by the contract."
  [ctx command]
  (m/assert ClaimCommand command)
  (request.fx/run-claim ctx command))
```

The choreography depends on that public contract. Notice that its parameters do not include a helper or principal identity. Choreo can carry the semantic command while leaving principal binding to the trusted Request authority:

```
(ns net.humanhelp.site.model.request.choreo
  (:require
   [gesso.choreo.core :as choreo]
   [net.humanhelp.site.model.request.core :as request]))

(defn claim-request
  [{:keys [request-id command-id]}]
  (choreo/at
    :request-authority

    (choreo/invoke
      {:operation :request/claim
       :contract request/claim-contract}

      (fn [ctx]
        (request/claim
          ctx
          {:request-id request-id
           :command-id command-id})))))
```

Nothing in that choreography needs to know whether the model uses one query, five queries, an XTDB transaction function, a Biff FX pipeline, or a future implementation with different private staging data.

Below `request.core`, the selected realization may be considerably richer. The following FX-like pipeline is illustrative rather than a commitment to exact Biff syntax, but it preserves two boundaries at once: invocation input is explicit, while the authenticated principal is rebound from trusted runtime context instead of being accepted from command data.

```
(ns net.humanhelp.site.model.request.fx
  (:require
   [com.biffweb :as biff]
   [net.humanhelp.auth.authority :as authority]
   [net.humanhelp.site.model.membership.core :as membership]
   [net.humanhelp.site.model.organization.core :as organization]
   [net.humanhelp.site.model.request.domain :as domain]
   [net.humanhelp.site.model.request.persistence :as persistence]))

(def claim-pipeline
  {:start
   (fn [ctx command]
     {:state :prepare
      :command command
      ;; Rebound at the trusted invocation boundary, not taken from command.
      :principal (authority/require-authenticated-principal ctx)})

   :prepare
   (fn [ctx {:keys [command principal]}]
     ;; Explicit intent and trusted principal move forward separately.
     ;; A model pre-read here could reject a compatible already-incorporated
     ;; retry before the authoritative command record is examined.
     {:state :commit
      :command command
      :principal principal})

   :commit
   (fn [ctx {:keys [command principal]}]
     ;; One public model-owned transaction first resolves any compatible
     ;; prior command under current disclosure policy. For a new command it
     ;; checks current Request, Membership, Organization and command binding,
     ;; then writes the claim, command result and delivery debt atomically.
     (let [result
```

```
          (persistence/submit-claim!
            ctx
            {:command command
             :principal principal})]
      (case (:commit/status result)
        :committed
        {:state :after-commit
         :command command
         :commit result}

        :already-incorporated
        {:state :done
         :result
         {:outcome :already-incorporated
          :request (:request result)
          :command-id (:command-id command)}}

        :rejected
        {:state :done
         :result
         {:outcome :rejected
          :reason (:reason result)
          :request (:request result)
          :command-id (:command-id command)}}

        :unknown
        {:state :done
         :result
         {:outcome :commit-unknown
          :command-id (:command-id command)}})))

   :after-commit
   (fn [ctx {:keys [command commit]}]
     ;; The durable outbox/delivery debt was committed with the claim.
     ;; Publication failure cannot become a claim rejection.
     (let [publication
           (try
             (persistence/publish-claim-consequence!
               ctx
               {:request-id (:request-id command)
                :command-id (:command-id command)
                :basis (:commit/basis commit)})
             (catch Exception _delivery-error
               ;; The durable debt remains available for a later worker.
               {:publication/status :deferred
                :command-id (:command-id command)}))]
       {:state :done
        :result
        {:outcome :committed
         :request (:request commit)
         :command-id (:command-id command)
         :publication publication}}))

   :done
   (fn [_ctx {:keys [result]}]
     result)})

(defn run-claim
  [ctx command]
```

```
  (biff/run-state-machine
    ctx
    claim-pipeline
    command))
```

The private `submit-claim!` entry above denotes an atomic transaction contract, not a new required public API. It returns an already incorporated result only after checking command meaning and permission to disclose that result. It resolves that case **before** treating the now-claimed Request as a new command with failed `:open` precondition. A read in `:prepare` supplies a candidate or diagnostics; a precheck rejection cannot be final while a prior incorporation remains possible. Any committed result also records delivery debt atomically so failure during later publication cannot erase the duty to notify or imply non-incorporation. The concrete store mechanism and evidence for those properties remain Part IV implementation obligations.

The private pipeline may be replaced later without changing the public meaning of `request/claim`. Conversely, the choreography must not reach down and call a private state directly:

```
;; Forbidden architectural shortcut.

(ns net.humanhelp.site.model.request.choreo
  (:require
   [net.humanhelp.site.model.request.fx :as request.fx]))

(defn claim-request [ctx command]
  ;; This bypasses the public semantic operation and couples choreography
  ;; directly to one FX realization.  Supplying a prepared value here would
  ;; also bypass the trusted principal/current-authority preparation boundary.
  ((:commit request.fx/claim-pipeline)
   ctx
   {:command command
    :prepared (:client-supplied-prepared-plan command)}))
```

That shortcut would make private transaction staging part of the distributed protocol even though `request.core` never promised it. The public model boundary exists precisely so Choreo can coordinate one stable semantic operation while the authoritative implementation remains free to be sophisticated underneath.

Atomic authoritative transitions

If moving authoritative domain state A to B must be atomic, there must never be an authoritative A-half state.

The computation deciding B may be multi-step. Establishment of B within the declared authoritative transaction scope is one atomic step. Publication of an invalidation or notification is a separate after-commit effect.

Conceptually:

```
read authoritative facts
        |
derive / validate / decide
        |
prepare proposed transition
        |
identify commit-time guards
        |
one atomic transaction
        |
B
```

The important race is between evaluation and commit.

For example:

```
read:
    Request R is unclaimed
    Membership U is active
    Organization permits the operation

decide:
    claim is allowed

              another transaction changes a relevant fact

commit:
    ???
```

An unconditional write at the end would be unsafe. The authoritative operation must protect the facts whose incompatible change would invalidate the prepared decision.

Semantically:

```
read relevant authoritative facts
             |
             v
derive proposed transition
             |
             v
transaction:
    assert required preconditions still hold
    +
    apply every authoritative write in the transition
             |
          success?
          /     \
        yes      no
        |         |
        B      no partial B
                  |
                  v
            re-evaluate or
            return a semantic outcome
```

The exact XTDB mechanism used for commit guards is an implementation question for the model/FX layer. The choreography language should express the authoritative commit boundary and rely on the public model contract rather than reproducing those guards itself.

Not every value read while computing a response must become a transaction guard. Only facts whose incompatible change could invalidate the semantic transition need protection.

Model boundaries are API boundaries, not necessarily transaction boundaries.

A Request operation may depend on facts exposed by Request, Membership, and Organization public interfaces. If several authoritative writes collectively constitute one invariant, they must be composed into one transaction rather than exposed through several independently committing model calls.

The operation should conceptually distinguish:

- preparation;
- atomic commit;
- after-commit effects.

For example:

```
{:commit
 {:guards [...]
  :writes [...]}

 :after-commit
 [[:live/invalidate ...]
  [:notification/send ...]]}
```

The exact FX syntax may differ.

A failure established before submission of the proposed commit, or a confirmed transaction rejection, establishes that this attempt did not commit its proposed transition. It does not establish that the whole authoritative world remains at A: other concurrent operations may have changed it.

Loss of contact after commit submission is different. If the trusted caller cannot establish whether the transaction committed, it must preserve an unknown commit outcome and use the declared resolution contract. An exception or timeout does not by itself locate the failure before the commit boundary.

Once the authoritative commit succeeds, this transition established B even if later publication, notification, SSE delivery, settlement delivery, or browser rendering fails. Subsequent authoritative operations may advance the state again; evidence of this commit is not a promise that B remains current forever.

Post-commit failure must therefore never be reported as though the authoritative mutation itself had failed. Post-commit work requires its own delivery, idempotency, retry, or convergence semantics.

If an operation spans an external system that cannot participate in the same atomic transaction, any externally observable intermediate state must itself be modeled as a valid domain state.

For example:

```
:pending-provisioning
:awaiting-external-activation
```

The system must not hide an externally real partial state and merely hope nobody observes it.

Authoritative transition contract

An authoritative operation exposes one semantic transition contract even when its implementation contains several computational phases.

The contract must distinguish at least:

```
prepare
    read and derive candidate transition

commit
    atomically establish the authoritative transition

after-commit
    publish, invalidate, notify, settle, or otherwise propagate consequences
```

Only the commit boundary changes authoritative domain truth.

This distinction is not merely operational bookkeeping. It determines what failure means.

When non-incorporation of this attempt is established:

```
failure before commit submission, or confirmed transaction rejection
    -> this attempt did not establish its proposed authoritative transition
```

When submission occurred but the result is unavailable:

```
commit request submitted
    -> acknowledgement/result lost
    -> commit outcome unknown to this participant
    -> resolve or retry only under the declared command/transaction contract
```

After commit:

```
publication failure
notification failure
settlement-delivery failure
Live-delivery failure
browser-render failure
    -> authoritative transition DID occur
    -> propagation/recovery problem remains
```

A caller must never be told ”the mutation failed” merely because a later delivery mechanism failed after the authoritative transaction committed.

Semantic rules become authoritative implementation obligations

The auditable semantic surface names the business rules that determine whether an operation is allowed and what it establishes. An authoritative operation must not lose those identities when it crosses from model semantics into transaction implementation.

For every correctness-relevant rule on which an authoritative transition depends, the operation must make clear how that rule participates in the transition contract.

A rule may be discharged in different ways. For example:

- a trusted principal binding may already have been established by an admitted server boundary and carried unchanged into the transaction;
- an immutable command field may be checked against the command's retained binding;
- a mutable authoritative fact may need to be read or asserted again at commit;
- a relationship or uniqueness predicate may require a transaction mechanism stronger than checking a previously read entity version;
- a postcondition may follow from the complete atomic write set rather than from a separate guard;
- an external or otherwise opaque proposition may remain an explicit trusted premise if the supported implementation cannot establish it mechanically.

The design does not require every semantic rule to become a literal database `ASSERT`. It requires a visible correspondence between the rule and the mechanism that is claimed to discharge it.

Conceptually, an authoritative operation should be able to expose information of this shape:

```
{:operation/id :request/claim

 :semantic/requires
 #{:request/open
   :request/unclaimed
   :membership/active-for-principal
   :membership/request-organization-match
   :organization/claiming-enabled}

 :semantic/establishes
 #{:request/claimed-by-principal
   :command/incorporated-result-recorded}

 :commit
 {:obligations
  [{:rule/id :request/open
    :discharged-by :transaction-current-guard}

   {:rule/id :request/unclaimed
    :discharged-by :transaction-current-guard}

   {:rule/id :membership/active-for-principal
    :discharged-by :transaction-current-guard}

   {:rule/id :membership/request-organization-match
    :discharged-by :transaction-current-relationship-check}

   {:rule/id :organization/claiming-enabled
    :discharged-by :transaction-current-guard}

   {:rule/id :request/claimed-by-principal
    :discharged-by :atomic-write-set}

   {:rule/id :command/incorporated-result-recorded
    :discharged-by :atomic-write-set}]}}
```

The exact representation and names above are illustrative. A real operation may derive several rules from one database predicate, or one rule may require several transaction primitives. The semantic identity must nevertheless remain available so later diagnostics, whole-application assembly, and formal correspondence can say **which rule** a mechanism is intended to establish.

This correspondence is especially important when preparation has already computed the same predicate. An earlier successful evaluation such as:

```
(= :active (:membership/status membership))
```

is evidence about the observation used during preparation. It is not by itself evidence that `:membership/active-for-principal` still holds when the claim commits. If that rule is required at commit, the authoritative realization must protect or re-establish it under the selected transaction semantics.

Conversely, an implementation need not repeat a rule mechanically when the operation contract establishes that its truth cannot change across the relevant boundary. What matters is that this reason is represented rather than inferred from incidental code layout.

This produces an explicit chain:

```
named business rule
        |
        v
operation precondition / postcondition
        |
        v
commit obligation
        |
        v
concrete authoritative mechanism
```

A failure in the last link is an implementation defect even if the pure domain predicate and the formal semantic rule are both correct. For example, if `:request/open` is modeled as a commit precondition but the XTDB realization performs only an earlier ordinary read followed by an unconditional write, the implementation has not discharged the operation's atomicity obligation.

The operation must also avoid the opposite failure: acquiring additional correctness-relevant policy only in incidental implementation code.

For example:

```
;; Suspicious unless this condition corresponds to a named semantic rule or
;; an explicit trusted premise.
(when (some-complicated-business-predicate ctx request)
  (submit-authoritative-write! ...))
```

If the branch changes whether an authoritative transition is admitted, what it writes, who may cause it, or what protected fact may be disclosed, then its semantic contribution belongs on the auditable surface described in Part II. The implementation should promote the predicate to a named rule, derive it from already named rules, or expose it as an explicit opaque/trusted premise. It must not gain assurance merely by being hidden inside ordinary host code.

This requirement does not turn every implementation detail into model semantics. Retry loops, logging branches, serialization choices, rendering mechanics, query optimizations, and other incidental conditions remain ordinary implementation when they do not alter a correctness-relevant judgment.

The authoritative-operation acceptance criterion is therefore stronger than ”the transaction is atomic”:

> Every correctness-relevant premise and result claimed by an authoritative operation must retain a visible path from its named semantic rule to the mechanism that establishes, protects, derives, or explicitly assumes it at the authoritative boundary. Atomic transaction machinery may discharge that obligation in implementation-specific ways, but an earlier observation or an unnamed host-language branch may not silently substitute for the declared semantic requirement.

Preparation may be rich without weakening atomic publication

The computation that prepares an authoritative transition may involve:

- several model reads;
- deterministic derivations;
- relationship checks;
- current authorization/policy evaluation;
- construction of multiple writes;
- generation of command/result metadata;
- identification of commit-time guards.

There is no requirement that all of this logic fit into one tiny function.

The requirement is instead that any facts whose incompatible change would make the prepared transition invalid are protected at commit.

Conceptually:

```
authoritative read set
        |
        v
derive candidate transition
        |
        v
identify semantic dependencies that must remain compatible
        |
        v
atomic commit:
    guards + complete authoritative write set
```

The model must not confuse ”was consulted during preparation” with ”must be guarded at commit.” Guard the facts and predicates whose incompatible change can invalidate the semantic transition.

Those predicates may concern absence, uniqueness, aggregate constraints, or relationship membership rather than just rows returned by an earlier read. Checking only previously read entity versions is insufficient if a concurrent insertion or relationship change can violate the invariant. The model/FX implementation must identify how its chosen transaction mechanism protects each relevant predicate under the actual database contract.

Preparation also needs observations compatible with the decision being made. Separate reads do not constitute a common snapshot merely because they were performed close together. The operation must establish the snapshot or other compatibility relation required by its invariant and preserve the required premises through commit.

A Request claim is a good example because the decision may depend on public semantics owned by Request, Membership, and Organization while the domain transition still needs one atomic authoritative result.

Assume preparation has acquired mutually compatible authoritative observations **and** a trusted principal binding established by the server boundary. The principal is deliberately separate from the command:

```
(def authoritative-inputs
  {:principal
   {:user/id U
    :authority/source :authenticated-session}

   :request
   {:request/id R
    :request/status :open
    :request/organization-id O
    :request/helper-id nil
    :model/version :request-v17}

   :membership
   {:membership/user-id U
    :membership/organization-id O
    :membership/status :active
    :model/version :membership-v9}

   :organization
   {:organization/id O
    :organization/claiming-enabled? true
    :model/version :organization-v31}

   ;; Caller-controlled semantic intent.  It does not assert who the helper is.
   :command
   {:operation :request/claim
    :command-id C42
    :request-id R}})
```

The domain decision can remain pure while using the trusted principal supplied by preparation rather than a helper identity asserted by command data. It checks that the Organization whose policy is consulted is the one owning both the Request and the helper's Membership:

```
(defn claim-decision
  [{:keys [principal request membership organization]}]
  (let [helper-id (:user/id principal)]
    (cond
      (not= :open (:request/status request))
      {:decision :rejected
       :reason :request-not-open}

      (not= :active (:membership/status membership))
      {:decision :rejected
       :reason :membership-inactive}

      (not= helper-id (:membership/user-id membership))
      {:decision :rejected
       :reason :membership-not-for-principal}

      (not=
        (:request/organization-id request)
        (:membership/organization-id membership))
      {:decision :rejected
       :reason :wrong-organization}

      (not=
        (:request/organization-id request)
        (:organization/id organization))
      {:decision :rejected
       :reason :wrong-organization}

      (not (:organization/claiming-enabled? organization))
      {:decision :rejected
       :reason :organization-disallows-claim}

      :else
      {:decision :allowed
       :transition
       {:request-id (:request/id request)
        :helper-id helper-id}})))
```

Preparation describes a candidate authoritative transaction, even when its earlier read suggests rejection: an existing compatible command may already have committed. The exact XTDB transaction primitives are intentionally abstracted here; the command lookup, current guards, and writes must satisfy one declared atomic decision contract.

```
(defn prepare-claim
  [{:keys [principal request membership organization command] :as inputs}]
  (let [helper-id (:user/id principal)]
    {:prepare/status :candidate

     ;; A pre-read may aid diagnosis, but must not reject an incorporated
     ;; retry before the transaction checks the retained command record.
     :observed/decision (claim-decision inputs)

     ;; Command resolution and the new-command guards/writes execute under
     ;; one authoritative contract. This is not an unprotected lookup.
     :commit/command-resolution
     {:binding {:command-id (:command-id command)
                :operation :request/claim
                :subject (:request/id request)
                :principal helper-id}
      :compatible-existing :return-permitted-recorded-result
      :incompatible-existing :reject
```

```
     :absent :evaluate-new-command}

    ;; Only for a new command, guard transaction-current facts and any
    ;; dependent relationship predicates; failure produces typed rejection.
    ;; The Request's organization association is guarded along with its
    ;; status. The Membership and policy checks refer to that same 0.
    :commit/new-command
    {:guards
     [[:request/still-open
       {:request-id (:request/id request)
        :expected-organization-id (:request/organization-id request)
        :expected-version (:model/version request)}]

      [:membership/still-active-for-principal
       {:user-id helper-id
        :organization-id (:request/organization-id request)
        :expected-version (:model/version membership)}]

      [:organization/still-allows-claim
       {:organization-id (:request/organization-id request)
        :expected-version (:model/version organization)}]]

     :writes
     [[:request/claim
       {:request-id (:request/id request)
        :helper-id helper-id
        :command-id (:command-id command)}]

      [:command/record-incorporated-result
       {:command-id (:command-id command)
        :operation :request/claim
        :subject (:request/id request)
        :principal helper-id}]

      [:delivery/record-publication-debt
       {:command-id (:command-id command)
        :request-id (:request/id request)}]]}}))
```

Suppose preparation succeeds, then another transaction deactivates Membership U before this commit is submitted:

```
(def prepared
  (prepare-claim authoritative-inputs))

;; Concurrent authoritative transaction:
(membership/deactivate!
  tx-context
  {:user-id U
   :organization-id O})

;; Our later commit resolves the command first; for a new command it
;; evaluates current guards atomically with all authoritative writes.
(def result
  (authoritative/submit!
    tx-context
    {:command-resolution (:commit/command-resolution prepared)
     :new-command (:commit/new-command prepared)}))

result
;; ⇒
{:commit/status :rejected
 :failed-guard
 [:membership/still-active-for-principal
  {:user-id U
   :organization-id O}]}
```

The key property is not that the implementation happened to notice a version number. It is that the authoritative transition could not incorporate the Claim while a required premise was false at commit. Nor could it publish only part of the promised write set. These are properties of the Claim transaction, not permanent restrictions on later Membership transitions:

```
;; Both of these hypothetical Claim commit records violate the contract.
;; In the first, a new Claim was incorporated despite a failed current guard.
{:claim-commit/status :incorporated
 :membership/status-at-claim-commit :inactive
 :request/status-after-claim :claimed}

;; In the second, the Request mutation committed without its command result.
{:claim-commit/status :incorporated
 :request/status-after-claim :claimed
 :command-result-record nil}
```

If claiming the Request and recording durable command incorporation jointly constitute the promised invariant, both writes belong to the same authoritative transaction. If Membership status must remain active for the claim to commit, the commit must protect that predicate as required by the selected database mechanism. A subsequent valid deactivation of Membership U can leave the Request claimed while U's Membership is inactive; that later state does not retroactively invalidate the earlier Claim.

Notice that the public model APIs can still remain separate:

```
;; Semantic ownership remains separate.
(request/current ctx R)
(membership/current-for-user ctx U O)
(organization/current ctx O)

;; The public Request claim operation receives command intent only.  It obtains
;; the authenticated helper from trusted ctx before constructing the atomic
;; transition shown above.
(request/claim
  ctx
  {:request-id R
   :command-id C42})
```

API separation did not force four independent commits. Conversely, merely concatenating write forms from several models would not prove that their public preconditions were preserved. The authoritative operation must compose the semantic constraints as well as the data writes.

API boundaries do not force transaction boundaries

A public operation may depend upon facts owned by several models while still forming one authoritative transaction.

For example, Request claim may require public semantic information from:

```
Request
Membership
Organization
```

If the resulting authoritative change is one domain invariant, the implementation may compose the necessary transaction work behind the public Request operation contract.

The design rejects this inference:

```
different model APIs
    => separate independent commits
```

Model APIs organize semantic ownership. Atomicity is determined by the domain transition and the scope supported by the authoritative system. Composition must retain every participating model's required constraints; combining write forms alone does not establish that their contracts have been satisfied.

If the selected stores cannot realize the required joint atomic transition, the operation cannot be admitted as providing that atomicity guarantee. It needs a different realization or an explicitly modeled multi-step domain protocol.

External systems require explicit valid intermediate states

When an external system cannot participate in the authoritative transaction, Gesso must not pretend the overall world changes atomically.

If external provisioning is observable between local commit and external success, that intermediate situation must be represented as a valid domain state, for example:

```
:pending-provisioning
:provisioning-failed
:awaiting-external-activation
```

The model may then define subsequent transitions that resolve that state. Compensation, where supported, is a new authoritative transition with its own conditions and failure semantics. It does not erase the fact that an earlier external effect occurred or guarantee that every external consequence can be undone.

What is forbidden is an externally real partial transition whose existence is hidden merely because the application wished the operation were atomic.

A store checkout is a useful example because a payment provider cannot normally participate in the same XTDB transaction that establishes local Order state.

Pretending otherwise would produce an imaginary atomic operation:

```
;; Misleading pseudo-operation.

(defn place-order! [ctx checkout]
  (atomic
    (payment-provider/charge!
      (:payment-method checkout)
      (:total checkout))

    (order-model/insert!
      ctx
      {:order/status :placed
       :checkout checkout})))
```

No local transaction wrapper can make the remote charge roll back when the process dies after the provider accepts it.

A truthful model instead exposes an intermediate authoritative state:

```
(def OrderStatus
  [:enum
   :payment-pending
   :placed
   :payment-failed
   :cancelled])

(defschema Order
  [:map
   [:order/id uuid?]
   [:order/customer-id uuid?]
   [:order/cart-id uuid?]
   [:order/status OrderStatus]
   [:order/total decimal?]
   [:order/payment-command-id uuid?]
   [:order/payment-method-id uuid?]
   [:order/provider-idempotency-key string?]
   [:order/provider-reference {:optional true} string?]])
```

In this illustrative product policy, a verified payment authorization is sufficient for the local Order to become `:placed`. If placement needs separate stock or fulfillment confirmation, that application must retain a distinct `:payment-authorized` state and declare its subsequent guarded transition.

The initial authoritative transaction can create the pending Order and bind a stable provider idempotency key:

```
(defn begin-order!
  [{:keys [db principal now]}
   {:keys [cart-id command-id payment-method-id]}]

  ;; The command record is resolved atomically before any read that could
  ;; reject a legitimate retry because its Cart has since changed or gone.
  (authoritative/submit!
    db
    {:command-resolution
     {:binding
      {:command-id command-id
       :operation :checkout/place-order
       :subject cart-id
       :principal (:user/id principal)
       :payment-method-id payment-method-id}
      :compatible-existing :return-permitted-recorded-order
      :incompatible-existing :reject}

     ;; The callback runs only on the command-absent branch *inside* the
```

```
    ;; atomic decision. It reads/guards current Cart state and constructs
    ;; the complete write set from that same guarded snapshot.
    :on-absent
    (fn [tx]
      (let [cart
            (cart-model/require-current-checkout-eligible!
              tx
              {:cart-id cart-id
               :customer-id (:user/id principal)})

            order-id (uuid/random)
            provider-key (str "order-payment/" command-id)]

        {:guards
         [[:cart/still-checkout-eligible
           {:cart-id cart-id
            :expected-version (:cart/version cart)}]]

         :writes
         [[:order/create
           {:order/id order-id
            :order/customer-id (:user/id principal)
            :order/cart-id cart-id
            :order/status :payment-pending
            :order/total (:cart/total cart)
            :order/payment-command-id command-id
            :order/payment-method-id payment-method-id
            :order/provider-idempotency-key provider-key
            :order/created-at now}]

          [:command/record
           {:command-id command-id
            :operation :checkout/place-order
            :subject cart-id
            :principal (:user/id principal)
            :payment-method-id payment-method-id
            :result-subject order-id}]

          [:payment/record-authorization-debt
           {:order-id order-id
            :provider-idempotency-key provider-key}]]}))}))
```

Command resolution, Cart guards, Order creation, command-result recording, and the durable payment-work obligation must be one supported atomic decision. The chosen persistence mechanism must prevent concurrent retries from both taking the `:absent` branch. A compatible retry checks permission to disclose the prior result and reuses its recorded Order and provider key; a client cannot substitute a different payment method under the same command identity.

At that point `:payment-pending` is not an embarrassing implementation detail. It is the truthful authoritative state: the Order exists locally, but external payment disposition is not yet established.

The external effect can then run with the stable provider key:

```
(defn authorize-payment!
  [ctx order]
  (payment-provider/authorize!
    (:payment-provider ctx)
    {:idempotency-key
     (:order/provider-idempotency-key order)

     :payment-method-id
     (:order/payment-method-id order)

     :amount
     (:order/total order)

     :metadata
     {:order-id (:order/id order)
      :command-id (:order/payment-command-id order)}}))
```

Provider success becomes input to a **later** authoritative transition:

```
(defn record-payment-authorized!
  [{:keys [db now]} order provider-result]
  (authoritative/submit!
    db
    {:guards
     [[:order/still-payment-pending
       {:order-id (:order/id order)}]

      [:provider-result/trusted-source
       {:provider-idempotency-key
        (:order/provider-idempotency-key order)}]

      [:provider-result/matches-order
       {:order-id (:order/id order)
        :amount (:order/total order)
        :provider-reference
        (:provider/reference provider-result)}]]

     :writes
     [[:order/record-payment-authorization
       {:order-id (:order/id order)
        :provider-reference
        (:provider/reference provider-result)
        :authorized-at now}]

      [:order/set-status
       {:order-id (:order/id order)
        :status :placed}]]}))
```

Provider rejection has its own valid transition:

```
(defn record-payment-failed!
  [{:keys [db now]} order provider-result]
  (authoritative/submit!
    db
    {:guards
     [[:order/still-payment-pending
       {:order-id (:order/id order)}]

      [:provider-result/verified-rejection
       {:order-id (:order/id order)
        :provider-idempotency-key
        (:order/provider-idempotency-key order)}]]

     :writes
     [[:order/payment-failure
       {:order-id (:order/id order)
        :reason (:provider/reason provider-result)
        :failed-at now}]

      [:order/set-status
       {:order-id (:order/id order)
        :status :payment-failed}]]}))
```

The difficult failure window is provider success followed by local process loss before `record-payment-authorized!` commits. That state is real and must have a resolution path. If the provider supports lookup by the same idempotency key, a reconciler may recover it:

```
(defn reconcile-pending-payment!
  [ctx order]
  (let [provider-result
        (payment-provider/lookup-by-idempotency-key
          (:payment-provider ctx)
          (:order/provider-idempotency-key order))]

    (case (:provider/status provider-result)
      :authorized
      (record-payment-authorized!
        ctx
        order
        provider-result)

      :rejected
      (record-payment-failed!
        ctx
        order
        provider-result)

      :not-found
      {:resolution :still-unknown
       :order-id (:order/id order)}

      :pending
      {:resolution :still-pending
       :order-id (:order/id order)})))
```

This example does not require every Gesso application to use an outbox, one payment API, or these exact states. It demonstrates the semantic requirement: when a remote system cannot share the authoritative transaction, any externally real intermediate condition must be representable and recoverable according to the declared application protocol. Compensation, if later added, would be another transition; it would not make the historical payment attempt vanish.

FX value-flow architecture

The Biff FX substrate used by Gesso must preserve a sharp distinction among:

- ambient execution context;
- machine invocation arguments;
- previous-state output.

The intended state-machine discipline is conceptually:

```
:start
(fn [ctx operation-input]
  ...)

:next
(fn [ctx previous-output]
  ...)
```

Ordinary state output must not become ambient context merely because an earlier state returned it.

This means:

```
ctx
    stable/ambient capabilities and environment

operation-input
    explicit value supplied when the machine is invoked

previous-output
    explicit value produced by the preceding state
```

This distinction is an architectural invariant, not a temporary compatibility constraint.

It aligns FX execution with the semantic boundaries established in Parts I–III:

- ambient authority/capability remains ambient;
- operation input remains explicit;
- intermediate computation remains explicit state flow;
- a later state cannot accidentally acquire authority or data merely because some earlier state placed a key into an accumulated context map.

Ambient context is capability-bearing, not a hidden data bus

Examples of values that may legitimately live in ambient context include:

- authenticated request/server environment;
- database/runtime handles;
- clock capability;
- randomness/seed capability;
- transaction execution facilities;
- trusted configuration;
- other explicitly supplied execution capabilities.

Examples of values that should normally move through explicit machine state instead include:

- operation input;
- queried facts produced by an earlier state;
- a prepared model command;
- a transaction plan;
- a previous semantic outcome;
- derived values needed by the next machine state.

This keeps state-machine meaning inspectable.

A machine that requires a value should receive it through a declared path rather than relying on accidental context accumulation.

An available clock or transaction facility is a capability; a particular sampled time or transaction outcome is an explicit value. Merely placing a value in `ctx` does not establish trust. The trusted invocation boundary owns the origin and permitted use of capability-bearing context, and untrusted input must not overwrite it through an ordinary map merge.

Required capabilities and their supplied realizations must be inspectable at construction. A structurally missing required clock, principal binding, or transaction facility must fail admission before first invocation when the absence is already knowable. Concrete authentication, availability, and policy remain runtime premises.

FX state boundaries preserve semantic ownership

An FX state may compute or realize part of a public model operation, but passing through an FX state does not upgrade provenance or authority.

For example:

```
client supplied argument
    -> validated operation input
    -> FX state
```

does not make the argument authoritative merely because trusted server code handled it.

Likewise:

```
query result
    -> machine state output
```

does not automatically become a choreography fact known by some remote actor.

Semantic provenance still comes from the appropriate model, authority, knowledge, or projection rule.

Ordered effects are explicit semantics

Where effect ordering matters, the FX representation must express ordering explicitly rather than relying on incidental collection interpretation.

The supported ordered-effect mechanism is part of the execution contract. It must distinguish sequencing from successful completion, and define what happens to later effects when an earlier one fails. Ordering by itself supplies neither atomicity across effects nor durable retry after process loss.

A sequence such as:

```
commit
then publish invalidation
then enqueue notification
```

must have a representation whose ordering semantics are unambiguous to the FX runtime.

A vector that is merely an effect's value must not be confused with a sequence of effects.

This matters because post-commit correctness often depends on distinguishing:

```
what must occur atomically
what must occur afterward
what may retry independently
```

Graph/multistate integrations obey the same value-flow rule

Any multistate graph/resolver facility built on the same FX execution model must respect the same separation:

```
ambient ctx
invocation/resolver input
previous state output
```

Later states may not silently recover original or intermediate values from ambient context unless those values are intentionally part of that context.

The exact graph API is subordinate to the invariant.

Operation contracts describe effects without exposing implementation details

A public model operation may expose semantic information such as:

- typed inputs;
- typed outcomes;
- authority requirements;
- atomicity;
- declared externally visible effects;
- preconditions;
- postconditions;
- model-defined failure classes.

It should not require Choreo to understand its internal FX state graph.

The later compositional-proof architecture may certify that an FX/operation implementation satisfies such a contract. The contract boundary exists independently of that stronger proof machinery.

Post-commit delivery debt is explicit

Once authoritative commit is established, outstanding propagation work is delivery debt. It does not undo that commit.

A participant lacking adequate evidence may nevertheless remain uncertain about its command. Authoritative disposition, propagation status, and each participant's knowledge are separate dimensions. The command uncertainty contract below applies even when another participant already knows that the commit succeeded.

Examples include:

- Live invalidation not yet published;
- notification not yet delivered;
- settlement response lost;
- browser disconnected before observing the result;
- downstream cache not yet refreshed.

The system should model or diagnose this state honestly.

Recovery may use:

- retry;
- idempotent publication;
- later authoritative reread;
- advisory invalidation;
- reconciliation by command identity;
- another application-specific convergence mechanism.

It must never ”roll back” the truth conceptually merely because an observer did not learn it immediately. An authoritative reread can recover current state without necessarily establishing which command produced it.

1. Publication and recovery contract

   Every claimed commit-to-observation guarantee must identify the propagation and recovery mechanism and its environmental premises. In particular, it must address process loss after commit but before publication, repeated publication, and loss of the original settlement where those failures are in scope.

   A connected observer that misses publication may have no evidence that anything changed. The design must not infer eventual notification merely from the existence of an SSE connection or from successful commit. The supported realization may provide durable publication intent, reconciliation on reconnect, periodic acquisition, explicit user Refresh, or another declared path. Each supports a different claim and must be described accordingly.

   If recovery depends on user Refresh, convergence is conditional on that action. If automatic eventual publication is claimed, the mechanism must retain or reconstruct the necessary work across the failures covered by that claim. Gesso does not require one universal outbox or durable workflow engine.

   Retrying propagation must not re-execute the authoritative mutation unless the command's retry contract explicitly permits that and establishes its safety. The operational acceptance must exercise the selected recovery path, including the commit-before-publication failure window, against the actual production realization.

Authoritative basis and XTDB

XTDB supplies the authoritative database progression/consistency coordinate.

Gesso is intentionally allowed to assume XTDB for its primary persistent application authority. Database/backend independence is not a current design goal when pursuing it would materially increase complexity or weaken correctness.

The semantic vocabulary should still avoid accidental coupling to a particular XTDB Java class or serialization format, and authority remains conceptually broader than one database because external systems may establish their own facts.

Gesso should reuse XTDB's supported consistency mechanisms rather than inventing a second generic global revision system.

The generic semantic layer distinguishes at least three related categories even when their concrete representation contains the same XTDB token:

- `AuthoritativeBasis` – a basis established by a trusted authoritative acquisition or trusted runtime boundary;
- `ObservedBasisClaim` – an adversarial participant's statement about the basis from which it claims to have acted;
- `ProgressionRequirement` – a requirement that a future authoritative acquisition must satisfy for some declared scope.

These names describe meanings, not a mandate for three public wrapper classes. The implementation may reuse one token representation while preventing the supported APIs from treating the categories as interchangeable without the required derivation/admission step.

A browser representation may be known to have been derived from a read consistent through some authoritative coordinate X17.

X17 does **not** mean:

- Request revision 17;
- the seventeenth state of the application;
- that Request changed seventeen times;
- that the database has not advanced since the browser rendered;
- that every actor knows the same facts.

When established by trusted acquisition, it records a consistency coordinate for that observation's declared scope. Carried back by an untrusted browser, the same bytes are only an observed-basis claim unless stronger evidence is established. Neither form alone states that the actor acquired the whole authoritative world.

A command may carry protocol data such as:

```
:command-id
:observed-basis     ; an ObservedBasisClaim at the untrusted boundary
:operation
:arguments
```

and may additionally carry model-specific fact/version/revision information where the model finds that useful.

The dimensions remain separate:

```
XTDB progression / consistency coordinate
    authoritative database visibility/progression

domain fact/version
    semantic identity or version of a relevant model fact

valid/effective time
    when a domain fact applies

actor knowledge frontier
    which authoritative facts a concrete actor has learned
```

The browser must not perform arithmetic comparisons on opaque XTDB tokens unless XTDB explicitly provides a supported comparison operation.

The consistency requirement is instead expressed using XTDB-supported read semantics: a refresh caused by an authoritative transaction must not accidentally return a representation that predates the authoritative change responsible for that refresh.

An authoritative observation should record enough provenance to justify the knowledge it establishes.

For example:

```
{:provenance :authoritative
 :basis X24
 :projection :request/card
 :facts {...}}
```

The exact representation is tentative and need not be exposed literally to application views.

A client command formed from X17 may reach authority after the database has advanced to X23.

Advancement alone is neither conflict nor rejection.

For example:

```
X17
    Request R is unclaimed

client forms command C42

X18
    unrelated Membership changes

X19
    another Request changes

X23
    authority receives C42
```

If the authoritative facts relevant to C42 remain compatible with the operation, the command may still be valid.

Conversely:

```
X17
    Request R is unclaimed

client forms command C42

X18
    another helper claims R

X23
    authority receives C42
```

the semantic situation has changed in a way that may matter.

The reason is not merely "X17 is stale." The reason is that authoritative facts on which `request/claim` depends have changed.

The model re-evaluates the semantic command against current authoritative facts and decides whether it is:

- still legal;
- already incorporated;
- rejected;
- reconcilable;
- transformable;
- otherwise handled by application policy.

Gesso supplies command identity, observed-basis provenance, consistency propagation, and protocol structure.

HumanHelp supplies the semantic interpretation.

Progression is visibility provenance, not domain identity

An authoritative progression/basis coordinate answers a visibility question:

> Through what authoritative database frontier was this observation known to be consistent?

It does not answer:

> What semantic version is this Request?

Those dimensions may correlate in a particular implementation but they are not the same concept.

Consequently, Gesso must not use progression values as ersatz:

- Request revisions;
- command versions;
- semantic operation versions;
- browser execution or resource-instance identities;
- domain valid-time values.

Each identity exists for its own reason.

Scope-aware progression and authoritative acquisition

A progression requirement is meaningful only together with the authoritative acquisition whose consistency it constrains. The design must therefore preserve the scope or coverage to which a requirement applies rather than treating a database progression token as a universal application freshness number.

Examples of acquisition scope include:

- one Request projection;
- a set of Requests selected by a declared board query;
- an Organization projection;
- a relationship needed by one model operation;
- another explicitly declared authoritative read scope.

A read that satisfies requirement P for scope S establishes only the facts and coverage justified by that read. It does not, merely because the returned XTDB basis is newer, establish that unrelated application scopes have been reacquired or adopted.

Conceptually:

```
read Request S
    scope:       [:request S]
    satisfies:   ⩾ P19

therefore:
    authoritative observation of S at a sufficient basis

not therefore:
    whole board adopted through P19
    every Request refreshed through P19
    every outstanding advisory resolved
```

Where an aggregate operation requires a common snapshot, causal relation, or other cross-object compatibility relation, that stronger requirement must be declared and established explicitly. Independently authoritative observations are not automatically mutually compatible for every aggregate computation.

A travel application makes scope-aware progression particularly obvious because two useful acquisitions may share one database while having nothing to do with one another semantically.

Suppose a seat-map invalidation requires a reread of Flight 17 at or beyond some authoritative progression coordinate:

```
(def flight17-requirement
  {:requirement/type :authoritative-acquisition
   :authority :travel-db

   :scope
   {:projection :flight/seat-map
    :flight/id #uuid "f1700000-0000-0000-0000-000000000017"
    :coverage :complete-offered-seats}

   :minimum-basis :P19})
```

An unrelated Hotel 9 acquisition may happen later in the same XTDB database:

```
(def hotel9-observation
  {:provenance :authoritative
   :authority :travel-db

   :scope
   {:projection :hotel/availability
    :hotel/id #uuid "a0900000-0000-0000-0000-000000000009"
    :coverage :rooms-for-date-range}

   :basis :P25

   :facts
   [{:room/type :queen
     :available 3}]})
```

Even if the database contract says `P25` is at least as advanced as `P19`, Hotel 9 does not discharge the Flight 17 requirement:

```
(defn satisfies?
  [observation requirement]
  (and
    (= (:authority observation)
       (:authority requirement))

    (scope/covers?
      (:scope observation)
      (:scope requirement))

    (progression/satisfies-minimum?
      (:basis observation)
      (:minimum-basis requirement))))

(satisfies?
  hotel9-observation
  flight17-requirement)
;; ⇒ false
```

The failure is not progression. It is scope.

A matching Flight 17 observation can satisfy the requirement:

```
(def flight17-observation
  {:provenance :authoritative
   :authority :travel-db

   :scope
   {:projection :flight/seat-map
    :flight/id #uuid "f1700000-0000-0000-0000-000000000017"
    :coverage :complete-offered-seats}

   :basis :P21

   :facts
   [{:seat/id :12A :status :available}
    {:seat/id :12B :status :reserved}
    {:seat/id :12C :status :available}]})

(satisfies?
  flight17-observation
  flight17-requirement)
;; ⇒ true
```

Coverage matters inside one logical subject too. A narrow read of Seat 12A is not automatically a complete seat-map observation:

```
(def seat12a-observation
  {:provenance :authoritative
   :authority :travel-db

   :scope
   {:projection :flight/seat
    :flight/id #uuid "f1700000-0000-0000-0000-000000000017"
    :seat/id :12A
    :coverage :one-seat}

   :basis :P23

   :facts
   [{:seat/id :12A
     :status :available}]})

(satisfies?
  seat12a-observation
  flight17-requirement)
;; ⇒ false
```

The narrower observation may be perfectly authoritative for Seat 12A and still be insufficient to establish complete membership of the seat map.

Requirement combination must preserve the same distinction. Two requirements for the same admitted scope may sometimes be conservatively reduced when one entails the other:

```
(def flight17-P19
  (assoc flight17-requirement
    :minimum-basis :P19))

(def flight17-P22
  (assoc flight17-requirement
    :minimum-basis :P22))

(requirement/entails?
  flight17-P22
  flight17-P19)
;; ⇒ true, only if the selected progression contract establishes it

(requirement/combine
  flight17-P19
  flight17-P22)
;; ⇒
;; {:requirement/type :authoritative-acquisition
;;  :scope {...Flight 17 complete seat map...}
;;  :minimum-basis :P22}
```

The Flight requirement and Hotel requirement are incomparable by scope, so a sound combination can retain both obligations instead of inventing a universal join:

```
(def hotel9-requirement
  {:requirement/type :authoritative-acquisition
   :authority :travel-db
   :scope
   {:projection :hotel/availability
    :hotel/id #uuid "a0900000-0000-0000-0000-000000000009"
    :coverage :rooms-for-date-range}
   :minimum-basis :P24})

(requirement/combine
  flight17-requirement
  hotel9-requirement)
;; ⇒
{:requirements
 [flight17-requirement
  hotel9-requirement]}
```

The same rule applies to HumanHelp board membership. A very new observation of one Request card is not proof that the complete authorized board query has been reacquired. Progression constrains **an acquisition of some declared scope**; it is not a freshness number that floats free of the facts and coverage that were actually observed.

1. Acquisition scope includes query meaning and coverage

   The scope contract must distinguish entity content from aggregate membership. For a board query it must identify the selection, authorization context, and relevant temporal interpretation on which the coverage claim depends. A page, partial result, or narrow projection must not be labeled as a complete board observation merely because it carries a sufficiently new database token.

   Sorting or filtering a locally adopted set does not itself establish new membership. A server-side search or query change establishes only the coverage declared for that acquisition and may not silently bypass the application's adoption policy.

   An empty result proves absence only within the authorized, successfully evaluated scope for which the acquisition contract establishes completeness. Failure, partial results, or denial of access must not be converted into authoritative domain absence. What distinctions may be disclosed remains subject to authorization.

   The contract may derive scope from an admitted acquisition declaration rather than repeating all of its fields in every request. It must still make scope identity, coverage, and compatibility mechanically available where required.

2. Requirement satisfaction and safe combination

The supported progression implementation must define the following semantic relations. The names here are notation, not a required public API.

- `satisfies(O, R)`: admitted authoritative observation O establishes the scope, coverage, and consistency required by R.
- `entails(R2, R1)`: every admitted observation satisfying R2 also satisfies R1.
- `combine(R1, R2)`: an admitted requirement, requirement bundle, or explicitly unsupported result for retaining both obligations.

Normalization must preserve the meaning of a requirement. Where stable identity depends on normalized representation, normalization must also be deterministic under the declared semantic and adapter versions.

A successful combination must be conservative:

```
satisfies(O, combine(R1, R2))
    implies satisfies(O, R1) and satisfies(O, R2)
```

This implication does not require a universal least upper bound. If one observation cannot satisfy both requirements, the representation may retain separate obligations discharged by their respective acquisitions. A claim of one common snapshot remains stronger and requires its own witness.

Discarding R1 because R2 supersedes it is valid only when the supported contract establishes `entails(R2, R1)`. A newer-looking token for another scope is not such evidence. Unknown comparability is not equality, and inability to compute a join is not permission to drop an obligation.

Where duplicate coalescing, reordering, or regrouping is used, the implementation must establish the corresponding idempotence, commutativity, or associativity of semantic obligation meaning. These laws need not imply byte-identical data structures. They must not be claimed for an operation whose supported semantics do not satisfy them.

Strengthening a combined requirement may be safe for observation safety while making progress harder. The acquisition contract must therefore state whether the strengthened requirement remains satisfiable by its supported read path and under which availability or progress assumptions. Combination is not permitted to manufacture an impossible requirement and still claim the original liveness guarantee.

3. Finite representations and boundedness claims

   A declaration of one queued acquisition does not prove that its retained scope or progression metadata is bounded. The implementation must identify the bound it actually supplies: request concurrency, retained obligation representation, admitted scope classes, or a property-specific abstract state space.

   For a supported finite scope domain, a canonical representative per scope may be sufficient if the scope's progression contract supports safe subsumption. For incomparable or dynamically growing scopes, the implementation must state whether it retains a larger bundle, partitions coordination, applies a sound authorized broader acquisition, or rejects an unsupported configuration. It must not silently discard requirements to maintain a claimed bound.

   Any broader acquisition requires its own permission and coverage contract. Acquiring data and adopting it into the visible application remain separate decisions where product policy distinguishes them. Coarsening requirements must not automatically admit unrelated board membership.

   An implementation claiming a finite mechanically explored model must state the abstraction from concrete requirements to that model and why it preserves the property checked. Part VI defines the browser coordinator using these contracts.

Mixed authoritative observations do not imply one aggregate snapshot

An actor may legitimately hold authoritative observations acquired at different times or bases. For example, it may retain a board membership acquired from one authorized read while incorporating a correlated authoritative result for one Request from a later operation.

Such a state is not inherently contradictory. The actor may know each displayed fact through a justified path without claiming that all displayed facts came from one database snapshot.

The representation must therefore avoid overclaiming:

```
Request A justified through P17
Request B justified through P19

≠

complete board snapshot at P19
```

This distinction is semantic and does not require a universal persisted fact graph, per-object integer revision, or one mutable data structure per category. It requires the implementation to retain enough provenance and scope information to make only the consistency claims it can justify.

Progression must flow through authoritative propagation

When an authoritative transaction commits and causes Live or another mechanism to tell an observer that relevant authority changed, the propagation path must preserve enough progression information to prevent the corresponding authoritative reread from going backward behind the change that triggered it.

Conceptually:

```
authoritative transaction commits
        |
        v
commit establishes progression P
        |
        v
primary change / invalidation carries requirement ⩾ P
        |
        v
observer refreshes
        |
        v
authoritative read satisfies the progression requirement
```

The exact representation of ”requirement >= P” must use supported XTDB/Gesso progression semantics rather than arithmetic over opaque tokens.

This is a correctness requirement even though the invalidation itself remains advisory.

Advisory obligations are scoped and discharged by coverage

An advisory may carry or imply a progression requirement for a particular authoritative acquisition scope. That creates an obligation to acquire enough authoritative information before the application may honestly claim that the corresponding advisory debt has been resolved.

A completed read may discharge only obligations actually covered by that read's justified scope and progression relation. Completion of an HTTP request named ”Refresh” is not itself evidence that every currently outstanding advisory has been satisfied.

For example:

```
Refresh starts with obligation O1:
    board query Q must be observed at ⩾ P18

while read is in flight:
    advisory O2 arrives requiring Q at ⩾ P20

response proves Q at P18

result:
    O1 may be discharged
    O2 remains outstanding unless the response independently satisfies it
```

The concrete implementation need not store obligations under those names. It must preserve the meaning: later relevant advisory information cannot disappear merely because an earlier acquisition finished.

Discharge requires an admitted observation establishing coverage, not only a request-start marker, successful HTTP status, echoed minimum requirement, or arrival time. A requested lower bound is a demand; the acquisition contract must establish that the result satisfies it.

The coordinator must retain the active acquisition's captured requirement and account for later obligations independently. A later obligation may be discharged by that response only if its actual justified coverage suffices; it need not force a redundant successor read when that stronger coverage has already been established.

The obligation must also state whether it requires acquisition or incorporation into a particular adopted projection. Merely fetching content that remains uninstalled does not justify claiming that the visible board has adopted it. If an observation cannot be installed because newer admitted state conflicts, the implementation must retain or transfer the outstanding incorporation obligation until a checked successor or reconciliation satisfies it.

Likewise, an older response must not overwrite a newer already-installed authoritative observation merely because the older request completed later in wall-clock time. Rejection, a successor acquisition, or another checked reconciliation strategy may be used where required. What is forbidden is silently manufacturing one compatible snapshot by merging observations whose compatibility has not been established.

The O1/O2 example above can be made executable without yet committing Part VI to one browser-coordinator implementation.

Assume Bob's board currently carries one outstanding advisory:

```
(def board-state
  {:projection
   {:kind :request/board
    :organization-id O
    :coverage :complete-authorized-membership}

   :adopted
   {:basis :P17
    :scope {:kind :request/board
            :organization-id O
            :coverage :complete-authorized-membership}
    :requests [R1 R2]}

   :requirements
   [{:requirement/id :O1
     :authority :request
     :scope
     {:kind :request/board
      :organization-id O
      :coverage :complete-authorized-membership}
     :minimum-basis :P18
     :requires :acquire-and-adopt}]

   :active-acquisition nil})
```

Starting Refresh captures the requirements the acquisition is intended to satisfy:

```
(defn begin-acquisition
  [state acquisition-id]
  (assoc state
    :active-acquisition
    {:acquisition/id acquisition-id
     :resource-instance/id [:acquisition-resource acquisition-id]
     :captured-requirements
     (:requirements state)
     :scope (:projection state)}))

(def refreshing
  (begin-acquisition
    board-state
    :A1))
```

While A1 is in flight, a later invalidation adds O2:

```
(def advisory-O2
  {:requirement/id :O2
   :authority :request
   :scope
   {:kind :request/board
    :organization-id O
    :coverage :complete-authorized-membership}
   :minimum-basis :P20
   :requires :acquire-and-adopt})

(def refreshing-with-O2
  (update
    refreshing
    :requirements
    conj
    advisory-O2))
```

Now the server returns a complete authoritative board observation that is only justified through P18:

```
(def response-A1
  {:acquisition/id :A1
   :resource-instance/id [:acquisition-resource :A1]

   :observation
   {:provenance :authoritative
    :authority :request

    :scope
    {:kind :request/board
     :organization-id O
     :coverage :complete-authorized-membership}

    :basis :P18

    :requests [R1 R2 R3]}})
```

Completion should evaluate each current requirement against the **actual admitted observation**:

```
(defn requirement-satisfied?
  [observation requirement]
  (and
    (= (:authority observation)
       (:authority requirement))
    (scope/covers?
      (:scope observation)
      (:scope requirement))

    (progression/satisfies-minimum?
      (:basis observation)
      (:minimum-basis requirement))))

(defn complete-acquisition
  [trusted-context state completion]
  (let [active (:active-acquisition state)]
    (when-not
      (and
        (some? (:acquisition/id active))
        (= (:acquisition/id active)
           (:acquisition/id completion))
        (= (:resource-instance/id active)
```

```
          (:resource-instance/id completion)))
      (throw
        (ex-info
          "Acquisition completion does not belong to the active resource."
          {:error/type :acquisition/wrong-owner
           :received (:acquisition/id completion)
           :active (:acquisition/id active)})))

    ;; This checks trusted source/correlation, current read authorization,
    ;; declared query scope and the actual consistency/coverage of the result.
    ;; The :provenance key in client-carried data alone proves nothing.
    (let [observation
          (acquisition/require-admitted!
            trusted-context
            active
            completion)

          installable?
          (presentation/can-install?
            (:adopted state)
            observation)

          discharged?
          (fn [requirement]
            (and (requirement-satisfied? observation requirement)
                 (case (:requires requirement)
                   :acquire-only true
                   :acquire-and-adopt installable?
                   false)))

          satisfied (filter discharged? (:requirements state))
          outstanding (remove discharged? (:requirements state))]

      (cond→
        (assoc state
          :requirements (vec outstanding)
          :active-acquisition nil
          :last-discharged (mapv :requirement/id satisfied))

        installable?
        (assoc :adopted
          {:basis (:basis observation)
           :scope (:scope observation)
           :requests (:requests observation)})

        (not installable?)
        (assoc :reconciliation
          {:reason :observation-not-installable
           :outstanding (mapv :requirement/id outstanding)})))))
```

With a P18 response:

```
(def after-A1
  (complete-acquisition
    trusted-acquisition-context
    refreshing-with-O2
    response-A1))

(:last-discharged after-A1)
;; ⇒ [:01]

(mapv :requirement/id
      (:requirements after-A1))
;; ⇒ [:02]
```

O2 did not vanish merely because an HTTP request named ”Refresh” completed.

If the actual response had been justified through P21 for the same complete scope, both requirements could be discharged even though O2 arrived after A1 started:

```
(def response-A1-stronger
  (assoc-in response-A1
    [:observation :basis]
    :P21))

(def after-strong-A1
  (complete-acquisition
    trusted-acquisition-context
    refreshing-with-O2
    response-A1-stronger))

(:requirements after-strong-A1)
;; ⇒ []
```

The captured requirements still matter operationally: they explain why A1 was started and what minimum consistency the acquisition requested. But discharge is justified by the result actually obtained, not by request-start metadata.

Adoption is a separate condition. If a newer installed state makes this response ineligible for installation, fetched bytes cannot discharge a debt that requires visible adoption. The `complete-acquisition` transition above is the only path in this example that clears acquisition debt. If presentation installation fails, acquisition alone may satisfy an `:acquire-only` requirement, but an `:acquire-and-adopt` requirement remains outstanding and a reconciliation successor is recorded. The transition uses the requirements in the **current** state, so an advisory received while A1 was in flight cannot vanish.

A real completion is accepted only after trusted acquisition admission. The `:provenance :authoritative` tag and an `assoc-in` that changes `:basis` in a test fixture cannot turn arbitrary browser bytes into a trusted observation; the P18 and P21 examples assume the fixture supplies correspondingly admitted, authorized results from the declared complete board query.

Part VI may choose a different concrete coordinator representation. The semantic obligation established here is independent of that representation: advisory debt is scoped, a read discharges only what its admitted observation covers, and later requirements cannot be erased by an earlier in-flight acquisition.

Advisory knowledge and authoritative knowledge remain separate

A Live invalidation may establish:

> There is relevant authority at or beyond progression P that you have not necessarily adopted.

It does not establish the changed business fact itself.

Only the subsequent authorized authoritative observation establishes the content.

This distinction permits HumanHelp's intended behavior:

```
other browser receives advisory
    -> knows relevant newer authority exists
    -> does not automatically install unrelated new board membership
    -> user explicitly refreshes
    -> authorized reread adopts the authoritative result for the covered scope
    -> only advisory obligations covered by that result are discharged
```

Knowledge that relevant authority advanced is therefore distinct from adopting a new authoritative projection, and adoption of one scoped observation is distinct from claiming that every related aggregate has been reacquired at the same basis.

Initial acquisition is part of a managed authoritative projection

A managed fragment cannot rely only on future invalidations.

It needs either:

- authoritative initial knowledge already rendered into it; or
- a declared permitted acquisition path capable of obtaining that knowledge.

A fragment with:

```
managed Live semantics
+ empty target
+ no initial projection
+ no acquisition route
```

is structurally incomplete even if its SSE subscription works perfectly.

Whole-application preflight will later verify that such acquisition paths are actually assembled. The semantic requirement is established here because it follows directly from the meaning of authoritative observation.

Observed basis remains a claim unless stronger evidence is verified

The foundational observation contract (#req-observation-evidence) applies at this XTDB boundary. A browser-supplied `:observed-basis` is an `ObservedBasisClaim`: useful protocol context for consistency, reconciliation, diagnostics, and comparison with current authority.

It is not upgraded to `AuthoritativeBasis` merely because:

- the token parses successfully;
- XTDB recognizes the token;
- the token equals a basis the server could have issued;
- the browser labels its provenance `:authoritative`;
- a Malli schema accepts the enclosing map.

Trusted server code may use the claim to construct a safe consistency request or to reason about the participant's asserted starting point without believing that the participant actually observed it. Token parsing, accepted authority/scope, and the supported consistency mode must be validated before use. A malformed, unsupported, or unsatisfiable client demand must follow declared rejection or recovery semantics rather than grant authority or weaken the required read. Operational waiting and resource limits must not turn an unmet requirement into a falsely successful authoritative observation.

If prior observation itself becomes security-relevant, the application requires an explicit server-verifiable mechanism such as an authenticated observation receipt, trusted server-side state, or another declared mechanism. Verification of such evidence may establish the historical issuance or other precise proposition defined by that evidence contract; it does not by itself establish delivery, installation, or human acknowledgement. It still does not silently establish current operation authorization.

Thus:

```
ObservedBasisClaim
    ≠ AuthoritativeBasis
    ≠ AuthenticatedObservationReceipt
    ≠ OperationCapability
    ≠ CurrentAuthorization
```

The concrete implementation may represent several of these using maps or the same underlying XTDB token. Their supported construction and consumption paths must nevertheless preserve the semantic distinctions.

Advancement alone does not make a command invalid

A command formed from X17 may be evaluated when the database is at X23.

That fact alone says nothing about whether the command remains semantically valid.

The authoritative operation must examine the facts relevant to the command.

Thus:

```
basis advanced because unrelated data changed
    -> command may remain valid
```

while:

```
basis advanced because relevant Request fact changed
    -> model may confirm / reconcile / reject / transform
```

This keeps conflict semantics in the model instead of inventing a generic ”stale basis means reject” rule.

Consistency mechanisms remain XTDB-specific where useful

Gesso is not required to invent a backend-neutral consistency algebra merely to avoid acknowledging XTDB.

The system may use XTDB's supported consistency and snapshot mechanisms directly when doing so materially simplifies correctness.

The semantic layer should still name concepts such as:

- authoritative basis;
- progression requirement;
- authoritative observation;
- visibility frontier;

rather than baking accidental concrete token classes into every public API.

This provides conceptual clarity without paying speculative portability cost.

Dependency upgrades may not silently weaken consistency

The concrete Biff/XTDB integration is a trusted/runtime boundary.

Changes in framework or database versions must not silently alter the progression contract on which Gesso relies.

The build and test architecture must therefore preserve regression coverage for:

- snapshot-token propagation;
- required-progress read semantics;
- authoritative commit progression;
- Live invalidation propagation;
- reread consistency.

This is not a requirement to freeze dependencies forever. It is a requirement that changing them cannot silently change the semantic contract.

The concrete adapter must identify the supported dependency versions and the read, transaction, token, and failure contracts on which it relies. These requirements do not claim that a particular XTDB release provides an unspecified guard or comparison operation. The production realization must establish that its selected mechanism supplies the contract being claimed.

Regression evidence must cover scope-sensitive failures as well as token flow: a narrow newer read cannot discharge a broader obligation; a partial query cannot prove complete membership; a read with an unmet minimum cannot be reported as satisfying it; and an unknown commit result cannot become confirmed non-incorporation through an adapter exception mapping.

Command identity and execution identity

`:command-id` and choreography execution identity are different.

`:command-id` identifies one semantic attempted operation.

An execution ID identifies one protocol execution.

The distinction is semantic even if both happen to use the same underlying representation, such as UUIDs, strings, or maps. Supported APIs must not make `:command-id` and execution identity interchangeable merely because their host- language representations are structurally compatible.

A nominal type, opaque wrapper, smart constructor, checked constructor, or another existing Gesso admission mechanism may make that distinction harder to misuse accidentally. This design does not require one particular Clojure representation. It requires the semantic categories and their construction/ consumption paths to remain distinct.

A retry of the same semantic command through another web node should retain the same command ID even if the protocol execution instance changes.

For example:

```
Client sends C42
   |
web node W1 fails
   |
retry of C42
   |
web node W3 handles request
```

Gesso propagates command identity.

HumanHelp/model persistence decides whether durable idempotency is required and how already-incorporated commands are represented.

Command identity is not projected-plan identity.

A browser may retry one command after a deployment using a new execution and a new compatible projected plan while preserving the same semantic command identity where protocol semantics allow that retry.

Semantic command identity survives physical retry

A command ID represents one semantic attempted operation.

It should therefore normally survive:

- retry through another HTTP connection;
- retry through another Aleph node;
- reconnect;
- transient transport failure;
- a compatible projected-plan replacement.

Physical attempts may acquire distinct request IDs or execution IDs while still referring to the same command.

Conceptually:

```
semantic command C42

attempt 1:
    execution E1
    transport request T1
    host W1

attempt 2:
    execution E2
    transport request T2
    host W3

both attempts refer to C42
```

T1 and T2 identify transport attempts, not HumanHelp domain Requests. The example uses a separate protocol execution for each attempt; a supported retry contract may instead permit multiple transport attempts within one execution. The command, execution, and transport identities remain distinct in either case.

Whether both attempts may safely execute is a model/protocol question involving idempotency or already-incorporated handling.

Execution identity scopes protocol-local resources

Execution identity exists because one semantic command may have more than one physical/protocol attempt and because one browser may have several concurrent operations.

Execution identity may scope:

- current machine state;
- transport-request correlation;
- settlement timers;
- provisional target ownership;
- continuity resources;
- resource-instance identities used to admit browser callbacks;
- cleanup.

It is not a durable domain identity.

When one execution is retired, stale callbacks carrying that execution identity or its retired resource-instance identity must not gain power over a replacement execution merely because they refer to the same semantic command or logical target. Replacing a resource can also retire that resource's callback rights without replacing the execution. Admission must identify which owner or instance lost its rights rather than infer retirement from one universal counter.

Command-outcome or authoritative evidence that remains useful may enter through its independently declared admission path. Accepting that evidence does not reactivate a retired execution or resource instance.

The precise browser isolation rules are specified in Part VI and follow owner-relative event admission.

Command identity is not idempotency policy

Gesso propagates semantic command identity.

The model decides what that identity means for durable behavior.

Possible policies include:

- reject duplicate incorporation;
- return the already-incorporated result;
- reconcile with current authoritative state;
- deliberately allow repeated commands where the domain operation is repeatable.

Gesso must not infer universal exactly-once semantics merely because a `:command-id` exists.

Where a command identity participates in retry or deduplication, the model or operation contract must define what semantic material that identity binds. At minimum this commonly includes the operation and its semantic arguments, and may also include principal, authority subject, or another model-defined scope.

Reusing an identifier with incompatible meaning must not silently count as a valid retry. For example:

```
C42 = claim Request R as principal P

retry C42 = same semantic command under the declared retry contract

C42 reused for cancel Request Q
    -> reject as incompatible command identity reuse
```

The exact binding remains model/operation policy. The framework requirement is that retry semantics be explicit enough to prevent identifier equality from being mistaken for semantic equivalence.

Where durable deduplication is claimed, concurrent attempts must not both pass an unprotected ”not seen before” check and incorporate the same command twice. The command record, identity binding, and model transition need the atomic or otherwise justified coordination required by that claim. Recording a result only after an unprotected mutation leaves a failure window and cannot establish durable at-most-once incorporation by itself.

The contract must state the deduplication scope, retention/expiry policy, and what retries mean after the evidence is no longer available. A cached result must not be disclosed to a different principal or subject merely because that caller supplied the same command identifier. Retry handling must preserve the required identity binding and current permission to obtain protected results.

Allowing repeated incorporation is a different explicit model policy. The runtime must not assume that retransmission under that policy is harmless. Command identity alone supplies neither deduplication nor permission to retry.

Command disposition and participant knowledge are distinct

Authoritative reality and a participant's evidence about that reality are different dimensions.

A participant may report a semantic command as incorporated or rejected only when the evidence available to that participant justifies that conclusion. Transport failure, timeout, browser replacement, or loss of a settlement response does not by itself establish either incorporation or rejection.

The design must distinguish at least three questions:

1. What is the current authoritative domain state?
2. What was the authoritative disposition of this semantic command?
3. What evidence about either fact is available to this participant?

Those questions may have different answers.

HumanHelp Claim can illustrate all three questions with one lost-response race.

Suppose Alice's browser forms semantic command C42. The wire command identifies what is being attempted and which semantic attempt this is; it does **not** assert that Alice is the authenticated principal:

```
(def C42
  {:operation :request/claim
   :command-id
   #uuid "c4200000-0000-0000-0000-000000000042"
   :request-id R})
```

When an attempt reaches the Request authority, trusted server context binds the principal independently of that command:

```
(defn admitted-claim-attempt
  [ctx command]
  {:command command
   :principal
   (authority/require-authenticated-principal ctx)})

(def attempt
  ;; The authenticated session represented by ctx belongs to Alice.
  (admitted-claim-attempt ctx C42))
```

The Request model chooses to make command incorporation durable for this operation. A successful claim transaction writes both the Request transition and a command result, and both bind to the trusted principal established above:

```clojure
(defn prepare-claim-tx
  [{:keys [request principal command]}]
  (let [command-id (:command-id command)
        helper-id (:user/id principal)]
    ;; The complete operation first resolves an existing compatible command
    ;; inside this transaction; this map shows only the absent/new branch.
    {:guards
     [[:request/still-open
       {:request-id (:request/id request)}]

      [:membership/active-for-principal
       {:principal helper-id
        :request-id (:request/id request)}]

      [:organization/currently-allows-claim
       {:request-id (:request/id request)}]]

     :writes
     [[:request/claim
       {:request-id (:request/id request)
        :helper-id helper-id
        :command-id command-id}]

      [:command/store-result
       {:command-id command-id
        :operation :request/claim
        :subject (:request/id request)
        :principal helper-id
        :outcome :committed
        :result
        {:request/id (:request/id request)
         :request/status :claimed
         :request/helper-id helper-id}}]

      [:delivery/record-publication-debt
       {:command-id command-id
        :request-id (:request/id request)}]]}))
```

Assume that transaction commits, but the HTTP connection disappears before Alice's browser receives settlement:

```
browser
    submit C42
        |
        v
server
    bind authenticated principal = Alice
    authoritative transaction commits:
        Request R -> claimed by Alice
        command C42 -> committed for Alice
        |
        X response connection lost
        |
browser
    timeout
```

At that moment the authoritative answers are:

```
{:current-request
 {:request/id R
  :request/status :claimed
  :request/helper-id Alice}

 :command-disposition
 {:command-id C42
  :principal Alice
  :outcome :committed}

 :alice-evidence
 :outcome-unknown}
```

The browser timeout changes only the third coordinate. It does not reverse the Request transition and does not turn the command into a confirmed rejection.

Suppose Alice first recovers by rereading Request R. The authorized read again uses the authenticated principal from trusted context rather than taking a principal supplied by the browser:

```
(def current-request
  (request/read-authorized-card
    ctx
    {:request-id R}))

current-request
;; ⇒
{:request/id R
 :request/status :claimed
 :request/helper-id Alice}
```

That may be enough to render the correct current UI. But without stronger command-specific evidence, the state alone need not prove that C42 caused it. Another concurrent command by Alice, an administrator operation, or another model-defined path might have produced an observationally equivalent current Request.

Because this model **does** retain command-specific results, Alice can separately resolve attribution. The resolver compares the retained binding with the current trusted principal; the lookup request does not get to assert who that principal is:

```
(defn resolve-command!
  [ctx {:keys [command-id operation subject]}]
  (let [principal
        (authority/require-authenticated-principal ctx)

        expected-binding
        {:operation operation
         :subject subject
         :principal (:user/id principal)}

        ;; The lookup itself enforces disclosure for the current principal.
        ;; Foreign command identities are indistinguishable from unavailable
        ;; evidence through this public query.
        record
        (command-model/find-disclosable-result
          (:db ctx)
          {:command-id command-id
           :principal (:user/id principal)})]

    (cond
      (nil? record)
      {:resolution :not-established
       :reason :no-retained-command-evidence}
```

```
      (not=
        expected-binding
        (select-keys
          record
          [:operation :subject :principal]))
      (throw
        (ex-info
          "Command id is bound to different semantic material."
          {:error/type :command/incompatible-reuse
           :command-id command-id}))

      :else
      {:resolution :established
       :record record})))

(resolve-command!
  ctx
  {:command-id (:command-id C42)
   :operation :request/claim
   :subject R})
;; ⇒
{:resolution :established
 :record
 {:command-id C42
  :operation :request/claim
  :subject R
  :principal Alice
  :outcome :committed
  :result
  {:request/id R
   :request/status :claimed
   :request/helper-id Alice}}}
```

A permitted retry can carry the same semantic command identity through a new transport request and even a new protocol execution. The new attempt still rebinds the current authenticated principal before comparing semantic material:

```
(def retry-attempt
  {:command C42

   :execution-id
   #uuid "e8000000-0000-0000-0000-000000000008"

   :transport-request-id
   #uuid "72000000-0000-0000-0000-000000000002"})

(defn handle-claim-command!
  [ctx {:keys [command]}]
  ;; The admitted route fixes :request/claim; client :operation bytes cannot
  ;; redirect this handler. The public model operation binds the principal
  ;; from ctx and atomically resolves duplicates and new claims.
  (request/claim
    ctx
    {:request-id (:request-id command)
     :command-id (:command-id command)}))
```

An earlier `find-result` followed by `request/claim` would leave a race between two attempts that both observe no record. The model-owned atomic decision described above is the authoritative retry boundary. A response that says `:already-incorporated` is disclosed only to a currently permitted principal for the original binding; an incompatible or foreign command id cannot be used to fetch another principal's result.

Reusing C42 for incompatible semantic material is not a retry. Even under the same authenticated Alice context, changing

operation or subject changes the semantic binding:

```
(command-model/require-compatible-binding!
  {:command-id (:command-id C42)
   :operation :request/claim
   :subject R
   :principal Alice}

  {:operation :request/cancel
   :subject Q
   :principal Alice})

;; ⇒
;; {:error/type :command/incompatible-reuse
;;  :command-id C42
;;  :expected {:operation :request/claim
;;             :subject R
;;             :principal Alice}
;;  :received {:operation :request/cancel
;;             :subject Q
;;             :principal Alice}}
```

Nor does inventing C43 solve an unknown C42 if duplicate effects matter:

```
;; This is a new semantic attempt, not "the retry of C42".
{:operation :request/claim
 :command-id C43
 :request-id R}
```

Whether a fresh command is permitted is a separate model/protocol decision.

This worked example separates the three questions explicitly:

```
{:authoritative-domain-state
 (request/read-current ctx R)

 :authoritative-command-disposition
 (command-model/find-result
   (:db ctx)
   (:command-id C42))

 :participant-current-evidence
 (client-state/command-knowledge
   browser-state
   (:command-id C42))}
```

Those values may converge later, but they are not definitionally the same. Current-state recovery can repair presentation without fabricating historical attribution, and command-specific evidence can establish attribution without reviving the retired browser execution that originally submitted the command.

For example:

```
client sends C42
    |
network outcome becomes unknown to client
    |
client later reads Request R = :claimed
```

The reread may be sufficient to install a correct authoritative presentation. It does not necessarily prove that C42 produced the claimed state. Another actor or command may have produced an observationally equivalent result.

Therefore current-state convergence and command attribution are separate claims. An application that requires command-specific attribution must obtain command-specific evidence under the model's declared command contract. An application that does not require attribution may recover by authoritative reread without inventing a conclusion about C42.

From a participant's perspective, the command semantics must be able to express at least the difference among:

- confirmed incorporation;
- confirmed rejection or another model-defined non-incorporation outcome;
- outcome not yet established for that participant.

No fixed public enum is required. The semantic distinction is required.

A runtime timeout must never be translated automatically into confirmed model rejection merely because the local execution needs to terminate.

Non-incorporation evidence must identify its scope. Rejection of one physical attempt does not necessarily establish that the semantic command was never incorporated by another concurrent or earlier attempt. A command-level rejection claim requires the stronger evidence defined by the command contract.

Likewise, failing to find a command record is not automatically proof of non-incorporation: the lookup may be stale, scoped differently, incomplete, or outside the record-retention period. The model contract must establish when a negative lookup justifies a negative command conclusion.

1. Retry and local retirement after an unknown outcome

   When outcome remains unknown, the protocol may reacquire current state, query command-specific evidence, retry the same bound command if permitted, or expose another declared recovery path. The choice must respect the model's retry and attribution contract.

   Creating a fresh command identifier is a new semantic attempt; it is not a safe substitute for resolving an earlier unknown attempt when duplicate effects matter. Conversely, retaining an identifier does not make a non-idempotent operation safe to repeat without its declared duplicate-handling mechanism.

   Local execution retirement revokes local ownership. It neither cancels a commit already in progress nor establishes server-side cancellation. Authoritative cancellation, where supported, is a separate model operation or protocol with its own race and outcome semantics.

   An outcome learned after retirement may still be relevant to current command-resolution or acquisition state. It must enter through an admitted owner and the current correlation contract. It must not revive the retired execution or bypass stale-callback rejection to deliver useful information. Part VI defines the browser realization of that distinction.

Command identity is distinct from operation identity

The operation identifies **what kind** of semantic action is requested.

The command identifies **this semantic attempt** of that action.

For example:

```
operation:
    :request/claim

commands:
    C42
    C77
    C103
```

All may invoke the same operation while representing different semantic attempts.

Command identity is distinct from projected-plan compatibility

A browser may retain or retry a semantic command across a deployment only when the current server/runtime contract defines that continuation as compatible.

The command may remain C42 even when:

```
old execution E7
old plan P7

becomes

new execution E8
new compatible plan P8
```

If the new plan is incompatible, the command's existence does not authorize an undefined continuation.

Stale-plan recovery belongs to Part VI.

Correlation identities may be more numerous than semantic identities

One command can produce several runtime correlation identities:

- transport-request ID for one HTTP attempt;
- execution ID;
- browser resource-instance identity;
- settlement correlation;
- fragment refresh correlation.

These identities must not be collapsed merely to reduce metadata.

Each exists to prevent a particular class of ambiguity or stale-event bug. In browser/runtime examples, `:transport-request-id` names the transport attempt; a domain `:request-id` continues to identify the HumanHelp Request. A specialized resource-instance identity may serve as the transport-request identity when the contract identifies exactly that resource; distinct semantics do not require redundant tokens for the same instance.

The design favors explicit identity over attempting to rediscover ownership later from DOM position, current URL, or another mutable physical fact.

Part V — Provisional state and executable presentation

This part defines provisional rendering, local continuity, semantic view bindings, portable execution, and the boundary between executable plans and diagnostic evidence. It builds on Part IV's scoped observations and command uncertainty. Part VI defines browser coordination and physical realization; Part VII closes the complete application path.

Optimism

Optimism is part of the semantic distributed operation.

It is not merely a DOM trick.

An optimistic projection is explicit provisional knowledge derived from an authoritative basis and associated with a semantic command.

Conceptually:

```
{:authority :provisional
 :command-id C42
 :observed-basis X100
 :projection P}
```

This example is conceptual. The `:authority :provisional` field classifies local projection status; it is not an authority grant. Under the semantic admission rules, neither that tag nor a copied basis token proves that the required observation was acquired.

A provisional projection never claims to be a future authoritative XTDB state.

It means approximately:

> Given the scoped authoritative observations this actor acquired at the stated basis, this is the permitted local projection if command C42 is incorporated in the modeled way. The provisional rendering is not a prediction guaranteed to become authoritative.

A later authoritative fact may relate to the provisional projection as:

```
P --confirmed-by----> F
P --reconciled-by---> F
P --rejected-by-----> F
P --superseded-by---> F
```

The verifier should enforce at least:

- every provisional projection has a known authoritative basis;
- every provisional projection has a semantic command identity;
- provisional state is explicitly marked provisional;
- provisional state cannot silently become authoritative;
- only an appropriate authority establishes authoritative resolution;
- an actor may derive optimism only from facts it knows;
- every provisional trajectory has a resolution or recovery path;
- authoritative supersession is a valid resolution;
- semantic ownership of resources used to realize optimism ends or transfers through an explicit admitted rule on every terminal path; physical cleanup follows its separate best-effort contract.

Multiple clients may hold mutually incompatible provisional projections without creating multiple authoritative histories.

HumanHelp Claim is a useful worked example because its provisional presentation is visible to the user while its authority remains entirely elsewhere.

Assume the browser has already acquired an authoritative Request card:

```
(def authoritative-card
  ;; Fixture notation for a trusted acquisition result, not a browser
  ;; constructor. A copied tag or map cannot establish admission.
  (projection/admitted
    {:projection/type :request/card
     :authority :authoritative
     :request/id R
     :request/status :open
     :request/helper-id nil
     :basis :X17
     :scope
     {:projection :request/card
      :request/id R}}

    {:admission/id :admission/request-card-X17
     :acquisition/id :acquisition/request-card
     :provenance :trusted-server
     :authorized-principal [:user Alice]
     :covered-through :X17}))
```

The view binds a semantic Claim operation:

```
(def bound-claim
  {:operation :request/claim
   :role :helper
   :args {:request/id R}
   :observation
   {:scope
    {:projection :request/card
     :request/id R}
    :basis :X17}})
```

When Alice clicks Claim, the runtime allocates a semantic command and may derive a provisional projection:

```
(def C42
  #uuid "c4200000-0000-0000-0000-000000000042")

(def E42
  #uuid "e4200000-0000-0000-0000-000000000042")

(def T42
  #uuid "91000000-0000-0000-0000-000000000001")

(def provisional-claim
  {:projection/type :request/card
   :authority :provisional

   :command/id C42

   :basis
   {:scope
    {:projection :request/card
     :request/id R}
    :observed-through :X17}

   :request/id R
   :request/status :claiming
   :request/helper-id Alice

   :presentation
   {:label "Claiming..."
    :claim-button :disabled
    :pending? true}})
```

That provisional projection is not a forecast that the model must eventually confirm. It is a typed local state associated with one command.

The runtime can make the transition explicit:

```
(defn begin-claim
  [{:keys [authoritative-card current-owner binding] :as runtime}
   command-id]
  (projection/require-admitted-for!
    runtime authoritative-card current-owner binding)
  (let [card
        (projection/value authoritative-card)
        principal
        (binding/authorized-principal binding)]

    {:machine/state :claim/pending

     :command/id command-id
     :execution/id (:execution/id current-owner)

     :authoritative-basis
     {:scope (:scope card)
      :basis (:basis card)
      :admission/id
      (projection/admission-id authoritative-card)}

     :provisional
     {:projection/type :request/card
      :authority :provisional
      :command/id command-id
      :request/id (:request/id card)
      :request/status :claiming
      :request/helper-id principal}

     :resources
     {:settlement-timeout
      {:resource-instance/id
       (:settlement-timeout/id runtime)
       :owner current-owner
       :status :active}}}))
```

Here the admitted start context carries `{:execution/id E42}` as its current owner and `T42` as its allocated settlement-timeout instance. The binding's principal is authenticated by trusted preparation; it is also subject to the server's independent authorization when the command arrives. A browser label or request body cannot create that authentication.

Now consider four legitimate endings.

First, direct confirmation:

```
(def accepted-settlement
  ;; Fixture notation for a trusted settlement result, not a browser
  ;; constructor. Boundary admission authenticates source and correlates
  ;; execution, command, resource, outcome, and scoped progression.
  (settlement/admitted
    {:event/type :request/claim-settled
     :execution/id E42
     :command/id C42
     :resource-instance/id T42
     :outcome :accepted}

    (projection/admitted
      {:request/id R
       :request/status :claimed
       :request/helper-id Alice
       :basis :X18}

      {:provenance :trusted-command-settlement
       :scope {:projection :request/card
               :request/id R}
       :covered-through :X18})))

(defn settle-accepted
  [runtime admitted-settlement]
  (settlement/require-admitted-for!
    runtime admitted-settlement
    {:outcome :accepted :target :request/card})

  (→ runtime
      (assoc
        :machine/state :claim/settled
        :authoritative
        (settlement/authoritative-projection
          admitted-settlement)
        :provisional nil)
      (update :resources retire-owned-resources)))
```

Second, authoritative rejection because Bob claimed first:

```
(def rejected-settlement
  (settlement/admitted
    {:event/type :request/claim-settled
     :execution/id E42
     :command/id C42
     :resource-instance/id T42
     :outcome :rejected
     :reason :request-not-open}

    (projection/admitted
      {:request/id R
       :request/status :claimed
       :request/helper-id Bob
       :basis :X18}

      {:provenance :trusted-command-settlement
       :scope {:projection :request/card
               :request/id R}
       :covered-through :X18})))

(defn settle-rejected
  [runtime admitted-settlement]
  (settlement/require-admitted-for!
    runtime admitted-settlement
    {:outcome :rejected :target :request/card})

  (→ runtime
      (assoc
        :machine/state :claim/rejected
        :authoritative
        (settlement/authoritative-projection
          admitted-settlement)
        :provisional nil
        :message
        "Another helper claimed this Request.")
      (update :resources retire-owned-resources)))
```

Third, local timeout with outcome still unknown:

```
(def timeout-event
  {:event/type :resource/timeout
   :execution/id E42
   :command/id C42
   :resource-instance/id T42})

(defn settle-timeout
  [runtime timeout]
  (if-not
    (and (= :claim/pending (:machine/state runtime))
         (event/current-resource-owner?
           runtime timeout :settlement-timeout))
    runtime
    (→ runtime
      (assoc
        :machine/state :claim/outcome-unknown

        ;; The optimistic execution ends.
        :provisional nil

        ;; This is participant knowledge, not a domain conclusion.
        :command-knowledge
        {:command/id (:command/id runtime)
         :disposition :unknown})

      (update :resources retire-owned-resources)

      (assoc
        :recovery
        {:kind :authoritative-reacquisition
         :scope
         (get-in runtime
           [:authoritative-basis :scope])})))))
```

The timeout does not synthesize `:rejected`. If the server committed before the connection disappeared, the authoritative Request may already be claimed.

Fourth, an authoritative reread may supersede the provisional trajectory even without command-specific settlement:

```
(def reread-result
  ;; This is not a decoded response map.  It is the result of the trusted
  ;; acquisition boundary after authorization, provenance, plan, scope,
  ;; progression, and resource-instance checks have succeeded.
  (acquisition/admitted
    {:projection/type :request/card
     :authority :authoritative
     :request/id R
     :request/status :claimed
     :request/helper-id Alice
     :basis :X19
     :scope
     {:projection :request/card
      :request/id R}}

    {:acquisition/id :acquisition/request-card
     :resource-instance/id
     #uuid "a1900000-0000-0000-0000-000000000019"
     :provenance :trusted-server
     :authorized-principal [:user Alice]
     :covered-through :X19}))

(defn install-authoritative-reread
  [runtime admitted-observation]
  ;; Admission is relative to the current target owner, authorized scope,
  ;; installed observation, and declared progression rule.
  (if
    (acquisition/admitted-for-current-target?
      runtime admitted-observation
      (get-in runtime [:authoritative-basis :scope]))
    (→ runtime
          (assoc
            :machine/state :claim/recovered
            :authoritative
            (acquisition/authoritative-projection
              admitted-observation)
            :provisional nil)

          ;; The current view is now authoritative even if command attribution
          ;; remains unresolved.
          (assoc
            :command-knowledge
            {:command/id (:command/id runtime)
             :disposition
             (or
               (get-in runtime
                 [:command-knowledge :disposition])
               :unknown)})

          (update
            :resources
            retire-owned-resources))

    runtime))
```

The browser can now render the correct authoritative Request card while still saying, internally, ”I do not know whether C42 caused this state.”

That difference matters when another cause could have produced the same current Request state.

The same example also shows why rollback is not unconditional. Suppose the browser preserved the old open card as a local snapshot:

```
(def optimistic-snapshot
  {:request/id R
   :request/status :open
   :request/helper-id nil
   :basis :X17})

(def optimistic-capture
  {:admitted-basis
   (projection/admission-id authoritative-card)
   :presentation optimistic-snapshot
   :owner {:execution/id E42}})
```

After `reread-result` at X19 has been admitted and installed, a late failure callback from the old execution must not do this:

```
;; Forbidden stale rollback.

(defn late-failure-handler [runtime _event]
  (assoc runtime
    :authoritative optimistic-snapshot
    :machine/state :claim/rolled-back))
```

The snapshot may have been justified when captured. It no longer owns the target after authoritative supersession.

A correct failure callback first checks current ownership and installation state:

```
(defn maybe-restore-snapshot
  [runtime event captured]
  (if
    (and (event/current-provisional-owner? runtime event)
         (snapshot/same-admitted-basis?
           captured (:authoritative-basis runtime))
         (not (authoritative/installed-after-capture?
                runtime captured)))
    (assoc runtime
      :presentation
      {:source :prior-justified-presentation
       :snapshot (:presentation captured)
       :status :recovery-pending})
    runtime))
```

Even in the last branch, restoring a familiar visual shape does not assert that authority reverted or that the command failed. Presentation may resemble the old card while its semantic classification remains recovery-pending or outcome-unknown.

The code is intentionally larger than the minimal runtime representation likely to ship. Its purpose is to make the semantics visible: command identity, authoritative basis, provisional projection, local resource ownership, settlement evidence, authoritative supersession, and command attribution are related but not interchangeable.

For example:

```
        authoritative F17
         /             \
provisional Alice   provisional Bob
```

The Request model may subsequently confirm one, reconcile both, reject one, or apply another domain policy.

That policy remains in the model.

A later authoritative reread may supersede the provisional projection even if the browser never receives the original command settlement.

That is an important recovery path for unreliable connections.

Optimism is typed provisional knowledge

An optimistic result is not ”pretend the database already changed.”

It is a local semantic statement:

> Relative to the authoritative facts I know, and for semantic command C, this is the provisional projection I am permitted to render while authority resolves the command.

That statement therefore carries provenance.

At minimum the runtime must be able to distinguish:

```
authoritative observation for declared scope
provisional(command C, scoped basis P)
authoritative absence within declared complete acquisition scope
unknown / not yet acquired / acquisition unsuccessful
```

A DOM node that merely looks like an authoritative card is insufficient. Likewise, a same-shaped Clojure map does not become an authoritative projection merely because its keys resemble one.

The supported runtime path must preserve the semantic distinction between provisional and authoritative values even where both are represented as ordinary immutable data. Nominal or opaque representations are permitted when they make accidental substitution harder, but the correctness requirement is admission through the appropriate semantic boundary rather than use of a particular host- language type.

Provisional state cannot confer authority

An adversarial browser may fabricate arbitrary provisional state.

The trusted server must treat such state only as untrusted protocol input or local presentation.

It may not accept:

- provisional role labels;
- provisional ”authorized” flags;
- claimed server-machine state;
- claimed authoritative provenance;
- claimed principal identity;

as authorization to execute a model transition.

Optimism improves latency and experience. It does not enlarge the principal's authority.

Every provisional trajectory requires resolution semantics

A permitted optimistic branch must specify how provisional ownership can end.

Resolution classes may include:

- confirmed by authority;
- reconciled with a different authoritative result;
- rejected by authority;
- superseded by a newer authoritative projection;
- timed out into a defined recovery path;
- retired because the plan/execution became incompatible.

The exact set is operation-specific, but ”remain provisional forever with no recovery path” is not a valid terminal design. Providing a recovery path is not itself proof that the path eventually executes. The relevant environment and application-action assumptions remain part of any progress claim.

Retiring local provisional ownership is not the same as determining the authoritative command disposition. A timeout, incompatible-plan retirement, or other local recovery boundary may end an execution while the participant still has an unresolved command outcome.

Conceptually:

```
local execution E42 retires
    -> provisional ownership ends
    -> recovery/reacquisition may begin
    ≠ command C42 proved rejected
```

A declared recovery state may therefore be terminal for one local execution without being a semantic statement that authority rejected the command.

Conditional liveness claims may depend on environment assumptions such as eventual connectivity or authoritative service availability. Those assumptions must remain visible.

Settlement and authoritative observation are complementary

A direct command settlement may tell the initiating browser how authority resolved its command.

A later authoritative reread may independently supersede the provisional state.

Such a reread establishes only what its authorized acquisition contract actually justifies. It may establish current authoritative state strongly enough to replace the provisional presentation while leaving the disposition or causal attribution of the original command unresolved.

Recovery of authoritative presentation must therefore not depend exclusively on receiving the original settlement. If the application also requires command-specific attribution, it needs a declared path to that evidence under its separate availability and retention assumptions.

For example:

```
client sends C42
    |
server commits authoritative result
    |
settlement response is lost
    |
permitted acquisition trigger or later reload causes authorized reread
    |
authoritative projection supersedes provisional C42
```

This is essential for unreliable networks and sleeping browsers.

If the application requires evidence that C42 itself committed, that conclusion must come from a command-specific settlement, durable command/result relation, idempotency contract, or another model-declared source of attribution. Merely observing the expected current state is insufficient where another cause could produce the same state.

Authoritative supersession prevents obsolete restoration

Once a logical target has installed an admitted authoritative observation that supersedes provisional ownership for the relevant scope, cleanup from that obsolete provisional execution must never restore a representation that violates the new observation or its ownership merely because a failure callback arrives late.

”Newer” is defined by the applicable scoped observation and installation contract, not wall-clock arrival or a maximum opaque database token. A narrow observation cannot erase unrelated provisional or advisory state without the required coverage and ownership justification. This is a prohibition on obsolete restoration, not a claim that all observations form one total order or that domain state can only grow. Any stronger monotonicity claim must identify the ordering and scope over which it holds.

The browser realization and proof obligations belong to Part VI and the scoped installation contract. The semantic rule is:

> Authoritative installation outranks obsolete provisional ownership.

Rollback is therefore conditional.

An old optimistic snapshot may be restored only while doing so remains valid relative to current ownership and authority. It is not a universal ”request failed, restore old DOM” rule. Restoring an earlier justified presentation does not assert that the command failed or that authority remains unchanged. Pending recovery, staleness, and unknown command outcome must retain their semantic classification even if the local presentation resembles its earlier appearance.

Multiple provisional views do not fork authority

Two browsers may optimistically predict incompatible outcomes from the same authoritative basis.

For example:

```
authoritative:
    Request R unclaimed

Alice:
    provisional claimed-by-Alice

Bob:
    provisional claimed-by-Bob
```

This creates two provisional knowledge states, not two authoritative histories.

The model's authoritative operation determines the semantic outcomes.

This distinction is one reason optimism belongs in Choreo/runtime semantics rather than being implemented as untracked DOM mutation.

Optimistic resource ownership is explicit

An optimistic execution may own resources such as:

- target ownership/lock;
- settlement timer;
- request correlation;
- provisional DOM projection;
- pending label/state;
- cleanup action;
- continuity transaction associated with replacement.

Those resources belong to the execution and must terminate or be semantically retired on every terminal path.

The detailed ownership transition system is specified in Part VI and follows owner-relative admission. A later outcome may update an admitted command-resolution owner without reviving a retired optimistic execution.

No silent downgrade of a semantically optimistic operation

If an affordance declares an operation whose semantics require:

- a projected optimistic trajectory;
- command identity;
- authoritative basis;
- settlement/progression behavior;

then the runtime or rendering layer must not silently emit a weaker ordinary HTMX request when any of those required semantic pieces are absent.

Forbidden:

```
missing authoritative basis
    -> emit naked hx-post anyway

missing optimistic command realization
    -> perform ordinary request anyway

missing projected plan
    -> guess transport and hope
```

An enabled realization must be rejected at the earliest boundary that knows it is incomplete. A view may instead represent explicitly unavailable operation identity under the declared rendering contract. That unavailable state is not a successful executable realization and does not satisfy a milestone requiring the capability to work in an eligible scenario.

A separately declared non-optimistic realization may be selected only when the authored operation contract permits it and its own realization is complete. Missing optimism metadata is not itself permission to make that selection.

This fail-closed rule does not make raw HTMX illegal. Ordinary non-Choreo interactions may use ordinary HTMX. What is forbidden is losing semantic identity or required semantics without an explicit change in the authored operation.

Optimistic rendering is not domain policy

The model remains responsible for:

- whether the command is legal;
- how conflicts resolve;
- what authoritative state results;
- whether already-incorporated commands succeed;
- what reconciliation means.

Choreo/runtime decides:

- whether provisional rendering is permitted;
- what provisional projection is associated with the command;
- how provisional ownership is tracked;
- how settlement/supersession terminates it.

Views decide how the provisional state looks.

These responsibilities must not collapse into one browser helper.

Continuity

Continuity is deliberately **not** part of choreographic truth.

Continuity preserves local browser interaction state across representation replacement.

Examples include:

- focus;
- input selection;
- scroll position;
- open/closed UI details;
- other local DOM interaction state.

Continuity belongs primarily in ClojureScript/browser code.

Conceptually:

```
choreographic semantic effect
        |
        | install authoritative projection
        v
browser effect adapter
        |
        +-- capture continuity
        +-- install/replace DOM
        +-- restore continuity
        +-- release continuity resources
        |
        v
effect complete
```

Portable choreography should not need semantic events such as `:continuity/restored`.

From Choreo's perspective, the browser effect either completes or fails.

Continuity may still be a managed browser resource owned by an execution, fragment replacement, or another declared browser effect scope. The runtime must manage that ownership correctly. Ordinary continuity does not establish domain facts, knowledge, authority, or agreement, and need not invent a command execution for an otherwise legitimate fragment replacement.

A continuity bug may harm local browser experience. It must not be able to change the semantic meaning of a Request operation.

Continuity identity must distinguish the current slot/resource instance rather than relying only on logical target id. It follows the resource instance identity contract; a scoped resource-instance counter is one possible representation. A delayed restore from an older replacement must not consume or mutate continuity state of a newer replacement.

Consider a HumanHelp board where Bob has:

- typed `cabinet` into the live search field;
- expanded Request R17;
- placed keyboard focus on the Refresh button;
- scrolled the board midway down.

An authoritative board reacquisition replaces the board fragment. The semantic result and local interaction continuity can be represented separately:

```
(def authoritative-board
  ;; Illustration of the value inside an admitted board observation.
  {:projection/type :request/board
   :authority :authoritative
   :basis :X31
   :requests [{:request/id R12}
              {:request/id R17}
              {:request/id R22}
              {:request/id R30}]})

(def admitted-board
  ;; Fixture notation: issued only by authorized, trusted acquisition.
  (acquisition/admitted
    authoritative-board
    {:scope {:projection :request/board}
     :authorized-principal [:user Bob]
     :covered-through :X31}))

(def continuity-capture
  {:slot :request-board

   :capture-instance/id
   #uuid "a1000000-0000-0000-0000-000000000001"

   :owner
   {:replacement/id
    #uuid "a2000000-0000-0000-0000-000000000001"}

   :local
   {:focus
    {:kind :element-key
     :key :board/refresh}

    :scroll
    {:top 842}

    :inputs
    {:board/search "cabinet"
     :board/options
     {:sort :newest
      :visibility :available}}

    :details
    {:open-request-ids #{R17}}}})
```

None of `:board/search`, scroll offset 842, or ”R17 details open” becomes a domain fact merely because the DOM from which it was captured was authoritative.

The replacement adapter can treat continuity as a local effect transaction:

```
(defn replace-board!
  [browser
   {:keys [observation replacement-id]}]

  (acquisition/require-admitted-for-current-target!
    browser :request-board observation)
  (let [capture
        (continuity/capture
          browser
          {:slot :request-board
           :owner
           {:replacement/id replacement-id}})

        install
        (dom/install-authoritative-if-current!
          browser
          {:target :request-board
           :observation observation})]

    (if (= :installed (:status install))
      {:semantic
       {:status :authoritative-installed
        :observation observation}

       :continuity
       (continuity/restore-if-current!
         browser
         {:capture capture
          :slot :request-board
          :installed-observation observation})}

      (do
        (continuity/retire-capture-if-current! browser capture)
        {:semantic
         {:status :installation-failed
          :observation observation}
         :continuity
         {:status :not-attempted}})))))
```

Suppose restore succeeds:

```
{:semantic
 {:status :authoritative-installed
  :observation admitted-board}

 :continuity
 {:status :restored
  :restored
  #{:focus :scroll :inputs :details}}}
```

The new DOM can show authoritative Request membership from X31 while preserving Bob's search text and open details locally. The install operation itself compares current ownership and scoped progression at its mutation boundary. The earlier admission check cannot substitute for that check if another replacement can intervene.

Now suppose focus restoration fails because the Refresh button is absent from the newly authorized rendering:

```
{:semantic
 {:status :authoritative-installed
  :observation admitted-board}

 :continuity
 {:status :partial
  :restored #{:scroll :inputs :details}
  :failed
  {:focus
   {:reason :element-no-longer-present
    :key :board/refresh}}}}
```

That is not an authoritative transition failure. The application must not respond by reinstalling the old board merely to get the old Refresh button back.

The race case is more interesting. Replacement O1 captures continuity instance I1. Before I1 restores, a second authoritative replacement O2 captures I2:

```
(def I1
  {:slot :request-board
   :capture-instance/id
   #uuid "a1000000-0000-0000-0000-000000000011"
   :owner
   {:replacement/id
    #uuid "a2000000-0000-0000-0000-000000000011"}
   :local {:board/search "cabinet"}})

(def I2
  {:slot :request-board
   :capture-instance/id
   #uuid "a1000000-0000-0000-0000-000000000012"
   :owner
   {:replacement/id
    #uuid "a2000000-0000-0000-0000-000000000012"}
   :local {:board/search "cabinet hinge"}})

(def continuity-runtime
  {:current
   {:slot :request-board
    :capture-instance/id
    (:capture-instance/id I2)
    :owner (:owner I2)}})
```

A delayed callback for I1 must be rejected:

```
(defn current-continuity-capture?
  [runtime capture]
  (and
    (some? (get-in runtime [:current :capture-instance/id]))
    (some? (:capture-instance/id capture))
    (some? (get-in runtime [:current :owner]))
    (some? (:owner capture))
    (= (get-in runtime [:current :slot])
       (:slot capture))
    (=
      (get-in runtime
        [:current :capture-instance/id])
      (:capture-instance/id capture))

    (=
      (get-in runtime
        [:current :owner])
      (:owner capture))))

(current-continuity-capture?
  continuity-runtime
  I1)
;; ⇒ false

(current-continuity-capture?
  continuity-runtime
  I2)
;; ⇒ true
```

The important state is not ”which capture number is numerically larger?” It is which concrete capture is currently owned by the replacement scope that may restore this continuity slot. The adapter checks its own current slot registry immediately before restoration rather than trusting a callback's owner claim. It retires the capture after restoration or failed installation.

A continuity implementation may also filter captured values against the new authoritative rendering:

```
(defn admissible-details
  [observation captured-details]
  (let [authorized-ids
        (set
          (map :request/id
               (:requests (acquisition/value observation))))]

    {:open-request-ids
     (set/intersection
       authorized-ids
       (:open-request-ids captured-details))}))
```

If Request R17 is no longer part of Bob's authorized board projection, continuity must not recreate its DOM or disclosure merely because R17 was previously open. Local UI preservation is subordinate to the semantics of the new projection.

This example shows why continuity deserves careful ownership despite not being choreographic truth. It can race, leak stale local state, or damage user experience, but it never becomes an authority channel.

Continuity is local preservation, not provenance

Focus, selection, scroll position, accordion openness, and similar values may be important to user experience.

They are not authoritative facts about the application domain.

A continuity value therefore cannot justify:

- model authorization;
- choreography branch knowledge;
- authoritative provenance;
- command validity;
- agreement.

This remains true even when continuity data was captured from DOM produced by an authoritative projection.

Continuity resources have declared owner and instance identity

Although continuity is not choreographic truth, its implementation can still race with semantic replacement.

A continuity capture/restore transaction must therefore carry sufficient identity to ensure that an old restore cannot mutate the local state of a newer replacement.

Conceptually:

```
logical target T, logical continuity slot S
    capture instance I1 owned by replacement scope O1
    capture instance I2 owned by replacement scope O2

late restore callback for I1
    must not consume/overwrite I2's captured state
```

The browser adapter's owner-relative isolation rules in Part VI govern this safety. A continuity owner may refer to an execution when one exists, but the admission test must use the declared owner rather than require an unrelated execution. Slot S names the logical capture location; I1 and I2 distinguish its concrete captures. O1 and O2 identify the declared replacement/effect owners. Their representation need not use ordered counters or separate redundant tokens when the ownership contract can preserve these distinctions with fewer values.

Failure of continuity must not corrupt semantic settlement

If authoritative DOM installation succeeds but continuity restoration fails, the application has a local interaction-quality failure.

It must not reinterpret that event as:

```
authoritative transition failed
```

nor may it restore a stale provisional/authoritative representation merely to recover focus.

Semantic truth outranks continuity convenience.

The effect result must distinguish authoritative installation status from optional continuity follow-up status. A generic failed effect must not conceal that authoritative installation already succeeded and thereby trigger stale rollback. If physical installation itself fails after semantic admission, the adapter records the admitted observation and incomplete realization according to its explicit recovery contract; it does not pretend the DOM necessarily matches the semantic state.

Continuity cleanup is best effort after semantic retirement

Once a continuity resource instance or its declared execution, fragment- replacement, or effect-scope owner has been retired, late physical cleanup may still occur. Retirement revokes the corresponding instance's restoration rights; it need not imply that every resource of the same execution was replaced.

Cleanup failure becomes diagnostic/runtime debt.

It must not preserve semantic authority for the retired resource.

This mirrors the general resource rule developed in Part VI:

> Physical resource lifetime may outlast semantic ownership; stale physical callbacks remain powerless after semantic retirement.

Portable Choreo observes only effect completion/failure

The portable choreography should not branch on browser-internal details such as:

```
focus restored
selection restored
scroll restored
MutationObserver completed
```

unless an application deliberately promotes some browser event into semantic meaning through a separately designed contract.

Ordinary continuity remains behind the browser-effect boundary.

From Choreo's point of view, the declared browser realization reports the outcome defined by its effect contract. Continuity failure must not be mapped to authoritative rejection or erase a completed installation. Browser-local diagnostics may describe the failed focus or selection restoration without making those details choreography branch discriminators.

Views and UI annotation

Views should be able to use choreographic operations/types exported by models to annotate UI affordances.

Views should not redefine the choreography.

A model may export:

`request.choreo/claim`

A preparation layer may bind that operation to a concrete principal, request, and authoritative basis, producing something capability-like:

```
(choreo/bind request.choreo/claim
  {:role :helper

   ;; This value comes from trusted preparation context.  A browser-supplied
   ;; user id may be carried as data, but it cannot establish this principal.
   :principal
   (trusted-context/authenticated-principal request-context)

   :args {:request-id request-id}
   :observed-basis observed-basis})
```

A view may then attach that bound operation to Hiccup:

```
(live/button
  {::choreo/op claim
   :class "gesso-action gesso-action--primary"
   :pending-label "Claiming..."}
  "Claim")
```

or, for an unbound operation:

```
[:button
 {::choreo/op request.choreo/claim
  ::choreo/role :helper
  :gesso/args {:request-id request-id}
  :class "gesso-action gesso-action--primary"}
 "Claim"]
```

The exact UI API remains tentative. These are alternative illustrative authoring forms, not a requirement to maintain several independent binding APIs. The supported ordinary API should normalize them, if more than one is offered, to the same admitted binding contract.

In the bound example, `:principal` records trusted preparation context or a claimed client context according to the boundary where binding occurs. A value supplied by the browser cannot establish authenticated principal identity. The server derives that identity independently and checks the declared binding.

The source-level meaning is:

> This affordance invokes Request.claim as the Helper role.

HTTP routes, HTMX attributes, command IDs, basis propagation, correlation metadata, and other transport details are realization details Gesso may derive.

The view owns presentation concerns such as:

- element choice;
- text;
- semantic CSS classes;
- pending labels;
- ARIA;
- layout;
- visual rendering of provisional/canonical states.

The view should not own:

- whether the domain operation is legal;
- Request transition semantics;
- membership/organization policy;
- stale-basis interpretation;
- authoritative persistence;
- choreography branch semantics;
- DOM continuity machinery.

Ideally, view preparation supplies only available typed operation capabilities.

Instead of:

```
(when (request/can-claim? request user)
  ...)
```

a view may receive:

```
{:actions
 {:claim bound-claim-operation
  :take-over nil}}
```

and render only the capabilities it was given.

This avoids reimplementing domain authorization/business policy in views. An absent action in such a prepared map may represent legitimate policy-based unavailability. It must not conceal a missing operation declaration, plan, or required binding. The preparation and assembly boundaries must distinguish those cases; independent capability acceptance detects intended actions omitted from the product.

A rendered capability is not durable authorization.

The authoritative server/model operation must re-establish authenticated principal identity and current authorization when the command is executed.

A prepared Request-card view can keep business-policy decisions out of Hiccup while still giving presentation code meaningful choices.

Suppose a preparation layer receives current authoritative model values and asks the semantic application layer which actions are available:

```
(defn prepare-request-card
  [{:keys [principal] :as ctx}
   request]

  (let [claim
        (request.choreo/prepare
          ctx
          :request/claim
          {:request request
           :principal principal})

        cancel
        (request.choreo/prepare
          ctx
          :request/cancel
          {:request request
           :principal principal})

        reassign
        (request.choreo/prepare
          ctx
          :request/reassign
          {:request request
           :principal principal})]

    {:request
     {:id (:request/id request)
      :title (:request/title request)
      :status (:request/status request)}

     :actions
     {:claim claim
      :cancel cancel
      :reassign reassign}}))
```

The prepared values can distinguish policy-based absence from a construction defect:

```
{:claim
 {:availability :available
  :binding
  {:operation :request/claim
   :role :helper
   :args {:request/id R}
   :observation
   {:scope {:projection :request/card
            :request/id R}
    :basis :X41}}}

 :cancel
 {:availability :unavailable
  :reason :request/not-owned-by-principal}

 :reassign
 {:availability :unavailable
  :reason :application/missing-required-target
  :diagnostic
  {:missing #{:helper/id}
   :operation :request/reassign}}}
```

The first is an executable affordance. The second is legitimate product unavailability. The third indicates that application preparation tried to expose an operation but failed to supply required semantic input.

A view can render those cases without knowing the underlying business rule:

```
(defn request-card-view
  [{:keys [request actions]}]

  [:article.request-card
   [:h3 (:title request)]
   [:p.status
    (name (:status request))]

   (case (action/require-prepared-availability!
          :request/claim (:claim actions))
     :available
     (gesso/action
       {:binding
        (get-in actions [:claim :binding])
        :pending-label "Claiming..."
        :class "gesso-action gesso-action--primary"}
       "Claim")

     :unavailable
     nil)

   (case (action/require-prepared-availability!
          :request/cancel (:cancel actions))
     :available
     (gesso/action
       {:binding
        (get-in actions [:cancel :binding])}
       "Cancel")

     :unavailable
     nil)

   (case (action/require-prepared-availability!
          :request/reassign (:reassign actions))
     :available
     (gesso/action
       {:binding
        (get-in actions [:reassign :binding])}
       "Reassign")

     :unavailable
     nil)])
```

Here `action/require-prepared-availability!` accepts only a validated `:available` binding or a declared policy-based `:unavailable` reason. It throws for missing/unknown availability, missing required input (including Reassign's `:helper/id` above), or a plan/binding construction defect for **every** action. A broken intended action cannot silently disappear.

The view never asks:

```
(and
  (= :open (:request/status request))
  (= :helper (:membership/role current-user))
  (:organization/claiming-enabled? organization)
  (not (:request/helper-id request)))
```

because doing so would create a second claim-policy implementation in presentation code.

The prepared binding can contain the information needed to derive realization metadata:

```
(def claim-binding
  {:operation :request/claim
   :role :helper
   :args {:request/id R}
   :observed-basis-claim :X41

   :provisional
   {:projection :request/claiming}

   :realization
   {:plan :plan/request-claim
    :compatibility :humanhelp-live-v12}})
```

Rendering may turn that into HTMX or another browser mechanism, but the semantic identity must remain visible through the process:

```
(render-action claim-binding)
;; ⇒
[:button
 {:class "gesso-action gesso-action--primary"
  :hx-post "/gesso/op/request-claim"
  :hx-swap "none"
  :data-gesso-operation "request/claim"
  :data-gesso-plan "plan/request-claim"
  :data-gesso-binding "...opaque encoded binding..."
  :aria-busy "false"}
 "Claim"]
```

The exact attributes are illustrative. The important point is that they are derived from or checked against `claim-binding` rather than independently authored strings.

If required realization data is missing, a managed action fails closed:

```
(render-action
  (dissoc-in
    claim-binding
    [:realization :plan]))

;; ⇒
{:error/type :view/incomplete-semantic-realization
 :operation :request/claim
 :missing #{:projected-plan}
 :fallback-rendered? false}
```

It must not silently produce:

```
[:button
 {:hx-post "/requests/claim"}
 "Claim"]
```

because that would erase the semantic operation while leaving an apparently functional mutation path.

The view remains free to decide whether provisional status appears as `"Claiming..."`, a spinner, reduced opacity, or another accessible treatment. The semantic layer says **what state this is**; the view says **how that state looks**.

Views declare semantic intent, not transport machinery

The ordinary view-level declaration should be as small as practical.

Conceptually:

```
{:choreo/op :request/claim
 :subject request-id}
```

or a prepared/bound equivalent is preferable to requiring the view author to repeat:

- projected plan registry key;
- bootstrap registry key;
- protocol envelope encoding;
- artifact stamp/digest;
- trusted server authority identity;
- command-wire parameter names;
- basis-header encoding;
- runtime participant override.

Those details should be derived from or verified against the assembled application whenever possible.

The exact public API remains a design choice. The semantic target does not.

Bound operation capability is not a durable authorization token

The word `capability` in this section describes a prepared operation affordance, not an unforgeable permission-bearing security capability. Implementations and diagnostics should qualify the term where ambiguity would otherwise arise.

A bound operation may carry useful presentation/execution context such as:

- semantic operation;
- role;
- subject/arguments;
- the authoritative observation context from which the affordance was prepared;
- provisional projection information;
- plan/compatibility identity as required by runtime.

Once observation information crosses an adversarial browser boundary and is returned by the client, an unauthenticated basis value is an `ObservedBasisClaim`, not an `AuthoritativeBasis`. If the operation requires server-verifiable evidence of prior issuance/observation, that requirement must be represented explicitly, for example by an authenticated observation receipt or another declared trusted mechanism.

Neither an observed-basis claim nor an authenticated historical receipt establishes current authorization to execute the operation.

It does not allow the browser to establish:

- authenticated principal identity;
- current model permission;
- server role authority;
- authoritative state.

The trusted server rebinds the authenticated principal from trusted context and re-evaluates current model policy.

A bound capability is therefore better understood as a typed, prepared affordance than as an irrevocable security capability.

This is another place where a nominal or opaque internal type may improve accidental-misuse resistance, but the public application API should not require downstream web developers to manually construct proof-like wrappers merely to render an ordinary action. Gesso should construct or admit stronger semantic values at the boundary that has enough information to justify them.

Capability visibility should come from prepared application semantics

Where practical, view preparation should compute which operation capabilities are available and pass them to rendering.

Preferred:

```
{:actions
 {:claim claim-capability
  :cancel cancel-capability
  :reassign nil}}
```

rather than duplicating domain checks inside Hiccup:

```
(when (and (request/open? request)
           (membership/helper? user)
           ...)
  ...)
```

This does not mean every dynamic authorization decision must be precomputed forever.

It means the view should consume prepared semantic availability rather than becoming a second independent business-rule engine.

The authoritative operation still rechecks current permission at execution.

Semantic operation identity is preserved through rendering

Once a view declares that an affordance is semantically managed as operation O, later rendering stages may add realization metadata but may not silently erase or substitute that identity. Preservation is not an ordering over operation identities.

Conceptually:

```
semantic affordance O
    -> bind O
    -> derive plan/transport metadata for O
    -> render executable O
```

Forbidden:

```
semantic affordance O
    -> required metadata missing
    -> silently render ordinary unrelated hx-post
```

If an enabled complete semantic realization cannot be produced, rendering or application preflight must fail precisely. An explicitly unavailable rendering may preserve the operation and its reason for unavailability under the declared contract. It must have no active weaker transport path and cannot count as usable capability completion.

This is the fundamental no-silent-downgrade invariant.

Raw HTMX remains a first-class ordinary mechanism

Not every interaction belongs to Choreo.

Applications may still use ordinary HTMX for interactions that deliberately do not claim Choreo/optimistic/progression semantics.

The distinction must be explicit:

```
ordinary HTMX affordance
    -> ordinary HTMX semantics

Choreo-managed affordance
    -> complete Choreo realization
       or explicit non-executable unavailability
       or precise rejection
```

Strictness applies to semantic identity once claimed. It does not require turning every button or form into a distributed protocol. An ordinary HTMX binding must not occupy a route reserved by the assembly for a semantic operation as a means of bypassing that operation's required protocol.

Presentation remains view-owned

Views retain ordinary presentation authority over:

- element type;
- text;
- CSS classes;
- pending labels;
- ARIA;
- layout;
- icons;
- local visual treatment;
- how provisional versus authoritative status is displayed.

These choices may be semantically informed but do not become domain policy.

For example, Choreo may establish:

```
projection authority = provisional
operation = :request/claim
```

while the view decides whether that appears as:

- a badge;
- pending text;
- reduced opacity;
- a spinner;
- another accessible presentation.

Views do not interpret stale basis or reconciliation policy

A view may display the outcome of reconciliation.

It must not decide what stale basis means.

Forbidden architecture:

```
if browser basis < newest basis
    then view decides Claim is rejected
```

Correct architecture:

```
model evaluates current authoritative facts
    -> semantic outcome
    -> Choreo/runtime propagates outcome
    -> view renders outcome
```

This prevents database/progression semantics from leaking into presentation code.

UI operation binding creates later whole-application obligations

Once a view exposes a Choreo-managed operation, the application acquires structural obligations that later preflight must close.

Before the enabled affordance is admitted for serving, the assembled application must establish the required path, analogous to:

```
rendered affordance
    -> semantic operation
    -> valid choreography trajectory
    -> projected executable plan
    -> exact browser artifact
    -> declared transport
    -> trusted server capability
    -> authoritative model operation
    -> settlement/progression
    -> authoritative acquisition path
    -> renderable result
```

Part VII defines that whole-application check, including dynamic rendered surfaces. This part defines the binding contract that creates those obligations; their later position in the document does not permit delayed validation after an invalid enabled control has already been served.

Server execution capabilities are separate from view authority

Some operations require trusted execution inputs/resources such as:

- authenticated principal;
- transaction capability;
- authoritative basis;
- clock;
- random seed;
- other declared server capabilities.

A view binding does not supply those capabilities.

Later application assembly must verify that the trusted server boundary can supply the capabilities required by the operation.

This is especially important because an operation can be semantically valid yet unrealizable in a particular assembled server environment.

Capability declaration is not authority declaration

A statement such as:

```
server supplies :clock
server supplies :transaction
```

means that the execution environment can provide those resources.

It does not mean the server principal is automatically authorized to perform a particular domain operation.

Execution capability and domain authority remain separate concepts.

UI diagnostics should preserve semantic context

When a managed affordance cannot be realized, diagnostics should be able to identify:

- source view/component;
- semantic operation;
- role/subject where known;
- missing semantic or realization edge;
- expected plan/transport/capability;
- available alternatives where mechanically known.

A missing plan should not degrade into a generic malformed-attribute error.

The rendering/preflight system knows the semantic intent and should preserve it for repair tooling.

Provisional state and view boundary invariant

At the end of the authority/progression/UI layer, the unified design requires:

> Authoritative business truth changes only through model-owned atomic transitions. Biff FX realizes those transitions using explicit value flow rather than hidden context accumulation. XTDB progression records authoritative visibility rather than domain revision. Commands, executions, plans, and physical requests retain distinct identities; command identity does not itself provide idempotency, and participant uncertainty about a command is not silently converted into rejection or incorporation. Current authoritative state may resolve presentation without proving command attribution. Optimistic state is explicit provisional knowledge with required resolution paths, and local provisional retirement does not invent an authoritative command outcome. Continuity is local browser state with no authority. Views declare typed semantic affordances without becoming business-policy or authorization engines, and a Choreo-managed affordance may never silently downgrade to weaker ordinary transport semantics.

Clojure and ClojureScript execution

The portable choreography representation and transition semantics form one shared executable semantic core.

Where the implementation can express that core portably, it lives in shared Clojure/ClojureScript code rather than in separate JVM and browser interpretations.

The responsibility boundary is semantic rather than namespace-name specific.

Clojure owns or hosts the trusted/server-side work needed for:

- global choreography construction and verification;
- model integration;
- authoritative model operations;
- XTDB consistency and transaction work;
- server-side projections where required;
- JVM execution of the portable executable semantic core;
- compiler/build-time verification and projection.

ClojureScript owns or hosts the browser-side realization needed for:

- execution of browser role projections;
- the same portable machine transition semantics;
- browser effect interpretation;
- semantic provisional projections;
- HTMX lifecycle integration;
- normalized browser event adaptation;
- managed browser effects;
- DOM realization;
- continuity.

This division does not authorize two semantic machines.

The same portable transition semantics must run under:

- JVM Clojure;
- Node ClojureScript;
- Chromium ClojureScript.

A common portable machine corpus must execute across all three environments to detect representation or compiler-target differences. Its results remain tested evidence for the exercised cases; sharing source and passing the corpus do not by themselves prove equivalence of every compiler/runtime behavior.

The portable data contract must define relevant numeric, equality, collection, serialization, and error semantics for values used in transitions. A platform-specific representation must not silently change branch selection, identity comparison, or requirement satisfaction. Any relied-upon compiler or runtime behavior outside the checked core remains an explicit trusted boundary.

Verification and projection machinery are build/construction concerns.

Once a validated compact projected plan has been emitted, the production browser must not need the global verifier, proof DAG, or choreography compiler in order to execute that plan.

The browser executes the accepted projection.

The same semantic transition core used by production code is the object of formal reasoning and model checking.

A small projected Claim machine makes the desired portability more concrete.

The immutable plan can be ordinary portable data:

```
(def executable-plan
  {:plan/version 3
   :plan/id :plan/request-claim
   :compatibility/id :humanhelp-live-v12
   :operation :request/claim

   :states
   [{:state :idle
     :transitions
```

```
     [{:event :ui/invoke
       :guard [:binding/complete]
       :next 1
       :effects
       [[:command/allocate]
        [:projection/install-provisional]
        [:transport/start]]}]}

    {:state :pending
     :transitions
     [{:event :claim/settled
       :guard
       [:and
        [:event/current-owner]
        [:event/matches-command]
        [:settlement/admitted-current-outcome]
        [:observation/admitted-progressive]]
       :next 2
       :effects
       [[:command/record-accepted]
        [:projection/install-authoritative]
        [:resources/retire-owned]]}

      {:event :claim/timeout
       :guard [:event/current-resource-owner]
       :next 3
       :effects
       [[:command/record-unknown]
        [:resources/retire-owned]
        [:recovery/request-authoritative-acquisition]]}

      {:event :claim/rejected
       :guard
       [:and
        [:event/current-owner]
        [:event/matches-command]
        [:settlement/admitted-current-outcome]
        [:observation/admitted-progressive]]
       :next 5
       :effects
       [[:command/record-rejected]
        [:projection/install-authoritative]
        [:resources/retire-owned]]}]}

    {:state :settled
     :transitions []}

    {:state :unknown
     :transitions
    [{:event :authority/acquired
       :guard
       [:and
        [:event/current-recovery-target-owner]
        [:observation/admitted-authorized-current-progressive]]
       :next 4
       :effects
       [[:projection/install-authoritative]]}]}

    {:state :recovered
```

```
     :transitions []}

    {:state :rejected
     :transitions []}]})
```

Runtime state is separate from the plan:

```
(def runtime-state
  {:machine/state :pending

   :execution/id
   #uuid "b1000000-0000-0000-0000-000000000010"

   :command/id
   #uuid "b2000000-0000-0000-0000-000000000042"

   :resources
   {:transport
    {:resource-instance/id
     #uuid "b3000000-0000-0000-0000-000000000006"
     :owner [:execution
             #uuid "b1000000-0000-0000-0000-000000000010"]
     :status :active}

    :timeout
    {:resource-instance/id
     #uuid "b3000000-0000-0000-0000-000000000007"
     :owner [:execution
             #uuid "b1000000-0000-0000-0000-000000000010"]
     :status :active}}

   :authoritative-basis
   {:scope {:projection :request/card :request/id R}
    :basis :X49}

   :projection
   {:authority :provisional
    :request/id R
    :request/status :claiming}})
```

A normalized event likewise contains runtime-specific values:

```
(def settled-event
  {:event/type :claim/settled

   :execution/id
   #uuid "b1000000-0000-0000-0000-000000000010"

   :command/id
   #uuid "b2000000-0000-0000-0000-000000000042"

   :resource-instance/id
   #uuid "b3000000-0000-0000-0000-000000000006"

   ;; Supplied by the trusted settlement admission boundary. The map by
   ;; itself would not satisfy :settlement/admitted-current-outcome.
   :settlement
   (settlement/admitted-fixture
     {:execution/id
      #uuid "b1000000-0000-0000-0000-000000000010"
      :command/id
      #uuid "b2000000-0000-0000-0000-000000000042"
      :resource-instance/id
      #uuid "b3000000-0000-0000-0000-000000000006"
      :outcome :accepted
      :scope {:projection :request/card :request/id R}
      :authoritative
      {:request/id R
       :request/status :claimed
       :request/helper-id Alice
       :basis :X50}})})
```

The portable transition function can remain pure:

```clojure
(defn step
  [plan state event]
  (let [state-index
        (plan/state-index plan (:machine/state state))
        candidates
        (get-in plan [:states state-index :transitions])
        transition
        (some #(when (= (:event %) (:event/type event)) %)
              candidates)]

    (cond
      (nil? transition)
      {:result :rejected
       :reason :event-not-admitted-in-state
       :state state
       :effects []}

      (not
        (guards/holds?
          (:guard transition)
          {:state state
           :event event}))
      {:result :rejected
       :reason :guard-failed
       :state state
       :effects []}

      :else
      (let [next-state
            (get-in plan [:states (:next transition) :state])
            elaborated
            (effects/elaborate (:effects transition)
              {:state state :event event})]
        ;; The *same pure core* updates semantic state and returns physical
        ;; requests. The browser adapter cannot invent a competing state.
        {:result :accepted
         :state (effects/apply-semantic
                  (assoc state :machine/state next-state)
                  elaborated)
         :effects (effects/physical-requests elaborated)})))))
```

Running the same input under JVM Clojure and ClojureScript should produce the same semantic result:

```
(step
  executable-plan
  runtime-state
  settled-event)
;; ⇒
{:result :accepted

 :state
 {:machine/state :settled
  :execution/id ...
  :command/id ...
  :resources {:transport {:status :retired} :timeout {:status :retired}}
  :projection {:authority :authoritative
               :request/id R
               :request/status :claimed
               :request/helper-id Alice
               :basis :X50}}

 :effects
 [[:projection/install-authoritative
   {:request/id R
    :request/status :claimed
    :request/helper-id Alice
    :basis :X50}]

  [:resources/retire-owned
   {:execution/id
    #uuid "b1000000-0000-0000-0000-000000000010"}]]}
```

The notation `settlement/admitted-fixture` represents a value supplied only by a trusted admission boundary, not a constructor exposed to the browser. `guards/holds?` checks the current execution/resource and admitted outcome; `effects/elaborate` extracts only the authorized projection from that admitted settlement. `effects/apply-semantic` is pure: it records the authoritative projection, accepted command disposition, and retired resource ownership. `:claim/timeout` records unknown disposition, and an admitted reread can recover presentation while leaving command attribution unknown. `:event/current-recovery-target-owner` refers to the adapter's recovery target owner after the timed-out execution retires; it never resurrects that execution. Acquisition events still require their own resource-instance correlation and current scoped installation admission. `:claim/rejected` requires its own admitted settlement; observing the current Request state does not by itself establish why the command failed. `effects/physical-requests` returns adapter instructions separately. An installation failure must enter the declared physical-effect recovery contract; the shell cannot silently reverse the semantic admission or pretend the DOM was installed. The abbreviated displayed result omits unrelated state fields.

The browser shell interprets abstract effects:

```
(defn interpret-effect!
  [browser effect]
  (case (first effect)

    :projection/install-authoritative
    (dom/install-if-current!
      browser
      (second effect))

    :projection/install-provisional
    (dom/install-provisional!
      browser
      (second effect))

    :transport/start
    (transport/start!
      browser
      (second effect))

    :resources/retire-owned
    (resources/retire!
      browser
      (second effect))

    :recovery/request-authoritative-acquisition
    (acquisition/request!
      browser
      (second effect))))
```

The shell may differ across Chromium, Node tests, or another host. The transition meaning should not.

A dangerous architecture would instead duplicate the branch logic:

```
;; JVM "reference semantics"
(defn jvm-step [state event]
  ...)

;; separately handwritten browser behavior
(defn browser-step! [dom event]
  (if (= "claim-settled" (.-type event))
    ;; independently decide what this means here
    ...))
```

Even if the two implementations currently pass the same tests, the proof/test object and production object are now different semantic machines.

The portable core avoids that correspondence gap by making platform-specific code interpret effects rather than redefine the state transition itself.

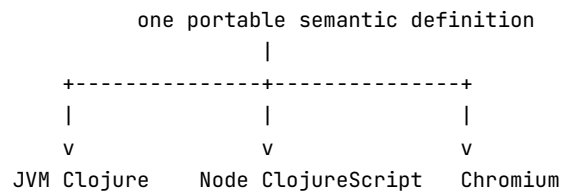
A separately proved paper/reference machine followed by an independently handwritten production semantic adapter is insufficient for the strongest correctness claim in this design.

One portable semantic machine, several execution environments

The JVM, Node, and Chromium implementations must not define separate semantic machines that merely resemble one another.

The portable state-transition meaning belongs in shared Clojure/ClojureScript data and functions wherever practical.

Conceptually:



Differences among those environments belong in effect interpretation and trusted integration boundaries, not in the meaning of a projected transition.

This is important for both testing and proof. A JVM proof or model check over a different ”reference” machine is weaker than reasoning about the transition function production actually uses.

Portable execution owns semantic state, not the browser shell

The portable executable semantic core and browser adapter must have an explicit ownership boundary for semantic state. An execution machine owns its projected trajectory; the adapter owns cross-execution browser coordination where required. Those are composed responsibilities, not competing copies of one state machine. Each semantic fact has one owner, with transitions and effects connecting the layers.

Together their admitted state includes:

- projected machine state;
- execution identity;
- concrete resource-instance identities required by admission;
- semantic command identity where operationally required;
- resource/effect ownership;
- authoritative versus provisional status;
- accepted progression requirements;
- correlation identity;
- terminal/retired status.

The imperative browser shell may own physical handles such as:

- DOM nodes;
- timer handles;
- HTMX request objects;
- EventSource instances;
- listeners;
- MutationObservers;
- continuity snapshots.

The shell must not maintain an independent competing interpretation of semantic ownership.

If a physical handle exists after the semantic execution is retired, the handle is cleanup debt. It is not evidence that the execution remains semantically alive.

Runtime inputs are normalized before semantic interpretation

Raw browser events are not semantic events.

A boundary layer translates raw external observations into a small normalized event vocabulary.

For example:

```
raw HTMX event
raw timer callback
raw EventSource message
raw DOM replacement notification
raw browser lifecycle event
        |
        v
normalize + attach opaque correlation
        |
        v
NormalizedEvent
        |
        v
portable transition core
```

Normalization may copy primitive data and opaque identities that were attached when the resource was created.

It may not infer:

- authority;
- principal identity;
- semantic ownership;
- provenance;
- whether a referenced execution or resource instance remains admitted;
- permission to resume a machine;

from mutable physical state.

Those questions belong to the appropriate declared semantic admission boundary. For browser events, the adapter resolves the owner and checks applicability before resuming an execution or applying a shared transition. Normalization may not choose a weaker ownership class from untrusted event fields.

A normalized event is therefore a representation presented for semantic admission, not evidence that the represented semantic claim is true. This remains so if the host-language value uses a nominal record, opaque wrapper, tagged map, or another category-preserving representation. Such representations may prevent accidental substitution inside cooperating code; they do not allow browser-supplied bytes to manufacture authenticated identity, authoritative provenance, current authority, or an admitted semantic result.

Crossing the browser/server or other untrusted serialization boundary removes any assurance that depended only on in-process construction discipline. The receiving trusted boundary must re-establish the relevant semantic category by the declared validation, authentication, verification, acquisition, or admission path.

Runtime state transition functions are total over the supported event vocabulary

For every supported portable state and normalized event class, semantic behavior should be explicit.

A normalized event may be:

- accepted and advance the machine;
- accepted and leave semantic state unchanged;
- rejected as stale;
- rejected as duplicate;
- converted into a recovery transition;
- classified as an external/runtime fault.

It should not fall through to ad-hoc callback behavior merely because the combination was inconvenient to model.

Where a combination is impossible by construction, that impossibility should be represented as an invariant and mechanically checked where feasible.

Unknown event classes, malformed decoded data, and unsupported plan versions must be classified or rejected at their admission boundary rather than passed to an undefined transition. An unexpected implementation exception is a runtime fault, not proof that the input state/event combination was impossible. Its retirement and recovery behavior is defined by the browser failure contract.

Test execution and production execution use the same plan

Development or proof-enabled tests must not secretly execute a richer client machine than production.

The design requires:

```
production
    ExecutablePlan
    + production event/effect shell

test/debug
    same ExecutablePlan
    + same semantic transition core
    + optional observers/diagnostics
```

This prevents a particularly dangerous class of false confidence in which:

```
proved/test machine
    ≠
shipped machine
```

and correspondence is trusted informally.

Verification does not ship as a browser prerequisite

The browser executes a checked projection.

It does not need to:

- reconstruct the global choreography;
- rerun projection;
- rebuild knowledge derivations;
- load the proof DAG;
- solve proof obligations;
- understand application source namespaces.

This keeps the production client small and keeps proof machinery from becoming a new runtime authority surface.

Runtime rules, executable guards, and references to required runtime inputs belong in the `ExecutablePlan`. Concrete command identities, acquired facts, ownership instances, and other execution-specific values belong in admitted runtime state or inputs governed by that plan. They must not be retrieved from an optional diagnostic sidecar.

Explanations of why the executable rules are sufficient belong outside runtime semantics. This distinction does not require embedding future runtime values in an immutable build artifact.

Portable representation version and semantic compatibility are explicit

The portable plan format may evolve.

The runtime must therefore distinguish at least:

- plan representation/version;
- wire/protocol compatibility;
- semantic operation identity/version where needed;
- application/server compatibility identity where needed;
- admitted assembly generation identity where required for realization binding.

Deployment identity records physical rollout or placement where needed. It must not substitute for plan compatibility or imply that every execution/resource shares one lifetime with the deployment.

A parser accepting bytes does not imply the server and suspended browser are semantically compatible.

Compatibility is a deliberate runtime predicate, developed further under stale continuation recovery below.

Executable projection and diagnostic sidecars

Verification evidence determines construction admission and supports later explanation. It does not supply an alternate transition input at runtime. The verification/projection pipeline should emit distinct executable and evidence products from the outset:

```
verified projection result
  |
  +--> ExecutablePlan
  |
  +--> DiagnosticProofSidecar
```

There is no production `AnnotatedPlan` that the browser executes and no semantics-preserving compiler optimization pass whose job is to erase proofs from such a plan. `ExecutablePlan` is the canonical runtime artifact. The sidecar is a separate compiler/checker artifact that may explain that runtime artifact but is never part of its semantic input.

The sidecar MUST also identify the exact `ExecutablePlan` artifact that it explains. The build should derive an `executable-digest`, artifact hash, or equivalent exact build identity from a canonical representation of the emitted executable payload and bind the sidecar to that identity. If the identity is embedded in the plan itself, the digest is computed over the canonical payload with that digest field excluded; alternatively the identity may remain entirely in build/asset metadata. Diagnostic tooling MUST refuse to correlate runtime locators with a sidecar whose executable identity does not match the deployed plan. There is no best-effort fallback to matching state or transition coordinates across different executable artifacts.

This exact artifact identity is distinct from semantic/protocol compatibility identity. A `plan-id` may intentionally identify a declared plan compatibility class or version, while `executable-digest` answers the much stricter diagnostic question: ”does this sidecar describe these exact emitted state and transition coordinates?” The digest may live in a build manifest, asset metadata, or the executable plan when needed by diagnostic tooling; its existence must not make proof metadata part of semantic execution.

The portable/browser transition interface should make this separation explicit:

```
AdapterStep(
  ExecutablePlan,
  AdapterState,
  NormalizedEvent)
    ->
  AdapterState', AbstractEffects
```

`AdapterStep` MUST NOT accept a proof sidecar, knowledge derivation tree, proof obligation, source span, compiler diagnostic object, or other verification-only metadata. The same rule applies to the portable projected-machine transition function. Diagnostic tooling may observe which runtime state/transition/effect occurred and then consult a sidecar, but no result of that lookup may flow back into semantic execution.

Every item needed to preserve runtime semantics belongs to the executable contract, admitted runtime state, or declared runtime inputs. It is not diagnostic metadata merely because the verifier reasoned about it.

The immutable `ExecutablePlan` contains the supported projected transition structure, executable guards and derivations, required input/state contracts, and plan/protocol identity where needed. A particular invocation's admitted runtime state and inputs carry values such as:

- current projected machine state;
- execution identity and resource-instance identity;
- semantic command identity where operationally required;
- resource ownership and correlation;
- authoritative or provisional status;
- scoped authoritative observations and progression requirements.

The plan specifies how those values may be consumed and changed. It is not mutated to absorb each execution's changing state. Shared fragment coordination may have a separate adapter-owned state under the same explicit boundary.

The verifier's explanation of why such a guard, predicate, transition, or identity is sufficient belongs in the sidecar. The executable object itself does not.

Three kinds of identity must not be conflated:

- a **runtime locator** identifies the state/transition/effect selected by the compact executable representation, for example `[state-index transition-index]` or another small generated coordinate;
- a **semantic identity** names a source-level state, transition, operation, or choreography concept for compiler/debugger use;
- a **proof-obligation identity** names a verifier obligation or derivation and is verifier-only unless diagnostics explicitly request it.

The term semantic identifier here denotes a source-level name or diagnostic identity. It must not be confused with the normalized semantic dependency identity defined in Part I: retaining a source-level name across a contract change does not preserve dependent certification.

A runtime locator need not be stable across recompilations. Stable semantic and proof-obligation identities belong in compiler/sidecar metadata unless some independent runtime requirement actually needs them. A `plan-id` or compatibility identity may remain in the executable plan because stale-plan detection is a runtime concern, not because proof diagnostics require it.

For example, a compact executable plan may conceptually contain:

```
{:plan-id P8
 :executable-digest H8
 :states [...]
 :transitions
 [[{:event :claim/settled
    :guard [:admission/execution-resource-current]
    :next 5}]
  ...]}
```

The guard tag is schematic executable vocabulary, not a prescribed public API. It requires that this event's declared execution and relevant resource instance retain the ownership needed for the transition. Concrete identities come from admitted runtime state and event correlation, not the immutable plan or sidecar. Shared-event paths use their own owner-relative admission rules. Passing this guard does not replace required command correlation, outcome validation, or other transition premises.

The executable semantic core evaluates the applicable transition and guards. The proof kernel checks derivations about the declared semantics, and dedicated semantic checkers establish their particular judgments. Neither checking role supplies runtime permission merely by having accepted the plan.

The optional sidecar maps compact runtime locators to richer semantic and proof information:

```
{:plan-id P8
 :executable-digest H8
 :transitions
 {[4 0]
  {:semantic-id :request-claim/helper/settled
   :knowledge-derivation ...
   :authority-derivation ...
   :projection-correspondence ...
   :proof-obligations ...
   :source ...}}}
```

Normal production packaging ships only `ExecutablePlan`. Test, development, and diagnostic tooling execute that exact same plan and may additionally load `DiagnosticProofSidecar`. Proof-enabled testing MUST NOT compile or execute a second semantically richer client machine. Before using a sidecar, diagnostic tooling MUST verify its exact executable identity; a mismatch disables the sidecar and is itself a diagnostic error rather than permission to guess at a correspondence.

A concrete build can make the separation mechanical.

The plan above is the canonical emitted runtime artifact. The build uses this very object for production, portable tests, and exact diagnostic correspondence; there is no second handwritten machine for the sidecar to explain.

The build computes an exact identity over the canonical executable payload:

```
(def executable-digest
  (artifact/digest
    {:algorithm :sha-256
     :encoding :canonical-edn-v1
     :value executable-plan}))
```

The sidecar can then explain the compact coordinates:

```
(def diagnostic-sidecar
  {:sidecar/version 1

   :plan/id :plan/request-claim

   :executable-digest executable-digest

   :states
   {0
    {:semantic-id :request-claim/helper/idle
     :source
     {:ns 'net.humanhelp.site.model.request.choreo
      :form 'request-claim}}

    1
    {:semantic-id :request-claim/helper/pending}

    2
    {:semantic-id :request-claim/helper/settled}

    3
    {:semantic-id :request-claim/helper/outcome-unknown}

    4
    {:semantic-id :request-claim/helper/recovered}

    5
    {:semantic-id :request-claim/helper/rejected}}

   :transitions
   {[1 0]
    {:semantic-id
     :request-claim/helper/accepted-settlement

     :knowledge-derivation
     {:premises
      [[:event/carries :authoritative-request]
       [:event/matches-command]
       [:event/current-owner]]

      :conclusion
      [:helper/may-install
       :authoritative-request]}

     :authority-derivation
     {:premises
      [[:settlement/from-trusted-boundary]
       [:request/authoritative-result]]

      :conclusion
      [:projection/status :authoritative]}
```

```
     :proof-obligations
     [:obligation/settlement-command-correlation
      :obligation/stale-owner-rejected
      :obligation/scoped-progression-admitted]}

    [1 1]
    {:semantic-id
     :request-claim/helper/timeout

     :knowledge-derivation
     {:premises
      [[:timeout/current-resource-owner]]

      :conclusion
      [:command/disposition :unknown]}

     :proof-obligations
     [:obligation/timeout-does-not-imply-rejection]}

    [1 2]
    {:semantic-id
     :request-claim/helper/rejected-settlement

     :knowledge-derivation
     {:premises
      [[:event/current-owner]
       [:event/matches-command]
       [:settlement/admitted-current-outcome]]
      :conclusion [:command/disposition :rejected]}

     :proof-obligations
     [:obligation/settlement-command-correlation
      :obligation/rejection-requires-admitted-outcome]}}})
```

The sidecar locates the rejected settlement at [1 2]. A missing explanation cannot be silently interpreted as proof that the transition is safe.

```
(get-in diagnostic-sidecar [:states 5 :semantic-id])
;; ⇒ :request-claim/helper/rejected
```

Production execution never calls:

```
(get-in diagnostic-sidecar
  [:transitions [1 0] :knowledge-derivation])
```

to decide whether settlement is admissible.

The transition function receives only executable semantics and current runtime inputs:

```
(step
  executable-plan
  runtime-state
  settled-event)
```

A development observer may watch that exact step result:

```
(defn observe-transition
  [{:keys [plan state-before event result]}
   sidecar]

  (when-not
    (and
      (= (:plan/id sidecar) (:plan/id plan))
      (= (:executable-digest sidecar)
         (artifact/digest
           {:algorithm :sha-256
            :encoding :canonical-edn-v1
            :value plan})))
    (throw
      (ex-info
        "Diagnostic sidecar does not describe this executable."
        {:error/type :diagnostic/executable-mismatch
         :sidecar-digest
         (:executable-digest sidecar)})))

  (let [locator
        (runtime/transition-locator
          plan
          state-before
          event)]

    {:runtime
     {:locator locator
      :event (:event/type event)
      :result (:result result)}

     ;; This is an untrusted diagnostic claim unless build provenance and
     ;; checker result are independently verified.
     :explanation/status :unverified-sidecar-claim
     :explanation
     (get-in sidecar [:transitions locator])}))
```

Its output can be rich:

```
{:runtime
 {:locator [1 0]
  :event :claim/settled
  :result :accepted}

 :explanation/status :unverified-sidecar-claim
 :explanation
 {:semantic-id
  :request-claim/helper/accepted-settlement

  :knowledge-derivation
  {...}

  :authority-derivation
  {...}

  :proof-obligations
  [...]}}
```

But observer failure must remain diagnostic:

```
(defn run-with-optional-observer
  [plan state event observer sidecar]
  (let [result
        (step plan state event)]

    (when observer
      (try
        (observer
          {:plan plan
           :state-before state
           :event event
           :result result}
          sidecar)

        (catch Throwable t
          (try
            (diagnostics/report!
              {:error/type :diagnostic/observer-failed
               :cause t})
            (catch Throwable _
              nil)))))

    ;; Semantic result is returned regardless of observer success.
    result))
```

Now corrupt the sidecar:

```
(def wrong-sidecar
  (assoc diagnostic-sidecar
    :executable-digest
    "sha256:not-this-plan"))

(run-with-optional-observer
  executable-plan
  runtime-state
  settled-event
  observe-transition
  wrong-sidecar)
;; ⇒
;; semantic step result is unchanged
;; diagnostic observer reports executable mismatch
```

Or remove it entirely:

```
(step
  executable-plan
  runtime-state
  settled-event)
;; ⇒
;; same semantic transition
```

The exact artifact identity is stricter than runtime compatibility. A rebuilt plan could have different state indices or emitted bytes while remaining compatible enough for ordinary execution under the server/runtime contract. A sidecar for the old bytes still cannot explain the new runtime coordinates. Matching the executable digest alone does not authenticate the explanation: a fabricated sidecar can copy that digest. Tooling that labels derivations as checker-verified must also validate sidecar provenance, its binding to the build/checker result, and the checked obligations. Otherwise it displays sidecar content only as an unverified diagnostic claim.

That is why the build needs both:

```
{:runtime-compatibility
 :humanhelp-live-v12

 :exact-executable-identity
 "sha256:6e4b..."}
```

One answers ”may this runtime continue?” The other answers ”does this diagnostic artifact describe these exact emitted coordinates?”

The sidecar architecture earns its complexity by preserving that separation: production runs one compact semantic machine; verification and debugging can explain that machine richly without becoming an alternate authority or control path.

Conceptually there are three modes:

```
verification/build
  global choreography
  -> ExecutablePlan
  +  DiagnosticProofSidecar / derivations / proof obligations

test/debug
  ExecutablePlan
  + optional read-only diagnostic observer using DiagnosticProofSidecar

production
  the same ExecutablePlan
  sidecar omitted
```

This separation is also a security boundary. A browser-side proof object is not a capability, authorization token, or evidence that the trusted server must accept. An adversarial browser can modify or fabricate any diagnostic sidecar, so authoritative security remains entirely on the trusted side.

The relevant correctness property is **diagnostic noninterference**, not a general verified compiler-erasure optimization. Attaching, removing, corrupting, or replacing a sidecar must not change semantic execution because the transition function has no sidecar input.

If `run` denotes execution without diagnostics and `run-with-observer` denotes the same execution with a read-only diagnostic observer, then for the same `ExecutablePlan` and event sequence:

```
semantic-trace(run(plan, events))
  =
semantic-trace(run-with-observer(plan, sidecar, events))
```

The equality is for the same admitted initial state and normalized event sequence, under the same declared runtime premises. It includes semantic state transitions and abstract effects, modulo diagnostic output. It does not claim that enabling instrumentation cannot affect wall-clock scheduling or resource consumption in a real browser.

The observer may emit additional diagnostic output only. It must not mutate runtime state, feed decisions back into event admission, suppress required effects, or turn an observer exception into a different domain outcome. Observer failure follows a contained diagnostic-failure path. This property should be reinforced structurally by module/API boundaries, checked mechanically where the implementation can inspect dependencies or effect categories, and exercised by property/model-based and Chromium tests. It is not intended to require proving bisimulation between a rich annotated AST and an erased production AST because no such runtime AST should exist.

A debug-only explanation facility may answer questions such as:

```
why did runtime transition [4 0] correspond to :request-claim/helper/settled?
what knowledge derivation justified this branch?
which authority obligation discharged this operation?
```

Such explanations are valuable precisely because they are detached from the production execution semantics.

ExecutablePlan is the canonical semantic runtime artifact

There must be exactly one canonical runtime semantic payload for a projected client role.

The design explicitly rejects:

```
rich proved plan
    |
    | optimize / erase
    v
different production plan
```

as the ordinary architecture.

Instead:

```
verified projection
    |
    +--> ExecutablePlan
    |
    +--> optional DiagnosticProofSidecar
```

The runtime plan contains the executable rules and declares the runtime inputs and state contracts it needs. The admitted runtime supplies concrete values through those contracts.

The sidecar may explain the plan but may never complete or modify its semantics.

Exact executable identity and compatibility identity are different

The system needs both concepts.

An exact executable identity answers:

> Does this diagnostic artifact describe these exact emitted runtime coordinates and bytes/terms?

A compatibility identity answers:

> May this projected runtime safely continue against this server/runtime contract?

Those questions are not interchangeable.

Two executable artifacts may be semantically compatible while having different exact digests.

Conversely, a matching operation name does not establish either exact artifact identity or compatibility.

Exact artifact matching fails closed for diagnostics

Diagnostic tooling must never ”best effort” correlate a sidecar with a different runtime artifact merely because:

- state counts look similar;
- semantic names overlap;
- a plan ID is close;
- timestamps suggest the files were built together;
- source locations happen to match.

If exact executable identity does not match, the sidecar is unavailable for that runtime artifact.

The application may continue to execute the plan if ordinary runtime compatibility permits it, but diagnostics must report the mismatch rather than inventing correspondence.

Diagnostic noninterference is architectural

The runtime transition function does not accept:

- proof certificates;
- derivation trees;
- source spans;
- checker objects;
- explanation metadata;
- suggested fixes.

Therefore changing any of those objects must not change runtime semantics through the defined interface. Excluding a sidecar parameter is necessary but not alone sufficient: ambient mutable state, shared callbacks, exception propagation, or hidden lookups must not reintroduce a diagnostic control path.

This should be reinforced by module and namespace dependency direction, not only by convention.

A proof sidecar that can accidentally influence runtime execution is not a sidecar.

The sidecar is not a capability

An adversarial browser may fabricate a perfect-looking diagnostic artifact.

The server must gain no authority from it.

In particular, a sidecar may not establish:

- principal identity;
- operation authorization;
- authoritative provenance;
- current server machine state;
- permission to read protected data;
- permission to subscribe to protected Live data.

Its only role is explanation/debug correspondence.

Runtime locator, semantic identity, and proof identity remain distinct

A compact runtime coordinate such as:

```
[4 0]
```

may identify one transition in an emitted plan.

A semantic identifier such as:

```
:request-claim/helper/settled
```

names source-level meaning.

A proof-obligation identifier names a checker obligation or derivation.

There is no requirement that runtime coordinates remain stable across builds.

Trying to make them stable for debugging convenience can accidentally couple runtime representation to source/compiler history.

The sidecar exists precisely so rich stable semantic/debug information does not have to pollute compact runtime structure.

Optional diagnostics observe; they do not drive

A debug observer may receive:

- current plan identity;
- runtime locator;
- normalized event;
- emitted abstract effects;
- transition outcome.

It may use those values to retrieve explanation.

It may not feed the result back into transition selection.

Conceptually:

```
transition core
    |
    +--> next semantic state
    +--> abstract effects
    +--> optional diagnostic observation
             |
             v
         explanation/logging only
```

The arrow never returns from explanation into semantic execution.

Build outputs retain enough correspondence for later application closure

The whole-application checker in Part VII must establish before serving that the browser artifact contains the admitted required projected plan set.

This part therefore requires build output to expose machine-checkable identity sufficient to compare:

- current projected executable plans;
- generated browser artifact contents/stamp;
- exact diagnostic sidecars when used.

It does not define the whole application assembly object yet.

It establishes the runtime/artifact identity facts that Part VII will consume.

The identity recipe must specify the object identified, canonical encoding or exact byte domain, excluded self-identifying fields if any, and the versions relevant to interpretation. A digest of canonical plan terms is not automatically a digest of the emitted JavaScript bundle. The build must establish the declared relationship among normalized plan, packaged artifact, and diagnostic locators without a circular hash dependency.

A matching hash establishes the claimed object correspondence under that recipe; it does not establish that the checker accepted the object's semantics. The assembly construction and admission phases own the complete production artifact gate.

Part VI — Browser realization and recovery

This part defines the pure browser adapter, its HTMX and physical-effect boundaries, fragment coordination, stale-plan recovery, managed effects, asynchronous admission, and authoritative absence. It applies the ownership rules of Part II, the scoped acquisition and command contracts of Part IV, and the executable/presentation boundaries of Part V. Target theorems identify required assurance; they do not claim that implementation or proof is complete.

Browser adapter refinement

Executable semantic core and browser integration boundary

The browser adapter must be structured so the Gesso semantic part is small and pure. This does not require Gesso to replace, wrap, or reimplement HTMX's normal request and DOM lifecycle.

Normalized Gesso-relevant events enter a deterministic transition function:

```
AdapterStep : ExecutablePlan x AdapterState x NormalizedEvent
           -> AdapterState x AbstractEffects
```

The plan supplies immutable executable rules; state and admitted events supply concrete runtime values, following Part V. An implementation may bind the plan before invoking the step function, but may not replace it with a diagnostic or test-only semantic object.

This adapter is an executable semantic core for browser coordination. The portable execution machine owns its projected trajectory; the two cores compose through their declared state/effect boundary. A proof kernel checks derivations about the semantics, and a dedicated semantic checker establishes its particular judgment. Neither supplies runtime facts or replaces concrete event admission.

Representative normalized events include:

```
{:event :htmx/before-request
 :execution-id E
 :transport-request-id H}

{:event :htmx/before-swap
 :execution-id E
 :transport-request-id H
 :target-id T}

{:event :htmx/after-swap
 :execution-id E
 :transport-request-id H
 :target-id T}

{:event :http/failed
 :execution-id E
 :transport-request-id H
 :error X}

{:event :live/invalidated
 :owner [:fragment F]
 :source-instance S
 :requirement R}

{:event :timer/fired
 :execution-id E
 :timer-id T
 :resource-instance I}

{:event :continuity/completed
 :owner O
 :resource-instance I
 :slot S}
```

These are illustrative event shapes, not complete wire schemas. In particular, the shared advisory resolves its declared source and scoped requirement without requiring a command execution. H identifies one transport attempt, not a HumanHelp domain Request; a retry has a distinct transport-request identity even when it retains the command identity. In these examples H also identifies the concrete request resource, so a redundant resource-instance field is unnecessary. For the timer, T names the logical timer and I identifies its concrete instance. The continuity owner may be an execution or an admitted fragment/effect scope. Correlation fields propose information for admission; their mere presence does not establish ownership.

Representative abstract effects/dispositions include:

```
[:machine/resume ...]
[:htmx/allow-request ...]
[:htmx/cancel-request ...]
[:htmx/allow-swap ...]
[:htmx/cancel-swap ...]
[:timer/start ...]
[:timer/cancel ...]
[:dom/install-provisional ...]
[:execution/retire ...]
```

For HTMX-owned operations, Gesso should normally decide semantic disposition–for example allow, cancel, ignore as stale, correlate, or observe completion–while HTMX continues to perform the request, response parsing, OOB handling, DOM swap, and settle work it already owns.

HTMX lifecycle listeners, EventSource/SSE observations, timers, MutationObserver callbacks, and Gesso-owned DOM hooks should submit Gesso-relevant observations to the executable semantic core rather than independently deciding choreography semantics, mutating execution ownership, or resuming machines through alternative code paths.

Gesso-owned physical effects that are not HTMX's responsibility, such as local provisional rendering, timers, continuity resources, and related cleanup, are interpreted by a narrow imperative effect shell. That shell performs physical browser operations but does not invent semantic state transitions.

Conceptually:

```
  ordinary HTMX lifecycle
           |
documented lifecycle event
           |
           v
    checked AdapterStep
           |
   allow/cancel/observe
           |
           v
  HTMX continues normally
           |
after-swap / after-settle
           |
           v
    checked AdapterStep
```

```
Gesso-owned browser event ---> checked AdapterStep ---> narrow browser FFI effect
```

Here `checked` identifies the transition object to which the stated assurance applies. It is not a claim that formal proof is already complete. The actual evidence class and remaining correspondence assumptions must be reported.

This architecture is a correctness requirement, not merely a code-style preference. It deliberately preserves HTMX as a useful declarative runtime instead of turning it into a transport primitive beneath a Gesso reimplementation.

A concrete HumanHelp Claim interaction shows how little semantic authority the HTMX listener layer actually needs.

Assume Part V emitted a Claim `ExecutablePlan` and the adapter has admitted an execution:

```
(def E42
  #uuid "e4200000-0000-0000-0000-000000000042")

(def C42
  #uuid "c4200000-0000-0000-0000-000000000042")

(def H1
  #uuid "a1100000-0000-0000-0000-000000000001")

(def claim-runtime
  {:execution/id E42

   :command/id C42

   :plan/id :plan/request-claim

   :compatibility/id :humanhelp-live-v12

   :machine/state :pending

   :target
   {:logical-id :request-card
    :request/id R}

   :authoritative-basis
   {:scope {:projection :request/card :request/id R}
    :basis :X17}

   :resources
   {:transport
    {:transport-request/id H1
     :owner [:execution E42]
     :status :active}

    :timeout
    {:timer/id :claim-settlement
     :resource-instance/id
     #uuid "a1000000-0000-0000-0000-000000000007"
     :owner [:execution E42]
     :status :active}}

   :projection
   {:authority :provisional
    :request/id R
    :request/status :claiming
    :command/id C42}})
```

The rendered button may still use ordinary declarative HTMX:

```
<button
  class="gesso-action gesso-action--primary"
  hx-post="/gesso/op/request-claim"
  hx-swap="none"
  data-gesso-execution="e42..."
  data-gesso-command="c42..."
  data-gesso-plan="plan/request-claim">
  Claiming...
</button>
```

The exact attributes are illustrative. They are correlation handles installed by an admitted realization, not authority assertions supplied by arbitrary client bytes.

A thin listener normalizes `htmx:beforeRequest`:

```
(defn on-htmx-before-request!
  [adapter raw-event]

  (let [normalized
        {:event :htmx/before-request

         :execution-id
         (correlation/execution-id raw-event)

         :transport-request-id
         (correlation/transport-request-id raw-event)

         :target-id
         (correlation/logical-target raw-event)}]

    (dispatch-adapter-event!
      adapter
      normalized)))
```

The pure adapter decides whether this physical request is still the request that the semantic execution owns:

```
(defn admit-before-request
  [state event]

  (cond
    (not (adapter/current-target-owner-and-plan?
           state event))
    {:result :rejected
     :diagnosis :stale-target-or-incompatible-plan
     :state state
     :effects
     [[:htmx/cancel-request
       {:transport-request-id
        (:transport-request-id event)}]]}

    (not=
      (:execution/id state)
      (:execution-id event))
    {:result :rejected
     :diagnosis :stale-execution
     :effects
     [[:htmx/cancel-request
       {:transport-request-id
        (:transport-request-id event)}]]}

    (not=
      (get-in state
        [:resources :transport :transport-request/id])
      (:transport-request-id event))
    {:result :rejected
     :diagnosis :wrong-transport-resource
     :effects
     [[:htmx/cancel-request
       {:transport-request-id
        (:transport-request-id event)}]]}

    (not=
      :active
      (get-in state
        [:resources :transport :status]))
    {:result :rejected
     :diagnosis :retired-transport-resource
     :effects
     [[:htmx/cancel-request
       {:transport-request-id
        (:transport-request-id event)}]]}

    :else
    {:result :accepted
     :state state
     :effects
     [[:htmx/allow-request
       {:transport-request-id
        (:transport-request-id event)}]]}))
```

The imperative shell does not reinterpret that answer:

```
(defn interpret-htmx-disposition!
  [raw-event effect]

  (case (first effect)

    :htmx/allow-request
    ;; No special action is needed. HTMX continues normally.
    nil

    :htmx/cancel-request
    (.preventDefault raw-event)

    :htmx/allow-swap
    nil

    :htmx/cancel-swap
    (.preventDefault raw-event)))
```

The Claim button declares `hx-swap"none"=`. Its response cannot be treated as proof that a Request-card DOM swap happened. A settlement response enters a separately admitted command-settlement path; a subsequent authorized card acquisition may use an HTMX swap. The following swap example applies to that **separate protected acquisition** and assumes its integration exposes a cancelable before-swap boundary for the entire protected mutation:

```
(defn on-claim-response!
  [adapter raw-event]
  (dispatch-adapter-event!
    adapter
    {:event :claim/settlement-candidate
     :execution-id (correlation/execution-id raw-event)
     :transport-request-id
     (correlation/transport-request-id raw-event)
     ;; Decoding yields untrusted data. Part V's settlement admission checks
     ;; current owner/resource/command and trusted source/progression.
     :candidate (response/decode-gesso-settlement raw-event)}))
```

The protected swap below belongs to an authorized acquisition owner with its own current transport resource; it need not share the Claim request's H1.

```
(defn on-htmx-before-swap!
  [adapter raw-event]

  (dispatch-adapter-event!
    adapter
    {:event :htmx/before-swap

     :execution-id
     (correlation/execution-id raw-event)

     :transport-request-id
     (correlation/transport-request-id raw-event)

     :target-id
     (correlation/logical-target raw-event)

     :candidate-observation
     (response/decode-gesso-observation
       raw-event)}))
```

The adapter can reject a stale response **before** it mutates protected presentation:

```
(defn admit-before-swap
  [state event]

  (let [candidate
        (:candidate-observation event)]

    (cond
      (not=
        (:execution/id state)
        (:execution-id event))
      {:result :rejected
       :diagnosis :stale-execution
       :effects
       [[:htmx/cancel-swap
         {:transport-request-id
          (:transport-request-id event)}]]}

      (not (adapter/current-transport-and-target?
             state event))
      {:result :rejected
       :diagnosis :stale-resource-or-target
       :effects
       [[:htmx/cancel-swap
         {:transport-request-id
          (:transport-request-id event)}]]}

      (not
        (acquisition/admitted-for-current-target?
          state event candidate))
      {:result :rejected
       :diagnosis :unadmitted-or-incompatible-observation
       :effects
       [[:htmx/cancel-swap
         {:transport-request-id
          (:transport-request-id event)}]
        [:recovery/reacquire-current-target
         {:scope (get-in state
                   [:authoritative-basis :scope])}]]}

      :else
      {:result :accepted
       :state
       (adapter/stage-protected-install
         state event candidate)

       :effects
       [[:htmx/allow-swap
         {:transport-request-id
          (:transport-request-id event)}]]})))
```

After HTMX performs that separate acquisition's supported swap, `htmx:afterSwap` reports its physical lifecycle step:

```
(defn on-htmx-after-swap!
  [adapter raw-event]

  (dispatch-adapter-event!
    adapter
    {:event :htmx/after-swap

     :execution-id
     (correlation/execution-id raw-event)

     :transport-request-id
     (correlation/transport-request-id raw-event)

     :target-id
     (correlation/logical-target raw-event)}))
```

The transition core may record the admitted observation's physical realization and then discharge incorporation-dependent obligations. It does not infer the Claim command's disposition from a DOM mutation. The Claim's own admitted settlement revokes timeout ownership at the settlement transition, before any slow rendering; it cannot wait for this acquisition's after-swap callback.

A failure callback follows the same route:

```
(defn on-htmx-send-error!
  [adapter raw-event]

  (dispatch-adapter-event!
    adapter
    {:event :http/failed

     :execution-id
     (correlation/execution-id raw-event)

     :transport-request-id
     (correlation/transport-request-id raw-event)

     :error
     (browser/raw-request-error raw-event)}))
```

If authoritative disposition is unknown, the semantic transition may retire the local transport and enter reconciliation:

```
(defn handle-http-failed
  [state event]

  (if
    (and
      (= (:execution/id state)
         (:execution-id event))
      (= :pending (:machine/state state))
      (=
        (get-in state
          [:resources :transport :transport-request/id])
        (:transport-request-id event))
      (= :active
         (get-in state [:resources :transport :status])))

    {:result :accepted

     :state
     (→ state
         (assoc-in
           [:resources :transport :status]
           :retired)

         (assoc-in
           [:resources :timeout :status]
           :retired)

         (assoc
           :machine/state
           :outcome-unknown

           :command-knowledge
           {:command/id (:command/id state)
            :disposition :unknown}))

     :effects
     [[:timer/cancel
       {:resource-instance/id
        (get-in state
          [:resources :timeout :resource-instance/id])}]
      [:recovery/request-authoritative-acquisition
       {:scope (get-in state
                 [:authoritative-basis :scope])}]]}

    {:result :rejected
     :diagnosis :stale-or-unknown-request
     :state state
     :effects []}))
```

The example is intentionally mundane. Its point is that HTMX is still doing HTMX work: starting the request, receiving the response, parsing it, and performing the supported swap. Gesso contributes narrow semantic gates and correlation around lifecycle points where protected state could otherwise be mutated by stale or incompatible work.

Nothing in the listener layer decides Request policy, current authorization, command disposition, or which execution ”probably” owns a response. Those decisions remain in the semantic adapter and trusted server/model boundaries.

HTMX integration contract

HTMX is a trusted external runtime, not a Gesso effect implementation.

Gesso should use HTMX through its ordinary declarative API and documented lifecycle events. HTMX remains responsible for

the behavior it already owns, including ordinary request execution, response processing, out-of-band response handling, swaps, and settlement.

Gesso may observe or intervene at documented lifecycle boundaries when a Gesso-owned semantic invariant requires it. Typical intervention is narrow: allow the lifecycle to continue, cancel/suppress stale work, attach or recover opaque correlation, capture continuity information, or observe completion.

A custom HTMX extension is NOT required by this architecture. Gesso should not introduce one merely to make HTMX look like an abstract effect executor. If a future concrete requirement cannot be implemented correctly through HTMX's ordinary declarative API and documented lifecycle hooks, an extension may be considered then as an explicit engineering choice with its own proof/test boundary.

The exact HTMX behavior on which Gesso relies forms a supported-contract subset. The implementation should pin and test the HTMX and htmx-ext-sse versions it supports and document any response headers, swap modes, OOB behavior, history features, or other options that can bypass or alter a lifecycle gate required by Gesso correctness. Unsupported combinations may be rejected rather than silently weakening the semantic guarantees.

For each supported response/swap mode, the integration must identify which documented gate precedes every relevant physical mutation and which event establishes completion or failure. Main-target approval must not implicitly authorize unrelated out-of-band targets, history restoration, or other effects whose ownership and scope have not been admitted. Response parsing alone is not authoritative installation.

If a required action cannot be suppressed before it affects protected semantic presentation under the selected lifecycle contract, that configuration cannot claim the corresponding invariant. The integration must reject it or provide a separately specified realization with the required correspondence evidence. These requirements do not assert particular hook behavior for an unexamined HTMX version.

The proof does not claim HTMX itself is correct. It claims that, assuming the supported HTMX lifecycle contract, Gesso's semantic decisions at those boundaries preserve the stated adapter invariants.

Browser FFI contract

The Gesso-owned browser FFI/effect shell is an explicit proof assumption and must remain deliberately weaker than the semantic adapter core. Its job is physical execution of Gesso-owned browser effects and faithful translation of primitive browser observations, not protocol reasoning. HTMX's native lifecycle is governed by the separate HTMX integration contract above rather than being reimplemented inside this shell.

The FFI shell may:

- observe primitive browser events and normalize their raw payloads;
- carry opaque owner/resource correlation identity installed when semantic logic created or admitted the physical resource;
- report physical success, failure, cancellation, or disappearance as declared normalized events;
- perform concrete Gesso-owned DOM, EventSource, timer, request, and listener operations requested by admitted abstract effects, without independently duplicating an HTMX-owned request or swap;
- record diagnostics about physical failures.

The FFI shell must not:

- infer semantic authority or authorization;
- infer domain facts merely from arbitrary DOM text or client-provided labels;
- choose an execution or shared owner by independent semantic reasoning;
- invent an admitted execution/resource identity or upgrade provenance, authority, basis, or ownership;
- decide whether rollback, reconciliation, supersession, or retirement is valid;
- transfer semantic ownership between executions;
- mutate semantic adapter state through a side channel;
- resume a projected machine except through an abstract effect emitted by the adapter transition core.

Correlation should be established when semantic logic creates or admits a physical resource and then carried opaquely by integration code. Browser code must not reconstruct semantic ownership later from mutable DOM position, CSS selectors, current target contents, or another physical coincidence.

A physical effect must not rely on an admission decision that has already lost its applicability. The execution contract must either perform the action within the admitted serialized step or validate its still-current permit/ownership at the last required boundary before mutation. A delayed effect queued while E1 was current cannot install obsolete DOM after E1 retires merely because its later completion callback would be rejected.

The difference between rejecting a late callback and preventing a late **effect** is subtle enough to deserve its own example.

Suppose E1 receives a valid settlement and AdapterStep emits an abstract effect to install authoritative Request content:

```
(def emitted-effect
  [:dom/install-authoritative
   {:permit
    {:permit/id P1
     :owner [:execution E1]
     :resource-instance/id I1}

    :target [:request-card R]

    :observation
    ;; Fixture: issued by the admitted acquisition/settlement boundary.
    (acquisition/admitted-fixture
      {:basis :X60
       :scope {:projection :request/card :request/id R}
       :request
       {:request/id R
        :request/status :claimed
        :request/helper-id Alice}})}])
```

The imperative shell queues the physical DOM work, but before the queue runs, E1 is retired and E2 becomes the current owner of the same logical target:

```
(def adapter-state
  {:owners
   {[:request-card R]
    {:owner [:execution E2]
     :resource-instance/id I2}}

   :permits
   {P1
    {:owner [:execution E1]
     :resource-instance/id I1
     :status :revoked}}})
```

It is not enough for the eventual **completion callback** of E1's DOM work to be rejected. The mutation itself must not happen after its permit expired.

A narrow physical boundary check can validate a permit that was created by the semantic adapter:

```clojure
(defn permit-current?
  [adapter-state permit target]

  (let [permit-state
        (get-in adapter-state
          [:permits (:permit/id permit)])

        current-owner
        (get-in adapter-state
          [:owners target])]

    (and
      (some? (:permit/id permit))
      (some? (:owner permit))
      (some? (:resource-instance/id permit))
      (some? current-owner)
      (= :active
         (:status permit-state))

      (= (:owner permit-state)
         (:owner permit))

      (=
        (:resource-instance/id permit-state)
        (:resource-instance/id permit))

      (= current-owner
         {:owner (:owner permit)
          :resource-instance/id
          (:resource-instance/id permit)}))))
```

The effect shell may then refuse the delayed mutation:

```clojure
(defn perform-authoritative-install!
  [browser adapter
   [_ {:keys [permit target observation]}]]

  ;; Serialize the final permit/admission check with the protected mutation.
  ;; A stale snapshot of adapter state would leave the same race open.
  (adapter/with-current-permit!
    adapter permit target
    (fn []
      (acquisition/require-admitted-for-current-target!
        adapter target observation)
      (dom/replace-target!
        browser target
        (views/render (acquisition/value observation)))
      {:physical/status :installed})))
```

Crucially, the shell did not decide **which execution should own the target**. That decision was already present in adapter state and in permit P1. The shell only checked whether the adapter-issued permit remained applicable at the last boundary before physical mutation. `with-current-permit!` returns `:physical/status :suppressed` when the permit is revoked. Its serialization contract prevents ownership changing between this final check and the protected mutation; merely passing a previously captured `adapter-state` to `permit-current?` would not establish that property.

If a browser primitive cannot provide a usable check before a protected mutation, then that primitive cannot support the corresponding Gesso invariant without a different realization strategy.

This closes a race that callback-only designs miss:

```
semantic event admitted for E1
    |
abstract DOM effect queued
    |
E1 retires / E2 takes target
    |
old queued DOM mutation runs
    |
completion callback rejected

too late -- protected DOM was already corrupted
```

The required ordering is instead:

```
semantic event admitted for E1
    |
effect receives adapter-issued permit P1
    |
E1 retires -> P1 revoked
    |
physical mutation reaches last protected boundary
    |
P1 no longer current -> mutation suppressed
```

That is the browser-side meaning of ”an admission decision must not outlive its applicability.”

This is a boundary check, not permission for the shell to independently decide ownership. The adapter owns the permit and its validity rule. Physical handles must remain associated with the resource instance created for them so cleanup cannot rediscover and remove a newer resource by selector alone.

The smaller this shell remains, the smaller the unproved correspondence boundary between the formal adapter and the real browser. Chromium integration tests are responsible for aggressively challenging both these assumptions and the supported HTMX lifecycle contract.

Property/model-based transition testing

Property- and model-based testing should be used early while the transition semantics are still being designed. This is complementary to proof, not a replacement for it.

For example, a generator may produce sequences such as:

```
start E1
request begins
invalidation arrives
retire E1
start E2
E1 response arrives
E1 timer fires
E2 response arrives
duplicate E1 response arrives
continuity callback arrives
```

and check the adapter invariants after every transition.

Useful generated properties include execution isolation, retired-resource rejection, declared fragment-coordination bounds, semantic ownership uniqueness, prevention of obsolete restoration after authoritative installation, and terminal retirement.

A generated counterexample can expose a bad invariant or transition rule before time is spent proving it. Passing generated tests does not justify the word `proved`; the same claimed safety property still requires the mechanical proof defined by this document.

Adapter refinement theorem

The target claim is approximately:

> Assuming the declared browser primitives and supported HTMX lifecycle contract satisfy their named assumptions, every semantic transition made by the Gesso browser adapter corresponds to a permitted transition of the formal browser/Choreo semantics, and the adapter preserves its stated safety invariants.

The production transition core should be generated from or directly implement the checked transition representation. A stronger formal claim additionally requires the production/formal correspondence evidence for that representation and checker. A separately handwritten semantic callback layer would reopen the gap; shared terminology or similar source code is not correspondence evidence.

The abstract event model must include stale, duplicate, malformed, missing, and reordered observations relevant to the property. Scheduler fairness, delivery, and cleanup assumptions must be named rather than excluded from the model by assuming the desired callback order.

The proof need not claim that Chrome, the JavaScript engine, the operating system, or the CPU is formally verified by Gesso.

Instead those primitives are explicit assumptions at the browser FFI or HTMX integration boundary.

Execution isolation theorem

For distinct executions E1 and E2, an asynchronous event produced by a resource owned exclusively by E1 must not alter:

- the Choreo state of E2;
- E2's provisional state;
- E2's target/resource ownership;
- E2's continuity-capture instance identity and ownership;
- an authoritative projection installed after E1 became stale;
- any other execution-scoped semantic state of E2.

Intentionally shared authoritative invalidation is not execution-private and is handled through its own declared path.

This theorem should subsume broad families of race bugs involving:

- old settlement timers;
- old HTMX completions;
- old continuity callbacks;
- old fragment requests;
- target replacement followed by a new execution;
- duplicated external callbacks.

Authoritative installation prevents obsolete restoration

Once a logical target has installed an admitted authoritative observation that supersedes provisional ownership for the relevant scope, cleanup or delayed effects from the superseded execution must not restore presentation that violates that observation or its ownership contract merely because a callback remains.

This prohibition does not impose a total ordering on observations or provenance, nor prevent legitimate later domain changes. Any stronger monotonicity claim must specify the ordering and scope for which it is established.

The implementation should rely on supported progression/ownership relations, not arithmetic ordering of opaque tokens.

This property prevents:

```
canonical authoritative response arrives
        |
old failure callback runs
        |
old optimistic snapshot is restored
```

The same rule applies when two authoritative acquisitions have different scopes. A response that is newer in one progression coordinate does not automatically supersede every independently justified observation already installed for the logical view. Supersession is permitted only where the declared scope, projection, ownership, and progression relations justify it.

Scoped authoritative installation and advisory discharge

The browser runtime must preserve the distinction between installing an authoritative observation and claiming that an entire aggregate view has been reacquired at one common snapshot.

An authoritative acquisition result must carry or resolve enough checked semantic information for the runtime to determine the scope of what the result actually establishes. Depending on the existing Gesso realization, that may be represented by projection identity, progression requirements, acquisition metadata, semantic ownership, or another checked mechanism. This requirement does not mandate a universal per-object revision number or a new persisted history structure.

For a scoped installation, the runtime must be able to preserve all of the following simultaneously:

- a newly installed authoritative observation for the acquisition's covered object or projection;
- other independently justified authoritative observations whose scopes are not superseded by that installation;
- unrelated advisory obligations not discharged by the acquisition;
- local presentation state that remains compatible with the adopted semantic view;
- provisional state whose ownership has not been superseded or otherwise resolved by the installation.

Therefore:

```
response completed
    ≠
all advisory information discharged

newer database progression observed for object S
    ≠
whole board reacquired at that progression
```

A completed acquisition may discharge only obligations satisfied under the Part IV advisory-discharge contract. An obligation requiring adoption into a visible projection is not discharged solely because the response was fetched or parsed. The runtime must distinguish justified observation, admitted installation, physical realization, and completion of any required incorporation step. An advisory that arrives while a read is in flight remains outstanding when the returned observation does not cover the later obligation.

Likewise, if a local correlated operation installs a newer justified authoritative observation before an older Refresh response arrives, the older response must not blindly replace the newer installation. The runtime may reject the older result, preserve independently compatible portions, schedule a successor acquisition, or use another checked reconciliation strategy already supported by Gesso. It must not merge incompatible snapshots merely to avoid another read.

1. Installation and physical realization are distinct stages

   An admitted authoritative observation is not invalidated by a later DOM or continuity failure. Nor does admission establish that its intended physical representation was successfully installed.

   The adapter must track enough status to distinguish the observation it has accepted, the representation currently realized, and any outstanding rendering or adoption obligation. A failed or partially completed installation must follow a declared repair path rather than claim a coherent DOM that does not exist. The affected presentation must not expose enabled semantic affordances based on an incoherent mixture of bindings and content.

   The integration need not promise one atomic transaction over arbitrary DOM effects. It must specify the admitted realization boundary and failure handling for the supported swap/effect set. It may preserve already compatible portions or reacquire and reconstruct the affected scope where the contract permits it. It must not re-enable obsolete provisional restoration to hide a rendering fault.

   Continuity failure after successful authoritative installation is a local presentation fault. It does not revoke that installation or the authoritative model result, as specified in Part V.

Fragment coordination contract

For each admitted logical coordination scope, the default supported coordinator admits at most one active logical acquisition and one queued acquisition slot:

```
active logical acquisition count ⩽ 1
queued acquisition slot count    ⩽ 1
```

The slot may contain a normalized bundle of scoped requirements. Its meaning follows Part IV's satisfaction and combination contract. Slot count is not a proof that this bundle or all retained metadata is bounded. A realization must state its supported scope domain, representation bounds, and treatment of incomparable requirements.

Physical requests may outlive logical retirement. Therefore the logical bound does not claim that only one physical request exists in the browser at every instant. Physical cancellation debt and its operational limits are tracked separately and cannot retain semantic installation rights.

1. Advisory debt is distinct from permission to acquire

   Receiving an advisory records the scoped requirement. Whether to start an acquisition depends on the declared application policy and current authorized acquisition path. A pending advisory does not by itself create a queued command to automatically adopt remote content.

   An explicit Refresh may admit acquisition of a captured requirement under a declared action scope. The contract must say whether that action also permits successor reads needed for the same recovery, or whether a later advisory needs another user action. A completion callback must not make that policy choice by accident. The requirement cannot disappear merely because permission to acquire it has not yet been supplied.

   Acquisition permission is local scheduling/adoption policy, not server-side authorization. Each protected read still follows the trusted authorization path.

2. Coordinator state and transitions

   The state must represent, directly or through admitted references:

   - logical coordination scope and current owner instance;
   - active acquisition identity, captured requirement, and relevant policy context;
   - outstanding requirements and any queued acquisition permitted to satisfy them;
   - admitted observations and adopted projection state needed for comparison;
   - physical-resource correlations and failure/recovery status relevant to safety.

   The following transitions define the required behavior without prescribing one public record layout:

Event	Required semantic result
Advisory arrives while idle	Retain its requirement; start only if declared policy permits acquisition
Advisory arrives while active	Preserve its requirement using safe combination; do not relabel it covered by the captured request
Refresh or other permitted trigger	Admit the requested acquisition scope and captured obligations; queue or start according to current ownership
Active response arrives	Validate correlation, authority/projection contract, scope, consistency, and installation compatibility before applying it
Response covers obligations	Discharge only the acquisition or incorporation obligations actually satisfied; retain the rest
Active read fails or is cancelled	End that logical attempt; preserve unsatisfied obligations and follow declared retry/recovery policy
Owner retires or is replaced	Revoke its private permits and correlations; explicitly retain, transfer, or re-establish still-required coordination state
Retired response arrives	Reject its obsolete installation path; do not infer ownership from the current target

   When an active attempt ends, at most one queued acquisition is admitted next, and only if its policy and resource conditions still hold. Failed reads must not create an uncontrolled immediate retry loop. Retry limits, delay, and availability assumptions belong to the chosen recovery contract; stopping a retry does not falsely discharge its requirement.

   A successor response may independently satisfy an obligation that arrived after the request began. In that case a redundant read is unnecessary, but coverage must be established from the returned observation rather than assumed from completion order.

   A complete HumanHelp Refresh race makes the coordinator contract easier to inspect.

   Start with Bob's board at X17 and one advisory O1:

```
(def board-runtime
  {:owner
   {:fragment/id :request-board
    :resource-instance/id
    #uuid "b1000000-0000-0000-0000-000000000001"}

   :projection
   {:kind :request/board
    :organization-id 0
```

```
    :coverage :complete-authorized-membership}

   :adopted
   {:basis :X17
    :requests [R1 R2]}

   :requirements
   [{:requirement/id :O1
     :kind :newer-authority-available
     :minimum-basis :X18
     :scope
     {:kind :request/board
      :organization-id O
      :coverage :complete-authorized-membership}
     :requires :acquire-and-adopt}]

   :active nil
   :installation-confirmations {}
   :queued nil})
```

Bob presses Refresh. The application policy permits an acquisition now:

```
(defn start-refresh
  [state acquisition-id resource-instance-id]
  (when (:active state)
    (throw
      (ex-info
        "Logical acquisition already active."
        {:error/type :fragment/acquisition-already-active})))

  (assoc state
    :active
    {:acquisition/id acquisition-id

     :resource-instance/id resource-instance-id

     :captured-requirements
     (:requirements state)

     :scope
     (:projection state)

     :permission
     {:kind :explicit-refresh
      :granted? true}}

    :queued nil))

(def A1
  (start-refresh
    board-runtime
    :A1
    #uuid "b2000000-0000-0000-0000-000000000010"))
```

While A1 is in flight, Alice creates another Request and Bob receives advisory O2 requiring authority through X20:

```
(def O2
  {:requirement/id :O2
   :kind :newer-authority-available
   :minimum-basis :X20
```

```
   :scope
   {:kind :request/board
    :organization-id O
    :coverage :complete-authorized-membership}
   :requires :acquire-and-adopt})
```

The coordinator must preserve it:

```
(defn on-advisory
  [state admitted-requirement]

  (advisory/require-admitted-for-fragment!
    state admitted-requirement)
  (let [requirement
        (advisory/value admitted-requirement)
        state*
        (update state
          :requirements
          requirement/combine-preserving-scope
          requirement)]

    ;; This HumanHelp policy does not grant new acquisition permission solely
    ;; because an advisory arrived.
    (if
      (and
        (:active state*)
        (get-in state*
          [:active :permission :granted?]))

      ;; A queued slot may remember that follow-up work could be useful,
      ;; but it does not claim the active response will cover it.
      (assoc state*
        :queued
        {:reason :requirement-arrived-while-active
         :requirements
         (:requirements state*)})

      state*)))

(def A1-plus-O2
  (on-advisory
    A1
    ;; Fixture issued after the current subscription and scope checks.
    (advisory/admitted-fixture
      O2
      {:fragment-owner (:owner A1)
       :subscription-instance/id :subscription-bob-board-current})))
```

Suppose A1 returns a complete authorized board observation at X18. The following is an **admitted installation completion** for that acquisition, after the authorized read and a current-permit DOM installation. A network response alone cannot trigger this event or discharge O1:

```
(def A1-response
  {:event :acquisition/installed

   :owner
   {:fragment/id :request-board
    :resource-instance/id
    #uuid "b1000000-0000-0000-0000-000000000001"}

   :acquisition/id :A1
```

```
   :resource-instance/id
   #uuid "b2000000-0000-0000-0000-000000000010"

   :observation
   ;; Fixture issued by the trusted, authorized acquisition boundary.
   (acquisition/admitted-fixture
     {:scope
      {:kind :request/board
       :organization-id 0
       :coverage :complete-authorized-membership}
      :basis :X18
      :requests [R1 R2 R3]}
     {:owner
      {:fragment/id :request-board
       :resource-instance/id
       #uuid "b1000000-0000-0000-0000-000000000001"}
      :acquisition/id :A1
      :resource-instance/id
      #uuid "b2000000-0000-0000-0000-000000000010"})})
```

Admission checks both the fragment owner and concrete acquisition resource:

```
(defn current-acquisition?
  [state event]
  (and
    (some? (:owner state))
    (some? (:owner event))
    (some? (get-in state [:active :acquisition/id]))
    (some? (get-in state [:active :resource-instance/id]))
    (= (:owner state)
       (:owner event))

    (=
      (get-in state
        [:active :acquisition/id])
      (:acquisition/id event))

    (=
      (get-in state
        [:active :resource-instance/id])
      (:resource-instance/id event))))

(defn admitted-acquisition-response?
  [state event]
  (and
    (current-acquisition?
      state
      event)

    (acquisition/admitted-for-current-target?
      state event (:observation event))

    (installation/confirmed-current?
      state event)))
```

The confirmation predicate consults adapter-owned installation records, not an `:installed?` bit from response bytes. The following evaluations assume the authorized current-permit installation has already completed and record that result in coordinator state:

Discharge is computed from the actual observation:

```
(defn complete-fragment-acquisition
  [state event]

  (if-not
    (admitted-acquisition-response?
      state
      event)

    {:result :rejected
     :state state
     :diagnosis :stale-or-incompatible-acquisition}

    (let [observation
          (acquisition/value (:observation event))

          [covered outstanding]
          ((juxt filter remove)
           #(advisory/satisfied-by?
              %
              observation)
           (:requirements state))]

      {:result :accepted

       :state
       (→ state
          (assoc
            :adopted
            {:basis (:basis observation)
             :requests (:requests observation)}

            :requirements
            (vec outstanding)

            :active nil)

          (assoc
            :queued
            (when
              (seq outstanding)
              {:reason :outstanding-requirements
               :requirements
               (vec outstanding)})))

       :discharged
       (mapv :requirement/id
             covered)})))
```

For X18, only O1 is discharged:

```
(def after-A1
  (complete-fragment-acquisition
    (installation/record-current-confirmation
      A1-plus-O2 A1-response)
    A1-response))

(get-in after-A1
  [:state :adopted :basis])
;; ⇒ :X18
```

```
(:discharged after-A1)
;; ⇒ [:01]

(mapv
  :requirement/id
  (get-in after-A1
    [:state :requirements]))
;; ⇒ [:02]
```

The coordinator can now show the Refresh indicator again or retain whatever product-specific ”newer Requests available” presentation corresponds to O2. What it may **not** do is infer a second automatic read merely because the queue slot is nonempty if HumanHelp policy requires a new explicit Refresh.

Now consider the opposite case. Suppose A1's actual authoritative response is through X21 because the database acquisition naturally covered O2 as well:

```
(def A1-response-X21
  (assoc A1-response
    :observation
    ;; Separate trusted fixture; mutating an admitted wrapper would not
    ;; confer admission on the modified result.
    (acquisition/admitted-fixture
      (assoc
        (acquisition/value (:observation A1-response))
        :basis :X21)
      {:owner (:owner A1-response)
       :acquisition/id :A1
       :resource-instance/id
       (:resource-instance/id A1-response)})))

(def after-A1-X21
  (complete-fragment-acquisition
    (installation/record-current-confirmation
      A1-plus-O2 A1-response-X21)
    A1-response-X21))

(get-in after-A1-X21
  [:state :requirements])
;; ⇒ []

(:discharged after-A1-X21)
;; ⇒ [:01 :02]
```

No redundant successor read is needed. The newer requirement was not considered covered merely because it arrived before completion; it was covered because the actual returned observation satisfied it.

The physical-request race remains separate. Suppose A1 is logically retired and A2 begins, but the browser cannot successfully abort A1's underlying fetch:

```
(def A2-state
  (→ (get after-A1 :state)
     (assoc
       :active
       {:acquisition/id :A2
        :resource-instance/id
        #uuid "b2000000-0000-0000-0000-000000000011"
        :scope
        {:kind :request/board
         :organization-id 0
         :coverage :complete-authorized-membership}
        :permission
```

```
         {:kind :explicit-refresh
          :granted? true}})))

(def late-A1
  (assoc A1-response
    :observation
    (acquisition/admitted-fixture
      (assoc
        (acquisition/value (:observation A1-response))
        :basis :X22)
      {:owner (:owner A1-response)
       :acquisition/id :A1
       :resource-instance/id
       (:resource-instance/id A1-response)})))
```

The late response is still stale because ownership changed:

```
(current-acquisition?
  A2-state
  late-A1)
;; ⇒ false

(complete-fragment-acquisition
  A2-state
  late-A1)
;; ⇒
{:result :rejected
 :diagnosis :stale-or-incompatible-acquisition
 :state A2-state}
```

A later database basis does not restore the retired physical request's installation rights.

This is exactly why the coordinator's claimed bound is about **logical** acquisitions. The browser may temporarily contain physical remnants of A1 and A2 simultaneously, but only the current admitted acquisition can install into the managed fragment.

3. Mechanical checking and concrete correspondence

   The coordinator's formal or exact-finite claim must specify the abstract states, events, coverage relations, policy choices, and retained-data abstraction used. Current/stale ownership and covered/uncovered obligation relations can be useful abstractions only when their soundness for the claimed property is justified. A small sample of concrete tokens or two queue slots does not establish this.

   The checked properties should include non-loss of obligations, absence of stale installation, preservation of independent scopes, policy-correct acquisition, and the stated logical concurrency bound. Progress claims additionally require their scheduling, user-action, resource, and authoritative-availability premises.

Resource/terminal invariant

Every physical resource represented in the semantic adapter state has exactly one declared semantic owner or explicitly shared ownership class while that ownership is active.

Retirement first revokes the execution's semantic ownership and the admission rights of its execution-private resource instances. After that semantic retirement, the execution owns zero execution-private semantic resources even if some corresponding physical browser objects have not yet been successfully destroyed.

Physical cleanup is then idempotent, best effort, and diagnostic. A timer, listener, HTMX request, continuity object, or other physical resource may outlive semantic retirement because cancellation raced, threw, was ignored by a library, or the browser was suspended. Such a leftover resource is cleanup debt, not a source of semantic authority. Any later callback from it must be rejected by the owner/resource-instance admission gate.

New semantic ownership therefore depends on successful revocation of old semantic ownership, not on proof that every old physical resource has already vanished.

The target theorem is:

> Once execution E is semantically retired, no subsequently delivered private event or deferred effect may use E's revoked ownership to alter current execution or presentation state, regardless of whether E's physical cleanup completed successfully. Independently admitted shared or command-resolution events retain only the rights of their own declared owner.

Cleanup failure in the physical shell is recorded as an external/runtime fault; it must not mutate the semantic transition history into an impossible earlier state and must not block safe logical reuse merely because physical cleanup is incomplete.

Adversarial-browser authority theorem

The formal server-side protocol should be checked under an adversarial client model rather than assuming the generated browser projection is honest.

The client may send any syntactically representable request bytes.

Subject to the named authentication/model/XTDB assumptions, arbitrary client behavior must not be sufficient to:

- establish another principal identity;
- acquire an undeclared authority role;
- manufacture authoritative provenance;
- jump the trusted server projection to a privileged state;
- perform a model transition not permitted to the authenticated principal;
- obtain a protected authoritative observation or Live subscription that the authenticated principal is not authorized to receive.

This is stronger than proving that the generated JavaScript follows the global choreography.

The honest projected browser is a usability/correctness implementation. Security on the trusted side must remain valid even when that implementation is replaced entirely.

Adapter state has one semantic owner

The browser adapter is the owner of browser coordination semantics and composes the portable execution machines under the Part V state ownership contract. Each fact has one semantic owner. A portable execution may own its trajectory while the adapter owns fragment coordination and the conditions under which that execution is resumed.

HTMX, EventSource callbacks, timers, continuity code, and DOM helpers may produce normalized observations or realize abstract effects.

They must not each keep a partial private copy of:

- which execution is current;
- which admitted execution or effect-scope owner holds the target;
- whether authority has superseded optimism;
- whether rollback remains legal;
- whether an event may resume a machine.

Duplicated ownership state is exactly how individually reasonable callbacks produce impossible global behavior.

Semantic admission precedes physical action when intervention is required

Where Gesso must decide whether an HTMX lifecycle action is semantically admissible, the order is:

```
raw lifecycle event
    |
normalize
    |
semantic adapter decision
    |
allow / cancel / ignore / correlate
    |
HTMX continues or is suppressed
```

The shell must not first perform the action and ask the semantic machine afterward whether it should have happened.

This does not require Gesso to control every ordinary HTMX lifecycle step. Intervention is narrow and only where a Gesso invariant requires it.

HTMX remains declarative infrastructure rather than a hidden transport beneath Gesso

The architecture deliberately avoids rebuilding ordinary HTMX behavior.

For supported interactions, HTMX continues to own:

- request execution;
- normal response processing;
- swaps;
- out-of-band processing;
- settle behavior;
- its documented lifecycle.

Gesso owns the semantic constraints layered around that behavior.

This distinction is valuable because it keeps the trusted integration contract small enough to document and test.

If a future HTMX version changes the lifecycle contract, the supported-contract adapter may change. Choreo/browser semantic invariants should not be rewritten merely because an external library renamed events or changed implementation mechanisms.

Supported HTMX behavior is a named assumption

The framework should define which externally documented HTMX behaviors it relies on.

Anything outside that supported subset may be:

- unverified;
- rejected;
- explicitly unsupported;
- admitted only under a weaker stated guarantee.

The system must not silently claim Gesso's stronger browser invariants while allowing configuration that bypasses the lifecycle gates on which those invariants depend.

Browser FFI is intentionally dumb

The browser FFI may know how to:

- start/cancel a timer;
- install provisional DOM;
- capture/restore continuity;
- observe a raw event;
- ask HTMX to cancel an operation through a supported hook;
- install/remove a listener;
- report physical failure.

It should not know why a rollback is legal, why a resource instance is currently admitted, or why a settlement wins.

Those semantic facts are outputs of the pure transition core.

This keeps the trusted correspondence boundary narrow and makes browser races amenable to explicit state-transition reasoning.

Correlation is installed, then carried opaquely

When semantic logic creates or admits a physical resource, the resource receives the identity needed to report back safely.

For example:

```
adapter emits
    [:timer/start
      {:execution-id E
       :timer-id T
       :resource-instance I}]

shell installs timer carrying E/T/I

timer fires

shell reports
    {:event :timer/fired
     :execution-id E
     :timer-id T
     :resource-instance I}
```

The shell must not rediscover E/T/I later from:

- DOM ancestry;
- CSS selector;
- target contents;
- ”currently active” global variables;
- URL shape.

Opaque installed correlation is more reliable than semantic reconstruction from mutable browser state.

Stale and duplicate events are ordinary modeled cases

Asynchronous browsers naturally produce events that arrive after their semantic usefulness has ended.

Examples:

- request completion after execution retirement;
- timer after settlement;
- continuity callback after target replacement;
- duplicate request completion;
- old fragment response after a newer authoritative install.

The runtime should treat these as explicit event classes admitted by the safety model, not as impossible programming errors.

The ordinary result is often:

```
recognized
classified stale/duplicate
semantic no-op
optional diagnostic record
```

This is stronger than hoping cancellation prevents delivery.

Cancellation is an optimization; invalidation is the safety mechanism

Physical cancellation can race or fail.

Therefore the safety argument may not depend on proving that every old:

- timer;
- request;
- listener;
- observer;
- callback;

was physically destroyed.

Semantic retirement revokes the affected owner or resource instance's semantic admission rights.

After that point a late physical callback is powerless.

Successful physical cancellation remains desirable for efficiency and resource use, but it is not the sole correctness gate.

Execution isolation is intentionally local except for declared shared events

Most asynchronous resources are execution-private.

Shared authoritative invalidation is different.

For example, an invalidation about a logical Request fragment may legitimately matter to a newer execution even if it was triggered by another operation.

The runtime therefore distinguishes:

```
execution-private callback
    -> must satisfy owner/resource-instance gate

declared shared authoritative invalidation
    -> follows fragment/progression coordination path
```

Treating all events as either globally shared or strictly private would be incorrect.

Fragment refresh uses the declared coordinator contract

Invalidations can arrive faster than authoritative reads complete. They must enter the fragment coordinator instead of starting an independent request per advisory. That contract defines the logical request bound, preserved scoped obligations, acquisition policy, and the evidence needed for any finite-state or retained-memory claim. It is the single normative owner of those coordination rules.

Authoritative installation revokes weaker obsolete restoration rights

A stale failure callback may still know an old optimistic snapshot.

That fact does not mean restoring the snapshot remains legal.

After authoritative installation, older provisional cleanup may:

- release resources;
- record diagnostics;
- retire execution state.

It may not reinstall weaker/older representation merely because its callback finally ran.

This is prevention of obsolete restoration after authoritative installation.

The comparison is semantic, based on authority/ownership/progression relations, not numeric arithmetic over opaque basis tokens.

Semantic retirement precedes physical cleanup

Retirement has a precise order:

```
1. mark execution semantically retired
2. revoke admission rights of execution-private resource instances
3. release semantic ownership
4. permit new semantic owner where valid
5. attempt physical cleanup best effort
```

This order matters.

If physical cleanup is required before semantic retirement, one uncancellable timer or third-party request can prevent logical progress forever.

If semantic ownership remains until cleanup finishes, stale callbacks may retain power they should already have lost.

Browser security assumes arbitrary client bytes

The trusted server may not rely on:

- the generated adapter having run;
- local plan state being honest;
- execution/resource-instance correlation being truthful;
- hidden inputs being untouched;
- client role labels being valid;
- a client-provided nominal tag, record name, wrapper, or discriminator having crossed the trusted construction path it claims to represent;
- a button having existed;
- a sidecar claiming verification.

The server validates syntax/protocol structure as needed, then derives authenticated principal and current permission from trusted state and model semantics.

This adversarial model is part of the browser architecture, not an optional security-hardening mode.

Projected-plan identity, deployment, and stale continuation recovery

A projected choreography may remain in a browser across deployment.

Old or suspended browsers must not be assumed compatible with a newly deployed server forever.

The design therefore distinguishes:

- choreography representation version;
- projected-plan version;
- semantic operation identity/version where needed;
- wire/protocol version;
- semantic command identity;
- protocol execution identity;
- application/server compatibility identity where needed;
- admitted assembly generation identity where required for realization binding.

Deployment identity records placement or rollout. It is not interchangeable with assembly, execution, resource-instance, or compatibility identity.

These do not need to become one global version integer.

A compatibility check should answer whether a suspended or retried projection is safe to continue with the current server/runtime contract.

For example:

```
browser loaded projected plan P7
        |
browser sleeps
        |
deployment introduces incompatible P8/server contract
        |
browser wakes
```

The safe default is:

```
detect incompatible continuation
        |
semantically retire stale execution
and revoke its execution-private callback rights
        |
initiate best-effort physical cleanup
        |
re-establish transport/authentication
        |
authoritative reread/reload
        |
instantiate current projection
```

Instantiation of the new projection must not depend on successful destruction of every stale physical browser resource. Once semantic retirement and ownership revocation has made stale callbacks powerless, physical cleanup may complete later or fail diagnostically without restoring the old execution's ownership.

The framework should not invent transparent migration of arbitrary suspended machine state unless an explicit migration facility is designed and tested.

Compatibility failure is a recovery condition, not evidence that authoritative model state is invalid or that a pending command was rejected. Recovery must preserve the distinction between current-state reconstruction and attribution of an earlier command, following Part IV.

Changing authenticated principal or protected view context is also an admission boundary. Old pending bindings, subscriptions, or observations must not be silently treated as belonging to the new principal. The runtime must retire or revalidate the affected scopes under the current contract; the server remains responsible for every protected read and mutation.

This is particularly important for low-end Android phones whose tabs may survive sleep, backgrounding, or long network outages across deployments.

Stale plan recovery protects authoritative truth, not suspended local illusion

When a browser wakes with an incompatible plan, the goal is not to preserve every detail of its suspended local machine.

The goal is to recover safely to current authoritative truth.

The default recovery sequence is therefore intentionally destructive of stale local execution state:

```
detect incompatibility
    |
revoke stale semantic ownership
    |
revoke stale resource-instance admission rights
    |
best-effort cleanup
    |
re-establish current transport/authentication
    |
authoritative reacquisition
    |
instantiate current projection
```

A stale optimistic or continuity state may be lost.

Authoritative model truth is not invalid merely because the old projection can no longer continue.

Compatibility is explicit and scoped

Compatibility may concern different dimensions:

- executable plan format;
- protocol envelope;
- operation meaning/version;
- settlement interpretation;
- progression representation;
- browser/server runtime contract.

The framework should not collapse these into one universal monotonically increasing version integer unless a particular layer genuinely has such a coordinate.

A compatibility predicate may therefore be more structured than equality.

What matters is that continuation is admitted only when the current runtime can justify it.

Parseability is not compatibility

A new server may still understand the bytes of an old command while no longer supporting the old continuation semantics.

Thus:

```
can decode
    ≠
safe to continue
```

The compatibility check occurs at the semantic/protocol boundary, not merely at deserialization.

Transparent arbitrary state migration is a non-goal

Automatically translating arbitrary suspended projected-machine state across deployments would require:

- source/target semantic correspondence;
- state migration rules;
- resource ownership migration;
- pending effect migration;
- timer/callback treatment;
- provisional state treatment;
- proof of compatibility.

The framework must not accidentally promise this by attempting heuristic migration.

Specific cross-version migrations may be defined for high-value cases only when their compatibility semantics are explicit.

The safe baseline is authoritative reacquisition.

A sleeping checkout tab gives the recovery sequence enough detail to see why arbitrary machine migration is unnecessary.

Suppose the browser suspended while command C77 was outcome-unknown under plan P7:

```
(def sleeping-runtime
  {:plan
   {:plan/id :checkout/place-order-P7
    :representation/version 2
    :compatibility/id :store-live-v7
    :wire/protocol :gesso-wire-v7
    :assembly/generation :A12}

   :principal [:user Alice]

   :operation
   {:id :checkout/place-order
    :semantic/version 4}

   :execution
   {:execution/id
    #uuid "c1000000-0000-0000-0000-000000000007"

    :command/id
    #uuid "c7700000-0000-0000-0000-000000000077"

    :state :outcome-unknown

    :resources
    {:retry-timer
     {:resource-instance/id
      #uuid "c2000000-0000-0000-0000-000000000031"
      :status :active}}

    :projection
    {:authority :provisional
     :order/status :submitting}}})
```

While the tab sleeps, deployment A13 introduces P8. The plan bytes may still be parseable, but the continuation contract is incompatible:

```
(def current-server-contract
  {:assembly/generation :A13

   :supported
   {:plan-representation #{3}
    :compatibility #{:store-live-v8}
    :wire-protocol #{:gesso-wire-v8}}

   :operations
   {:checkout/place-order
    {:semantic/version 5}}})
```

The wake path asks a semantic compatibility predicate, not merely the EDN parser:

```
(defn compatible-continuation?
  [runtime server]

  (let [op-id (get-in runtime [:operation :id])
        op-version (get-in runtime [:operation :semantic/version])
        server-op-version
        (get-in server [:operations op-id :semantic/version])]
    (and
    (some? op-id)
    (some? op-version)
    (some? server-op-version)
    (contains?
      (get-in server
        [:supported :plan-representation])
      (get-in runtime
        [:plan :representation/version]))

    (contains?
      (get-in server
        [:supported :compatibility])
      (get-in runtime
        [:plan :compatibility/id]))

    (contains?
      (get-in server [:supported :wire-protocol])
      (get-in runtime [:plan :wire/protocol]))

    (= op-version server-op-version))))
```

Assume it returns false. Recovery first revokes old semantic ownership:

```
(defn retire-incompatible-execution
  [runtime]

  (→ runtime
      (assoc-in
        [:execution :status]
        :retired)

      (assoc-in
        [:execution :retired-reason]
        :incompatible-plan)

      (update-in
        [:execution :resources]
        (fn [resources]
          (into {}
            (map
              (fn [[k resource]]
                [k
                 (assoc resource
                   :status :retired
                   :callback-rights :revoked)]))
            resources)))

      (assoc
        :recovery
        {:state :needs-current-authority
         :preserved-command
         (get-in runtime
           [:execution :command/id])})))
```

Physical timer cancellation happens afterward:

```
(defn cleanup-retired-runtime!
  [browser runtime]

  (doseq [[_ resource]
          (get-in runtime
            [:execution :resources])]

    (try
      (physical/cancel-resource!
        browser
        resource)

      (catch Throwable t
        (try
          (diagnostics/report!
            {:error/type :cleanup/failed
             :resource resource
             :cause t})
          (catch Throwable _
            nil)))))

  runtime)
```

If the browser later receives the old retry timer callback, the callback is powerless because its resource instance was semantically retired even if physical cancellation failed:

```
(def old-timer-event
  {:event :timer/fired

   :execution-id
   #uuid "c1000000-0000-0000-0000-000000000007"

   :resource-instance/id
   #uuid "c2000000-0000-0000-0000-000000000031"})

(admit-async-event
  retired-runtime
  old-timer-event)
;; ⇒
{:result :rejected
 :diagnosis :stale-resource}
```

Recovery then re-establishes current authentication and reads authority:

```
(defn recover-checkout!
  [{:keys [session] :as ctx}
   retired-runtime]

  (let [principal
        (auth/current-principal!
          session)

        command-id
        (get-in retired-runtime
          [:recovery :preserved-command])

        command-resolution
        (order-model/resolve-command
          ctx
          {:principal principal
           :command-id command-id})

        current-cart-or-order
        (order-model/read-current-recovery-view
          ctx
          {:principal principal
           :command-id command-id})]

    {:principal principal

     :command-resolution
     command-resolution

     :authoritative-view
     current-cart-or-order

     :current-plan
     (artifact/require-current-plan!
       ctx
       :checkout/place-order)}))
```

There are several legitimate outcomes.

If durable command evidence establishes that C77 committed:

```
{:command-resolution
 {:command/id C77
  :disposition :committed
  :order/id O900}

 :authoritative-view
 {:order/id O900
  :order/status :placed}

 :current-plan
 {:plan/id :checkout/place-order-P8
  :compatibility/id :store-live-v8}}
```

the new runtime can instantiate an authoritative settled presentation under P8.

If current Order state is known but command attribution is not:

```
{:command-resolution
 {:command/id C77
  :disposition :unknown}

 :authoritative-view
 {:order/id O900
  :order/status :placed}

 :current-plan
 {:plan/id :checkout/place-order-P8
  :compatibility/id :store-live-v8}}
```

the browser can still render current authoritative Order state while preserving the fact that C77's causal disposition is unresolved.

If no current compatible artifact can be loaded:

```
{:recovery/state :unavailable
 :reason :current-plan-unavailable
 :retry
 {:policy :explicit-backoff
  :after-ms 5000}}
```

the runtime must not fall back to P7 merely because it still has the bytes.

The example also exposes the principal-change boundary. If the tab slept as Alice and wakes after Bob has authenticated in the same browser profile, P7's old protected observations and subscriptions do not become Bob's:

```
(when-not
  (=
    (:principal sleeping-runtime)
    (auth/current-principal session))

  (recovery/retire-protected-scopes!
    sleeping-runtime
    {:reason :principal-changed}))
```

The new principal acquires a new authorized projection through the normal trusted read path.

The safe recovery goal is therefore intentionally modest: recover current authority under a current compatible plan. Preserving arbitrary suspended local machine detail is optional future functionality, not a prerequisite for robust deployment recovery.

Same command may survive when execution cannot

A semantic command and its current projected execution have different identity.

An old execution may be retired while the application still recognizes the same semantic command for reconciliation or retry.

For example:

```
command C42 persists semantically

old:
    plan P7
    execution E7

deployment

new:
    plan P8
    execution E8

C42 may be reconciled/retried if operation policy permits
```

This is one reason command identity, execution identity, and plan identity must remain separate.

Recovery must itself obey authorization and progression rules

Authoritative reacquisition after stale-plan detection is not a privileged backdoor.

The new read/subscription path must still:

- bind the authenticated principal;
- authorize protected reads;
- satisfy required progression semantics;
- install only current compatible projections.

Recovery changes runtime topology, not security policy. It also does not waive explicit adoption policy: reconstructing a board after reconnect must follow the declared policy for its existing observations and remote membership. If a full reload intentionally establishes a new initial observation, that is a separately declared acquisition boundary rather than a hidden advisory-triggered adoption.

Recovery must handle failure to acquire a compatible plan or authoritative read without repeatedly restarting an incompatible continuation. The unavailable or retry state and the conditions for another attempt must be explicit. A missing current artifact is not permission to execute the stale plan.

Offline duration does not imply replay obligation

A browser offline through many authoritative changes need not replay every intermediate invalidation or projection.

If application semantics permit it, recovery may jump directly to a sufficiently current authoritative observation.

This keeps Live advisory and convergence semantics compatible with very long sleep/offline periods.

Browser managed effects

A choreography execution may cause browser implementation effects with lifetimes longer than one pure machine step.

These must be owned explicitly.

Semantic ownership decisions belong to the pure browser adapter transition core. The physical browser effect shell may allocate/cancel concrete resources only in response to abstract effects emitted by that core; it must not maintain a second competing semantic ownership model.

No supported execution or shared browser activity may create ongoing managed browser resources without registering their declared owner with the runtime. This requirement concerns Gesso-managed effects, not every unrelated browser resource created by the surrounding application.

Effect categories may include:

- **resource** – must be released;
- **reversible** – may be compensated while still safe;
- **authoritative** – records installed canonical state and must never be automatically rolled back merely because later cleanup failed;
- **async** – timers/callback sources that must be cancelled or invalidated;
- **ephemeral** – recorded for diagnostics but leaves no ongoing resource.

An execution should have one managed effect scope.

Examples of owned browser effects:

- optimistic target lock;
- settlement timer;
- request correlation;
- continuity slot/transaction;
- provisional DOM state;
- pending browser state;
- callback/event subscriptions associated with the execution.

Important invariants include:

```
every active managed effect scope has an admitted execution or shared owner
every active execution's managed effects belong to its declared effect scope
every managed timer/lock/continuity object has declared owner and instance identity
retired execution retains no execution-private semantic ownership or callback authority
physical remnants after retirement are cleanup debt only
```

Catastrophic cleanup should be:

- reverse acquisition order where required by the declared resource dependencies;
- idempotent;
- best effort;
- continuing through remaining permitted cleanup actions even when one throws;
- diagnostic about cleanup failures;
- subordinate to the original failure;
- forbidden from undoing already-established authoritative DOM merely because a later cleanup failed.

Stale-plan retirement uses the same ownership rules: an incompatible old execution must first lose semantic ownership and its execution-private callback rights. A new projection may then assume the logical target without waiting for every old physical effect to be destroyed. Cleanup of those stale physical effects is best effort, and any callbacks they later emit remain powerless because they carry retired execution or resource-instance correlation.

One effect scope per active execution

An active projected execution should have one semantic effect scope that records the resources and reversible/provisional obligations it owns.

Conceptually:

```
Execution E
    |
    +--> target ownership
    +--> provisional projection
    +--> settlement timer
    +--> request correlation
    +--> continuity slot
    +--> subscriptions/callback registrations
```

The exact representation may be compact.

The important property is that resource ownership is explicit and queryable by the semantic transition system.

Effect category affects legal cleanup

Not every effect supports the same compensation.

A useful conceptual distinction is:

- resource – requires release/invalidation;
- reversible – may be compensated while the compensation remains semantically legal;
- authoritative – must never be undone by local cleanup merely because later delivery failed;
- async – creates future callbacks whose semantic admission rights must be revocable independently of best-effort physical cancellation;
- ephemeral – has no ongoing semantic ownership.

An implementation may encode categories differently.

What matters is that cleanup logic cannot blindly run ”undo” on every historical effect.

Rollback rights expire

An optimistic execution may initially own the right to restore a captured local snapshot if its request fails before stronger authority supersedes it.

That right is conditional.

It expires when, for example:

- a newer authoritative projection installs;
- another current execution legitimately owns the target;
- the relevant execution or resource instance is retired;
- reconciliation chooses a different canonical representation.

Therefore rollback is a semantic operation guarded by current ownership, not a generic exception handler.

Catastrophic cleanup preserves the original semantic failure

If cleanup action 1 throws, cleanup actions 2 through N should still be attempted.

Cleanup exceptions are diagnostics subordinate to the original failure that triggered cleanup.

A later cleanup error must not rewrite history so the system appears to have failed for a different semantic reason.

Conceptually:

```
primary failure F
    |
semantic retirement
    |
cleanup A  -> fails CA
cleanup B  -> succeeds
cleanup C  -> fails CC

reported:
    primary F
    cleanup diagnostics [CA CC]
```

Cleanup is idempotent

The runtime should tolerate repeated cleanup attempts because:

- timeout and request failure may race;
- browser lifecycle recovery may repeat retirement;
- exception handling may retry cleanup;
- physical cancellation may be uncertain.

Idempotent physical cleanup plus owner/resource-instance admission safety is preferable to fragile exactly-once cleanup assumptions.

Physical remnants are observable debt, not hidden success

When cleanup cannot destroy a resource, the runtime should retain enough diagnostic information to explain the remnant.

It should not silently claim complete physical cleanup. Semantic isolation does not prove a bound on physical leaks. The operational contract must define how retained diagnostics and remnant resources are bounded, reclaimed, or reported as an unrecoverable local resource condition where repeated failure would otherwise exhaust the browser. A resource limit may suspend new work through a declared unavailable state; it must not revive obsolete ownership.

At the same time, the semantic system may proceed once the resource's callback authority has been revoked.

This separation lets the framework remain safe under imperfect browsers and third-party runtimes without concealing operational problems.

Shared resources require an explicit shared ownership class

The ”one owner” rule applies to execution-private resources.

Some resources may be intentionally shared, for example a fragment-level coordinator or shared Live connection.

Such sharing must be represented explicitly.

An unowned global variable that several executions happen to mutate is not a shared ownership model.

The semantic design needs to know the class of sharing well enough to prevent an execution's retirement from destroying resources still required by legitimate owners.

Async event safety

Every asynchronous event delivery must first be normalized at the browser FFI boundary and then pass the semantic adapter safety gate. There must not be an alternate callback path that directly mutates execution state or resumes the portable machine.

The common gate follows Part II's owner-relative admission rule. For an execution-private event that resumes a projected machine, the runtime must establish:

- the execution still exists;
- correlation identifies the admitted execution instance and the concrete resource instance whose event may resume it;
- the resource that produced the event is still owned;
- the event is accepted in the machine's current state;
- where relevant, the projected plan is still compatible with the runtime/server contract.

Otherwise the event is stale, duplicate, or incompatible and is ignored, converted into recovery, or recorded diagnostically.

Retirement forbids resuming the retired owner; it does not prove that the associated command failed or that every later authoritative result is useless. Command-outcome or authoritative evidence may enter through an independently declared admission path without reactivating the old execution or granting its callbacks permission to mutate replacement state. Normalization cannot invent that alternate path by relabeling a rejected callback as a shared event.

All of the following event sources use the common admission boundary, with the checks required by their declared ownership class:

- timers;
- HTMX lifecycle callbacks;
- request failures;
- settlements;
- Live/canonical supersession;
- continuity callbacks;
- fragment request completions;
- Refresh/authoritative-acquisition completions and their captured requirements;
- browser lifecycle recovery;
- other asynchronous browser events.

If machine/FX execution throws unexpectedly:

- mark the execution failed;
- never restore an old pre-drive machine as though the failed transition did not occur;
- revoke the failed owner's semantic permits and retire the affected execution or admitted coordination scope before attempting physical cleanup;
- attempt managed cleanup without reviving obsolete ownership;
- retain failure history and cleanup diagnostics;
- report/rethrow the original error.

When settlement or authoritative supersession wins, settlement timeout ownership should be cancelled immediately before slow/local rendering work such as continuity restoration.

One admission gate for every asynchronous callback

The browser must not have one safety gate for timers, another informal rule for HTMX, and a third special case for continuity.

Every asynchronous callback that can influence Gesso semantic state passes through the same conceptual admission sequence:

```
normalize callback
    |
resolve declared event class, owner, and correlation
    |
owner admitted and applicable?
    |
resource instance / execution identity current where required?
    |
resource still semantically owned under that class?
    |
event legal in the owning execution or coordinator state?
    |
scope / progression / plan compatibility satisfied where required?
    |
accept transition
```

Any failure before the final step means the callback cannot directly perform the proposed semantic mutation. The gate may emit the specified diagnostic or recovery transition. It must not redispatch a rejected private event as shared to bypass the failed check.

An idle fragment may accept an admitted shared advisory with no command execution. A stale subscription instance may not impersonate the replacement. An old settlement may inform a current command-resolution process only through its separately admitted command-evidence path; it does not resume the retired execution whose private callback was rejected.

Three callbacks arriving at the same browser can demonstrate why owner class must be resolved before event admission.

Assume execution E1 has been retired and replaced by E2:

```
(def browser-state
  {:executions
   {E1
    {:status :retired
     :command/id C42
     :resources
     {:timeout
      {:resource-instance/id I1
       :status :retired}}}

    E2
    {:status :active
     :command/id C77
     :resources
     {:transport
      {:resource-instance/id I2
       :status :active}}}}

   :shared
   {:board-subscription
    {:subscription/id S2
     :resource-instance/id I3
     :status :active
     :scope
     {:organization-id O
      :projection :request/board}}}

   :command-resolution
   {C42
    {:status :awaiting-evidence
     :principal [:user Alice]
     :subject {:request/id R}}}})
```

First, E1's timeout fires late:

```
(def late-timeout
  {:event/class :execution-private

   :event :timer/fired

   :owner
   [:execution E1]

   :resource-instance/id I1

   :timer/id :claim-settlement})
```

The event is stale because its declared owner and resource are retired:

```
(admit-event
  browser-state
  late-timeout)
;; ⇒
{:result :rejected
 :diagnosis :stale
 :owner [:execution E1]
 :resource-instance/id I1}
```

The adapter may log that result, but it cannot reinterpret the callback as belonging to E2 merely because both executions touched the same Request card.

Second, the current shared board subscription delivers an advisory while no command execution is relevant:

```
(def board-advisory
  {:event/class :shared

   :event :live/invalidated

   :owner
   [:subscription S2]

   :resource-instance/id I3

   :scope
   {:organization-id O
    :projection :request/board}

   :requirement
   {:kind :newer-authority-available
    :minimum-basis :X90}})
```

That event can be admitted through the subscription owner:

```
(defn admit-shared-subscription-event
  [state event]

  (let [[_ subscription-id]
        (:owner event)

        current
        (get-in state
          [:shared :board-subscription])]

    (cond
      (not=
        (:subscription/id current)
        subscription-id)
      {:result :rejected
       :diagnosis :stale-subscription}

      (not=
        :active
        (:status current))
      {:result :rejected
       :diagnosis :retired-shared-owner}

      (not=
        (:resource-instance/id current)
        (:resource-instance/id event))
      {:result :rejected
       :diagnosis :stale-subscription-instance}

      (not
        (scope/compatible?
          (:scope current)
          (:scope event)))
      {:result :rejected
       :diagnosis :wrong-shared-scope}

      (not
        (advisory/admitted-for-current-subscription?
          state event))
      {:result :rejected
       :diagnosis :unadmitted-advisory}

      :else
      {:result :accepted
       :effects
       [[:fragment/record-advisory
         {:requirement
          (:requirement event)}]]})))

(admit-shared-subscription-event
  browser-state
  board-advisory)
;; ⇒ accepted
```

No fake execution ID is needed.

Third, suppose a trusted settlement response for old command C42 arrives after E1 retired. It contains useful command-specific evidence:

```
(def late-command-evidence
  {:event/class :command-evidence

   :event :command/result-observed

   :command/id C42

   :evidence
   ;; Fixture issued by the trusted command-result boundary.
   (command-evidence/admitted-fixture
     {:command/id C42
      :principal [:user Alice]
      :subject {:request/id R}
      :disposition :committed
      :request
      {:request/id R
       :request/status :claimed
       :request/helper-id Alice}})})
```

It must **not** be admitted as E1's private continuation:

```
(admit-execution-private-event
  browser-state
  (assoc late-command-evidence
    :owner [:execution E1]))
;; ⇒ rejected :stale
```

But the separately declared command-resolution owner may consume it:

```
(defn admit-command-evidence
  [state event]

  (let [command-id
        (:command/id event)

        resolver
        (get-in state
          [:command-resolution command-id])]

    (cond
      (nil? resolver)
      {:result :rejected
       :diagnosis :unknown-command-resolution-owner}

      (not= :awaiting-evidence (:status resolver))
      {:result :rejected
       :diagnosis :already-resolved-or-retired}

      (not
        (command-evidence/admitted-for-current-owner?
          state resolver event))
      {:result :rejected
       :diagnosis :unadmitted-command-evidence}

      :else
      {:result :accepted

       :state
       (assoc-in state
         [:command-resolution command-id]
         {:status :resolved
          :evidence (:evidence event)})

       :effects
       [[:recovery/reconcile-current-presentation
         {:command-id command-id
          :evidence (:evidence event)}]]})))

(admit-command-evidence
  browser-state
  late-command-evidence)
;; ⇒ accepted
```

Useful information survived retirement. E1's transition rights did not. The evidence predicate checks the command, authenticated principal, subject, outcome class, and current resolver; copying a `:source` label is insufficient. The reconciliation effect may update presentation only after separately checking the **current** target owner and its authorized observation. The old execution's settlement alone does not grant a retired DOM permit.

The common conceptual gate can therefore begin with classification:

```
(defn admit-event
  [state event]

  (case (:event/class event)

    :execution-private
    (admit-execution-private-event
      state
      event)

    :shared
    (admit-shared-subscription-event
      state
      event)

    :command-evidence
    (admit-command-evidence
      state
      event)

    {:result :rejected
     :diagnosis :unknown-event-class}))
```

This is more precise than a generic `current-generation?` check. The same physical target can host execution-private work, fragment-level coordination, and command-resolution evidence with different semantic owners and different admission rules.

It also prevents a dangerous fallback:

```
;; Forbidden:
(or
  (admit-execution-private-event state event)
  ;; "maybe it was shared?"
  (admit-shared-subscription-event
    state
    (assoc event
      :event/class :shared)))
```

A rejected private callback does not earn a second chance by relabeling itself. The event class and owner are part of the declared integration contract.

Unknown, stale, duplicate, and incompatible are distinct diagnoses

All may result in ”do not resume the machine,” but they describe different conditions.

- unknown – no recognized owner/resource correlation exists;
- stale – correlation refers to a retired execution or replaced/retired resource instance;
- duplicate – equivalent event was already consumed;
- incompatible – continuation exists but current plan/runtime contract forbids it;
- illegal-state – event belongs to an admitted owner but is not admissible from that owner's current execution or coordination state.

Keeping these classifications separate improves diagnostics and testing.

Late callback safety must not rely on ordering luck

JavaScript event scheduling, network completion, HTMX lifecycle ordering, and browser suspension make many orderings possible.

The design must therefore be correct for adversarial scheduling within the documented external contracts.

Tests should deliberately permute:

- response versus timeout;
- canonical install versus old request failure;
- execution retirement versus continuity completion;
- invalidation versus fragment response;
- advisory arrival during an in-flight Refresh;
- newer local authoritative installation versus an older Refresh response;
- reconnect versus old callback;
- duplicate external events;
- deferred physical effect versus owner retirement;
- shared advisory arrival while no command execution exists;
- explicit Refresh permission versus later advisory arrival;
- partial installation failure versus continuity completion;
- authentication-context replacement versus old subscription or request events.

The proof/model abstraction should classify events by semantically relevant relations such as admitted/retired owner and current/replaced resource instance, rather than a few chosen concrete integers. A resource can be replaced without retiring its execution, and different owners can reuse a logical target without sharing admission rights. A sound abstraction must preserve these distinctions where they affect the property.

Unexpected semantic-machine failure retires rather than rewinds

If the semantic machine throws while driving a transition, the runtime must not restore an earlier machine object and pretend no transition was attempted.

The safe response is:

```
record primary failure
    |
retire affected semantic owner / revoke effect permits
    |
invalidate private resource correlations
    |
cleanup physical resources best effort
    |
surface original failure
```

Rewinding arbitrary partial semantic execution would require explicit transactional semantics that the browser machine does not claim to provide. The runtime may retain the last committed state for diagnosis and recovery, but must not use it to replay possibly issued effects without the effect contract's duplicate-handling guarantees. A failure after effect dispatch must account for the possibility that physical work already occurred.

Winning terminal events cancel competing timeout authority immediately

When a settlement or authoritative supersession wins, any settlement timeout must lose semantic ownership before slower local presentation work runs.

For example:

```
settlement accepted
    |
cancel/invalidate timeout ownership
    |
then perform continuity / DOM follow-up
```

Otherwise a timeout may fire during a slow local effect and incorrectly compete with an already resolved semantic outcome.

Duplicate delivery is part of the protocol environment

External systems may duplicate:

- SSE messages;
- request completions;
- browser events;
- timer observations after cancellation races.

Where a duplicate is semantically harmless, the transition system should make that idempotence explicit.

Where it is not harmless, correlation/state must make the second delivery rejectable.

”Should only happen once” is not a sufficient browser invariant.

Authoritative absence

The runtime must be able to represent that an authoritative result establishes absence of a previously rendered target within a declared projection scope. That does not automatically establish nonexistence of the domain entity.

A missing canonical target is not always an error.

For example, a Request may legitimately disappear from a filtered board because it became terminal or ceased to match the current view.

The semantic protocol therefore needs a notion equivalent to authoritative absence/tombstone for projections.

The browser realization may then remove the corresponding DOM safely while preserving the choreography's authority semantics.

An actor can acquire authoritative knowledge of absence in the same sense that it can acquire authoritative knowledge of presence.

Presence and absence are both authoritative projection outcomes

A projection result should distinguish:

```
present authoritative value
authoritative absence
not yet acquired
failed to acquire
```

These are not equivalent.

In particular:

```
empty DOM
```

does not by itself tell the runtime whether the authoritative target is absent or simply has not been loaded.

This distinction is necessary for both initial acquisition and safe removal.

Authoritative absence may legitimately remove local representation

Examples include a Request that:

- becomes terminal and is filtered out;
- moves to another board scope;
- no longer satisfies the current search/filter projection;
- is deleted/withdrawn under domain policy where such absence is authoritative.

The browser may remove the representation when the authoritative projection establishes absence.

That removal has the same authority semantics as installing an authoritative present representation.

Absence supersedes obsolete provisional presence

If a provisional execution rendered a target and authority later establishes that the target is absent from the relevant projection, the provisional target must not survive merely because it has a physical DOM node.

Likewise a stale rollback cannot resurrect the target after authoritative absence has been installed.

This is another instance of prevention of obsolete restoration after authoritative installation.

Absence is scoped to a projection

Authoritative absence from one projection does not necessarily mean the domain entity ceased to exist.

For example:

```
Request R absent from
    "open requests visible to this helper"

does not imply

Request R does not exist
```

The provenance of absence therefore includes the authoritative projection/read whose result established it, including the completeness and consistency needed to justify the negative result. Access denial, a partial page, a failed request, or missing HTML is not interchangeable with authoritative absence.

Local presentation filtering is also distinct. Hiding a terminal card using an option over already-adopted data does not necessarily perform a new acquisition or revoke adopted membership. Re-showing it must use retained justified content or a permitted acquisition, not resurrect a target whose authoritative absence or ownership supersession has since been established. It must not admit unrelated remote membership merely because a filter changed.

This keeps filtering/view semantics separate from entity existence semantics.

A HumanHelp board can exercise all four states – present, authoritatively absent, not yet acquired, and failed acquisition – with the same Request ID.

Assume Bob currently displays Request R17 because his adopted authoritative board contained it:

```
(def bob-board
  {:projection
   {:kind :request/board
    :organization-id 0
    :filter :open-for-helper
    :coverage :complete-authorized-membership}

   :adopted
   {:basis :X100

    :members
    {R17
     {:status :present
      :value
      {:request/id R17
       :request/status :open
       :request/title "Move a cabinet"}
      :provenance :authoritative}}}})
```

Alice later completes R17. A complete authorized reacquisition of the **same projection** may return an explicit absence result:

```
(def board-result-X105
  {:provenance :authoritative

   :basis :X105

   :scope
   {:kind :request/board
    :organization-id 0
    :filter :open-for-helper
    :coverage :complete-authorized-membership}

   :members
   {R17
    {:status :absent
     :reason :does-not-match-projection}

    R22
    {:status :present
     :value
     {:request/id R22
      :request/status :open
      :request/title "Need help installing shelves"}}}})

(def admitted-board-X105
  ;; Fixture issued by authorized, complete acquisition for Bob's
  ;; :open-for-helper projection. A copied :provenance field is insufficient.
  (acquisition/admitted-fixture
    board-result-X105
    {:principal [:user Bob]
     :scope (:scope board-result-X105)
     :covered-through :X105}))
```

That gives the adapter authority to remove R17 from the **open-for-helper board**:

```
(defn install-member-outcome
  [board request-id admitted-observation]
  (acquisition/require-current-authorized-complete!
    board request-id admitted-observation)
  (let [observation (acquisition/value admitted-observation)
        outcome (get-in observation [:members request-id])]
    (when-not outcome
      (throw (ex-info
               "Complete acquisition did not justify this member outcome."
               {:request/id request-id})))
    (case (:status outcome)

      :present
      (assoc-in board
        [:adopted :members request-id]
        {:status :present
         :value (:value outcome)
         :provenance :authoritative
         :presentation-status :pending-realization
         :basis (:basis observation)})

      :absent
      (assoc-in board
        [:adopted :members request-id]
        {:status :absent
         :reason (:reason outcome)
         :provenance :authoritative
         :presentation-status :pending-realization
         :basis (:basis observation)
         :scope (:scope observation)})

      (throw
        (ex-info
          "Unsupported projection-member outcome."
          {:outcome outcome})))))

(def after-X105
  (install-member-outcome
    bob-board
    R17
    admitted-board-X105))
```

This call records one justified member outcome and an outstanding physical realization obligation. It does not claim that any card has yet been removed from the DOM, and it cannot advance the board-wide adopted basis until that separate incorporation succeeds. The admission check enforces the current target owner, Bob's authorization, matching projection/filter, complete membership coverage, and progression relative to the installed outcome.

The DOM effect can then remove the card:

```
[[:dom/remove-authoritative-member
  {:target [:request-card R17]
   :permit
   (adapter/issue-current-install-permit
     after-X105 [:request-card R17])
   :reason :authoritative-projection-absence
   :basis :X105
   :scope
   {:kind :request/board
    :organization-id 0
    :filter :open-for-helper}}]]
```

The effect shell checks the adapter-issued permit and target owner immediately before removal. Completion clears `:presentation-status :pending-realization`; failure preserves a repair obligation and cannot enable an old snapshot to reinsert R17.

This does **not** prove that Request R17 ceased to exist. Bob may still be allowed to open a different ”completed Requests” projection where R17 is present:

```
(def completed-board-result
  {:provenance :authoritative

   :basis :X105

   :scope
   {:kind :request/board
    :organization-id 0
    :filter :completed-by-helper
    :coverage :complete-authorized-membership}

   :members
   {R17
    {:status :present
     :value
     {:request/id R17
      :request/status :completed
      :request/title "Move a cabinet"}}}})
```

The two observations are compatible because their scopes differ.

Now contrast authoritative absence with ”not yet acquired.” A freshly mounted details panel might start as:

```
{:projection
 {:kind :request/details
  :request/id R17}

 :acquisition
 {:status :not-started}

 :content
 {:status :not-yet-acquired}}
```

An empty DOM container at this point conveys no negative domain fact.

Contrast both with a failed read:

```
{:projection
 {:kind :request/details
  :request/id R17}

 :acquisition
 {:status :failed
  :error :network-unreachable}

 :content
 {:status :unknown}}
```

The network error does not authorize removal or a ”Request does not exist” message.

A server may also return an authorization failure:

```
{:projection
 {:kind :request/details
  :request/id R17}

 :acquisition
 {:status :rejected
  :reason :not-authorized}

 :content
 {:status :not-disclosed}}
```

That is still not authoritative absence of R17. It establishes that this principal may not acquire that protected projection under the current policy.

Finally, suppose an optimistic local workflow had provisionally inserted R30 into the open board but the authoritative complete board says R30 is absent from that projection:

```
(def provisional-R30
  {:request/id R30
   :authority :provisional
   :execution/id E99
   :command/id C99
   :request/status :creating})

(def authoritative-R30-absence
  {:status :absent
   :provenance :authoritative
   :basis :X110
   :scope
   {:kind :request/board
    :organization-id 0
    :filter :open-for-helper
    :coverage :complete-authorized-membership}})

(def admitted-R30-absence
  (acquisition/admitted-fixture
    authoritative-R30-absence
    {:principal [:user Bob]
     :scope (:scope authoritative-R30-absence)
     :covered-through :X110}))
```

Installing that absence retires the obsolete provisional presence for the covered projection:

```
(defn install-authoritative-absence
  [runtime request-id admitted-absence]
  (acquisition/require-current-authorized-complete-absence!
    runtime request-id admitted-absence)
  (let [absence (acquisition/value admitted-absence)
        provisional (get-in runtime [:provisional request-id])]
    (cond→ (→ runtime
               (assoc-in [:members request-id] absence)
               (update :provisional dissoc request-id))
      (some? (:execution/id provisional))
      (update :resources
        retire-resources-owned-by
        [:execution (:execution/id provisional)]))))
```

A late callback from C99 cannot reinsert the card merely because its DOM node or snapshot still exists.

This example is why an absence outcome needs provenance, scope, and acquisition completeness just as a present value does. ”Nothing is here” is only a semantic fact when an authoritative projection actually established that negative result.

Failure, liveness, and convergence

Choreography verification must distinguish safety from liveness.

A safety property might say:

> Every reachable machine state is allowed by the global choreography.

A liveness property might say:

> Under eventual delivery, successful authoritative observation, and continued execution, every provisional claim eventually reaches a resolving outcome.

The verifier must not claim unconditional termination merely because every reachable state has some path to a terminal state.

Liveness guarantees depend on explicit assumptions such as:

- message/event delivery where communication is required;
- authoritative dependencies remaining reachable;
- authoritative rereads eventually completing where observation is required;
- participants continuing to execute;
- clocks/timers firing;
- browser/runtime compatibility or successful recovery after incompatibility;
- absence of permanent unresolved application-defined conflict.

These assumptions should be named rather than hidden.

A continuously changing universe may prevent all actors from simultaneously observing the latest state.

The correctness target is that actor knowledge remains valid relative to its known basis and provenance and that relevant actors have a finite path toward newer authoritative knowledge.

For HumanHelp, replay of every intermediate invalidation is not required for convergence.

A client that was offline from X17 through X900 may recover by obtaining a current authoritative observation at X900 or later rather than replaying every intermediate change.

Safety, liveness, and recovery are separate claims

A safe protocol may still wait forever if an external dependency never responds.

A live protocol under favorable assumptions may still be unsafe if it can enter an invalid state before eventual recovery.

The design therefore reports these classes separately.

Representative safety claim:

```
stale execution callback can never mutate newer execution state
```

Representative conditional liveness claim:

```
if authority remains reachable and the actor continues executing,
every provisional command eventually reaches authoritative resolution
or a declared recovery terminal
```

The assumptions belong in the claim.

Path-to-terminal is not termination proof

Finding a terminal path from every state does not prove the environment will take it.

For example, a machine may have:

```
waiting
    -- settlement --> confirmed
    -- timeout ----> local recovery / outcome still unknown
```

but if neither settlement nor timer delivery is assumed eventually to occur, unconditional termination has not been established.

The verifier must not promote graph reachability into liveness.

Agreement debt may converge under quiescence

Actors can legitimately hold different authoritative frontiers while the world changes.

Where the application requires convergence, the system should make the debt visible and provide a finite route toward newer authoritative knowledge.

Under suitable assumptions such as:

- relevant authority eventually stops changing long enough;
- required communication/invalidations are eventually delivered or replaced by authoritative reacquisition;
- actors continue running;
- authoritative reads succeed;
- any user/application acquisition trigger required by policy occurs;

agreement debt should be able to fall to zero for the scoped facts that require agreement.

This is a conditional convergence statement, not a claim of instantaneous global consistency.

Replay of every intermediate change is usually unnecessary

A browser that slept through:

```
X17 ... X900
```

may often recover directly by authoritative observation at X900 or later.

This is especially appropriate when Live invalidations are advisory rather than durable semantic events.

The system needs current justified knowledge, not a reenactment of every notification packet.

Applications whose domain semantics genuinely require every event must model a durable event/log contract explicitly instead of assuming SSE replay.

Failure classes preserve authoritative commit truth

Failures should remain classified according to where they occur.

At minimum:

```
failure established before commit submission, or confirmed rejection of attempt
    this attempt did not commit its proposed transition

commit submitted but result unavailable
    outcome unknown to this participant

authoritative commit succeeded
then propagation failed
    mutation occurred
    delivery/recovery debt remains

browser local effect failed
    authoritative truth unchanged
    local execution may recover/retire

stale/incompatible execution detected
    authoritative truth remains valid
    local projection reacquires
```

Collapsing these categories into a generic ”operation failed” destroys the information needed for safe recovery.

These categories describe authoritative reality when that reality is actually known at the observing boundary. They must not be inferred solely from a participant's transport symptom. A timeout, connection loss, or missing response may leave that participant unable to establish whether authoritative commit occurred. The command/optimism layer's participant-relative uncertainty therefore remains in force here: runtime classification must not turn unknown commit disposition into either confirmed rejection or confirmed incorporation.

Recovery must not fabricate success

A recovery path may:

- reread authority;
- supersede provisional state;
- instantiate a current plan;
- retry publication;
- reconcile a semantic command.

It must not report success for an authoritative transition that did not occur.

Similarly it must not report authoritative failure merely because local display or notification failed after commit.

When command disposition remains unresolved, recovery may restore a correct authoritative presentation without claiming that the participant has learned which command produced that state. Presentation convergence, command attribution, and authoritative command disposition remain separate conclusions.

Runtime fault and semantic outcome remain different

A model may legitimately return semantic outcomes such as:

- rejected;
- already incorporated;
- conflict/reconciled;
- no longer applicable.

Those are not runtime failures.

Conversely:

- broken HTMX contract;
- malformed response envelope;
- browser FFI exception;
- impossible adapter state;
- unsupported plan version;

are runtime/integration failures, not domain outcomes.

Diagnostics and tests should keep the distinction explicit.

Convergence does not require automatic UI adoption

An actor may learn that newer authority exists without immediately adopting that projection into visible UI.

HumanHelp deliberately uses this distinction for remote creation. Its policy for remote lifecycle changes to already-admitted Requests remains the explicit product decision recorded in the HumanHelp part; generic browser semantics do not choose that policy.

Therefore a convergence path may include:

```
advisory knowledge of newer authority
    |
user/application chooses refresh
    |
authoritative reacquisition
    |
visible adoption
```

The system can remain semantically correct during the interval as long as the actor does not perform behavior requiring knowledge it has not adopted and the application exposes the required path toward current authority.

Adoption remains scoped. Installing one newer authoritative observation does not itself admit unrelated remote membership changes, clear unrelated advisory debt, or establish that every displayed object belongs to one common snapshot. A Refresh indication may clear only when the authoritative acquisition actually satisfies the obligations whose presence caused that indication.

Runtime/browser guarantees are layered

The strongest useful statement combines several assurance kinds:

```
proved / mechanically checked:
    portable transition invariants
    execution isolation
    execution/resource-instance ownership invariants
    explicitly stated logical coordination bounds and requirement preservation
    prevention of obsolete restoration after authoritative installation
    projection/refinement properties for supported subset

runtime-enforced premises:
    event correlation
    owner/resource-instance admission checks
    plan compatibility
    schema validation
    progression decoding

trusted/external assumptions:
    browser primitives
    supported HTMX lifecycle
    scheduler/network liveness where claimed
    authentication/model/XTDB contracts
```

Real Chromium testing remains essential because it challenges the bindings and external assumptions that the pure semantic model deliberately does not prove.

Browser realization invariant

At the end of the portable/browser runtime layer, the unified design requires:

> Production executes one canonical projected semantic plan through a pure portable transition core. Browser integrations normalize observations and realize abstract effects without independently deciding semantics. Every asynchronous event is admitted according to its declared owner, resource instance, correlation, legal state, and applicable scope/compatibility checks. Shared events do not require an unrelated execution. Deferred physical effects cannot use expired permits to mutate a newer representation. Semantic retirement makes stale physical resources powerless before best-effort cleanup. New authoritative installations cannot be undone by obsolete provisional callbacks, and scoped authoritative installation discharges only advisory obligations actually covered by its acquisition. Older Refresh responses cannot erase newer justified state or silently convert a mixed authoritative view into a fictitious common snapshot. Transport/runtime failure does not invent command rejection or incorporation when disposition remains unknown. Incompatible stale plans recover through authoritative reacquisition rather than heuristic state migration. Proof/diagnostic artifacts may explain execution but can never affect it, and liveness/convergence claims remain explicitly conditional on their named environment assumptions.

Part VII — Whole-application construction

This part defines the canonical application assembly and the production boundaries that enforce its closure. It connects the semantic and runtime contracts of Parts I–VI to generated artifacts, actual routes, prepared server operations, acquisition paths, rendering, and lifecycle. Structural closure, production enforcement, and independent product completeness retain their distinct evidence and scope.

Whole-application realization and preflight closure

A verified choreography, a correct model operation, a valid browser projection, and a working HTTP route do not by themselves constitute a correct application.

The complete application has another structural correctness boundary:

> Every semantic operation that the application exposes must have one coherent, checked path from authored affordance through executable browser/server realization to authoritative mutation or observation and back to a renderable authoritative result.

This boundary is called **whole-application closure**.

Whole-application closure exists because individually valid components can be assembled into a system that is structurally impossible.

Examples include:

- a view exposes a valid semantic operation for which no browser plan is shipped;
- a browser plan exists but the generated browser artifact is stale;
- a plan emits a command transport for which no trusted route exists;
- a trusted route exists but does not resolve the declared semantic operation;
- a server operation exists but its execution environment cannot supply the required clock, transaction, principal, basis, or other capability;
- a model operation commits correctly but no declared settlement/progression path can reacquire the successor authoritative state;
- a Live fragment subscribes to invalidation but has no initial authoritative knowledge and no permitted acquisition route;
- a view silently degrades a Choreo-managed optimistic action into an ordinary HTMX request when a required basis is absent;
- tests construct a shadow registry that is internally coherent while the production application uses a different incompatible assembly.

None of those failures is necessarily visible inside the individual component that is locally correct.

The unified design therefore treats the assembled application as a first-class semantic object that can itself be checked.

Whole-application closure is not a second choreography

The application assembly does not replace the global choreography.

The layers remain distinct:

```
global choreography
    establishes allowed distributed semantic behavior

projection
    derives role-local executable semantics

application assembly
    establishes that concrete application realization actually supplies
    every required physical/trusted edge for those semantics
```

A choreography answers:

> What distributed behavior is semantically allowed?

Application closure answers:

> Has this application actually assembled a complete realization of that behavior?

A route being present does not make an invalid choreography valid.

A valid choreography does not make a missing route appear.

Both layers are required.

Assurance terminology remains precise

Whole-application closure must not inflate its terminology.

Mechanically proved properties of explicit formal semantics may be called `proved` under the assurance rules established in Part I.

Closed graph/assembly checks should be described as, for example:

- `preflight-closed`;
- `assembly-verified`;
- `verified application assembly`.

A Chromium test is behavioral evidence.

A startup check is runtime enforcement.

A trusted route or authentication binding may remain a trusted/runtime premise.

These classifications may coexist in one application report.

For example:

```
operation :request/claim

semantic choreography:
    proved / mechanically checked for supported properties

browser artifact correspondence:
    assembly-verified

server capability closure:
    assembly-verified

authenticated principal value:
    runtime-enforced

current Request authorization:
    runtime/model-enforced

authentication middleware correctness:
    trusted assumption

real HTMX/browser lifecycle:
    tested external assumption
```

A useful application explanation preserves those distinctions instead of collapsing them into one green `:verified?` boolean.

Three distinct application assurance claims

The authoritative definition of declared closure, production enforcement, and behavioral completeness is in Part I. This part supplies the construction and enforcement contracts for the first two. Independent user-capability acceptance remains necessary for the third.

An application report must retain the scope of each claim. An unused operation need not have a user affordance unless required by the declared realization or product milestone; omitting a required user capability nevertheless fails that milestone even if the remaining graph is closed.

ApplicationAssembly

The application has one canonical source-owned value or constructor that describes the realization facts needed for whole-application closure.

This conceptual object is called `ApplicationAssembly`.

The exact Clojure API is intentionally not fixed by this design document.

The semantic requirements are fixed.

An `ApplicationAssembly` contains or deterministically derives enough information to describe at least:

- semantic operations exposed by the application;
- global/verified choreography references required by those operations;
- role-local projected browser plans;
- browser plan registry identity;
- generated browser artifact correspondence;
- rendered affordance declarations or the rules by which they are checked;
- physical command transports;
- trusted HTTP/server entrypoints;
- server-side semantic operation registry;
- authoritative model-operation correspondences;
- required and supplied execution capabilities;
- settlement/outcome declarations;
- progression requirements;
- Live/authoritative acquisition paths;
- source/ownership locations used for diagnostics.

The assembly is not merely a bag of registries.

It is a graph whose edges are semantic obligations.

One canonical source-owned assembly

An application must not have separate independently authored assemblies for:

- browser build;
- application preflight;
- startup;
- runtime rendered-surface checking;
- tests.

Those consumers may derive different views of one assembly, but they must share one source of semantic identity.

Conceptually:

```
application source declarations
        |
        v
canonical ApplicationAssembly
        |
        +--> browser build manifest
        |
        +--> browser artifact currentness check
        |
        +--> static application preflight
        |
        +--> runtime rendered-surface preflight
        |
        +--> application explanation
        |
        +--> tests
```

Tests must consume the production assembly.

A test-only reconstruction is not evidence that the production application is closed. Tests may instantiate the canonical constructor with declared fixture configuration and may deliberately introduce controlled defects. They must identify the configuration differences relevant to the claim; fixture-specific providers or routes cannot stand in for an unexamined production binding.

Source ownership is explicit

Every independently authored application realization fact should have an obvious owner.

Examples:

```
semantic operation/choreography
    model or flow choreography namespace

physical routes
    route namespace

Live fragment/acquisition declarations
    Live integration namespace

trusted server operation realization
    server/optimistic integration namespace

browser plan declaration
    Choreo/browser assembly source

view affordance
    view/component source
```

The `ApplicationAssembly` composes those facts.

It should not become a second namespace that manually recopies all of them.

The preferred direction is:

```
source owner
    -> exported normalized declaration
    -> assembly composition
```

not:

```
source owner says X
application_preflight.clj manually says X again
test manually says X a third time
```

Assembly identity

The normalized application assembly has a deterministic semantic identity.

That identity should change when a realization fact relevant to closure changes.

Examples include:

- operation registry membership;
- browser plan semantic identity;
- transport correspondence;
- route method/template;
- server operation correspondence;
- execution capability requirements;
- acquisition/progression binding;
- required browser plan set and manifest input contract.

It should not change merely because of irrelevant source formatting or comments.

The exact normalization/fingerprinting mechanism belongs to the later compositional-proof/hardening layer where appropriate.

The application layer nevertheless requires a stable normalized identity now so that build, startup, diagnostics, and artifact currentness can compare the same assembly.

Normalized declaration identity, generated artifact identity, and admitted generation identity may identify different objects. They must not be collapsed into a self-referential hash. The identity recipe must specify which object and dependencies each identifier binds, following the construction phases below. Source provenance or concrete realization identity may additionally be needed to bind test evidence to the implementation actually exercised; a stable public contract name does not transfer that evidence to a changed handler body.

Assembly construction phases and admitted assembly generation

One canonical source owner does not require every construction product to exist at the same instant. The design distinguishes three phases conceptually:

1. source-owned declarations and normalized semantic inputs;
2. generated executable artifacts plus the evidence that binds them to those inputs;
3. a validated assembly generation admitted for serving.

These are phases and products of one ownership model, not three independently maintained semantic registries. Existing Gesso representations should express these phases where they already suffice; this specification does not require a new public runtime type merely because the phases are semantically distinct.

The construction dependency graph must be acyclic. In particular, an artifact stamp embedded in generated bytes must not require the hash of the final bytes that themselves contain that stamp unless an explicit construction defines how the cycle is broken. A detached build/artifact receipt is one possible realization, not a universal requirement.

Each identity used by the assembly must make clear:

- which semantic or physical object it identifies;
- which normalized inputs determine it;
- which compiler/checker/protocol versions contribute where semantically relevant;
- whether diagnostic-only material is excluded;
- whether the identity denotes normalized semantic meaning or exact generated bytes;
- when the identity becomes fixed;
- which changes invalidate dependent evidence.

An admitted assembly generation binds the application realization accepted for serving. In this part, an unqualified “admitted generation” refers to that assembly generation, not to a browser execution or resource instance. Deployment identity records physical rollout/placement where needed; it is not an alias for assembly generation. Plan/protocol compatibility and exact artifact identity remain separately declared relationships.

An admitted assembly generation is immutable for the scope of the claims made about it. Source, semantic, compiler, manifest, route, or generated-artifact changes that invalidate those claims require construction and validation of a new generation or another explicitly checked transition. Development reload may automate that process, but it must not mutate an already admitted generation in place while retaining stale evidence.

Concrete use of an admitted generation must remain coherent. Routes, handler wrapping, prepared operations, capability providers, rendering rules, and artifact bindings consumed by one admitted request must resolve to that generation or to an explicitly checked compatibility transition. Capturing an assembly map is insufficient if an unchecked mutable registry or redefined var can change its realized behavior while the old report still claims validity. The supported implementation must freeze relevant bindings, resolve through generation-owned references, or invalidate and re-admit affected use.

Evidence produced during construction determines whether an artifact is eligible to execute. It does not become runtime authority. Semantic hashes, artifact receipts, certificates, sidecars, and assembly reports must not grant a running execution a principal, authoritative fact, current authorization, branch decision, or operation capability that the runtime semantics require from trusted current premises.

Assembly construction may derive facts

The obvious coherent formulation should be the shortest formulation.

If the semantic operation declaration already establishes:

- operation identity;
- browser participant;
- command schema;
- required progression;
- expected settlement;
- projected plan identity;

then the assembly should derive or validate those values rather than require each downstream layer to type them again.

The design favors:

```
one semantic declaration
    -> derived browser metadata
    -> derived required transport
    -> derived expected server operation
    -> derived diagnostics
```

over:

```
operation declaration
+ button operation name
+ browser-plan name
+ protocol name
+ route operation string
+ server registry operation string
+ test operation string
```

Where two independent physical choices really do exist, they remain separate and the assembly checks their correspondence.

A HumanHelp Claim realization can make the intended authoring direction concrete. The application should not begin by hand-writing one enormous assembly map. Each source owner exports the facts it genuinely owns.

The Request choreography namespace owns semantic operation identity and projected requirements:

```
(ns net.humanhelp.site.model.request.choreo)

(def claim
  {:operation/id :request/claim

   :roles
   {:helper
    {:principal :user
     :actor :client}

    :request-authority
    {:authority :request}}

   :input
   [:map
    [:request/id uuid?]]

   :browser
   {:role :helper
    :plan :plan/request-claim
    :command-contract :contract/request-claim-command
    :provisional :request/claiming}

   :server
   {:required-capabilities
    #{:authenticated-principal
      :transaction
      :clock
      :authoritative-request-read}}

   :effect
   {:authority :request-authority
    :model-operation :request/claim}

   :settlement
   {:accepted
    {:reacquire :request/card}

    :rejected
    {:reacquire :request/card}}

   ;; Unknown is participant knowledge after missing delivery, not a
   ;; fabricated authoritative server outcome.
   :delivery-uncertainty
   {:recover
    {:acquire :request/card
     :command-attribution :unknown}}})
```

The component owns the fact that one rendered affordance invokes the operation:

```
(ns net.humanhelp.example.components.request-card)

(defn declaration []
  {:view/id :request/card

   :semantic-affordances
   {:claim
    {:operation :request/claim
     :args
     [:map
      [:request/id uuid?]]}}})
```

The route namespace owns physical dispatch:

```
(ns net.humanhelp.example.routes)

(def routes
  [{:route/id :route/request-claim

    :method :post

    :template
    "/app/requests/:request-id/claim"

    :path-parameters
    {:request-id
     {:to [:args :request/id]
      :codec :uuid}}

    :accepts
    {:command-contract :contract/request-claim-command
     :correlation #{:command/id :execution/id}}

    :semantic-operation
    :request/claim

    :trusted-server-capability
    :server/request-claim}

   {:route/id :route/request-card
    :method :get
    :template "/app/requests/:request-id/card"
    :path-parameters
    {:request-id
     {:to [:scope :request/id]
      :codec :uuid}}
    :authoritative-acquisition :request/card
    :trusted-server-capability
    :server/read-request-card}])
```

The trusted server integration owns the implementation capability it provides:

```
(ns net.humanhelp.example.server.request)

(def operations
  {:server/request-claim
   {:semantic-operation :request/claim

    :handler #'claim-handler

    :model-operation :request/claim
    :accepts-command :contract/request-claim-command

    :supplies
    #{:authenticated-principal
      :transaction
      :clock
      :authoritative-request-read}

    ;; An actual provider binding must deliver the operation's required
    ;; shape; naming :clock alone cannot supply :biff.fx/now.
    :capability-bindings
    {:clock
     {:provider :application/system-clock
      :context-key :biff.fx/now
      :value-contract :instant}}}

   :server/read-request-card
   {:authoritative-acquisition :request/card
    :handler #'read-request-card-handler
    :authorization :current-principal-request-view
    :supplies
    #{:authenticated-principal
      :authoritative-request-read}}})
```

The Live/acquisition integration owns the authoritative projection that closes the return path:

```
(ns net.humanhelp.example.live.request)

(def projections
  {:request/card
   {:projection/id :request/card

    :authority :request

    :scope
    [:map
     [:request/id uuid?]]

    :initial-acquisition
    :route/request-card

    :reread
    :route/request-card

    :authorized-principal-source
    :authenticated-session

    :progression
    :xtdb/snapshot}})
```

Browser construction owns the supported plan compiler/build entrypoint, not a second manually maintained operation registry:

```
(ns net.humanhelp.example.browser)

(def browser
  {:plans
   {:source :derive-from-application-assembly}

   :artifact
   {:output-root "resources/public"
    :logical-asset-path "/gesso-live.js"

    :builder
    #'build-current-browser-artifact!}})
```

The canonical application constructor composes those exported declarations:

```
(ns net.humanhelp.example.application)

(defn application-assembly
  [config]
  (gesso.application/assemble
    {:application/id :humanhelp/example

     :operations
     [request.choreo/claim
      request.choreo/create
      request.choreo/cancel]

     :views
     [(request-card/declaration)
      (request-board/declaration)]

     :routes
     routes/routes

     :server-operations
     request.server/operations

     :acquisitions
     request.live/projections

     :browser
     browser/browser

     :capability-providers
     (server/capability-providers
       config)

     :lifecycle
     (application/lifecycle-declarations
       config)}))
```

No source owner is recopied into a test-only registry. Tests call the same constructor with fixture configuration:

```
(def test-assembly
  (application-assembly
    {:environment :test
     :db test-db
     :clock fixed-clock
     :artifact-path test-artifact-path}))
```

The constructor can normalize those declarations into a graph:

```
(def normalized
  (gesso.application/normalize
    (application-assembly config)))

(select-keys normalized
  [:application/id
   :semantic-identity
   :operations
   :plans
   :routes
   :server-operations
   :acquisitions
   :capability-providers])
```

The exact public API is provisional. The design pressure is not: semantic facts are authored at their real owners, the assembly derives what can be derived, and independent physical choices remain visible as edges that must correspond. The displayed constructor also names Create, Cancel, and a Request-board declaration whose source bodies are omitted for brevity; those declarations must exist and close independently. The two displayed Request-card routes, the path-argument mapping, and the trusted clock-to-`biff.fx/now` binding are concrete obligations of the Claim path, not substitutes for omitted declarations or proof that the handler body supplies them correctly. The plan's command contract must also reach the rendered control and trusted route, including the required correlation. A working `hx-post` whose form omits the command field accepted by the route fails the transport/rendered surface edge despite its valid method and URL. The selected realization defines the actual wire field and checks its propagation at both boundaries.

A structurally dangerous alternative would duplicate the same Claim identity in several unrelated maps:

```
;; Avoid: six independently authored statements about one semantic intent.

(def view-config
  {:claim-op "claim"})

(def browser-config
  {:claim-plan "request-claim-plan-v14"})

(def route-config
  {:claim-route "/requests/:id/claim"})

(def server-config
  {:claim-handler "claim-handler"})

(def model-config
  {:claim-op "request/claim"})

(def test-config
  {:expected-op "request-claim"})
```

All six strings can look reasonable while disagreeing. The assembly model exists to make that class of application defect mechanically visible rather than a cross-namespace archaeology exercise.

Assembly construction is closed-world for declared registries

Where the application claims a registry is closed, unknown members are errors.

Examples include:

- unknown semantic operation referenced by a view;
- browser plan key not present in the current plan registry;
- route declaration referring to an unknown trusted server operation;
- acquisition path referring to an unknown fragment/projection;
- server realization referring to an unknown model operation contract.

The application should not discover these facts lazily on first use if the complete registry is already available during construction or preflight.

Whole-application closure graph

The application checker operates over explicit nodes and edges.

It should not grow into a bag of unrelated boolean predicates.

A graph representation provides:

- a place to attach source ownership;
- a place to attach identities;
- a place to record obligations;
- a place to explain which edge failed;
- a place to inspect the complete application;
- a basis for later proof-DAG integration.

Semantic nodes

At minimum the semantic side includes concepts equivalent to:

- global choreography;
- semantic operation;
- projected participant/role;
- authoritative model operation;
- typed semantic outcome/settlement;
- authoritative progression requirement;
- authoritative acquisition/projection requirement.

Depending on the operation, it may also include:

- provisional projection;
- command schema;
- model operation contract;
- authority requirement;
- resource/effect contract.

Application realization nodes

At minimum the physical/application side includes concepts equivalent to:

- rendered affordance declaration;
- browser executable plan;
- browser assembly manifest;
- generated browser artifact/stamp;
- physical transport binding;
- route/server entrypoint;
- trusted server semantic operation registry;
- server execution capability provider;
- Live fragment/acquisition declaration;
- authoritative reread endpoint/projection;
- application lifecycle/startup module.

Important closure edges

The basic path is:

```
rendered affordance
        |
        v
semantic operation
        |
        v
valid trajectory in global choreography
        |
        v
projected browser executable plan
        |
        v
exact generated browser artifact
        |
        v
declared physical command transport
        |
        v
trusted route / server entrypoint
        |
        v
trusted server semantic operation
        |
        v
authoritative model operation
        |
        v
typed settlement / progression
        |
        v
permitted authoritative acquisition path
        |
        v
renderable authoritative projection
```

Not every application operation requires every node.

For example, a server-only choreography has no browser plan.

An ordinary authoritative read may have no optimistic provisional projection.

The assembly representation should express operation-specific requirements rather than inventing fake nodes solely to satisfy one universal pipeline.

Following one Claim affordance all the way through the graph makes the closure boundary much less abstract.

A normalized operation realization could look approximately like:

```
(def claim-realization
  {:operation :request/claim

   :nodes
   {:affordance
    {:node/id :view/request-card.claim
     :kind :rendered-affordance
     :source
     {:ns 'net.humanhelp.example.components.request-card
      :component :request-card
      :slot :claim}}
```

```
    :operation
    {:node/id :op/request-claim
     :kind :semantic-operation
     :identity :request/claim}

    :trajectory
    {:node/id :choreo/request-claim.helper
     :kind :choreography-trajectory
     :role :helper}

    :plan
    {:node/id :plan/request-claim
     :kind :browser-executable-plan
     :identity :plan/request-claim}

    :artifact
    {:node/id :artifact/gesso-live
     :kind :browser-artifact
     :exact-identity
     "sha256:artifact-A12"}

    :transport
    {:node/id :transport/request-claim
     :kind :http-transport
     :method :post
     :route-template
     "/app/requests/:request-id/claim"}

    :route
    {:node/id :route/request-claim
     :kind :trusted-route
     :method :post
     :route-template
     "/app/requests/:request-id/claim"}

    :server
    {:node/id :server/request-claim
     :kind :trusted-server-operation
     :identity :server/request-claim}

    :model
    {:node/id :model/request-claim
     :kind :authoritative-model-operation
     :identity :request/claim}

    :settlement
    {:node/id :settlement/request-claim
     :kind :typed-settlement
     :outcomes
     #{:accepted :rejected}}

    :recovery
    {:node/id :recovery/request-claim-unknown
     :kind :unknown-command-outcome-recovery
     :acquire :request/card
     :command-attribution :unknown}

    :acquisition
    {:node/id :acquire/request-card
```

```
     :kind :authoritative-acquisition
     :projection :request/card}

    :render
    {:node/id :render/request-card
     :kind :renderable-authoritative-projection
     :projection :request/card}}})
```

The edges are the actual obligations:

```
(def claim-edges
  [{:edge/id :claim/affordance->operation
    :kind :affordance-realizes-operation
    :from :view/request-card.claim
    :to :op/request-claim}

   {:edge/id :claim/operation->trajectory
    :kind :operation-admitted-by-choreography
    :from :op/request-claim
    :to :choreo/request-claim.helper}

   {:edge/id :claim/trajectory->plan
    :kind :projection-correspondence
    :from :choreo/request-claim.helper
    :to :plan/request-claim}

   {:edge/id :claim/plan->artifact
    :kind :artifact-contains-exact-plan
    :from :plan/request-claim
    :to :artifact/gesso-live}

   {:edge/id :claim/plan->transport
    :kind :plan-emits-transport
    :from :plan/request-claim
    :to :transport/request-claim}

   {:edge/id :claim/transport->route
    :kind :physical-dispatch-correspondence
    :from :transport/request-claim
    :to :route/request-claim}

   {:edge/id :claim/route->server
    :kind :trusted-dispatch
    :from :route/request-claim
    :to :server/request-claim}

   {:edge/id :claim/server->model
    :kind :delegates-to-model-operation
    :from :server/request-claim
    :to :model/request-claim}

   {:edge/id :claim/model->settlement
    :kind :typed-outcome-correspondence
    :from :model/request-claim
    :to :settlement/request-claim}

   {:edge/id :claim/plan->unknown-recovery
    :kind :uncertain-delivery-recovery
    :from :plan/request-claim
    :to :recovery/request-claim-unknown}
```

```
   {:edge/id :claim/recovery->acquisition
    :kind :permitted-authoritative-reacquisition
    :from :recovery/request-claim-unknown
    :to :acquire/request-card}

   {:edge/id :claim/settlement->acquisition
    :kind :permits-authoritative-reacquisition
    :from :settlement/request-claim
    :to :acquire/request-card}

   {:edge/id :claim/acquisition->render
    :kind :projection-renderable
    :from :acquire/request-card
    :to :render/request-card}])
```

A checker can retain evidence for each edge rather than return one boolean:

```
(def checked-claim
  (mapv
    (fn [edge]
      (application/check-edge
        normalized-assembly
        edge))
    claim-edges))

(first checked-claim)
;; ⇒
{:edge/id :claim/affordance->operation
 :kind :affordance-realizes-operation

 :from
 {:node/id :view/request-card.claim
  :source
  {:ns 'net.humanhelp.example.components.request-card
   :component :request-card
   :slot :claim}}

 :to
 {:node/id :op/request-claim
  :identity :request/claim}

 :evidence
 {:class :assembly-verified
  :checker :gesso.application/affordance-operation-edge
  :inputs
  {:declared-operation :request/claim
   :known-operations #{...}}}

 :status :accepted}
```

Another edge may depend on exact artifact correspondence:

```
{:edge/id :claim/plan->artifact
 :kind :artifact-contains-exact-plan

 :from
 {:plan/id :plan/request-claim
  :manifest/id :browser-manifest-M17}

 :to
 {:artifact/id :artifact/gesso-live
  :receipt/id :artifact-receipt-R17
  :artifact-digest "sha256:artifact-A12"}

 :evidence
 {:class :assembly-verified
  :relation
  :manifest-artifact-exact-currentness}

 :status :accepted}
```

And the route-to-server edge may explicitly retain what is **not** proven:

```
{:edge/id :claim/route->server
 :kind :trusted-dispatch

 :from
 {:route/id :route/request-claim}

 :to
 {:server-operation :server/request-claim}

 :evidence
 {:class :assembly-verified
  :relation :router-dispatch-binding}

 :remaining-premises
 [{:class :trusted
   :proposition
   :router-obeys-declared-dispatch}

  {:class :tested
   :proposition
   :handler-body-corresponds-to-declared-server-operation}]

 :status :accepted}
```

That last report deliberately does not promote structural binding into a theorem about arbitrary handler code.

Now break one edge at a time.

If the generated artifact omits the plan:

```
(def broken-artifact
  (assoc-in normalized-assembly
    [:browser :artifact :contained-plans]
    #{:plan/request-create
      :plan/request-cancel}))

(application/check-operation
  broken-artifact
  :request/claim)
;; ⇒
{:status :rejected
 :error/kind :missing-browser-plan
 :failed-edge :claim/plan->artifact
 :operation :request/claim
 :required :plan/request-claim
 :available
 #{:plan/request-create
   :plan/request-cancel}}
```

If the route points to Cancel instead of Claim:

```
(def wrong-route
  (assoc-in normalized-assembly
    [:routes :route/request-claim
     :semantic-operation]
    :request/cancel))

(application/check-operation
  wrong-route
  :request/claim)
;; ⇒
{:status :rejected
 :error/kind :server-operation-mismatch
 :failed-edge :claim/transport->route
 :expected :request/claim
 :actual :request/cancel}
```

If the trusted execution lacks the required clock:

```
(def no-clock
  (update-in normalized-assembly
    [:server-operations
     :server/request-claim
     :supplied-capabilities]
    disj
    :clock))

(application/check-operation
  no-clock
  :request/claim)
;; ⇒
{:status :rejected
 :error/kind :missing-server-capability
 :operation :request/claim
 :required :clock
 :provider :server/request-claim}
```

If mutation can settle but there is no legal Request-card reread:

```
(def no-reacquisition
  (update normalized-assembly
    :acquisitions
    dissoc
    :request/card))

(application/check-operation
  no-reacquisition
  :request/claim)
;; ⇒
{:status :rejected
 :error/kind :missing-authoritative-reread
 :failed-edge :claim/settlement->acquisition
 :projection :request/card}
```

These failures all concern the same semantic operation, but they belong to different graph edges and therefore different local repairs.

A closed operation report can preserve that structure:

```
{:operation :request/claim

 :status :preflight-closed

 :edges
 {:accepted 13
  :rejected 0
  :open 0}

 :runtime-premises
 #{:authenticated-principal
   :current-request
   :current-membership
   :current-organization}

 :assurance
 {:choreography
  {:semantic-checker :accepted
   :proof-kernel :accepted-under-stated-premises}
  :application-path :assembly-verified
  :current-authorization :runtime-enforced
  :browser-integration :tested}}
```

The dynamic premises are not ”open edges.” Closure says the application has a structural place to obtain and enforce them when execution occurs. Whether Alice is currently authorized remains a runtime/model question.

This is the practical payoff of making `ApplicationAssembly` a graph rather than a pile of checks: the same Claim operation can be inspected end-to-end and each contradiction names the relation that actually failed.

Where two realization values have the same ordinary Clojure representation but different semantic status, shape alone is not evidence that they are interchangeable. A raw declaration, admitted executable plan, current artifact stamp, trusted server realization, or checked certificate may all be represented with ordinary immutable data while belonging to different semantic categories. The supported construction path must preserve those distinctions strongly enough that accidental substitution is rejected at the earliest legitimate boundary.

Nominal records/types, opaque wrappers, smart constructors, checked constructors, or existing admitted representations are permitted implementation techniques. No particular technique is required here. The requirement is semantic: code must not gain stronger admission, provenance, authority, or certification merely by constructing a same-shaped map or discriminator key.

Every edge has evidence

Each assembly edge should be inspectable as something conceptually equivalent to:

```
{:edge/id ...
 :edge/kind :operation->browser-plan
 :edge/source ...
 :edge/from ...
 :edge/to ...
 :edge/expected ...
 :edge/evidence ...
 :edge/status :assembly-verified}
```

Exact fields are tentative.

The required semantics are:

- which relation is being checked;
- which source introduced the obligation;
- which source supplies the candidate fulfillment;
- what identities are compared;
- how failure is classified;
- what diagnostic evidence is available;
- which premises remain trusted or runtime-enforced rather than mechanically established by that edge check.

An edge's own `:edge/status :assembly-verified` field is not its admission rule. The checker must derive acceptance from the appropriate identities, contracts, and admitted dependencies. Raw or stale reports cannot certify themselves. An arbitrary handler annotated with an operation name does not thereby prove that its body implements that operation. The supported constructor/dispatch binding supplies structural correspondence; remaining implementation behavior is trusted or tested unless stronger evidence actually discharges it.

An edge should not disappear after successful checking if retaining its explanation is useful for application inspection.

No orphan realization nodes

An application should detect realization artifacts that are supposed to serve a semantic role but are unconnected.

Examples:

- generated browser plan with no semantic operation;
- route marked semantic but absent from operation registry;
- Live acquisition declaration for no managed fragment;
- server operation registry entry that claims a semantic operation no current authoritative model contract supplies.

Some unused declarations may be legal.

They must be explicitly classified as unused/available rather than silently confused with realized application behavior.

No unresolved required edge in a closed application

A preflight-closed application has no unresolved structural obligations.

Dynamic runtime premises may remain.

For example:

```
"structurally, authenticated principal will be supplied here"
    may be closed at assembly time

"the actual principal is authorized for Request R right now"
    is runtime/model state
```

The first can be an assembly requirement.

The second should not be mislabeled as an open preflight obligation.

Semantic operation identity is preserved through realization

Once source code declares that a rendered action means a semantic operation, later stages may add physical realization details but may not silently erase or substitute that operation identity. This is preservation through binding, not an ordering over identities.

The intended realization path is:

```
semantic operation
    |
    + operation arguments/subject
    |
    + browser plan
    |
    + render binding
    |
    + physical transport
    |
    + trusted route
    |
    + trusted execution
    |
    + settlement/publication/acquisition
```

Failure to realize a required edge of an enabled action causes rejection. An explicitly unavailable rendering may be admitted under its own contract, with no active weaker transport and without counting as a usable capability. Neither case erases the earlier semantic declaration.

No implicit fallback realization

Suppose `:request/claim` has an optimistic Choreo realization that requires:

- authoritative observation basis;
- protocol command;
- projected browser plan;
- settlement/progression contract.

If the current render cannot supply the required basis, the following is forbidden:

```
cannot realize optimistic :request/claim
    |
    v
omit :choreo/op
    |
    v
emit ordinary hx-post /claim
```

That turns a known semantic operation into a weaker anonymous transport request.

The correct possibilities are:

- realize the declared semantic operation completely;
- render it explicitly unavailable/disabled if the UI contract permits that;
- choose another realization that was independently declared and preflighted for the same semantic operation;
- reject the malformed render/application.

Explicit alternate realization is legal

The no-fallback rule does not mean one semantic operation can have only one physical realization forever.

An operation may deliberately support, for example:

```
optimistic browser realization
non-optimistic semantic realization
server-only realization
```

if those alternatives are:

- explicitly declared;
- semantically related to the same operation;
- individually checked;
- selected by a rule the application understands;
- visible to diagnostics.

What is forbidden is inventing the weaker realization because the stronger one was missing required data.

Operation identity survives unavailable realization

A missing basis means:

> this realization is unavailable

not:

> this control no longer means :request/claim

This distinction is central to fail-closed rendering.

Ordinary HTMX and semantic-operation routes

Ordinary HTMX remains a supported mechanism.

Gesso does not require every:

- form;
- link;
- refresh;
- filter;
- search interaction;
- local UI control;

to become a choreographic operation.

The application must instead distinguish ordinary interactions from semantic operation realizations.

Ordinary HTMX remains legal

An ordinary HTMX interaction may use a route that is not assembled as a semantic-operation route.

For example:

```
ordinary filter UI
ordinary search fragment
ordinary non-semantic preference update
```

may use ordinary HTMX according to application design.

The framework must not reject ordinary HTMX merely because Choreo exists.

Anonymous use of an assembled semantic route is rejected

If method/path correspondence identifies a physical route as a realization of a semantic operation, an ordinary anonymous action must not occupy that route while omitting the semantic identity required by the assembly.

Conceptually:

```
assembly:
    POST /app/requests/:request-id/claim
        realizes :request/claim

rendered:
    hx-post="/app/requests/123/claim"
    no semantic operation identity

result:
    REJECT
```

This rule is a backstop against semantic identity erasure.

Method and route-template identity matter

The collision check must use the application's actual route semantics, including the relevant base path, decoding/normalization, route precedence, and method dispatch of the supported router. An independently guessed string-matching approximation is insufficient where it disagrees with actual dispatch.

The supported constructor should derive checking and dispatch from the same normalized route declarations or establish their correspondence explicitly. Unsupported dynamic routing must be identified as such rather than accepted as closed merely because a checker cannot analyze it.

It should distinguish, as appropriate:

- HTTP method;
- route template;
- concrete path parameters;
- query-string normalization where semantically relevant;
- optional/alternate physical realizations.

For example:

```
POST /requests/:id/claim
```

must match a concrete:

```
POST /requests/550e8400-e29b-41d4-a716-446655440000/claim
```

for collision purposes.

A GET to the same path should not automatically collide with the POST operation unless the application declared it as another semantic realization.

Parameterization cannot bypass semantic-route identity

Route-template matching must not be defeated by ordinary parameter values.

A semantic route does not cease to be semantic merely because a concrete UUID, slug, or other path value was substituted.

Likewise, query parameters should not accidentally create a second anonymous realization when the router regards the request as the same semantic endpoint.

Forms and inherited attributes are part of the effective rendered action

The rendered-surface checker must reason about the effective action, not only a particular button attribute.

Potential semantic transport can arise from:

- a form's action/method;
- an `hx-post` on the form;
- a child control inheriting HTMX behavior;
- nested/inherited HTMX attributes;
- component helpers that generate the action indirectly.

The exact HTML/HTMX analysis should stay within the supported contract subset. It must include any supported submitter override, inherited target or action, and generated attribute that changes effective dispatch. Conflicting authored transport attributes and derived semantic bindings must be rejected or resolved by an explicit checked rule; silent last-write-wins merging is not closure.

Arbitrary client-side JavaScript can change DOM after delivery. Structured render checking does not prove that such mutation never occurs, and the server must still reject malformed or unauthorized protocol use independently.

The principle is that a helper cannot bypass semantic-route checking simply by moving the physical action one level up the DOM.

Multiple realizations must be explicit, not ambiguous

If one semantic operation has multiple physical routes, the assembly records that relationship.

If one physical route can dispatch several semantic operations, the route must have a trusted discriminator/registry relationship that preflight can inspect.

Ambiguity is not resolved by guessing from:

- parameter names;
- button text;
- source namespace;
- nearby plan names.

Browser assembly manifest

The browser plan set used by application closure is normalized into an explicit build manifest.

This conceptual artifact is called `BrowserAssemblyManifest`.

The exact format is tentative.

Its semantic role is not.

The manifest identifies the executable browser plans that the current application assembly requires.

It should include enough stable identity to answer:

- which plans are required;
- which declared plan/protocol compatibility identities or versions they target;
- which semantic operations they realize;
- the build contract that binds the required plan set to the generated artifact.

The pre-build manifest need not contain a digest of bytes that do not yet exist. A generated receipt or another acyclic construction may bind the manifest to the completed artifact. Its relationship must be explicit under the assembly construction phases.

Browser build consumes the canonical assembly

The supported browser build must derive its required plan set from the same source-owned application assembly used by preflight.

Forbidden architecture:

```
production preflight registry A

browser build registry B

tests registry C
```

even if A, B, and C happen to match today.

The preferred architecture is:

```
canonical ApplicationAssembly
    |
    v
BrowserAssemblyManifest
    |
    v
browser build
```

Generated artifact has exact currentness identity

The generated runtime artifact carries or is accompanied by an exact identity corresponding to the manifest from which it was built.

This conceptual fact is called `BrowserArtifactStamp`.

A successful currentness report requires:

- the current manifest identity is valid;
- the generated artifact stamp is valid;
- the identities correspond exactly according to the defined artifact currentness relation.

A boolean `:valid? true` supplied independently of the compared identities is not sufficient evidence. Nor is a copied manifest stamp sufficient to establish that an arbitrary file contains the intended runtime. The supported build and admission path must bind the expected manifest, executable plan payloads, and actual emitted artifact under the declared identity recipe.

Where exact bytes are claimed, the checker must establish correspondence to those bytes through the checked build receipt, content digest, or another declared mechanism. A stamp's presence in a filename, comment, or metadata map does not independently establish that relationship. The assumptions of any trusted build/receipt mechanism remain visible.

Exact artifact currentness is stricter than plan compatibility

A stale generated file may still contain a plan whose protocol is broadly compatible.

That does not make it the current application artifact.

The build/startup check asks:

> Was this artifact generated from the current required browser assembly?

This is an exact correspondence question.

The stale-plan runtime compatibility question from Part VI asks:

> Can a browser already executing an older plan safely recover or continue?

Those checks serve different purposes.

Stale artifact fails before serving

If the current application requires manifest M2 but the generated browser artifact corresponds to M1:

```
current manifest M2
generated artifact stamp M1
M1 ≠ M2

=> startup/build rejection
```

The application must not start normally and hope the missing browser behavior is not exercised.

The artifact checked during admission must be the artifact served or loaded by the corresponding generation. Mutable asset paths, cached responses, or later replacement of generated files must not silently sever that binding. The supported realization must preserve exact serving correspondence or detect and reject the mismatch before relying on it.

This does not require an already-running browser to have the newest artifact at every moment. Such a browser follows the separately declared stale-plan compatibility and recovery contract. Exact current serving identity and allowed continuation of an older client are distinct claims.

Rebuild makes the artifact current

The supported development/build workflow should make browser-artifact generation an ordinary reproducible step.

The design target is:

```
source changes plan set
    |
currentness check becomes red
    |
supported browser build
    |
artifact stamp matches current manifest
    |
startup may proceed
```

A manually remembered side command whose omission merely causes subtle browser failure is not an adequate final workflow.

A concrete build pipeline shows how manifest identity and exact artifact identity can remain acyclic.

Suppose the normalized application assembly requires three browser plans:

```
(def browser-manifest-input
  {:manifest/version 1

   :application/id :humanhelp/example

   :assembly/semantic-identity
   "sha256:semantic-assembly-44ae"

   :compiler
   {:gesso/projector-version 12
    :cljs-target :browser
    :cljs-build-identity
    (build/identity :cljs-compiler)
    :browser-runtime-source-identity
    (build/identity :gesso-browser-runtime)
    :build-options-identity
    (build/identity :humanhelp-browser-options)}

   :required-plans
   [{:plan/id :plan/request-create
     :operation :request/create
     :compatibility :humanhelp-live-v12}

    {:plan/id :plan/request-claim
     :operation :request/claim
     :compatibility :humanhelp-live-v12}

    {:plan/id :plan/request-cancel
     :operation :request/cancel
     :compatibility :humanhelp-live-v12}]})
```

The manifest identity is computed over normalized semantic/build inputs:

```
(def manifest-id
  (artifact/digest
    {:algorithm :sha-256
     :encoding :canonical-edn-v1
     :value browser-manifest-input}))

(def browser-manifest
  (assoc browser-manifest-input
    :manifest/id manifest-id))
```

The `build/identity` calls are schematic inputs backed by fingerprints or pinned versions of the actual compiler, runtime source, and relevant options. Reusing a literal `:current` label across changed inputs would leave the stale-artifact check unsound. A source or toolchain change that can alter the emitted runtime must change an appropriate normalized input or force a trusted rebuild whose artifact correspondence is rechecked.

Projection emits the exact executable payloads required by that manifest:

```
(def projected-plans
  (into {}
    (map
      (fn [{:plan/keys [id]}]
        [id
         (projector/emit-executable-plan
           normalized-assembly
           id)]))
    (:required-plans browser-manifest)))
```

The browser builder then emits JavaScript and computes an exact content identity over the actual served bytes:

```
(defn build-browser-artifact!
  [{:keys [manifest plans output-path]}]

  (browser/require-exact-projected-plan-set!
    manifest plans)
  (let [javascript-bytes
        (cljs-build/compile-gesso-runtime!
          {:plans plans
           :output-path output-path})

        artifact-digest
        (artifact/digest-bytes
          {:algorithm :sha-256
           :bytes javascript-bytes})

        ;; Build output includes an inspectable plan inventory bound to the
        ;; emitted runtime registry, not an inventory typed into this receipt.
        emitted-plan-digests
        (cljs-build/inspect-emitted-plan-registry
          javascript-bytes)

        receipt
        {:receipt/version 1

         :manifest/id
         (:manifest/id manifest)

         :artifact/path
         output-path

         :artifact/digest
         artifact-digest

         :plan-digests
         emitted-plan-digests}]

    (when-not
      (= emitted-plan-digests
         (artifact/plan-digests plans))
      (throw (ex-info
               "Emitted plan registry differs from projected plans."
               {:error/kind :artifact-content-mismatch})))

    (write-receipt!
      (str output-path ".receipt.edn")
      receipt)

    receipt))
```

The receipt is detached from the bytes whose digest it records, so there is no self-hash cycle. Receipt fields alone cannot establish that the served JavaScript actually contains the claimed plans: a receipt could repeat the expected digests beside an unrelated bundle. The illustrative `cljs-build/inspect-emitted-plan-registry` contract requires an inspectable inventory derived from the **served bytes** and tied by the trusted builder to the runtime registry it emits. If the selected compiler cannot establish that binding, the checker must report the missing correspondence rather than calling receipt equality a proof of plan inclusion. Compiler fidelity and actual runtime behavior still require their separately named evidence.

Startup compares the current manifest, current projected payloads, receipt, and served artifact. This check reads the same asset path the admitted server will serve; deployment must reject replacing those bytes after admission or revalidate the replacement against a new assembly generation:

```
(defn check-browser-artifact-currentness
  [{:keys [manifest plans artifact-path receipt]}]

  (browser/require-exact-projected-plan-set!
    manifest plans)
  (let [actual-bytes
        (read-bytes artifact-path)

        actual-artifact-digest
        (artifact/digest-bytes
          {:algorithm :sha-256
           :bytes actual-bytes})

        expected-plan-digests
        (artifact/plan-digests plans)

        actual-plan-digests
        (cljs-build/inspect-emitted-plan-registry
          actual-bytes)

        verified-manifest-id
        (artifact/digest
          {:algorithm :sha-256
           :encoding :canonical-edn-v1
           :value (dissoc manifest :manifest/id)})]

    (cond
      (not=
        verified-manifest-id
        (:manifest/id manifest))
      {:status :rejected
       :error/kind :invalid-browser-manifest
       :reason :manifest-identity-mismatch}

      (not=
        artifact-path
        (:artifact/path receipt))
      {:status :rejected
       :error/kind :artifact-content-mismatch
       :reason :served-path-does-not-match-receipt}

      (not=
        (:manifest/id manifest)
        (:manifest/id receipt))
      {:status :rejected
       :error/kind :stale-browser-artifact
       :reason :manifest-mismatch
       :current-manifest
       (:manifest/id manifest)
       :receipt-manifest
       (:manifest/id receipt)}

      (not=
        expected-plan-digests
        (:plan-digests receipt))
      {:status :rejected
       :error/kind :artifact-content-mismatch
       :reason :projected-plan-set-mismatch
       :expected expected-plan-digests
```

```
       :receipt (:plan-digests receipt)}

      (not=
        expected-plan-digests
        actual-plan-digests)
      {:status :rejected
       :error/kind :artifact-content-mismatch
       :reason :served-plan-registry-does-not-match-projection
       :expected expected-plan-digests
       :actual actual-plan-digests}

      (not=
        actual-artifact-digest
        (:artifact/digest receipt))
      {:status :rejected
       :error/kind :artifact-content-mismatch
       :reason :served-bytes-do-not-match-receipt
       :expected (:artifact/digest receipt)
       :actual actual-artifact-digest}

      :else
      {:status :accepted
       :manifest/id (:manifest/id manifest)
       :artifact/digest actual-artifact-digest})))
```

The exact-set check rejects missing, extra, duplicate, nil, or operation/compatibility-mismatched plan entries before compiling or serving. An inventory match cannot compensate for a manifest that omitted an operation required by the source assembly; normalization checks that source-to-manifest edge independently.

Now suppose the developer adds `:request/reassign` to its source-owned declarations. The next transformation illustrates the **derived** normalized manifest change; developers do not hand-edit the production manifest:

```
(def changed-manifest-input
  (update browser-manifest-input
    :required-plans
    conj
    {:plan/id :plan/request-reassign
     :operation :request/reassign
     :compatibility :humanhelp-live-v12}))

(def changed-manifest-id
  (artifact/digest
    {:algorithm :sha-256
     :encoding :canonical-edn-v1
     :value changed-manifest-input}))
```

In this fixture, `changed-projected-plans` is the new valid projected set, `old-receipt` is the receipt from the old build, and `old-manifest-id` is `manifest-id`. The old receipt still names the previous manifest:

```
(check-browser-artifact-currentness
  {:manifest
   (assoc changed-manifest-input
     :manifest/id changed-manifest-id)

   :plans changed-projected-plans

   :artifact-path
   (:artifact/path old-receipt)

   :receipt old-receipt})
;; ⇒
{:status :rejected
 :error/kind :stale-browser-artifact
 :reason :manifest-mismatch
 :current-manifest changed-manifest-id
 :receipt-manifest old-manifest-id}
```

The application does not begin serving and hope nobody clicks Reassign.

A supported development command can make the normal path explicit:

```
(defn prepare-application-generation!
  [config]

  (let [assembly
        (application-assembly config)

        normalized
        (gesso.application/normalize assembly)

        manifest
        (browser/derive-manifest normalized)

        plans
        (browser/project-required-plans
          normalized
          manifest)

        candidate-artifact-path
        (browser/staged-artifact-path
          manifest)

        receipt
        (browser/build-browser-artifact!
          {:manifest manifest
           :plans plans
           :output-path
           candidate-artifact-path})

        preflight
        (gesso.application/preflight
          {:assembly normalized
           :browser-manifest manifest
           :browser-plans plans
           :browser-artifact-receipt receipt
           :served-artifact-path
           candidate-artifact-path})]

    (gesso.application/admit-generation!
      preflight)))
```

Here `admit-generation!` returns a generation whose `:preflight/status` is `:accepted` and whose handler, route, provider, and served-artifact bindings are resolved from this same validated construction. The currentness check above runs on the served bytes, not just on the receipt. The staged path is unique to the candidate build and remains immutable while its assembly generation is admitted. The public logical asset route resolves through the request's admitted generation. Building a rejected A13 candidate cannot overwrite A12's served JavaScript. Activation switches new requests and asset bindings together; in-flight A12 requests keep their old binding. Browser cache/asset retention and old client compatibility follow the Part VI contract.

This still leaves stale already-running browsers to Part VI's compatibility and recovery rules. Exact startup currentness answers a different question: whether the artifact the **new generation intends to serve** is exactly the artifact constructed from its required plan set.

No test-only artifact surrogate

Tests that verify application startup/currentness must use the same manifest and artifact correspondence mechanism as the running application.

A fake stamped artifact may be useful in a narrow unit test of the checker.

It must not replace the real artifact path in the top-level application acceptance/startup test.

Rendered affordance closure

A rendered semantic affordance introduces a concrete whole-application obligation.

A view declaration such as:

```
{:choreo/op :request/claim
 :subject request-id}
```

means more than:

> put some metadata on a button

It means:

> the assembled application claims that this visible action can be realized as the declared semantic operation.

Views should state minimal semantic intent

The ordinary view/component should not normally have to repeat:

- projected plan registry internals;
- browser bootstrap registry internals;
- protocol envelope encoding;
- browser artifact digest;
- trusted server operation implementation;
- authority identity;
- exact progression header/wire representation.

Those values are derived or checked by the assembled application.

The view owns:

- which semantic operation the affordance represents;
- subject/arguments not already bound;
- presentation;
- explicit UI availability state where needed.

Rendered capability is not authorization

The view may render an operation because trusted server-side preparation says it is presently sensible to expose.

Execution still re-establishes:

- authenticated principal;
- current domain state;
- model authorization;
- commit-time guards.

Preflight checks structural availability.

It does not turn the rendered affordance into durable permission.

Rendered enabled action must have a complete realization

If a semantic action is rendered as executable, the application must be able to trace its closure path.

Conceptually:

```
view source
    |
operation
    |
browser plan
    |
artifact
    |
transport
    |
route
    |
trusted server op
    |
model op
    |
settlement/progression
    |
acquisition
```

An enabled visible action with a missing edge is a malformed assembled application.

Unavailable semantic operation remains semantically named

An application may deliberately render an operation unavailable.

For example:

```
Claim
    disabled
    reason: no authoritative basis yet
```

That may be a valid UI state if the product design permits it.

The important point is that the operation identity remains known.

The UI must not replace it with an anonymous ordinary POST merely to keep the control clickable. Unavailability must suppress the effective executable path, including inherited or form-level transport. Styling a control as disabled while leaving an active semantic bypass does not satisfy this contract.

Required-capability acceptance remains independent: a permanently unavailable required action does not count as operational completion merely because its disabled rendering is structurally valid.

Transport and route closure

A browser-visible semantic operation must have a complete trusted transport path.

The transport layer is physical realization.

It does not establish semantic authority.

Transport declaration

A projected operation may require a transport contract containing facts such as:

- transport kind;
- HTTP method/path template where relevant;
- command envelope/protocol version;
- correlation requirements;
- operation discriminator requirements;
- progression/basis propagation;
- expected settlement channel.

The exact schema should remain small and grow from concrete Gesso requirements.

Physical transport maps to trusted route

The application assembly checks that the required transport has a corresponding trusted server entrypoint.

Conceptually:

```
browser plan says:
    operation :request/claim
    transport T

routes say:
    T accepted by route R

server says:
    R dispatches through trusted semantic operation registry
```

A matching URL string alone is not enough when method, protocol, or operation discriminator differs.

Route declares what it accepts

A trusted semantic route should make its accepted contract inspectable enough for application closure to determine:

- supported transport/protocol;
- semantic operation(s);
- path parameter mapping;
- required command envelope;
- authenticated-context requirements;
- server execution entrypoint.

This does not mean route code contains business policy.

It means route realization is explicit rather than discovered by arbitrary handler introspection at runtime.

Browser cannot grant trusted state through transport

The browser may send:

- requested semantic operation;
- command ID;
- operation arguments;
- observed basis/progression context;
- correlation metadata.

The browser may not grant itself:

- authenticated principal;
- authority role;
- trusted server machine state;
- model permission;
- authoritative provenance.

The trusted route/server boundary reconstructs or validates those facts from trusted context.

Route selection and operation identity are checked together

If a route parameter selects a Request ID and the command envelope also names a Request ID, trusted server integration must ensure they correspond where the contract requires correspondence.

Likewise, if the route selects the semantic operation physically while the protocol envelope contains an operation identifier, mismatches are rejected.

The system must not accept contradictory independent selectors and let one silently override the other. The trusted route validates the supported envelope and required selectors before invoking the authoritative model. This enforcement does not depend on the request originating from checked rendered HTML.

Transport closure is not authorization closure

Application preflight may establish:

```
there exists a trusted route capable of receiving :request/claim
```

It cannot establish:

```
user U may claim Request R at 14:03:17
```

The latter depends on runtime authoritative state.

The diagnostic system must classify these failures differently.

Trusted server operation closure

After transport reaches trusted server code, the application must resolve the semantic operation through a trusted registry or equivalent closed dispatch mechanism.

The server must not execute arbitrary browser-supplied machine state.

Trusted operation registry

The server-side registry maps semantic operation identity to the trusted prepared execution realization.

Conceptually:

```
{:request/claim
 {:operation ...
  :prepared-server ...
  :model-operation ...
  :requires #{...}}}
```

Exact representation is tentative.

The registry must be derived from or checked against the same semantic operation sources used by Choreo/application assembly.

Server realization delegates to public model operation

The trusted semantic operation eventually invokes the authoritative public model operation that owns the domain transition.

For Request claim:

```
trusted :request/claim server projection
        |
        v
public Request model claim operation
        |
        v
authoritative transaction
```

The server realization must not reproduce a second copy of Request business policy.

It may perform protocol work such as:

- trusted principal binding;
- command decode/validation;
- role/operation correspondence;
- progression context setup;
- execution capability binding;
- calling the public model operation;
- normalizing semantic settlement.

Exact prepared-server correspondence

Where a projected browser plan expects a particular trusted server projection or prepared operation contract, application closure must compare the actual server realization against that expectation.

A server function with a convenient name is not enough evidence.

The application needs stable identity or normalized contract information sufficient to establish correspondence.

The checked route must dispatch through the admitted prepared realization, not an unrelated function carrying equivalent-looking metadata. The report must distinguish structural binding to that realization from proof of its arbitrary host-language implementation. Contract-level identity, concrete assembly- generation binding, and implementation evidence serve different purposes.

This requirement prevents:

```
browser plan generated for server semantics S1
running route dispatches to server semantics S2
```

from being accepted merely because both are called `claim`.

No duplicate hidden operation truth

The server registry should not become an independently maintained list of operation semantics.

Its semantic operation identities and contracts should derive from or be checked against model/Choreo declarations.

The registry owns trusted execution dispatch.

It does not own the meaning of the business operation.

Server execution capability closure

A structurally valid server operation may still be unrealizable because its execution environment lacks a required resource.

The application assembly therefore represents required and supplied execution capabilities.

Required capabilities

Concrete operations may require capabilities such as:

- authenticated principal;
- transaction execution;
- authoritative database access;
- authoritative basis/progression context;
- clock/current time;
- random seed / deterministic UUID source;
- publication/notification facility;
- other trusted runtime services.

The set must grow from concrete operation needs.

Gesso should not invent a universal abstract effect system merely to make the capability list theoretically complete.

Supplied capabilities

A trusted server boundary declares the capability classes it can provide.

Conceptually:

```
operation requires
    #{:authenticated-principal
      :transaction
      :authoritative-basis
      :clock
      :random-seed}

server boundary supplies
    #{:authenticated-principal
      :transaction
      :authoritative-basis
      :clock
      :random-seed
      :live-publication}

initial structural condition:
    required capability classes are supplied

complete structural condition:
    each requirement resolves to an admitted compatible provider contract
```

The actual values remain runtime values.

Preflight checks structural supply. Set inclusion over names is a useful first check, not a complete provider contract when stronger properties matter. Each required capability must resolve to a compatible provider with the declared input/output shape, trusted origin, invocation scope, and lifecycle dependency. For example, a function yielding sampled time and an already-sampled timestamp are not interchangeable merely because both are labeled `:clock`.

Provider composition must reject missing, ambiguous, or incompatible supply where statically knowable. A provider can remain unavailable at runtime, and the operation must retain its declared failure or recovery behavior. Provider annotations alone do not prove arbitrary implementation correctness.

Capability is not authority

A server having:

```
:transaction
:clock
:random-seed
```

does not authorize an operation.

Capabilities answer:

> Can this trusted execution environment provide the resource/input required by the operation?

Authority answers:

> May this authenticated principal perform this semantic operation against the current authoritative world?

Those are separate checks.

Capability provenance matters

Some capabilities are meaningful only when supplied by a trusted boundary.

A browser-supplied:

```
{:clock ...}
```

does not satisfy the requirement for trusted current time merely because the value shape matches.

Likewise a client-supplied principal ID does not satisfy `:authenticated-principal`.

The application assembly can verify the source class.

Runtime validates or supplies the concrete value.

Missing capability is an assembly error when statically knowable

If the operation declares `:clock` and the trusted server environment declares no clock provider, the application should fail before the first invocation.

Waiting for a runtime `missing :biff.fx/now` exception throws away information already available at assembly time.

Dynamic capability failure remains possible

A structurally supplied external capability may fail at runtime.

For example:

- remote provider unavailable;
- database connection failed;
- clock function threw;
- publication system unavailable.

Preflight establishes the path exists.

It cannot promise external runtime availability unless a stronger premise is actually justified. Where the supported operation contract requires a handling or recovery path for such a failure, preflight must establish the declared path's structural presence and compatibility. The static handling-coverage requirement below defines that check. A declared recovery path does not prove eventual recovery without its named environmental, scheduling, or user-action premises.

Authoritative settlement and progression closure

A successful authoritative operation does not complete whole-application closure merely by committing.

The application must also know how the initiating and other relevant actors can learn the authoritative result.

Typed semantic outcome

The authoritative server/model path produces a declared semantic outcome.

Examples may include:

- confirmed;
- reconciled;
- rejected;
- already incorporated;
- superseded;
- application-specific outcomes.

The outcome schema/contract belongs to the semantic operation.

Transport-specific response shapes realize that outcome.

Settlement closure

Where the initiating browser expects direct settlement, the assembly checks that the server realization can produce a settlement compatible with the projected browser plan.

This may include:

- command correlation;
- execution correlation where required;
- typed outcome;
- progression requirement;
- authoritative/provisional resolution information.

A route that merely returns HTTP 200 is not necessarily a valid semantic settlement.

Post-commit settlement loss does not undo authority

Application closure describes a path for settlement.

It does not assume settlement delivery is guaranteed.

Part IV established:

```
authoritative commit succeeded
settlement lost

=> authoritative mutation remains true
```

Part V established that a later authoritative reread can supersede provisional presentation while command attribution may remain unknown. Assembly closure must not confuse a current-state read with command-specific outcome evidence.

Whole-application closure must therefore ensure there is some valid recovery or authoritative acquisition path, not merely one optimistic happy-path response.

Progression propagation closes the visibility relation

When a mutation establishes authoritative progression P, any downstream authoritative refresh caused by that mutation must be able to require a read consistent with the appropriate progression relation.

The application assembly records enough correspondence among:

- authoritative model operation;
- publication/invalidation;
- progression requirement;
- refresh/acquisition route;
- authoritative projection;

to ensure these pieces are not independently incompatible.

Publication declaration

If an operation requires Live invalidation or another publication to provide the application's convergence path, that publication is part of application closure.

An operation that commits but never emits the declared advisory event may still be model-correct while leaving the application realization incomplete.

The assembly should therefore know which publication mechanism is expected.

Runtime/browser testing verifies that it actually fires in the real system and that the selected publication recovery contract holds for the failures in scope. A declared publication function proves neither delivery nor recovery after a crash between commit and publication. The report must state whether convergence depends on durable publication, reconnect, explicit Refresh, or another implemented path.

Live initial acquisition and authoritative reread closure

Live invalidation is advisory.

A managed fragment must have an authoritative knowledge path.

Initial authoritative knowledge

A managed fragment may begin with authoritative content rendered in the initial page.

If so, the assembly should be able to identify:

- authoritative projection/read;
- progression/basis provenance where relevant;
- logical fragment owner;
- browser fragment identity.

Explicit acquisition path

Alternatively, the fragment may begin without content if it has an explicit permitted acquisition path.

Conceptually:

```
fragment F
    |
    + initial source: none
    |
    + acquisition:
        GET /...
        trusted authorized read
        projection P
        progression semantics Q
```

The absence of both initial content and acquisition is a structural error.

Invalidation scope

A Live declaration identifies what authoritative changes may invalidate the fragment's adopted projection.

The invalidation does not carry trusted domain content merely because it arrives through a server-originated channel.

It establishes advisory knowledge such as:

> relevant authority may be newer

Authoritative reread path

The fragment declaration identifies how the browser may obtain the current authorized authoritative representation.

The reread path must itself:

- bind trusted authentication;
- authorize protected data;
- satisfy progression requirements;
- return a projection compatible with the fragment;
- represent authoritative absence where appropriate.

Advisory frontier and adopted frontier may differ

An application may deliberately maintain:

- scoped advisory requirements indicating newer authority may need acquisition;
- adopted authoritative observations representing what the current UI has incorporated.

These need not be scalar frontiers or one common snapshot. Closure checks the supported scope and requirement-combination contract rather than treating token shape or one maximum value as sufficient evidence.

This is not automatically a bug.

It supports HumanHelp's refresh model and other applications where remote changes should be announced before visible adoption.

The assembly must nevertheless provide a valid path from advisory knowledge to new authoritative adoption where required.

Stable owner versus replaceable fragment

Where Live/continuity semantics distinguish a stable logical owner from a replaceable physical fragment, the assembly should preserve that relation.

Physical DOM replacement must not create a new semantic subscription owner accidentally.

The details of browser ownership, resource instances, and continuity remain governed by Parts V and VI. The assembly must identify the acquisition trigger and adoption policy used by the fragment coordinator; an advisory subscription does not itself authorize automatic acquisition or visible adoption.

Static application preflight

The complete normalized `ApplicationAssembly` is checked before ordinary application service begins.

This build/startup check is called application preflight.

Preflight is mandatory compilation/startup work

Application preflight is not an optional debugging command.

The supported Gesso application startup path invokes it automatically.

An application author should not need to remember:

```
run server
but first maybe remember to call require-application-assembly!
```

The supported abstraction should make the checked path the ordinary path.

Preflight consumes the production assembly

The preflight function receives or derives the canonical production `ApplicationAssembly`.

It does not reconstruct:

- a test registry;
- a simplified browser plan set;
- a mocked route graph;
- a second ”preflight-only” Live graph.

A production source declaration removed or changed must affect preflight.

Structural obligations fail closed

Preflight rejects when a required structural edge cannot be established.

Representative failures include:

- unknown semantic operation;
- exposed operation with no browser plan;
- mismatched operation/plan identity;
- stale generated browser artifact;
- missing transport;
- missing trusted route;
- route accepting wrong protocol;
- missing trusted server operation;
- server/model operation mismatch;
- missing execution capability;
- missing or incompatible declared handling/recovery path required by an operation's supported dynamic-failure contract;
- undeclared required publication;
- missing Live initial knowledge;
- missing authoritative acquisition path;
- incompatible progression/acquisition declaration;
- ambiguous physical realization;
- duplicate incompatible semantic identity.

The application does not ”try anyway” in normal supported startup.

Preflight does not predict dynamic domain state

Application preflight should succeed for a structurally valid Request Claim even when the particular current user is not authorized for a particular Request at runtime.

Dynamic examples include:

- another helper claimed the Request after render;
- Membership became inactive;
- Organization closed;
- transaction guard fails due a race;
- external provider is unavailable.

These remain trusted runtime/model outcomes.

Trying to encode all current database facts into startup preflight would make the checker both unsound and unusable.

Static coverage of declared dynamic-condition handling

Preflight need not predict dynamic facts. It must nevertheless check the structural presence and compatibility of handling/recovery paths required by the supported operation contract for dynamic conditions that could otherwise strand an operation. The existence of a provider, happy-path route, or terminal state alone does not establish that coverage.

The contract identifies the failure/outcome classes in scope and the handling obligations they create. These may include typed rejection, bounded retirement, explicit uncertain-outcome presentation, permitted authoritative reacquisition, or another declared recovery action. Preflight checks the declared continuations, required inputs/correlation, owners, and realization bindings needed by those paths where structurally knowable. It must not rely on a bare assertion that “recovery exists” when its supported declarations permit those edges to be checked.

For example:

- If a capability can be unavailable and the operation contract requires handling that outcome, the declared continuation must accept the corresponding outcome shape and have the required realization.
- If loss after submission can leave command attribution unknown, the handling contract must preserve that state and expose its declared recovery path. An unrelated current-state read cannot be wired as unconditional proof of success or rejection.
- If acquisition is the required recovery mechanism, its declared route, progression/scope contract, and triggering permission must correspond. Merely recording advisory debt does not authorize automatic acquisition or adoption.
- If continuity restoration fails after successful authoritative installation, the declared handling must not turn that failure into rollback of the already installed observation or rejection of a committed mutation.

These checks establish structural coverage of the declared supported cases. They do not prove that arbitrary handler bodies conform, that every external failure is enumerated, that authorization will hold later, or that a recovery attempt will eventually succeed. Implementation behavior and actual activation of the path require their own runtime/conformance evidence. Unsupported analysis must remain explicit rather than being presented as completed static checking.

Controlled-defect tests must remove or misbind a required handling edge and observe rejection at the applicable production checking boundary. Corresponding valid fixtures and fault scenarios must exercise the intended path. Failure conditions first knowable at invocation or concrete rendering remain checked at those boundaries; preflight does not invent the unavailable dynamic facts.

Earliest legitimate failure layer

The design follows:

> Fail at the earliest layer that possesses enough information.

Thus:

```
definition/construction
    malformed choreography
    unknown role
    impossible local transition
    inconsistent operation metadata

application preflight
    missing browser plan
    stale artifact
    missing route
    missing server capability
    missing acquisition path

runtime
    current authorization changed
    transaction guard failed
    network disappeared
```

This partition is part of the diagnostic contract.

Dynamic rendered-surface preflight

Static assembly cannot know every dynamically rendered Hiccup tree.

The running application therefore also enforces semantic rendered-surface invariants at the canonical structured response boundary.

This is not a second independent preflight system.

It is runtime enforcement of assembly rules against concrete rendered output.

Check before serialization

The preferred Gesso response path preserves structured render data long enough for the application-preflight handler boundary to inspect it before final HTML serialization.

Conceptually:

```
application handler
    |
structured Gesso/Hiccup response
    |
dynamic rendered-surface preflight
    |
serialize HTML
    |
Ring response
```

A malformed semantic affordance should be rejected before its anonymous/weakened HTML reaches the browser.

Canonical Gesso response constructors participate automatically

Applications using the supported Gesso HTML response path should receive the dynamic check through the installed application handler boundary.

They should not manually invoke a checker at every render call.

The boundary exists precisely to make runtime enforcement systemic.

Already-serialized HTML requires an explicit assurance boundary

If application code manually constructs an already-serialized HTML response before Gesso's structured guard can inspect semantic metadata, the ordinary rendered-surface proof cannot apply.

Such paths must be:

- avoided in the canonical semantic application path;
- explicitly documented as escape hatches;
- separately audited/tested if they remain necessary.

The framework must not pretend it inspected semantic Hiccup that no longer exists. Naming an escape hatch does not make its output covered by the structured-render claim. A report must identify the excluded surface or the separate checked realization that covers it. An alternate complete realization may be admitted only through its own explicit contract and evidence.

The same principle applies to streaming output, cached fragments, redirects to alternate render paths, and error handlers where they can emit managed affordances. The supported surface must identify the last boundary before delivery at which its semantic obligations can be enforced. An unchecked path must not inherit a blanket claim that every emitted semantic affordance was validated.

Render checking uses the production assembly

The dynamic guard compares concrete rendered affordances against the same production assembly used by build/startup preflight.

It must not have a different route registry or operation list. The response must be checked against the admitted generation whose bindings will realize its operations, rather than whichever mutable registry happens to be current when serialization begins.

Forged rendered metadata is not trusted authority

Rendered metadata is browser-bound output and can later be modified by the client.

The dynamic guard uses it only to verify that trusted server rendering produced a coherent semantic affordance.

The trusted execution path still re-authenticates and re-authorizes the command.

Semantic-route collision is enforced dynamically

A concrete rendered ordinary HTMX action that maps to an assembled semantic route while omitting the required semantic operation identity is rejected.

This rule applies even when:

- the path contains concrete parameters;
- a form owns the HTMX action;
- a component helper generated the action;
- rendering conditions caused semantic metadata to disappear.

The concrete render is where some of these contradictions first become visible.

Render rejection does not undo an earlier authoritative effect

Dynamic render checking must run before prohibited output is delivered. It does not automatically precede an authoritative model operation performed earlier in request handling and cannot roll back a transaction already committed.

A Request card can demonstrate why startup closure and concrete render closure need the same assembly but occur at different times.

Assume startup has already admitted an assembly containing a valid Reassign operation:

```
{:operation/id :request/reassign

 :input
 [:map
  [:request/id uuid?]
  [:helper/id uuid?]]

 :plan :plan/request-reassign

 :route :route/request-reassign

 :server-operation :server/request-reassign}
```

Whether a particular Request card can expose Reassign depends on the concrete helper target selected by application preparation.

A correct structured render might produce:

```
(def prepared-card
  {:view/id :request/card

   :request
   {:request/id R
    :request/status :claimed
    :request/helper-id Alice}

   :actions
   {:reassign
    {:operation :request/reassign

     :args
     {:request/id R
      :helper/id Bob}

     :realization
     {:plan :plan/request-reassign}}}})
```

Before HTML serialization, the response constructor checks the concrete semantic surface against the generation that will realize it:

```
(defn semantic-response
  [ctx structured-view]

  (let [generation
        (:admitted-assembly-generation ctx)

        checked
        (gesso.render/check-surface!
          generation
          structured-view)]

    (→ checked
        views/render-hiccup
        html/serialize
        ring/html-response)))
```

Now introduce the kind of defect that static assembly cannot see: the concrete render omits `:helper/id`.

```
(def malformed-card
  (update-in prepared-card
    [:actions :reassign :args]
    dissoc
    :helper/id))

(gesso.render/check-surface!
  admitted-generation
  malformed-card)
;; ⇒
{:error/type :gesso.application/render-error
 :error/kind :incomplete-operation-arguments
 :operation :request/reassign
 :view :request/card
 :missing #{:helper/id}
 :available #{:request/id}}
```

The invalid enabled control never reaches the browser.

A different defect erases semantic identity while keeping the physical route:

```
(def anonymous-action
  [:form
   {:hx-post
    (str "/app/requests/" R "/reassign")}

   [:input
    {:type "hidden"
     :name "helper-id"
     :value Bob}]

   [:button
    {:type "submit"}
    "Reassign"]])
```

The effective-action analyzer resolves method/template using the same normalized route semantics the application dispatches through:

```
(def effective
  (gesso.render/effective-action
    admitted-generation
    anonymous-action))

effective
;; ⇒
{:method :post
 :concrete-path
 (str "/app/requests/" R "/reassign")

 :matched-route
 :route/request-reassign

 :route-template
 "/app/requests/:request-id/reassign"

 :assembled-semantic-operation
 :request/reassign

 :rendered-semantic-operation
 nil}
```

That is rejected as semantic identity erasure:

```
(gesso.render/check-effective-action!
  admitted-generation
  effective)
;; ⇒
{:error/type :gesso.application/render-error
 :error/kind :anonymous-semantic-route
 :method :post
 :route-template
 "/app/requests/:request-id/reassign"
 :operation :request/reassign
 :rendered-semantic-operation nil}
```

Moving `hx-post` to an ancestor form does not avoid the check because effective dispatch, not one attribute location, is what matters.

By contrast, an ordinary search form on an unassembled ordinary route remains ordinary HTMX:

```
[:form
 {:hx-get "/app/request-search"
  :hx-target "#request-board"}

 [:input
  {:name "q"
   :value "cabinet"}]]
```

If `GET /app/request-search` is not a semantic-operation route, there is no need to invent a Choreo operation just to satisfy the checker.

The post-commit response case is deliberately different. Suppose a valid Claim command commits successfully:

```
(def mutation-result
  {:outcome :committed
   :command/id C42
   :request
   {:request/id R
    :request/status :claimed
    :request/helper-id Alice}
   :basis :X81})
```

While building the response, a bug constructs a malformed **different** affordance for Cancel:

```
(def malformed-response-view
  {:view/id :request/card

   :request (:request mutation-result)

   :actions
   {:cancel
    {:operation :request/cancel
     ;; required request/id accidentally missing
     :args {}}}})
```

Render preflight rejects the response:

```
(semantic-response
  ctx
  malformed-response-view)
;; throws before HTML delivery:
{:error/kind :incomplete-operation-arguments
 :operation :request/cancel
 :missing #{:request/id}}
```

That exception does not retroactively turn the already committed Claim into a rejection.

The request boundary must retain the known mutation result for settlement or recovery:

```
(defn handle-claim-request
  [ctx command]

  (let [result
        (request/claim
          ctx
          command)]

    (try
      (semantic-response
        ctx
        (request-card/prepare
          ctx
          (:request result)))

      (catch clojure.lang.ExceptionInfo render-error

        (recovery/respond-after-committed-render-failure
          ctx
          {:command command
           :authoritative-result result
           :render-error render-error}))))))
```

A suitable recovery response might carry trusted settlement metadata instructing the client to perform authoritative reacquisition rather than claim the mutation failed:

```
{:http/status 500

 :gesso/settlement
 ;; Produced by the trusted response boundary from the retained committed
 ;; result; an arbitrary client or copied response map cannot issue it.
 {:command/id C42
  :authoritative-outcome :committed
  :presentation-outcome :response-render-failed
  :recover
  {:kind :authoritative-reacquisition
   :projection :request/card
   :request/id R}}}
```

The exact transport is provisional. The semantic distinction is not. A client may use the committed outcome only after its settlement admission checks the authenticated source, command and owner correlation, outcome, and compatibility. If the 500 response body is lost or the selected HTMX error lifecycle does not expose that settlement path, the client must keep command disposition unknown and use the declared command-resolution or authoritative reacquisition path. The server's known commit remains committed either way. The selected supported response protocol must define how such a failure is delivered and normalized; the displayed map is not itself an admission certificate.

This is why production-path tests need to state whether they are proving ”malformed output was not delivered” or ”protected mutation did not occur.” Static assembly defects should normally prevent serving entirely. Dynamic render defects must prevent prohibited output. Neither boundary is allowed to invent a different authoritative history.

Command/protocol admission and model guards must reject invalid invocation before the protected action. If a valid mutation commits but constructing its response reveals a malformed affordance, rendering fails without reclassifying the mutation as rejected. The response/recovery path must preserve the known commit outcome or participant-relative uncertainty and permit the declared settlement or reacquisition recovery.

Integration tests must distinguish ”no malformed output was delivered” from ”no mutation occurred.” They are different claims with different enforcement boundaries.

Production-path enforcement evidence

Checker correctness and integration enforcement are separate acceptance levels.

A checker-correctness test establishes that a malformed structure is rejected when presented to the checker. An integration-enforcement test establishes that the actual supported application path routes the corresponding malformed structure through that checker before the prohibited output or effect occurs. Claims that depend on production enforcement require both forms of evidence where the distinction is relevant. A corresponding valid fixture must remain usable; rejecting every request or preventing all startup is not successful enforcement of a usable application contract.

Representative controlled defects include:

- an exposed semantic operation missing a basis or other required realization premise;
- an operation whose declared physical route no longer corresponds to its assembly binding;
- a generated browser artifact that is stale relative to the current required manifest;
- removal of the structured-rendering guard so a managed affordance would otherwise serialize unchecked.

For each controlled defect, acceptance must identify:

- the deliberate defect;
- the real production entrypoint exercised;
- the earliest boundary at which enough information exists to reject it;
- the stable diagnostic category;
- the prohibited mutation, unchecked fragment, or externally visible action that must not occur;
- the supported recovery or explicit-unavailability behavior where one exists.

Not every defect belongs at startup. Some information exists only during rendering or concrete request handling. The requirement is rejection at the earliest legitimate production boundary, not artificial promotion of every dynamic check into static preflight.

A validator cannot reject a fragment it never receives. Therefore the supported application construction path must make checked structured rendering systemic, or an alternate path must be explicitly classified as an escape hatch outside the corresponding assurance scope. Integration acceptance should become red if that enforcement connection is removed.

Controlled defects must be introduced through isolated test construction or fixtures using the supported production path. The acceptance evidence identifies the checked generation, entrypoint, expected diagnostic, and observed absence of the prohibited effect/output. It must not depend only on an exception thrown by a test-only wrapper before the production boundary is reached.

Top-level browser acceptance must operate the application's actual rendered controls. It must not repair a malformed control by manufacturing the semantic protocol command that the control failed to produce. Lower-level protocol tests may still construct valid commands when their stated scope is the trusted protocol/server boundary itself.

Biff 2 lifecycle architecture

Application preflight is integrated into native Biff 2 lifecycle semantics.

The final architecture does not preserve the preview-era model of arbitrary component functions accumulating stop callbacks.

Lifecycle-bearing modules use stable explicit identities and native start/stop contracts.

Conceptually:

```
{:biff.core/id    :gesso.live.application-preflight/handler
 :biff.core/start ...
 :biff.core/stop  ...}
```

Exact IDs and namespace names remain implementation choices. The example is schematic; the concrete integration must use the official contract of its selected final Biff 2 release and record that dependency. Requirements here describe behavior Gesso's supported assembly/startup must enforce, not a claim that every check is performed automatically by an unspecified Biff version.

Stable lifecycle identity

Every lifecycle-bearing module that participates in application startup has a stable qualified identity.

Examples conceptually include:

- Biff configuration;
- Biff XTDB;
- Gesso application preflight;
- Gesso Live;
- application/Aleph server.

Lifecycle ordering is stated in terms of identities rather than anonymous function position.

This makes the startup graph inspectable and rejectable.

Duplicate and unknown lifecycle IDs are invalid

The supported lifecycle assembly rejects:

- unknown IDs requested in startup ordering;
- duplicate module IDs;
- required module IDs that cannot be resolved;
- contradictory lifecycle assembly, including a required dependency cycle.

An application must not silently run whichever duplicate happened to appear last.

Preflight is a lifecycle dependency of serving

The critical partial order is:

```
whatever trusted/config/build state preflight requires
        |
        v
application preflight succeeds
        |
        v
network server may begin accepting application requests
```

The design does not require one universal total order among config, XTDB, Live, and every application module.

It requires that Aleph or any equivalent externally serving application boundary cannot expose the application before closure has succeeded and the admitted handler binding for the admitted assembly generation is ready. Any separately scoped health or administrative endpoint must be identified as such and cannot expose unchecked application operations under that exception.

Supported Gesso startup inserts/enforces preflight

A caller using the supported Gesso application-start abstraction must not need to manually remember the preflight lifecycle ID.

Gesso's wrapper/assembly layer inserts or enforces the required module and ordering itself.

The unsafe shape:

```
(biff/start system modules
  ;; caller forgot preflight
  [:config :xtdb :aleph])
```

must not be the ordinary Gesso path.

The supported shape is conceptually:

```
start Gesso application with ApplicationAssembly
    |
Gesso constructs/enforces lifecycle requirements
    |
preflight mandatory
    |
server permitted only after success
```

A native lifecycle composition can make that dependency explicit without asking the application author to remember preflight ordering manually.

The exact Biff 2 API remains subject to the selected release contract, but the shape should resemble one stable module per lifecycle owner:

```
(def config-module
  {:biff.core/id
   :humanhelp/config

   :biff.core/start
   (fn [ctx]
     {:config
      (load-config ctx)})})

(def xtdb-module
  {:biff.core/id
   :humanhelp/xtdb

   :biff.core/start
   (fn [ctx]
     {:db
      (start-xtdb!
        (:config ctx))})

   :biff.core/stop
   (fn [_ctx {:keys [db]}]
     (stop-xtdb! db))})

(def gesso-preflight-module
  {:biff.core/id
   :gesso/application-preflight

   :biff.core/start
   (fn [ctx]
     (let [candidate
           (prepare-application-generation!
             ctx)]

       {:admitted-generation
```

```
        candidate}))})

(def live-module
  {:biff.core/id
   :humanhelp/gesso-live

   :biff.core/start
   (fn [ctx]
     {:live-runtime
      (gesso.live/start!
        {:generation
         (:admitted-generation ctx)
         :db (:db ctx)})})

   :biff.core/stop
   (fn [_ctx {:keys [live-runtime]}]
     (gesso.live/stop!
       live-runtime))})

(def web-module
  {:biff.core/id
   :humanhelp/web

   :biff.core/start
   (fn [ctx]
     (let [generation
           (:admitted-generation ctx)

           handler
           (gesso.application/checked-handler
             generation
             (routes/application-handler
               (generation/handler-bindings
                 generation ctx)))]

       {:server
        (aleph/start-server!
          handler
          {:port
           (get-in ctx
             [:config :port])})}))

   :biff.core/stop
   (fn [_ctx {:keys [server]}]
     (.close server))})
```

The supported Gesso application start function can enforce the required partial order itself:

```
(defn start-humanhelp!
  [system modules]

  (gesso.application/start!
    {:system system

     :modules modules

     :application
     #'application-assembly

     :requires
     {:before-serving
      #{:humanhelp/config
        :humanhelp/xtdb
        :gesso/application-preflight}

      :serving
      :humanhelp/web}

     :ordered
     [:humanhelp/config
      :humanhelp/xtdb
      :gesso/application-preflight
      :humanhelp/gesso-live
      :humanhelp/web]}))
```

If the browser artifact is stale, preflight fails before `:humanhelp/web` starts:

```
(with-redefs
  [browser/current-receipt
   stale-receipt]

  (start-humanhelp!
    system
    modules))
;; ⇒
{:startup/status :failed

 :primary-error
 {:error/kind :stale-browser-artifact
  :current-manifest :M18
  :artifact-manifest :M17}

 :started-modules
 [:humanhelp/config
  :humanhelp/xtdb]

 :never-started
 [:humanhelp/gesso-live
  :humanhelp/web]}
```

Lifecycle cleanup still accounts for modules that did start:

```
{:cleanup
 [{:module :humanhelp/xtdb
   :status :stopped}

  {:module :humanhelp/config
   :status :no-stop-required}]}
```

The important externally observable fact is that the Aleph application listener never started.

The same boundary helps development reload remain coherent. Suppose A12 is currently serving and source changes produce candidate A13:

```
(def current
  {:generation/id :A12
   :status :serving})

(def candidate
  (prepare-application-generation!
    changed-source-context))
```

Candidate construction happens before activation:

```
(defn activate-candidate!
  [runtime candidate]

  (when-not
    (= :accepted
       (:preflight/status candidate))
    (throw
      (ex-info
        "Candidate generation is not admitted."
       {:error/type :application/candidate-not-admitted
        :candidate candidate})))

  ;; The implementation serializes this compare-and-activate operation,
  ;; preserves old bindings for in-flight requests, and retires them only
  ;; after all declared owners release their references.
  (generation/activate-if-current!
    runtime
    (:current-generation runtime)
    candidate))
```

A request that began under A12 keeps an A12 generation reference:

```
(def request-context
  {:transport-request/id H77
   :generation/ref :A12})

;; Later source reload activates A13.
;; This request still resolves routes, prepared operations, and render checks
;; through its admitted A12 bindings unless an explicit checked transition is
;; defined.
```

New requests use A13:

```
{:transport-request/id H78
 :generation/ref :A13}
```

Long-lived browser continuations are not silently mutated into A13; Part VI's compatibility/recovery contract decides whether they may continue, must retire, or reacquire under the new realization.

If A13 fails preflight:

```
{:candidate :A13
 :preflight/status :rejected
 :error/kind :missing-server-capability}
```

the runtime may keep the still-valid A12 serving generation:

```
{:current-generation :A12
 :candidate
 {:generation/id :A13
  :status :rejected}}
```

What it cannot expose is a half-A13 application with A13 routes, A12 browser artifact, and stale A12 preflight evidence.

This example is why ”assembly generation” is useful as a scoped identity. It binds the coherent application realization admitted for serving; it is not a universal freshness number and does not replace command, execution, resource, deployment, or artifact identities.

Native lifecycle modules are preferred over compatibility shims

Gesso-owned and application-owned lifecycle integration should use the official Biff 2 module contract directly.

Compatibility shims that preserve older component semantics enlarge the conceptual surface and make lifecycle identity less explicit.

They are not the target architecture.

Stop semantics are module-owned

Each lifecycle module owns cleanup of the resources it starts.

The final architecture does not use an application-wide convention in which arbitrary components append stop functions into a shared vector.

This aligns shutdown ownership with startup identity.

If startup fails after earlier modules acquired resources, the supported lifecycle must stop or otherwise account for those successfully started resources according to their dependency and cleanup contracts. It preserves the primary startup failure while reporting cleanup failures separately. A failed preflight must not leave a background application listener serving unchecked routes. Stopping an old generation must also respect any explicitly shared resources still owned by an admitted replacement generation.

Failed preflight means server never becomes externally available

A deliberately malformed application must fail before normal network service.

The requirement is stronger than:

```
first request returns a nice preflight error
```

The supported startup path should not expose an application that is already known to be structurally invalid.

Lifecycle preflight and dynamic render checking complement each other

Startup preflight checks the closed static assembly.

Dynamic render checking checks concrete rendered surfaces whose exact shape depends on runtime data.

Therefore:

```
startup preflight
    verifies application graph and artifact closure

dynamic rendered-surface preflight
    verifies concrete semantic affordances against that graph
```

Neither makes the other redundant.

Supported application startup boundary

The end-state Gesso application has one supported checked startup path.

That path binds:

- canonical `ApplicationAssembly`;
- browser artifact currentness;
- static application preflight;
- checked/render-aware application handler;
- lifecycle module graph;
- Live/runtime integration;
- network server startup.

No accidental second unchecked entrypoint

The application should not simultaneously expose:

```
checked dev startup
unchecked production startup
```

or:

```
checked test handler
unchecked real Aleph handler
```

unless an unchecked path is deliberately named as an unsupported escape hatch and excluded from the corresponding assurance claim. Merely documenting such a path does not satisfy a milestone requiring the real application entrypoint to be checked.

The ordinary path used for real development and deployment must be the checked one.

Handler wrapping is observable in actual runtime

It is not sufficient that a unit test can construct the checked handler.

The handler ultimately invoked by Aleph must pass through the Gesso application-preflight boundary.

This should be testable by deliberately rendering a malformed concrete semantic affordance and observing rejection before serialization/browser delivery.

Preflight state is immutable for an assembly generation

Once startup accepts an assembly generation and its bound artifact identities, the runtime may cache the normalized closure result.

If application realization declarations change, a new assembly generation must be built/preflighted.

The system must not mutate a previously verified closure report in place to accommodate changed source declarations.

Runtime reload/development mode must retain closure semantics

Development convenience may support rebuilding/reloading.

The invariant remains:

```
new source/assembly
    |
new manifest/artifact if required
    |
new preflight
    |
new serving generation
```

Hot reload must not silently pair a new server assembly with an old browser artifact merely because the process stayed alive.

The supported reload path constructs and validates the candidate generation before activating it. Activation must not expose a mixture of old routes, new prepared operations, stale artifact bindings, and unrelated rendering rules. An in-flight request remains bound to its admitted generation or follows an explicitly checked transition. Long-lived streams and browser continuations follow the compatibility and retirement contracts rather than an implicit generation switch.

If candidate construction fails, the system may retain a still-valid previously admitted generation or suspend serving the affected application. It must not partially activate the failed candidate. The implementation must state how generation references, activation, and retirement enforce these conditions; this design does not mandate one hot-reload mechanism.

Application explanation

A closed application should be inspectable.

Tooling should be able to explain the complete realization of an operation without forcing a human or LLM to reconstruct it from dozens of namespaces.

Conceptually useful queries include:

```
(choreo/explain-application application)
(choreo/explain-operation application :request/claim)
(choreo/open-obligations application)
```

These names are illustrative only.

The capability is normative.

For a closed HumanHelp Claim operation, an explanation should be rich enough that a developer does not need to grep the repository to reconstruct the path.

An illustrative query:

```
(gesso.application/explain-operation
  admitted-generation
  :request/claim)
```

could produce structured data resembling:

```
{:operation
 {:id :request/claim
  :source
  {:ns 'net.humanhelp.site.model.request.choreo
   :var 'claim}

  :role :helper

  :semantic-status
  {:semantic-checker/status :accepted
   :proof-kernel/status :accepted-under-stated-premises
   :certificate
   :cert/request-claim-v12}}

 :exposure
 [{:view :request/card
   :slot :claim
   :source
   {:ns
    'net.humanhelp.example.components.request-card
    :var 'request-card}
   :status :assembly-verified}]

 :browser
 {:plan
  {:id :plan/request-claim
   :status :assembly-verified}

  :manifest
  {:id :M18
   :status :assembly-verified}

  :artifact
  {:path "resources/public/gesso-live.M18.js"
   :digest "sha256:..."
   :receipt :R18
```

```
  :status :assembly-verified}}

:transport
{:method :post
 :route-template
 "/app/requests/:request-id/claim"
 :route :route/request-claim
 :status :assembly-verified}

:trusted-server
{:operation :server/request-claim

 :source
 {:ns 'net.humanhelp.example.server.request
  :var 'claim-handler}

 :required-capabilities
 #{:authenticated-principal
   :transaction
   :clock
   :authoritative-request-read}

 :supplied-capabilities
 #{:authenticated-principal
   :transaction
   :clock
   :authoritative-request-read}

 :closure-status :assembly-verified

 :runtime-premises
 #{:current-principal
   :current-membership
   :current-request
   :current-organization}}

:model
{:operation :request/claim
 :source
 {:ns 'net.humanhelp.site.model.request.core
  :var 'claim}

 :authority :request}

:settlement
{:outcomes
 #{:accepted :rejected}

 :status :assembly-verified}

:delivery-uncertainty
{:knowledge :unknown
 :recovery :route/request-card
 :command-attribution :unknown
 :status :assembly-verified}

:acquisition
{:projection :request/card
```

```
  :initial
  :route/request-card

  :reread
  :route/request-card

  :progression :xtdb/snapshot

  :status :assembly-verified}

 :assurance
 {:semantic-operation
  {:semantic-checker :accepted
   :proof-kernel :accepted-under-stated-premises}
  :application-closure :assembly-verified
  :startup-enforcement :runtime-enforced
  :current-authorization :runtime-enforced
  :htmx-integration :tested
  :router-correctness :trusted}

 :assembly
 {:generation :A13
  :status :preflight-closed}}
```

That output is valuable even when everything is green because it preserves source ownership and evidence classification.

The same query on an incomplete candidate should expose open obligations rather than fabricate a near match.

Suppose the route exists, but the trusted server operation no longer declares `:clock`:

```
(gesso.application/explain-operation
  broken-candidate
  :request/claim)
;; ⇒
{:operation
 {:id :request/claim}

 :assembly
 {:generation :candidate-A14
  :status :rejected}

 :open-obligations
 [{:obligation/id
   :request-claim/server-clock

   :edge
   [:server/request-claim
    :requires-capability
    :clock]

   :introduced-by
   {:operation :request/claim
    :source
    {:ns 'net.humanhelp.site.model.request.choreo
     :var 'claim}}

   :expected
   {:capability :clock}

   :provider
   {:id :server/request-claim
    :declared
    #{:authenticated-principal
      :transaction
      :authoritative-request-read}}

   :error/kind
   :missing-server-capability}]}
```

A repair tool can suggest what is mechanically known without changing execution:

```
{:suggestions
 [{:kind :inspect-provider
   :source
   {:ns 'net.humanhelp.example.server.request
    :var 'operations}}

  {:kind :available-capability-provider
   :capability :clock
   :provider :application/system-clock}]}
```

The suggestion does not automatically bind the clock. Only source correction and new assembly admission can do that.

An unknown operation can likewise show near names diagnostically:

```
(gesso.application/check-rendered-operation
  admitted-generation
  {:operation :request/claime
   :source
   {:view :request/card
    :slot :claim}})
;; ⇒
{:error/type :gesso.application/render-error
 :error/kind :unknown-operation
 :operation :request/claime

 :known
 #{:request/claim
   :request/create
   :request/cancel
   :request/reassign}

 :near
 [:request/claim]

 :auto-corrected? false}
```

The machinery can also distinguish a structural error from a dynamic denial. If Alice is no longer authorized to Claim R, the explanation of the assembly remains closed:

```
(gesso.application/explain-operation
  admitted-generation
  :request/claim)
;; ⇒
{:assembly
 {:status :preflight-closed}
 ...}
```

The invocation can still return a runtime/model outcome:

```
(request/claim
  current-runtime-context
  {:request/id R
   :command/id C42})
;; ⇒
{:outcome :rejected
 :reason :membership-inactive}
```

A diagnostic that rewrote that as `:application-assembly-corrupted` would be actively misleading.

For a complex operation, explanation can expose the graph itself:

```
(gesso.application/explain-path
  admitted-generation
  :request/claim)
;; ⇒
[{:from :view/request-card.claim
  :edge :affordance-realizes-operation
  :to :op/request-claim
  :status :accepted}

 {:from :op/request-claim
  :edge :operation-admitted-by-choreography
  :to :choreo/request-claim.helper
  :status :accepted}

 {:from :choreo/request-claim.helper
```

```
 :edge :projection-correspondence
 :to :plan/request-claim
 :status :accepted}

{:from :plan/request-claim
 :edge :artifact-contains-exact-plan
 :to :artifact/gesso-live
 :status :accepted}

{:from :artifact/gesso-live
 :edge :emits-transport
 :to :transport/request-claim
 :status :accepted}

{:from :transport/request-claim
 :edge :dispatches-to-route
 :to :route/request-claim
 :status :accepted}

{:from :route/request-claim
 :edge :trusted-dispatch
 :to :server/request-claim
 :status :accepted}

{:from :server/request-claim
 :edge :delegates-to-model-operation
 :to :model/request-claim
 :status :accepted}

{:from :model/request-claim
 :edge :typed-settlement
 :to :settlement/request-claim
 :status :accepted}

{:from :settlement/request-claim
 :edge :authoritative-reacquisition
 :to :acquire/request-card
 :status :accepted}

{:from :acquire/request-card
 :edge :renderable-projection
 :to :render/request-card
 :status :accepted}]
```

The value of such an explanation is not that application runtime should carry a large graph on every request. The graph is construction and diagnostic evidence. It lets humans, tests, and future proof tooling inspect the same realization that startup actually admitted. The linear path displayed above shows direct settlement. The closure report also includes the separate `:claim/plan→unknown-recovery` and `:claim/recovery→acquisition` edges when response delivery is lost. That reacquisition recovers the Request view without claiming command attribution.

That shared inspectable object is the opposite of the failure mode this part is trying to eliminate: a working-looking application whose browser, route, server, model, and test layers each tell a slightly different story about what operation is supposedly happening.

Whole-application semantic audit surface

The whole-application assembly should expose the auditable semantic surface introduced in Part II as a compact, structured application artifact.

This is not a second policy document maintained beside the application. It is a view derived from the same admitted semantic identities, rules, operations, authorities, realization edges, trusted premises, and source correspondence that construction and

preflight actually use.

A large application may contain tens or hundreds of thousands of lines of Clojure, ClojureScript, Hiccup, CSS, adapters, queries, and integration code. Whole-application inspection should not require a reviewer to read all of that material merely to discover the correctness-relevant policy that the admitted application claims to implement.

For an admitted generation, tooling should therefore be able to derive a semantic audit summary resembling:

```
HumanHelp semantic surface

scope: admitted declarations, checked construction products, and explicitly
       registered opaque implementation boundaries

business rules                     173
authoritative operations            35
lifecycle transitions               18
authorization/disclosure policies   12
cross-operation invariants           9
explicit opaque/trusted premises     4
unresolved declared bindings         0
```

The counts are illustrative. Their purpose is not to create a quality score. Their purpose is to expose the size and shape of the application's claimed semantic surface and to make exceptional or opaque portions easy to find. Zero unresolved **declared** bindings means that the admitted graph closed over its declared scope. It does not establish that arbitrary Clojure bodies contain no undiscovered policy branch. Each body outside the supported checked subset whose conformance matters to a claim must remain an explicit implementation boundary or premise; the report must state what was inspected and what remains opaque. A count cannot certify the completeness of undeclared business intent.

The audit surface should support at least the following derived views where the underlying application contains the corresponding semantics.

1. Business-rule catalog

   The rule catalog enumerates each named correctness-relevant business rule with its stable semantic identity.

   Where available, an entry should preserve or link to:

   - its machine-checkable proposition or relation;
   - its domain meaning;
   - its rationale and intentionally surprising exceptions;
   - the operations, transitions, observations, or invariants that depend on it;
   - its authority or authorities where authority is relevant;
   - the source declaration that owns it;
   - the declared realization or enforcement mechanism and the separate evidence, if any, that it actually discharges the rule;
   - any explicit opaque or trusted premise on which it depends;
   - the assurance classification currently justified for the rule.

   For example:

```
{:rule/id
 :request/claim-requires-active-membership

 :meaning
 "At Claim commit, the authenticated helper has an active Membership in the
  Organization of the Request being claimed."

 :rationale
 "Claiming transfers operational responsibility for the Request."

 :applies-to
 #{:request/claim}

 :authority
 :request
```

```
:enforcement
{:required-kind :authoritative-transaction-guard
 :operation :request/claim}

:source
{:ns 'net.humanhelp.site.model.request.rules
 :var 'claim-requires-active-membership}

:assurance
{:declared-rule-to-guard-binding
 {:class :assembly-verified
  :subject :request/claim-membership-guard-edge}

 :guard-implementation-conformance
 {:class :trusted
  :premise :humanhelp/claim-transaction-guards-conform}}}
```

The exact data shape is illustrative. The requirement is that a reviewer can move from semantic rule to meaning, use, source, and enforcement without reconstructing the relationship by repository-wide search. In this example, `:assembly-verified` applies to the checked declaration/closure edge. It does not establish that the XTDB transaction protects active Membership and the Request/Organization association against concurrent changes. The separate `:trusted` premise makes that unproved implementation obligation visible; a stronger applicable checker or concrete runtime evidence may justify a more specific classification. Neither classification is earned by writing the sample map or by giving a transaction the right name.

2. Operation cards

   Each semantic operation should have a compact operation card summarizing the correctness-relevant contract already present in the assembly.

   An operation card may include:

   - actor or principal requirements;
   - named preconditions and business rules;
   - authority and authoritative observations;
   - atomic state transition or effect;
   - typed outcomes;
   - command identity and retry semantics;
   - delivery-uncertainty behavior;
   - settlement and reacquisition path;
   - source ownership and realization status.

   The operation card is a summary view of the canonical operation and its closure edges. It must not become an independently editable source of semantics.

3. Authorization and disclosure view

   Where operations or observations carry authorization or disclosure conditions, tooling should be able to present them together rather than forcing a reviewer to inspect each route or model function independently.

   A conceptual view might show:

```
operation / observation        principal condition        protected by

request/read                   Organization member         :request/read-policy
request/claim                  active helper               :request/claim-policy
request/complete               current claimant            :request/complete-policy
request/private-note/read      current claimant or author  :request/note-read-policy
```

   Such a view is especially useful for exposing asymmetry. An unexpectedly weaker rule on one analogous operation becomes visible even when every individual operation is internally well formed.

   The view does not infer human intent. It faithfully reports the semantics the application actually admitted.

4. Lifecycle and transition view

Where the semantic model defines lifecycle states or transition restrictions, whole-application explanation should be able to render the admitted transition relation compactly.

For example:

```
:open      -> :claimed
:open      -> :cancelled
:claimed   -> :completed
:claimed   -> :cancelled
:completed -> [terminal]
:cancelled -> [terminal]
```

The representation may be structured data, a graph, a table, or another inspectable form. It should be generated from admitted semantic transition facts rather than reconstructed from route names or UI affordances.

5. Rule-dependency view

   Rules and invariants often depend on other semantic facts. The assembly should retain enough dependency information to answer questions such as:

   ```
   which operations depend on :membership/active?
   which invariants depend on :request/single-owner?
   which rules rely on :organization/request-policy?
   which assured claims rely on an opaque premise?
   ```

   This dependency view supports both local repair and impact analysis. Changing a rule can reveal the semantic objects whose assurance must be reconsidered without pretending that ordinary source dependency alone captures semantic dependency.

6. Opaque-premise report

   Every correctness-relevant dependency that crosses an admitted opaque or trusted boundary should be easy to enumerate.

   A report should identify at least:

   - the premise's semantic identity;
   - why it is opaque to the supported semantic path;
   - which rules, operations, or assurance claims depend on it;
   - the source or external system that supplies it;
   - the evidence classification permitted for claims that rely on it.

   A four-line opaque-premise report in a very large application is more useful than allowing those four dependencies to disappear among thousands of ordinary function calls.

   The presence of an opaque premise is not by itself an error. Hiding an opaque premise while presenting dependent conclusions as if they were mechanically established is an error in assurance accounting. This report enumerates registered boundaries. It cannot prove that uninspected arbitrary host code contains no additional policy simply because no such premise was registered.

7. Assurance-coverage view

   Whole-application explanation should be able to summarize how the admitted semantic surface is covered. Each row names the particular proposition or obligation to which its evidence class applies. A checked assembly edge, a guard implemented by a production transaction, and an abstract invariant checked over a finite model are different subjects even when all concern Claim.

   Conceptually:

   ```
   subject / obligation                                classification

   :request/open-at-claim declared guard binding        assembly-verified
   :request/open-at-claim transaction conformance       trusted
   :membership/current-at-claim transaction conformance trusted
   :request/single-owner within stated finite model     exact-finite-checked
   :request/single-owner in production                   trusted
   :sms/provider-delivers-message                        trusted
   :browser/fetch-conforms-to-envelope                    tested
   ```

The vocabulary must use the assurance classifications defined elsewhere in this design; the report must not collapse distinct evidence classes into percentages or a single ”verified” score. The illustrative `trusted` rows require named premises and explicit applicability; an implementation with stronger checked evidence can report that evidence for its exact subject. A finite model result does not by itself establish the corresponding property for arbitrary production behavior.

Coverage views are diagnostic accounting. Part IX determines what compositional certification may actually be claimed from the underlying evidence.

8. One semantic source, many audit projections

   The business-rule catalog, operation cards, authorization view, transition view, dependency graph, opaque-premise report, and assurance-coverage view are multiple projections of one admitted semantic and assembly object.

   They must not become separately maintained documents whose contents may drift:

```
source semantic declarations
        |
        v
checked semantic / application assembly
        |
        +--> executable realization
        +--> browser/server closure
        +--> business-rule catalog
        +--> operation cards
        +--> authorization/transition views
        +--> opaque-premise report
        +--> assurance-coverage view
```

   Explanatory prose retained from semantic declarations may appear in these views, but prose does not gain mechanical authority merely because it is rendered beside a checked proposition.

   Likewise, absence from an audit view must not be interpreted as proof that no corresponding real-world business rule exists. The report describes the semantic surface the application has expressed and admitted, together with its declared opaque boundaries. It must identify the inspected/generated subset for which rule/implementation correspondence is complete and the arbitrary host bodies for which conformance remains a premise. Human or heuristic review may still discover that an intended rule is missing entirely or that an uninspected host branch changes policy without a corresponding declaration.

   The whole-application audit acceptance criterion is:

   > A reviewer should be able to inspect the correctness-relevant semantic policy of an admitted Gesso application without reconstructing it from the complete implementation. The audit views must be generated from the same semantic and assembly objects used for execution and assurance, must expose explicit opaque premises rather than bury them, must state their inspection and conformance scope, and must preserve enough source and dependency correspondence to move from a suspicious rule or claim back to the code that owns and realizes it.

Explain exposed operations

For each exposed semantic operation, explanation should identify at least:

- semantic operation identity;
- source declaration;
- participating role/projection;
- views/components exposing it;
- browser executable plan where applicable;
- exact browser artifact identity;
- transport binding;
- trusted route;
- trusted server operation;
- authoritative model operation;
- required server capabilities;
- settlement/progression contract;
- authoritative acquisition path;
- current assembly status.

Explain source ownership

The report should distinguish:

```
operation comes from:
    request.choreo

route comes from:
    application routes

server prepared execution comes from:
    trusted server integration

acquisition comes from:
    Live fragment declaration
```

This makes contradictory declarations repairable locally.

Explain assurance classification

The report should identify whether each statement is:

- mechanically proved under named assumptions;
- proof-kernel-checked or exact-finite checked within its stated scope;
- assembly-verified;
- runtime-enforced;
- trusted or application-asserted;
- supported by tests.

Liveness premises qualify the corresponding claim; they are not a separate evidence mechanism. Explanation must preserve the classifications from Part I rather than compress them into a single confidence label.

This prevents ”verified” from becoming an ambiguous prestige label.

Open obligations are explicit

An incomplete assembly should be able to enumerate unresolved structural obligations.

For example:

```
operation :request/claim

OPEN:
    browser-plan -> generated-artifact
        current manifest: M17
        generated artifact: M16

OPEN:
    server-operation -> required-capability :clock
        supplied capabilities do not include :clock
```

Once structural closure succeeds, open structural obligations are zero.

Dynamic runtime premises remain classified separately rather than pretending to be unresolved compile-time obligations.

Explanation is read-only

Application explanation is diagnostic.

It may inspect the assembly/check result.

It must not repair missing edges automatically or choose a fuzzy near match for execution.

The source declaration is corrected and the assembly is checked again.

Structured diagnostics are part of the application contract

Important application-closure failures use stable structured diagnostic data.

Human-readable strings are rendered from that data.

The framework should not make application authors or LLMs parse arbitrary prose to determine what broke.

A conceptual diagnostic shape is:

```
{:error/type :gesso.choreo/application-error
 :error/kind :missing-server-capability
 :operation :request/claim
 :source {:namespace 'net.example.views
          :semantic-location :request-card/claim}
 :assembly-id ...
 :closure-edge [:transport :server-capability]
 :expected {:capability :clock}
 :actual nil
 :available #{:transaction
              :authenticated-principal
              :authoritative-basis}
 :suggestions [...]}
```

Exact keys remain a design decision.

The information requirements do not.

Minimum diagnostic questions

A useful closure failure answers:

1. What failed?
2. Which semantic operation/trajectory/fragment was involved?
3. Where was the obligation introduced?
4. Which edge of whole-application closure failed?
5. What was required?
6. What was actually available?
7. What source owns the missing or conflicting declaration?
8. What known legal alternatives exist, if mechanically enumerable?
9. Which assurance layer detected the problem?
10. Which admitted generation and relevant semantic/artifact identities were used?

Diagnostics must describe only evidence actually available. They must not invent the provenance of an unknown value or present a declared provider as a proved implementation. Sensitive principal, request, token, or model information must follow the disclosure policy of the diagnostic surface; useful internal repair evidence need not be exposed verbatim to an unauthorized client.

Stable error kinds

Important classes should have stable machine-readable identity.

Candidate classes include concepts equivalent to:

- `:unknown-operation`;
- `:missing-browser-plan`;
- `:stale-browser-artifact`;
- `:missing-transport`;
- `:incompatible-transport`;
- `:missing-route`;
- `:ambiguous-route`;
- `:missing-server-operation`;
- `:server-operation-mismatch`;
- `:missing-server-capability`;
- `:missing-initial-acquisition`;
- `:missing-authoritative-reread`;
- `:progression-acquisition-mismatch`;
- `:anonymous-semantic-route`;
- `:implicit-semantic-downgrade`;
- `:duplicate-semantic-identity`;
- `:lifecycle-preflight-missing`;
- `:artifact-content-mismatch`;
- `:incompatible-capability-provider`;
- `:assembly-generation-mismatch`;
- `:unchecked-semantic-surface`.

Exact keyword namespace/names may differ.

The semantic categories should remain stable once public.

Unknown operation diagnostic

Given a rendered affordance referencing an unknown operation:

- fail before browser startup/delivery;
- identify the view/source;
- identify the unknown operation;
- list known operation IDs when cheap;
- suggest nearby names diagnostically;
- never auto-correct the identifier for execution.

Missing browser plan diagnostic

Given a valid semantic operation exposed to a browser but absent from the required projected plan set:

- identify the operation;
- identify the exposing source;
- state that semantic verification succeeded far enough to know the operation;
- identify browser realization as the missing edge;
- list available compatible plan keys when useful.

Stale generated artifact diagnostic

Given:

```
current BrowserAssemblyManifest ≠ BrowserArtifactStamp
```

the error should report:

- current assembly/manifest identity;
- generated artifact identity;
- relevant mismatching plan set if available;
- the supported rebuild action conceptually required.

It must not guess currentness from timestamps or filenames.

Missing transport/route diagnostic

A missing physical route should identify:

- semantic operation;
- required transport contract;
- expected method/template/protocol;
- available candidate routes;
- whether the problem is absence or incompatibility.

A route that exists with the wrong method/protocol is not ”missing” in the same sense as no route at all.

Anonymous semantic-route diagnostic

When a concrete rendered ordinary HTMX action occupies a semantic route without the required semantic identity, the diagnostic should identify:

- concrete method/path;
- matched semantic route template;
- semantic operation associated with that route;
- rendered source;
- missing semantic identity;
- why ordinary HTMX would be a downgrade here.

This is much more useful than:

```
invalid button
```

Missing server capability diagnostic

The diagnostic should identify:

- operation(s) introducing the capability requirement;
- missing capability;
- trusted server boundary expected to supply it;
- capabilities that boundary declares;
- source declaration of the requirement where available.

This turns a late runtime ”missing now” exception into an early local repair.

Missing acquisition diagnostic

A managed fragment with neither initial authoritative knowledge nor a permitted acquisition path should report:

- fragment/logical owner;
- expected authoritative projection;
- progression requirement;
- available routes/acquisition declarations;
- source responsible for the fragment.

The system should not allow a blank technically-running UI to masquerade as a closed application.

Dynamic authorization is not an assembly error

If preflight is structurally closed but the current authenticated principal is not authorized for one current Request, the error/outcome should be classified as runtime/domain authorization.

The diagnostic should not claim:

```
application assembly corrupted
```

merely because current business state denied the action.

This anti-goal is important.

Over-eager static checking is as misleading as under-checking.

Near matches expose search space but never execute automatically

For a closed registry:

```
unknown :request/claime
nearby :request/claim
```

is useful diagnostic information.

The source still must be corrected.

Security-sensitive operation dispatch must not silently apply fuzzy correction.

Diagnostics retain exact artifact correspondence

If an error references a browser runtime locator or generated plan coordinate, diagnostic tooling must first verify exact artifact/sidecar correspondence as specified in Part V.

Application diagnostics must not use stale semantic metadata to explain a new artifact by best effort.

Preflight and browser acceptance are complementary

Whole-application closure is powerful precisely because it rejects impossible applications before browser execution.

It does not prove every physical integration assumption.

The final system therefore requires structural and concrete runtime evidence, alongside the independent behavioral-completeness evidence defined in Part I. The following distinction does not collapse production enforcement into a standalone checker test.

Structural/pre-browser evidence

Application closure should reject before normal browser execution:

- semantic route rendered anonymously;
- missing browser plan;
- mismatched plan/operation;
- stale browser artifact;
- missing physical route;
- incompatible transport;
- wrong/missing trusted server correspondence;
- missing server execution capability;
- undeclared required publication;
- broken authoritative acquisition path;
- malformed rendered semantic affordance;
- unavailable semantic realization silently downgraded to ordinary HTMX.

These are assembly contradictions.

Runtime/browser evidence

Real browser acceptance verifies that:

- the actual generated runtime loads;
- actual view controls activate the browser runtime;
- real HTMX lifecycle integration matches the supported contract;
- the expected protocol command is emitted;
- the real physical route receives it;
- trusted server execution reaches the production model;
- authoritative mutation really commits;
- settlement/Live/acquisition really reaches the DOM;
- reload confirms authoritative truth;
- multi-client advisory/adoption behavior works;
- unexpected console/page/request failures do not reveal mismatches in trusted external integration;
- deliberately injected network or rendering faults produce the declared diagnostics and recovery without inventing success or rejection.

Expected fault evidence must be asserted precisely rather than suppressed by a broad ”ignore console errors” rule. A fault test need not pretend the browser emits no diagnostic when the intended scenario deliberately breaks a request.

These are concrete implementation/assumption checks.

Neither class substitutes for the other.

A third acceptance class remains application behavioral completeness. Structural closure and production-path enforcement can both succeed for an application that simply omitted a required user capability from its declarations. Application acceptance must therefore remain independently sensitive to removal of promised capabilities even when the reduced assembly is internally coherent. Later HumanHelp acceptance expresses those requirements as user-visible capabilities and authoritative outcomes without maintaining a second route/operation registry.

Acceptance tests must not manufacture the missing browser edge

A lower-level route test may deliberately construct a valid protocol command to test:

```
valid protocol command
    -> trusted server/model path
```

A top-level browser acceptance test must not inject the command manually.

It must test:

```
rendered semantic control
    -> real browser runtime
    -> real protocol command
    -> real trusted route
```

Otherwise the test skips the exact closure edge the application is supposed to prove in use.

Whole-application security boundary

Whole-application closure must not accidentally move security authority into browser metadata or preflight artifacts.

Preflight establishes structure, not current permission

A closed assembly may establish:

```
there exists a trusted route for :request/claim
the route binds authenticated principal from trusted context
the route invokes the public Request claim operation
```

It does not establish:

```
browser-supplied user-id is authentic
current user may claim current Request forever
```

Those remain runtime trusted/model checks.

ApplicationAssembly is not a browser capability

The browser may learn some derived realization metadata.

Possessing or forging an assembly-like object cannot authorize server execution.

The server uses its own trusted assembly/registry and authenticated context.

Artifact stamps are correspondence evidence, not authorization tokens

An admitted current `BrowserArtifactStamp` supports only the defined artifact correspondence under its checked binding and trusted build assumptions. A raw stamp value proves neither content identity nor semantic validity by itself.

It does not prove:

- authenticated identity;
- operation permission;
- authoritative provenance;
- command freshness;
- model transition legality.

Identity evidence must not be promoted into authority.

Preflight result is not runtime authority

A successful application-preflight report means the structure was closed for the checked assembly generation.

It is not a token the browser sends to skip server checks.

The trusted server continues to validate every dynamic premise.

Whole-application construction invariant

At the end of the whole-application realization layer, the unified design requires:

> Every Choreo-managed affordance exposed by the application retains its semantic identity without silent erasure or substitution through browser plan, exact generated artifact, physical transport, trusted route, trusted server operation, authoritative model operation, settlement/progression, and authoritative reacquisition. Missing edges reject rather than silently downgrade. One canonical source-owned ApplicationAssembly feeds browser build, static preflight, runtime rendered- surface checking, startup, explanation, and tests while permitting distinct construction products and immutable admitted generations. Browser artifacts must correspond exactly to the current required manifest. Server execution capabilities are checked structurally without being confused with domain authority. Required declared handling paths for supported dynamic conditions are structurally checked without pretending to predict those conditions or prove unconditional recovery. Live invalidation remains advisory and every managed authoritative projection has an initial or permitted acquisition path. Native Biff 2 lifecycle identity makes application preflight a mandatory predecessor of external service, so Aleph cannot begin serving a structurally invalid application. Diagnostics identify the exact broken closure edge, while dynamic authorization and other genuinely current-world facts remain runtime/model responsibilities. Declared closure, production enforcement, and behavioral completeness are separate claims with separate evidence; none is promoted into the others merely because the same application satisfies all three. Preflight, production-path rejection tests, application acceptance, and real-browser acceptance provide complementary, not interchangeable, evidence.

Part VIII — HumanHelp proving application

This part specifies the proving application that exercises the generic contracts of Parts I–VII against production HumanHelp semantics. Its required capabilities are independent of its assembly declarations. Structural closure, production enforcement, and successful user behavior remain separate completion claims. Unresolved product choices are explicit milestone dependencies, not policies silently supplied by Gesso or inferred from an illustrative execution.

HumanHelp as the proving application

HumanHelp is the primary reference application for the executable Gesso design.

Its purpose in this architecture is not to define Gesso semantics.

Its purpose is to force those semantics to compose into a real application with:

- authoritative production models;
- authenticated principals;
- browser projections;
- optimistic lifecycle operations;
- Live advisory propagation;
- explicit authoritative reacquisition;
- rich view state;
- multiple simultaneous users;
- real HTMX/browser behavior;
- real failure and reconnect conditions.

HumanHelp is valuable precisely because it exercises the boundaries between layers.

A framework may have:

- correct model operations;
- correct choreography;
- correct browser transitions;
- correct preflight predicates;

and still fail as an assembled application.

HumanHelp therefore serves as the first broad proof-by-use surface for whole-application closure. Here, proof-by-use means scoped integration and acceptance evidence, not a formal proof that every permitted application execution is correct.

HumanHelp is a reference consumer, not framework policy

HumanHelp-specific business rules remain HumanHelp-specific.

Gesso must not acquire built-in concepts such as:

- Request claim eligibility;
- HumanHelp Membership policy;
- HumanHelp Organization policy;
- HumanHelp Request lifecycle rules;
- HumanHelp board sorting policy.

When HumanHelp exposes a missing abstraction, the design asks:

> Is this missing fact a generic requirement for constructing correct Gesso applications, or is it HumanHelp product/domain policy?

Generic examples include:

- semantic operation identity;
- operation binding;
- browser plan correspondence;
- transport requirement;
- trusted server capability;
- authoritative progression;
- acquisition path;
- application closure edge;
- structured diagnostic obligation.

Those may become Gesso concepts.

HumanHelp-specific business meaning remains in HumanHelp.

The proving application is deliberately broad

The reference surface is not one isolated Claim button.

The HumanHelp proving application must exercise:

- login/authenticated identity;
- Request creation;
- Request lifecycle actions;
- creator-local immediate incorporation;
- remote-client advisory state;
- explicit refresh/adoption;
- live search;
- sorting;
- visibility options;
- Request details;
- continuity where appropriate;
- terminal-state rendering;
- response flow;
- reconnect behavior;
- multi-client progression;
- fail-closed malformed-operation rejection.

This breadth matters because composition bugs tend to appear at joins.

A single happy-path mutation can remain green while:

- initial acquisition is missing;
- remote invalidation is fake;
- rendered operation identity disappears;
- route and browser registries disagree;
- view state has no producer;
- stale optimistic callbacks overwrite current authority.

The rich board forces those relationships into one actual application.

HumanHelp authority boundary

HumanHelp production models own authoritative domain truth.

The production model namespace family remains:

`net.humanhelp.site.model.*`

The temporary reference/proving UI remains:

`net.humanhelp.example.*`

These layers have intentionally different responsibilities.

Production model layer

The production model layer owns:

- Request business rules;
- Request state transitions;
- Membership semantics;
- Organization semantics;
- authoritative model queries;
- authoritative persistence;
- transaction guards;
- model-defined reconciliation;
- model-defined failure/outcome classes;
- public operation contracts.

The model layer remains meaningful without the example UI.

It remains meaningful without HTMX.

It remains meaningful without a browser.

It remains meaningful without Choreo for callers that use public model operations directly.

Production Request Choreo layer

The production Request choreography owns distributed programs built from the public Request model interface.

It may describe:

- roles;
- entry knowledge;
- authoritative observations;
- semantic command identity;
- provisional projections;
- settlement;
- convergence;
- distributed continuation.

It does not reimplement Request business policy.

For example:

```
Request Choreo
    |
    v
public Request core operation
    |
    v
authoritative Request semantics
```

not:

```
Request Choreo
    |
    v
copy of Request eligibility rules
```

Example layer

The example application owns integration and presentation concerns such as:

- physical routes;
- browser-facing page/fragment/stream composition;
- board presentation state;
- semantic operation binding to UI controls;
- Live advisory presentation;
- explicit refresh interaction;
- application assembly composition;
- trusted protocol boundary integration;
- example-specific HTML/Hiccup;
- CSS.

It does not become a parallel HumanHelp domain layer.

No example-domain authority

There must not be:

```
production Request rules
        +
example Request rules
```

with the example application deciding which rule set wins.

The example layer consumes production semantics.

If the production model says an operation is rejected, the example application renders/propagates that outcome.

It does not reinterpret it into success.

Production model migration precedes example workaround

When framework substrate changes require model adaptation, the production model layer is migrated rather than bypassed.

The example app must not create temporary replacement business operations merely to avoid updating production model code.

This is especially important for:

- FX state-flow changes;
- transaction semantics;
- public operation contracts;
- authoritative progression;
- authenticated principal binding.

The proving UI only proves the intended architecture if it actually consumes the production model layer.

HumanHelp application topology

The target reference topology is:

```
browser
    |
    v
net.humanhelp.example views/components
    |
    v
Gesso bound semantic operations / browser runtime
    |
    v
example physical routes
    |
    v
example trusted protocol integration
    |
    v
production Request choreography/server projection
    |
    v
production Request core operation
    |
    v
production model FX / XTDB transaction
    |
    v
authoritative state
    |
    +--> direct correlated settlement to initiator
    |
    +--> Live advisory invalidation
              |
              v
         permitted authoritative reread
              |
              v
         production-backed board projection
```

The exact namespace/function names may evolve.

The ownership direction must not.

The topology becomes more useful when read as an actual vertical slice rather than only as namespace arrows. The following code is intentionally incomplete in application breadth but substantial enough to show which layer owns which kind of decision.

The production Request model owns the domain transition:

```
(ns net.humanhelp.site.model.request.core
  (:require
   [malli.core :as m]
   [net.humanhelp.site.model.request.domain :as domain]
   [net.humanhelp.site.model.request.fx :as request.fx]))

(def ClaimCommand
  [:map
   [:request-id uuid?]
   [:helper-id uuid?]
   [:command-id uuid?]])

(defn claim
  [ctx command]
  (m/assert ClaimCommand command)
  (request.fx/run-claim
    ctx
    command))
```

Its domain layer owns the business rule:

```
(ns net.humanhelp.site.model.request.domain
  (:require
   [net.humanhelp.site.model.organization.core :as organization]))

(defn claim-decision
  [{:keys [request membership organization helper-id]}]
  (cond
    (not= :open
          (:request/status request))
    {:decision :rejected
     :reason :request-not-open}

    (not= :active
          (:membership/status membership))
    {:decision :rejected
     :reason :membership-inactive}

    (not=
      (:request/organization-id request)
      (:membership/organization-id membership))
    {:decision :rejected
     :reason :wrong-organization}

    (not
      (organization/allows-claim?
        organization
        membership
        request))
    {:decision :rejected
     :reason :organization-policy}

    :else
    {:decision :allowed
     :transition
     {:request/id (:request/id request)
      :request/status :claimed
      :request/helper-id helper-id}}))
```

The production choreography owns distributed use of that public model operation, not a copy of the rule:

```
(ns net.humanhelp.site.model.request.choreo
  (:require
   [gesso.choreo :as choreo]
   [net.humanhelp.site.model.request.core :as request]))

(def claim
  (choreo/operation
    {:operation/id :request/claim

     :actor :helper

     :input
     [:map
      [:request/id uuid?]]

     :runtime-premises
     #{:authenticated-principal
       :current-request
       :current-membership
       :current-organization}

     :provisional
     {:projection :request/claiming}

     :effect
     {:authority :request-authority
      :invoke request/claim}

     :settlement
     {:accepted
      {:reacquire :request/card}

      :rejected
      {:reacquire :request/card}}

     ;; Unknown is participant knowledge after lost delivery, not a
     ;; server assertion about the authoritative Claim outcome.
     :delivery-uncertainty
     {:recover
      {:acquire :request/card
       :command-attribution :unknown}}}))
```

The temporary proving UI binds a real control to that operation:

```
(ns net.humanhelp.example.components.request-card.core
  (:require
   [gesso.view :as gesso.view]))

(defn request-card
  [{:keys [request actions]}]

  [:article.request-card
   {:data-request-id
    (:request/id request)}

   [:header
    [:h3
     (:request/title request)]

    [:span.request-status
     (name
       (:request/status request))]]

   [:p
    (:request/summary request)]

   (case (gesso.view/require-prepared-availability!
           :request/claim (:claim actions))
     :available
     (gesso.view/action
       {:binding (get-in actions [:claim :binding])
        :class "request-card__claim"
        :pending-label "Claiming..."}
       "Claim")
     :unavailable nil)])
```

Preparation decides whether Claim is semantically available and supplies a bound operation rather than forcing the view to reimplement claim policy:

```
(ns net.humanhelp.example.board
  (:require
   [gesso.prepare :as gesso.prepare]
   [net.humanhelp.site.model.request.choreo :as request.choreo]))

(defn prepare-request-card
  [ctx request]

  {:request
   (select-keys request
     [:request/id
      :request/title
      :request/summary
      :request/status
      :request/helper-id])

   :actions
   {:claim
    (gesso.prepare/availability
      ctx
      request.choreo/claim
      {:request/id
       (:request/id request)})}})
```

Here `availability` returns either a validated `:available` binding or an explicit policy-based `:unavailable` result. An unknown operation, missing basis, malformed required argument, or absent projected plan is a construction error, never silent `nil` from the view. The preparation layer consults the production semantic operation instead of copying its `:open` eligibility test.

The example route is a physical realization owned by the example application:

```
(ns net.humanhelp.example.routes)

(def routes
  [{:route/id :route/request-claim
   :method :post
   :template "/app/requests/:request-id/claim"
    :path-parameters
    {:request-id {:to [:args :request/id] :codec :uuid}}
    :command-contract :contract/request-claim-command
    :semantic-operation :request/claim
    :trusted-capability :server/request-claim}

   {:route/id :route/request-card
    :method :get
    :template "/app/requests/:request-id"
    :path-parameters
    {:request-id {:to [:scope :request/id] :codec :uuid}}
    :projection :request/card
    :trusted-capability :server/read-request-card}

   {:route/id :route/request-board
    :method :get
    :template "/app/requests"
    :projection :request/board
    :trusted-capability :server/read-request-board}])
```

The trusted protocol integration owns principal rebinding and the current runtime/model premises:

```
(ns net.humanhelp.example.optimistic
  (:require
   [gesso.server :as gesso.server]
```

```
   [net.humanhelp.site.auth :as auth]
   [net.humanhelp.site.model.membership.core :as membership]
   [net.humanhelp.site.model.organization.core :as organization]
   [net.humanhelp.site.model.request.core :as request]))

(defn claim-handler
  [{:keys [db session] :as ctx}
   ring-request]

  (let [binding
        (gesso.server/require-binding!
          ctx
          :route/request-claim)

        command
        (gesso.server/decode-command!
          binding
          ring-request)

        principal
        (auth/current-principal!
          session)

        request*
        (request/require-current!
          ctx
          (:request/id command))

        membership*
        (membership/require-current!
          ctx
          {:user-id
           (:user/id principal)

           :organization-id
           (:request/organization-id
            request*)})

        organization*
        (organization/require-current!
          ctx
          (:request/organization-id
           request*))]

    (gesso.server/invoke!
      binding
      (assoc ctx
        :principal principal
        :current-request request*
        :current-membership membership*
        :current-organization organization*)
      command)))
```

The Claim server capability supplies the trusted conversion from a decoded semantic wire command to the model's internal command. This conversion occurs inside the admitted server operation used by `gesso.server/invoke!`:

```
(def server-operations
  {:server/request-claim
   {:semantic-operation :request/claim
    :supplied-capabilities
    #{:authenticated-principal :current-request
      :current-membership :current-organization
      :transaction :clock :authoritative-request-read}
    :model-command
    (fn [{:keys [principal]} command]
      {:request-id (:request/id command)
       :helper-id (:user/id principal)
       :command-id (:command/id command)})
    :invoke request/claim}})
```

The model-command converter is installed as trusted server code and is not selectable or replaceable from browser input. The decoded command ID is checked by the binding for its operation, contract, and correlation before conversion. The real assembly must also declare the two protected read capabilities `:server/read-request-card` and `:server/read-request-board` that the example routes refer to; otherwise application preflight rejects it.

The model operation then runs below that trusted boundary:

```
;; Internal model command constructed by the trusted prepared server
;; projection, with helper-id bound to the authenticated principal.
(request/claim
  trusted-ctx
  {:request-id R
   :helper-id Alice
   :command-id C42})
;; ⇒
{:outcome :committed
 :request
 {:request/id R
  :request/status :claimed
  :request/helper-id Alice}
 :basis :X24
 :command-id C42}
```

The public model command shown here is an internal trusted call shape. The untrusted wire command carries Request identity and semantic command correlation; the server projection independently binds `:helper-id` from the authenticated principal and checks the corresponding Membership/Organization. Neither a posted helper ID nor copied browser principal metadata can establish that model input. The example's `:route/request-card` and `:route/request-board` also require real protected read handlers in the application assembly; route declarations alone do not perform authorization.

The creator/initiator settlement path can return correlated authoritative evidence:

```
{:gesso/settlement
 {:operation :request/claim
  :command/id C42
  :outcome :accepted
  :authoritative
  {:projection :request/card
   :basis :X24
   :request
   {:request/id R
    :request/status :claimed
    :request/helper-id Alice}}}}
```

The Live publication path carries only advisory information needed by other clients:

```
{:live/event :request-authority-advanced

 :scope
 {:projection :request/board
  :organization-id 0}

 :requirement
 {:kind :newer-relevant-authority
  :minimum-basis :X24}}
```

Bob's browser does not install Request content from that payload. It records an outstanding advisory and later obtains canonical content through the authorized board acquisition route.

Finally, the canonical HumanHelp assembly composes those declarations:

```
(ns net.humanhelp.example.application-preflight)

(defn application
  [config]

  (gesso.application/assemble
    {:application/id
     :humanhelp/example

     :operations
     [request.choreo/create
      request.choreo/claim
      request.choreo/unclaim
      request.choreo/on-the-way
      request.choreo/complete
      request.choreo/cancel
      request.choreo/reassign]

     :views
     [(request-card/declaration)
      (request-board/declaration)]

     :routes
     routes/routes

     :server-operations
     optimistic/server-operations

     :acquisitions
     live/acquisitions

     :browser
     browser/declaration

     :capability-providers
     (server/capability-providers
       config)

     :lifecycle
     (runtime/lifecycle
       config)}))
```

The exact forms are provisional, but the vertical ownership is not. The example UI never becomes the owner of Request eligibility, and production model code never needs to know that the proving surface happens to use HTMX buttons, Refresh glow, or a temporary example namespace.

That is what makes HumanHelp useful as a proving application rather than merely a demo: the browser can exercise the entire stack without moving domain authority upward into the integration layer.

Global application points at the proving UI during this stage

The application startup used to test the Gesso/HumanHelp integration must route the active proving surface through the example application.

This does not mean the unfinished production site UI becomes the migration target.

The eventual production UI may later consume the same production models and Gesso machinery.

For this stage, the example application is the deliberate integration surface.

The unfinished production UI remains outside this proving scope

Files under the unfinished production site UI are not modified merely to make the proving application more aesthetically complete.

They enter scope only when a concrete shared dependency is required for the production models or the example proving surface.

This prevents two migrations from being confused:

```
prove Gesso against production HumanHelp semantics
    ≠
finish the eventual production HumanHelp UI
```

HumanHelp ApplicationAssembly

HumanHelp has one canonical source-owned `ApplicationAssembly` as defined in Part VII.

The application-specific constructor composes facts from their real owners.

Conceptually:

```
production Request Choreo
    -> semantic operation truth

example.routes
    -> physical route truth

example.optimistic
    -> trusted server realization truth

example.live
    -> Live/acquisition truth

browser declaration
    -> browser plan requirements

view/component declarations
    -> rendered semantic affordances

all
    |
    v
example.application-preflight
    |
    v
HumanHelp ApplicationAssembly
```

Application preflight namespace is the sole assembly composer

The application-preflight namespace owns composition of the canonical HumanHelp assembly.

It does not own the semantic facts it composes.

It should expose only the small production constructors needed by:

- browser build;
- startup;
- application explanation;
- static preflight;
- dynamic rendered-surface preflight;
- tests.

Tests consume the production constructor

HumanHelp preflight tests must not independently reconstruct:

- operation registry;
- route registry;
- browser manifest;
- Live acquisition graph;
- server registry.

A test should call the production assembly constructor.

Therefore changing or deleting a production declaration must cause the test to observe the resulting failure.

A shadow assembly that keeps passing after production is broken is invalid test architecture.

Browser build consumes the same assembly

The HumanHelp browser artifact is generated from the browser declaration/manifest derived from the same assembly.

The application has:

- one intended browser artifact output;
- one manifest identity;
- one currentness mechanism;
- one startup correspondence check.

The running app does not use a manually stale runtime while tests validate a different generated artifact.

Startup consumes the same assembly

The active HumanHelp development/runtime startup:

- obtains the canonical assembly;
- verifies browser artifact currentness;
- performs application preflight;
- installs the checked structured-response handler boundary;
- starts Aleph only after preflight succeeds.

There is no ordinary unchecked startup path accidentally used by the real application.

HumanHelp source ownership by namespace

The example source tree has explicit architectural responsibilities.

These responsibilities are normative even if future refactoring changes exact function names.

`example/app.clj`

`example/app.clj` owns trusted HTTP composition for the proving application.

It is responsible for:

- handler composition order;
- page/fragment/stream boundary selection;
- protocol-command extraction at the trusted boundary;
- lifecycle transport dispatch;
- Request creation route integration;
- Request lifecycle mutation route integration;
- route handler composition;
- preserving structured responses long enough for Gesso render preflight;
- carrying required trusted request context into downstream execution;
- avoiding duplicate publication/notification caused by overlapping handlers.

It does not own:

- Request business rules;
- optimistic operation semantics independent of production Choreo;
- route facts that belong canonically to `example.routes`;
- browser operation identity invented from request shape.

The hard rule is:

> A server route may require a Choreo/optimistic protocol only when the rendered semantic operation is realized through that protocol. The server does not make an anonymous ordinary POST legal merely to accommodate a broken view.

1. Structured response boundary

   Where the app returns canonical Gesso Hiccup/HTML responses, they remain structured until the dynamic application-preflight boundary has had the opportunity to inspect semantic affordances.

   The example app should not serialize semantic pages early and thereby bypass the render check.

   Any intentionally pre-serialized HTML path must identify its excluded assurance scope or its separate checked realization under Part VII. An audit label alone does not establish that emitted affordances passed structured render checking. Invocation validation must precede protected mutation; response-render rejection does not undo an already committed valid operation.

2. Creation and lifecycle mutation remain distinct operations

   Creating a Request and mutating an existing Request's lifecycle are separate semantic operations even if they share:

   - authentication;
   - transaction machinery;
   - board publication;
   - Live invalidation.

   The app should not route all mutations through one untyped generic handler that erases operation identity.

`example/application_preflight.clj`

This namespace composes the canonical HumanHelp application assembly.

It must:

- derive the browser manifest from the actual browser declaration;
- compose the actual production Request operation/choreography registry;
- consume physical route declarations from `example.routes`;
- consume Live acquisition/publication declarations from `example.live`;
- consume the exact trusted prepared-server realization from `example.optimistic`;
- include every lifecycle operation actually exposed;
- explicitly classify supported-but-not-rendered operations;
- use the same data for build and runtime preflight;
- expose inspectable assembly/explanation state.

It must not:

- hand-copy the same route list into a second registry;
- create a second production Request operation registry;
- claim an acquisition path that the running route graph does not provide;
- use a test-only browser artifact stamp.

`example/board.clj`

`example/board.clj` owns the production-backed board projection and board-level presentation state.

It is responsible for:

- obtaining Request data through production-backed query interfaces;
- representing the viewer context needed by presentation;
- search/filter state;
- sort state;
- terminal visibility;
- mine-first ordering;
- unclaimed-first ordering;
- authoritative progression/fact-version information needed by the view contract;
- preparing/binding lifecycle operation capabilities for Request cards;
- board-level advisory versus adopted-frontier state where architecturally appropriate.

It must distinguish:

```
presentation eligibility
    ≠
authoritative permission
```

and:

```
optimistic realization available
    ≠
semantic operation exists
```

1. Missing basis never erases operation identity

   If Claim requires an authoritative basis for its optimistic realization and no basis is currently available:

   ```
   operation :request/claim
       still exists

   optimistic realization
       unavailable
   ```

   The board layer may:

   - arrange for required authoritative basis to be supplied;
   - mark the action explicitly unavailable;
   - allow a separately declared non-optimistic realization if one exists;
   - cause checked rendering to reject an impossible enabled action.

It may not reinterpret missing basis as permission to emit an anonymous authoritative POST.

2. Search state

   Search is live presentation filtering.

   Typing in the search input updates the visible board according to current adopted board data.

   Search does not:

   - establish new authoritative facts;
   - change the adopted authoritative frontier by itself;
   - authorize hidden Request operations;
   - turn advisory-but-unadopted remote Requests into adopted board entries.

   This restriction is part of the HumanHelp proving contract, including when search uses a server-side read. Search is not an alternate remote-adoption action.

3. Sort and visibility options

   Board options control presentation of the adopted board projection.

   Required behaviors include:

   - newest/oldest ordering;
   - mine-first;
   - unclaimed-first;
   - show/hide terminal Requests.

   These controls must operate on real Request data and state.

   They should not rely on dead hidden fields or placeholder values that never change.

`example/components/refresh_button/attr.clj`

The Refresh attribute namespace remains presentation-only.

It may map semantic/presentation state into:

- classes;
- ARIA attributes;
- labels;
- data attributes required by the component.

It does not derive whether the board is stale from CSS or presentation.

It receives state produced elsewhere.

In particular it must not manufacture:

```
:stale? true
```

merely to trigger a visual effect.

example/components/refresh_button/core.clj

The Refresh component realizes the explicit advisory/adoption interaction.

Its state is driven by actual advisory versus adopted frontier state.

Required behavior:

```
no newer advisory
    -> ordinary Refresh appearance

newer advisory exists
    -> visible stale/new indication
    -> glow/pulse or equivalent intended treatment
    -> pending/new count or toast context where supplied

user activates Refresh
    -> authoritative reread
    -> adopted frontier advances on successful authoritative observation
    -> advisory state recomputed/cleared as appropriate
```

Refresh is not a hidden automatic adoption trigger.

The component must not install remote canonical content merely because an invalidation arrived.

1. Refresh accessibility

   Visual stale indication must have an accessible state/label counterpart where the component design requires one.

   A glow or pulse alone is not sufficient state communication.

   The exact wording is presentation design.

   The architectural requirement is that accessibility state is driven from the same real advisory state as visual presentation.

example/components/request_card/attr.clj

The Request-card attribute namespace maps real Request-card semantic/presentation state into attributes.

Special states such as:

- provisional;
- terminal;
- pending;
- status-specific styling;

must have actual producers and consumers.

The namespace does not preserve historical/dead attributes merely because CSS still names them.

It does not infer domain state from appearance.

example/components/request_card/core.clj

The Request-card component is the canonical view-level owner of lifecycle affordance rendering.

Every lifecycle control retains unconditional semantic identity.

At minimum:

```
Claim            -> :request/claim
Unclaim          -> :request/unclaim
Mark on the way  -> corresponding production semantic operation
Complete         -> :request/complete
Cancel           -> :request/cancel
Reassign         -> :request/reassign
```

Exact operation keywords follow production Request Choreo.

The essential invariant is:

> Availability of a particular realization may vary; semantic operation identity does not conditionally disappear.

1. Request-card physical realization is separate

   The component receives or derives a checked binding/realization.

   It does not decide:

   ```
   basis present?
       yes -> semantic Choreo action
       no  -> anonymous hx-post
   ```

   Instead:

   ```
   semantic operation known
       |
   checked binding/realization available?
       |
       +-- yes --> render enabled semantic action
       |
       +-- no --> explicit unavailable state or render rejection
   ```

   according to the application contract.

2. Presentation eligibility is not authorization

   A button may be hidden/disabled because server-prepared view state indicates the action is not currently useful.

   That is presentation.

   The trusted production model operation still re-establishes actual authorization at execution.

   The component must not contain a second authoritative implementation of Request policy.

3. Target identity matches DOM ownership

   Optimistic target IDs, logical target identity, and rendered DOM structure must correspond.

   A component must not claim one target while the browser runtime actually swaps or restores another.

   This correspondence is part of application/browser correctness.

example/live.clj

`example/live.clj` owns the example application's Live publication, invalidation, advisory, and authoritative reacquisition wiring.

It is responsible for:

- mapping semantic Request publication to Live invalidation;
- preserving authoritative progression requirements;
- defining managed board fragment identity;
- defining initial authoritative board knowledge or acquisition;
- defining refresh/reread acquisition;
- producing real advisory state;
- producing pending/new counts from real frontier relationships;
- producing `:stale?` from real state;
- supplying toast/advisory updates;
- handling reconnect without collapsing advisory into automatic adoption.

It does not own Request business policy.

1. No hardcoded advisory success state

   Values conceptually equivalent to:

   ```
   {:pending-open-count 0
    :stale? false}
   ```

   are only correct when derived from actual state.

   They may not be permanent placeholders while the UI claims to implement remote advisory behavior.

2. Creator-local incorporation

   When a browser creates a Request and receives the correlated authoritative outcome, the creator should be able to incorporate that Request immediately.

   The creator does not need to wait for an unrelated global Refresh merely because other browsers use advisory adoption.

   Conceptually:

   ```
   Browser A creates Request R
       |
   authoritative commit
       |
   correlated settlement / authoritative result
       |
   A adopts R immediately
   ```

   The incorporated representation must ultimately correspond to authoritative state.

   It is not merely an optimistic card that remains after commit without authoritative confirmation.

3. Remote-client advisory

   Another browser B that is already viewing the board follows different semantics:

   ```
   Request R commits
       |
   Live invalidation reaches B
       |
   B learns newer relevant authority exists
       |
   B does NOT automatically install R
       |
   B shows advisory/Refresh stale state
   ```

   This distinction is intentional product behavior.

   The invalidation establishes advisory knowledge, not remote canonical content.

4. Explicit adoption

   When B activates Refresh:

   ```
   Refresh
       |
   authorized authoritative reread
       |
   read satisfies progression requirement
       |
   new authoritative board projection
       |
   B advances adopted visible frontier
       |
   R becomes visible
   ```

   The board's visible state changes because B performed authoritative acquisition, not because SSE pushed trusted Request content into the DOM.

5. Advisory pending count

   If the UI displays a count of newly available Requests, that count must be derived from actual application semantics.

   A difference between progression coordinates is not, by itself, a count of Requests. Neither invalidation count nor transaction distance establishes the number of newly available, authorized Requests. Any displayed exact count needs an explicit counting scope and justified application metadata or read contract, including duplicate invalidations, permissions, terminal state, and local selective adoption. Such a read must not itself expand adopted membership.

   When only the existence of pending work is known, the UI may show an update indicator. It must not fabricate an exact count or show zero as if absence of count evidence established absence of pending work.

   The exact counting method is a HumanHelp product/design question.

   It must not be a permanently hardcoded decorative value.

6. Reconnect

   Reconnect may re-establish Live transport and learn that authority advanced.

   Reconnect must not silently force the board to adopt remote content if the intended HumanHelp semantics require explicit Refresh.

   The advisory/adopted distinction survives reconnect.

   If the current adopted projection cannot safely continue because of stale plan or other runtime incompatibility, the generic Part VI recovery rules may require authoritative reconstruction.

   That is distinct from ordinary remote-update adoption policy. The reconstruction contract must say which adopted membership and local choices survive, and which require a fresh authorized acquisition. It cannot use reconnect as an implicit exception to explicit adoption. If safe continuation is impossible, expose the declared recovery action rather than silently claiming normal continuity.

example/optimistic.clj

`example/optimistic.clj` owns trusted example-layer integration between the browser optimistic protocol and production Request Choreo/server realizations.

It preserves:

- the exact production Request Choreo operation registry;
- authenticated principal normalization from trusted request context;
- one prepared trusted server realization used by actual execution and preflight;
- route-selected operation/request identity checking;
- command decoding/validation;
- required/supplied server execution capability closure;
- settlement normalization.

It must not become a second business-rule layer.

1. Same prepared server for preflight and execution

   If application preflight validates prepared server realization S, actual `run-command` execution must use the admitted assembly generation's S, or an explicitly checked compatible transition. Matching operation names or equivalent-looking metadata do not establish correspondence to that admitted assembly generation. This binding is distinct from the identity of an individual command, protocol execution, or browser resource instance.

   Forbidden:

   ```
   preflight checks prepared server S1

   runtime uses separately constructed S2
   ```

   even if the two currently look equivalent.

   This is a whole-application correspondence guarantee.

2. Trusted principal binding

   Browser payload may identify a requested principal or role for protocol correlation.

   The actual HumanHelp principal is derived from authenticated trusted context.

   The server does not accept:

   ```
   browser says user U
   ```

   as proof that U is the authenticated principal.

3. Route-selected identity correspondence

   Where route parameters select:

   - operation;
   - Request ID;
   - other semantic subject;

   and the command envelope contains corresponding identities, the trusted integration checks that they agree.

   A browser may not use contradictory selectors and rely on whichever one is more privileged.

`example/routes.clj`

`example/routes.clj` is the single owner of physical route facts for the proving application.

It defines:

- page route;
- semantic lifecycle routes;
- creation route;
- fragment/acquisition routes;
- stream/advisory routes;
- any explicit physical route for Reassign or other lifecycle operations.

It exports route facts in a form consumable by both:

- runtime routing;
- `ApplicationAssembly` preflight.

It does not require application preflight to retype the route graph manually.

1. Semantic route correspondence

   Every semantic lifecycle route has one explicit operation correspondence.

   The method/path identity used by actual runtime routing and preflight is the same identity.

   The application must not have:

   ```
   preflight says:
       POST /requests/:id/claim

   runtime actually routes:
       POST /request/:id/claim
   ```

   through separately maintained maps.

2. Rendered exposure versus route availability

   A physical route may exist for an operation that the current UI does not expose.

   For example, Reassign may have a trusted route before a complete target-helper selector is present in the proving UI.

   The assembly distinguishes:

   ```
   operation supported by trusted server
   operation exposed by current UI
   ```

   rather than pretending they are identical sets.

3. Fragment and stream routes match acquisition assembly

   Every route used for:

   - initial board acquisition;
   - fragment refresh;
   - advisory stream;
   - authoritative reread;

   must correspond to the acquisition/Live declarations consumed by application preflight.

   A stale route left in the router but absent from acquisition assembly is not silently treated as the canonical path.

example/views.clj

`example/views.clj` owns composition of the proving UI.

It includes:

- page shell;
- initial board/fragment installation;
- toolbar;
- search UI;
- Board Options;
- sorting controls;
- Request list;
- Request details/accordion;
- create dialog;
- lifecycle extras;
- advisory toast;
- Refresh state;
- OOB content where supported;
- hidden state fields that have actual semantic purpose.

It does not independently rebuild transport semantics for lifecycle operations.

Lifecycle action rendering delegates through the checked Request-card/Gesso binding path.

1. Every visible special state has a producer

   If a UI state is visible, there must be a real upstream producer.

   Examples include:

   - stale Refresh;
   - pending/new count;
   - provisional Request card;
   - terminal fade/status;
   - selected/open Request state.

   Decorative state that can never occur is removed or explicitly documented as future/unwired rather than presented as implemented behavior.

2. Every hidden state field has a purpose

   Hidden form/DOM values may be useful for:

   - filter state;
   - sort state;
   - correlation;
   - board option persistence;
   - semantic transport.

   They must not accumulate as unexplained migration debris.

   A hidden field that no current producer/consumer uses should be removed or documented rather than becoming accidental protocol state.

3. Component CSS

   The component CSS files are part of the semantic integration audit insofar as they advertise application states.

   At minimum:

   ```
   example/components/refresh_button/refresh_button.css
   example/components/request_card/request_card.css
   ```

   must satisfy:

   - every special selector corresponds to an active state producer; or
   - the selector is explicitly non-active/future and does not misrepresent current behavior.

CSS does not define semantic truth.

It renders state supplied by application components.

Request lifecycle operation set

The proving application uses one generic Gesso operation-binding and application-closure mechanism for the Request lifecycle.

The required lifecycle set for the full proving milestone includes:

- Claim;
- Unclaim;
- Mark on the way;
- Complete;
- Cancel;
- Reassign.

All exposed operations use the same architectural path.

There is not one special hardcoded Gesso integration for Claim and ordinary anonymous HTMX for the rest.

Common lifecycle path

For each exposed operation O:

```
production Request Choreo O
    |
bound semantic affordance
    |
projected browser plan if required
    |
current browser artifact
    |
declared transport
    |
trusted route
    |
trusted Request server realization
    |
public production Request model operation
    |
authoritative transaction/outcome
    |
settlement/progression
    |
authoritative acquisition
    |
rendered authoritative state
```

The details may differ by operation.

The closure discipline does not.

Claim

Claim is the most detailed reference operation and remains the canonical worked example below.

Claim exercises:

- authoritative basis;
- provisional projection;
- semantic command identity;
- trusted principal binding;
- model policy;
- atomic authoritative transition;
- settlement;
- Live invalidation;
- authoritative reread;
- multi-client knowledge lag.

Unclaim

Unclaim follows the same generic operation-binding path.

Its business eligibility and resulting Request transition belong to the production Request model.

The example UI may only:

- expose a bound Unclaim capability;
- render provisional/pending state permitted by Choreo;
- send the declared protocol;
- render the authoritative outcome.

Mark on the way

Mark-on-the-way follows the same generic path.

The exact production operation identity and business preconditions come from Request Choreo/model contracts.

No example component hardcodes a substitute state transition.

Complete

Complete follows the same generic path.

If completion makes the Request terminal or removes it from the current board projection, authoritative absence/removal semantics from Part VI apply.

The UI may animate/present terminal state.

It does not decide that the Request is terminal independently of authority.

Cancel

Cancel follows the same generic path.

If cancellation changes visibility, terminal status, or board membership, those effects are learned from authoritative model results/projections.

Reassign

Reassign follows the same semantic operation-binding path when exposed.

Its UI may require additional subject/input such as a target helper.

The current proving application may distinguish:

```
trusted operation/route exists
    but
UI exposure unavailable until legitimate target-helper selection exists
```

This is preferable to exposing a malformed partial Reassign control during an intermediate checkpoint. It does not satisfy the full proving milestone. That milestone requires legitimate target-helper input and a reachable eligible case in which Reassign succeeds through the actual rendered control. The model still decides target eligibility and the resulting transition.

No lifecycle-specific fallback hole

For every lifecycle operation:

> If the semantic operation is rendered, missing realization metadata causes unavailability or rejection, never semantic erasure.

This rule is tested across the lifecycle matrix.

Independent lifecycle capability acceptance

The required set above is an independent product acceptance input. It must not be generated by enumerating only the currently exposed actions or current assembly entries. Removing an action must fail its capability test even if all remaining declarations are closed.

Capability	Required positive evidence	Negative or concurrent evidence
Claim	Eligible principal claims an available Request	Competing claim/current policy governs outcome
Unclaim	Eligible principal performs the model-defined release	Ineligible or changed ownership is handled
Mark on the way	Eligible actor reaches the model-defined resulting state	Invalid lifecycle basis is handled
Complete	Eligible actor reaches the model-defined completed state	Repeated/concurrent completion follows model policy
Cancel	Eligible actor reaches the model-defined cancelled state	Unauthorized or disallowed cancellation is handled
Reassign	Legitimate selected target is accepted in an eligible case	Ineligible, missing, or stale target is handled

Each positive case uses actual rendered affordances and verifies the specific production-model outcome through fresh acquisition. The fixture records the preconditions and expected outcome independently of the component being tested; these are not new framework business rules. A negative case may correctly render unavailability or receive a typed rejection. Passing negative cases alone never substitutes for a required positive case.

An intermediate report may identify a partial lifecycle checkpoint. It must name missing capabilities and must not call that checkpoint full HumanHelp completion.

HumanHelp claim example

Consider a Request claim initiated by helper principal U through browser actor C.

The Request card shown by C was derived from an authoritative read known to be consistent through XTDB progression/basis coordinate X17.

X17 is not a Request revision and is not a claim that the database is still exactly as it was when C rendered the card. It records the authoritative observation frontier from which C is acting, within the read's declared Request scope. It is not evidence that the whole board is a snapshot at X17. Any browser-carried basis or observation receipt must pass the trusted protocol's validation; a client assertion alone does not establish observed authority.

The Request facts represented by the card therefore enter C's knowledge with `:authoritative` provenance relative to that observation.

C creates semantic command C42 to claim Request R.

Because the Request claim choreography permits optimism, C may derive a provisional local projection from the authoritative facts it knows at X17.

Conceptually:

```
{:authority :provisional
 :command-id C42
 :observed-basis X17
 :operation :request/claim
 :arguments {:request-id R}
 :projection :request/claimed-by-principal}
```

This is not authoritative Request state and it does not reserve a future Request revision.

The projection describes a semantic provisional outcome. It need not be a literal patch to Request persistence data. The view/browser realization may render it as, for example, ”claimed by U” while keeping its provisional provenance explicit.

The projected Helper role communicates the semantic command, Request identity, and observed-basis information required by the protocol to the Request-authority role.

The client does not establish its own trusted principal identity merely by including ”principal U” in a payload.

The principal on whose behalf the Request authority acts is established from the authenticated server context and bound to the choreography according to HumanHelp authentication/authorization rules.

Conceptually:

```
browser actor
    C

authenticated principal
    U

choreographic role
    :helper

authoritative role
    :request-authority
```

A concrete web node may execute the authority projection, but web-node identity is not Request-authority identity. A retry may reach another web node without changing the semantic command or authority.

For example:

```
C42 first attempt
    |
Aleph node A

node A fails

C42 retry
    |
Aleph node C
```

The semantic command remains C42 only under the operation's declared retry and identity contract. A new transport attempt is not a new logical intent. Safe cross-node deduplication, where claimed, requires the declared durable identity, transaction, and retention contract; a command identifier alone cannot provide exactly-once effects. Node failure after submission can leave C's outcome unknown. This distributed sketch does not require multiple deployed web nodes for the initial proving milestone.

The Request-authority role invokes the public Request semantic operation:

```
request.core/claim
```

The choreography does not reproduce Request claim policy.

The authoritative Request operation may need facts exposed through public model interfaces such as:

- Request;
- Membership;
- Organization, including Organization-owned location facts where relevant.

Those checks belong to model semantics, not to the choreography simply because the choreography initiated the operation.

The authoritative database may have advanced beyond X17 before C42 is evaluated.

That is expected.

X17 records the basis from which C42 was formed. It is context for semantic re-evaluation, not a framework-level ”stale command” rejection rule.

For example:

```
X17
    R is unclaimed

C creates C42

X18
    unrelated data changes

X23
    Request authority evaluates C42
```

C42 may remain completely valid.

Alternatively:

```
X17
    R is unclaimed

C creates C42

X18
    another helper claims R

X23
    Request authority evaluates C42
```

the Request model may now confirm, reconcile, reject, or otherwise resolve C42 according to Request semantics.

The authoritative operation reads the relevant authoritative facts, derives a proposed transition, and identifies the commit-time preconditions required for that transition to remain valid.

If successful claim is an atomic domain transition, all authoritative writes constituting the claim and all required commit-time guards are applied in one XTDB transaction.

Conceptually:

```
read relevant authoritative facts
        |
derive Request claim transition
        |
prepare commit
        |
atomic transaction:
    guards + writes
        |
        +-- success --> authoritative B
        |
        +-- guard failure --> no partial B
                              re-evaluate or return
                              semantic outcome
```

Not every fact read for rendering or diagnostics needs to be guarded. The model protects the facts whose incompatible change would invalidate the claim.

Possible semantic outcomes include:

- claim confirmed;
- claim reconciled with newer authoritative state;
- claim rejected;
- command already incorporated;
- another model-defined result mapped to the choreography's typed continuation.

Once the XTDB transaction commits, the Request authority has moved from authoritative A to authoritative B.

Settlement delivery, Live invalidation, fragment refetch, and browser rendering occur after that authoritative boundary. Failure in one of those later mechanisms must not reclassify the committed mutation as rejected. The browser may nevertheless lack command-specific outcome evidence and must represent that uncertainty accurately.

The committed transaction establishes authoritative progression information that must survive the Live propagation path.

Conceptually:

```
atomic commit
    |
    v
authoritative progression X24
    |
    v
Live invalidation
    |
    v
adoption permission under the selected policy or explicit user action
    |
    v
authorized affected-fragment acquisition
    |
    v
read consistent with the invalidating transaction
    |
    v
canonical representation
```

The initiating browser may learn the command outcome through direct settlement or an authoritative result that actually attributes an outcome to C42. A later ordinary reread can establish current Request state without establishing whether C42 caused it. Superseding provisional presentation and settling command attribution are separate transitions.

Other actors do not need a dedicated ”Alice claimed R” choreography message.

Suppose Bob's browser is connected through another Aleph node.

```
Alice C
  |
Aleph A
  |
claim commits X24
  |
Live / Valkey
  |
Aleph C
  |
Bob browser receives advisory evidence
  |
selected remote-lifecycle policy or explicit Refresh permits acquisition
  |
authoritative fragment reread covering the relevant X24 requirement
```

Bob now knows the authoritative claim fact because he has observed an authoritative projection containing it.

That knowledge has `:authoritative` provenance.

Bob's later behavior may depend on the fact without requiring Alice's execution to locate Bob's actor or deliver a direct participant message.

Under the declared acquisition, scheduling, and availability assumptions, the client's provisional projection can be confirmed, reconciled, rejected, or superseded by justified evidence. This conditional convergence claim is not an unconditional delivery guarantee. Current authoritative state can supersede a projection while attribution of C42 remains unknown. Late attempt callbacks cannot restore retired provisional presentation.

Other actors may still know only facts visible through X17 or another earlier frontier. That is knowledge lag, not partial authoritative state.

Live propagation and later authoritative rereads move relevant actor knowledge forward.

No requirement says every browser or every web process must synchronously agree on X24 before the claim is authoritative.

The atomic model commit establishes B.

The choreography and Live system then provide valid paths by which actors that need B can acquire justified knowledge of B.

Request creation and creator visibility

Request creation is part of the reference board behavior even though the v4.5 worked example focuses on Claim.

The creation path must use production Request semantics and authoritative persistence.

Creation authority

The example application gathers creation input and invokes the production Request creation operation through the intended trusted server/model path.

The example app does not write XTDB directly to make the board demo work.

Any required:

- current time;
- random/UUID seed;
- authenticated principal;
- transaction capability;

is supplied explicitly by the trusted execution environment according to the Biff/Gesso capability model from Parts IV and VII.

Missing server capability fails before user interaction where knowable

If Request creation requires a clock or seed and the server assembly cannot supply it, application preflight should reject the structural omission.

The user should not discover the defect as a runtime exception such as:

```
missing :biff.fx/now
```

after filling out the Create Request dialog.

Authoritative creation result

Once creation commits, the creator receives a correlated result or performs a reread sufficient to install the authoritative new Request.

The intended user-visible behavior is immediate creator visibility once the correlated authoritative result is available, without requiring another manual Refresh. This is not a guarantee of response delivery after commit. Lost results follow the declared command/reacquisition recovery contract. Acceptance must also record how this visibility requirement interacts with active search and visibility options, rather than silently dropping either requirement.

Conceptually:

```
Browser A creates R
    |
authoritative creation commits
    |
A receives correlated authoritative result
    |
A's adopted board includes R
```

Creator immediate does not imply remote auto-adoption

The creator's immediate incorporation is correlated to the operation it initiated.

It does not imply that every connected browser automatically replaces its board projection.

Other clients follow the advisory path described below.

Board projection semantics

The board is not a generic client-side source of truth.

It is a rendered projection of authoritative production Request data plus local presentation state.

The board contract must distinguish at least these semantic concerns:

- **membership:** which Requests have been admitted to the currently adopted board;
- **observation provenance:** which authoritative observations justify the Request facts that are currently displayed;
- **advisory information:** evidence that relevant authority may contain changes that the board has not yet adopted;
- **local presentation:** search, sorting, visibility, focus, continuity, and related browser-local state;
- **provisional state:** command-correlated predictions and their runtime ownership while resolution remains pending.

These are semantic distinctions. They do not require five independent mutable stores, per-object integer revisions, or a universal persisted fact graph.

The governing rule is:

> Adoption is scoped. Incorporating a correlated authoritative result for one operation, object, or acquisition scope does not by itself establish adoption of every relevant change through that result's database progression.

Consequently, a newer authoritative observation of Request S does not by itself establish that:

- the complete board was reacquired at S's basis;
- every Request visible at that basis has been admitted;
- every previously displayed Request was refreshed;
- every outstanding advisory obligation has been discharged.

A board may therefore contain individually justified authoritative observations without claiming that the whole rendered board is one complete database snapshot. Where a view or operation requires a common snapshot, causal relation, progression relation, or other cross-object compatibility condition, that requirement must be declared and satisfied explicitly.

Adopted authoritative frontier

When the adopted board projection was acquired as one scope with one meaningful common progression relation, it is useful to describe that relation as an adopted authoritative frontier.

That phrase must not be overextended to mixed observations. If the board has selectively incorporated a newer correlated result for one Request while other membership remains intentionally older, the implementation must retain enough scope/provenance information to describe what has actually been adopted rather than pretending that the entire board advanced to the maximum observed basis.

This information does not need to be displayed to the user.

It is needed to reason about:

- which membership and Request facts are currently justified;
- whether newer advisory information remains outstanding;
- what progression requirement Refresh must satisfy;
- which observation or observed-basis claim an optimistic action is formed from;
- whether an aggregate operation requires stronger cross-object consistency.

Advisory frontier

The board may know that relevant authority has advanced beyond what it has currently adopted visibly.

Where one scoped progression relation adequately represents that knowledge, the design may describe an advisory frontier such as:

```
adopted board coverage:
    through X17 for scope Q

advisory requirement:
    observe scope Q sufficiently to satisfy X24

meaning:
    authority may contain relevant newer state
    board intentionally retains its currently adopted membership until Refresh
```

The exact representation may be a progression requirement, a scope-aware frontier relation, or another supported Gesso abstraction.

There need not be one scalar advisory number for every outstanding change. A newer observation in one scope must not erase advisory debt in another scope merely because its database token compares as newer.

Provisional state

A Request card may temporarily show provisional state associated with a semantic command.

That provisional state records or is correlated with:

- command identity;
- the relevant observed-basis claim or admitted authoritative observation;
- operation identity;
- provisional provenance;
- execution identity and declared resource-instance ownership as required by the browser runtime admission contract.

It is not folded into adopted authoritative state merely because it is visible.

Local presentation state

Search text, sort selection, visibility options, focus, open accordion state, and similar values are local/presentation state.

They do not change authoritative Request facts or board membership.

Some may survive fragment replacement through continuity mechanisms.

They do not become choreographic truth, and changing them must not admit Requests that are currently withheld by the board's adoption policy.

Advisory versus adopted authority

The HumanHelp board intentionally separates:

```
knowledge that relevant newer authority may exist
    ≠
adoption of authoritative content that satisfies that knowledge
```

This distinction is a first-class acceptance requirement.

Remote creation scenario

The canonical scenario uses at least two browser contexts.

```
Browser A
    creates Request R
        |
        v
authoritative commit at P18

Browser A
    correlated authoritative result
        |
        v
adopts R immediately

Browser B
    already viewing board adopted through P17
        |
        v
receives advisory requiring relevant observation ⩾ P18
        |
        v
does NOT automatically show R
        |
        +--> toast / update near Refresh
        +--> Refresh becomes visually stale/active
        +--> pending/new state reflects real advisory information
```

The advisory is evidence that B has an adoption obligation or opportunity. It is not authoritative Request content and does not itself add R to board membership.

Selective local adoption while remote work remains pending

The creator-local rule composes with outstanding remote advisory information. For example:

```
B has adopted its board through P17.

A creates remote Request R at P18.
B receives advisory for R but does not adopt R.

B then creates Request S at P19.
B receives the correlated authoritative result for S.

Required result:
    S appears on B's board.
    R remains absent.
    advisory information covering R remains outstanding.

B changes search or sorting.
    presentation changes over currently adopted membership.
    R still does not appear.

B explicitly Refreshes with sufficient scope/progression coverage.
    R may then be admitted through authoritative acquisition.
    advisory information is recalculated from what that read actually covered.
```

The fact that S was authoritatively observed at P19 does not imply that B performed a complete board read at P19. The implementation must not use a single maximum database token as proof that unrelated board membership has been adopted.

The same rule applies when B receives a correlated lifecycle result for an already-admitted Request while an unrelated remote creation remains awaiting Refresh.

The selective-adoption rule is easier to trust when expressed as executable state rather than as a verbal exception.

Assume Browser B has already adopted a complete authorized board through P17:

```
(def B0
  {:actor [:client :B]
   :principal {:user/id Bob}

   :owner
   {:fragment/id :request-board
    :resource-instance/id :bob-board-fragment-1}

   :live
   {:subscription/id :bob-board
    :resource-instance/id :bob-board-instance-1}

   :projection
   {:kind :request/board
    :organization-id O
    :coverage :complete-authorized-membership}

   :adopted
   {:basis :P17
    :members
    {R1
     {:request/id R1
      :request/status :open
      :request/title "Move a dresser"}

     R2
     {:request/id R2
      :request/status :open
      :request/title "Install a light"}}}

   :advisory
   {:requirements []}

   :options
   {:search ""
    :time-order :newest
    :mine-first? false
    :unclaimed-first? false
    :show-terminal? false
    :open-details #{}}})
```

Browser A creates remote Request R at P18:

```
(def create-R-result-for-A
  ;; Fixture notation for the trusted settlement boundary, not a
  ;; browser-callable constructor or an admission tag in a plain map.
  (settlement/admitted-fixture
    {:operation :request/create
     :command/id C18
     :outcome :accepted
     :observation
     {:basis :P18
      :scope
      {:projection :request/card
       :request/id R}
      :request
      {:request/id R
       :request/status :open
       :request/title "Need help moving a cabinet"
       :request/customer-id Alice}}}))
```

A may incorporate that correlated result immediately into A's own board under the declared creator-local rule:

```
(defn incorporate-local-created-request
  [board admitted-result]

  (settlement/require-current-creator-result!
    board admitted-result :request/create)
  (let [result (settlement/value admitted-result)
        observation (:observation result)
        request
        (:request observation)]

    (→ board
       (assoc-in
         [:adopted :members
          (:request/id request)]
         request)

       ;; This admits one correlated Request observation only.  It does not
       ;; manufacture a complete P18 board observation.
       (assoc-in
         [:local-correlated
          (:request/id request)]
         {:basis
          (:basis observation)

          :source
          [:admitted-command-result
           (:command/id result)]})))))
```

B instead receives an advisory:

```
(def advisory-R
  (advisory/admitted-fixture
    {:live/event :request-authority-advanced

     :scope
     {:kind :request/board
      :organization-id 0
      :coverage :complete-authorized-membership}

     :requirement
     {:requirement/id :remote-R
      :kind :newer-relevant-authority
      :minimum-basis :P18
      :scope
      {:kind :request/board
       :organization-id 0
       :coverage :complete-authorized-membership}
      :requires :acquire-and-adopt}}
    {:subscription/id :bob-board
     :resource-instance/id :bob-board-instance-1}))

(defn record-board-advisory
  [board admitted-advisory]

  (advisory/require-current-subscription!
    board admitted-advisory)
  (update-in board
    [:advisory :requirements]
    requirement/combine-preserving-scope
    (:requirement (advisory/value admitted-advisory))))

(def B1
  (record-board-advisory
    B0
    advisory-R))
```

B1 still has the same adopted membership:

```
(set (keys
  (get-in B1
    [:adopted :members])))
;; ⇒ #{R1 R2}

(get-in B1
  [:advisory :requirements])
;; ⇒
[{:requirement/id :remote-R
  :minimum-basis :P18
  :requires :acquire-and-adopt
  ...}]
```

Presentation derives the Refresh indication from real advisory state:

```
(defn refresh-presentation
  [board]

  (let [pending
        (count
          (get-in board
            [:advisory :requirements]))]

    {:pending pending

     :label
     (if (pos? pending)
       (str "Refresh (" pending ")")
       "Refresh")

     :class
     (cond→ #{"refresh-button"}
       (pos? pending)
       (conj
         "refresh-button--attention"
         "refresh-button--pulse"))

     :aria-live-message
     (when (pos? pending)
       (str pending
            " refresh requirement"
            (when-not (= 1 pending)
              "s")
            " pending."))}))
```

Nothing in that rendering decodes Request R from the advisory.

Now B creates local Request S at P19:

```
(def create-S-result-for-B
  (settlement/admitted-fixture
    {:operation :request/create
     :command/id C19
     :outcome :accepted
     :observation
     {:basis :P19
      :scope
      {:projection :request/card
       :request/id S}
      :request
      {:request/id S
       :request/status :open
       :request/title "Help carry boxes"
       :request/customer-id Bob}}}))

(def B2
  (incorporate-local-created-request
    B1
    create-S-result-for-B))
```

The desired state is now intentionally mixed:

```
(set (keys
  (get-in B2
    [:adopted :members])))
;; ⇒ #{R1 R2 S}

(mapv
  :requirement/id
  (get-in B2
    [:advisory :requirements]))
;; ⇒ [:remote-R]

(get-in B2
  [:local-correlated S])
;; ⇒
{:basis :P19
 :source [:admitted-command-result C19]}
```

The fact that S was observed authoritatively at P19 does not admit R. B did not perform a complete authorized board read at P19.

Search and sorting run over the adopted/local-correlated membership B actually has:

```
(defn visible-board-members
  [board]

  (board.presentation/present-board board))
```

Changing search to `cabinet` does not cause R to appear:

```
(def B3
  (assoc-in B2
    [:options :search]
    "cabinet"))

(map :request/id
     (visible-board-members B3))
;; ⇒ () for these local R1/R2/S titles
;; never remote R merely because its title would match
```

Only explicit Refresh **requests** this board acquisition/adoption under the settled remote-creation policy. It does not grant authority to read Requests; server-side authorization remains an independent trusted obligation:

```
(defn begin-explicit-refresh
  [board acquisition-id resource-instance-id]

  (assoc board
    :refresh
    {:acquisition/id acquisition-id

     :resource-instance/id resource-instance-id
     :owner (:owner board)

     :trigger
     {:kind :explicit-user-refresh}

     :captured-requirements
     (get-in board
       [:advisory :requirements])

     :requested-scope
     (:projection board)}))

(def B4
  (begin-explicit-refresh
    B3
    :refresh-20
    :refresh-resource-20))
```

The browser sends only that acquisition request. At the trusted server boundary, the current authenticated principal is rebound independently of the browser payload, authorization is evaluated for the requested board scope, the physical acquisition is correlated with the declared plan/resource instance, and the returned observation is admitted for provenance, scope, progression, and execution identity before it can reach installation code:

```
(def admitted-refresh-result
  ;; Fixture for an admitted current-permit installation completion, never
  ;; a client-constructible {:admission/kind ...} map.
  (acquisition/admitted-installed-fixture
    {:acquisition/id :refresh-20
     :observation
     {:scope
      {:kind :request/board
       :organization-id 0
       :coverage :complete-authorized-membership}
      :basis :P20
      :requests
      [R1-value
       R2-value
       R-value
       S-value]}}
    {:owner (:owner B4)
     :resource-instance/id :refresh-resource-20
     :principal {:user/id Bob}
     :authorization :request-board/read-authorized}))
```

Installation replaces complete-board membership only from that admitted observation while preserving compatible local presentation state: The admission predicate below checks the current fragment owner, acquisition instance, authorized scope, progression relative to installed state, and adapter-confirmed physical installation. A network response or copied `:admission/kind` tag cannot discharge a visible-adoption obligation.

```
(defn install-refreshed-board
  [board admitted-result]

  (acquisition/require-installed-for-current-board!
    board admitted-result)
  (let [observation
        (:observation (acquisition/value admitted-result))
        members
        (into {}
          (map
            (juxt :request/id identity))
          (:requests observation))

        remaining
        (remove
          #(advisory/satisfied-by?
             %
             observation)
          (get-in board
            [:advisory :requirements]))]

    (→ board
        (assoc
          :adopted
          {:basis (:basis observation)
           :members members})

        (assoc
          :local-correlated {})

        (assoc-in
          [:advisory :requirements]
          (vec remaining))

        (dissoc :refresh))))

(def B5
  (install-refreshed-board
    B4
    admitted-refresh-result))
```

Now R is legitimately adopted:

```
(contains?
  (get-in B5
    [:adopted :members])
  R)
;; ⇒ true

(get-in B5
  [:advisory :requirements])
;; ⇒ []
```

If a new advisory O2 arrives after Refresh captured its requirements and the returned observation covers only through P20 while O2 requires P21, O2 remains:

```
(def O2
  {:requirement/id :remote-T
   :minimum-basis :P21
   :scope
   (:projection B4)
   :requires :acquire-and-adopt})

(def B4-with-O2
  (record-board-advisory
    B4
    (advisory/admitted-fixture
      {:scope (:projection B4)
       :requirement O2}
      {:subscription/id :bob-board
       :resource-instance/id :bob-board-instance-1})))

(def B5-with-O2
  (install-refreshed-board
    B4-with-O2
    admitted-refresh-result))

(mapv
  :requirement/id
  (get-in B5-with-O2
    [:advisory :requirements]))
;; ⇒ [:remote-T]
```

The Refresh indicator therefore stays active.

This worked sequence is the HumanHelp-specific proof surface for several generic Gesso distinctions at once:

- one object-specific authoritative result does not imply whole-board adoption;
- advisory knowledge is not Request content;
- local presentation transforms only admitted membership;
- an explicit acquisition/adoption trigger remains distinct both from knowledge that newer authority exists and from trusted server authorization to read it;
- advisory discharge follows actual acquisition coverage;
- local correlated creation and remote selective adoption can coexist without a global ”latest revision” shortcut.

Explicit Refresh

Browser B activates Refresh.

Refresh performs a trusted authoritative reread with a declared acquisition scope and progression requirement. The returned observation establishes only the coverage justified by that read.

On a simple successful full-board Refresh:

```
board acquisition satisfying captured requirement
    |
authorized authoritative content installed
    |
newly covered membership becomes eligible for display
    |
advisory obligations actually satisfied by the acquisition are discharged
    |
other outstanding advisory information, if any, remains represented
```

Completion of the HTTP request is not itself proof that every currently known advisory has been resolved.

In particular:

- an advisory arriving after the Refresh captured its requirement remains outstanding unless the returned observation actually covers it;
- a local authoritative result installed while Refresh is in flight must not be undone by an older response;
- a response that is incompatible with newer justified state may be rejected, followed by a successor read, or handled by another checked reconciliation strategy;
- the implementation must not merge incompatible snapshots merely to avoid a successor read;
- failed Refresh leaves existing adopted content correctly classified and does not erase unresolved advisory information;
- the application retains a finite supported recovery path after failure.

The implementation must declare the authorization scope of one Refresh action: which membership/query scope it may adopt, what requirement it captures, and whether an automatic successor acquisition remains authorized by that action. Accumulating advisory debt does not itself extend acquisition or adoption permission. In particular, the contract must decide whether later advisories are eligible within that action or need another explicit action; tests exercise the selected rule. A successor read used to resolve an installation race must not silently become perpetual automatic remote adoption.

Discharge requires justified coverage and successful semantic adoption under the browser realization contract, not merely receipt of a newer token or a successful HTTP response. Partial failure must not erase uncovered obligations or enable controls against an incoherent partially installed projection.

These rules use the generic progression, scope, and browser-ownership semantics from the preceding parts rather than inventing a HumanHelp-specific revision counter.

SSE/Live is advisory

The SSE/Live stream does not push canonical Request card HTML that becomes authoritative merely because it arrived over a server connection.

The stream may carry:

- invalidation;
- progression requirement;
- advisory scope;
- other non-authoritative coordination metadata.

Canonical domain content is obtained through the permitted authoritative read path.

Why explicit adoption matters

The HumanHelp behavior demonstrates that Gesso can represent a useful state in which:

- the browser knows its visible projection may be stale or incomplete relative to relevant newer authority;
- every displayed authoritative fact retains justified observation provenance;
- currently adopted membership remains intentional rather than being expanded by incidental newer observations;
- the user is notified;
- the application offers a finite path to current authority;
- new remote content is not forced into the visible board without explicit adoption.

This is stronger than treating every invalidation as an automatic DOM refresh.

Remote lifecycle changes

Remote creation has a settled product rule: a new Request is not automatically admitted to another browser's board. Advisory indication plus explicit Refresh performs that adoption.

Remote lifecycle changes to Requests already admitted to the board are a separate HumanHelp product decision. Canonical r4 records the choice explicitly rather than allowing different sections to imply different policies. The decision remains unresolved in this draft: the alternatives below are not a selected policy, and the worked Claim example does not choose one implicitly.

The permitted choices are:

- require explicit Refresh before adopting remote lifecycle changes;
- automatically perform an authorized authoritative reacquisition of an already-admitted Request;
- define a precise policy by transition or visibility consequence.

Whichever policy HumanHelp selects must preserve all of these constraints:

- Live invalidation is advisory evidence, not authoritative domain content;
- authoritative content comes through the permitted authorized acquisition path;
- a stale card does not preserve authorization or reserve an operation;
- commands undergo current authoritative model policy and appropriate commit-time validation;
- reacquiring an existing Request does not implicitly admit unrelated remote new Requests;
- absence, lack of access, and terminal domain state remain distinct where the principal is permitted to distinguish them;
- the specification does not pretend that the browser can be made to forget information that was already disclosed.

The selected policy must state behavior for at least another helper claiming a visible Request, author cancellation, terminal transition, a visibility-changing transition, and loss of permission to reacquire protected content.

This product decision must be resolved before the HumanHelp operational completion milestone can claim the affected multi-client lifecycle behavior. Until then, the generic architectural invariant is only:

```
advisory invalidation
    ≠
authoritative content installation
```

Search, sorting, and visibility

The proving board must retain its intended rich interaction.

These interactions are deliberately part of acceptance because they interact with authoritative refresh and DOM replacement.

Live search

As the user types, search filters the currently adopted board projection.

Required properties:

- filtering is responsive to current search text;
- it does not require an authoritative mutation;
- it composes with current sort/visibility options;
- fragment replacement does not leave search state incoherent;
- search does not expose Requests absent from the adopted authoritative projection.

Newest/oldest ordering

The board can order Requests by the product's defined relevant temporal field.

The exact field belongs to HumanHelp model/view semantics.

The control must actually affect rendered order.

It must not be decorative.

Mine-first ordering

Mine-first promotes Requests associated with the current authenticated viewer according to production-backed Request facts.

The view may use trusted/prepared projection data.

It must not invent ownership from provisional CSS state.

Unclaimed-first ordering

Unclaimed-first uses production-backed Request claim state.

A provisional local Claim may affect the local provisional presentation as declared by Choreo, but authoritative ordering after settlement is based on authoritative Request state.

Terminal visibility

Show/hide terminal controls whether Requests whose production model state is terminal remain visible.

The view does not define which statuses are terminal independently of production model semantics.

Option composition

Board options compose deterministically.

For example:

```
adopted authoritative Requests
    |
terminal visibility
    |
search filter
    |
sorting / mine-first / unclaimed-first
    |
render
```

The precise ordering of presentation transforms is part of HumanHelp UI semantics and should be tested.

It must not depend on accidental DOM insertion order.

A concrete board presenter can keep these controls substantial enough to test without turning them into another domain model.

Suppose the adopted board contains authoritative Request facts already prepared for presentation:

```
(def adopted-requests
  [{:request/id R1
    :request/title "Move a cabinet"
    :request/status :open
    :request/helper-id nil
    :request/customer-id Alice
    :request/created-at #inst "2026-09-21T18:00:00Z"
    :request/terminal? false}

   {:request/id R2
    :request/title "Install shelves"
    :request/status :claimed
    :request/helper-id Bob
    :request/customer-id Carol
    :request/created-at #inst "2026-09-21T18:05:00Z"
    :request/terminal? false}

   {:request/id R3
    :request/title "Carry boxes"
    :request/status :completed
    :request/helper-id Bob
    :request/customer-id Alice
    :request/created-at #inst "2026-09-21T17:30:00Z"
    :request/terminal? true}])
```

Local board options can be ordinary immutable presentation data:

```
(def default-options
  {:search ""
   :time-order :newest
   :mine-first? false
   :unclaimed-first? false
   :show-terminal? false
   :open-details #{}})
```

Terminal filtering depends on production-prepared `:request/terminal?` rather than re-declaring which statuses are terminal in the view:

```
(defn apply-terminal-visibility
  [requests {:keys [show-terminal?]}]

  (if show-terminal?
    requests
    (remove :request/terminal?
            requests)))
```

Search is local over the adopted projection:

```
(defn normalize-search
  [s]
  (→ (or s "")
      str/lower-case
      str/trim))

(defn matches-search?
  [request query]

  (let [q
        (normalize-search query)]

    (or
      (str/blank? q)

      (str/includes?
        (normalize-search
          (:request/title request))
        q))))

(defn apply-search
  [requests {:keys [search]}]
  (filter
    #(matches-search?
       %
       search)
    requests))
```

Time ordering is deterministic:

```
(defn created-order
  [mode]

  (let [direction
        (case mode
          :newest -1
          :oldest 1
          (throw
            (ex-info
              "Unsupported Request order."
              {:mode mode})))]
    (fn [left right]
      (let [by-time
            (compare
              (:request/created-at left)
              (:request/created-at right))]
        (if (zero? by-time)
          (compare
            (str (:request/id left))
            (str (:request/id right)))
          (* direction by-time))))))

(defn apply-time-order
  [requests {:keys [time-order]}]

  (sort
    (created-order time-order)
    requests))
```

Mine-first uses the authenticated/principal-relative prepared fact:

```
(defn mine?
  [principal request]

  (= (:user/id principal)
     (:request/helper-id request)))

(defn apply-mine-first
  [requests principal {:keys [mine-first?]}]

  (if-not mine-first?
    requests

    (sort-by
      (fn [request]
        [(if
           (mine? principal request)
           0
           1)])
      requests)))
```

Unclaimed-first similarly uses authoritative claim state:

```
(defn unclaimed?
  [request]
  (nil?
    (:request/helper-id request)))

(defn apply-unclaimed-first
  [requests {:keys [unclaimed-first?]}]

  (if-not unclaimed-first?
    requests

    (sort-by
      (fn [request]
        [(if
           (unclaimed? request)
           0
           1)])
      requests)))
```

One deterministic pipeline composes them:

```
(defn present-board
  [{:keys [principal adopted options]}]

  (→ (vals (:members adopted))

       (apply-terminal-visibility
         options)

       (apply-search
         options)

       (apply-time-order
         options)

       (apply-mine-first
         principal
         options)

       (apply-unclaimed-first
         options)

       vec))
```

The exact sort priority is HumanHelp product/view semantics and can be changed without becoming a generic Gesso rule. The important property is that one declared deterministic pipeline owns it.

Typing in search updates only presentation state:

```
(defn on-search-input
  [board text]

  (assoc-in board
    [:options :search]
    text))
```

No authoritative Request is created, deleted, claimed, or acquired by that action.

Opening details likewise records local continuity:

```
(defn toggle-details
  [board request-id]

  (update-in board
    [:options :open-details]
    (fn [open]
      (let [open (or open #{})]
        (if
          (contains? open request-id)

          (disj open request-id)

          (conj open request-id))))))
```

When an authoritative same-identity board replacement occurs, local options may be retained:

```
(defn replace-authoritative-board
  [board admitted-observation]

  (acquisition/require-installed-for-current-board!
    board admitted-observation)
  (let [observation
        (acquisition/value admitted-observation)
        adopted-ids
        (set
          (map :request/id
               (:requests observation)))

        open-details
        (set/intersection
          adopted-ids
          (or (get-in board
                [:options :open-details]) #{}))]

    (→ board
        (assoc
          :adopted
          {:basis (:basis observation)
           :members
           (into {}
             (map (juxt :request/id identity))
             (:requests observation))})

        (assoc-in
          [:options :open-details]
          open-details))))
```

If R3 disappears from the newly authorized complete projection, its open-details bit is retired. The continuity layer does not recreate R3 merely because the old accordion was open.

A rendered board can now be a simple result of adopted authority plus local options:

```
(defn board-view
  [board]

  [:section.request-board

   (board-controls
     (:options board))

   (refresh-control
     (:advisory board))

   [:div.request-board__cards
    (for [request
          (present-board board)]

      ^{:key
        (:request/id request)}

      [request-card
       {:request request
        :open?
        (contains?
          (get-in board
            [:options :open-details])
          (:request/id request))}])]])
```

This is deliberately a presentation pipeline, not a second Request policy engine. It operates over production-backed facts and current adopted membership, which is exactly why it is useful for the proving application: Refresh, local creation, terminal transitions, and continuity all pressure the same local presentation logic without moving domain authority into it.

Request details and response flow

The proving application includes the ability to view Request details and the intended response interaction.

The source plans require these behaviors as browser acceptance surface but do not fully specify all product semantics.

This unified design therefore fixes only the architecture that the source plans support.

Request details

A Request card/details/accordion interaction exposes current adopted Request information.

It may preserve open/selected local state through continuity where practical.

Opening details is not itself authoritative mutation unless the production application explicitly defines such an operation.

Continuity

When an authoritative fragment refresh replaces Request-card DOM, local interaction state such as:

- selected/open accordion;
- focus;
- text selection where relevant;

may be restored through Parts V and VI continuity machinery.

Continuity failure must not alter authoritative Request truth.

Response flow

The real-browser acceptance suite includes the Request response flow.

The exact business semantics and authoritative operations of that flow remain owned by production HumanHelp model/choreography code.

The example UI must route the interaction through those production semantics rather than inventing a demo-only response store.

Where the response flow includes semantic mutation, the same application-closure discipline applies:

```
affordance
    -> semantic operation
    -> trusted server
    -> production model
    -> authoritative result
    -> acquisition/render
```

Independent response-flow acceptance contract

Status: unresolved product semantics. The architecture requires the response capability, but the source does not define a sufficient independent acceptance oracle. Naming a handler or making its current output the expected output does not close this gap. HumanHelp must record a selected contract before claiming the full rich-board milestone.

The contract must state:

- the user intent, eligible actor, and Request preconditions;
- the actual affordance and required input, including validation;
- the successful user-visible result and who may observe it;
- the production model operation and persisted result, if mutation is intended;
- the expected reload/reacquisition behavior and disclosure scope;
- rejection, concurrent-change, duplicate-submission, and uncertain-outcome behavior where applicable;
- the relation to lifecycle actions and existing details presentation.

This document does not invent a response entity, messaging system, permission policy, or storage model. If the selected interaction is non-mutating, its observable result and authority boundary must still be specified independently. If it mutates authority, normal semantic binding, production model truth, settlement, and acquisition rules apply.

Acceptance includes at least one eligible user completing the actual interaction and observing the specified result, plus the applicable invalid/concurrent case. A missing or permanently unavailable response control fails this capability regardless of whether application preflight passes. Until the contract is chosen and tested, report response-flow completion as unresolved.

Authentication and principal binding

HumanHelp browser acceptance begins with real authentication/login sufficient to establish distinct principals.

The generated browser is adversarial from the server's perspective.

Browser identity is not trusted principal identity

The browser may display a user name or include user IDs in protocol data.

The server derives the authenticated HumanHelp principal from trusted authentication/session context.

The trusted server projection then binds that principal to the choreography role.

Multiple personas

Cross-user acceptance uses independent browser contexts/personas.

Each context has its own authenticated principal.

Tests must not simulate ”another user” merely by mutating a client-side ID while sharing one trusted server session.

Authentication is an explicit assumption boundary

Gesso may assume the trusted HumanHelp authentication layer correctly establishes the authenticated principal, subject to runtime tests and the named trusted assumption.

Choreo then reasons from that trusted binding.

It does not prove the implementation of HumanHelp authentication merely because the role binding is valid.

HumanHelp Live and publication semantics

Production Request mutations that affect board visibility/publication produce the declared Live invalidation path.

The commit boundary remains primary.

Commit precedes invalidation

Conceptually:

```
production Request transaction commits
    |
authoritative progression P
    |
publication/invalidation
    |
interested client advisory
```

If invalidation delivery fails after commit, authoritative Request state remains committed.

Publication is not a second model mutation

Live publication announces possible newer authority.

It does not persist a duplicate copy of Request truth into a Live-specific store that can disagree with XTDB.

Publication correlation for initiator

The initiating browser may receive direct correlated settlement independent of the general advisory stream.

The architecture does not require the creator to learn its own committed result only through broadcast invalidation.

Other clients learn through authority

Another browser ultimately knows the new Request or lifecycle fact because it performs an authorized authoritative read that contains it.

It does not need a durable participant-to-participant message from the creator.

HumanHelp optimistic lifecycle semantics

Optimism is enabled only where production Request Choreo declares a provisional trajectory.

The example application does not infer optimism from:

- button class;
- route method;
- response latency;
- presence of a target DOM node.

Binding requirements

An optimistic Request lifecycle operation requires enough information to form the declared semantic provisional state, including where required:

- semantic operation identity;
- Request identity;
- authenticated-role binding on trusted side;
- observed authoritative basis;
- semantic command identity;
- browser target identity;
- projected plan compatibility.

Missing required information makes that realization unavailable.

No anonymous fallback

The following is always invalid for an operation whose only declared realization is optimistic Choreo:

```
optimistic binding cannot be constructed
    |
omit semantic metadata
    |
emit ordinary hx-post to semantic lifecycle route
```

The page must instead:

- obtain required basis;
- render explicit unavailable state;
- use another explicitly declared semantic realization;
- or fail checked rendering.

Pending state

Pending presentation belongs to the active command/execution.

It terminates when:

- settlement resolves;
- authoritative supersession resolves;
- execution retires;
- recovery path takes ownership.

A stale provisional pending label cannot survive after justified authoritative supersession has retired its ownership. A distinct unresolved-command status may remain where current-state evidence does not establish command attribution; it must expose the declared recovery path rather than continue an owned provisional spinner for an already retired execution.

Rollback

Rollback remains conditional on current semantic ownership.

If newer authoritative state has already installed, an old failure callback may not restore the pre-command Request card.

This is tested in the HumanHelp browser matrix because it is an application-level manifestation of Part VI's prohibition on obsolete restoration after justified authoritative installation. The rule revokes the superseded provisional owner's restoration rights; it does not require all Request observations or domain transitions to form one monotonically increasing history.

HumanHelp route and protocol safety

The proving application must reject malformed optimistic lifecycle requests at the trusted boundary.

Protocol-required routes

A route whose semantic operation requires the Gesso optimistic command protocol rejects a request that lacks or malforms the required command.

The server does not silently reinterpret it as an ordinary model call.

Render/server consistency

Because the UI may not silently downgrade, an enabled lifecycle control rendered by the application must be capable of producing the required protocol through the real browser runtime.

This is a join invariant.

A unit test proving route rejection is incomplete without a higher-level test showing valid rendered UI actually supplies the protocol.

Operation/route mismatch

If the route physically denotes Claim while the command declares Cancel, trusted server integration rejects the mismatch.

Likewise Request subject identity mismatches are rejected where the contract requires equality.

Unsupported anonymous semantic-route requests

An ordinary HTMX request may not occupy a semantic lifecycle route while omitting semantic identity.

This is rejected by rendered-surface preflight where produced by trusted views and by trusted route/protocol validation at execution as applicable.

HumanHelp Biff 2 runtime integration

The HumanHelp proving application runs on the final Biff 2 lifecycle/value-flow architecture specified in Parts IV and VII. Concrete APIs and lifecycle behavior must correspond to the selected supported final Biff 2 release; schematic examples here do not establish compatibility with an unspecified dependency.

Native lifecycle modules

HumanHelp startup uses native lifecycle-bearing modules for:

- Biff configuration;
- XTDB;
- Gesso application preflight;
- Gesso Live;
- Aleph/application serving.

Preview-era component conventions are not part of the end-state design.

Preflight precedes Aleph service

The actual HumanHelp serving path cannot begin accepting requests until:

- canonical assembly exists;
- required browser artifact is current;
- static preflight succeeds;
- checked application handler is installed.

This is a real runtime property.

It is not only a unit-test construction.

FX explicit value flow

HumanHelp production model machines use final Biff explicit state flow:

```
ambient ctx
explicit invocation input
explicit previous-state output
```

The example application does not rely on preview-era merged context to make production models work.

Explicit execution capabilities

Where production operations need:

- current time;
- seed/randomness;
- transaction;
- authenticated principal;
- authoritative basis;

the trusted HumanHelp server environment supplies them explicitly.

Application assembly checks structural supply.

HumanHelp test architecture

The reference application uses a ladder of tests with honest scope.

No single layer is allowed to stand in for another.

Model tests

Production model tests establish:

- pure domain rules;
- operation contracts;
- transaction behavior;
- model-defined outcomes;
- authority/business policy for the exercised operation contracts.

They do not prove browser operation binding.

Choreo tests

Request Choreo tests establish:

- valid global/local structure;
- knowledge/provenance rules;
- provisional trajectory;
- settlement continuations;
- projection invariants.

They do not prove the HumanHelp route graph or generated browser artifact is current.

Application preflight tests

Preflight tests consume the production HumanHelp assembly.

They establish structural closure:

- operation exists;
- browser plan exists;
- artifact current;
- route exists;
- server realization exists;
- capabilities supplied;
- Live acquisition exists;
- no semantic downgrade.

They do not prove current runtime authorization for arbitrary database state.

Rendered-surface tests

Concrete Hiccup/render tests establish that real dynamically rendered controls remain coherent with the assembly.

They specifically challenge:

- missing basis;
- conditional lifecycle buttons;
- route-template collision;
- forms/nested HTMX action ownership;
- unavailable realizations.

Lower-level server integration tests

A server integration test may deliberately manufacture a valid protocol command.

Such a test proves:

```
valid protocol command
    -> trusted HumanHelp server/model path
```

It does not prove:

```
real rendered button
    -> browser runtime
    -> valid protocol command
```

Test names/docstrings should preserve this distinction.

Real Chromium/Playwright tests

Browser acceptance uses:

- actual HumanHelp server;
- actual generated Gesso runtime;
- actual supported HTMX;
- actual rendered controls;
- actual HTTP requests;
- actual production model operations;
- actual DOM swaps/settlement;
- multiple independent browser contexts where required.

This is mandatory evidence for the proving application.

First real-browser lifecycle scenario: Claim

Claim is the first deep end-to-end scenario.

The browser test performs conceptually:

```
start real HumanHelp with local/current Gesso
    |
login real test principal
    |
establish claimable production Request
    |
GET /app
    |
find actual Claim control
    |
click actual control
    |
real browser runtime constructs command
    |
real HTMX/Gesso request
    |
trusted route
    |
production Request claim commits
    |
settlement / authoritative acquisition
    |
DOM reaches authoritative claimed state
    |
reload
    |
authoritative claimed state remains
```

The test never injects the browser command

Top-level acceptance does not manually create:

`__gesso_live_optimistic_command`

or any equivalent protocol envelope.

That value must arise from the real generated browser/runtime path.

Otherwise the test skips the core rendered-affordance-to-command edge.

Failure conditions

The Claim acceptance fails on:

- unexpected console error;
- unexpected page error;
- unexpected HTTP 4xx/5xx;
- unexpected failed browser request;
- missing protocol command;
- malformed protocol command;
- stale provisional DOM left after resolution;
- missing authoritative transition;
- visible claimed state without committed authoritative claim;
- reload showing that the apparent mutation was never persisted.

Reload is authoritative confirmation

The test reloads/reacquires after mutation.

This guards against a false success where only provisional DOM changed.

A successful user-visible optimistic interaction is not accepted until persisted authoritative state is observable after fresh acquisition.

Lifecycle browser matrix

After Claim, the same end-to-end pattern applies to every required lifecycle operation, independently of which controls the current implementation exposes.

At minimum:

- Unclaim;
- Mark on the way;
- Complete;
- Cancel;
- Reassign, including legitimate target-helper input.

For each operation the test checks:

- actual rendered operation identity;
- actual browser command;
- trusted route correspondence;
- authoritative production model mutation/outcome;
- final DOM;
- fresh authoritative reacquisition.

Where the operation is intentionally unavailable in a state, tests verify unavailability rather than expecting an anonymous fallback. These cases are additional to the eligible positive case required for every lifecycle capability.

Rich-board browser acceptance matrix

The rich board is the broad whole-application acceptance surface.

This matrix is intentionally independent of mere assembly closure. Removing a required user capability must make behavioral acceptance fail even if the remaining ApplicationAssembly is internally coherent.

Required behaviors include:

1. login establishes the intended principal;
2. Create Request works through production model semantics;
3. creator sees the new Request immediately;
4. another already-connected client does not automatically adopt the new Request;
5. the remote client receives advisory evidence that newer authority exists;
6. a toast/update appears near Refresh;
7. Refresh changes appearance/glows/pulses while advisory state is pending;
8. explicit Refresh performs authoritative reread/adoption;
9. after sufficient Refresh coverage the remote client sees the new Request;
10. a newer correlated local result does not implicitly admit unrelated remote Requests awaiting Refresh;
11. a newer correlated local result does not clear unrelated advisory debt;
12. an advisory arriving during Refresh remains outstanding when the returned acquisition does not cover it;
13. an older Refresh response cannot restore obsolete authoritative presentation over a newer justified installation;
14. failed Refresh preserves a usable recovery path and does not erase unresolved advisory information;
15. live search filters while typing without expanding adopted membership;
16. newest/oldest ordering works;
17. mine-first works;
18. unclaimed-first works;
19. show/hide terminal works;
20. Request details/accordion works;
21. selected/open details continuity survives supported same-identity replacement, while removal or lost access follows the declared retirement behavior;
22. terminal rendering is correct;
23. lifecycle actions work through the generic bound-operation path;
24. response flow works through production semantics;
25. reconnect preserves the advisory-versus-adopted distinction where applicable;
26. stale optimistic work cannot overwrite newer authoritative state;
27. malformed optimistic requests are rejected rather than downgraded;
28. the selected remote-lifecycle adoption policy is exercised for already-visible Requests once that product decision is resolved.

At least two independent browser contexts

The remote advisory scenario requires at least two browser contexts/personas.

One browser cannot prove:

```
creator immediate incorporation
    versus
remote advisory without adoption
```

because those are intentionally different actor behaviors.

A real two-context Playwright scenario should therefore look more like an application test than a protocol-unit test.

The following JavaScript is illustrative test code. The important points are that it drives the real controls, uses independent authenticated contexts, and cross-checks browser-visible state against production model truth.

```
test("creator-local adoption and remote advisory require explicit Refresh",
  async ({ browser }) => {

    const alice = await browser.newContext();
    const bob = await browser.newContext();

    const alicePage = await alice.newPage();
    const bobPage = await bob.newPage();
```

```
    const aliceErrors = [];
    const bobErrors = [];

    for (const [page, errors] of [
      [alicePage, aliceErrors],
      [bobPage, bobErrors],
    ]) {
      page.on("console", (msg) => {
        if (msg.type() === "error") {
          errors.push({
            kind: "console",
            text: msg.text(),
          });
        }
      });

      page.on("pageerror", (error) => {
        errors.push({
          kind: "pageerror",
          text: String(error),
        });
      });

      page.on("requestfailed", (request) => {
        errors.push({
          kind: "requestfailed",
          url: request.url(),
          failure: request.failure(),
        });
      });
    }

    await loginAs(alicePage, fixtures.alice);
    await loginAs(bobPage, fixtures.bob);

    await Promise.all([
      alicePage.goto(appUrl("/app")),
      bobPage.goto(appUrl("/app")),
    ]);

    await expect(
      alicePage.getByTestId("request-board")
    ).toBeVisible();

    await expect(
      bobPage.getByTestId("request-board")
    ).toBeVisible();

    const bobInitialIds =
      await visibleRequestIds(bobPage);

    const aliceTitle =
      uniqueTestTitle("Need help moving a cabinet");

    const bobTitle =
      uniqueTestTitle("Help carry boxes");

    await alicePage
      .getByRole("button", { name: "Create Request" })
```

```
    .click();

  await alicePage
    .getByLabel("Title")
    .fill(aliceTitle);

  await alicePage
    .getByLabel("Description")
    .fill("Cabinet is upstairs.");

  await alicePage
    .getByRole("button", { name: "Create" })
    .click();

  const aliceCreatedCard =
    alicePage
      .getByTestId("request-card")
      .filter({ hasText: aliceTitle });

  await expect(
    aliceCreatedCard
  ).toBeVisible();

  const createdId =
    await requestIdOf(aliceCreatedCard);

  expect(bobInitialIds).not.toContain(createdId);

  const created =
    await readRequestFromModel(createdId);

  expect(created).toMatchObject({
    status: "open",
    customerId: fixtures.alice.userId,
  });

  const bobRemoteCard =
    requestCardById(bobPage, createdId);

  await expect(
    bobRemoteCard
  ).toHaveCount(0);

  await expect(
    bobPage.getByTestId("refresh-advisory")
  ).toContainText(/update/i);

  await expect(
    bobPage.getByRole("button", { name: /Refresh/ })
  ).toHaveClass(/refresh-button--attention/);

  await bobPage
    .getByLabel("Search Requests")
    .fill("cabinet");

  await expect(
    bobRemoteCard
  ).toHaveCount(0);
```

```
    await bobPage
      .getByRole("button", { name: "Create Request" })
      .click();

    await bobPage
      .getByLabel("Title")
      .fill(bobTitle);

    await bobPage
      .getByLabel("Description")
      .fill("Three boxes in garage.");

    await bobPage
      .getByRole("button", { name: "Create" })
      .click();

    await bobPage
      .getByLabel("Search Requests")
      .fill("");

    const bobCreatedCard =
      bobPage
        .getByTestId("request-card")
        .filter({ hasText: bobTitle });

    await expect(
      bobCreatedCard
    ).toBeVisible();

    const bobCreatedId =
      await requestIdOf(bobCreatedCard);

    expect(
      (await readRequestFromModel(bobCreatedId)).customerId
    ).toBe(fixtures.bob.userId);

    await expect(
      bobRemoteCard
    ).toHaveCount(0);

    await expect(
      bobPage.getByRole("button", { name: /Refresh/ })
    ).toHaveClass(/refresh-button--attention/);

    await bobPage
      .getByRole("button", { name: /Refresh/ })
      .click();

    await expect(
      bobRemoteCard
    ).toBeVisible();

    await expect(
      requestCardById(bobPage, bobCreatedId)
    ).toBeVisible();

    await expect(
      bobPage.getByRole("button", { name: /^Refresh$/ })
    ).not.toHaveClass(/refresh-button--attention/);
```

```
    const bobBoard =
      await readAuthorizedBoardFromModel(
        fixtures.bob.userId
      );

    expect(
      bobBoard.map((r) => r.id)
    ).toContain(created.id);

    expect(aliceErrors).toEqual([]);
    expect(bobErrors).toEqual([]);

    await alice.close();
    await bob.close();
  });
```

That test already covers more than ”SSE fired.” It distinguishes Alice's correlated creation result, Bob's advisory knowledge, Bob's unchanged adopted membership, and Bob's later explicit authoritative reacquisition.

The Claim concurrency scenario can use the same two contexts:

```
test("two helpers racing Claim resolve through production Request authority",
  async ({ browser }) => {

    const alice = await authenticatedHelperContext(
      browser,
      fixtures.helperAlice
    );

    const bob = await authenticatedHelperContext(
      browser,
      fixtures.helperBob
    );

    const alicePage = await alice.newPage();
    const bobPage = await bob.newPage();

    await alicePage.goto(appUrl("/app"));
    await bobPage.goto(appUrl("/app"));

    const request =
      await createClaimableRequestFixture();

    await Promise.all([
      refreshUntilVisible(
        alicePage,
        request.id
      ),

      refreshUntilVisible(
        bobPage,
        request.id
      ),
    ]);

    const aliceCard =
      requestCardById(
        alicePage,
        request.id
      );
```

```
  const bobCard =
    requestCardById(
      bobPage,
      request.id
    );

  await expect(
    aliceCard.getByRole("button", { name: "Claim" })
  ).toBeVisible();

  await expect(
    bobCard.getByRole("button", { name: "Claim" })
  ).toBeVisible();

  // The fixture holds both outgoing Claim requests before server dispatch;
  // an uncontrolled pair of clicks need not produce a real race.
  const race = await holdClaimRequestsAtServerBoundary(
    [alicePage, bobPage], request.id
  );

  const clicks = Promise.all([
    aliceCard
      .getByRole("button", { name: "Claim" })
      .click(),

    bobCard
      .getByRole("button", { name: "Claim" })
      .click(),
  ]);

  await race.waitForBothRequests();
  await race.releaseTogether();
  await clicks;

  const [aliceSettlement, bobSettlement] = await Promise.all([
    expectClaimExecutionSettled(
      alicePage,
      request.id
    ),

    expectClaimExecutionSettled(
      bobPage,
      request.id
    ),
  ]);

  // Settlement helpers return outcomes for these command identities,
  // rather than reporting that a button stopped showing a spinner.
  expect(
    [aliceSettlement.status, bobSettlement.status].sort()
  ).toEqual(["accepted", "rejected"]);

  const authoritative =
    await readRequestFromModel(
      request.id
    );

  expect(
```

```
    authoritative.status
  ).toBe("claimed");

  expect([
    fixtures.helperAlice.userId,
    fixtures.helperBob.userId,
  ]).toContain(
    authoritative.helperId
  );

  expect(
    authoritative.helperId
  ).not.toBeNull();

  expect(
    authoritative.helperId === fixtures.helperAlice.userId
      ? aliceSettlement.status
      : bobSettlement.status
  ).toBe("accepted");

  // The remote-lifecycle adoption policy remains an explicit product
  // decision. This helper follows the selected fixture policy rather than
  // choosing auto-adoption inside this example.
  await reconcileVisibleRequest(
    alicePage,
    request.id
  );

  await reconcileVisibleRequest(
    bobPage,
    request.id
  );

  await expectRequestOwner(
    alicePage,
    request.id,
    authoritative.helperId
  );

  await expectRequestOwner(
    bobPage,
    request.id,
    authoritative.helperId
  );

  await alicePage.reload();
  await bobPage.reload();

  await expectRequestOwner(
    alicePage,
    request.id,
    authoritative.helperId
  );

  await expectRequestOwner(
    bobPage,
    request.id,
    authoritative.helperId
  );
```

```
    await alice.close();
    await bob.close();
  });
```

This example deliberately refuses to choose the still-open HumanHelp product decision about automatic adoption of remote lifecycle changes. The helper `reconcileVisibleRequest` must be implemented according to the selected policy once that decision is made.

The important acceptance property is already fixed: two optimistic browsers do not create two authoritative histories, and the losing browser does not turn its provisional Claim into domain truth.

Creator scenario

```
A creates R
    |
commit
    |
A immediately adopts authoritative R
```

The test verifies both:

- DOM;
- authoritative state.

Remote scenario

```
B already viewing
    |
invalidation arrives
    |
B still does not show R
    |
advisory UI appears
```

This is a success condition.

A browser test that immediately shows R before Refresh is red because it violated the intended product semantics.

Refresh scenario

```
B clicks Refresh
    |
authoritative board reread with captured scope/requirement
    |
covered membership/observations are installed
    |
R appears when the acquisition justifies admitting R
    |
only covered advisory obligations clear/recompute
```

The test validates that the content came from the authoritative acquisition path, not from cached invalidation payload.

The browser suite must also cover the interaction cases that make Refresh meaningful rather than ceremonial:

- an advisory arrives after Refresh has captured its requirement and remains pending if the response does not cover it;
- B has remote R awaiting Refresh, then creates local S and installs S's correlated authoritative result; S appears, R remains absent, and R's advisory remains outstanding;
- a newer authoritative local installation occurs before an older Refresh response returns, and the older response cannot restore obsolete state;
- Refresh fails without erasing adopted content classification or advisory debt.

Search scenario

The test types search text and observes live filtering.

It verifies no full page reload or unintended authoritative mutation is required unless the chosen implementation explicitly uses a server-side read for search.

The semantic requirement is live correct filtering of adopted board content.

Sorting scenario

The test exercises:

- newest/oldest;
- mine-first;
- unclaimed-first.

It verifies actual order, not merely control state.

Visibility scenario

The test toggles terminal Request visibility and verifies:

- correct Requests appear/disappear;
- authoritative Request state is unchanged;
- later Refresh preserves the option semantics.

Details/continuity scenario

The test opens a Request details accordion, causes a supported same-identity fragment replacement, and verifies the declared open/selected continuity. It also checks the declared retirement behavior when that Request is no longer rendered. Additional focus/selection restoration is tested where supported; these optional extensions do not replace the required details scenario.

Continuity is accepted only as local interaction preservation.

The Request's authoritative state remains independently verifiable.

A single browser scenario can exercise these local presentation features against stable adopted authority without conflating them with mutation.

```
test("search, sorting, visibility, and details remain local board presentation",
  async ({ browser }) => {

    const context =
      await authenticatedHelperContext(
        browser,
        fixtures.helperBob
      );

    const page =
      await context.newPage();

    const fixture =
      await seedBoard({
        requests: [
          {
            title: "Move a cabinet",
            createdAt: "2026-09-21T18:00:00Z",
```

```
        status: "open",
        helperId: null,
      },
      {
        title: "Install shelves",
        createdAt: "2026-09-21T18:05:00Z",
        status: "claimed",
        helperId: fixtures.helperBob.userId,
      },
      {
        title: "Carry boxes",
        createdAt: "2026-09-21T17:30:00Z",
        status: "completed",
        helperId: fixtures.helperBob.userId,
      },
    ],
  });

await page.goto(appUrl("/app"));

await refreshUntilBoardContains(
  page,
  [fixture.byTitle["Move a cabinet"].id,
   fixture.byTitle["Install shelves"].id]
);

const initialModel =
  await readAuthorizedBoardFromModel(
    fixtures.helperBob.userId
  );

await page
  .getByLabel("Search Requests")
  .fill("cabinet");

await expect(
  requestCardByTitle(
    page,
    "Move a cabinet"
  )
).toBeVisible();

await expect(
  requestCardByTitle(
    page,
    "Install shelves"
  )
).toHaveCount(0);

expect(
  await readAuthorizedBoardFromModel(
    fixtures.helperBob.userId
  )
).toEqual(initialModel);

await page
  .getByLabel("Search Requests")
  .fill("");
```

```
    await page
      .getByLabel("Sort order")
      .selectOption("oldest");

    expect(
      await visibleRequestTitles(page)
    ).toEqual([
      "Move a cabinet",
      "Install shelves",
    ]);

    await page
      .getByLabel("Sort order")
      .selectOption("newest");

    expect(
      await visibleRequestTitles(page)
    ).toEqual([
      "Install shelves",
      "Move a cabinet",
    ]);

    await page
      .getByLabel("Mine first")
      .check();

    expect(
      (await visibleRequestTitles(page))[0]
    ).toBe("Install shelves");

    await page
      .getByLabel("Unclaimed first")
      .check();

    expect(
      (await visibleRequestTitles(page))[0]
    ).toBe("Move a cabinet");

    await expect(
      requestCardByTitle(
        page,
        "Carry boxes"
      )
    ).toHaveCount(0);

    await page
      .getByLabel("Show completed Requests")
      .check();

    await expect(
      requestCardByTitle(
        page,
        "Carry boxes"
      )
    ).toBeVisible();

    const afterVisibilityToggle =
      await readRequestFromModel(
        fixture.byTitle["Carry boxes"].id
```

```
    );

  expect(
    afterVisibilityToggle.status
  ).toBe("completed");

  const cabinetCard =
    requestCardByTitle(
      page,
      "Move a cabinet"
    );

  await cabinetCard
    .getByRole("button", { name: /Details/ })
    .click();

  await expect(
    cabinetCard.getByTestId("request-details")
  ).toBeVisible();

  await page
    .getByRole("button", { name: /^Refresh/ })
    .click();

  await expect(
    requestCardByTitle(
      page,
      "Move a cabinet"
    )
    .getByTestId("request-details")
  ).toBeVisible();

  await expect(
    page.getByLabel("Show completed Requests")
  ).toBeChecked();

  await expect(
    page.getByLabel("Unclaimed first")
  ).toBeChecked();

  await context.close();
});
```

The scenario intentionally checks model truth before and after presentation-only interactions. That prevents an implementation from making a test green by silently rewriting authoritative Request data to implement filtering or sort order.

A separate retirement case should remove an open Request from the authorized projection and assert that continuity does **not** resurrect its details DOM. That case exercises Part VI's scope and retirement rules rather than merely proving that accordion state survives benign replacement.

Terminal transition scenario

An operation that makes a Request terminal must yield correct authoritative terminal state.

The board then follows its current terminal visibility option:

- show terminal -> render correct terminal representation;
- hide terminal -> local presentation excludes the terminal card.

Hiding is not an authoritative observation of Request absence and does not by itself remove adopted membership. If an authoritative acquisition separately establishes absence from a scoped complete projection, apply Part VI absence rules. Denial, an empty error response, and a local visibility option do not establish that fact.

Reconnect scenario

The test interrupts and re-establishes the supported Live connection through a controlled fixture. It asserts the recovery behavior and relevant availability assumptions of that supported path.

It verifies:

- no illegal stale execution resumes;
- advisory state can recover;
- reconnect does not force remote content adoption when explicit Refresh is the product rule;
- authoritative refresh still provides convergence.

Stale optimistic scenario

The test deliberately arranges or simulates an ordering where:

```
old optimistic execution
newer authoritative install
late old failure/callback
```

and verifies the late old callback cannot restore obsolete DOM.

This exercises the generic Part VI invariant in the real HumanHelp surface.

Structural rejection acceptance matrix

HumanHelp also supplies deterministic malformed fixtures that must fail before normal user interaction.

Those fixtures are most useful when they are mutations of the same production assembly constructor rather than a parallel ”bad app” representation.

A small helper can produce controlled defects while retaining the rest of the real application:

```
(defn production-candidate
  []
  (fixture/with-production-artifact-inventory
    (gesso.application/normalize
      (application-preflight/application
        test-config))))

(defn with-controlled-defect
  [candidate defect]
  (case defect

    :unknown-operation
    (assoc-in candidate
      [:views
       :request/card
       :semantic-affordances
       :claim
       :operation]
      :request/claime)

    :missing-browser-plan
    (update-in candidate
      [:browser :artifact :contained-plans]
      disj
      :plan/request-claim)

    :wrong-server-operation
    (assoc-in candidate
      [:routes :route/request-claim
       :trusted-capability]
      :server/request-cancel)

    :missing-clock
    (update-in candidate
      [:server-operations
       :server/request-claim
       :supplied-capabilities]
      disj
      :clock)

    :missing-board-acquisition
    (update candidate
      :acquisitions
      dissoc
      :request/board)

    :missing-claim-reread
    (update candidate
      :acquisitions
      dissoc
      :request/card)

    candidate))
```

For the structural-only plan defect, the normalized fixture includes the artifact inventory populated from the production browser build. Changing its inventory is a checker fixture; the separate currentness tests below check the real served artifact and receipt. All source declarations still come from the one production constructor.

Static defects then pass through the real preflight checker:

```
(deftest missing-claim-plan-fails-production-preflight
  (let [candidate
        (with-controlled-defect
          (production-candidate)
          :missing-browser-plan)

        result
        (gesso.application/check-operation
          candidate :request/claim)]

    (is (= :rejected
           (:status result)))

    (is (= :missing-browser-plan
           (:error/kind result)))

    (is (= :request/claim
           (:operation result)))

    (is (= :plan/request-claim
           (:required result)))))
```

The missing capability case likewise uses the actual server declaration:

```
(deftest missing-clock-fails-before-server-start
  (let [candidate
        (with-controlled-defect
          (production-candidate)
          :missing-clock)]

    (is (= :missing-server-capability
           (:error/kind
             (gesso.application/check-operation
               candidate :request/claim))))

    ;; The fixture starts the real build/preflight/lifecycle with this
    ;; candidate injected at its declared normalized-assembly boundary.
    (is (thrown?
          clojure.lang.ExceptionInfo
          (fixture/start-from-normalized-assembly!
            candidate)))

    (is (false?
          (test-runtime/server-started?)))))
```

The stale artifact defect uses the actual artifact-currentness boundary rather than merely putting a fake boolean in the assembly:

```
(deftest stale-artifact-bytes-fail-currentness
  (let [{:keys [manifest plans]}
        (artifact-fixture/current-projection
          (production-candidate))

        {:keys [receipt artifact-path]}
        (artifact-fixture/previous-build)

        result
        (browser/check-browser-artifact-currentness
          {:manifest manifest
           :plans plans
           :artifact-path artifact-path
           :receipt receipt})]

    (is (= :rejected
           (:status result)))

    (is (= :stale-browser-artifact
           (:error/kind result)))))
```

A stronger adversarial fixture copies the **current** manifest identity into stale metadata while leaving incompatible bytes:

```
(deftest forged-current-stamp-does-not-certify-stale-bytes
  (let [{:keys [manifest plans]}
        (artifact-fixture/current-projection
          (production-candidate))

        {:keys [receipt artifact-path]}
        (artifact-fixture/previous-build)

        forged
        (assoc receipt
          :manifest/id
          (:manifest/id manifest)
          :plan-digests
          (artifact/plan-digests plans))

        result
        (browser/check-browser-artifact-currentness
          {:manifest manifest
           :plans plans
           :receipt forged
           :artifact-path artifact-path})]

    (is (= :rejected
           (:status result)))

    (is (= :artifact-content-mismatch
           (:error/kind result)))))
```

Dynamic rendered-surface defects are introduced later, at the actual render boundary, because static preflight does not know the concrete arguments:

```
(deftest anonymous-claim-route-is-rejected-before-serialization
  (let [generation
        (fixture/admitted-production-generation)

        malformed
        [:form
         {:hx-post
          (str "/app/requests/"
               R
               "/claim")}

         [:button
          {:type "submit"}
          "Claim"]]]

    (is
      (thrown-with-msg?
        clojure.lang.ExceptionInfo
        #"anonymous semantic route"
        (gesso.render/serialize-checked!
          generation
          malformed)))

    (is (false?
          (fixture/output-delivered?)))

    (is (= :open
           (:request/status
             (request-model/read-current
               test-ctx
               R))))))
```

Malformed browser protocol is a different boundary. The route exists, but the untrusted client modifies or invents protocol data:

```
(deftest forged-browser-command-cannot-claim-request
  (let [response
        (http/post
          test-server
          (str "/app/requests/"
               R
               "/claim")

          {:headers
           authenticated-alice-headers

           :form-params
           {"__gesso_live_optimistic_command"
            (pr-str
              {:operation :request/claim
               :command-id C42
               :request/id R
               :principal Mallory
               :plan :forged-plan})}})]

    (is (= 400
           (:status response)))

    (is (= :open
           (:request/status
             (request-model/read-current
               test-ctx
               R))))))
```

The trusted boundary does not believe `:principal Mallory` merely because the client encoded it, and it does not accept an arbitrary plan identifier as proof of an admitted realization.

That malformed-plan test alone does not prove principal rebinding: the request could have been rejected before principal handling. A separate isolated fixture submits an otherwise valid browser command with a forged client identity:

```
(deftest valid-plan-with-forged-principal-cannot-claim-as-mallory
  (let [wire-command
        (fixture/valid-claim-wire-command
          {:request/id R
           :command/id C43})

        response
        (http/post
          test-server
          (str "/app/requests/" R "/claim")
          {:headers authenticated-alice-headers
           :form-params
           {"__gesso_live_optimistic_command"
            (pr-str
              (assoc wire-command
                :principal Mallory))}})

        current
        (request-model/read-current test-ctx R)]

    ;; Strict decoding may reject the extra field; a decoder that discards
    ;; it may proceed only with Alice's authenticated server identity.
    (is (≤ 200 (:status response) 499))
    (is (not= Mallory (:request/helper-id current)))
    (if (< (:status response) 400)
      (do
        (is (= :claimed (:request/status current)))
        (is (= Alice (:request/helper-id current))))
      (is (= :open (:request/status current))))))
```

The test fixture must make both Alice and Mallory independently eligible to Claim, so an accidental Mallory binding cannot pass by failing domain policy.

Finally, every malformed fixture should have a nearby valid control case:

```
(deftest valid-claim-generation-remains-usable
  (let [candidate
        (production-candidate)

        generation
        (fixture/admitted-production-generation)

        page
        (fixture/render-app
          generation
          {:principal Alice})]

    (is (= :accepted
           (:status
             (gesso.application/check-operation
               candidate :request/claim))))

    (is (fixture/visible-claim-control?
          page R))

    (is (= :preflight-closed
           (:status
             (gesso.application/report
               generation))))))
```

Rejecting everything would satisfy many negative tests while proving nothing about a usable application. The paired valid fixture ensures the enforcement boundary is selective rather than accidentally disabling the proving surface.

The same controlled-defect style can cover every row in the structural rejection matrix while preserving one crucial property: tests break the actual production construction at one declared edge and then observe which real enforcement boundary catches it.

Unknown semantic operation

A view references an operation not present in production Request Choreo.

Expected:

- construction/preflight rejection;
- view/component source identified;
- unknown operation identified;
- known candidates supplied where cheap;
- no fuzzy execution.

Missing browser plan

A valid exposed Request operation is omitted from the required browser plan set.

Expected:

- preflight rejection;
- semantic operation remains recognized;
- browser realization edge named missing;
- no ordinary HTMX fallback.

Stale browser artifact

The application assembly changes browser plan identity without rebuilding the runtime artifact.

Expected:

- startup failure before normal service;
- expected manifest and admitted artifact identities reported;
- copied stamps on incompatible artifact bytes are also rejected;
- the checked artifact corresponds to the artifact actually served;
- rebuilding restores currentness.

Missing route

The browser plan requires a semantic transport whose physical route is absent.

Expected:

- preflight rejection;
- operation named;
- required method/path/protocol named;
- available routes reported where useful.

Wrong trusted server realization

The route points at a server realization incompatible with the plan/operation.

Expected:

- preflight rejection;
- expected and actual semantic server identity/contract reported.

Missing execution capability

A production Request operation requires a capability such as trusted current time but the server boundary does not supply it.

Expected:

- startup/preflight rejection where statically knowable;
- missing capability named;
- operation/source named;
- no first-click runtime discovery.

Missing initial acquisition

A managed board fragment has:

- no authoritative initial content;
- no permitted load/reread path.

Expected:

- preflight rejection;
- fragment identity named;
- expected authoritative projection/progression named.

Anonymous lifecycle route

A rendered control emits an ordinary HTMX POST to an assembled lifecycle route while omitting semantic operation identity.

Expected:

- dynamic pre-serialization rejection;
- concrete method/path identified;
- matched semantic route template identified;
- intended operation identified;
- missing semantic identity identified.

Malformed browser protocol

An adversarial/modified browser sends malformed optimistic protocol data.

Expected:

- trusted boundary rejects it;
- no production Request transition occurs merely because route method/path was valid.

HumanHelp acceptance and production model truth

Every browser-visible success that represents authoritative mutation must be backed by production model truth.

DOM-only success is failure

A test is red if:

```
DOM says claimed
but authoritative Request remains unclaimed
```

or:

```
DOM says completed
but fresh authoritative read remains active
```

Optimistic UI may temporarily display provisional state.

Final accepted success requires authoritative confirmation.

Production model rejection is legitimate behavior

A lifecycle action may be structurally valid but rejected at runtime because current domain state changed.

For example:

- another helper claimed the Request;
- Membership no longer permits the operation;
- Organization policy changed;
- commit-time guard failed.

The example application must render/propagate the model-defined outcome.

It must not classify this as broken application assembly.

Already-incorporated/reconciled outcomes

Where the production model supports already-incorporated or reconciliation semantics, browser tests should verify the declared behavior rather than assuming every retry must be a hard error.

The model owns the policy.

Gesso owns the protocol structure needed to transport and render it safely.

Connected enforcement and recovery evidence

At least the Claim vertical slice records the admitted assembly generation, the real rendered operation, its browser plan/artifact binding, trusted route, prepared server realization, relevant capability providers, model outcome, settlement, and acquisition/installation evidence. These references come from the canonical assembly and observed execution, not a second hand-maintained operation registry. Evidence must connect the actual boundaries exercised.

Paired valid and controlled-defect fixtures establish both usability and rejection. Required defects include a missing basis in a concrete enabled render, an absent browser plan, stale or mismatched artifact content, an incompatible server binding, a missing trusted capability, and malformed browser protocol. Tests identify the first enforcing boundary and verify the absence of the prohibited output or mutation appropriate to that boundary. An earlier valid commit is not undone by a later response-render failure.

Recovery cases distinguish:

- rejection before submission;
- transaction submission followed by loss of command-specific outcome;
- known commit followed by failed settlement delivery;
- crash after commit and before Live publication;
- acquisition failure or an older response arriving after newer installation;
- retired callbacks arriving after provisional presentation has been superseded.

Publication recovery must exercise the implementation's selected mechanism under #req-post-commit-recovery. Durable publication, reconnect, and explicit Refresh make different liveness claims; merely declaring a publication function is not evidence that a commit-to-publication crash recovers. Where recovery requires a user action, the test performs that action and reports the premise.

A current-state read can resolve presentation without resolving command attribution. Recovery tests must not infer that a command committed merely because another actor could have produced the same current state. Fault tests assert expected diagnostics narrowly and still reject unrelated failures.

Removal of dead migration state

The final proving surface should not retain historical compatibility state that no longer participates in semantics.

Potential categories include:

- obsolete demo-store/reset mechanisms;
- dead visible-revision concepts;
- duplicate route registries;
- duplicate operation registries;
- comments describing removed fallback behavior;
- CSS selectors with no producer;
- hidden board fields with no consumer;
- tests whose only purpose was to bless transitional contradictions.

Removal happens after real behavior is grounded.

Deletion discipline

When a source/test file is removed, project workflow produces deletion documentation identifying:

- exact path;
- reason;
- replacement/superseding location where applicable.

This keeps cleanup explicit rather than losing architectural history through silent file disappearance.

HumanHelp completion gates

The HumanHelp proving application is complete for this architectural stage only when all required gates hold simultaneously.

These gates are an independent behavioral specification, not a second semantic registry. They state required user capabilities and authoritative outcomes without duplicating route strings, operation identities, generated-plan metadata, or model contracts already owned by the canonical assembly. A required capability removed from the product must fail the affected gate even when the remaining assembly is structurally valid. Conditional unavailability under current model policy is different from accidental omission.

Gate 1 – semantic-route invariant

An anonymous ordinary HTMX action cannot occupy an assembled Request semantic route without preflight rejection.

Gate 2 – actual runtime preflight adoption

The running HumanHelp application demonstrably uses the checked Gesso application handler/startup path.

Gate 3 – current browser artifact

HumanHelp startup refuses stale browser assembly/artifact correspondence.

Gate 4 – Claim end to end

The actual browser can click the actual Claim control and reach persisted authoritative claimed state without manufactured browser protocol data.

Gate 5 – lifecycle closure

Every required Request lifecycle capability uses the generic semantic operation path and succeeds in an eligible positive fixture, including Reassign with legitimate target selection. State-dependent unavailability and rejection are also verified. Removing a capability or leaving it permanently unavailable fails this gate even if the remaining assembly is closed.

Gate 6 – rich board

The board supports:

- creation;
- immediate creator visibility;
- remote advisory without auto-adoption;
- explicit Refresh;
- search;
- sorting;
- visibility options;
- details/continuity;
- terminal rendering;
- response flow.

The response-flow contract at #decision-humanhelp-response-flow must be selected and independently tested. Architectural wiring without that product outcome does not satisfy this gate.

Gate 7 – multi-client adoption behavior

At least two independent browser contexts demonstrate:

```
authority advanced
    ≠
automatic adoption
```

The acceptance includes selective local adoption while unrelated remote work remains pending, scope-correct advisory discharge, and the concurrent-Refresh race cases defined above.

Explicit Refresh advances only the remote projection/membership justified by the acquisition it completes.

Before this gate is fully satisfied for remote lifecycle changes to already visible Requests, HumanHelp must resolve the product decision recorded at `decision-humanhelp-remote-lifecycle-adoption` and test the selected policy.

Gate 8 – production assembly in tests

Application preflight tests consume the real production assembly constructor.

No shadow assembly can remain green while production declarations are broken.

Gate 9 – structural malformed fixtures

The important closure failures reject before ordinary browser interaction and produce stable structured diagnostics.

Gate 10 – real browser acceptance

The required Chromium/Playwright scenarios pass with:

- no unexpected console errors;
- no page errors;
- no unexpected failed requests;
- no unexpected 4xx/5xx;
- deliberately injected fault diagnostics asserted precisely, not suppressed;
- authoritative persistence confirmed;
- correct multi-client semantics.

Gate 11 – final Biff/Gesso substrate

HumanHelp uses the final Biff 2 execution/lifecycle contracts assumed by this unified design.

There is no compatibility behavior retained solely to preserve an accidental preview-era semantic assumption.

HumanHelp as evidence for generic Gesso evolution

HumanHelp failures are evidence.

They are not automatically framework features.

When a failure occurs, classify it.

Generic semantic gap

Examples:

- no representation of required server capability;
- no way to express acquisition path;
- no fail-closed operation binding;
- no exact artifact correspondence;
- no semantic-route collision check.

These justify generic Gesso work when the abstraction is reusable.

HumanHelp application assembly bug

Examples:

- route declaration missing from assembly;
- example view bypasses Request-card binding;
- Live fragment declaration points at wrong route;
- browser artifact was not rebuilt.

These are fixed in HumanHelp unless they reveal a generic guard that should prevent the class everywhere.

HumanHelp business-rule bug

Examples:

- incorrect Request lifecycle eligibility;
- wrong Organization policy;
- wrong terminal-state interpretation.

These remain HumanHelp model concerns.

Gesso should not absorb them merely because they surfaced during integration.

External/runtime assumption failure

Examples:

- HTMX lifecycle changed;
- browser behavior differs from assumed contract;
- network failure;
- XTDB/Biff integration changed.

These may require adapter/test/runtime updates without changing choreography semantics.

HumanHelp proving-application invariant

At the end of the HumanHelp reference-application layer, the unified design requires:

> The active HumanHelp proving application consumes production `net.humanhelp.site.model.*` semantics rather than duplicating them in `net.humanhelp.example.*`. One canonical HumanHelp ApplicationAssembly composes production Request Choreo, browser plans/artifact, physical routes, trusted server realization, Live acquisition, and rendered affordances. Every exposed Request lifecycle action retains unconditional semantic identity and uses the generic checked Gesso operation-binding path; missing optimistic basis or other realization data never produces an anonymous HTMX fallback. Request creation and lifecycle mutations commit through production models. The creator may adopt its own correlated authoritative result immediately, while remote new Requests remain outside another client's adopted membership until an authorized Refresh acquires sufficient scope/progression coverage. Selective incorporation of a newer local result neither admits unrelated remote Requests nor clears unrelated advisory debt. Refresh discharges only advisory obligations actually covered by its authoritative acquisition, and stale Refresh responses cannot overwrite newer justified installations. Remote lifecycle adoption for already-visible Requests follows the explicit HumanHelp product policy selected before the affected operational gate is claimed; Live invalidation itself never becomes authoritative domain content. Search, sorting, visibility options, Request details, continuity, terminal rendering, and response flow remain functional without accidentally expanding adopted membership. Real Playwright/Chromium tests use the actual generated runtime, actual controls, actual routes, and multiple authenticated contexts, never manufactured browser protocol commands, and every apparent mutation is confirmed through fresh authoritative state. Behavioral acceptance remains independent of assembly closure: deleting a promised HumanHelp capability does not redefine the application into correctness merely because the remaining assembly is internally valid. Structural contradictions are rejected by production preflight, dynamic business authorization remains in the production models, and HumanHelp drives generic Gesso improvements only when a failure exposes a genuinely generic semantic or application-closure gap.

Part IX — Compositional certification

This part defines the stronger certification layer over the semantic and application contracts of Parts I–VIII. It specifies what certified interfaces export, how assumptions and evidence compose, and how checking binds claims to the program and realization being admitted. These are end-state requirements; this specification is not evidence that the implementation already meets them. The operational, compositional, external-formal, and multi-node milestones retain their separate completion scopes.

Compositional proof and certified semantic interfaces

The stronger Gesso assurance architecture is compositional.

Its central rule is:

> A downstream semantic artifact may rely only on guarantees actually established by checked upstream semantic interfaces, plus assumptions that are explicitly named at the boundary where they enter.

This rule replaces **proof by reputation**.

A namespace, value, artifact, or cached file is not trusted merely because:

- it has a familiar name;
- it was produced by Gesso code;
- its source has not moved;
- a prior test suite passed;
- a map says `:verified? true`;
- a digest matches;
- its shape validates against Malli.

Those facts may be useful evidence about identity, construction history, or runtime shape.

They are not substitutes for the proposition the downstream stage needs.

Purpose

Parts I through VIII define:

- the semantic ontology;
- choreography and projection;
- authoritative model boundaries;
- browser execution;
- whole-application closure;
- HumanHelp as the proving application.

Part IX strengthens the way those layers depend on one another.

The target is not:

```
good-looking namespace
    |
    v
downstream code assumes what it probably means
```

The target is:

```
raw declaration
    |
normalize / elaborate
    |
check obligations
    |
certified semantic interface
    |
compose with certified dependencies
    |
derive next certified artifact
```

A mismatch stops construction at the first layer that can know the mismatch.

The malformed program does not continue toward a runnable browser merely because later layers could discover another symptom.

Relation to runtime semantics

Certification does not create a second executable meaning.

The semantic/runtime path remains:

```
domain/schema/rules
        |
        v
public authoritative operation contracts
        |
        v
global choreography
        |
        v
verified projection
        |
        v
ExecutablePlan
        |
        v
ApplicationAssembly / whole-application realization
        |
        v
runtime execution
```

The proof/certificate path runs alongside it:

```
semantic interface
        |
assumptions + guarantees
        |
obligations + derivations
        |
certificate
        |
dependency DAG
        |
root assurance report
```

Certificates participate in deciding whether construction is accepted under the selected certification profile. Each artifact class must define its required judgments and admissible assumption classes. The stronger compositional milestone requires the corresponding certificates; it must not be conflated with the earlier operational milestone's structural and runtime evidence. An unsupported proof feature cannot silently downgrade a build requesting that stronger profile into an ordinary operational build.

They do not alter runtime transitions after an executable artifact has been accepted.

A proof sidecar, derivation tree, semantic fingerprint, root report, or external formal proof is never a browser/server authorization capability.

One proof DAG, several assurance classes

The root application assurance graph may contain evidence of different kinds.

For example:

```
schema typing
    machine-checked by the identified semantic checker or proof kernel

operation implementation conformance
    trusted or separately checked

choreography knowledge obligation
    machine checked

projection preservation
    mechanically checked/proved for supported subset

browser artifact correspondence
    assembly verified

rendered affordance closure
    runtime enforced against assembly

authenticated principal
    runtime enforced by trusted authentication boundary

HTMX/browser primitive behavior
    trusted + real-browser tested
```

The fact that all of these appear in one root report does not flatten them into one assurance class.

The report must preserve exactly what kind of evidence supports each claim, using Part I's assurance classifications. Evidence attaches to individual propositions and their dependencies, not merely to the containing artifact's name. A certificate can contain a kernel-checked conditional proposition and a trusted implementation premise without claiming that the implementation itself has been proved.

Certification is declaration-first

Application developers should primarily declare:

- schemas/domain types;
- relationships;
- business-rule expressions;
- authoritative operation contracts;
- choreography;
- application realization facts.

Gesso should perform the proof bookkeeping:

- normalization;
- semantic identity;
- dependency tracking;
- type/refinement checking;
- knowledge/provenance checking;
- authority/capability checking;
- derivation construction;
- proof-kernel checking;
- certificate issuance;
- projection;
- executable-plan construction;
- application closure;
- diagnostics.

Where only the application can know a proposition, the application supplies an explicit assertion or assumption.

Gesso must not disguise such application truth as a derived theorem.

The governing dependency invariant

For every semantic layer L:

> Every guarantee exported by L has a checked justification appropriate to its assurance class and retains the dependencies and assumptions under which it holds. A proposition supplied without such a derivation is an explicit assumption or assertion, not an unconditional derived guarantee.

No stage gets credit for the intended meaning of a predecessor that the predecessor did not actually establish.

Vocabulary and assurance classes

The compositional architecture uses a small explicit vocabulary.

Raw declaration

User/application/framework data that has not yet crossed the relevant semantic checker.

Examples:

- raw Malli/domain schema;
- raw business-rule expression;
- raw operation contract;
- raw choreography;
- raw authority declaration;
- raw derivation;
- raw dependency reference.

Raw data is permitted at construction boundaries but may not masquerade as a certified semantic artifact.

Semantic category and admitted value

A representation may satisfy the shape of a semantic object without yet being admitted as that kind of semantic object.

The design therefore distinguishes at least:

```
representation shape
    ≠
semantic category
    ≠
admission through the required checker/trusted boundary
    ≠
proof of the propositions associated with that category
```

For example, an ordinary Clojure map may carry the same fields as a checked certificate, a verified choreography, an authoritative basis, or an executable plan. Shape equality, a discriminator key, or a familiar constructor name is not sufficient evidence that the stronger semantic category has been admitted.

An **admitted semantic value** is a value the supported construction path permits later stages to consume as a particular semantic category because the required construction, checking, trusted acquisition, or validation boundary has been crossed.

Admission is deliberately weaker than proof. A value may be admitted as an `:authoritative-observation` because a trusted acquisition path produced it, while claims about the correctness of the underlying model may still depend on runtime enforcement or trusted assumptions. Likewise, a nominal value named `VerifiedChoreography` does not prove choreography validity merely by possessing that runtime type; its trustworthy construction remains tied to the verifier or certificate-checking boundary.

Nominal types, private records, opaque wrappers, checked constructors, validated handles, tagged internal representations, and equivalent mechanisms are permitted ways to make semantic categories difficult to confuse accidentally. No one representation technique is required merely because two semantic values must remain distinct.

The ordinary downstream application author should not need to wrap routine business values manually simply to satisfy framework ceremony. Stronger category boundaries should preferentially be introduced behind elaboration, verification, authoritative acquisition, model operations, and other framework boundaries that can construct the admitted value themselves.

Semantic interface

The normalized meaning exported by one checked component to its dependents.

It should include only facts that downstream proofs are allowed to rely on.

Examples:

- declared value types;
- enumerated variants;
- cardinality;
- optionality/refinement rules;
- relationships;
- operation input/output facts;
- preconditions;
- postconditions;
- authority requirements;
- provenance requirements;
- effects/resources;
- named assumptions;
- named guarantees.

Implementation details not relevant to downstream reasoning should not become part of the semantic interface merely because they are convenient to serialize.

Assumption

A proposition accepted from outside the current proof.

Assumptions must be named and classified.

Useful assumption classes include:

- `:application-asserted`;
- `:trusted-model-contract`;
- `:authentication`;
- `:external-system`;
- `:runtime-primitive`;
- `:environment-liveness`.

An assumption is not a failure. An invisible assumption is.

Guarantee

A proposition established by the current certified component relative to its dependencies and assumptions.

Obligation

A proposition that must be established before a certificate may be issued.

Derivation

Machine-readable evidence describing how a guarantee follows from certified premises and inference rules.

Certificate

A closed artifact stating:

- what semantic object was checked;
- its semantic identity;
- certified dependency identities;
- assumptions;
- guarantees;
- proof obligations;
- derivations or references to checked derivations;
- applicable semantic-checker or proof-kernel version;
- result classification.

A certificate is evidence accepted by the checker, not a boolean. Serialized certificate data is a candidate until admitted by the applicable checking boundary. Its result classification cannot determine its own acceptance. A certificate may legitimately establish a conditional theorem; its admission must still reject unresolved obligations that its artifact class requires to be discharged rather than assumed.

Semantic identity

A deterministic identity of a normalized semantic interface.

This is deliberately not the same as a source-file checksum.

Whitespace, comments, local helper names, and non-semantic implementation refactors should not necessarily invalidate downstream proofs.

A change to a relied-upon type, rule contract, authority contract, operation postcondition, or other semantic commitment must.

Proof DAG

The complete dependency graph from a root verified application artifact through all certified semantic dependencies and named assumptions.

The unified design preserves explicit distinctions among mechanically established claims, runtime-enforced premises, and trusted assumptions.

Machine-checked framework properties

Properties justified mechanically from explicit semantics and derivations.

Examples:

- rule expression type safety;
- knowledge entailment;
- provenance entailment;
- authority derivation and non-escalation constraints;
- message payload closure and knowledge-transfer constraints;
- projection preservation for the supported subset;
- resource invariants;
- progression algebra invariants;
- browser-adapter transition invariants;
- semantic dependency compatibility.

Runtime-enforced premises

Properties whose concrete values must be checked when static proof meets the world.

Examples:

- Malli value shape;
- authenticated principal binding;
- operation registry lookup;
- execution identity and relevant resource-instance identity;
- correlation identity;
- plan compatibility;
- progression header decoding;
- server-side authorization;
- consistency basis interpretation;
- protected read/subscription authorization.

Trusted assumptions

Properties outside the proved core.

Examples:

- correctness of authentication middleware;
- declared model operation contract until the model layer itself becomes certified strongly enough to discharge it;
- XTDB's documented transaction and consistency behavior;
- HTMX lifecycle contracts;
- DOM/EventSource/browser primitive behavior;
- network and scheduler liveness premises where liveness is claimed.

Tests remain important evidence that trusted assumptions and implementation bindings behave as expected. They do not change classification from ”trusted” to ”proved.”

Non-negotiable architectural invariants

No accidental raw-to-certified substitution after certification begins

Once a stage has crossed into certified construction, downstream semantic APIs must not accept arbitrary same-shaped data as though it had crossed that boundary.

Bad:

```
(project
 {:verified? true
  :states ...})
```

The supported path should instead consume a value whose admission status was established by the corresponding checker boundary, conceptually:

```
(project verified-choreography)
```

This requirement is behavioral rather than a mandate that every stage use a specific Clojure nominal type. A private record, opaque wrapper, checked handle, validated certificate reference, or another representation may satisfy the boundary if ordinary cooperating code cannot accidentally substitute a raw map for the admitted value.

Clojure cannot make deliberate forgery or host-language bypass metaphysically impossible. Inside the trusted application boundary the purpose of these construction barriers is primarily to prevent accidental semantic substitution, make unsupported bypass conspicuous, and preserve precise diagnostics. Code that deliberately obtains lower-level authority and bypasses the supported boundary is outside the corresponding assurance claim.

No self-certification

A component may not establish an obligation solely by restating its own claimed guarantee.

The checker must distinguish:

```
claim
evidence
inference rule
checked conclusion
```

No circular semantic dependency without an explicit fixed-point rule

The certificate dependency DAG remains acyclic. A deliberately formalized recursive/fixed-point construction may occur inside a checked component using its own induction, guardedness, termination, or coinduction rule as appropriate. It does not permit certificates to discharge each other's premises through an unsupported dependency cycle.

Accidental namespace cycles do not become logical recursion merely because Clojure can load around them.

Proof artifacts never affect runtime meaning

A `DiagnosticProofSidecar`, proof trace, explanation, or external theorem file may help determine whether build-time certification succeeds.

Once an `ExecutablePlan` has been emitted, optional proof/diagnostic artifacts cannot alter its runtime transitions.

Executable plans must retain certified provenance

The production plan should retain enough immutable identity to establish:

- which verified choreography generated it;
- which semantic dependency set it was generated against;
- which plan/protocol version it uses;
- whether the runtime accepts that compatibility class.

The production browser does not need the full proof DAG.

Semantic identity is not proof

A fingerprint provides identity evidence under the specified canonicalization, encoding, and digest assumptions. It does not prove the semantic claim represented by those bytes. Exact equality claims must state whether they compare canonical terms directly or rely on a collision-resistance assumption.

Assertions remain visible

`:asserted` or application-trusted premises remain legitimate escape hatches where Gesso cannot establish domain truth.

They must not be silently promoted to `:derived` or `:authoritative`.

No theorem from finite examples

Property tests, generated traces, and Chromium scenarios provide scoped test evidence. Exhaustive checking of an exact finite state space establishes the property over that model and has the stronger `exact-finite-checked` class. Neither class automatically establishes a property of an unbounded production system. Extending the claim requires the sound abstraction or correspondence argument specified in Part I.

Certified construction pipeline

Certified construction is a sequence of semantic boundaries rather than a flag added to arbitrary data.

A HumanHelp Claim rule can be followed through those boundaries without asking the application author to write proof terms by hand.

Suppose the author writes a pleasant rule against certified domain names:

This is a deliberately small **illustrative policy conjunct**. The production HumanHelp Claim policy in Part VIII additionally relies on the relevant Membership/Organization rules and trusted current-state checks. Proving this rule well typed does not prove that it is the complete production policy or that its Clojure implementation conforms to it.

```
(defrule claimable?
  {:inputs
   {:request :humanhelp/Request
    :membership :humanhelp/Membership
    :organization :humanhelp/Organization}

   :returns :boolean}

  (and
    (= request.status :open)

    (= membership.status :active)

    (= membership.organization-id
       request.organization-id)

    organization.claiming-enabled?))
```

At this point the form is still a **raw declaration**. It is useful source data, but downstream certified construction must not consume it merely because it came from `defrule`.

Normalization resolves the semantic names and removes irrelevant authoring variation:

```
(def normalized-claimable
  {:term/kind :rule

   :rule/id :request/claimable?

   :inputs
   {:request
    {:semantic-type :humanhelp/Request}

    :membership
    {:semantic-type :humanhelp/Membership}

    :organization
    {:semantic-type :humanhelp/Organization}}

   :returns
   :gesso/Boolean

   :body
   [:and

    [:=
     [:field :request :request/status]
     [:enum :open]]

    [:=
     [:field :membership :membership/status]
     [:enum :active]]

    [:=
     [:field :membership :membership/organization-id]
     [:field :request :request/organization-id]]

    [:field
     :organization
     :organization/claiming-enabled?]]

   :dependencies
   {:request-schema
    :semantic/request-schema-S17

    :membership-schema
    :semantic/membership-schema-M11

    :organization-schema
    :semantic/organization-schema-O9}

   :normalization/version 4})
```

Formatting differences in the author's source need not change this normalized term. Changing `request.status` to a different semantic field does.

Elaboration can then resolve the operator/type structure and propose the derivations it expects will be needed:

```
(def elaborated-claimable
  {:subject
   normalized-claimable

   :resolved-fields
   {[:request :request/status]
```

```
   {:type :humanhelp/RequestStatus
    :presence :required}

   [:membership :membership/status]
   {:type :humanhelp/MembershipStatus
    :presence :required}

   [:membership :membership/organization-id]
   {:type :humanhelp/OrganizationId
    :presence :required}

   [:request :request/organization-id]
   {:type :humanhelp/OrganizationId
    :presence :required}

   [:organization :organization/claiming-enabled?]
   {:type :gesso/Boolean
    :presence :required}}

  :proposed-derivations
  [{:goal
    [:well-typed
     [:=
      [:field :request :request/status]
      [:enum :open]]]

    :proof
    [:eq-same-type
     [:field-type
      :request
      :request/status
      :humanhelp/RequestStatus]

     [:enum-member
      :open
      :humanhelp/RequestStatus]]}

   {:goal
    [:well-typed
     [:=
      [:field :membership :membership/status]
      [:enum :active]]]

    :proof
    [:eq-same-type
     [:field-type
      :membership
      :membership/status
      :humanhelp/MembershipStatus]

     [:enum-member
      :active
      :humanhelp/MembershipStatus]]}

   {:goal
    [:well-typed
     [:=
      [:field :membership :membership/organization-id]
      [:field :request :request/organization-id]]]
```

```
   :proof
   [:eq-same-type
    [:field-type
     :membership
     :membership/organization-id
     :humanhelp/OrganizationId]

    [:field-type
     :request
     :request/organization-id
     :humanhelp/OrganizationId]]}

  {:goal
   [:well-typed
    [:field
     :organization
     :organization/claiming-enabled?]]

   :proof
   [:field-type
    :organization
    :organization/claiming-enabled?
    :gesso/Boolean]}]})
```

The elaborator's proposed list is not itself the definition of ”all obligations that matter.” The rule checker independently derives the required coverage from the normalized subject and certification profile:

In particular, the checker recomputes referenced fields and reachable subexpressions from the normalized body. The `:resolved` set and the numbered child references in the following proof terms are claims to check against that body, not proof by self-declaration.

```
(def required-obligations
  (rule-checker/required-obligations
    {:subject normalized-claimable
     :profile :gesso/rules-v1}))
;; ⇒
[{:obligation/id :claimable/body-boolean
  :goal
  [:returns
   :request/claimable?
   :gesso/Boolean]}

 {:obligation/id :claimable/body-total
  :goal
  [:total
   :request/claimable?]}

 {:obligation/id :claimable/status-equality
  :goal
  [:well-typed
   [:=
    [:field :request :request/status]
    [:enum :open]]]}

 {:obligation/id :claimable/membership-equality
  :goal
  [:well-typed
   [:=
    [:field :membership :membership/status]
    [:enum :active]]]}

 {:obligation/id :claimable/organization-id-equality
  :goal
  [:well-typed
   [:=
    [:field :membership :membership/organization-id]
    [:field :request :request/organization-id]]]}

 {:obligation/id :claimable/claiming-enabled-boolean
  :goal
  [:well-typed
   [:field
    :organization
    :organization/claiming-enabled?]]}

 {:obligation/id :claimable/all-input-fields-resolved
  :goal
  [:all-field-references-resolved
   :request/claimable?]}]
```

The checker can now compare coverage:

```
(def coverage
  (obligation/check-coverage
    {:required required-obligations
     :proposed
     (:proposed-derivations
      elaborated-claimable)}))

coverage
;; ⇒
{:status :incomplete
 :missing
 [{:obligation/id :claimable/body-boolean
   ...}
  {:obligation/id :claimable/body-total
   ...}
  {:obligation/id :claimable/all-input-fields-resolved
   ...}]}
```

That does not mean the rule is false. It means the elaborator did not yet supply all evidence required for this artifact class.

A corrected elaborator output can add those derivations:

```
(def complete-derivations
  (vec (concat
    (:proposed-derivations
     elaborated-claimable)

    [{:goal
      [:all-field-references-resolved
       :request/claimable?]

      :proof
      [:closed-field-walk
       {:subject-id :request/claimable?
        :resolved
        #{[:request :request/status]
          [:membership :membership/status]
          [:membership :membership/organization-id]
          [:request :request/organization-id]
          [:organization :organization/claiming-enabled?]}}]}

     {:goal
      [:returns
       :request/claimable?
       :gesso/Boolean]

      :proof
      [:and-returns-boolean
       [:expression-returns-boolean
        0]
       [:expression-returns-boolean
        1]
       [:expression-returns-boolean
        2]
       [:expression-returns-boolean
        3]]}

     {:goal
      [:total :request/claimable?]

      :proof
      [:all-terms-total
       {:subject-id :request/claimable?
        :field-presence :required
        :operators #{:and := :field :enum}}]}])))
```

The accepting rule-set checker recomputes required coverage from the subject, then asks the proof kernel to check each derivation against already admitted semantic interfaces:

For this example, `rule-checker/check-all` derives the obligation set again instead of trusting `required-obligations` supplied from outside. The proof kernel checks that every numbered child refers to the actual corresponding expression and resolves each `:field-type` premise against an admitted dependency. Without those checks, the apparently complete proof below would not establish its advertised goal. It also checks totality of the actual field accesses and primitive operators; the `:all-terms-total` payload alone cannot assert totality for an omitted term, optional field, or partial primitive.

```
(def checked
  (rule-checker/check-all
    {:subject normalized-claimable

     :profile :gesso/rules-v1

     :dependencies
     {:request-schema
      verified-request-schema

      :membership-schema
      verified-membership-schema

      :organization-schema
      verified-organization-schema}

     :derivations
     complete-derivations}))

checked
;; ⇒
{:status :accepted

 :covered
 #{:claimable/body-boolean
   :claimable/body-total
   :claimable/status-equality
   :claimable/membership-equality
   :claimable/organization-id-equality
   :claimable/claiming-enabled-boolean
   :claimable/all-input-fields-resolved}

 :guarantees
 #{[:rule/well-typed :request/claimable?]
   [:rule/total :request/claimable?]
   [:rule/returns
    :request/claimable?
    :gesso/Boolean]}}
```

Only then does the certificate issuer produce an admitted rule-set value:

```
(def verified-claim-rules
  (certificate/issue-rule-set!
    {:subject
     normalized-claimable

     :semantic-id
     (semantic/id
       normalized-claimable)

     :dependencies
     {:request-schema
      (certificate/semantic-id
        verified-request-schema)

      :membership-schema
      (certificate/semantic-id
        verified-membership-schema)

      :organization-schema
      (certificate/semantic-id
        verified-organization-schema)}

     :checker
     {:profile :gesso/rules-v1
      :kernel/version 3}

     :checked-result
     checked}))
```

Downstream APIs accept that admitted product, not a raw map that merely resembles its serialized certificate:

```
(operation/verify
  raw-claim-operation
  {:rule-set
   verified-claim-rules})
;; ⇒ permitted to continue

(operation/verify
  raw-claim-operation
  {:rule-set
   {:semantic-id
    (certificate/semantic-id
      verified-claim-rules)

    :valid? true

    :guarantees
    #{[:rule/well-typed
       :request/claimable?]}}})
;; ⇒
{:error/type :certification/unadmitted-dependency
 :expected :VerifiedRuleSet
 :supplied :raw-map}
```

The ordinary HumanHelp developer still wrote the first `defrule`, not the proof objects. Gesso normalized, elaborated, generated the required obligation set, constructed/checks derivations, and issued the admitted artifact.

That is the practical meaning of ”declaration-first” certification: the proof machinery is substantial, but the authoring surface does not require the application developer to manually reproduce it.

Raw declaration

A raw declaration is ordinary application/framework input before the relevant semantic checker has accepted it.

Examples include:

- raw schema;
- raw relationship declaration;
- raw rule term;
- raw operation contract;
- raw choreography;
- raw projection request;
- raw application assembly.

Raw declarations remain useful, inspectable data.

They simply cannot be consumed by APIs that require the corresponding certified artifact.

Normalization

Normalization resolves representation choices that should not affect semantic identity.

Depending on artifact class, normalization may include:

- canonical ordering of closed declarations;
- resolved semantic field IDs;
- resolved nominal types;
- canonical operator terms;
- explicit default values;
- normalized assumption/guarantee forms;
- canonical dependency references.

Normalization must not silently erase semantically meaningful differences. Its contract identifies the source subset, semantic equivalences used, canonical encoding, and normalization version. It must preserve distinctions such as ordered versus unordered collections, absent versus explicit nil, nominally distinct identifiers, and closed versus extensible variants when they matter.

A certificate over normalized terms establishes a claim about those terms. If preservation from author syntax to those terms is not independently checked, that translation remains an explicit trusted premise. Normalization is not outside the assurance boundary merely because it runs before hashing.

Elaboration

Elaboration turns pleasant author syntax into explicit semantic data.

For example:

```
author DSL
    |
    v
closed rule AST
    |
    v
typed/resolved rule term + obligations
```

or:

```
choreography authoring syntax
    |
    v
normalized global transition representation
    |
    v
knowledge / provenance / authority obligations
```

Elaboration may be sophisticated.

It is not trusted to prove its own output.

Obligation generation

The artifact-class checking specification defines the propositions that must hold before the artifact may be certified. The elaborator may propose these obligations, but the accepting checker must establish that the required set is complete for the normalized subject and requested guarantee. Checking only the obligations volunteered by an elaborator cannot detect an omitted obligation.

Examples include:

- a field exists;
- a rule operator is defined for the operand types;
- an optional value has been refined before access;
- a role knows a branch discriminator;
- stronger provenance is justified;
- a required authority/capability exists;
- a dependency exports a required guarantee;
- a projection preserves required behavior;
- a closed registry contains the referenced member.

Derivation checking

Where an obligation is machine-derived, the system constructs evidence in the proof vocabulary.

The proof kernel or the appropriate dedicated checker validates that evidence.

A compiler/elaborator saying:

```
{:valid? true}
```

is not itself the derivation.

Certificate issuance

Only after all required obligations for the artifact class are discharged may the checker produce the corresponding certified value.

Conceptually:

```
RawRuleSet
    |
check
    v
VerifiedRuleSet

RawChoreography
    |
check using certified dependencies
    v
VerifiedChoreography

RawApplicationAssembly
    |
closure checks using certified semantic dependencies
    v
VerifiedApplicationAssembly / assembly-verified root
```

Downstream composition

Downstream construction consumes the certified artifact, not a same-shaped raw map.

For semantic reasoning, a downstream stage may rely only on the semantic interface that the upstream certificate exports. Diagnostic inspection of separate source/proof metadata is permitted under the sidecar boundary; it cannot introduce undeclared semantic premises.

It may not rely on private implementation detail merely because the value happens to contain it.

Certified construction is fail closed

If certification fails, the certified value does not exist.

The next stage therefore cannot proceed through the ordinary public API.

This gives the architecture a useful structural property:

```
failed upstream obligation
    |
no certified upstream artifact
    |
no legal downstream composition
```

The system should not construct a partial ”mostly verified” value and hope a later stage interprets the missing guarantee conservatively unless that partial artifact is itself an explicitly different artifact class.

Checked construction remains usable from Clojure

The architecture does not require a new host language.

Clojure remains the implementation and authoring host.

The stronger boundary may be represented through a combination of:

- checked constructors;
- opaque/private or nominal certified values where they materially prevent category confusion;
- predicates/accessors;
- explicit certificate data;
- semantic IDs;
- validated handles or registry references where those are already part of the design;
- public APIs that require admitted inputs.

These are realization choices, not a requirement to nominalize every domain value. A distinction deserves stronger representation when confusing two same-shaped values would violate a semantic invariant, cross a trust/admission boundary, or invalidate an assurance claim.

Clojure cannot make forgery metaphysically impossible.

The target is that ordinary correct use cannot accidentally bypass certification and that any deliberate bypass is unmistakably outside the supported interface. Routine downstream code should encounter less ceremony than the internal proof machinery: Gesso should construct admitted values mechanically whenever their status can be derived from declarations and trusted boundaries.

Typed business-rule language and distributed admissibility

A core requirement of this architecture is to reject malformed expressions of business rules without pretending Gesso knows whether the business chose the correct policy.

The distinction is:

```
well-formed but wrong policy
  (= request.status :open)
when the organization actually wanted some other policy

Gesso may accept this.

malformed policy expression
  (< request.status 10)

Gesso should reject this if request.status is an enum.
```

Required rule capabilities

The certified rule language should support only the smallest useful closed subset needed by real Gesso/HumanHelp code.

Candidate primitives:

- literals;
- typed field/reference access;
- equality/inequality where types support them;
- ordered comparison where the type declares order;
- boolean connectives;
- presence checks and presence refinements;
- membership over declared set/collection types;
- enum/variant predicates;
- declared deterministic derivations;
- relationship predicates;
- capability/role predicates supplied through certified model interfaces;
- explicit match/case over closed variants.

Do not expose arbitrary host-language execution inside the proved rule language. A host predicate or derivation used through an explicitly trusted extension must be named, typed, and classified as such. Declaring it deterministic does not prove purity, termination, or conformance. The proved subset and an extension that relies on asserted semantics are different assurance scopes.

Rule evaluation and refinement contract

The supported closed subset must have explicit evaluation semantics as well as typing rules. The contract states evaluation order and short-circuit behavior, absence/nil semantics, nominal equality, supported ordering, collection membership, and each declared primitive's failure behavior. It must not implicitly inherit incompatible host-language coercions or truthiness.

For every admitted primitive, either evaluation is total over its certified input domain or its typed failure/partiality is represented explicitly. Effects, time, randomness, database reads, and ambient authorization cannot enter a pure rule through an untracked helper. Required external facts enter as declared inputs with appropriate provenance and scope.

Refinements are path-sensitive and bound to the same subject, value observation, and environment on which they were established. A presence test for one Request cannot justify dereferencing another, nor can a check against an old observation justify a field in an unrelated replacement value. Branch joins retain only refinements established for every admitted incoming path, unless a stronger checked rule establishes the conclusion.

The smallest supported language may omit recursion and general quantification. Unsupported syntax receives an explicit diagnostic; it is not evaluated as arbitrary Clojure and then labeled certified. The supported subset must be sufficient for the real Claim pilot, rather than expanded speculatively.

Rule type checking

The checker should reject at least:

- unknown fields;
- unknown enum variants;
- operands of incompatible types;
- ordered comparison over unordered types;
- collection operations over scalar values;
- invalid cardinality assumptions;
- dereference of optional values without a proven presence refinement;
- branches that are non-exhaustive when exhaustiveness is claimed;
- impossible refinement combinations where the domain proves them impossible;
- invalid relationship endpoints;
- use of an operation result field not produced by that operation.

Choreographic admissibility of a valid rule

A rule can be intrinsically well formed but invalid at a particular distributed program point.

For example:

```
rule:
  (= request.private-review :approved)

intrinsic type:
  Boolean
  VALID

browser use before browser knows request.private-review:
  INVALID AT THIS PROGRAM POINT
```

Rule use should therefore add obligations for:

- knowledge;
- provenance where relevant;
- authority/trust where relevant;
- optionality/refinement state;
- basis/progression constraints where the rule relies on authoritative observation.

Rule syntax

The internal proof representation should be a closed data AST.

Application syntax may be a macro/DSL that elaborates into that AST.

Conceptual surface syntax:

```
(rule claimable?
  (and
   (has-role? current-user :helper)
   (= request.status :open)
   (= request.organization-id
      current-user.organization-id)))
```

Conceptual normalized term:

```
[:and
 [:has-role?
  [:principal :current]
  [:role :helper]]
 [:= [:field :request/status]
     [:enum :open]]
 [:= [:field :request/organization-id]
     [:field :current-user/organization-id]]]
```

The exact syntax is intentionally deferred until the proof vocabulary is small and explicit.

Mechanical rejection, formal counterexample, and intent review

The distinction above must remain visible in diagnostics and reports. Gesso has at least three importantly different ways to say that something deserves attention:

```
MECHANICALLY INVALID

The submitted construction does not satisfy the admitted language or one of
its checked semantic obligations.

Examples:
  enum compared as a number
  unknown relationship endpoint
  missing authoritative guard required by an operation contract
  certified rule used where its facts are not known

Result:
  reject the construction or certification step


COUNTEREXAMPLE

The construction is meaningful and admitted, but the modeled semantics permit
an execution that violates a declared invariant or other checked property.

Result:
  the affected formal claim is not established; report the counterexample and
  its source-level correspondence


SUSPICIOUS

The construction is internally meaningful and may satisfy every declared
invariant, but a human or heuristic reviewer finds the stated business policy
surprising, asymmetric, unusually permissive, or apparently inconsistent with
its explanatory prose or neighboring rules.

Result:
  review finding only; it does not become a proof premise, theorem, or
  certification failure merely because the reviewer dislikes the policy
```

These classes must not be collapsed into one generic warning/error channel. In particular, a successful model check cannot establish that the organization chose a sensible business policy, and a heuristic reviewer cannot turn its opinion about product policy into a kernel-checked theorem.

The distinction is also useful for automated repair. A repair tool may be allowed to change implementation or declarations in response to a mechanical error or counterexample, but changing the **meaning** of a business rule is a policy change. Such a change must remain visible as a semantic change rather than being disguised as an implementation repair.

Correctness-relevant policy must remain semantically visible

A large application may contain far more ordinary implementation code than business policy. Certification should therefore operate over a deliberately small semantic surface rather than requiring reviewers to rediscover policy by reading the whole program.

For a construction that claims Gesso assurance, conditions that determine a correctness-relevant semantic decision should be traceable to one of the following **within the semantic and implementation scope actually inspected**:

- an admitted named rule or derivation;
- another certified semantic interface that exports the required proposition;
- a runtime-enforced premise with an explicit acquisition/enforcement boundary;
- an explicitly named opaque/trusted assumption.

The last case can name the conformance of an entire arbitrary Clojure realization to its declared operation contract. It does not assert that Gesso has enumerated every branch inside that body. Reports identify the body, affected propositions, and the assurance profile under which this premise is permitted. Where complete correspondence is required, an opaque conformance premise cannot silently count as a checked derivation.

Correctness-relevant decisions include at least those that can change:

- authorization or capability;
- disclosure or visibility of protected information;
- eligibility for an authoritative operation;
- ownership or assignment;
- lifecycle/state-transition permission;
- command admission or incorporation;
- authoritative outcome classification;
- value movement or other domain effects protected by an invariant;
- whether a supposedly terminal fact may be superseded.

This requirement does **not** make every conditional in application code a business rule. Presentation formatting, collection plumbing, logging, resource cleanup, and other incidental branches need not pollute the semantic model merely because they use `if` or `case`. The boundary is whether the condition participates in the semantic decision for which assurance is being claimed.

The desired effect is semantic compression. A hundred-thousand-line application may legitimately have a correctness-relevant surface consisting of hundreds of named rules, dozens of authoritative operations, and a much smaller set of cross-operation invariants. Humans, formal tools, and optional heuristic reviewers should be able to inspect that smaller object without pretending it is the entire implementation.

Semantic-policy escape is an assurance defect

When Gesso can observe that a correctness-relevant decision depends on a predicate that is not represented by the admitted semantic model, it should report a semantic-policy escape rather than silently treating the host-language branch as implementation detail.

Conceptually:

```
SEMANTIC POLICY ESCAPE

operation:
  :request/claim

source realization:
  net.humanhelp.site.model.request/claim!

observed dependency:
  a branch in an inspected implementation subset affects whether the
  authoritative ownership transition is admitted

problem:
  the predicate is not discharged by an admitted rule, certified dependency,
  runtime-enforced premise, or named trusted assumption

repair choices:
  1. promote it to a named semantic rule;
  2. derive it from existing admitted rules;
  3. identify the certified/runtime boundary that establishes it; or
  4. expose it explicitly as an opaque trusted premise
```

The checker must state where it can establish absence of such escapes. For code generated from the admitted semantic representation, or code written in a closed checked subset, Gesso can require complete correspondence over that specified subset. For an arbitrary Clojure body it does not analyze, silence does not establish that no hidden policy branch exists. The body remains an explicit implementation-conformance/trust boundary until applicable analysis, constrained lowering, or other checked evidence discharges that obligation. The report must not turn ”no escapes detected” into ”all business rules in this body were discovered.”

Once such a boundary is explicitly registered, it is a named opaque premise, not an unresolved **detected** escape. Its actual behavior still needs evidence or trust appropriate to the requested profile. Naming the premise is more honest than making the body invisible: its subject and dependencies appear in assurance coverage and the root report. It cannot establish the completeness of the application's intended business policy.

Explanatory prose is review evidence, not an implicit theorem

Named business rules should retain enough human-facing material to make their meaning auditable without reverse engineering the normalized proof term. Useful attached material includes:

- a plain statement of the rule's meaning;
- the business rationale for the rule;
- a correctness note describing the stale-read, authority, atomicity, privacy, or lifecycle mistake the rule is intended to prevent;
- examples or counterexamples where they improve review.

This prose should travel with the semantic subject through generated audit and diagnostic views where practical. It is valuable input to humans and heuristic reviewers, and it can make a formal counterexample intelligible in application terms.

However, prose does not silently become a checked proposition. If a comment says ”completed Requests are terminal” while the admitted transition model permits `:completed` → `:open`, certification reasons from the admitted semantic model. The mismatch is an excellent review finding, not permission to pretend that terminality was proved.

The repair is to decide which statement expresses the intended policy and, when the prose is correct, promote that intent into an explicit rule or invariant.

Heuristic intent review is deliberately outside the proof kernel

A whole-application audit surface may be reviewed by a human, an LLM, a static heuristic, or another advisory tool for suspicious policy choices. Such review can look for patterns that formal validity alone does not identify, for example:

- one destructive operation has weaker authorization than analogous reads or writes;
- one lifecycle operation omits a requirement consistently present on its peers;
- a supposedly terminal state has a surprising outgoing transition;
- two near-identical operations apply materially different disclosure rules;
- an unusually permissive rule conflicts with its rationale or examples;
- an important domain state is unreachable or an important restriction appears conspicuously absent.

The reviewer should receive the compressed semantic/audit representation when available rather than being forced to infer business policy from an entire repository. This is particularly important for model-assisted review: the review task should be ”inspect these explicit business rules and their relationships”, not ”discover all policy hidden somewhere in one hundred thousand lines of Clojure”.

A heuristic finding has no authority to issue, weaken, or invalidate a certificate by itself. It should be represented separately, for example as:

```
review finding:
  :status :suspicious
  :subject :request/complete
  :observation "completion does not require active Membership although claim does"
  :basis [:request/claim :request/complete]
```

If the application owner confirms that the finding identifies a real business requirement, the requirement is added to the semantic declarations and the normal admission/certification pipeline runs again. At that point later violations can fail mechanically or produce formal counterexamples.

This creates a useful one-way ratchet:

```
heuristic suspicion
        |
        v
human/application-policy confirmation
        |
        v
explicit semantic rule or invariant
        |
        v
normal mechanical/formal enforcement thereafter
```

The heuristic layer discovers candidate intent; only the admitted semantic layer establishes what the system is actually required to enforce.

Assurance coverage is proposition coverage, not line coverage

Reports should make the semantic surface, inspection scope, and assurance status inspectable. At minimum, a whole-application report should be able to account for named correctness-relevant rules and dependencies by their actual evidence class and identify the implementation code it did not inspect:

```
semantic policy inventory for admitted generation G:
  declared business-rule identities                    173
  kernel-checked propositions                           80
  exact-finite-checked propositions                     61
  runtime-enforced premises                             18
  explicit trusted/opaque premises                      14

inspection coverage:
  complete correspondence scope: generated/closed checked subset
  unresolved *detected* escapes in that subset            0
  registered opaque implementation bodies                 4
  absence of hidden policy within those bodies: not established

advisory intent review:
  open suspicious findings                              3
```

The exact numbers are illustrative and are not a partition of the 173 rules: one rule can have a checked proposition, a runtime premise, and a trusted implementation correspondence. The report must give the subject, evidence, assumptions, and applicable checked scope behind each count. In particular, zero unresolved detected escapes means only that the inspected subset has no such unresolved finding. It cannot certify the absence of undiscovered policy inside the four opaque bodies. An artifact-wide ”certified” label must not conceal which propositions are checked, which are runtime enforced, which remain trusted, and which construction could not be accounted for.

Advisory findings remain outside the assurance totals unless and until they are promoted into admitted propositions. Conversely, an opaque premise remains in the assurance accounting even when tests or a reviewer make it look plausible.

Semantic interfaces and certificate hierarchy

The proof hierarchy follows semantic ownership.

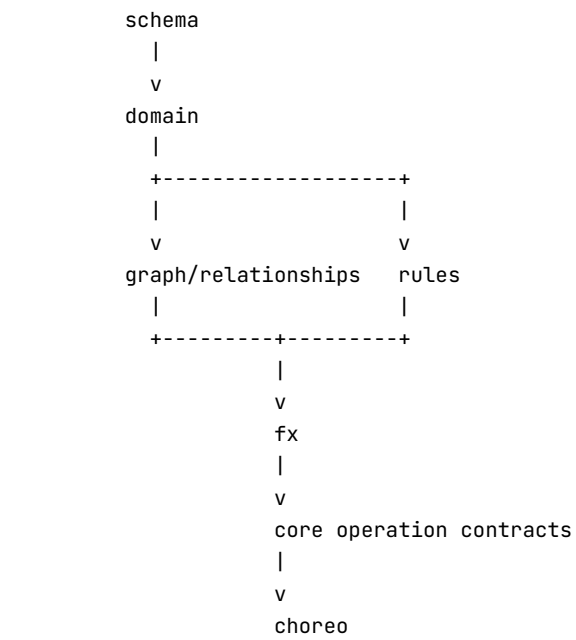
It is not simply a mirror of the filesystem.

A namespace participates only when it establishes or consumes semantic commitments relevant to downstream reasoning.

The goal is not to force every Clojure namespace into the proof system.

Namespaces that establish or consume semantic commitments should have an explicit certified interface.

The HumanHelp semantic dependency progression is conceptually:



HumanHelp source ownership remains governed by the production model boundaries defined earlier; this diagram describes semantic dependency, not necessarily source-file load order.

A concrete Claim dependency closure can preserve that ownership while still producing one compositional proof DAG.

The following maps illustrate the **readable contents** of interfaces and reports produced by admitted construction. Writing these `def` forms does not create certified values: downstream composition consumes checker-issued handles and rechecks serialized representations at the relevant trust boundary. Labels such as `:semantic/request-schema-S17` stand in for computed semantic identities; production certificates bind actual normalized subjects and dependency IDs, not these mnemonic symbols.

The schema certificates export only schema facts:

The Request status list and Organization boolean below are schematic example interfaces, not claims that these exact fields and variants exist in the current production model. The actual pilot must derive its closed enum from the production Request schema and either certify the real Organization policy interface or expose `organization/allows-claim?` as a named, typed, explicitly trusted primitive. Inventing `:organization/claiming-enabled?` to make the example typecheck would not certify the actual HumanHelp Claim path.

```
(def request-schema-interface
  {:interface/kind :schema

   :semantic-id
   :semantic/request-schema-S17

   :types
   {:humanhelp/Request
    {:fields
     {:request/id
      {:type :humanhelp/RequestId
       :presence :required}

      :request/status
      {:type :humanhelp/RequestStatus
       :presence :required}

      :request/organization-id
      {:type :humanhelp/OrganizationId
       :presence :required}

      :request/helper-id
      {:type
       [:optional :humanhelp/UserId]
       :presence :required}}}

    :humanhelp/RequestStatus
    {:kind :closed-enum
     :variants
     #{:open
       :claimed
       :on-the-way
       :completed
       :cancelled}}}})

(def organization-schema-interface
  {:interface/kind :schema

   :semantic-id
   :semantic/organization-schema-09

   :types
   {:humanhelp/Organization
    {:fields
     {:organization/id
      {:type :humanhelp/OrganizationId
       :presence :required}

      :organization/claiming-enabled?
      {:type :gesso/Boolean
       :presence :required}}}}})
```

Domain certificates may add refinements and declared application assumptions:

```
(def organization-domain-interface
  {:interface/kind :domain

   :semantic-id
   :semantic/organization-domain-014

   :dependencies
   {:schema
    :semantic/organization-schema-09}

   :guarantees
   #{[:field/boolean
      :humanhelp/Organization
      :organization/claiming-enabled?]

     [:policy/input-available
      :request/claimable?
      :organization/claiming-enabled?]}

   :assumptions
   #{{:assumption/id
      :humanhelp/organization-policy-source-trusted

      :class
      :trusted-model-contract

      :proposition
      [:model/read-current-organization
       :returns-current-policy-state]}}})
```

The Membership relationship certificate can state path/cardinality semantics without pretending that path existence is authorization:

```
(def membership-relationship-interface
  {:interface/kind :relationship

   :semantic-id
   :semantic/membership-rel-R8

   :guarantees
   #{[:relationship
      :membership/user
      {:from :humanhelp/Membership
       :to :humanhelp/User
       :cardinality :exactly-one}]

     [:relationship
      :membership/organization
      {:from :humanhelp/Membership
       :to :humanhelp/Organization
       :cardinality :exactly-one}]}

   :does-not-guarantee
   #{:authorization
     :current-database-conformance}})
```

The verified Claim rule set consumes those public interfaces:

```
(def claim-rule-certificate
  {:certificate/kind :rule-set

   :semantic-id
   :semantic/claim-rules-C21

   :dependencies
   {:request-schema
    :semantic/request-schema-S17

    :membership-schema
    :semantic/membership-schema-M11

    :organization-domain
    :semantic/organization-domain-O14

    :membership-relationships
    :semantic/membership-rel-R8}

   :guarantees
   #{[:rule/well-typed
      :request/claimable?]

     [:rule/returns
      :request/claimable?
      :gesso/Boolean]}

   :assumptions
   #{{:assumption/id
      :humanhelp/organization-policy-source-trusted

      :from
      :semantic/organization-domain-O14}}})
```

The operation certificate adds the public authoritative contract:

```
(def claim-operation-certificate
  {:certificate/kind :operation

   :semantic-id
   :semantic/request-claim-op-O31

   :dependencies
   {:rules
    :semantic/claim-rules-C21

    :request-schema
    :semantic/request-schema-S17}

   :interface
   {:operation :request/claim

    :input
    [:map
     [:request/id :humanhelp/RequestId]]

    :runtime-premises
    #{:authenticated-principal
      :current-request
      :current-membership
```

```
    :current-organization}

 :authority
 :request-authority

 :outcomes
 #{:accepted
   :rejected}

 :delivery-uncertainty
 {:recover {:acquire :request/card
            :command-attribution :unknown}}}

:guarantees
#{[:operation/outcomes-closed
   :request/claim]

  [:operation/authority-required
   :request/claim
   :request-authority]}

:implementation-assumptions
#{{:assumption/id
   :humanhelp/request-claim-implementation-conforms

   :class
   :trusted-model-contract

   :binding
   {:ns
    'net.humanhelp.site.model.request.core

    :var
    'claim}

   :proposition
   [:implementation/conforms
    :request/claim
    :semantic/request-claim-op-031]}})
```

The Choreo certificate depends on that operation interface rather than reaching into Request FX:

```
(def claim-choreo-certificate
  {:certificate/kind :choreography

   :semantic-id
   :semantic/request-claim-choreo-H12

   :dependencies
   {:operation
    :semantic/request-claim-op-031}

   :guarantees
   #{[:role/entry-knows
      :helper
      :request/id]

     [:authority/non-escalation
      :helper
      :request-authority]

     [:settlement/covered
      :request/claim
      #{:accepted
        :rejected}]

     [:delivery-uncertainty/recovery-declared
      :request/claim
      :request/card]

     [:projection/knowledge-sufficient
      :helper]}})
```

The helper projection and executable plan each carry their own correspondence claims:

```
(def helper-projection-certificate
  {:certificate/kind :projection

   :semantic-id
   :semantic/request-claim-helper-P7

   :dependencies
   {:choreography
    :semantic/request-claim-choreo-H12}

   :guarantees
   #{[:projection/preserves
      :request-claim/helper
      :supported-observables-v1]}})

(def executable-plan-certificate
  {:certificate/kind :executable-plan

   :semantic-id
   :semantic/request-claim-plan-E5

   :dependencies
   {:projection
    :semantic/request-claim-helper-P7}

   :guarantees
   #{[:lowering/preserves
      :request-claim/helper
      :gesso-plan-v3]}

   :implementation-assumptions
   #{{:assumption/id
      :gesso/plan-emitter-conforms

      :class :trusted

      :binding
      {:component :gesso.plan/emitter
       :version 3}}}})
```

Whole-application realization then adds evidence that is **not** automatically a proof-kernel theorem:

```
(def claim-assembly-evidence
  {:artifact/kind :application-realization

   :assembly/generation :A13

   :dependencies
   {:plan
    :semantic/request-claim-plan-E5

    :operation
    :semantic/request-claim-op-031}

   :claims
   [{:proposition
     [:view-affordance/realizes
      :request/card.claim
      :request/claim]

     :evidence
     :assembly-verified}

    {:proposition
     [:artifact/contains-exact-plan
      :artifact/gesso-live
      :semantic/request-claim-plan-E5]

     :evidence
     :assembly-verified}

    {:proposition
     [:trusted-route/binds
      :route/request-claim
      :server/request-claim]

     :evidence
     :assembly-verified}

    {:proposition
     [:server/runtime-enforces
      :current-authentication
      :request/claim]

     :evidence
     :runtime-enforced}

    {:proposition
     [:htmx-adapter/behaves-as-required
      :request/claim]

     :evidence
     :tested}]})
```

The root report can now preserve the dependency structure instead of flattening everything to `:verified true`:

```
(def claim-root-report
  {:root
   {:application :humanhelp/example
    :assembly/generation :A13
    :operation :request/claim}
```

```
  :semantic-dependencies
  [:semantic/request-schema-S17
   :semantic/membership-schema-M11
   :semantic/organization-schema-O9
   :semantic/organization-domain-O14
   :semantic/membership-rel-R8
   :semantic/claim-rules-C21
   :semantic/request-claim-op-O31
   :semantic/request-claim-choreo-H12
   :semantic/request-claim-helper-P7
   :semantic/request-claim-plan-E5]

  :claims
  [{:proposition
    [:rule/well-typed
     :request/claimable?]
    :evidence :kernel-checked}

   {:proposition
    [:projection/preserves
     :request-claim/helper
     :supported-observables-v1]
    :evidence :kernel-checked}

   {:proposition
    [:delivery-uncertainty/recovery-declared
     :request/claim
     :request/card]
    :evidence :kernel-checked}

   {:proposition
    [:application/path-closed
     :request/claim]
    :evidence :assembly-verified}

   {:proposition
    [:current-principal/re-established
     :request/claim]
    :evidence :runtime-enforced}

   {:proposition
    [:real-chromium/claim-path-exercised]
    :evidence :tested}]

  :open-assumptions
  #{{:assumption/id
     :humanhelp/organization-policy-source-trusted
     :class :trusted-model-contract}

    {:assumption/id
     :humanhelp/request-claim-implementation-conforms
     :class :trusted-model-contract}

    {:assumption/id
     :browser/htmx-lifecycle-contract
     :class :runtime-primitive}}})
```

A developer inspecting this report can answer both ”what depends on Organization semantics?” and ”which Claim facts are still trusted rather than proved?” without confusing either question with whether Alice is currently authorized at runtime.

That is the compositional benefit: each layer inherits exactly the public guarantees it needs and carries forward the assumptions that have not actually been discharged.

Schema certificate

May establish:

- nominal semantic type names;
- Malli/value representation;
- fields;
- required/optional status;
- enum variants;
- cardinality;
- normalized compatibility identity.

It must not establish business authority merely because a value validates. A declaration that a field is required or has a particular type is a schema contract; it does not establish that every existing database record satisfies it. Concrete data must enter through the corresponding validated or explicitly trusted acquisition boundary.

Domain certificate

May establish:

- domain refinements;
- legal state variants;
- relationships among domain concepts;
- deterministic pure derivations;
- state invariants that can actually be checked from the declared domain;
- application assumptions that cannot be derived.

Relationship/graph certificate

May establish:

- valid relationship endpoints;
- cardinality constraints;
- cross-model references;
- path typing;
- derived relationship facts.

It must not silently infer authorization from graph reachability. A declared cardinality is not proof that stored relationships currently obey it. A graph certificate must distinguish path typing, assumptions about model invariants, and facts actually obtained from an admitted observation.

Rule-set certificate

May establish:

- each rule is intrinsically well typed;
- each referenced semantic field exists;
- each refinement is coherent;
- each declared derivation is valid under its inputs;
- named assumptions are explicit.

FX/operation certificate

May establish:

- required typed inputs;
- produced typed outputs;
- declared effects/resources;
- atomic model boundary ownership;
- preconditions;
- postconditions;
- expected domain outcomes;
- failure classes;
- authority requirements.

A certificate may initially rely on a trusted assertion that the ordinary Clojure implementation satisfies the contract. Stronger implementation-conformance checking can reduce that trusted surface where useful. The assumption must identify the implementation realization or binding whose conformance is trusted. A contract's unchanged semantic identity does not transfer implementation evidence to a changed function body.

Core certificate

May establish that public semantic operations:

- refer to certified operation contracts;
- preserve model ownership boundaries;
- expose only declared outcomes;
- do not advertise stronger guarantees than underlying FX/model contracts.

Choreo certificate

Consumes only certified semantic interfaces and establishes distributed properties such as:

- entry knowledge;
- communication knowledge transfer;
- authoritative observation;
- deterministic derivation;
- branch distinguishability;
- provenance;
- authority flow;
- resource discipline;
- typed continuations;
- terminal behavior.

Projection certificate

Establishes the stated projection property for the supported global/local semantics under an explicit theorem or checked property. The claim identifies observable events, permitted internal steps, communication/queue assumptions, branch knowledge, and safety or progress scope. An individual local simulation must not be described as correctness of the composed participant system unless the corresponding composition obligation has also been established.

Executable-plan certificate

Establishes the declared correspondence between the certified projection and lowered runtime data under the named format/protocol version and dependency identities. The claim must distinguish checked lowering correctness from trusted compiler conformance. Merely recording the input certificate's identity on the output plan proves no preservation property.

Whole-application realization belongs in the same dependency story

The hierarchy does not stop at `ExecutablePlan`.

Part VII established that an application can have individually valid semantic pieces and still be impossible as an assembled program.

Therefore the root assurance graph extends through application realization.

The certificate hierarchy extends beyond the original model/choreography chain to the whole-application realization introduced in Part VII.

This extension must preserve the distinction between semantic proof and assembly verification.

Application-assembly certificate

An application-assembly certificate may establish that a normalized `ApplicationAssembly` is structurally closed relative to its certified semantic dependencies and declared realization facts.

It may record guarantees such as:

- every exposed semantic operation resolves to a known certified operation;
- every browser-exposed operation has a compatible certified projection;
- every required browser plan is represented in the current browser manifest;
- browser artifact identity corresponds exactly to the required manifest;
- required transport bindings exist;
- trusted routes correspond to declared semantic operations;
- trusted server realizations correspond to certified operation contracts;
- required server capabilities resolve to compatible admitted provider contracts, including the declared origin, value shape, scope, and lifecycle dependencies;
- required settlement/progression relationships are represented;
- every managed authoritative fragment has initial knowledge or a permitted acquisition path;
- semantic-route collisions cannot silently erase operation identity.

These guarantees retain their actual evidence class, commonly `:assembly-verified`. They may be called proved only when the corresponding proposition and derivation justify that stronger classification. Classification is not automatically upgraded when the implementation reaches a later stage.

The root must also retain independent production-enforcement and required user-capability evidence under #req-application-assurance-claims. Neither the certificate hierarchy nor an internally closed assembly establishes that the production entrypoint uses it or that HumanHelp includes all promised actions.

Browser-artifact correspondence certificate

A browser-artifact correspondence artifact may establish:

- current browser manifest semantic identity;
- exact generated artifact stamp/digest;
- set of required executable plans;
- exact correspondence between required plans and generated artifact.

It establishes only the defined build correspondence under the checked build binding and named trust assumptions. The manifest, emitted payload, and actual artifact bytes must be related by the acyclic construction and serving contracts of #req-browser-assembly-manifest. A copied stamp is insufficient.

It does not establish:

- runtime authorization;
- correctness of HTMX/browser primitives;
- current server compatibility of a suspended browser;
- truth of domain facts.

Rendered-surface verification evidence

Dynamic rendered-surface checking may produce structured verification evidence for a concrete render.

That evidence may establish:

- every semantic affordance retains its operation identity;
- every effective semantic transport corresponds to the assembly;
- no anonymous ordinary HTMX action occupies a semantic route;
- unavailable realizations are not silently downgraded;
- concrete subjects/route parameters satisfy declared structural constraints.

Because the exact render depends on runtime data, this evidence is normally runtime-enforced/assembly verification rather than a static certificate over all possible renders. Evidence binds the concrete checked render and admitted assembly generation; it cannot be reused as a certificate for later renders with different data or bindings. Excluded serialized/streamed paths retain explicit assurance boundaries. A valid render is not a substitute for server protocol admission or current model authorization.

Root application proof report

The root application report should be able to traverse:

- certified model/schema dependencies;
- rule and relationship certificates;
- operation contracts;
- choreography certificate;
- projection certificate;
- executable-plan certificate;
- application-assembly certificate/report;
- browser artifact correspondence;
- runtime-enforced premises;
- trusted assumptions.

Conceptually:

```
ROOT APPLICATION REPORT

semantic proof chain:
    schema/domain/rules
      -> operation
      -> choreo
      -> projection
      -> ExecutablePlan

application realization chain:
    affordance
      -> plan
      -> artifact
      -> transport
      -> route
      -> trusted server
      -> model operation
      -> settlement/progression
      -> acquisition

runtime premises:
    authenticated principal binding
    concrete schema checks
    execution/resource-instance identity and event correlation
    dynamic authorization
    plan compatibility

trusted assumptions:
    XTDB
    browser primitives
    HTMX lifecycle
    authentication implementation
```

The root report is an explanation of assurance. It identifies the root subject, admitted assembly generation, selected certification profile, dependency closure, and the actual evidence status of each required claim. Missing, unsupported, or unresolved evidence must remain distinguishable from accepted conditional claims and from tested behavior.

It is not one undifferentiated theorem.

Certificate representation

A certificate records the checked semantic boundary.

A conceptual certificate shape:

```
{:gesso.proof/type :verified-rule-set
 :gesso.proof/version 1

 :semantic-id "..."
 :certification-profile :rule-set/closed-typed
 :semantics-id "..."

 :subject
 {:kind :rule-set
  :id :humanhelp.request/claim-rules}

 :dependencies
 [{:kind :schema
   :id :humanhelp.request/schema
   :semantic-id "..."}
  {:kind :schema
   :id :humanhelp.user/schema
   :semantic-id "..."}]

 :assumptions
 [{:id :humanhelp/request-claim-policy
   :class :application-asserted
   :proposition ...}]

 :guarantees
 [...]

 :obligations
 [...]

 :derivations
 [...]

 :checker
 {:proof-kernel-version 1
  :rule-set-id "..."
  :elaborator-version 1}}
```

This shape is conceptual. Final keys and namespaces should follow the canonical Gesso artifact vocabulary in the implementation while preserving the semantics specified here. Each guarantee must resolve to its proposition, assurance class, justification, and open assumptions; an artifact-wide status cannot replace those bindings. Any bounded result also records its model/exploration scope. The checker establishes these relationships rather than trusting the fields.

Certified values and public API discipline

The certificate is evidence.

The public Clojure API must also prevent ordinary downstream construction from treating arbitrary data as certified merely because it has the same shape.

Clojure does not provide Rust's nominal compile-time privacy guarantees, and the design does not depend on pretending that it does.

The requirement is that ordinary correct use is strongly guided and accidental category substitution is difficult through the supported API.

Permitted techniques include:

- private implementation records/types where appropriate;
- constructors that perform checking and return opaque or nominal admitted values;
- public predicates/accessors rather than map-shape conventions;
- sealed-ish protocol ownership inside Gesso namespaces;
- validated handles or references to checker-owned admitted artifacts;
- no public `map→VerifiedThing` style escape hatch where it would allow a raw representation to impersonate an admitted value;
- structural validation and the required semantic/derivation rechecking at certificate deserialization boundaries;
- semantic fingerprint validation before composition;
- proof-kernel checking before accepting externally stored/cached certificates.

The implementation should choose the smallest mechanism that preserves the semantic distinction. A nominal type is especially useful where two values can have the same representation but materially different provenance, authority, lifecycle, or assurance status. It is unnecessary when ordinary structural data already carries all semantics required by the consumer.

Do not claim that an opaque or nominal Clojure value is itself a proof.

The proof is the checked derivation and dependency chain represented by it; an admitted runtime category records that the required construction boundary was crossed.

The public API must make checked construction the path of least resistance.

Certified values are not map conventions

A value does not become certified because it contains:

```
{:valid? true
 :semantic-id "..."}
```

Certified downstream APIs consume values created by checker-controlled constructors or validated deserialization boundaries.

The semantic identity is evidence of which normalized interface was checked.

The derivation/certificate establishes why guarantees may be relied upon.

Raw and certified values are different products

The API should make a visible distinction between:

```
RawRuleSet
VerifiedRuleSet

RawChoreography
VerifiedChoreography

RawProjection
VerifiedProjection

RawApplicationAssembly
VerifiedApplicationAssembly
```

The exact Clojure representation may use private records, protocols, opaque handles, or another mechanism.

The key requirement is behavioral:

> Ordinary public downstream construction cannot bypass checking merely by assembling a map with the right keys.

This distinction should impose little routine ceremony on downstream web code. Application authors should normally write the source declarations, rules, operations, and choreography. Elaboration, verification, authoritative acquisition, and model boundaries should construct the stronger admitted values. Manual wrappers are appropriate mainly when application code intentionally crosses one of those semantic boundaries itself.

A useful API acceptance criterion is:

> Where two same-shaped values have materially different semantic meanings, accidental substitution through the supported API is either structurally unavailable or rejected immediately with a diagnostic naming the expected semantic category, the supplied category, and the missing admission or derivation step.

Deserialization rechecks trust boundaries

Cached or persisted certificates re-enter through a validation boundary.

The system must validate at least:

- certificate structure;
- applicable semantic-checker or proof-kernel version compatibility;
- semantic identity;
- dependency identities;
- derivation validity where required;
- signatures or transport integrity if the declared storage trust boundary requires them;
- the normalized subject and proposition identities actually bound by the certificate, not only supplied identifier strings;
- all required transitive dependencies and assumption scopes.

A signature authenticates an issuer or bytes under its key trust contract; it is not a derivation of the signed proposition. Likewise, unsigned locally cached data may be accepted after complete required rechecking. Do not require a signing infrastructure when rechecking supplies the necessary trust boundary. Unknown rule tags, unsupported semantic/checker profiles, malformed references, or unavailable required evidence fail admission rather than falling back to `:valid?` or a previous successful report.

A stored certificate is not trusted merely because it came from yesterday's build directory.

Likewise, crossing an untrusted serialization boundary does not preserve a purely in-process nominal or opaque admission claim. Encoded bytes re-enter as transport data and regain stronger semantic status only through the verification, authentication, authoritative acquisition, or certificate-checking boundary appropriate to that category.

Checker-controlled constructors do not imply proof of implementation bodies

A certified operation contract may initially state:

```
trusted assumption:
    production Clojure function satisfies this operation contract
```

That is legitimate when explicit.

The fact that the contract object is certified does not prove arbitrary Clojure implementation conformance.

Stronger implementation-conformance work may shrink that trusted boundary.

Until then, diagnostics and root reports keep it visible.

Certified interfaces should be minimal

A semantic interface should export only commitments downstream reasoning is allowed to depend on.

Over-exporting implementation details has two costs:

- it creates unnecessary proof dependencies;
- it invalidates downstream artifacts on irrelevant implementation changes.

The target is a normalized semantic contract, not a serialized source namespace.

Compatibility starts strict

The default safe rule is exact semantic dependency identity.

Compatibility/subtyping may be introduced only when a concrete evolution problem justifies it and the compatibility relation itself is checked.

This avoids an early system in which vague ”compatible enough” logic silently recreates the same proof-by-reputation problem the certificate architecture was introduced to remove.

Composition, semantic identity, and stale-dependency rejection

Composition is where independent certified artifacts become one larger trusted semantic construction.

The default rule is strict:

> A dependent stage accepts only dependencies whose certified semantic interfaces satisfy the exact assumptions and identity requirements of the dependent construction.

A dependent stage must reject immediately if:

- a required semantic dependency is missing;
- a supplied dependency has the wrong certificate kind;
- semantic identities do not match the dependency named in the certificate;
- the required semantic-checker or proof-kernel version is unsupported;
- an upstream guarantee needed by the dependent stage is absent;
- an assumption is supplied with an incompatible proposition/type;
- two dependencies export incompatible definitions of the same semantic identity;
- a proof obligation remains unresolved;
- a supposedly closed rule/message/variant set contains an undeclared member;
- a dependency graph contains an unsupported cycle;
- a stale certificate is paired with a changed semantic interface;
- a certificate claims a theorem not derivable under its recorded premises.

A failure at `request.rules` means no `VerifiedRequestRules` exists.

Therefore:

```
no VerifiedRequestRules
  -> no VerifiedRequestOperationRegistry
  -> no VerifiedRequestChoreography
  -> no VerifiedProjection
  -> no certified ExecutablePlan
```

A production build should not ”try anyway.”

Goals

Semantic identity must allow:

- stale proof detection;
- deterministic caching;
- dependency mismatch rejection;
- reproducible build diagnostics;
- proof DAG inspection.

It must avoid invalidating the world for irrelevant source edits.

Normalize before fingerprinting

Fingerprint normalized semantic terms, not source text.

For example, a rule's semantic identity may include:

- normalized operator tree;
- resolved nominal types;
- resolved semantic field ids;
- declared assumptions;
- declared guarantees.

It should exclude:

- whitespace;
- comments;
- local source formatting;
- unrelated helper implementation;
- diagnostic-only source spans.

Source locations remain attached separately for diagnostics.

Dependency identities are transitive evidence, not transitive hashes alone

A certificate records direct certified dependency identities.

The root proof DAG can reconstruct the transitive dependency closure.

Do not mistake a Merkle-like dependency hash for proof validity. The checker still validates certificate structure and derivations.

Compatibility rather than equality where justified

Not every downstream dependency requires byte-identical semantics.

Some certified interfaces may admit an explicit compatibility relation such as:

```
new interface provides every guarantee required by old dependent
and does not invalidate a relied-upon closed-world assumption
```

Do not implement semantic subtyping/compatibility speculatively. Begin with exact semantic identity where correctness is clearer, then generalize only for a concrete evolution problem.

Identity evidence is not proof evidence

Semantic identity answers:

> Is this the normalized semantic interface whose identity the dependent artifact records?

It does not answer:

> Was that interface correctly certified?

Both checks matter.

A copied semantic ID on a forged raw value is not a certificate.

A valid certificate for the wrong semantic dependency is not compatible merely because the artifact class matches.

Staleness propagates through dependency edges

Suppose:

```
organization.domain O1
    |
claim-rules R1 depends on O1
    |
claim-operation C1 depends on R1
    |
claim-choreo H1 depends on C1
    |
projection P1 depends on H1
```

If the semantic interface becomes O2 and R1 relied on changed meaning, R1 no longer composes.

The ordinary dependency chain stops there.

Later stages do not receive permission to keep using C1/H1/P1 simply because their own serialized bytes have not changed.

The HumanHelp Claim chain gives a concrete example of how that staleness should propagate through an otherwise successful cache.

Assume Organization domain version O14 exports:

```
(def organization-domain-O14
  {:semantic-id
   :semantic/organization-domain-O14

   :guarantees
   #{[:field/boolean
      :organization/claiming-enabled?]

     [:policy/input-available
      :request/claimable?
      :organization/claiming-enabled?]}})
```

Claim rules C21 were certified against exactly that interface:

```
(def claim-rules-C21
  {:semantic-id
   :semantic/claim-rules-C21

   :dependencies
   {:request-schema
    :semantic/request-schema-S17
    :membership-schema
    :semantic/membership-schema-M11
    :organization-domain
    :semantic/organization-domain-O14
    :membership-relationships
    :semantic/membership-rel-R8}

   :guarantees
   #{[:rule/well-typed
      :request/claimable?]}})
```

Now the application intentionally evolves Organization policy. Instead of a boolean field, the certified semantic interface becomes a closed policy enum:

```
(def organization-domain-O15
  {:semantic-id
   :semantic/organization-domain-O15

   :guarantees
   #{[:field/type
      :organization/claim-policy
      :humanhelp/ClaimPolicy]

     [:enum/variants
      :humanhelp/ClaimPolicy
      #{:claim-policy/open
        :claim-policy/helpers-only
        :claim-policy/closed}]}})
```

The change may be a perfectly coherent product evolution. O15 is not ”malformed.”

But old C21 explicitly depends on O14:

```
(certification/compose
  {:artifact verified-claim-rules-C21

   :dependencies
   {:organization-domain
    verified-organization-domain-O15}})
;; ⇒
{:status :rejected

 :error/type
 :certification/semantic-dependency-mismatch

 :artifact
 :semantic/claim-rules-C21

 :dependency
 :organization-domain

 :expected
 :semantic/organization-domain-O14

 :supplied
 :semantic/organization-domain-O15}
```

The cache may still physically contain C21, operation O31, choreography H12, projection P7, and plan E5:

Here `cached-C21` and its peers denote checker-issued handles (or persisted candidates rechecked before use). The earlier `def` maps only displayed their public interfaces; supplying those maps to `compose` would instead fail as unadmitted input, regardless of the stale-dependency check.

```
(def cache
  {:semantic/claim-rules-C21
   cached-C21

   :semantic/request-claim-op-O31
   cached-O31

   :semantic/request-claim-choreo-H12
   cached-H12

   :semantic/request-claim-helper-P7
   cached-P7

   :semantic/request-claim-plan-E5
   cached-E5})
```

Their presence is not permission to admit them into the new closure.

A dependency-aware cache lookup can require the current normalized dependency set:

```
(defn reusable?
  [cached
   {:keys [subject
           dependencies
           checker-profile]}]

  (and
    ;; This checks a checker-issued handle or a fully rechecked persisted
    ;; certificate, never a raw map's :valid? flag.
    (certificate/admitted? cached)

    (=
      (certificate/subject-semantic-id cached)
      (semantic/id subject))

    (=
      (certificate/dependency-semantic-ids cached)
      (into {}
        (map
          (fn [[k certified]]
            [k
             (certificate/semantic-id
               certified)]))
        dependencies))

    (=
      (certificate/checker-profile cached)
      checker-profile)

    (certificate/admissible?
      cached)))
```

The lookup for C21 now fails:

```
(reusable?
  cached-C21
  {:subject
   current-claim-rules-source

   :dependencies
   {:organization-domain
    verified-organization-domain-O15}

   :checker-profile
   :gesso/rules-v1})
;; ⇒ false
```

Gesso re-elaborates Claim rules against O15. The old source itself may no longer typecheck because it still references `organization.claiming-enabled?`:

```
(check-rule
  '(and
     (= request.status :open)
     (= membership.status :active)
     organization.claiming-enabled?)

  {:organization-domain
   verified-organization-domain-O15})
;; ⇒
{:status :rejected

 :error/type
 :certification/unknown-field

 :field
 :organization/claiming-enabled?

 :available
 #{:organization/claim-policy}}
```

The developer repairs the rule:

Accepting both `:claim-policy/open` and `:claim-policy/helpers-only` without checking the helper role would be **well typed but policy wrong**. The repaired source explicitly declares a principal input and uses a certified role predicate. A missing role interface is an unresolved dependency, not a reason to treat every active member as a helper:

```
(defrule claimable?
  {:inputs
   {:request :humanhelp/Request
    :membership :humanhelp/Membership
    :organization :humanhelp/Organization
    :principal :humanhelp/AuthenticatedPrincipal}}

  (and
    (= request.status :open)

    (= membership.status :active)

    (= membership.organization-id
       request.organization-id)

    (= organization.id
       request.organization-id)

    (case organization.claim-policy
      :claim-policy/open true
      :claim-policy/helpers-only
      (has-role? principal :helper)
      :claim-policy/closed false)))
```

This still proves only the rule's declared subset under its certified inputs. The server must bind the current authenticated principal, Request, Membership, and Organization, and the implementation-conformance premise remains explicit.

Certification produces a new rule identity C22:

```
{:certificate/kind :rule-set

 :semantic-id
 :semantic/claim-rules-C22

 :dependencies
 {:request-schema
  :semantic/request-schema-S17
  :membership-schema
  :semantic/membership-schema-M11
  :organization-schema
  :semantic/organization-schema-O9
  :organization-domain
  :semantic/organization-domain-O15
  :principal-role-interface
  :semantic/principal-helper-role-PR1}

 :guarantees
 #{[:rule/well-typed
    :request/claimable?]}}
```

Any downstream artifact whose **semantic interface** actually depends on the changed rule set must now be rebuilt or recertified:

```
organization O15
    |
claim rules C22
    |
claim operation O32
    |
claim choreography H13
    |
helper projection P8
    |
executable plan E6
```

This does not mean every implementation artifact in HumanHelp must change. A CSS file, an unrelated Search view, or a Request serialization helper may be unaffected.

Nor does it mean every change to Organization source forces O15. If the source change is only a comment or a local helper refactor and normalization still produces exactly the O14 semantic interface, the semantic ID may remain O14 and contract-level downstream proofs can be reused.

Implementation evidence is narrower. Suppose the Organization semantic interface remains O14 but the implementation function behind its trusted model contract changes:

```
{:semantic-interface
 :semantic/organization-domain-O14

 :implementation-binding
 {:before
  {:ns 'net.humanhelp.site.model.organization.core
   :var 'current
   :implementation-digest "sha256:impl-44"}

  :after
  {:ns 'net.humanhelp.site.model.organization.core
   :var 'current
   :implementation-digest "sha256:impl-45"}}}
```

Rule/choreography derivations depending only on O14 can remain valid, while implementation-conformance or production test evidence attached to `"sha256:impl-44"` cannot simply be copied to `"sha256:impl-45"`.

That is the reuse model this section is aiming for:

```
semantic interface unchanged
    -> semantic derivations may be reusable

implementation binding changed
    -> implementation evidence may need renewal

semantic interface changed
    -> dependent semantic certificates do not compose until checked again
```

Caching follows the proof DAG rather than source-file timestamps and follows implementation identities for claims that actually concern implementation.

Compatibility must be earned

Exact semantic identity is the safe initial rule.

An explicit compatibility relation may admit evolution such as:

```
new interface
    provides every guarantee required by old dependent
    preserves all closed-world assumptions that dependent relied upon
```

but that compatibility relation is itself semantic machinery requiring checks.

”Looks backwards compatible” is not a supported proof rule.

Assumption propagation and discharge

Assumptions are scoped propositions, not just names. Their identity includes the relevant subject, parameters, semantic context, and dependencies. Two equal labels with different propositions cannot be merged. The report must make the assumptions of each exported guarantee inspectable through the dependency DAG; flattening a root list must not lose which claims depend on which premise.

A downstream derivation may discharge an assumption only by supplying an accepted proof of the required proposition under a checked substitution and compatible scope. Its replacement proof's own assumptions remain dependencies. The original unresolved premise may disappear from the open-assumption list, but the discharge evidence remains reconstructible. Renaming or copying the premise into another certificate is not discharge.

Runtime authorization and authenticated-principal premises require the actual invocation boundary; a static certificate that names a principal or capability does not satisfy them for future requests. Environmental liveness premises remain attached to the claims that use them. No certificate supplies fairness, network recovery, or a user's eventual Refresh action simply by requiring it.

An artifact class must state which assumptions it permits and which obligations must be checked rather than asserted. Unsupported syntax, missing dependencies, and invalid proof rules cannot be excused by automatically converting failure into an application assumption. An explicitly weaker artifact/profile is a different result and cannot satisfy a stronger requested milestone.

The checker rejects detectable contradictions within its supported logic and incompatible definitions of the same scoped proposition. It need not decide arbitrary consistency of application policy or opaque trusted predicates. A conditional theorem under externally inconsistent assumptions does not prove a realizable usable application. Positive HumanHelp scenarios and independent behavioral acceptance remain necessary to prevent vacuous completion claims.

Semantic reuse and implementation evidence

Exact semantic identity permits reuse only for claims that depend on that semantic interface. Evidence about a concrete implementation, provider binding, compiler, generated artifact, or browser integration must remain attached to the corresponding implementation and admitted assembly-generation identities, where relevant, and the environment scope. An implementation change can preserve all downstream contract-level derivations while requiring renewed implementation conformance or test evidence.

Caching must key admission on the normalized subject, dependencies, rule set, checker semantics/version, certification profile, and relevant assumptions. Source spans may change without semantic invalidation, but diagnostics must still identify the source associated with the current construction. An old accepted result cannot override an explicitly unsupported or withdrawn checker profile in the current admission policy.

Admission of one immutable dependency closure must not race mutable lookups that later substitute a different dependency. Construction either retains the admitted closure or detects changes and rechecks before use. This follows Part VII's admitted assembly-generation binding; it does not require one global lock or storage implementation.

Proof kernel and proof-producing elaboration

The logical trusted base should remain much smaller than the compiler that constructs semantic artifacts.

The end state should have a deliberately small trusted proof kernel.

The large compiler/elaborator is allowed to be complicated. The proof kernel must remain small enough to inspect independently.

The canonical terminology from Part I distinguishes three responsibilities:

- The **executable semantic core** determines production transitions from executable rules, admitted state, and inputs/events, producing state and declared effects.
- The **proof kernel** accepts or rejects derivations under explicit inference rules and premises. It does not execute the application trajectory or supply runtime facts, ownership, or authorization.
- A **dedicated semantic checker** establishes its stated judgment through its own checking mechanism, such as structural closure or exact finite exploration. Its result retains the evidence class and trusted premises of that mechanism.

These components may share canonical semantic definitions, but sharing does not merge their responsibilities or prove implementation conformance. The proof kernel checks proposed derivations independently of the elaborator's claimed success. The production executable semantic core remains necessary regardless of the certification profile; introducing the later proof kernel does not create a second runtime semantics.

This terminology does not prescribe a new namespace, expand the supported rule language, or settle the judgment-calculus definitions deferred by this part.

Conceptually:

```
raw application syntax
        |
        v
large elaborator/compiler
        |
        | proposes terms + derivations
        v
small proof kernel
        |
        | accepts/rejects derivation
        v
certificate
```

A compiler bug that generates an invalid derivation must therefore fail at the proof-kernel boundary. This guarantee alone does not cover omitted obligations, incorrect source normalization, or incorrect executable lowering. Those require their own checked correspondence or explicit trusted premises.

Candidate proof-kernel responsibilities

Keep the proof kernel small enough to audit.

Possible responsibilities:

- nominal type equality;
- closed enum/variant membership;
- rule-expression typing;
- presence/refinement entailment;
- knowledge entailment;
- provenance entailment;
- authority/capability entailment;
- conjunction/disjunction of propositions;
- declared deterministic derivation application;
- message contract matching;
- transition precondition/postcondition compatibility;
- certificate dependency matching;
- selected resource linearity/ownership rules;
- selected properties of the declared progression requirement-combination contract, without assuming every implementation supplies a universal join;
- selected projection proof obligations.

Some properties may remain separate model checkers at first if forcing them into one proof kernel would obscure rather than reduce the trusted base.

Proof-kernel non-responsibilities

Do not put these in the proof kernel:

- arbitrary Clojure evaluation;
- DOM behavior;
- HTMX implementation;
- network scheduling;
- XTDB internals;
- arbitrary arithmetic theorem proving;
- general higher-order logic;
- arbitrary user predicates;
- automatic proof search over an unbounded domain;
- full dependent typing.

Existing verifier code should progressively move from:

```
{:valid? true}
```

toward producing explicit evidence.

For example:

```
static goal:
  after admitted observation O of Request R at program point q,
  browser knows O's request.status value for R

required premises:
  trusted acquisition contract produces an observation of R
  O's scope includes R and its status field
  projection includes that field under the relevant branch
  browser receives and admits O through the declared event boundary
  provenance and any required basis/compatibility obligations hold

checked conclusion:
  Knows(browser, status-of(R, O), after(q))

not concluded:
  the browser has already received a future observation
  every Request or the whole board is current at O's basis
  observation alone authorizes a later mutation
```

The exact proof-term representation should remain ordinary immutable data.

The diagnostic renderer and external formalization bridge should be able to consume the same proof vocabulary. The static derivation establishes a conditional transition property. Concrete observation admission remains a runtime boundary under #req-authoritative-observation; a proof sidecar cannot manufacture O.

Obligation coverage and executable correspondence

The accepting checker must validate a complete judgment for its artifact class, not merely accept a bag of individually valid derivations. It must bind every exported conclusion to the normalized subject and verify the required coverage: for example, every relevant rule term, reachable transition, branch, communication contract, and terminal/resource case in the claimed subset. Unreachable-case exclusion itself needs the justification required by that subset. Removing a troublesome branch from the proposed obligation list cannot make the original choreography certifiable.

A checker can accomplish this by deriving required obligations itself, checking a syntax-directed derivation covering the subject, or another explicit complete coverage mechanism. The mechanism and any trusted generator remain named in the assurance report; this document does not require one implementation.

There are distinct correspondence boundaries from author syntax to normalized subject, from that subject to certified projection, and from projection to the actual executable plan consumed by production. An accepted proof about a silently changed subject is not proof about the author's original declaration. Tests provide evidence for these bindings; only a stated checked preservation argument permits a stronger correspondence claim.

The logical trusted base includes inference-rule semantics, checker code, primitive semantics, and the representation/parser assumptions used in checking. A small proof kernel reduces that surface; it does not prove its own implementation sound merely by being small. Dedicated checkers add their own stated trusted surface or supply evidence accepted by a smaller checker.

Checking failure, resource exhaustion, timeout, missing evidence, and unsupported features yield distinct non-admitted results where relevant. They never mean that an obligation was proved false, nor that it passed. Resource limits must not turn incomplete search or exploration into successful certification.

Proof-kernel/elaborator separation is a fault-containment boundary

The elaborator may contain bugs.

The architecture is useful only if an elaborator bug that proposes an invalid derivation can be rejected by the smaller checker.

Therefore mutation tests should deliberately construct:

- missing premises;
- wrong dependency references;
- invalid type conclusions;
- unjustified provenance strengthening;
- bogus authority entailment;
- malformed refinement derivations;
- fake rule tags;
- stale dependency identities;
- omitted obligations for a retained branch or term;
- a valid derivation for the wrong subject or program point;
- an unjustified assumption discharge or changed scoped proposition;
- a semantically different normalized term paired with the original declaration;
- an executable plan altered after valid projection certification.

The appropriate checker or correspondence boundary must reject each defect; a proof kernel is not credited with checking an external binding it does not inspect.

The proof kernel accepts or rejects the proof term independently of the elaborator's claim that the term is valid.

A deliberately buggy elaborator makes that fault-containment boundary concrete.

Suppose a HumanHelp helper projection contains this branch:

```
(branch
  (= request.private-review :approved)

  (then
    (enable :request/claim))

  (else
    (disable :request/claim)))
```

The normalized projection state says the helper knows only:

This is a static program-point environment. A derivation about a future authoritative observation is conditional on an event admitted by the runtime; the proof kernel cannot assert that any particular browser has already received such an observation.

```
{:role :helper

 :program-point :request-card/claim-branch

 :knowledge
 #{[:request/id R]
   [:request/status R :open]}

 :provenance
 {[:request/id R]
  :authoritative

  [:request/status R :open]
  :authoritative}}
```

There is no premise establishing `request.private-review`.

A buggy elaborator might nevertheless generate:

```
{:goal
 [:knows
  :helper
  [:request/private-review R]
  :request-card/claim-branch]

 :proof
 [:knowledge-from-global-syntax
  {:role :helper
   :field :request/private-review
   :reason
   "field appears in choreography branch"}]

 :elaborator/status :valid}
```

The kernel does not recognize `:knowledge-from-global-syntax` as an inference rule:

```
(proof.kernel/check
  kernel-context
  buggy-proof)
;; ⇒
{:status :rejected

 :error/type
 :proof/unknown-inference-rule

 :rule
 :knowledge-from-global-syntax

 :goal
 [:knows
  :helper
  [:request/private-review R]
  :request-card/claim-branch]}
```

A subtler bug uses a real rule with a missing premise:

```
(def forged-observation-proof
  {:goal
   [:knows
    :helper
    [:request/private-review R]
    :request-card/claim-branch]

   :proof
   [:knowledge-from-authoritative-observation

    ;; Required premise 1:
    [:observation/scope-covers
     :request-card-observation
     [:request/private-review R]]

    ;; Required premise 2 should establish that the helper admitted
    ;; a matching observation in the conditional receive transition.
    ;; The elaborator accidentally omits it.
    ]})
```

The kernel can reject the malformed application of the inference rule:

An admitted receive **contract** suffices for the static conditional claim. A concrete observation still requires runtime scope, owner, freshness, and resource-instance checks before the browser may adopt it. Supplying an asserted client observation as this premise

is invalid.

```
(proof.kernel/check
  kernel-context
  forged-observation-proof)
;; ⇒
{:status :rejected

 :error/type
 :proof/missing-premise

 :rule
 :knowledge-from-authoritative-observation

 :required-premise
 [:observation/admitted-by
  :helper
  :request-card-observation]

 :goal
 [:knows
  :helper
  [:request/private-review R]
  :request-card/claim-branch]}
```

Now consider a different elaborator bug: instead of constructing an invalid derivation, it quietly omits the troublesome branch from the obligation list.

The normalized choreography still contains two reachable branches:

```
(def normalized-claim-choreo
  {:states
   {:card
    {:branches
     [{:branch/id :ordinary
       :guard
       [:= request.private-review :approved]
       :next :claim-enabled}

      {:branch/id :not-approved
       :guard
       [:not
        [:= request.private-review :approved]]
       :next :claim-disabled}]}}})
```

The buggy elaborator proposes coverage only for `:ordinary`:

```
(def proposed
  {:obligations
   [{:obligation/id
     :branch/ordinary-discriminator-known

     :subject
     [:state :card
      :branch :ordinary]

     :goal
     [:knows
      :helper
      [:request/private-review R]
      :card]}]

   :derivations
   [...]})
```

If the checker accepted only the volunteered list, removing an obligation could make certification easier.

Instead the artifact-class checker derives required coverage from the normalized subject:

```
(def required
  (choreo-checker/required-obligations
    {:subject normalized-claim-choreo
     :profile :gesso/choreo-v4}))
;; ⇒
[{:obligation/id
  :branch/ordinary-discriminator-known
  :subject
  [:state :card
   :branch :ordinary]
  ...}

 {:obligation/id
  :branch/not-approved-discriminator-known
  :subject
  [:state :card
   :branch :not-approved]
  ...}

 {:obligation/id
  :state/card-continuations-typed
  ...}

 {:obligation/id
  :state/card-terminal-resource-accounting
  ...}]
```

Coverage checking fails before any certificate is issued:

```
(obligation/check-coverage
  {:required required
   :proposed
   (:obligations proposed)})
;; ⇒
{:status :rejected

 :error/type
 :certification/unresolved-obligation

 :missing
 [{:obligation/id
   :branch/not-approved-discriminator-known}

  {:obligation/id
   :state/card-continuations-typed}

  {:obligation/id
   :state/card-terminal-resource-accounting}]}
```

There is a third correspondence failure the proof kernel alone cannot catch. Suppose a valid certificate was produced for normalized subject S1, then a buggy lowering stage emits an executable plan from different subject S2 while copying S1's certificate identity:

```
(def certified-projection
  {:subject/id :projection-S1
   :certificate :cert-P1})

(def emitted-plan
  {:plan/id :plan-request-claim
   :claimed-source :projection-S1

   ;; Bug: executable branch actually came from S2.
   :states
   (lower
     projection-S2)})
```

The derivation in `:cert-P1` may be perfectly valid for S1. The relevant projection-to-plan correspondence checker must reject the mismatch. It reconstructs the canonical executable payload from the admitted S1 subject (or checks an equivalent independent preservation derivation) and compares it to the **actual** emitted payload. Reading `:claimed-source` on its own would accept the forged plan. If the lowering checker and emitter share a bug or the served bytes differ from the checked payload, those are separate named implementation and artifact correspondence premises, not proof-kernel conclusions:

```
(plan-checker/check-correspondence
  {:certified-projection
   certified-projection

   :plan
   emitted-plan})
;; ⇒
{:status :rejected

 :error/type
 :certification/executable-correspondence-failed

 :certified-subject
 :projection-S1

 :reason
 :emitted-payload-differs-from-checked-lowering}
```

This is why the proof kernel is deliberately **small**, not omnipotent. It catches invalid derivations under its logic. Obligation coverage catches omitted judgments. Correspondence checkers bind proved subjects to author syntax, projection, and executable artifacts.

Together they make a large elaborator/compiler less dangerous without pretending that one tiny kernel checks every implementation boundary in the system.

Dedicated semantic checkers may coexist with the proof kernel

Not every property must immediately be forced into one universal logic.

A small exact transition-system checker may remain the right tool for:

- bounded fragment coordination;
- adapter ownership invariants;
- particular finite-state properties.

Such a result can still participate in the root assurance report with an honest classification. A finite checker must record the modeled state dimensions, initial states, transitions, property, and exploration scope. One active request and one queued slot do not make retained metadata or physical retired requests finite. A claim about the concrete coordinator requires the relevant sound abstraction/refinement relation to #req-fragment-coordinator.

The report distinguishes completed exact exploration, a discovered counterexample, and incomplete exploration. It cannot equate running every currently generated trace with exploring every behavior of the claimed model.

The goal is a smaller trusted story, not maximal unification for its own sake.

Malformedness boundary and policy correctness

The proof architecture deliberately distinguishes malformed semantic expressions from coherent but product-wrong policies.

Every negative downstream test should classify the defect.

Categories include:

- `:syntax`;
- `:unknown-semantic-reference`;
- `:type-mismatch`;
- `:invalid-cardinality`;
- `:missing-refinement`;
- `:closed-world-violation`;
- `:knowledge-not-established`;
- `:provenance-not-established`;
- `:authority-not-established`;
- `:capability-not-established`;
- `:relationship-mismatch`;
- `:operation-contract-mismatch`;
- `:continuation-mismatch`;
- `:message-contract-mismatch`;
- `:projection-obligation-failed`;
- `:resource-obligation-failed`;
- `:semantic-dependency-mismatch`;
- `:stale-certificate`;
- `:unsupported-certificate-version`;
- `:proof-invalid`;
- `:unresolved-obligation`.

A stable taxonomy is useful both for developer diagnostics and for proof/mutation coverage accounting.

This architecture must preserve this boundary rigorously.

Accepted but potentially wrong business policy:

```
(rule adult?
  (> user.age 18))
```

where `user.age` is an integer.

Gesso cannot know whether the business intended “older than 18” or “at least 18.” The product owns that choice.

Rejected malformed rule:

```
(rule adult?
  (> user.email 18))
```

where `user.email` is an email/string-like nominal type.

Rejected valid rule at an invalid choreography point:

```
(rule approved?
  (= request.private-review :approved))
```

if the browser tries to branch on `approved?` before it has justified knowledge of `request.private-review`.

The proof system checks coherence and admissibility, not product intent.

Distributed admissibility remains a second check

A rule may be intrinsically well typed yet unusable by a role at a particular point.

Therefore the full judgment is conceptually:

```
rule intrinsically valid?
        |
        v
required facts known here?
        |
required provenance available?
        |
required authority/trust justified?
        |
required optional refinements established?
        |
required authoritative basis/progression available?
        |
        v
rule admissible at this distributed program point
```

This separation prevents ordinary type correctness from being mistaken for distributed knowledge correctness.

Diagnostics as proof and repair output

A rejected composition is only useful if the system preserves enough semantic context to explain the first failed obligation.

A failed proof composition should explain why the proof cannot be constructed.

Example:

```
Gesso proof error: :type-mismatch

Rule:
  humanhelp.request/claimable?

Expression:
  (< request.status 10)

Operator:
  <

Required:
  Ordered<A> x Ordered<A> -> Boolean

Resolved operands:
  request.status : RequestStatus
  10             : Integer

RequestStatus has no declared ordering.

Primary source:
  request/domain/rules.clj:...

Proof chain stopped at:
  VerifiedRequestRules

Downstream artifacts not constructed:
  VerifiedRequestOperationRegistry
  VerifiedRequestChoreography
  VerifiedRequestProjection
  ExecutablePlan
```

Another:

```
Gesso proof error: :semantic-dependency-mismatch

request.rules was certified against:
  request.schema semantic-id S17

current request.schema:
  semantic-id S18

Relevant semantic change:
  RequestStatus removed :claimed
  RequestStatus added :assigned

No stale request.rules certificate may be composed.
```

Another:

```
Gesso proof error: :knowledge-not-established

Role:
  :helper-browser

Behavior diverges on:
  request.private-review

Rule itself:
  VERIFIED

Use at this choreography state:
  REJECTED

Available provenance:
  none

Required:
  knowledge of this field for this subject at this program point,
  with the provenance required by the branch's contract

Candidate establishment path (subject to authorization and scope):
  authoritative request projection at state ...
```

Systematic downstream mutation corpus

The mutation corpus is the adversarial executable specification of what malformed downstream construction must not cross a certified boundary.

This should become a major acceptance suite.

Begin with several known-good certified programs.

Generate or hand-author semantic mutations one at a time and require classified rejection at the first applicable boundary. A structurally valid browser request containing an untrusted principal assertion may require runtime rejection rather than construction-time rejection; acceptance names the actual enforcing boundary.

Each case records its valid baseline, the isolated change, affected proposition, expected result, first enforcing boundary, stable diagnostic category, and whether the prohibited artifact/effect was prevented. Re-run the positive baseline through that same path: a checker that rejects every program does not satisfy the acceptance contract.

Schema/domain mutations

- remove a referenced field;
- change a field's nominal type;
- change required to optional;
- change optional to required;
- remove an enum variant;
- add a closed-world variant where exhaustiveness matters;
- change relationship cardinality;
- change relationship endpoint type.

Rule mutations

- compare incompatible types;
- order an unordered type;
- access optional value without refinement;
- use unknown field;
- use unknown variant;
- omit a match branch;
- use a relationship predicate on the wrong model;
- reference an undeclared operation output.

Knowledge/provenance mutations

- remove communication;
- remove authoritative observation;
- give entry knowledge to the wrong role;
- use optional communicated data as definite knowledge;
- turn authoritative provenance into asserted provenance where stronger provenance is required;
- derive from an input the actor does not know;
- branch before discriminator knowledge exists.

Authority/security mutations

- browser supplies role;
- browser supplies principal;
- browser supplies authoritative provenance;
- browser supplies capability;
- browser requests undeclared operation;
- browser asks for protected observation/subscription without server authorization;
- operation contract requires authority absent from the actor projection.

Choreography/projection mutations

- make remote behavior depend on an unobservable branch;
- remove required correlation;
- overlap receive contracts;
- change continuation type;
- omit terminal path;
- leak resource on one branch;
- reuse a retired execution or resource-instance identity to claim current ownership;
- pair a projection with the wrong verified choreography identity.

Certificate mutations

- alter semantic-id;
- alter dependency id;
- alter guarantee without changing derivation;
- drop a premise;
- inject a fake proof rule;
- change checker version;
- compose certificate against stale dependency;
- create dependency cycle;
- replace certified value with same-shaped raw map;
- omit a required obligation while retaining its subject transition;
- keep a guarantee but alter its assumption scope;
- substitute an implementation or served artifact while retaining its old report;
- return a partial exploration result as a successful exact check.

The expected classification is specified before executing the mutant. A changed schema need not be intrinsically malformed: the required rejection may be the old dependent certificate composing against it. An unchanged-semantic edit or a coherent product-wrong rule can be an intentional accepted control. Mutation acceptance must not retroactively relabel an unexpected checker miss as an application-truth exception.

Every completed mutant case must be either:

```
REJECTED
with stable reason and proof location
```

or deliberately recorded as:

```
ACCEPTED BY DESIGN
property belongs to application truth or an explicitly trusted boundary
```

There should be no ambiguous ”probably okay” bucket. Infrastructure failure, timeout, or an unsupported test mechanism is an incomplete test, not either successful outcome above. An expected unsupported-feature rejection may pass its explicit negative case; it does not demonstrate support for that feature. A score over mutants is evidence about that corpus, not a soundness theorem.

The corpus can be organized as data so every mutant records the baseline, isolated change, expected boundary, and diagnostic before the test runs.

Ordinary source mutants start from fresh **unadmitted** production declarations and go through the same constructor/checkers as the positive control. A certificate-forgery mutant instead changes serialized candidate bytes at the deserialization boundary. The test fixture exposes these distinct injection points; it never assumes a checker-issued opaque value is a writable map.

For example:

```
(def certification-mutants
  [{:mutant/id
    :request-schema/remove-status

    :baseline
    :humanhelp/request-claim

    :mutate
    (fn [program]
      (update-in program
        [:request-schema
         :declaration :types
         :humanhelp/Request
         :fields]
        dissoc
        :request/status))

    :affected
    [:rule/reference
     :request/claimable?
     :request/status]

    :expect
    {:result :rejected
     :boundary :rule-set/check
     :error/type
     :certification/unknown-field}}

   {:mutant/id
    :organization/stale-domain-certificate

    :baseline
    :humanhelp/request-claim

    :exercise :certificate/compose

    :mutate
    (fn [program]
      (→ program
          (assoc-in [:organization-domain :declaration]
                    (fixtures/organization-policy-source
                      :claim-policy-enum))
          (assoc-in [:certificate-cache :claim-rules]
                    (fixtures/serialized-certificate cached-C21))))

    :affected
    [:dependency
     :claim-rules
     :organization-domain]

    :expect
    {:result :rejected
     :boundary :certificate/compose
     :error/type
     :certification/semantic-dependency-mismatch}}

   {:mutant/id
    :choreo/remove-helper-knowledge
```

```
   :baseline
   :humanhelp/request-claim

   :mutate
   (fn [program]
     (update-in program
       [:claim-choreo
        :entry
        :helper
        :knowledge]
       disj
       :request/id))

   :affected
   [:projection
    :helper
    :request/id]

   :expect
   {:result :rejected
    :boundary :choreo/check
    :error/type
    :certification/knowledge-not-established}}

  {:mutant/id
   :certificate/forge-valid-map

   :baseline
   :humanhelp/request-claim

   :exercise :certificate/deserialization

   :mutate
   (fn [program]
     (assoc-in program
       [:certificate-cache :claim-rules]
       {:certificate/kind :rule-set
        :semantic-id
        :semantic/claim-rules-C21
        :valid? true
        :guarantees
        #{[:rule/well-typed
           :request/claimable?]}}))

   :affected
   [:admission
    :VerifiedRuleSet]

   :expect
   {:result :rejected
    :boundary :certificate/deserialization
    :error/type
    :certification/unadmitted-dependency}}

  {:mutant/id
   :projection/wrong-choreo-identity

   :baseline
   :humanhelp/request-claim
```

```
 :exercise :projection/compose

 :mutate
 (fn [program]
   (assoc-in program
     [:certificate-cache :helper-projection]
     (fixtures/serialized-certificate
       verified-create-helper-projection)))

 :affected
 [:projection
  :request-claim/helper]

 :expect
 {:result :rejected
  :boundary :projection/compose
  :error/type
  :certification/semantic-dependency-mismatch}}

{:mutant/id
 :assembly/remove-transport-edge

 :baseline
 :humanhelp/request-claim

 :mutate
 (fn [program]
   (update-in program
     [:application-assembly
      :routes]
     (fn [routes]
       (vec
         (remove
           #(= :route/request-claim (:route/id %))
           routes)))))

 :affected
 [:application
  :request/claim
  :transport]

 :expect
 {:result :rejected
  :boundary :application/preflight
  :error/type
  :application/missing-transport}}

{:mutant/id
 :product-policy/change-manager-limit

 :baseline
 :expense/approval

 :mutate
 (fn [program]
   (assoc-in program
     [:policy :manager-limit]
     5000M))
```

```
   :affected
   [:application-policy
    :expense/manager-limit]

   :expect
   {:result :accepted
    :reason
    :coherent-product-policy-change}

   :classification :application-truth}])
```

The runner checks a fresh positive baseline for **every** case before applying its isolated mutation. The fixture contains source declarations built by the production constructors plus any explicitly selected cached candidate bytes. An `:exercise` selector invokes the named **supported** boundary with the fixture inputs; omitted selectors run the whole production construction. Thus the stale-certificate case tests the rejected composition attempt itself. A normal cache may instead miss, re-elaborate against O15, and reject the unchanged old rule as an unknown field. A separate production-path case must show that neither route admits a stale dependent. Deserialization validates cache admission before it can reuse a certificate:

The two `:prohibited-*` fields in the runner come from instrumented fixture constructors and effect sinks, not from a mutable result supplied by the mutant. If that instrumentation is unavailable, the case is incomplete; `nil` cannot silently stand for `false`.

```
(defn run-mutant!
  [{:mutant/keys [id]
    :keys [baseline mutate expect exercise]}]

  (let [program
        (fixtures/fresh-certification-candidate
          baseline)

        baseline-result
        (certification/exercise-supported-boundary
          program (or exercise :production-construction))


        _
        (when-not (= :accepted (:result baseline-result))
          (throw
            (ex-info
              "Positive certification control failed."
              {:mutant/id id
               :baseline baseline
               :result baseline-result})))

        mutated
        (mutate program)

        result
        (certification/exercise-supported-boundary
          mutated (or exercise :production-construction))]

    {:mutant/id id

     :expected expect

     :actual
     (select-keys result
       [:result
        :boundary
        :error/type
        :prohibited-artifact-constructed?
        :prohibited-effect-observed?])

     :pass?
     (and
       (mutation/expected? expect result)
       (or (= :accepted (:result result))
           (and
             (false? (:prohibited-artifact-constructed? result))
             (false? (:prohibited-effect-observed? result)))))}))
```

A test asserts the predeclared classification:

```
(deftest certification-mutation-corpus
  (doseq [mutant
          certification-mutants]

    (testing
      (name
        (:mutant/id mutant))

      (let [report
            (run-mutant!
              mutant)]

        (is
          (:pass? report)
          (pr-str report))))))
```

The same runner executes the positive baseline through the same supported path:

```
(defn verify-positive-control!
  [baseline]

  (let [program
        (fixtures/fresh-certification-candidate
          baseline)

        result
        (certification/run-through-supported-boundaries
          program)]

    (when-not
      (= :accepted
         (:result result))
      (throw
        (ex-info
          "Positive certification control failed."
          {:baseline baseline
           :result result})))

    result))
```

That catches the useless checker that rejects every input.

The runner also distinguishes infrastructure failure from semantic rejection:

```
{:result :incomplete-test

 :mutant/id
 :projection/wrong-choreo-identity

 :reason
 :checker-timeout

 :counted-as
 :neither-accepted-nor-rejected}
```

A timeout cannot be relabeled as ”mutant killed.”

A detailed corpus report can preserve first-enforcing-boundary evidence:

```
{:corpus
 :humanhelp/request-claim-v1

 :baseline
 {:status :accepted}

 :mutants
 [{:mutant/id
   :request-schema/remove-status

   :expected
   {:result :rejected
    :boundary :rule-set/check
    :error/type
    :certification/unknown-field}

   :actual
   {:result :rejected
    :boundary :rule-set/check
    :error/type
    :certification/unknown-field}

   :prohibited-artifact
   :VerifiedClaimRules

   :constructed?
   false}

  {:mutant/id
   :product-policy/change-manager-limit

   :expected
   {:result :accepted
    :reason :coherent-product-policy-change}

   :actual
   {:result :accepted}

   :classification
   :application-truth}]}
```

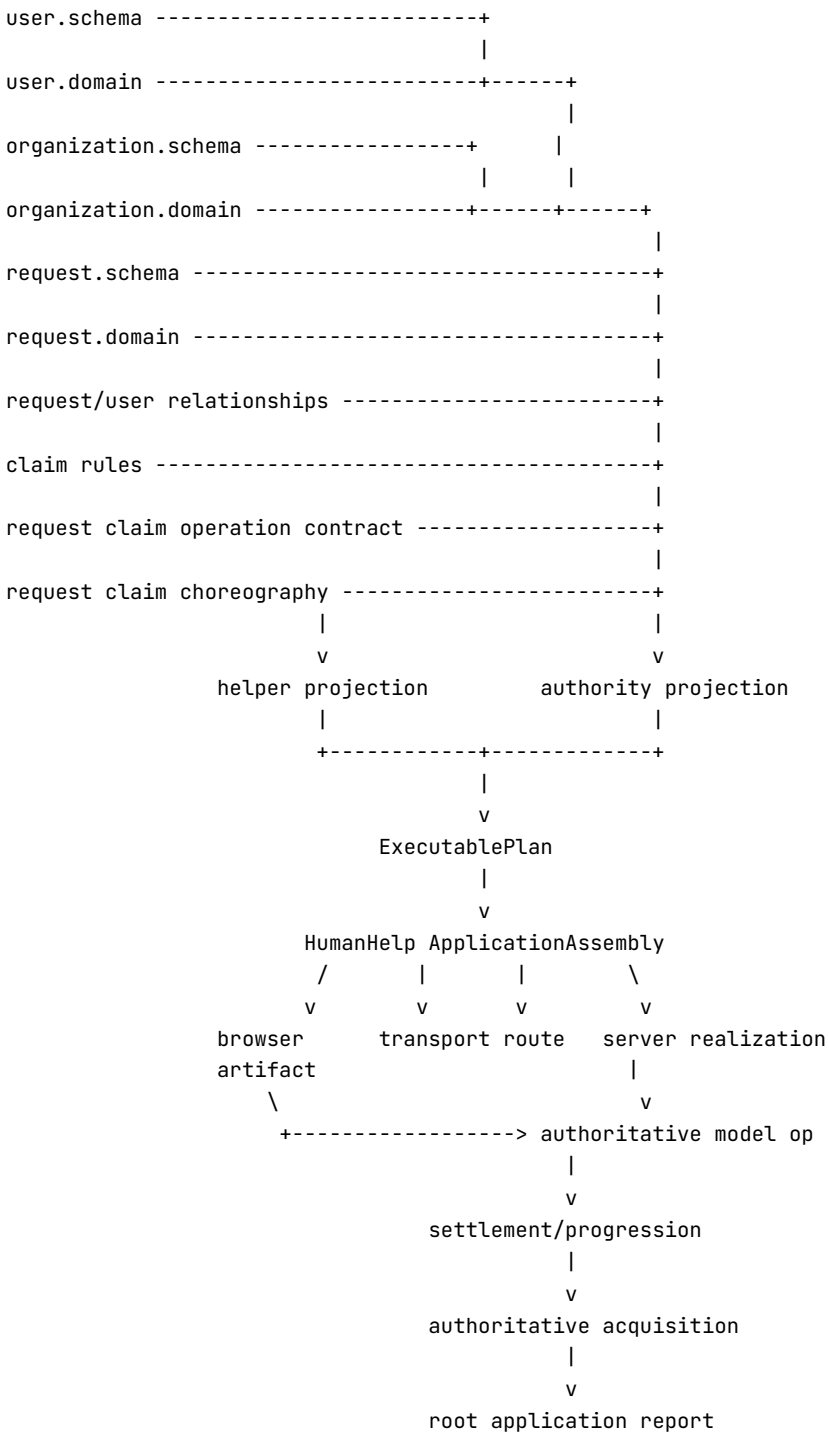
The corpus can therefore demonstrate something more useful than a raw kill percentage: malformed changes are rejected at their intended boundaries, valid controls still construct, and coherent product-policy changes remain outside the framework's attempt to infer human intent.

HumanHelp proof DAG and compositional pilot

HumanHelp Request Claim is the first substantial application proof chain.

The graph should extend through the whole application rather than stopping at the projected runtime artifact.

Conceptually:



The root report does not claim every edge is a theorem checked by one proof kernel.

It records the assurance class of each edge. The diagram mixes semantic dependencies with explanatory execution flow; only certified justification edges form the proof DAG. Repeated runtime acquisition or protocol loops do not create permission for circular certificate justification.

An upstream semantic mismatch must nevertheless prevent downstream construction when the downstream artifact depends on the changed commitment.

HumanHelp Request provides the first real downstream demonstration that framework guarantees compose across application-owned semantic layers.

The pilot is not successful merely because a toy proof term works.

It should cover one meaningful production operation with real cross-model dependencies.

Request Claim is the preferred conceptual target when its production contracts are stable enough.

Pilot dependency chain

The root proof DAG should include, where the operation genuinely depends on them:

- Request schema;
- User/Membership schema and domain commitments;
- Organization schema/domain commitments;
- Request domain refinements;
- relationship/graph facts;
- Claim rule set;
- Request Claim authoritative operation contract;
- Request Claim choreography;
- Helper projection;
- Request-authority projection;
- executable browser/server plan artifacts;
- HumanHelp application assembly realization;
- explicit application/trusted assumptions.

The chain must not invent dependencies solely to make the graph look impressive.

Upstream semantic breakage stops downstream construction

Suppose `organization.domain` changes a semantic guarantee relied upon by Claim rules.

The desired result is:

```
organization.domain semantic-id changes
    |
old claim-rules dependency no longer matches
    |
VerifiedClaimRules cannot be constructed
    |
Claim operation certificate cannot compose
    |
Claim choreography cannot certify
    |
projection cannot certify
    |
ExecutablePlan cannot certify
    |
HumanHelp application root cannot close
```

The system should fail at the first stage that has enough information to know the dependency mismatch.

The browser must never be asked to discover it.

Application truth remains explicit

The Claim pilot will still contain HumanHelp-specific assumptions that Gesso cannot derive from generic semantics.

For example, the application may assert a particular business policy relation.

That assertion is acceptable if:

- named;
- typed;
- attached to its source;
- propagated transitively into dependent certificates;
- visible in the root report.

It is not silently converted into a framework theorem.

Cross-model authority stays public-interface based

If Claim depends on Membership and Organization semantics, the proof DAG consumes their certified public semantic interfaces.

It does not reach into their private FX/query implementation merely because that would make proof extraction easier.

The proof architecture must reinforce, not erode, the model ownership boundary from Part IV.

Pilot must include adversarial mutants

The HumanHelp pilot includes mutants such as:

- remove a required Request status variant;
- change Membership relationship cardinality;
- remove an Organization guarantee required by Claim;
- stale the Claim rule certificate;
- forge a same-shaped operation certificate;
- remove browser knowledge of a branch discriminator;
- give the browser asserted provenance where authoritative provenance is required;
- pair the projection with the wrong choreography identity;
- omit the application transport edge;
- pair the plan with a stale browser artifact.

Each mutant should fail at the earliest appropriate layer with a stable classification.

Pilot root report

A successful pilot emits a report that answers at least:

- what Claim operation is certified;
- which model semantic interfaces it depends on;
- which assumptions remain application/trusted;
- what Choreo properties are proved/checked;
- what projections were derived;
- what executable plan was emitted;
- what whole-application edges are assembly-verified;
- what runtime premises remain;
- what trusted external assumptions remain;
- which normalized subject, checker profile, implementation bindings, and actual admitted assembly generation the evidence covers;
- which required operational claims are separately supported by real-browser and production-enforcement evidence;
- which certification obligations remain unsupported or unresolved.

The pilot must connect to the Claim vertical slice of Part VIII, including its actual controls, production model operation, and authoritative reacquisition. A successful isolated proof fixture cannot replace that application evidence. Conversely, a working Claim browser test does not discharge missing proof obligations. The compositional milestone requires both in their proper scopes.

The goal is not merely ”green.”

The goal is inspectable inheritance of guarantees through a real application.

A successful Claim pilot can make that inheritance inspectable as one root artifact rather than a collection of prose claims.

An illustrative root report might look like:

```
(def humanhelp-claim-root
  {:report/version 1

   :root
   {:application/id
    :humanhelp/example

    :assembly/generation
    :A13

    :operation
    :request/claim

    :certification/profile
    :gesso/compositional-v1}

   :implementation-bindings
   {:request-model
    {:ns 'net.humanhelp.site.model.request.core
     :var 'claim
     :implementation-identity :fixture/request-claim-impl-A13}
    :claim-server
    {:capability :server/request-claim
     :implementation-identity :fixture/claim-server-impl-A13}}

   :browser-artifact-binding
   {:manifest/id :fixture/browser-manifest-A13
    :served-path :fixture/served-gesso-live-A13
    :served-bytes-digest :fixture/browser-bytes-A13
    :emitted-plan-registry :fixture/claim-plan-registry-A13
    :currentness-report :fixture/artifact-currentness-A13}
```

```
  :normalized-subjects
  {:request-schema
   :semantic/request-schema-S17

   :membership-schema
   :semantic/membership-schema-M11

   :organization-schema
   :semantic/organization-schema-O9

   :organization-domain
   :semantic/organization-domain-O14

   :membership-relationships
   :semantic/membership-rel-R8

   :claim-rules
   :semantic/claim-rules-C21

   :claim-operation
   :semantic/request-claim-op-O31

   :claim-choreography
   :semantic/request-claim-choreo-H12

   :helper-projection
   :semantic/request-claim-helper-P7

   :executable-plan
   :semantic/request-claim-plan-E5}

  :certificates
  [{:kind :schema
    :subject
    :semantic/request-schema-S17
    :status :admitted}

   {:kind :schema
    :subject
    :semantic/membership-schema-M11
    :status :admitted}

   {:kind :schema
    :subject
    :semantic/organization-schema-O9
    :status :admitted}

   {:kind :domain
    :subject
    :semantic/organization-domain-O14
    :status :admitted}

   {:kind :relationship
    :subject
    :semantic/membership-rel-R8
    :status :admitted}

   {:kind :rule-set
    :subject
```

```
    :semantic/claim-rules-C21
    :status :admitted}

   {:kind :operation
    :subject
    :semantic/request-claim-op-O31
    :status :admitted}

   {:kind :choreography
    :subject
    :semantic/request-claim-choreo-H12
    :status :admitted}

   {:kind :projection
    :subject
    :semantic/request-claim-helper-P7
    :status :admitted}

   {:kind :executable-plan
    :subject
    :semantic/request-claim-plan-E5
    :status :admitted}]

  :checked-guarantees
  [{:proposition
    [:rule/well-typed
     :request/claimable?]

    :evidence
    {:class :kernel-checked
     :certificate
     :semantic/claim-rules-C21}}

   {:proposition
    [:operation/outcomes-closed
     :request/claim
     #{:accepted
       :rejected}]

    :evidence
    {:class :kernel-checked
     :certificate
     :semantic/request-claim-op-O31}}

   {:proposition
    [:delivery-uncertainty/recovery-declared
     :request/claim
     :request/card]

    :evidence
    {:class :kernel-checked
     :certificate
     :semantic/request-claim-choreo-H12}}

   {:proposition
    [:helper/branch-knowledge-sufficient
     :request/claim]

    :evidence
```

```
    {:class :kernel-checked
     :certificate
     :semantic/request-claim-choreo-H12}}

   {:proposition
    [:projection/preserves
     :request-claim/helper
     :supported-observables-v1]

    :evidence
    {:class :kernel-checked
     :certificate
     :semantic/request-claim-helper-P7}}

   {:proposition
    [:lowering/preserves
     :request-claim/helper
     :gesso-plan-v3]

    :evidence
    {:class :kernel-checked
     :certificate
     :semantic/request-claim-plan-E5}}]

  :application-realization
  [{:proposition
    [:view/exposes
     :request/card.claim
     :request/claim]

    :evidence
    {:class :assembly-verified
     :assembly/generation :A13}}

   {:proposition
    [:browser-artifact/contains
     :semantic/request-claim-plan-E5]

    :evidence
    {:class :assembly-verified
     :currentness-report
     :fixture/artifact-currentness-A13}}

   {:proposition
    [:transport/routes
     :request/claim
     :route/request-claim]

    :evidence
    {:class :assembly-verified
     :assembly/generation :A13}}

   {:proposition
    [:server/binds-model-op
     :server/request-claim
     :request/claim]

    :evidence
    {:class :assembly-verified
```

```
     :assembly/generation :A13}}

   {:proposition
    [:settlement/permits
     :request/card
     :authoritative-reacquisition]

    :evidence
    {:class :assembly-verified
     :assembly/generation :A13}}]

  :runtime-premises
  [{:proposition
    [:invocation/has-current
     :authenticated-principal]

    :evidence
    {:class :runtime-enforced
     :boundary
     :server/request-claim}}

   {:proposition
    [:invocation/has-current
     :membership]

    :evidence
    {:class :runtime-enforced
     :boundary
     :server/request-claim}}

   {:proposition
    [:invocation/has-current
     :request]

    :evidence
    {:class :runtime-enforced
     :boundary
     :server/request-claim}}

   {:proposition
    [:invocation/has-current
     :organization]

    :evidence
    {:class :runtime-enforced
     :boundary
     :server/request-claim}}]

  :trusted-assumptions
  [{:assumption/id
    :humanhelp/request-claim-implementation-conforms

    :class
    :trusted-model-contract

    :binding
    {:ns
     'net.humanhelp.site.model.request.core
```

```
    :var
    'claim}}

  {:assumption/id
   :humanhelp/organization-read-contract

   :class
   :trusted-model-contract}

  {:assumption/id
   :xtdb/transaction-semantics

   :class
   :runtime-primitive}

  {:assumption/id
   :browser/htmx-lifecycle-contract

   :class
   :runtime-primitive}]

 :operational-evidence
 [{:proposition
   [:production-startup/enforces
    :application-preflight]

   :evidence
   {:class :runtime-enforced
    :boundary :humanhelp/production-startup
    :supporting-test :humanhelp/preflight-startup}}

  {:proposition
   [:real-chromium/two-client-claim
    :exercised]

   :evidence
   {:class :tested
    :suite
    :humanhelp/chromium-multiclient}}

  {:proposition
   [:creator-local-vs-remote-advisory
    :exercised]

   :evidence
   {:class :tested
    :suite
    :humanhelp/chromium-selective-adoption}}]

 :unresolved
 []

 :status
 :accepted})
```

A report with `:status :accepted` is meaningful only because the requested compositional profile defines which certificate classes and obligations must be closed. It does not mean every trusted assumption has been turned into a theorem. The symbolic fixture identities above represent separately checked bindings; an actual root report must name the served asset path and digest **and** the manifest and emitted registry established from those served bytes under Part VII's currentness check. A copied digest field

alone is insufficient. Binding the model and server implementation identities also prevents reusing evidence from A13 after their implementations change without changing semantic source.

The report also makes partial progress explicit. For example, before checked operation-implementation conformance exists:

```
{:proposition
 [:implementation/conforms
  :request/claim
  :semantic/request-claim-op-031]

 :evidence
 {:class :trusted-model-contract
  :binding
  {:ns
   'net.humanhelp.site.model.request.core
   :var
   'claim}}}
```

Later work may replace that trusted premise with stronger evidence without changing the semantic operation itself.

Likewise, a future external formalization may establish a theorem about the canonical proof rules:

```
{:proposition
 [:proof-kernel/rule-sound
  :knowledge-from-authoritative-observation]

 :evidence
 {:class :externally-theorem-checked
  :formalization :lean/gesso-kernel-v1}}
```

That theorem joins the root assurance story at its actual scope. It does not retroactively prove HTMX, XTDB, the Claim implementation body, or current Alice authorization.

Finally, the report remains connected to Part VIII's real proving surface:

```
{:application-evidence
 {:control
  {:view :request/card
   :slot :claim
   :operation :request/claim}

  :production-model
  {:ns
   'net.humanhelp.site.model.request.core
   :var
   'claim}

  :authoritative-reacquisition
  {:projection :request/card}

  :browser-scenario
  :humanhelp/chromium-two-helper-claim}}
```

An isolated toy proof lacking those application edges cannot substitute for this root. Conversely, a green browser test cannot fill in a missing rule, Choreo, or projection certificate.

The compositional milestone is precisely the point where both stories meet: certified semantic inheritance reaches the real application assembly, while operational evidence remains honestly classified at the boundaries that still depend on runtime enforcement, trusted implementations, and tests.

Developer experience and acceptance examples

The proof architecture succeeds only if ordinary application development remains declaration-first and rejected programs are repairable.

Ordinary downstream development should ideally look like:

```
declare domain/schema/rules/operations/choreo
        |
        v
Gesso elaborates and checks
        |
        +-- malformed -> immediate precise rejection
        |
        +-- coherent -> certified artifacts
                            |
                            v
                       projection
                            |
                            v
                     executable plan
```

The developer should mostly write the program and domain declarations.

Gesso should carry the proof bookkeeping.

Application developers supply domain truth where only the application can know it; they should not need to become proof-assistant specialists to inherit Gesso's generic guarantees.

The same ergonomic rule applies to nominal or opaque semantic categories. Downstream web developers should not have to wrap routine IDs, strings, maps, or domain values merely because Gesso internally distinguishes admitted semantic stages. Stronger wrappers or handles should appear at the comparatively small number of boundaries where confusing same-shaped values would violate a real invariant, and Gesso should construct them automatically wherever possible.

The first implementation tranche should not be considered useful until all of these are possible.

Valid rule accepted

```
(rule claimable?
  (and
   (= request.status :open)
   (has-role? current-user :helper)))
```

assuming the relevant certified domain types.

Malformed rule rejected

```
(rule claimable?
  (< request.status 10))
```

Valid rule rejected at invalid distributed location

A browser cannot branch on a private authoritative fact it does not know.

Upstream schema change invalidates dependent certificate

Rename/remove `:claimed` and prove stale Request rule certificate cannot compose.

Model contract mismatch rejected

A choreography requiring `:claim-confirmed` cannot consume an operation contract that no longer exports that outcome.

Forged certificate rejected

A same-shaped map with copied semantic-id and `:valid? true` cannot enter the certified API.

Proof mutation rejected

Remove one premise from an otherwise valid derivation and require the proof kernel to reject it.

Root proof report generated

One real HumanHelp operation produces a complete assumptions/guarantees/dependency report.

The developer should not manage proof plumbing manually

Ordinary authoring should not require developers to:

- copy semantic IDs between namespaces;
- construct certificate maps;
- manually list transitive dependencies;
- handwrite derivation trees;
- understand proof-kernel-internal tags;
- write Lean.

The implementation should derive those artifacts from source-owned semantic declarations wherever the dependency is mechanically knowable.

Explicit assumptions are still authored

When only the application knows a proposition, the developer may need to name that proposition explicitly.

For example:

```
application assertion:
    HumanHelp product policy treats this relationship as claim-eligible
```

That is legitimate.

The system should make the assertion visible in the root report rather than forcing a fake derivation or an opaque escape hatch.

Compositional certification invariant

At the end of the compositional-proof layer, the unified design requires:

> Gesso constructs downstream semantic artifacts from certified interfaces rather than namespace reputation, copied identifiers, map shape, or `:valid?` flags. Raw declarations and certified values are distinct products. Every certified layer exports a minimal semantic interface consisting only of guarantees that downstream reasoning may rely upon, together with explicit assumptions and machine-checkable dependency identity.
>
> Business-rule expressions use a closed typed semantic language that rejects malformed expressions while still permitting coherent but product-wrong policy. Intrinsic rule validity is not sufficient for distributed use: knowledge, provenance, authority, refinement, and progression obligations must also hold at the program point where the rule is consumed.
>
> The certificate hierarchy follows semantic ownership from schema/domain and relationships through rules, authoritative operation contracts, Choreo, projection, `ExecutablePlan`, and whole-application realization. Semantic fingerprints establish normalized identity, not truth. Stale or incompatible dependency identities prevent downstream certification before browser execution. Compatibility beyond exact identity is admitted only through an explicit checked compatibility relation.
>
> A rich elaborator may generate normalized terms, obligations, and derivations, but a deliberately smaller proof kernel or dedicated exact checker decides the claims it is responsible for. Required obligation coverage and correspondence from declared subject through projection to executable plan are explicit checking or trust boundaries. An omitted obligation is not discharged merely because every submitted derivation passes. Certificates are evidence accepted by those checkers, not booleans, and optional proof/diagnostic artifacts never influence runtime semantic transitions or authorization.
>
> Assumptions remain scoped and transitively attributable to the guarantees that use them. Discharge requires accepted evidence; converting failed checking into an implicit assertion is forbidden. Semantic contract reuse does not preserve evidence about a changed implementation or admitted assembly generation. Finite checking, conditional theorems, runtime enforcement, and tests retain their actual scope; incomplete checking never becomes a successful certificate.
>
> Rejected programs retain stable structured diagnostics describing the failed obligation, dependencies, available versus required type/knowledge/provenance/ authority facts, assumptions, and the downstream artifacts that were not constructed. A systematic mutation corpus demonstrates that malformed schema/rule/knowledge/authority/choreography/certificate/application variants stop at the intended boundary.
>
> HumanHelp Request Claim is the first real compositional pilot. Its assurance graph extends from production model interfaces through Claim rules, operation contract, choreography, projections, executable plan, HumanHelp `ApplicationAssembly`, exact browser artifact, transport, trusted server/model execution, settlement/progression, and authoritative reacquisition. An upstream semantic mismatch prevents construction of the stale downstream application artifact, while application-specific policy assumptions remain visible rather than being mislabeled as framework theorems.

Part X — Formal correspondence and system boundaries

This part specifies the scope of stronger proof claims and their connection to production, together with the trusted boundaries and multi-node contracts that those claims must acknowledge. It consolidates the repeated formalization material into one correspondence contract without treating test coverage as a substitute for a theorem.

Formal strengthening and system boundaries

This part connects the compositional architecture of Part IX to system-wide assurance claims, external formalization, trusted implementation boundaries, and physical multi-node realization. It introduces no second semantic layer. Release/completion gates and the ordered realization roadmap follow in Parts XI and XII. Historical plan ordering does not override the canonical contracts.

Assurance is layered rather than binary

Gesso does not have one global state named `proved`.

Different claims may legitimately have different evidence.

The accepted assurance classes are:

- **machine-proved / externally machine-checked** – a theorem follows from an explicit formal semantics under named assumptions;
- **kernel-checked** – a production proof term or derivation is accepted by the small trusted proof kernel;
- **exact-finite checked** – the stated property was exhaustively checked over a finite semantic state space, with the scope of that claim stated precisely;
- **assembly-verified** – a closed application graph/artifact correspondence check succeeded;
- **runtime-enforced** – a concrete premise is validated or supplied at runtime;
- **trusted** – correctness depends on an external implementation or boundary outside the current proof;
- **asserted** – the application explicitly supplies a domain proposition Gesso cannot derive;
- **tested** – behavioral evidence exists, but the property is not promoted to a stronger assurance class.

The root assurance report may contain all of these at once, with the meanings fixed by Part I. Conditional liveness qualifies a progress claim and its environmental premises; it is not an additional evidence mechanism. A conditional-liveness claim can be theorem-backed, model- checked, or supported by tests, and the report must say which.

The design must not collapse them into one marketing label.

Stronger proof never changes semantic ownership

Formalization may strengthen confidence in:

- type safety;
- knowledge/provenance use;
- projection preservation;
- certificate composition;
- browser-adapter transition invariants;
- progression algebra;
- stale-dependency rejection;
- whole-application correspondence.

It does not move HumanHelp business truth into Gesso.

For example, Gesso may prove that a distributed program invokes the declared `:request/claim` operation only under the modeled knowledge/authority conditions.

It does not prove that HumanHelp product policy chose the correct Claim rule unless that rule itself is separately specified and verified against a stronger external requirement.

The final formal boundary follows the production semantics

The object of formal reasoning must remain the semantic object production uses.

The architecture rejects:

```
formal model A
    |
proof of A
    |
handwritten production semantics B
```

when the intended correctness claim concerns B.

The desired direction is:

```
explicit production executable semantics
        |
        +--> executable use
        |
        +--> model checking
        |
        +--> proof-term generation
        |
        +--> external formal correspondence
```

The same rule applies to projected plans, browser transition semantics, and whole-application assembly.

The executable semantic core realizes production transitions. The proof kernel checks derivations, while dedicated semantic checkers establish their particular judgments through declared mechanisms. These roles may share canonical semantic definitions without being one component or one assurance class. An external proof-assistant kernel is another explicitly named checking boundary, not an alternate application runtime. Correspondence connects the checked subject to the production object; terminology alone does not establish that connection.

System-wide assurance target matrix

The complete architecture aims to establish the following guarantees.

Each guarantee must be accompanied by an honest assurance classification in the implementation/root report.

The section states the desired property.

It does not imply that every property has already reached its strongest target classification.

Choreography language and projection

The supported global choreography language has explicit transition semantics.

The system should establish that:

- a global choreography has a closed, inspectable semantic representation;
- projected machines arise from an accepted global choreography;
- the supported projection algorithm preserves the declared global/local trace or refinement relation;
- projected behavior does not invent undeclared communication;
- projection preserves every knowledge-producing mechanism required by local behavior;
- behavior-sensitive branch divergence is allowed only after the local role has justified knowledge of the required discriminator or an equivalent fact;
- unsupported semantic constructs are rejected or explicitly classified outside the proved subset rather than silently inheriting stronger guarantees.

Target evidence:

- explicit semantics;
- proof-kernel checking or a dedicated exact-model checker for the stated construction judgments;
- external theorem support for stable projection rules where practical.

Knowledge, provenance, and agreement

The system should establish that:

- senders know the values they send;
- consumers know the facts they consume;
- communication establishes knowledge only for the declared payload;
- authoritative observation establishes only the facts justified by the observed authoritative projection and its provenance/basis;
- deterministic derivation establishes only declared outputs from known inputs;
- asserted knowledge remains visibly asserted/trusted;
- weaker provenance cannot silently become stronger provenance;
- browser assertion alone cannot become authoritative provenance;
- agreement claims are justified through declared communication, compatible authoritative observation, checked derivation, or explicit assertion;
- agreement over evolving facts is scoped to a meaningful frontier rather than assumed from co-presence or shape equality;
- privacy/authorization may intentionally produce different valid knowledge projections for different actors.

Target evidence:

- proof-kernel-checkable knowledge/provenance rules for the closed semantic subset;
- runtime enforcement for concrete authoritative reads and authentication;
- application assertions where domain truth is external to Gesso.

Types, schemas, and rule use

The system should establish that:

- message payloads satisfy declared Malli/value schemas;
- machine contexts satisfy state schemas at checked boundaries;
- supported typed rule operators are used only on values in their semantic domains;
- unknown fields/variants are rejected;
- optional values are not dereferenced without a valid refinement;
- required closed matches are exhaustive;
- relationship predicates use valid endpoint/cardinality contracts;
- operation output fields cannot be consumed before they are produced;
- a rule that is intrinsically well typed is still rejected at a distributed program point when required knowledge, provenance, authority, refinement, or progression is absent.

Target evidence:

- runtime schemas;
- typed rule elaboration;
- kernel-checked semantic judgments;
- mutation tests for unsupported/malformed expressions.

Authoritative model transitions

The system should establish that:

- public model operations own authoritative business semantics;
- authoritative transitions advertised as atomic have one atomic commit boundary;
- failures established to precede commit cannot expose half of the promised atomic authoritative transition;
- failures after commit cannot reclassify or implicitly revert that transition;
- lost outcomes after transaction submission preserve participant-relative uncertainty rather than inventing rejection;
- known post-commit publication/settlement failure creates delivery/recovery obligations, not evidence that the mutation failed;
- FX inputs and outputs are explicit and do not rely on hidden context mutation;
- model operation contracts distinguish required capability, authority, typed input/outcome, atomicity, and trusted assumptions;
- Choreo depends on public model contracts rather than private model FX/graph internals.

Target evidence:

- runtime-enforced transaction guards;
- model/property tests;
- certified operation contracts;
- implementation conformance trusted initially where arbitrary Clojure bodies are not formally verified.

Progression and authoritative observation

The system should establish that:

- authoritative progression is distinct from domain revision, valid time, command identity, and browser execution/resource-instance identities;
- progression requirements are canonical;
- composition of progression requirements is conservative;
- commit-derived invalidation and reread preserve sufficient scoped progression requirements when delivered;
- loss between commit and publication follows the declared recovery contract, with the assumptions needed for eventual acquisition made explicit;
- advisory invalidation never fabricates authoritative content or claims that the recipient has already adopted the advertised progression;
- a raw browser-supplied basis is not proof of prior observation; any admitted observation receipt establishes only its explicitly validated issuance and scope contract, not delivery, installation, or human awareness;
- stale basis is exposed to model policy rather than interpreted by Gesso as an automatic conflict;
- convergence may skip intermediate advisory events when a newer authoritative observation satisfies the application's semantics.

Target evidence:

- explicit progression algebra;
- XTDB integration tests;
- exact finite/proof checks for algebraic properties where practical;
- trusted XTDB consistency contract.

Optimism

The system should establish that:

- optimistic state is distinguishable from authoritative state;
- every optimistic projection carries semantic command and authoritative-basis provenance;
- provisional state cannot increase server authority;
- every provisional trajectory has a declared resolution/recovery path;
- direct settlement loss permits later authoritative supersession through the declared recovery path under its availability and scheduling assumptions;
- current-state supersession does not by itself settle command attribution;
- newer authoritative installation revokes stale rollback rights;
- multiple browsers may hold conflicting provisional projections without creating multiple authoritative histories;
- a semantic optimistic operation never silently downgrades to an anonymous ordinary request because basis/command/plan data is absent.

Target evidence:

- portable transition semantics;
- browser adapter invariant checking;
- whole-application rendered-surface enforcement;
- real Chromium adversarial scheduling tests.

Browser transition safety

The production browser semantic adapter should itself be the checked transition system.

The system should establish that:

- every semantically active async effect has one declared owner or explicit shared-ownership class;
- execution-private callbacks are admitted only for the current execution identity and relevant resource-instance ownership state;
- stale or duplicate events cannot resume an illegal machine state;
- stale callbacks cannot acquire validity merely because a logical target still exists;
- authoritative install cannot be undone by an obsolete provisional callback;
- semantic retirement revokes callback authority before best-effort physical cleanup;
- physical cancellation is an optimization rather than the sole safety gate;
- fragment coordination satisfies its stated bound on logical active/queued slots; retained metadata and retired physical requests have separately stated bounds or abstractions;
- shared fragment/subscription events use their declared owner-relative admission path without fabricated command execution identity;
- deferred physical mutation revalidates the relevant effect permission before changing managed DOM, not only when its callback begins;
- continuity may preserve local UI state but cannot alter domain/choreography semantics;
- incompatible stale projected plans retire and use the declared recovery or authoritative reconstruction path, preserving the product's acquisition and adoption policy rather than treating reconnect as automatic permission.

Target evidence:

- exact transition-system/model checking;
- property tests over adversarial event orders;
- JVM/Node/Chromium conformance;
- real browser integration for FFI/HTMX assumptions.

Portable execution and artifact identity

The system should establish that:

- the same portable semantic transition core runs under JVM, Node, and Chromium for the supported subset;
- production and proof-enabled tests execute the same canonical `ExecutablePlan`;
- proof/diagnostic sidecars are runtime-nonsemantic;
- exact sidecar lookup requires exact executable-artifact identity;
- compatibility identity remains distinct from exact artifact identity;
- generated browser artifacts correspond exactly to the current required browser manifest before the application is served.

Target evidence:

- deterministic plan/artifact identities;
- conformance tests;
- build/startup closure checks;
- structural noninterference by runtime API shape, with optional diagnostics isolated from semantic control flow and failures;
- equivalence for the same initial state and admitted event stream, without claiming identical wall-clock scheduling or external resource behavior.

Whole-application closure

The system should establish that every exposed Choreo-managed affordance has a coherent path through:

```
view affordance
    -> semantic operation
    -> verified trajectory
    -> projected browser plan
    -> exact generated artifact
    -> declared transport
    -> trusted route
    -> trusted server operation
    -> authoritative model operation
    -> settlement/progression
    -> authoritative acquisition
    -> renderable authoritative projection
```

The system should reject:

- unknown semantic operations;
- missing browser plans;
- stale generated artifacts;
- missing/incompatible transport;
- missing trusted route/server operation;
- missing required server capability;
- missing acquisition path;
- anonymous ordinary HTMX use of an assembled semantic route;
- implicit semantic downgrade;
- duplicate/incompatible identities.

Target evidence:

- `ApplicationAssembly` closure checking;
- runtime rendered-surface enforcement;
- lifecycle-enforced preflight before serving;
- real-browser acceptance through the same admitted assembly generation.

Closure quantifies over declared affordances. Independent required-capability acceptance must also fail when a promised action is omitted. Production entrypoint enforcement must demonstrate that the checked assembly and artifact are those actually used. These three claims remain separate under #req-application-assurance-claims.

Browser/server authority boundary

Under named authentication/model/XTDB assumptions, arbitrary browser behavior must not grant additional authoritative privilege.

The browser may propose:

- operation;
- arguments;
- command ID;
- observed basis;
- correlation metadata.

The browser may not establish:

- authenticated principal;
- server role entitlement;
- authoritative provenance;
- current model authorization;
- protected read/subscription permission;
- trusted server machine state.

Target evidence:

- trusted-side operation registry;
- authenticated principal binding;
- runtime authorization;
- adversarial browser tests;
- semantic authority-flow theorem for the modeled protocol boundary.

Liveness and convergence

The architecture should support explicit conditional claims such as:

> If relevant authority eventually becomes reachable, the actor continues to execute, the declared acquisition/adoption policy permits the required work, any required user action occurs, the scheduler fairly selects enabled recovery, required messages/reads eventually succeed, and the authoritative world is quiescent long enough, then the specifically named presentation or agreement obligation resolves. Command attribution additionally requires command-specific outcome evidence when current-state observation is insufficient.

The system must not infer unconditional liveness merely because:

- a terminal path exists;
- a timeout transition exists;
- a refresh route exists;
- a message could theoretically arrive.

Every liveness/convergence claim names its environment assumptions and the exact obligation that eventually resolves. A reachable terminal state proves the possibility of progress, not eventual progress on every fair execution. Any argument over all fair executions must justify the fairness and progress steps it uses. Safety must continue to hold when liveness premises fail; remaining unresolved is preferable to inventing an outcome or permission.

Target evidence:

- graph/state-machine analysis;
- conditional theorem statements;
- integration/fault testing for the external assumptions.

Framework theorem families

The following are the principal theorem families over the stable semantic definitions. Their subjects and evidence mechanisms must be stated separately; the executable semantic core is not itself the proof kernel.

Certified composition soundness

If certificate B is accepted using certificate A, every premise B attributes to A must follow from A's exported guarantees under checked scope/substitution and compatibility rules. Additional premises admitted at B remain explicit assumptions of the resulting conclusion, not guarantees attributed to A. Required obligation coverage and subject binding follow Part IX. A conditional composition theorem does not establish its external premises or the conformity of arbitrary implementation bodies.

Business-rule type safety

A certified rule cannot apply a supported operator to values outside the operator's declared semantic domain.

Rule-use knowledge safety

If a certified projected role branches/acts on a rule result, all facts required to evaluate that rule are justified by the role's certified knowledge at that point, modulo named assumptions.

Provenance safety

A stronger provenance class is never derived solely from a weaker class unless a declared inference rule permits it.

In particular, browser assertion alone cannot become authoritative provenance.

Authority derivation and non-escalation safety

No projected/browser-supplied semantic artifact can increase the authority of the authenticated principal without a trusted server/model rule that establishes the capability.

Projection preservation

For the supported Choreo subset, accepted local projections preserve the stated global trace/refinement property.

Executable-plan provenance

Every certified production plan corresponds to one accepted projection under the recorded semantic dependencies and plan-format version.

Stale certificate safety

A certificate whose required semantic dependency identity no longer matches cannot be used to produce a certified downstream artifact.

Progression preservation

The existing progression algebra should eventually participate in the same proof story:

- requirements are canonical;
- composition is conservative;
- success/failure rules preserve unsatisfied requirements appropriately;
- advisory invalidation does not fabricate authoritative progression.

Browser semantic adapter safety

Existing adapter state-machine properties should eventually be connectable to the root proof report while the DOM/HTMX/browser shell remains an explicit trusted/tested realization boundary.

Formal proof strategy and production correspondence

External formalization follows the stable production semantics. The intended progression is explicit semantic data, compositional Clojure certificates, a small proof kernel, proof-producing elaboration, stable proof-kernel semantics, external soundness theorems, and checked production/formal correspondence.

Formalization aims to reduce trust in Gesso's own verifier. It does not replace Clojure, Malli, XTDB, HTMX, or Chromium, and does not move HumanHelp business policy into the framework. This is a target architecture, not a statement that its theorems or correspondence mechanisms already exist.

Worked correspondence slice: HumanHelp Claim has one production meaning

A Claim slice makes the correspondence direction concrete. The formal and production objects may differ in representation, but transferred guarantees must concern both the object production admits and, where execution safety is claimed, the concrete behavior production can actually perform. The example therefore keeps **admission correspondence** and **execution refinement** separate.

The browser command contributes correlation and domain intent. It does not contribute the authenticated helper identity used by the authoritative transition:

```
(def claim-command-shape
  {:operation :request/claim
   :domain-input
   {:request/id :uuid}
   :correlation
   {:command/id :uuid}
   :observed-basis :optional-untrusted-observation-reference})

(def claim-subject
  {:operation :request/claim
   :state-type :humanhelp/Request
   :command-shape claim-command-shape
   :trusted-context
   {:principal :authenticated-principal
    :membership :current-authoritative-membership
    :request :current-authoritative-request
    :organization :current-authoritative-organization}
   :preconditions
   [[:= [:trusted :request :request/status] :open]
    [:= [:trusted :membership :membership/status] :active]
    [:= [:trusted :membership :membership/organization-id]
        [:trusted :request :request/organization-id]]
    ;; Policy is a typed, named HumanHelp model premise. Whether its
    ;; implementation conforms is separate trusted/checked evidence.
    [:organization/allows-claim?
     [:trusted :organization]
     [:trusted :membership]
     [:trusted :request]]
    [:authorized? [:trusted :principal] :request/claim]]
   :transition
   {:request/status :claimed
    :request/helper-id [:trusted :principal :user/id]}
   :commit-contract
   {:guard #{:current-request :current-membership
             :current-organization :bound-command-identity}
    :atomic-writes #{:request-ownership :command-result-record}}
   :outcomes #{:accepted :rejected}
   :delivery-uncertainty
   {:recover {:acquire :request/card
              :command-attribution :unknown}}})

(def claim-normalized
```

```
  (gesso.semantics/normalize-operation claim-subject))

(def claim-formal-subject
  (gesso.formal/translate-operation claim-normalized))

(def claim-production-plan
  (gesso.compile/executable-operation claim-normalized))

;; An instance-level check for this normalized Claim subject. A theorem about
;; every supported operation needs a separately checked quantified proof.
(def claim-admission-correspondence
  (gesso.formal/check-correspondence
   {:formal-subject claim-formal-subject
    :production-subject claim-production-plan
    :proposition
    [:this-normalized-subject
     [:production-admits? :request/claim]
     [:formal-judgment-holds? :claim-admissible]]
    :direction :production-admission-implies-formal-validity
    :representation :gesso.correspondence/operation-v1
    :dependencies
    {:rule-set :semantic/claim-rules-C21
     :operation :semantic/request-claim-op-031
     :kernel :gesso-proof-kernel/K7}}))

;; This is a distinct obligation. An opaque arbitrary Clojure implementation
;; cannot be proved to refine a plan by naming its artifact in a map.
(def claim-execution-obligation
  {:semantic-plan claim-production-plan
   :production-binding
   {:server-capability :server/request-claim
    :model-var 'net.humanhelp.site.model.request.core/claim
    :implementation-identity :fixture/claim-implementation-A44}
   :observables
    #{:authoritative-read
      :authorization-decision
      :durable-commit
      :settlement
      :progression-publication}
   :proposition
   [:every-relevant-production-trace
    [:refines :request/claim-semantic-trace]]
   :dependencies
   {:lowering :gesso.lowering/L12
    :adapter :humanhelp.request.adapter/A9
    :runtime-profile :humanhelp.production/P4}})

(def claim-execution-refinement
  (gesso.formal/check-execution-refinement
    claim-execution-obligation))

;; The selected profile may admit a named trusted implementation premise,
;; or require a checked refinement. An unsupported/unchecked result cannot
;; satisfy the latter. Whole-application admission still follows Part VII.
(def claim-correspondence-gate
  {:normalized-subject claim-normalized
   :plan claim-production-plan
   :admission-evidence claim-admission-correspondence
   :execution-evidence claim-execution-refinement
```

```
   :selected-profile :gesso/external-formal-v1})

;; Wrong shape: prove F, then independently execute G.
(def formal-F
  {:when :open
   :then {:status :claimed
          :helper :trusted-principal}})

(defn production-G [request browser-command]
  ;; Browser-controlled actor data has slipped into the authoritative write,
  ;; and G also contains behavior not represented by F.
  (if (:request/priority-lock? request)
    request
    (assoc request
           :request/status :claimed
           :request/helper-id (get-in browser-command [:actor :id]))))
```

The admitted shape does not treat a command-shaped map as an admitted semantic object merely because its keys look right. Authentication, current Membership, current Request state, authorization, normalization, and the required correspondence evidence all belong to their declared trusted/admission boundaries.

The `:commit-contract` records an obligation, not evidence that an arbitrary Request implementation actually performs one atomic guarded write. Production Claim must check the command record's operation, Request argument, and bound principal before a permitted replay; guard the current Request, Membership, Organization, and association together with the ownership and command-result write; and retain unknown delivery as participant knowledge. The formal profile cannot report checked execution refinement of those actions without a real implementation binding and a proof or other justified correspondence mechanism. Calling `check-execution-refinement` in this illustrative code does not assert its result is `:checked`. In Part VII, preflight and admission apply to the whole assembled application after the profile's requirements are met.

The final `production-G` shape leaves two independent gaps. First, the formal subject and the production behavior are not connected by a checked execution refinement. Second, browser-controlled actor identity has escaped its proper trust boundary. Shared names, examples, digest metadata, or agreement on an acceptance test do not repair either gap. Where arbitrary Clojure remains behind the public operation contract, its conformance remains an explicit trusted/tested premise until stronger evidence discharges that obligation.

Formal obligations must return to executable production boundaries

A formally valid model is useful only to the extent that its correctness-relevant assumptions are either realized by production or remain visible as explicit premises. Gesso therefore requires both forward semantic correspondence from admitted source to its formal and production subjects, and reverse traceability from formal results and assumptions back to source and production obligations. Reverse traceability is not itself a proof that every formal transition is executable in production or that every production behavior is covered by the formal model.

The forward direction is familiar:

```
Gesso semantic object
        |
        v
normalized semantic IR
        |
        +--> executable production realization
        |
        +--> formal subject / transition system
```

The reverse traceability path is equally required:

```
formal assumption, obligation, or counterexample
        |
        v
correspondence identity
        |
        v
semantic rule / operation / transition
        |
        v
production binding and source location
```

The purpose of the reverse path is not merely nicer error reporting. It keeps a formal claim from floating free of the program whose behavior the claim is meant to constrain.

1. Atomic formal actions create atomic production obligations

   If a formal transition treats several observations and writes as one atomic action, production must justify an equivalent atomic boundary for the claimed property. The implementation need not resemble the formal syntax and need not run on one thread or one host. It must provide the same correctness-relevant semantics.

   For example, an abstract Claim action may say that all of these belong to one atomic authoritative transition:

   ```
   Request is still open
   Membership is still active
   Membership and Request still belong to the same Organization
   command identity is still bound to this operation, principal, and Request
   Request ownership changes
   command result is durably recorded
   ```

   Production may discharge that obligation through an XTDB transaction, a conditional write, another serializable authoritative mechanism, or a separately proved refinement. A sequence of ordinary reads followed by an unconditional write does not discharge the obligation merely because every individual read was correct when performed.

   The same rule applies to other formal assumptions. Examples include:

   - an authenticated principal used by a transition requires a production binding to trusted authentication/session state rather than browser bytes;
   - a current authoritative observation requires a production read/guard whose freshness semantics match the claim;
   - exactly-once incorporation may be realized through idempotent command identity and durable result recording rather than literally once-only network delivery;
   - resource retirement requires the browser realization to reject effects from retired resource instances even if those callbacks physically arrive later;
   - an abstract nondeterministic failure or late delivery must be covered by the concrete adapter/environment behavior admitted by the refinement claim.

   An obligation is discharged only by named evidence appropriate to the selected assurance profile. If Gesso cannot establish the production mechanism, the assumption remains a visible trusted/tested premise. It must not disappear because the generated formal model happened to be safe under that assumption.

   A correspondence record may therefore carry information of this shape:

   ```
   {:obligation/id :request/claim-current-and-atomic
    :formal-subject :formal/request-claim
    :semantic-subject :semantic/request-claim-op-031
    :requires
    #{:current-request
      :current-membership
      :current-organization
      :bound-command-identity
      :atomic-request-and-command-result-write}
    :production-binding
    {:operation :request/claim
     :model-var 'net.humanhelp.site.model.request.core/claim}
    :discharge
    {:mechanism :xtdb/guarded-transaction
   ```

```
  :evidence :correspondence/request-claim-X19}
 :source
 {:ns 'net.humanhelp.site.model.request.core
  :var 'claim}}
```

This is illustrative semantic data, not a promise that source metadata alone proves atomicity. The `:discharge` entry must itself be justified by whatever checked correspondence, constrained lowering, integration evidence, or trusted premise the active profile requires.

2. Generated formal artifacts are derivatives, not a second application

   Where Gesso can derive a formal transition system mechanically from admitted semantic IR, that generated artifact is not an independently authored source of business meaning. It is a derivative representation of the same admitted semantic object.

   The preferred path is therefore:

   ```
   application semantic declarations
           |
           v
   checked semantic IR
           |
           +--> production lowering
           |
           +--> formal/model-checking lowering
   ```

   rather than:

   ```
   application Clojure  ---- manually synchronized ----  handwritten formal spec
   ```

   A supported backend may render the derived transition system into TLA+/TLC, Quint, Recife/TLC, or another formal representation. Backend choice does not change semantic ownership: the admitted Gesso object remains the source of the application meaning being transferred.

   Developers and reviewers may inspect generated formal artifacts. Framework maintainers may of course work directly on backend translations. But changing a generated formal file to make a failed check pass does not repair the application. The repair must occur in the semantic source, production binding, formal lowering, or explicitly named assumption from which that artifact is derived, after which the artifact is regenerated and rechecked.

   This prevents a particularly dangerous false repair:

   ```
   counterexample found
       -> weaken generated invariant
       -> checker becomes green
       -> production remains wrong
   ```

   If the application owner intentionally weakens an application invariant, that is a semantic-policy change and must be visible as such. It is not an implementation fix.

3. Counterexamples must be interpreted back into application semantics

   Raw model-checker traces are valuable evidence but are a poor final diagnostic surface for ordinary Gesso development. Gesso should retain enough provenance through formal lowering to translate a counterexample back into the semantic objects whose modeled behavior permits it. The production obligation ledger then identifies corresponding implementation questions. A trace of the formal subject alone does not establish that arbitrary production code took that trace or identify the production defect that caused it.

   For example, suppose a **candidate** Claim semantic transition checks that a Request was open when observed but omits the required current-open guard from the atomic commit action. A backend could then report an execution equivalent to:

   ```
   A observes Request open
   B observes Request open
   A commits Claim of R
   B also incorporates Claim of R without an intervening release
   the declared claim-from-open-at-commit property is false
   ```

This trace is possible in the defective modeled transition. It would not be a counterexample to a formal action that truly guards `:open` atomically with incorporation. If a purportedly guarded generated model nevertheless produces it, inspect the translation, property statement, bounds, and abstraction before claiming that the production transaction is at fault.

The application-level interpretation should identify at least:

- the violated named invariant or business rule;
- the semantic operation and transition participating in the failure;
- the relevant executions/actors/authorities where meaningful;
- the smallest useful prefix of the counterexample;
- the semantic rule, formal lowering, or stated assumption whose change would exclude this modeled trace, to the extent the evidence identifies it;
- the bound Clojure/Gesso source object or source span that owns that semantic decision;
- the related production obligation and its current evidence class, without claiming the trace has demonstrated an implementation defect;
- any abstraction or trusted premise relevant to interpreting the trace.

A diagnostic may therefore read conceptually:

```
COUNTEREXAMPLE IN THE FORMAL SUBJECT

Declared property:
  :request/claim-requires-open-at-commit

Operation:
  :request/claim

Trace:
  execution A observed Request R open
  execution B observed Request R open
  execution A incorporated Claim
  execution B incorporated Claim without an intervening release

Modeled cause:
  the candidate Claim action tests the earlier observation but does not
  require Request R to remain open at its atomic commit

Semantic repair owner:
  admitted :request/claim transition / current-open requirement

Corresponding production binding:
  net.humanhelp.site.model.request.core/claim

Production obligation to check separately:
  current-open guard, ownership write, and command-result write share the
  required authoritative atomic transition

Production conclusion from this formal trace alone:
  not established
```

The formal result rejects the candidate model's claimed property. Repair the semantic source or a faulty formal lowering as appropriate, regenerate, and recheck it. A production realization might independently have a correct atomic guard or might share the defect; resolving that question requires the applicable production correspondence and implementation evidence. Conversely, a formal model with a correct atomic guard can pass while production uses an unguarded pre-read. In that case the failure is an unresolved or refuted production correspondence obligation, not a counterexample to the guarded formal model.

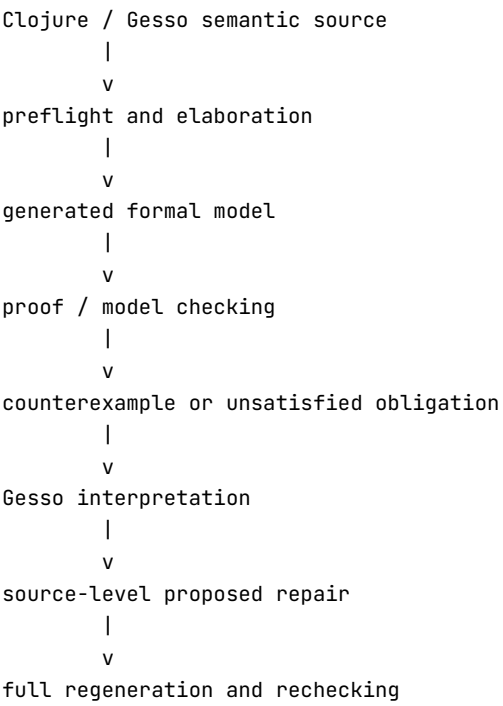
The interpretation must not invent certainty that the formal result did not establish. If an abstract counterexample has not been concretized under the abstraction relation, it is labeled accordingly. Several possible semantic, translation, or implementation repair locations should remain alternatives until evidence distinguishes them.

The same correspondence metadata is useful before failure. A reviewer should be able to ask what a production operation means formally and what formal obligations depend on a source construct without manually comparing two languages.

4. Source-level repair is the supported automation boundary

   This correspondence path is especially important for automated repair. A tool or language model may be very good at ordinary Clojure and application concepts while being much less reliable at constructing or repairing temporal-logic specifications. Gesso should exploit that asymmetry rather than make formal syntax the ordinary repair surface.

   A supported automated loop is:



   The automated assistant may suggest correcting an admitted semantic transition, fixing a formal lowering, changing an authoritative transaction when separate implementation evidence identifies that defect, adding a missing admitted premise, refining choreography, or promoting a confirmed policy into an explicit invariant. It must label which evidence supports each proposed repair. It does not gain authority to edit generated proof/model artifacts as an independent way to obtain a green result.

   This does not make an automated repair trustworthy merely because it was suggested at the source level. The repaired source must pass the same preflight, correspondence, proof/model-checking, concrete test, and admission boundaries as a human-authored change.

5. Acceptance criterion

   For every formal or model-checked claim transferred to production, the assurance report can answer both directions:

   (a) **What production semantic object and executable boundary does this formal subject correspond to?**
   (b) **What production evidence discharges each correctness-relevant assumption used by the formal claim?**

   For a failed formal check, the report can identify the corresponding semantic rules/operations and their production bindings without requiring the developer to reverse-engineer backend syntax. Unsupported or opaque implementation behavior remains explicit rather than being silently covered by the formal claim.

External proof assistant role

Lean is the leading candidate, but the architectural role matters more than the particular proof assistant. Initial formal objects should remain smaller than the full implementation:

- proposition syntax and semantic type judgments;
- knowledge/provenance entailment;
- authority/capability entailment;
- the declared progression requirement algebra;
- certificate dependency composition;
- selected transition invariants and projection properties.

Application developers continue to author Clojure/Clojure data and Gesso's closed semantic languages. Ordinary development does not require Lean, tactic knowledge, manual proof trees, or hand translation of HumanHelp rules into a second language. The framework derives the relevant formal objects where their meaning is mechanically available.

Exact claim and correspondence direction

A theorem about a formal checker is insufficient if production accepts different derivations or executes a different plan. Every transferred claim must identify:

- the formal subject and proposition, including assumptions and supported subset;
- the concrete production object and its immutable identity/version;
- the representation relation between formal and production subjects;
- the checking, translation, or refinement relation used for transfer;
- the direction and scope of that relation;
- the remaining implementation and toolchain trust premises.

For sound admission, a typical required direction is: whenever the production boundary accepts a subject for guarantee G, the corresponding formal judgment holds under the recorded premises. Agreement on rejected examples alone does not establish this direction. Completeness, meaning acceptance of every valid supported subject, is a different property. A sound checker may be conservative; usability and supported-feature acceptance must still prevent “reject all” from being treated as a successful implementation.

For execution safety, a trace/refinement argument must account for every relevant concrete behavior and its observable meaning. Mere acceptance-set agreement does not prove that the runtime executes the certified plan correctly. For liveness, correspondence must additionally address divergence, scheduling, and fairness as needed; a safety simulation that allows arbitrary stuttering cannot alone establish progress.

Correspondence mechanisms and their evidence

Select the smallest mechanism that demonstrably reduces the trusted gap. The following mechanisms have different assurance implications.

Mechanism	What it can establish	Remaining obligation
Externally checked proof terms	The particular admitted proposition follows from accepted premises	Bind proof, subject, dependencies, and actual production artifact; enforce the check
Generated or extracted checker	Implementation originates from the formal definition	Justify translation/extraction and execution toolchain, or identify them as trusted
Mechanically proved comparison/refinement	A stated relation holds for all objects in the proved domain	Identify domain, direction, representation relation, and assumptions
Exhaustive comparison over a finite domain	Acceptance/results agree throughout that specified finite domain	Do not extend to unbounded inputs without a sound coverage argument
Generated conformance vectors	Tested implementations agree on those vectors	Tests alone do not prove correspondence outside the exercised cases

Using every currently generated vector, or covering every inference-rule tag, does not establish coverage of all rule parameters, combinations, recursive proof trees, malformed encodings, or production paths. Such results remain conformance tests unless a separate argument establishes exhaustive coverage for the claimed domain. More than one mechanism may be needed.

A source review saying “the Clojure rule looks like the Lean theorem” is review evidence, not formal correspondence. Similarly, using a formally specified algorithm does not prove a handwritten implementation conforms to it.

Bind formal checking to the admitted production artifact

The correspondence path must bind the normalized subject, complete required obligations, derivation, dependencies, proof-kernel semantics, and result to the object actually admitted. The boundaries identified in #req-proof-kernel-boundary remain relevant: normalization, projection, executable lowering, and runtime realization each need appropriate checked correspondence or named trust.

An independently valid proof about subject A must not authorize construction of subject B. Copying A's identity onto B is insufficient. Build/startup must bind the admitted plan and generated artifact to the admitted assembly generation actually served, following Part VII. Immutable references or revalidation must prevent a later mutable lookup from substituting a different object after checking.

If an external check is a required construction gate, unavailable checking, timeout, unsupported features, or invalid evidence prevent that admission. The system may retain an already admitted valid assembly generation under the declared lifecycle contract; it must not silently waive the requested certification profile. Proof sidecars remain unnecessary for runtime semantic decisions after admission and never become authorization capabilities.

Formal work follows stable semantics

Make the semantics explicit, use them in production, exercise adversarial cases, stabilize the relevant checking rules, and then formalize them. The aim is to reduce a specific trusted surface rather than repeatedly translating unstable representations or encoding the whole implementation wholesale.

The selected formal scope may grow incrementally. Each report names the actual subset proved and the properties still checked, tested, or trusted. A theorem for one primitive cannot silently cover every expression using other primitives.

Finite checking and abstraction

Exact finite exploration establishes a property over its specified finite model. The model must name its initial states, transitions, data domains, property, and exploration completeness. A timeout or partially explored graph is incomplete evidence, not a successful exact check or a counterexample.

Checking resource-instance counter values 0 through 3 establishes only that bounded result unless a sound abstraction argument covers all relevant concrete resource instances, counter scopes, and reuse/retirement behavior. Opaque instance identities may require a different abstraction; no universal counter is assumed. The same applies to command identities, progression tokens, dependency depth, queue metadata, and retired physical effects. Bounded queue slots do not imply a finite overall state space.

An abstraction supporting a concrete safety claim must preserve the relevant initial states and simulate all concrete transitions needed for that claim, with the abstract invariant implying the concrete safety property. Merely mapping each state to some

abstract state is insufficient. A liveness transfer needs the additional progress/fairness/divergence conditions its theorem uses. An abstract counterexample may require concretization before being reported as a realizable production failure.

1. Worked abstraction slice: resource ownership without a fake finite universe

   Browser resource-instance identities may be unbounded even when the safety property depends on only a small relation among them. Checking four chosen UUIDs would therefore prove only an artificial four-value model.

```
(def concrete-state
  {:target :request-list
   :current-resource-instance #uuid "62eae6bf-d8e5-4faa-9c65-5387408ec111"
   :retired-resource-instances
   #{#uuid "e3ff7f3d-a62e-4995-9ef9-314298250812"}
   :pending
   {#uuid "62eae6bf-d8e5-4faa-9c65-5387408ec111"
    {:effect :fragment-fetch
     :phase :awaiting-response}}})

(defn callback-admitted? [state callback]
  (and (= (:resource-instance callback)
          (:current-resource-instance state))
       (not (contains? (:retired-resource-instances state)
                       (:resource-instance callback)))
       (contains? (:pending state)
                  (:resource-instance callback))))

(defn abstract-instance [state instance]
  (cond
    (= instance (:current-resource-instance state)) :instance/current
    (contains? (:retired-resource-instances state) instance) :instance/retired
    :else :instance/other))

(defn abstract-callback [state callback]
  {:instance-class
   (abstract-instance state (:resource-instance callback))
   :pending?
   (contains? (:pending state) (:resource-instance callback))
   :retired?
   (contains? (:retired-resource-instances state)
              (:resource-instance callback))
   :event (:event callback)})

(defn abstract-admitted?
  [{:keys [instance-class pending? retired?]}]
  (and (= instance-class :instance/current)
       pending?
       (not retired?)))

(def resource-ownership-obligation
  {:initial :every-concrete-initial-state-has-related-abstract-state
   :step :every-relevant-concrete-transition-is-simulated
   :fresh-name :allocation-preserves-required-equality-and-inequality
   :retirement :current-identity-cannot-remain-permitted-when-retired
   :metadata :pending-and-retired-sets-are-bounded-or-soundly-abstracted
   :property
   [:implies
    [:abstract-callback-admitted?]
    [:concrete-callback-owned-by-current-resource?]]
   :scope
   #{:start-resource :retire-resource :receive-callback
     :authoritative-install}})
```

If the abstraction relation and simulation obligation are machine-checked, exhaustive exploration of the finite abstract state space can support a claim about every concrete identity covered by that theorem. Without that bridge, the same exploration is merely bounded evidence. Liveness still needs its own scheduling/fairness argument rather than inheriting this safety result. Here `abstract-callback` is only a **local observation abstraction**, not a complete finite transition system. A real proof must specify the abstract state and all transitions, preserve equality and freshness when an arbitrary new identity becomes current, and account for unbounded pending/retired metadata. It must also state the invariant that a retired current instance cannot retain effect permission. This slice checks resource ownership alone; execution identity, effect phase, and deferred DOM mutation require their own admission conditions. None of the symbols in `resource-ownership-obligation` is itself a discharged theorem.

Evidence package and maintenance

A formal claim is reviewable only when its evidence identifies:

- theorem statement and formal semantic definitions;
- proof/checker versions, relevant options, and any admitted axioms;
- normalized subject, dependency closure, and artifact identities;
- representation/correspondence definitions and their evidence class;
- exact finite bounds or abstraction soundness evidence where used;
- the production boundary enforcing admission;
- open assumptions and excluded behavior;
- reproducible checking instructions and the observed result.

A changed semantic rule, primitive, checker, translation, or production binding must trigger the rechecking appropriate to its dependency edges. Unchanged interface-level proofs may remain reusable while implementation correspondence needs renewed evidence. The maintenance contract must not confuse a theorem's continuing mathematical validity with applicability to a changed release.

Trusted computing base, security boundaries, and failure modes

The formal architecture is credible only when the trusted computing base is named and kept small.

Assurance ledger

For each release/milestone, the root assurance report should identify:

- proof-kernel namespaces/size;
- elaborator/compiler namespaces;
- runtime-enforced boundary namespaces;
- trusted external libraries/systems;
- application assertions;
- properties machine-proved;
- properties kernel-checked;
- properties exact-finite checked;
- properties assembly-verified;
- properties runtime-enforced;
- properties supported only by tests;
- exact implementation, dependency, and semantic/checker versions in scope;
- correspondence and admission boundaries that bind the evidence to production;
- conditional-liveness premises and unresolved obligations.

Namespace size is useful audit information, not a proof of soundness. The ledger must distinguish the logical trusted base from the larger runtime, security, and availability dependencies. Each claim identifies the relevant subset; listing every dependency without saying which claim relies on it is insufficient.

A smaller explicitly named TCB is preferable to a much larger system described vaguely as formal.

Typical trusted/external boundaries

The exact list evolves with implementation, but likely boundaries include:

- JVM/Clojure runtime;
- ClojureScript/JavaScript engine;
- browser primitive behavior;
- supported HTMX lifecycle contract;
- Aleph/Ring transport implementation;
- authentication/session implementation;
- XTDB transaction/consistency implementation;
- operating system/network behavior;
- external proof assistant and its proof-checking kernel implementation where used;
- parsing, serialization, normalization, extraction/code generation, and build tooling wherever their correctness is not independently checked;
- primitive operator semantics and cryptographic assumptions used for identity or authenticated evidence.

These are not ignored.

They are named premises.

Real integration and adversarial testing exists partly to challenge them.

Certificate system does not replace runtime authorization

A malicious browser may forge:

- serialized maps;
- DOM;
- JavaScript;
- headers;
- semantic IDs;
- plan-looking objects;
- certificate-looking objects;
- role/principal fields;
- basis/provenance claims.

Therefore:

- server authority comes from trusted server context and current model policy;
- endpoints resolve operations through trusted registries;
- protected reads/subscriptions remain server-authorized;
- cached/persisted certificates re-enter through structural/semantic validation;
- artifact signing may protect integrity but does not prove semantic correctness;
- a structural certificate about a route never substitutes for validating its concrete invocation before protected execution.

The security claim is relative to its specified threat model and boundary. A browser may propose arbitrary bytes; hostile code already holding server-side transaction authority is a different trust condition. Server authorization must hold even if checked rendering is bypassed. Authentication, request-origin/CSRF controls where relevant, input/resource limits, and protected-data disclosure follow the actual transport contract; a non-escalation theorem about the modeled protocol does not automatically prove the security of the entire web stack.

Hashes are identities, not proofs

A checked digest provides correspondence evidence for specified bytes or normalized content under the canonicalization and collision-resistance assumptions. A digest string copied into unverified metadata provides no such binding by itself.

It does not establish the truth of the proposition associated with those bytes.

The architecture always distinguishes:

```
identity evidence
    from
semantic proof evidence
```

A cache lookup makes the distinction operational. This small data example compares **claimed** artifact IDs; it does not inspect served bytes or establish that either ID is genuine. Even when the real Part VII currentness check has established byte identity, old semantic evidence can be stale under a new dependency closure.

```
(def cached-browser-artifact
  {:artifact/id "sha256:91b..."
   :proof-report/id :proof/request-board-R12
   :dependencies
   {:claim-operation :semantic/request-claim-op-031
    :membership :semantic/membership-M18}
   :assembly-generation :assembly/H42})

(def current-application
  {:required-artifact/id "sha256:91b..."
   :dependencies
   {:claim-operation :semantic/request-claim-op-031
    :membership :semantic/membership-M19}
   :assembly-generation :assembly/H43})

(defn claimed-artifact-id-equal? [cached current]
  (= (:artifact/id cached) (:required-artifact/id current)))

(defn claim-compatible-with-current-closure? [cached current]
  (and (claimed-artifact-id-equal? cached current)
       (= (:dependencies cached) (:dependencies current))
       (= (:assembly-generation cached) (:assembly-generation current))))

(claimed-artifact-id-equal?
  cached-browser-artifact current-application) ;⇒ true
(claim-compatible-with-current-closure?
  cached-browser-artifact current-application) ;⇒ false
```

The copied digest claim cannot authorize carrying the old Membership-dependent proposition into the new assembly. To establish actual byte correspondence, the checker must inspect the asset production will serve and relate its digest and emitted plan registry to the current manifest/receipt under Part VII's build and serving contract. Only then is artifact identity evidence available; it still does not prove the proposition associated with those bytes.

Elaborator/checker shared-bug risk

A rich elaborator and a checker implemented from the same misunderstanding may share a bug.

Mitigations include:

- deliberately small proof kernel;
- adversarial malformed derivations;
- independent rule tests;
- external formalization;
- production/formal correspondence.

The purpose of the independent proof-kernel/formal step is specifically to reduce the amount of semantic correctness that depends on trusting the compiler that generated the evidence.

Certificate forgery

Same-shaped map construction must not enter certified APIs.

A public API that accepts:

```
{:semantic-id X
 :valid? true}
```

as equivalent to a checker-issued certified value defeats the architecture.

Certified values use checker-controlled constructors and validated persistence boundaries.

Circular trust

An operation contract cannot prove its own implementation merely by declaring the postcondition it wants.

Likewise:

- a rule cannot certify itself;
- a certificate cannot accept itself as its own dependency;
- a proof of projection cannot assume the preservation theorem it is intended to establish.

Certificate dependencies remain acyclic. Supported recursive semantics use an explicit checked fixed-point/induction rule within a component, with a soundness argument for that rule. They do not permit circular certificate justification. Unsupported cycles reject.

Assertion inflation

`:asserted` is a legitimate proof-DAG leaf.

It is not an escape hatch for every difficult obligation.

Root reports should count and surface assertions prominently so an application whose ”proof” is mostly assertion is visibly different from one with derived guarantees. Counts are diagnostic information, not a confidence score: one central opaque premise may matter more than many minor assertions. Reports must show which guarantees depend on each premise. Required proof-kernel obligations cannot be converted into assertions merely because checking them failed.

Semantic-fingerprint under-approximation

If normalization omits a semantically relevant field, a stale certificate may appear current.

Fingerprint normalization therefore needs adversarial mutation tests.

Semantic-fingerprint over-approximation

If normalization includes irrelevant implementation/source details, harmless edits invalidate too much of the proof DAG and developers are incentivized to bypass certification.

Semantic interfaces should therefore be minimal.

Proof artifacts must remain noninterfering

Proof sidecars, derivation trees, theorem metadata, and diagnostics never become inputs to production transition decisions.

A proof system that changes runtime meaning when enabled has created a second semantic channel.

”Verified” remains qualified

Public reports should prefer specific classifications:

- machine-proved;
- kernel-checked;
- exact-finite checked;
- assembly-verified;
- runtime-enforced;
- trusted;
- asserted;
- tested.

The unqualified word `verified` should not be allowed to hide materially different evidence. Conditional-liveness qualifications remain attached to the relevant claim alongside, rather than in place of, its evidence classification.

Arbitrary Clojure is not the formal target

The strongest proof effort concerns:

- the closed semantic rule language;
- semantic interfaces;
- choreography;
- projection;
- adapter transition semantics;
- certificate composition.

It does not attempt to prove arbitrary host-language behavior by pretending all Clojure is inside the small logic.

Implementation conformance outside the closed semantic subset stays explicitly trusted/tested until separately constrained or verified.

Validation ladder

Stronger formal assurance does not remove the need for concrete testing layers.

Retain:

- unit tests;
- property tests;
- exact finite-state exhaustive checks;
- JVM/Node/Chromium portable conformance;
- real HTMX/SSE/EventSource integration;
- adversarial browser tests;
- XTDB integration tests;
- multi-node tests when multi-node Live exists;
- generated-artifact freshness tests.

Their role becomes clearer:

```
formal proof
  -> semantic theorem

certificate checking
  -> downstream theorem composition

runtime checks
  -> concrete premises

integration/adversarial tests
  -> scoped evidence about shell correspondence and external assumptions
```

Evidence layers are complementary

A useful failure can live outside the theorem boundary.

For example:

- a proof may establish adapter execution/resource-instance isolation;
- a Chromium test may reveal that an HTMX lifecycle event is normalized incorrectly;
- a property test may find an elaborator bug before the proof kernel sees its output;
- an XTDB integration test may reveal an incorrect assumption about snapshot propagation;
- a HumanHelp multi-client test may reveal a missing whole-application edge.

The correction may occur in:

- semantics;
- implementation;
- trusted-assumption definition;
- application assembly;
- application policy.

The framework should not distort the proof boundary merely to make every bug a theorem violation. Acceptance records the first enforcing boundary, the actual admitted assembly generation, and the evidence scope. Expected network/console failures in fault scenarios are asserted precisely; unrelated failures still fail the scenario. A test-only wrapper rejecting before the real boundary is reached is not evidence that production enforces that boundary.

Multi-node realization and authoritative rendezvous

Multi-node deployment realizes the semantic distinctions already established.

It does not introduce a second authority model.

Roles, actors, hosts, and authorities remain distinct

When several Aleph nodes exist:

- a role remains a semantic choreography role;
- an actor remains a role/principal realization;
- a host remains a physical process/node;
- an authority remains the trusted semantic owner of authoritative facts or operations.

A concrete node may temporarily realize an authority projection.

The node itself does not become semantic authority.

For example:

```
role:
    :request-authority

host for invocation:
    web-node A

retry host:
    web-node C

semantic authority:
    [:authority :request R]
```

Host replacement must not change command identity or authoritative meaning. The new host must recover or receive the actual command identity through its trusted protocol; physical relocation does not reconstruct missing intent. Retry safety follows the model's explicit idempotency/deduplication and retention contract. A node-local cache or the same command string alone cannot establish cross-node exactly-once effects. Outcome loss after submission can remain unknown even when another node can read current authoritative state.

Durable model state remains the ordinary rendezvous point

For ordinary HumanHelp cross-user interactions:

```
Alice browser
    |
Aleph node A
    |
Request authoritative operation
    |
XTDB commit
    |
distributed advisory invalidation
    |
Aleph node C
    |
Bob browser receives advisory
    |
product policy or explicit Refresh authorizes acquisition/adoption
    |
authorized authoritative reread
```

Alice and Bob do not require a persistent cross-node participant-message process merely because they interact with the same Request.

The durable Request fact is the rendezvous point.

Distributed invalidation is not participant messaging

A distributed invalidation says approximately:

> Relevant authority may have changed. A future acquisition must satisfy the advertised scope/progression requirement; the application's policy determines when acquisition and adoption are permitted.

A choreography participant message says approximately:

> This execution received this declared message.

A future implementation may transport both through related infrastructure.

Their semantics remain different.

Direct participant routing is opt-in

A genuinely ephemeral protocol may require receipt of a participant message itself to carry semantic meaning.

That use case may introduce:

- distributed actor-interest registration;
- message routing;
- correlation;
- duplicate/loss semantics;
- retry/delivery policy;
- authenticated routing, protected payload disclosure, and recipient admission;
- resource-instance retirement and the applicable compatibility contract across host replacement or a change of admitted assembly generation.

Delivery acknowledgments, broker persistence, execution admission, and domain commit are distinct facts. Any stronger delivery guarantee must name its scope and recovery assumptions rather than inheriting them from the broker's label.

Those mechanisms should be added when a real choreography requires them.

They must not redefine ordinary authoritative convergence.

1. Worked multi-node contrast: durable Request versus ephemeral typing message

   Two cross-user features may traverse the same cluster while requiring different semantics. First, Alice submits a Claim intent through node A. The wire command contains correlation plus the domain Request identity; the authoritative helper identity is rebound from node A's authenticated server context rather than accepted from the browser:

```
(def browser-claim-command
  {:command/id #uuid "76d7d7e1-fd03-47f7-82f7-66915ec283d9"
   :operation :request/claim
   :request/id #uuid "412090ce-0e79-4bc8-9d66-a4690ee52022"})

(defn admit-claim-through-node-a
  [http-request wire-command]
  (let [principal
        (auth/require-authenticated-principal node-a-system http-request)

        request
        (request/read-current-authoritative
         node-a-system
         (:request/id wire-command))

        membership
        (membership/read-current-authoritative
         node-a-system
         {:organization/id (:request/organization-id request)
          :user/id (:user/id principal)})

        organization
        (organization/read-current-authoritative
         node-a-system
         (:request/organization-id request))]

    ;; Admission validates the wire contract and binds these trusted facts;
    ;; execute-admitted-claim! rechecks mutable facts and policy in its
    ;; atomic commit. Declaring a requirement is not proof it was satisfied.
    (choreo/admit-command
     {:operation :request/claim
      :wire-command wire-command
      :trusted
      {:principal principal
       :request request
       :membership membership
```

```
       :organization organization}
      :requirements
      #{:authenticated-principal
        :current-request
        :current-membership
        :current-organization}}))))

(def admitted-claim
  (admit-claim-through-node-a incoming-http-request browser-claim-command))

(def claim-result
  (request/execute-admitted-claim! node-a-system admitted-claim))

;; A rejected result has no committed Request and produces no success advisory.
;; A response lost after submission is uncertain to the browser until it has
;; command-specific evidence; a current-state read alone may not attribute it.
(def committed-request
  (when (= :committed (:outcome claim-result))
    (:authoritative/request claim-result)))

(when committed-request
  (assert (= (:request/helper-id committed-request)
             (:user/id
              (choreo/admitted-trusted-principal admitted-claim)))))

;; This publication attempt may itself fail after a successful commit. It
;; carries board progression information, not protected Request contents.
(def advisory
  (when committed-request
    {:kind :request/authority-may-have-advanced
     :scope
     {:projection :request/board
      :organization-id (:request/organization-id committed-request)
      :coverage :complete-authorized-membership}
     :requires
     (progression/require-board-requirement!
       claim-result)}))

(defn refresh-through-node-c
  [http-request trigger]
  (let [principal
        (auth/require-authenticated-principal node-c-system http-request)

        scope
        (board/require-current-authorized-scope!
         node-c-system principal)

        ;; An admitted advisory may add a progression requirement. Explicit
        ;; Refresh still works when no advisory arrived at Bob's browser.
        requirement
        (advisory/requirement-if-admitted-for-scope
         trigger scope)

        acquisition
        (board/admit-authoritative-acquisition
         node-c-system
         {:principal principal
          :scope scope
          :requirement requirement
```

```
          :trigger trigger})

        observation
        (board/read-authoritative node-c-system acquisition)]

    (if (and requirement
             (not (progression/satisfies?
                   observation requirement)))
      {:status :retryable-insufficient-progression
       :required requirement}
      {:status :acquired
       :observation observation})))
```

If publication is lost before Bob sees it, Bob cannot supply its requirement; his explicit Refresh or another declared recovery trigger must perform a new authorized authoritative acquisition against the durable model. With a previously received advisory, the acquisition must satisfy its scoped progression requirement; a lagging node returns retryable insufficiency without discharging it. Acquisition alone does not establish browser installation: the adapter admits and installs the observation under the current resource-instance and adoption policy. The advisory branch illustrates one selected remote-lifecycle notification policy; Part VIII leaves automatic adoption of already-visible remote lifecycle changes as an explicit HumanHelp product decision. It does not reconstruct the Claim from node A's process memory, and possession of the advisory does not itself authorize the read. The reread can establish current Request state without proving whether Alice received her direct settlement response or even whether node C can identify the exact execution that produced that state.

Now contrast an ephemeral typing signal whose **admitted participant-message receipt** is itself the semantic event. The browser still does not get to assert a trusted sender or recipient identity. The routing boundary binds those from authenticated/current collaboration state:

```
(def browser-typing-intent
  {:message/id #uuid "0a675872-2ba3-4575-a76f-4024d2c73938"
   :kind :collaboration/typing
   :document/id #uuid "389bdfac-48c9-4563-8b83-83bfae42d32e"})

(def typing-route-contract
  {:delivery :best-effort
   :durability :ephemeral
   :duplicates :permitted
   :late-message :discard
   :sender-binding :authenticated-principal
   :recipient-binding :current-collaboration-route
   :authorization :current-document-membership
   :semantic-effect :recipient-observed-admitted-message})

(defn route-typing-intent!
  [http-request wire-intent]
  (let [sender
        (auth/require-authenticated-principal node-a-system http-request)

        document
        (document/read-current-authoritative
         node-a-system
         (:document/id wire-intent))


        _
        (auth/require-current-document-membership!
         node-a-system sender document)

        recipients
        (routing/current-collaboration-recipients
         node-a-system
         {:document document
```

```
          :excluding (:user/id sender)})

        admitted-message
        (routing/admit-participant-message
         {:intent wire-intent
          :trusted
          {:sender sender
           :recipients recipients
           :document document}
          :execution/id (execution/new-id)
          :expires-at (time/plus (time/now) (time/seconds 4))
          :contract typing-route-contract})]

    (routing/deliver! node-a-system admitted-message)))

(defn receive-typing!
  [recipient-session delivered-message]
  (let [receipt
        ;; Checks trusted sender/recipient binding, current recipient access,
        ;; admitted routing scope, expiry, and current execution/resource owner.
        ;; A wire map or broker delivery by itself is not a receipt.
        (routing/admit-recipient-receipt
         {:recipient-session recipient-session
          :message delivered-message
          :contract typing-route-contract})]

    (when receipt
      (ui/show-typing-indicator!
       (get-in receipt [:trusted :sender :display-name])))))
```

There may be no durable business fact for node C to reread later. If the typing message is lost, its declared best-effort semantics permit that loss; a stronger product guarantee would need an explicit stronger routing, persistence, acknowledgment, or recovery contract. Conversely, broker persistence or a delivery acknowledgment would not by itself turn the message into an authoritative domain commit. Reusing Request invalidation semantics would conflate two meanings merely because both can travel through the same broker or load-balanced hosts.

First physical multi-node topology may be simple

The first useful multi-node environment does not need sophisticated orchestration.

A development harness may use, as concretely useful:

- multiple Aleph nodes;
- HAProxy or another load balancer;
- shared XTDB/object-store dependencies;
- Valkey or another Live invalidation-routing substrate;
- Redpanda or another log only where a concrete requirement warrants it.

The service list is implementation-specific.

The semantic gate is real host replacement and cross-node propagation through the same trusted model/protocol boundaries. Naming services is not evidence of those properties. The selected topology must record the actual dependency versions and supported consistency/durability contracts; this document neither requires every listed service nor certifies an unspecified deployment.

Multi-node assumptions enter the assurance graph

Once the physical implementation exists, the root assurance report should name the contracts needed for:

- host replacement;
- distributed invalidation routing;
- interest registration;
- post-commit publication;
- duplicate publication;
- lost publication;
- stale routing knowledge;
- reconnect through another node;
- node-local process loss;
- progression-safe reread from another node;
- command identity/deduplication scope and retention where retry guarantees rely on them;
- cross-node session/principal binding and compatible application realizations under the declared plan/protocol and admitted assembly contracts;
- acquisition/adoption policy across reconnect and initial reconstruction.

Authority committed while publication was lost must remain discoverable through the implemented post-commit recovery contract. If that contract relies on explicit Refresh, the liveness claim includes that user action. Reconnect alone is not evidence that lost advisory information has been recovered. A lagging node must not label an insufficient read as satisfying the requested progression merely because it returned successfully.

These assumptions arise from the actual implementation.

They should not be invented speculatively before the topology exists.

Required multi-node acceptance evidence

The multi-node milestone must identify the actual topology and test at least:

- a mutation on one node becoming authoritatively observable through another;
- an already connected remote browser receiving advisory without unauthorized new-Request adoption, followed by successful permitted Refresh;
- a node lost before submission versus one lost after submission/commit, with participant uncertainty and any permitted retry handled distinctly;
- crash after commit before publication, followed by the declared recovery path;
- duplicate or reordered advisory delivery preserving scoped outstanding debt;
- reconnect through a different node preserving the declared adoption policy;
- a read unable to meet the required progression preserving recovery rather than falsely claiming current authority;
- current authentication and authorization at the replacement host;
- incompatible old/new plan versions following the selected compatibility or retirement contract.

These cases use real node/transport boundaries where the claimed failure depends on physical deployment. Portable model tests remain complementary evidence. The report separates safety under interruption from progress after the named recovery conditions hold. A single successful load-balanced request does not establish the multi-node milestone.

Multi-node does not require durable suspended choreography machines

A node may disappear while the durable business process survives.

Ordinary long waits remain represented as authoritative model state and later short choreographies.

Persisting arbitrary suspended choreography-machine state across process death is a separate opt-in facility with its own:

- persistence;
- versioning;
- migration;
- ownership;
- recovery;
- delivery semantics.

Proof work and multi-node work can overlap

Full multi-node productionization is not a prerequisite for compositional proof hardening.

Conversely, a small real multi-node prototype may expose missing semantic distinctions that should be corrected before deeper formalization.

The correct relationship is iterative:

```
stable semantic model
    |
small real multi-node realization
    |
new failure evidence
    |
refine named assumptions / semantics if necessary
    |
proof/root report incorporates the actual contracts
```

The semantic invariant remains constant:

> Physical topology may change; semantic authority does not derive from host identity.

Formal correspondence and system-boundary invariant

A formal theorem applies to production only through its stated representation, checking, or refinement relation. Conformance vectors remain tests unless their coverage justifies a stronger claim. Complete obligation coverage, immutable subject/artifact binding, and explicit translation assumptions prevent a proof of one object from being attributed to another. Exact finite checking does not silently become an unbounded theorem, and safety correspondence does not silently establish liveness.

The root report preserves the evidence class, open assumptions, trusted boundary, and applicable subject/artifact identities and admitted assembly generation of each claim where relevant. Runtime authorization and protected acquisition remain necessary regardless of certificates. Multi-node placement changes hosts, not authority; retries, publication recovery, current reads, and command attribution follow their explicit contracts. HumanHelp's selective adoption policy survives these realizations. Operational success, compositional certification, external formal correspondence, and multi-node completion remain distinct milestones with distinct evidence.

Part XI — Release and completion

This part defines evidence reuse, candidate admission, release reports, global non-goals, and four distinct completion scopes. It makes a release claim reviewable against the actual artifacts and required capabilities without requiring every later proof milestone before an operational application exists.

Incremental certification, build, and release

A proof system that rebuilds the world after every edit will be resisted.

Design for incremental recomputation:

- normalized semantic identities, certified dependency identities, checker/rule profiles, and relevant assumptions determine certificate cache eligibility;
- direct dependency identities identify what must be reconsidered;
- unchanged semantic interfaces retain dependent validity where safe;
- diagnostic/source metadata can change independently of semantic identity;
- cache entries are re-admitted under the current checker/version and certification-profile contract;
- cache corruption cannot create a certificate the proof kernel or applicable dedicated semantic checker would otherwise reject.

Correctness dominates cache hit rate.

The build system is where semantic identity, proof dependency identity, whole-application closure, and exact browser artifact correspondence become one release discipline.

Deterministic semantic regeneration

For unchanged normalized semantic inputs, regeneration should produce stable:

- semantic identities;
- certificate dependency identities;
- projected plan identity where representation is deterministic;
- normalized application assembly identity;
- browser assembly manifest identity.

Determinism is relative to the complete declared inputs, including relevant normalization, compiler, dependency, and format versions. It does not claim that semantic equality alone makes all compiler output byte-identical. Exact artifact bytes have their own checked identity and build binding. Build timestamps or other diagnostic metadata may vary only where their exclusion from the relevant identity is explicit.

Diagnostic-only source metadata may change without forcing semantic re-certification when the normalized interface is unchanged.

Release gate

A production release must pass the gates for its declared supported scope and claimed milestone. It must not be admitted with any of the following defects within that required scope:

- unresolved obligations required by its selected certification profile;
- stale semantic dependency identities;
- invalid derivations submitted to the proof kernel or applicable dedicated semantic checker;
- stale browser artifact relative to the current assembly manifest;
- application assembly closure errors;
- a missing mandatory preflight lifecycle module;
- generated runtime that is stale relative to authored ClojureScript;
- proof/diagnostic data included as a runtime semantic decision input;
- missing required operational capabilities or failed independent acceptance;
- a production entrypoint that bypasses the required construction/render gates.

The operational milestone does not require completion of the later compositional or external-formal milestones. Conversely, a build requesting a stronger profile cannot silently fall back to operational assurance when certification fails. Admitted trusted premises remain visible; they are not unresolved proof tasks unless the selected profile requires their discharge.

Release scope and required capabilities are declared before gate evaluation. Removing a failing operation or shrinking the proof subset during that evaluation cannot preserve the original completion claim. A deliberately narrower release must be identified as such, with its missing capabilities and excluded claims. This does not redefine the canonical HumanHelp milestone into a smaller target.

Ordinary runtime test gates remain required. A failed, skipped, unavailable, timed-out, or incomplete required check is not a pass. Reports distinguish a counterexample or invalid artifact from infrastructure failure and unsupported checking. An expected rejection counts as success only for a negative test whose expected boundary and diagnostic were specified in advance.

The release gate produces a concrete candidate and evidence; it is not itself a deployment action. Activation follows the assembly-generation and lifecycle contracts in Part VII and must use the artifact that the gate admitted.

Incremental invalidation is dependency-driven

When semantic identity A changes:

```
A changed
    |
direct dependents of A reconsidered
    |
only affected downstream proof DAG rebuilt
```

The system should not rebuild unrelated artifacts merely because they live in the same source file or Git commit.

At the same time, cache correctness outranks speed.

A cache hit never bypasses validation by the proof kernel or dedicated semantic checker required by its current version and applicable certification profile. Reuse may avoid repeated work through an admitted immutable in-process result or another explicitly checked cache contract. Merely finding a stored successful report is insufficient.

Implementation changes require separate consideration. An unchanged semantic interface can preserve downstream contract-level derivations while invalidating conformance or test evidence for a changed handler, provider, compiler, or runtime artifact. Reuse follows each claim's actual dependencies under #req-certificate-composition, rather than a blanket semantic-ID shortcut.

1. Worked invalidation slice: one policy edit rebuilds only its semantic cone

   Suppose HumanHelp changes the rule that admits a helper to Claim a Request. The source edit matters because it changes normalized semantic input, not because it shares a file or commit with other code.

   ```
   (def claim-eligibility-source-v7
     {:artifact/class :semantic/rule
      :rule/id :request/claim-eligible?
      :requires #{:membership/current :request/open}
      :result :helper-may-claim})

   (def claim-eligibility-v7
     (semantic/normalize-rule
   ```

```
    claim-eligibility-source-v7))

(def claim-operation-v12
  {:artifact/class :semantic/operation
   :operation :request/claim
   :semantic/id :operation/request-claim-O12
   :depends-on #{(semantic/id claim-eligibility-v7)
                 :schema/request-S9}
   :effect :request/open->claimed})

(def request-card-plan-v18
  {:artifact/class :browser/plan
   :plan/id :plan/request-card-P18
   :depends-on #{:operation/request-claim-O12
                 :projection/request-card-J6}})

(def search-plan-v4
  {:artifact/class :browser/plan
   :plan/id :plan/request-search-P4
   :depends-on #{:projection/request-search-J3}})

;; Product policy changes: suspended helpers may no longer Claim. This
;; source edit does not assign or forge its next semantic identity.
(def claim-eligibility-v8
  (semantic/normalize-rule
    (update claim-eligibility-source-v7
      :requires conj :helper/not-suspended)))

(def invalidation
  (release/reconsider
   {:changed #{(semantic/id claim-eligibility-v7)}
    :replacement {(semantic/id claim-eligibility-v7)
                  (semantic/id claim-eligibility-v8)}
    :dependency-index production-dependencies}))

;; Conceptual result:
invalidation
;; ⇒
{:rebuild
 #{:operation/request-claim
   :certificate/request-claim
   :plan/request-card
   :assembly/humanhelp
   :browser-artifact/humanhelp}
 :retain
 #{:plan/request-search
   :certificate/request-search
   :projection/request-search}}
```

The operation must be reconstructed against `R8` because its former dependency identity is stale. The Claim-facing projection/plan and the root assembly are then reconsidered because their dependency closure contains that operation. If the generated browser artifact incorporates the changed Claim plan, its exact bytes and artifact identity change as well. Here `R7` and `R8` are mnemonic labels for identities computed from the normalized rules, and `production-dependencies` is the index derived from the admitted source graph. The new `:helper/not-suspended` predicate must have a typed, appropriately authoritative model input; if it does not, the rule is rejected rather than allowing the illustrated rebuild to proceed.

The search plan does not become stale merely because its implementation lives in the same repository, release, or source namespace. Its dependencies did not change. This is the useful meaning of incremental certification: retain claims whose applicability is mechanically established, while rebuilding the complete semantic cone of the changed subject.

Cached certificate corruption fails closed

A corrupted cache entry must not become accepted simply because its filename or cache key matches.

The cache stores candidate evidence.

The current trusted checker still performs the admission checks required for that certificate class: normalized subject, structure, dependencies, assumptions, checker/rule compatibility, and derivation validity where required. A copied semantic ID or stale success flag cannot override those checks.

Corruption may reject the candidate or trigger fresh reconstruction from valid inputs; it must never relax acceptance. Cached values crossing serialization boundaries regain admission only through the validation specified in Part IX.

1. Worked cache-admission slice: a green file is not a certificate

   A cache lookup may return bytes that look exactly like an earlier successful certificate. Admission still checks whether those bytes establish the current claim under the current checker contract.

```
(def current-subject
  {:semantic/id :operation/request-claim-013
   :depends-on #{:rule/claim-eligible-R8
                 :schema/request-S9}})

(def cached-report
  {:cache/key [:operation/request-claim
               :profile/compositional-v3]
   :subject :operation/request-claim-012
   :dependencies #{:rule/claim-eligible-R7
                   :schema/request-S9}
   :checker :checker/operation-v6
   :result :valid
   ;; A hash is not the derivation or its checked subject binding.
   :derivation-hash "71ad..."})

(def admission
  (certificate/admit-cached
   {:expected-subject current-subject
    :required-checker :checker/operation-v7
    :required-profile :profile/compositional-v3
    :candidate cached-report}))

;; A checker that collects safe independent diagnostics can report these;
;; a fail-fast checker may report the first blocking error instead.
{:admitted? false
 :errors
 [{:type :certificate/subject-mismatch
   :expected :operation/request-claim-013
   :actual :operation/request-claim-012}
  {:type :certificate/stale-dependency
   :expected :rule/claim-eligible-R8
   :actual :rule/claim-eligible-R7}
  {:type :certificate/checker-mismatch
   :required :checker/operation-v7
   :actual :checker/operation-v6}
  {:type :certificate/missing-required-derivation
   :reason :digest-without-checkable-proof}]}
```

   A hostile variant could copy the new subject ID and leave the old derivation bytes. That still fails when the applicable checker validates structure, dependency binding, assumptions, and derivation validity. A cache hit is an optimization opportunity; it is never an alternate trust path.

2. Worked implementation slice: unchanged contract, changed adapter

   A different kind of edit changes production realization while leaving the semantic operation unchanged. That does not

justify throwing away every proof, but neither may the old implementation evidence be silently reused. Crucially, semantic evidence is retained only after its applicability to the *current normalized semantic subject* has been checked; matching a semantic-ID string is not itself that check.

```
(def claim-contract
  {:normalized/semantics
   {:pre #{:request/open :helper/eligible}
    :post #{:request/claimed-by-authenticated-helper}}
   :depends-on #{:rule/claim-eligible-R8
                 :schema/request-S9}})

(def current-claim-subject
  (semantic/normalize claim-contract))

(def claim-semantic-id
  (semantic/id current-claim-subject))

(def xtdb-claim-adapter-a
  {:implementation/id :impl/claim-xtdb-A44
   :implements claim-semantic-id
   :handler #'request.model/claim-v1})

(def xtdb-claim-adapter-b
  {:implementation/id :impl/claim-xtdb-B45
   :implements claim-semantic-id
   :handler #'request.model/claim-v2})

(def previous-evidence
  {:semantic-certificate
   ;; A complete previously issued certificate candidate, including the
   ;; normalized subject, derivations, dependencies, and assumption scopes.
   ;; The checker re-admits it below; a report saying :result :valid would
   ;; not suffice.
   (certificate-fixture/serialized-checked-operation-certificate
    {:subject current-claim-subject
     :dependencies (:depends-on claim-contract)
     :checker :checker/operation-v7
     :profile :profile/compositional-v3})

   :implementation-conformance
   {:subject :impl/claim-xtdb-A44
    :against claim-semantic-id
    :result :pass}

   :integration
   {:subject :assembly/humanhelp-G31
    :uses :impl/claim-xtdb-A44
    :result :pass}})

(def semantic-reuse
  (certificate/admit-reuse
   {:current-subject current-claim-subject
    :current-dependencies (:depends-on claim-contract)
    :required-checker :checker/operation-v7
    :required-profile :profile/compositional-v3
    :candidate (:semantic-certificate previous-evidence)}))

;; Conceptually, this is true only because the current checker establishes that
;; the certificate still applies to the current normalized subject and its
;; dependencies.  The computed semantic ID is useful correlation, not proof.
```

```
{:admitted? true
 :reason :subject-dependencies-and-derivation-rechecked}

(def reconsidered
  (release/reconsider-implementation
   {:old xtdb-claim-adapter-a
    :new xtdb-claim-adapter-b
    :semantic-reuse semantic-reuse
    :evidence previous-evidence}))

;; Evidence about A44 cannot establish facts about B45.  The semantic theorem
;; may be retained only through the admitted applicability result above.
{:retain #{:semantic-certificate/operation-request-claim-013}
 :rerun #{:conformance/claim-adapter
          :integration/humanhelp-claim
          :real-browser/request-claim}
 :revalidate #{:server-provider-binding
               :assembly/humanhelp
               :startup-preflight/humanhelp}
 :rebuild-if-bound #{:browser-manifest/humanhelp
                     :browser-artifact/humanhelp}}
```

The semantic certificate is reusable because the current checker establishes that its normalized subject, semantic dependencies, checker contract, and profile still apply. It is *not* reusable merely because both versions carry the same computed Claim identity. A copied or stale semantic ID cannot establish semantic sameness.

The conformance result is not reusable because it was evidence about implementation `A44`. Assembly and artifact work is recomputed precisely where the admitted implementation identity participates in their construction. The `A44/B45` labels stand for implementation identities established from the actual bound handler, build, and relevant dependencies. Assigning those labels in a map is not itself evidence that production executes those implementations; activation checks the current binding under Part VII's assembly lifecycle.

This distinction prevents two opposite pathologies: rebuilding every theorem because a handler was refactored, and retaining either semantic or production evidence on the strength of matching labels that no current checker has shown to remain applicable.

Root release report

A production build should be able to emit one machine-readable and human-readable root report describing at least:

- application/assembly semantic identity;
- certified artifact classes;
- proof kernel and dedicated semantic checker versions, as applicable;
- unresolved obligations, expected zero for certified production scope;
- explicit assumptions/assertions;
- whole-application closure status;
- current browser-artifact identity and correspondence;
- runtime/test/conformance gate results;
- TCB classification;
- declared milestone, supported subset, and certification profile;
- concrete source/build provenance, admitted assembly generation, and dependency versions;
- identities of the actual tested and served artifacts;
- required user-capability acceptance and production-enforcement evidence;
- exclusions, unresolved product choices, and environmental/liveness premises;
- result and evidence reference for each required gate, including incomplete or skipped outcomes.

Each evidence reference identifies the subject and configuration it covers. Passing tests for a different fixture assembly, old artifact, or shadow model cannot establish current release behavior. Reuse is permitted only where its applicability is justified by unchanged dependencies or a checked compatibility relation. A prior timestamp or a matching operation name is not that relation.

The report is release evidence describing what was built and which assurance claims support it. It is not runtime authorization and does not become a semantic input merely because the build that produced it was admitted.

Candidate integrity and activation

The admitted candidate binds declarations, prepared server realizations, providers, routes, normalized semantic/certificate identities, executable plans, manifest, and actual generated bytes according to the acyclic construction phases in #req-assembly-construction-phases. Build and test evidence must not be paired with a different artifact at deployment time.

Unchanged normalized inputs produce deterministic identities where claimed, including certificate content/identity when its encoding is deterministic. Diagnostic source metadata remains separate. Where exact reproducible build bytes are claimed, the toolchain and all byte-affecting inputs must also be controlled; otherwise the report claims semantic reproducibility and records the actual artifact bytes separately.

A dependency or mutable binding changed after admission requires the appropriate revalidation before activation. A failed candidate may leave a still-valid old assembly generation running under the lifecycle contract; it may not partially replace that assembly generation. Old browser plans use the explicitly supported compatibility or retirement/recovery path.

Reactivating an older assembly generation does not roll back committed domain state. Any release involving model/schema evolution must identify its supported state compatibility and recovery constraints before claiming that rollback is safe. This requirement does not introduce a universal database migration or workflow engine.

Bounded rolling-release compatibility

A release boundary must distinguish at least four different identities that are easy to collapse incorrectly:

- the semantic identity of an operation or plan;
- the exact identity of emitted browser/server artifacts;
- the admitted assembly-generation identity of a releasable whole application;
- the compatibility relation that says whether an execution originating in one admitted assembly generation may continue against another.

Compatibility is not inferred from matching names, numerical version ordering, Git ancestry, build count, wall-clock recency, or the fact that two releases were produced from similar source. It is a checked relation over the continuation- relevant semantics and deployment capabilities of the two admitted generations.

The baseline operational policy is deliberately bounded. Ordinary rolling deployment needs to support a small overlap between the generation being replaced and the generation replacing it; it does not require arbitrary old browser executions to remain live forever. A typical supported window is:

```
assembly generation N
        <---- checked rolling compatibility ---->
assembly generation N+1

assembly generation N+2 activates
        |
        +-- N execution is outside the supported window
            -> retire stale execution universe
            -> preserve only explicitly recoverable semantic intent
            -> reacquire current authoritative realization
```

The symbols `N`, `N+1`, and `N+2` name an example **activation order**, not numerical assembly-generation identities or a proof of compatibility. The exact supported overlap window is a release capability. An application may choose a different bound, but it must be explicit, finite, and checked.

Part VII fixes when an admitted assembly generation exists: a complete realization is constructed and validated for serving. It may be admitted during development without being selected for a production release. Admission binds the actual routes, prepared operations, providers, plans, and generated artifact for that generation; relevant changes require a new admission or an explicitly checked transition even if the semantic continuation contract is unchanged. A release selects admitted generation(s) for activation and records their rolling compatibility. Development rebuilds or commits alone neither mutate an existing admitted generation nor imply that its release will be activated. Deployment/activation order and assembly-generation identity remain different coordinates.

1. Direction matters during overlap

   Rolling compatibility may be directional. If deployment/routing permits both of these situations:

   ```
   old browser / old plan  -> new server
   new browser / new plan  -> old server
   ```

   then the release gate must establish both directions separately. If routing or activation rules make one direction impossible, the report may rely on that operational fact, but it must name and test it rather than silently assuming symmetry.

   The compatibility question is therefore not merely:

   ```
   compatible?(N, N+1)
   ```

   but, where direction matters:

   ```
   may-continue?(client-generation, server-generation, operation/plan)
   ```

   A successful result means that every continuation-relevant assumption required by the suspended execution remains valid under the target generation, or that an explicit checked adaptation contract establishes the required refinement.

2. Compatibility is derived from continuation-relevant semantics

   Where practical, Gesso should derive a normalized continuation contract from the same admitted semantic objects that produce the executable plan. The contract contains the facts that can affect whether suspended work may safely continue, for example:

   - operation and plan semantics;

- command argument, result, and outcome contracts;
- authority and principal requirements;
- event vocabulary and relevant state transitions;
- retry, duplicate, uncertainty, and reconciliation semantics;
- progression/acquisition obligations;
- required runtime/browser capabilities;
- route and transport envelope correspondence, including the target server's actual decoder and the runtime artifact that may send the envelope;
- settlement and continuation obligations;
- any other dependency whose change can invalidate an already projected continuation.

A conservative implementation may use equality of a soundly normalized, complete continuation-contract identity for the **semantic** portion of the compatibility decision. That equality is useful because a change represented by the contract retires the old continuation unless a stronger rule has been justified. It is not by itself evidence that the target server actually serves the bound route/decoder, that the old client artifact and target runtime support the same envelope/events, or that the relevant implementation and assembly bindings passed admission. The release gate checks those concrete facts for each client-to-server direction it permits, and the runtime still re-establishes current authorization and other dynamic premises at use.

A more permissive implementation may establish a directional compatibility or migration relation that proves an older continuation remains safe under newer semantics. Such a relation is additional checked evidence. A handwritten assertion that two plans are compatible is, at best, an explicit trusted premise unless the selected assurance profile supplies a checker for it.

The release report records which rule established compatibility. It must not reduce that fact to a label such as `v3` whose meaning depends on undocumented human convention. The report identifies both the semantic relation and the actual admitted client/server realizations and evidence to which it applies.

3. Retirement preserves semantic intent, not stale execution authority

   When an execution falls outside the supported compatibility window, the safe default is retirement followed by authoritative reacquisition. Retirement is not an attempt to deserialize and continue arbitrary old machine state under the new release.

   Some application context may survive because it denotes stable semantic intent. Examples can include, where explicitly supported:

   - the logical route or view the user was visiting;
   - stable domain subject identifiers, such as the Request being viewed;
   - filters, sort choices, tabs, or other local presentation choices whose meaning is still admitted;
   - draft user input that the application has explicitly classified as recoverable.

   An outstanding semantic command has a different status from a browser execution. If its delivery or incorporation is uncertain, retirement must not silently erase the command-outcome question. Where the operation's declared retention and recovery contract permits it, the stable command identity, original semantic binding, and unresolved reconciliation obligation survive as recoverable **command-specific intent and knowledge**. They do not carry the old execution's authority, and a browser-held identifier is never proof of incorporation or permission to disclose a recorded result. A new generation must admit any lookup or compatible retry under the current authenticated principal and the model's command-binding and disclosure rules. If attribution cannot be recovered under the declared contract, the result remains unknown; current-state acquisition may still repair the displayed view.

   The following are execution artifacts and do not survive merely because they were present in the old browser:

   - execution and resource-instance identities;
   - pending callbacks or continuation handles;
   - old authority observations or knowledge frontiers;
   - provisional settlement state whose basis is no longer current;
   - old artifact bindings and runtime capabilities;
   - continuation rights established only by the retired plan;
   - assumptions about current authoritative domain state.

   Retiring provisional presentation therefore does not convert its uncertain command into a confirmed failure. Nor may recovery silently assign a fresh command ID and resubmit the same business intent as if the old command were known not to have committed.

   A recoverable draft may preserve bytes or structured user input, but the new generation must re-admit and revalidate that

input under current semantics before it can affect authoritative state. Recovery never turns possession of stale browser state into authority.

The governing rule is:

> Release recovery preserves stable application intent and reacquires its current authoritative realization; it does not preserve stale execution semantics merely because they were present in the old browser.

For example, a user who was viewing Request `R` before retirement may be returned to the current authoritative view of `R`. If `R` became completed while the old execution was suspended, recovery shows the completed Request. It does not resume the old plan at an earlier lifecycle state merely to preserve continuity.

Suppose Claim command `C42` was sent before a deployment, the response was lost, and the originating execution is then retired. Its callbacks and provisional card are discarded. An authorized recovery may retain `C42` to resolve its recorded result or retry that **same** command under the model's compatibility, retention, and disclosure contract. Rereading `R` can establish its current state without proving that `C42` caused it. If no permitted command-specific evidence is available, the participant's attribution remains unknown even when the current card is rendered correctly.

4. Unsupported skew fails closed

An incompatible or unsupported generation pairing must be detected before the stale execution can perform a continuation that relies on changed semantics. The permitted outcomes are bounded compatibility, semantic retirement/recovery, or an explicit typed rejection when recovery is not permitted. Silent fallback to an older contract is forbidden.

A release gate for a rolling deployment therefore checks at least:

- the assembly generations that may coexist during activation;
- every client/server direction that routing can actually expose;
- the continuation contracts or checked compatibility evidence used for that coexistence;
- concrete admitted target route/decoder, runtime capability, and artifact evidence for each permitted client/server direction;
- the retirement/recovery behavior for generations outside the supported window;
- preservation and re-admission rules for recoverable application intent;
- preservation or honest unresolved status of command-specific outcome obligations across execution retirement;
- real runtime tests showing that unsupported skew does not continue silently.

This bounded policy gives deployment overlap a precise meaning without turning Gesso into a general historical workflow-migration system.

Global non-goals

The following constraints keep Gesso's semantic and proof boundary deliberately smaller than the set of all possible distributed-system machinery.

Applications may opt into stronger mechanisms when their domain benefits.

Gesso does not require those mechanisms universally.

No universal persisted fact DAG

Gesso does not require a second persisted fact graph containing every:

- authoritative fact;
- provisional fact;
- supersession relation;
- command;
- resolution;
- dependency edge.

Fact-graph concepts are useful for reasoning.

Application models and their database remain the persistence authority for domain facts and any domain history they choose to retain.

No universal Git-like history

Gesso does not require every model to persist:

- branches;
- merge commits;
- common ancestors;
- three-way history;
- universal conflict records.

The Git analogy is useful for reasoning about:

```
authoritative basis
provisional local state
semantic commands
re-evaluation
reconciliation
```

Gesso may provide reusable reconciliation machinery.

The model provides the meaning of reconciliation.

Applications remain free to adopt stronger domain histories such as:

- event sourcing;
- CRDTs;
- ancestry;
- manual reconciliation workflows;
- base/ours/theirs resolution.

No universal global integer revision

Gesso does not invent one global monotonically increasing integer that stands for all authoritative application state.

That would incorrectly collapse:

- XTDB progression;
- model/entity revision;
- valid/effective time;
- command causality;
- actor knowledge frontier.

Models may define their own integer revisions when domain semantics benefit.

No universal cross-node participant-message fabric

Durable cross-user facts normally rendezvous through authoritative model state.

A global runtime directory mapping every actor to a current Aleph node is not a baseline requirement.

Direct cross-node participant routing is added only for protocols whose message receipt is itself semantically required.

No universal durable suspended workflow engine

Ordinary Choreo does not persist every suspended server-side execution for hours or days.

Human-scale waiting normally lives in authoritative model state and later short executions.

A durable workflow engine would be an explicit separate capability.

No transparent arbitrary stale-plan migration

The framework does not promise to translate arbitrary suspended browser machine state from one incompatible deployment into another.

The safe default is:

```
detect incompatibility
    |
semantic retirement
    |
best-effort cleanup
    |
permitted authoritative reacquisition / explicit recovery action
    |
current-plan restart
```

Specific compatibility/migration can be added where explicitly justified. Retirement cannot be used to silently bypass HumanHelp's acquisition/adoption policy. Recovery identifies which local choices and adopted membership survive and what requires a new authorized acquisition.

No proof of the entire browser/OS/hardware stack

Gesso does not claim to formally verify:

- Chromium/Firefox;
- JavaScript engine;
- JVM;
- OS;
- CPU;
- network hardware;
- HTMX internals;
- every third-party dependency.

Formal browser-adapter claims are relative to explicit primitive/FFI contracts.

Real integration/fault testing remains mandatory evidence for those boundaries.

No inference of correct product policy from syntax

Gesso may establish that a business-rule expression is:

- well typed;
- compositionally compatible;
- used only where the actor has justified knowledge/authority;
- faithfully preserved through projection/execution.

It cannot infer that the product organization chose the desired policy.

A coherent but wrong business rule remains possible.

No arbitrary Clojure theorem proving

The formal subset is deliberately closed and smaller than Clojure.

The framework does not initially attempt to prove arbitrary host-language functions.

Unsupported constructs fail explicitly or remain outside the stronger proof boundary under a declared weaker scope. A required stronger judgment cannot be satisfied by silently treating unsupported behavior as an assertion.

No elimination of runtime validation

Static/certificate proof does not remove runtime validation for:

- untrusted inputs;
- schema boundaries;
- authenticated principal binding;
- current authorization;
- plan/runtime compatibility;
- concrete progression values;
- external system failure.

Those checks are premises of the complete correctness story.

No elimination of adversarial/integration testing

Formal proof does not replace:

- real Chromium;
- HTMX/SSE integration;
- XTDB integration;
- fault injection;
- load/resource testing;
- multi-node testing.

Those test the trusted implementation shell and external assumptions.

No requirement for Typed Clojure or unrestricted dependent types

The typed semantic rule/choreography languages are purpose-built closed languages.

The architecture does not depend on adopting a universal host-language type system or unrestricted dependent types.

No requirement that application developers write proof-assistant code

Application authors declare domain semantics, rules, operations, choreography, and explicit assumptions.

They do not manually construct Lean proof scripts or internal proof-kernel terms for ordinary development.

No conflation of semantic identity with source identity

File path, namespace name, Git commit, line number, and artifact timestamp are not semantic identity.

They may be useful provenance/diagnostic data.

They are not substitutes for normalized semantic identity.

No proof-sidecar runtime semantics

Proof/diagnostic sidecars remain optional read-only explanation artifacts.

They never participate in production transition decisions.

No arbitrary unconditional distributed liveness claim

Liveness/convergence claims name their environment assumptions.

The framework does not prove that an unavailable network, dead client, or failed external system will eventually recover without such an assumption.

No requirement that every actor know every fact

Knowledge and agreement remain scoped by:

- relevance;
- role;
- authorization;
- privacy;
- actual behavioral need.

A correct system may intentionally expose different authoritative projections to different actors.

No automatic conflict policy

Gesso may provide typed reconciliation protocol structure.

The model owns domain-specific conflict meaning and resolution.

No speculative database portability

Gesso may use XTDB-specific mechanisms where doing so materially strengthens correctness or simplicity.

Semantic concepts remain separate from accidental XTDB representation details, but database neutrality is not purchased at the expense of the current design.

Progress metrics and completion boundaries

Project progress should track semantic coverage and rejection quality rather than raw test count or revision count.

Useful progress metrics

Track, where meaningful:

- number of certified semantic artifact classes;
- number of framework properties with explicit proof/check rules;
- number of application assumptions surfaced;
- number of trusted implementation boundaries;
- TCB size;
- proportion of downstream mutation classes rejected before runtime;
- number of semantic dependency mismatches caught at the first possible stage;
- number of production plans traceable to a root assurance DAG;
- number of externally formalized proof-kernel properties;
- number of production checker rules with an explicit formal-correspondence story;
- diagnostic completeness for rejected mutants;
- number of exposed application operations with complete whole-application closure;
- number of real-browser/multi-client acceptance scenarios exercising actual production artifacts.

Raw test count is not a primary progress metric.

Each coverage metric needs a stable denominator: the required capability set, mutation classes, artifact classes, or theorem scope it measures. Reports retain missing and blocked items instead of improving percentages by dropping them. Count required usable operations separately from exposed closed operations.

Assertion counts and TCB size are audit aids, not scalar confidence scores. One central unproved premise may dominate many derived facts. Report the claims that depend on it. Rejection quality includes an appropriate first boundary, repairable diagnostics, and paired valid cases; rejecting every program is not progress.

Operational application completion

The first bounded completion milestone is the usable, structurally closed Gesso and HumanHelp proving application.

This milestone requires, at minimum:

- the supported final Biff lifecycle substrate;
- authoritative production-model ownership with explicit FX value flow;
- correct XTDB progression/consistency integration;
- executable Choreo, projection, browser, optimism, continuity, and Live semantics for the supported operational subset;
- production execution of the same canonical `ExecutablePlan` whose semantics are checked and tested;
- mandatory whole-application assembly and rendering checks on the supported production path;
- exact current browser-artifact correspondence;
- no silent semantic downgrade;
- required HumanHelp creation, details, response, lifecycle, search, sorting, visibility, advisory, explicit Refresh, continuity, and authorization behavior;
- real Chromium acceptance using actual controls, routes, generated artifacts, and multiple authenticated contexts where the behavior is multi-client;
- representative generic adversarial and controlled-defect tests demonstrating that the production path invokes the promised checks.

All required HumanHelp capabilities are interpreted by Part VIII, independently of the implementation's exposed-action list. Claim, Unclaim, Mark on the way, Complete, Cancel, and Reassign require eligible positive browser scenarios; state-dependent unavailability is additional evidence, not a substitute.

The response-flow contract at #decision-humanhelp-response-flow and remote lifecycle policy at #decision-humanhelp-remote-lifecycle-adoption must be selected and tested before full completion. Refresh action scope, concurrent advisory handling, and successor-read permission must be explicit under #req-humanhelp-refresh-coverage. These decisions belong to HumanHelp; Gesso does not choose them to make a gate pass.

Evidence separately establishes declared closure, actual production enforcement, and required user behavior under #req-application-assurance-claims. A server starting, no console errors, or HTTP 200 responses establishes none of these alone. Mutation success requires production-model truth and fresh authoritative acquisition; a provisional DOM change is insufficient.

This milestone establishes a usable application under its explicitly stated assurance and environmental assumptions. Until all required gates pass, a partial checkpoint must name the capabilities or decisions still missing.

It does not claim completion of compositional certification, external formal verification, or production multi-node realization.

Compositional certification completion

The compositional-certification milestone requires the intended semantic interfaces and dependency chain to be operational for the artifact classes in scope.

It includes:

- certified semantic interfaces for the selected model/schema/domain/rule/ operation layers;
- a closed typed business-rule subset that rejects malformed expressions while permitting coherent but product-wrong policy;
- distributed admissibility checks for knowledge, provenance, authority, refinement, and progression;
- certificate composition and stale-dependency rejection;
- a deliberately small trusted proof kernel or dedicated semantic checker boundary where claimed;
- proof-producing elaboration or equivalent checked derivation construction for the supported classes;
- systematic downstream mutation sensitivity;
- inspectable proof dependencies and repair-oriented diagnostics;
- at least one substantial HumanHelp operation certified through the semantic and whole-application chain;
- complete required-obligation coverage and subject/derivation binding;
- explicit transitive assumption propagation and checked discharge;
- certificate re-admission after cache/serialization boundaries;
- evidence binding semantic claims to the implementation or admitted assembly generation actually covered, without conflating their different identities.

The Claim pilot follows #req-humanhelp-certification-pilot and connects to the real operational scenario. Positive valid cases accompany the mutation corpus. An isolated proof fixture, a root graph containing mostly unsupported claims, or a checker accepting only trivial rules does not satisfy the intended production operation scope.

The executable semantic core remains the production transition semantics; the proof kernel checks derivations, and dedicated semantic checkers establish their named judgments. Completion of this milestone does not merge these roles or turn proof evidence into runtime transition input.

The milestone must name exactly which artifact classes and application scopes are certified. It must not imply certification of arbitrary Clojure code or business intent beyond the stated contracts.

External formal verification completion

The external-formalization milestone requires an independent machine-checked account of the stable semantic subset for which Gesso makes formal claims.

It includes:

- the formalized semantic subset;
- exact theorem statements;
- named assumptions and trusted boundaries;
- soundness, refinement, or correspondence results appropriate to each theorem;
- an explicit relationship between formal objects and the production semantic objects/checker rules they justify;
- release/correspondence evidence preventing a disconnected formally verified model from being mistaken for proof about unrelated production semantics.

The correspondence evidence follows #req-formal-production-correspondence. Conformance vectors alone do not discharge a required theorem about production acceptance or execution. If the chosen correspondence mechanism retains trusted translation/extraction or runtime assumptions, the exact conditional claim and those premises must be reported. A theorem for a different or outdated subject cannot satisfy the current release gate.

Exact finite exploration, bounded testing, and unbounded theorem proving remain distinct evidence classes throughout this milestone. Incomplete exploration or an unavailable external checker cannot be reported as successful verification.

Multi-node production completion

Production multi-node completion requires the semantic host/authority model to be realized over actual multiple application nodes without deriving authority from host identity.

It includes:

- physical node topology and routing;
- distributed invalidation/interest realization;
- host replacement and process loss;
- duplicate/lost publication behavior;
- reconnect through another node;
- progression-safe authoritative reread;
- deployment/version compatibility;
- multi-node adversarial, load, and fault evidence;
- incorporation of the resulting implementation assumptions into the root assurance report;
- the concrete host-loss, publication-loss, duplicate/reordered advisory, authorization, and compatibility cases in #req-multi-node-realization;
- preservation of selective adoption and explicit Refresh across reconnect;
- declared command retry/deduplication and retention contracts where claimed;
- stated capacity/resource bounds and load conditions for the production claim.

Tests distinguish an operation's durable outcome from another participant's knowledge of it. Current-state reads do not automatically resolve command attribution, and lost publication must exercise the implemented recovery path. Progress claims include any necessary user action and availability assumptions. A load-balanced demonstration without this failure evidence is a prototype.

A bounded multi-node prototype may occur earlier when sufficient executable and authority foundations exist and when it can expose semantic mistakes cheaply. That exploratory feedback does not constitute this production completion milestone.

These four milestones are completion scopes, not aliases for stage numbers. Scheduling may overlap where dependencies permit, while every milestone retains its own acceptance boundary. Production multi-node completion does not imply external formal completion, nor does external proof establish deployment resilience. A compositional pilot can be developed alongside operational work, but does not authorize calling incomplete HumanHelp capabilities complete.

Completion records and continued applicability

A completion record names the milestone, canonical requirement set, supported scope, candidate assembly generation, evidence references, test environments, assumptions, open decisions, and exclusions. Every required gate has a result; missing evidence is not presumed successful. Required unresolved decisions block the full claim rather than being buried in release notes.

Completion applies to that admitted scope and assembly generation. Later changes trigger re-evaluation of the affected guarantees and gates through their dependency relationships. A previous completion record remains historical evidence; it does not confer permanent correctness on later builds. Unaffected evidence may be reused where its applicability is established, avoiding unnecessary full rechecking while preserving the claim.

Documentation changes alone cannot establish implementation completion. This specification defines the gates; actual implementation evidence is required to claim that any milestone has been met.

1. Worked completion record: one release, four different answers

   A release report should make partial progress boringly explicit. A usable HumanHelp release may have completed the operational milestone while later assurance scopes remain incomplete. The report also names the exact admitted assembly-generation coordinate rather than using a bare, ambiguous ”generation” field.

   This is a **hypothetical future** release after HumanHelp has selected and tested the still-open response-flow and remote-lifecycle policies. The two decision references below are placeholders for independently recorded, substantive HumanHelp contracts; this example does not choose either policy or claim that the present application satisfies the operational milestone.

```
(def humanhelp-release-2026-09
  {:release/id :humanhelp/2026-09-r4-candidate-17
   :assembly
   {:semantic/id :assembly/humanhelp-A61
    :assembly-generation/id :assembly-generation/G204
    :assembly-generation/admission-evidence
    :evidence/assembly-generation-G204-admission
    :browser-artifact
```

```
 {:manifest/id :fixture/browser-manifest-G204
  :served-path :fixture/browser-asset-G204
  :served-bytes-digest "sha256:83c4..."
  :emitted-plan-registry :fixture/plan-registry-G204
  :currentness-report :evidence/browser-currentness-G204}
 :source {:gesso "819117f..."
          :humanhelp "dad5c7c..."}}

:canonical-requirements :humanhelp/part-viii-operational-v1
:selected-product-contracts
{:response-flow :humanhelp/response-contract-selected-v1
 :remote-lifecycle-adoption
 :humanhelp/remote-lifecycle-policy-selected-v1}

:milestones
{:operational-application
 {:status :complete
  :scope #{:request/create :request/details :request/respond
           :request/search :request/sort :request/board-options
           :request/show-terminal :request/hide-terminal
           :request/terminal-render
           :request/claim :request/unclaim :request/on-the-way
           :request/complete :request/cancel :request/reassign
           :creator/immediate-adoption :remote/creation-advisory
           :remote/lifecycle-selected-policy
           :advisory/explicit-refresh
           :advisory/late-event-retention
           :continuity/reconnect :authorization/protected-read
           :claim/concurrent-two-client-settlement}
  :gate-results
  {:gate/production-model-truth :passed
   :gate/required-capabilities :passed
   :gate/product-decisions :passed
   :gate/assembly-closure :passed
   :gate/startup-preflight :passed
   :gate/production-entrypoint :passed
   :gate/rendered-surface :passed
   :gate/browser-currentness :passed
   :gate/browser-runtime-source :passed
   :gate/plan-compatibility-or-retirement :passed
   :gate/jvm :passed
   :gate/node :passed
   :gate/chromium :passed
   :gate/real-http :passed
   :gate/multi-client :passed
   :gate/adversarial-defects :passed}
  :evidence-index
  :evidence/operational-G204-gates}

 :compositional-certification
 {:status :partial
  :scope #{:request/claim}
  :complete #{:typed-rule-subset
              :dependency-composition
              :claim-pilot}
  :missing #{:full-required-artifact-class-coverage}}

 :external-formal-verification
 {:status :not-complete
```

```
    :checked #{:formal/progression-finite-model}
    :in-progress #{:formal/knowledge-safety-draft}
    :missing #{:production-correspondence-theorem
               :stable-release-bound-evidence}}

   :multi-node-production
   {:status :prototype
    :complete #{:two-node-authoritative-reread
                :duplicate-advisory-test}
    :missing #{:host-loss-suite
               :deployment-compatibility
               :production-load-bounds}}}

  :open-decisions #{}
  :release-claim :operational-application})
```

The report can admit this candidate for the operational claim if every required operational gate passes and the referenced assembly-generation admission actually establishes that this report describes the candidate being released. The external evidence index must resolve **each** gate to its actual result, test environment, production-bound subject/artifact, and independent product contract. The report cannot turn a missing gate into `:passed` merely by including a keyword in this example. In particular, the two selected contracts must be real and independently tested for the full operational claim; absent either one, `:status :complete` and `:open-decisions #{}` are invalid. The map does not make `G204` admitted merely by naming an admission-evidence ID; that evidence remains independently checked under the release boundary.

It cannot summarize the same record as ”R4 complete” or silently reinterpret `:partial`, `:not-complete`, and `:prototype` as green. Conversely, unfinished formal or multi-node work does not erase evidence that already establishes the explicitly bounded operational milestone.

Later releases reevaluate only the statuses whose subjects or premises changed. The historical record remains useful because it names the exact admitted assembly generation, scope, and evidence to which each answer applied.

Working rule for every future semantic change

For every framework or application change that claims participation in the provable chain, ask:

> What proposition does this layer establish, what premises does it rely on, what evidence supports the proposition, and can the next layer mechanically verify that the premises and guarantees match?

If the answer is not precise, the component is not yet part of the compositional proof chain.

The intended end state is not:

> Everything is formally verified.

It is:

> Gesso knows which claims are proved, which are kernel-checked, which are exact- finite checked, which are assembly-verified, which are runtime-enforced, which are conditional on liveness assumptions, which are trusted, which are asserted by the application, and which are supported only by tests. Downstream programs are built compositionally from those claims, and construction stops at the first semantic mismatch instead of allowing incompatible assumptions to drift into an application that merely appears to work.

Part XII — Implementation order and final invariants

This final part preserves the eighteen-stage implementation order while making its dependencies, milestone relationships, and cross-stage acceptance explicit. It is an end-state roadmap, not a progress report or evidence that a stage has already been implemented. The detailed contracts in Parts I–XI govern each gate.

Unified ordered implementation stages

The canonical implementation order is the following.

The stages are semantic gates, not revision quotas. Their normative details remain in Parts I–XI; this roadmap summarizes dependencies and acceptance rather than defining weaker alternate contracts. A stage report identifies its supported scope, actual implementation evidence, remaining gaps, and applicable milestone.

Each stage must preserve a runnable validation path for the work it claims to complete. Intermediate construction may be incomplete, but a passing isolated fixture must not be reported as a completed application stage. Concrete defects in an earlier layer are repaired at that layer and downstream evidence is rechecked where affected.

Work inside one stage may expose a missing prerequisite in an earlier stage.

Low-cost later hardening may be pulled forward when it closes a concrete hole without obscuring the current acceptance target.

Stage 1 – Final Biff 2 substrate and lifecycle foundation

Goal:

> Make the underlying Biff execution/lifecycle model match the semantic assumptions used by Gesso.

Required work:

- select and record the supported final Biff 2 dependency and use its official execution/lifecycle contracts; schematic examples are not API verification;
- preserve explicit XTDB version choice unless separately changed;
- migrate generated/model FX machines to explicit invocation input and previous output rather than preview-era merged context;
- audit multistate Graph/FX usage;
- express Gesso lifecycle integration as stable-ID native Biff modules;
- establish a mandatory preflight/admitted-handler lifecycle dependency of serving; Stage 6 completes the closure checks carried by that boundary;
- migrate HumanHelp config/XTDB/Gesso Live/Aleph startup to native lifecycle modules;
- remove compatibility behavior that exists solely to preserve accidental preview semantics.

Gate:

- Gesso and HumanHelp compile/run on official Biff 2;
- no production machine relies on merged prior output in `ctx`;
- lifecycle IDs are explicit and validated;
- Aleph cannot serve before preflight;
- core JVM/Node/Chromium and consistency suites for the declared substrate scope are green, with existing unrelated failures reported separately;
- failed startup cleans up acquired resources without leaving an unchecked application listener serving.

Stage 1 establishes the integration boundary, not a claim that Stage 6's full checker already exists. A placeholder success report cannot satisfy mandatory whole-application preflight or the operational completion milestone.

Stage 2 – Authoritative model and consistency foundation

Goal:

> Make authoritative model transition semantics, XTDB progression, and trusted execution capabilities explicit and stable.

Required work:

- preserve model ownership of business truth;
- enforce preparation / atomic commit / after-commit distinction;
- carry authoritative progression through mutation, publication, and reread;
- distinguish command ID from execution ID;
- preserve trusted authenticated-principal binding;
- expose required/supplied server execution capabilities;
- reject known missing capabilities before first invocation;
- keep stale basis as model context rather than generic conflict.

Gate:

- atomic transitions cannot expose half-state;
- known post-commit publication failure is not mutation failure;
- outcome loss after submission is represented as uncertainty when appropriate;
- crash between commit and publication has the declared recovery path and explicit availability/user-action premises;
- XTDB snapshot/progression tests remain green;
- production model operations run through explicit Biff value flow;
- missing =clock=/seed/transaction/principal/basis requirements are structurally visible and resolve to compatible provider contracts, not merely matching names.

Stage 3 – Portable Choreo semantics, verifier, and projection

Goal:

> Stabilize the smallest useful global/local executable semantic core from which browser and server plans are derived.

Required work:

- explicit global and role-local transition semantics;
- typed roles/principals/actors/hosts/authorities;
- knowledge/provenance algebra;
- behavior-sensitive branch checking;
- closed messages;
- explicit operation contracts;
- explicit FX requires/outputs;
- resource and terminal semantics;
- authoritative observation as a knowledge path;
- deterministic derivation;
- trusted assertions remaining visible;
- projection preservation/refinement checks for supported subset;
- portable machine corpus across JVM, Node, Chromium.

Gate:

- supported choreography cannot consume facts the actor cannot justify;
- projection does not invent knowledge;
- all supported message/operation/state boundaries retain runtime schemas;
- production runtime executes generated projection rather than a handwritten parallel semantics.

Stage 4 – Browser semantic adapter, optimism, continuity, stale recovery

Goal:

> Make the browser runtime one explicit transition system whose race safety can be mechanically checked.

Required work:

- pure `AdapterState x NormalizedEvent → State x AbstractEffects` core;
- documented HTMX integration boundary;
- narrow browser FFI;
- execution and resource-instance isolation;
- semantic resource ownership;
- fragment single-flight/queued-refresh invariant, with logical slot bounds distinguished from metadata and retired physical-request bounds;
- owner-relative admission for shared fragment/subscription events;
- effect permission revalidation at deferred physical mutation boundaries;
- prevention of obsolete restoration after authoritative installation under the declared scope and supersession relation;
- typed provisional state;
- settlement/timeout/supersession semantics, keeping current-state supersession distinct from command-specific outcome knowledge;
- authoritative presence/absence;
- continuity separated from domain semantics;
- stale projected-plan compatibility and authoritative reconstruction;
- adversarial event-order/property tests.

Gate:

- stale callbacks cannot mutate newer execution state;
- retired executions and resource instances cannot exercise revoked ownership rights despite physical cleanup debt;
- optimistic state cannot silently become authoritative;
- canonical install cannot be rolled back by stale provisional work;
- same `ExecutablePlan` drives production and proof-enabled tests;
- partial installation or continuity failure cannot invent successful adoption, erase unresolved obligations, or expose incoherent enabled controls.

Stage 5 – Gesso Live publication, acquisition, and convergence

Goal:

> Close the path from authoritative commit to advisory invalidation to authorized authoritative reacquisition.

Required work:

- explicit publication after commit;
- progression carried through invalidation;
- managed fragment identity;
- initial authoritative knowledge or explicit acquisition;
- progression-safe refresh;
- authoritative absence;
- advisory versus adopted frontier distinction where applications need it;
- reconnect semantics;
- convergence without replaying every advisory invalidation.

Gate:

- no managed fragment is structurally blank with no acquisition path;
- invalidation never masquerades as authoritative content;
- admitted refresh results satisfy their scoped progression requirements;
- insufficient or incomparable observations preserve unsatisfied obligations rather than pretending a successful HTTP response establishes coverage;
- advisory debt does not itself authorize acquisition or adoption;
- successor acquisitions stay within the declared user-action/policy permission;
- an authorized explicit acquisition path remains available when a relevant advisory is never delivered and no debt was recorded for it;
- reconnect does not grant stale callbacks authority.

Stage 6 – Whole-application assembly and mandatory preflight

Goal:

> Reject applications whose individually valid pieces do not assemble.

Required work:

- canonical source-owned `ApplicationAssembly`;
- operation/view/plan/artifact/transport/route/server/model/settlement/acquisition closure graph;
- exact `BrowserAssemblyManifest` / generated artifact correspondence;
- required/supplied trusted server capabilities;
- semantic-route collision detection;
- no silent realization fallback;
- static build/start preflight, including structural coverage of required declared handling/recovery paths for supported dynamic failure conditions;
- dynamic pre-serialization rendered-surface preflight;
- structured error kinds;
- operation/application explanation;
- a source-derived semantic audit surface containing the business-rule catalog, operation cards, authorization/disclosure view, lifecycle/transition view, rule-dependency view, opaque-premise report, and assurance-coverage view;
- semantic-policy escape detection at every checked boundary able to determine that a condition influences authorization, disclosure, eligibility, ownership, authoritative transition, command acceptance, or another declared assurance;
- complete classification of correctness-relevant predicates **within each generated or closed checked subset** as admitted semantics, derived semantics, or an explicitly named premise;
- registration of arbitrary host-code realizations outside those subsets as scoped implementation-conformance/trust boundaries, with the operation contracts and assurance claims depending on them identified;
- startup lifecycle integration.

Gate:

- a statically knowable missing required closure edge rejects construction;
- required dynamic-condition handling/recovery paths are declared and structurally compatible under #req-preflight-dynamic-handling-coverage; preflight does not have to predict whether those conditions will occur;
- a contradiction first knowable in a concrete render rejects before delivery through the actual production render boundary;
- invalid invocation is rejected before protected mutation; a later render failure does not undo an earlier valid commit;
- stale browser artifact prevents serving;
- anonymous HTMX cannot occupy a semantic route;
- ordinary genuinely non-Choreo HTMX remains legal;
- startup cannot omit preflight accidentally;
- routes, providers, prepared server realizations, rendering, and served artifact stay bound to one admitted assembly generation or a checked compatible transition;
- the whole-application semantic audit surface is generated from the same admitted semantic/assembly objects that production uses rather than from a separately maintained policy inventory, and states its inspection scope;
- where a checked boundary can see that correctness-relevant host-language logic affects an admitted operation but cannot associate it with admitted semantics, that logic is promoted, derived, or exposed as an opaque/trusted premise rather than silently counted as mechanically assured;
- unresolved semantic-policy escapes prevent the affected stronger admission or assurance claim;
- zero detected escapes establishes closure only over the reported inspected scope; an unanalysed Clojure body does not gain a complete policy inventory by silence, and a profile requiring checked conformance cannot treat its opaque implementation premise as discharged;
- paired valid fixtures remain usable while controlled defects fail at the intended production boundary.

Release evidence, actual artifact binding, and operational test gates from Part XI apply here. They are not deferred until the later formal-release stage.

Stage 7 – HumanHelp production integration and rich-board proving surface

Goal:

> Demonstrate the architecture against production HumanHelp model semantics in a real browser.

Required work:

- example app consumes `net.humanhelp.site.model.*`;
- canonical HumanHelp application assembly;
- all six required Request lifecycle operations use the generic bound-operation path and have eligible positive browser scenarios, including legitimate target-helper selection for Reassign;
- Create Request works through production model;
- creator sees own authoritative creation immediately;
- other clients receive advisory without automatic adoption;
- Refresh adopts only justified membership/observations, discharges only covered advisory obligations, and preserves newer compatible local installation;
- live search;
- sort and visibility controls;
- Request details/continuity;
- a selected, independently testable response-flow contract;
- a selected remote lifecycle adoption policy, without equating advisory delivery with permission to show or acquire a changed Request;
- terminal rendering;
- login/principal separation;
- multiple browser contexts;
- real Playwright/Chromium acceptance.

Gate:

- actual Claim control emits real protocol command without test injection;
- persisted state survives reload;
- no missing optimistic contract silently downgrades;
- remote browser does not show new Request until explicit Refresh under intended product semantics;
- malformed application fixtures fail at the first applicable construction, rendering, or trusted-invocation boundary;
- all required lifecycle and rich-board capabilities pass independently of which actions the current implementation happens to expose;
- dropped advisory delivery still has an independent permitted Refresh or other declared recovery path; reconnect alone cannot be its only trigger;
- local creation while unrelated remote work is pending preserves selective adoption and advisory debt;
- local hiding is not misclassified as authoritative absence;
- response-flow and remote-lifecycle policy decisions from Part VIII are resolved and tested, including concurrent Refresh and successor acquisition permission.

Claim alone is the first vertical slice, not this stage's full completion gate. A permanently unavailable required operation or an unresolved response flow remains an explicit blocker even when the surviving assembly is closed.

Stage 8 – Generic adversarial closure and additional reference applications

Goal:

> Demonstrate that the semantic/application design is Gesso-generic rather than accidentally HumanHelp-specific.

Required work:

- small reference choreographies such as checkout, approval, and an ephemeral message/presence case;
- forged browser inputs;
- unauthorized reads/subscriptions;
- unrelated versus relevant authoritative advancement;
- duplicate/lost settlement;
- dropped invalidation;
- already-incorporated command;
- authoritative disappearance;
- browser suspension;
- stale plan;
- continuity failure;
- load/fault testing;
- HTMX/SSE lifecycle conformance.

Gate:

- reference applications need no HumanHelp-specific framework hooks;
- adversarial browser cannot gain authority;
- trusted external shell assumptions are exercised in real Chromium;
- unsupported semantic constructs fail explicitly rather than degrading;
- injected failures assert expected diagnostics precisely while unexpected console/page/request failures still fail acceptance;
- recovery tests distinguish known rejection, lost command attribution, known commit with failed delivery, and current-state supersession.

Stage 9 – Certified semantic interfaces and proof DAG

Goal:

Replace proof-by-reputation with compositional checked semantic dependencies.

Required work:

- inventory current assurance claims;
- canonical proposition/certificate vocabulary;
- raw versus certified artifact boundary;
- semantic IDs for normalized interfaces;
- stale dependency rejection;
- root proof DAG/report;
- explicit certification profiles and artifact-class obligations;
- cache/deserialization re-admission and scoped assumption identity.

Gate:

- same-shaped raw map cannot impersonate certified artifact;
- semantically stale dependency cannot compose;
- at least two independent existing checks can share certificate vocabulary without losing their distinctions.

Stage 10 – Typed downstream business-rule language

Goal:

Reject malformed downstream business-rule expressions before they enter model or choreography composition.

Required work:

- closed rule AST;
- nominal type/field resolution;
- enum/variant typing;
- optional presence refinement;
- relationship predicates;
- deterministic derivations;
- exhaustive closed match/case where required;
- source-aware diagnostics;
- explicit evaluation, absence/nil, ordering, and primitive failure semantics;
- observation- and subject-bound path refinements;
- declared totality or typed partiality for each admitted primitive.

Gate:

- malformed rule cannot become certified;
- wrong-but-well-formed policy remains representable;
- rule use at a distributed point still requires knowledge/provenance/authority.

Stage 11 – Certificate hierarchy across namespaces

Goal:

> Make schema/domain/relationship/rule/operation/Choreo/projection/plan/application guarantees compose transitively.

Required work:

- schema certificates;
- domain certificates;
- relationship/graph certificates;
- rule-set certificates;
- operation/FX contracts;
- core/public API certificates;
- Choreo certificates;
- projection certificates;
- executable-plan certificates;
- application-assembly/report integration.

Gate:

- changing relied-upon upstream semantics invalidates downstream construction at the earliest dependent stage;
- closed registries cannot gain undeclared members;
- unsupported dependency cycles fail;
- assumptions propagate per guarantee and can be discharged only through accepted evidence with compatible scope;
- semantic contract reuse does not preserve stale implementation evidence.

Stage 12 – Minimal proof kernel and proof-producing elaboration

Goal:

> Shrink the logical TCB so the small proof kernel decides whether derivations produced by the elaborator/compiler are valid.

Required work:

- small proposition syntax;
- first inference rules;
- derivation checking;
- richer elaborator/compiler producing proof terms;
- malformed-derivation mutation tests;
- explicit assertions as leaves rather than fake proofs;
- complete required-obligation coverage tied to the normalized subject;
- explicit normalization/projection/lowering correspondence or trusted premises;
- rejection of omitted obligations, wrong-subject proofs, and false assumption discharge, not merely malformed submitted derivations.

Gate:

- deliberately buggy elaborator output cannot convince the proof kernel of an invalid conclusion;
- proof-kernel responsibilities remain small and inspectable;
- diagnostic/proof sidecars remain runtime-nonsemantic;
- incomplete checking, timeout, and unsupported features cannot yield successful certification for a profile requiring that judgment.

The executable semantic core established earlier remains responsible for runtime transition semantics. The proof kernel checks derivations; dedicated semantic checkers establish their named judgments. This stage adds proof checking without replacing the executable semantic core or making proof sidecars runtime inputs.

Stage 13 – HumanHelp compositional proof pilot

Goal:

> Carry one real production HumanHelp operation through the semantic proof DAG and application realization report.

Required work:

- Request/Membership/Organization semantic dependencies where actually needed;
- Claim rule/operation contract;
- Choreo certificate;
- projection;
- `ExecutablePlan`;
- application assembly/artifact realization;
- explicit application assumptions;
- adversarial upstream/downstream mutants.

Gate:

- upstream semantic change prevents stale Claim plan construction;
- root report distinguishes proved, assembly-verified, runtime, asserted, and trusted claims;
- real runtime still uses the same executable artifact;
- the certificate chain connects to the actual Claim control, trusted model transition, and authoritative reacquisition exercised in Part VIII;
- omission of an obligation or substitution of an implementation/artifact cannot preserve an inapplicable success report.

Stage 14 – Systematic downstream mutation campaign

Goal:

> Demonstrate rejection quality across every supported proof/composition rule.

Required work:

- hand-authored diagnostic mutants;
- generated one-step mutants;
- schema/domain mutations;
- rule/type mutations;
- knowledge/provenance mutations;
- authority mutations;
- choreography/projection mutations;
- certificate/staleness/forgery mutations;
- application-closure mutations.

Gate:

- every supported rule's applicable rejection conditions have negative cases with expected outcomes fixed before execution, alongside valid controls;
- intrinsically valid changes that break an old dependency reject at composition, while accepted-by-design policy variations retain their explicit classification;
- incomplete test execution is not counted as either a successful rejection or an intentional acceptance;
- root failure is not obscured by later secondary ”missing plan” damage;
- structured diagnostics identify expected versus actual evidence.

Stage 15 – Explainability and repair surface

Goal:

> Make the proof/application state inspectable enough that humans and LLMs can repair local failures without reverse-engineering the entire runtime.

Required work:

- explain operation;
- explain knowledge/provenance derivation;
- explain application closure path;
- explain stale dependency;
- open-obligation report;
- generated semantic-policy inventory and assurance-coverage explanation;
- a source-linked explanation format for formal/model-checking counterexamples and unresolved production obligations, using correspondence metadata to identify their respective semantic and implementation subjects;
- stable structured diagnostics that distinguish mechanically invalid construction, a formal counterexample to declared semantics, and a merely suspicious but internally valid business policy;
- source ownership;
- valid candidate alternatives for closed registries;
- optional human/LLM intent review over the compact semantic surface, kept explicitly outside the trusted certification path.

Gate:

- malformed fixtures expose the failed obligation, actual/required evidence, responsible source, and appropriate repair boundary;
- a representative counterexample fixture can be explained in application vocabulary and traced to the semantic source and any related production obligation without treating the abstract trace as proof of a production bug;
- the explanation records its evidence class and whether the trace arose from a real generated backend, a fixture, or an abstraction not yet concretized;
- when generated TLA+/Quint/Recife or other formal artifacts exist, they are inspectable derivatives, not an independent repair surface that may be edited until the checker turns green;
- suggested alternatives preserve declared semantics; diagnostics do not claim a uniquely correct business-policy repair where the product must decide;
- heuristic intent review may label a well-formed policy suspicious, but such a label neither proves a defect nor changes admission; a confirmed concern gains mechanical force only after it is promoted into an admitted rule, invariant, contract, or other checked semantic object;
- explanation and heuristic review are read-only and cannot affect semantic execution.

Stage 15 establishes the explanation contract and tests its scope with an appropriate fixture. It does not require an external backend whose formalization and production correspondence are developed in later stages. A release that claims a generated-backend counterexample explanation must exercise that path with an actual generated trace under Stage 17's applicable evidence profile.

Stage 16 – Independent formalization of the stable small proof kernel

Goal:

> Machine-check soundness of production-used logical rules outside the Clojure implementation that proposes them.

Required work:

- formal proposition/type semantics;
- knowledge/provenance algebra;
- capability/authority entailment;
- progression algebra;
- certificate composition;
- selected transition safety;
- soundness theorem for the selected production-used rule class;
- explicit assumptions, admitted axioms, supported subset, and formal-tool version;
- stated proof scope, with finite exploration and unbounded claims distinguished.

Gate:

- at least one production-used certificate class relies on inference rules with externally machine-checked soundness;
- the report distinguishes soundness of those formal rules from correspondence of the production checker, which Stage 17 must establish at its claimed level.

Stage 17 – Production/formal correspondence and release integration

Goal:

> Connect production certificate acceptance to the externally formalized rules and make stale proof/application artifacts impossible to ship accidentally.

Required work:

- choose an explicit correspondence method and record its actual evidence class;
- do not treat generated conformance vectors as a universal correspondence proof without a sound exhaustive-coverage argument;
- bind formal subjects, complete obligations, dependencies, translations, and actual admitted production artifacts;
- maintain an explicit obligation ledger from every formal assumption relevant to a claimed result to the production mechanism, checked capability, or named trusted premise that discharges it;
- require an atomic action in the formal model to correspond to an executable authoritative boundary providing the same required atomicity/serialization property rather than merely to similarly named source code;
- preserve reverse correspondence metadata sufficient to interpret formal counterexamples in terms of admitted semantic objects and their source bindings;
- where a supported generated backend is included in the claimed profile, exercise an actual counterexample through the source-level explanation path, distinguishing the modeled violation from a separately evidenced production correspondence failure;
- deterministic certificate regeneration;
- dependency-aware incremental rebuild;
- build gates for unresolved obligations/stale semantic IDs;
- deterministic executable plan generation;
- exact browser artifact closure;
- root release report;
- ordinary JVM/Node/Chromium/adversarial tests retained.

Gate:

- release cannot ship a stale certified plan after semantic dependency change;
- formal checker story applies to actual production acceptance rather than a disconnected model;
- every assumption needed by a requested formal claim is either discharged by applicable production evidence with matching subject/scope or remains visible as a trusted/open premise; model success cannot hide an unconnected implementation obligation;
- source repair after a formal failure regenerates the semantic/formal artifacts and reruns their checks; editing a generated formal artifact cannot establish production correspondence;
- proof sidecars remain excluded from production semantics;
- required unavailable or incomplete checking blocks the requested admission without silently weakening its assurance profile.

This stage strengthens the operational release discipline already required by Stages 1–8. It does not introduce artifact freshness, real-browser gates, or production-path enforcement for the first time.

Stage 18 – Multi-node realization

Goal:

> Realize the already-defined host/authority/invalidation semantics across several actual application nodes.

Required work:

- multiple Aleph nodes;
- concrete routing/load balancing;
- distributed Live invalidation/interest implementation;
- host replacement;
- duplicate/lost publication behavior;
- reconnect through another node;
- process loss;
- progression-safe authoritative reread;
- multi-node adversarial/load/fault tests;
- update root assumptions/contracts from actual implementation evidence.

Gate:

- physical node replacement never changes semantic authority or command identity;
- node-local state may disappear without losing authoritative model truth;
- users on different nodes converge through authority under the declared acquisition/adoption policy, recovery actions, and environmental premises;
- cross-node convergence does not depend on delivery of each advisory, including when the only advisory for a change is lost;
- retry/duplicate handling follows the model's declared durable identity and retention contract where claimed;
- crash after commit before publication exercises the actual recovery path;
- another-node current-state reads do not invent command attribution;
- reconnect preserves authenticated principal binding and selective adoption;
- direct participant messaging, if added, remains semantically distinct from advisory invalidation;
- multi-node assumptions are incorporated into the root assurance report.

Worked evolving slice – the same HumanHelp Claim through the roadmap

The stages are easier to read if one operation survives all of them. The following is intentionally one evolving Claim slice rather than eighteen toy applications. Names are illustrative. In particular, functions whose names contain `admit`, `prepare`, `issue`, or `check` stand for the corresponding supported checking boundary described in the preceding Parts; a raw map with the same fields is not thereby an admitted value.

These source blocks are contract sketches, not verified Biff, Gesso, HumanHelp, or formal-tool APIs. Their identifiers denote distinct checked results where the text says they are admitted; evaluating a sketch or assigning a `def` does not implement a checker, run a proof, or grant admission.

The example also keeps three kinds of identity separate throughout:

- the semantic identity of the Claim operation and its derived artifacts;
- a durable command identity belonging to one submitted semantic command;
- an execution identity belonging to one concrete attempt to realize that command.

None of those is the authenticated principal, the authoritative Request state, or an admitted assembly generation.

At Stages 1-2, Claim is first an authoritative HumanHelp transition running on the supported substrate. The declaration describes the operation's requirements and business owner. Per-command and per-execution identities are deliberately absent from the static operation declaration.

```
(def claim-operation
  {:operation/id :request/claim
   :authoritative-owner :humanhelp/request-model
   :domain-input #{:request/id}
   :trusted-requirements
   #{:principal/current
     :clock/current
     :request/current
     :membership/current
     :organization/current}
   :prepare
   [:load-current-request
    :authorize-current-principal-as-helper
    :derive-open->claimed-transition]
   :commit :humanhelp.request/claim!
   :after-commit :publish-request-changed})
```

A concrete request crosses the trusted server boundary separately. The browser may carry an operation/command envelope, but it cannot grant the principal, current Request, clock, role, authority, basis, or provenance used by the model. The execution identity is created for this realization attempt rather than being part of the operation's meaning.

```
(def admitted-claim-execution
  (trusted-server/admit-operation!
   {:operation claim-operation
    :transport-envelope
    (protocol/read-command-envelope http-request)

    ;; Trusted bindings are reconstructed on the server side.
    :authenticated-principal
    (session/current-principal http-request)
    :clock
    (system/current-clock system)
    :route-selection
    {:request/id (routes/request-id http-request)}
    :authoritative-input
    {:request/current
     (request-model/read-current db (routes/request-id http-request))
     :membership/current
     (membership-model/read-for-principal
      db (session/current-principal http-request))
     :organization/current
     (organization-model/read-for-request
      db (routes/request-id http-request))}

    ;; Distinct from the durable command identity carried by an admitted
    ;; semantic command.
    :execution-id
    (execution/new-id!)}))
```

Only after this boundary has checked operation identity, route/transport correspondence, authenticated principal binding, structural command requirements, and the operation's trusted requirements does the HumanHelp model receive the admitted invocation. The model remains the sole owner of the Claim business transition. Trusted observations and commit-time guards must still obey the model's consistency contract; a pre-commit read alone does not make future authorization or Request status true at commit.

By Stage 3, the same operation gains a portable semantic description. Knowledge is not represented merely as a bag of convenient fact names: the projected role must be able to justify each fact through a declared input, communication, authoritative observation, deterministic derivation, or explicit trusted assertion.

```clojure
(def claim-choreo
  {:roles #{:helper :request-authority}
   :operation :request/claim

   :knowledge
   {:helper
    {:request/status
     {:source :authoritative-observation
      :observation :request/current
      :scope :request/card}
     :request/progression
     {:source :authoritative-observation
      :observation :request/current
      :scope :request/card}}

    :request-authority
    {:principal/current
     {:source :trusted-assertion
      :boundary :trusted-server}
     :request/current
     {:source :authoritative-observation
      :observation :request/current}}}

   :branch
   {:when [:known= [:helper :request/status] :open]
    :continue :request/claim-binding
    :otherwise :claim/unavailable}

   ;; Availability comes from model-owned preparation with trusted current
   ;; membership/organization context. Observing :open is necessary here,
   ;; never sufficient to assert eligibility or server authorization.
   :binding {:source :model-owned-preparation
             :operation :request/claim
             :render-control :only-when-bound}

   :authoritative-observation :request/current})

(def helper-projection
  (choreo/project claim-choreo :helper))
```

Projection may take this branch only when the helper can justify its observation. The rendered Claim control also requires a model-owned eligible binding, and a later trusted server admission rechecks current principal and model preconditions. The choreography does not create another copy of the HumanHelp Claim policy; it describes interaction around the authoritative operation and the knowledge needed to project it safely.

Stages 4-5 make that projection executable in the browser and connect successful or uncertain execution to settlement, advisory invalidation, and permitted authoritative reacquisition. Provisional presentation begins from an admitted observation/basis and a semantic command; neither a client-supplied basis-shaped map nor a later current-state reread proves an uncertain command's outcome.

```
(def claim-plan
  {:semantic-operation :request/claim
   :basis-requirement
   {:kind :admitted-authoritative-observation
    :scope :request/card}
   :command-requirement
   {:kind :admitted-semantic-command
    :operation :request/claim}
   :provisional-effect :request/show-claimed-by-self
   :authoritative-settlement #{:accepted :rejected}
   :delivery-uncertainty {:participant-knowledge :outcome-unknown
                          :recovery :permitted-authoritative-reread}
   :managed-fragment :request/card})

(def request-change-advisory
  {:kind :advisory/invalidation
   :resource :request/card
   :request/id request-id
   :remote-progression 42
   :acquisition :request/current
   :adoption-permission :from-selected-humanhelp-policy})
```

Here `:remote-progression` is advisory data for the scoped resource, not a newly admitted authoritative observation. This sketch intentionally leaves remote lifecycle adoption to the HumanHelp decision required by Stage 7; for a newly created remote Request the explicit board Refresh remains the declared visible adoption action. The advisory contains neither the authoritative Request value nor permission to adopt unrelated Requests. A later permitted and admitted `:request/current` observation can supersede provisional presentation, but current state and command-outcome knowledge remain distinct. Resource instance ownership is also checked independently, so a callback from an obsolete browser resource cannot settle or mutate its successor.

At Stages 6-8, the Claim path must close as part of the actual application. The closure boundary does substantially more than confirm that a bag of names exists. It checks the concrete edges among rendered control, generated plan, transport, physical route, prepared trusted server operation, model operation, settlement, authoritative acquisition, and the exact generated browser artifact for one admitted assembly generation.

```
(def claim-closure-requirement
  {:operation :request/claim
   :path
   [:view/request-card-claim
    :plan/request-claim
    :transport/request-claim
    :route/request-claim
    :server/request-claim
    :model/request-claim
    :settlement/request-claim
    :acquisition/request-card]
   :required-server-capabilities
   #{:principal/current
     :clock/current
     :request/current
     :membership/current
     :organization/current}
   :required-browser-manifest-entry :plan/request-claim})

(def admitted-humanhelp-generation
  (application/admit-generation!
   {:canonical-assembly
    (humanhelp/application-assembly config)
    :inspection-targets #{claim-closure-requirement}
    :prepared-server-realizations prepared-server-realizations
    :routes humanhelp/routes
    :acquisitions humanhelp/acquisitions
    :generated-browser-artifact generated-browser-artifact}))

(def claim-closure-report
  (application/explain-path
   admitted-humanhelp-generation
   :request/claim))
```

The `:inspection-targets` set in this sketch requests explanation of one Claim path; it does not define the application's required product capabilities. The canonical assembly and independent Part VIII product acceptance input still require all six lifecycle operations and the remaining rich-board behaviors before Stage 7 or the operational milestone can pass. Admission returns a checker-owned assembly generation whose report identifies checked edges, exact artifact correspondence, supplied capabilities, and retained runtime or trusted premises; a self-declared `:valid? true` is insufficient. HumanHelp then exercises that same production assembly in real Chromium:

```
rendered Claim control
  -> generated semantic command
  -> real HTTP route
  -> trusted server admission
  -> HumanHelp model commit
  -> settlement / advisory publication
  -> authoritative reread when permitted
  -> resulting Request card
```

Stages 9-15 add assurance around these same semantic subjects rather than replacing them with proof-only twins. A certificate is never represented here as just a memorable ID. Each admitted certificate binds a normalized subject, propositions, dependency identities, assumptions, and checked derivations under a specific certification profile.

```
(def normalized-claim-rule
  (rules/normalize
   {:rule/id :request/helper-may-claim
    :source :humanhelp/request-model/claim-policy
    :subject :request
    :when
    [:and
     [:= [:request/status] :open]
     [:= [:membership/status] :active]
     [:= [:request/organization-id] [:membership/organization-id]]
     [:organization/allows-claim?
      [:organization/current]
      [:membership/current]
      [:request/current]]]
    :result :claim/allowed}))

(def admitted-claim-rule-certificate
  (certificate/issue-rule-set!
   {:subject normalized-claim-rule
    :profile :gesso/rules-v1
    :dependencies
    {:request-schema request-schema-certificate
     :membership-relationship membership-certificate
     :organization-policy organization-policy-certificate
     :organization-allows-claim-primitive
     admitted-organization-claim-primitive-contract}
    :assumptions
    #{{:assumption/id :humanhelp/current-membership-read-is-authoritative
       :proposition :membership/current-read-authoritative}}
    :derivations claim-rule-derivations}))

(def admitted-claim-operation-certificate
  (certificate/issue-operation!
   {:subject (operation/normalize claim-operation)
    :profile :gesso/operation-v1
    :dependencies
    {:claim-rule admitted-claim-rule-certificate}
    :required-guarantees
    #{:trusted-principal-binding
      :authoritative-current-request
      :current-membership-and-organization
      :open->claimed-transition-owned-by-model}
    :derivations claim-operation-derivations
    :implementation-assumptions claim-implementation-assumptions}))
```

The rule is an illustrative normalized view of the single HumanHelp model-owned policy, not another browser-maintained decision. Its organization predicate is admitted only under a typed, totality/partiality-declared primitive contract and the relevant dependencies. The certificate does not prove that an arbitrary implementation of `humanhelp.request/claim!` evaluates that rule or protects commit-time preconditions: that correspondence retains its separately checked or explicitly trusted implementation obligation.

The Choreo, helper projection, executable plan, and application assembly are then certified against the exact admitted dependencies they consume. Their serialized forms may contain identities, but those identities are lookup/correspondence data; they are not substitutes for re-admission of the certificate and its dependency closure.

```
(def claim-certification-closure
  (certificate/compose-application!
   {:assembly-generation admitted-humanhelp-generation
    :certification-profile :humanhelp/claim-assurance-v1
    :dependencies
    {:operation admitted-claim-operation-certificate
     :choreography admitted-claim-choreo-certificate
     :projection admitted-helper-projection-certificate
     :executable-plan admitted-claim-plan-certificate}
    :required-propositions
    #{:claim/authority-remains-model-owned
      :claim/helper-projection-non-escalating
      :claim/plan-settlement-complete
      :claim/application-path-closed}}))

(explain/certificate claim-certification-closure)
```

Changing the normalized Claim rule changes its semantic identity and invalidates the dependent Claim certificate closure unless an explicitly supported compatibility relation proves the required proposition is preserved. It does not invalidate unrelated Search evidence merely because both live in the same application.

Stages 16-17 connect the stable production-used proof rules to an independent formalization and then bind the applicable formal result back to the production subjects that release admission will actually ship. Different correspondence claims remain different: checker/admission correspondence does not silently stand in for execution trace/refinement.

```
(def claim-formal-correspondence
  {:formal-subject formal/request-claim
   :production-subject
   (certificate/subject admitted-claim-operation-certificate)

   :admission-correspondence
   (formal/check-admission-correspondence!
    {:formal-rules formal/gesso-kernel
     :production-checker production/gesso-kernel
     :subject (certificate/subject
               admitted-claim-operation-certificate)})

   :execution-refinement
   (formal/check-execution-refinement!
    {:formal-transition formal/request-claim-transition
     :production-realization humanhelp.request/claim!
     :observable-boundary :authoritative-request-transition
     :correspondence-evidence claim-model-correspondence-evidence
     :assumptions claim-execution-correspondence-assumptions})})

(def admitted-claim-release
  (release/admit!
   {:assembly-generation admitted-humanhelp-generation
    :certification claim-certification-closure
    :formal-correspondence claim-formal-correspondence
    :browser-artifact (application/browser-artifact
                       admitted-humanhelp-generation)
    :real-browser-evidence claim-chromium-evidence
    :implementation-evidence claim-production-evidence}))
```

The `:correspondence-evidence` above must be a checked witness for the stated production relation or an explicitly classified trusted premise compatible with the requested profile. The name `check-execution-refinement!` does not supply a proof by itself, and admission correspondence alone cannot establish this execution claim. If the requested profile requires a proof and has only a trusted premise, admission fails. A successful release result concerns the exact admitted subjects and shipped artifact; a theorem about a neighboring formal subject, a certificate for a predecessor semantic identity, or a Chromium run against different bytes

cannot substitute for them.

Finally Stage 18 changes physical topology, not Claim's semantic owner or the meaning of any earlier identity. A command can be admitted on node A, commit through shared authority, cause an advisory to be delivered through node B, and later be reread through node C. The current-state observation and the knowledge of the original command outcome remain separately represented.

```
(def multi-node-claim-observation
  {:semantic-operation :request/claim
   :command-id admitted-command-id
   :execution-id node-a-execution-id
   :submitted-via :aleph/node-a

   :authoritative-commit
   {:request/id request-id
    :progression 42}

   :advisory-delivered-via :aleph/node-b

   :current-state-observation
   {:admitted-via :aleph/node-c
    :request/id request-id
    :request/status :claimed
    :request/helper current-helper
    :progression 42}

   ;; This does not become :committed merely because current state happens to
   ;; match the command's intended postcondition.
   :command-outcome-knowledge
   (or admitted-command-outcome-evidence
       :outcome-unknown)})
```

Across all eighteen stages, the operation becomes better specified, better realized, better checked, and better evidenced. It never becomes a browser-owned transition, a proof-owned business rule, a certificate-by-label, or a node-owned fact. Each stronger category is obtained only through its declared construction or admission boundary, and each piece of evidence remains scoped to the exact subject it actually establishes. That continuity is why the roadmap is an accumulation of guarantees rather than a succession of parallel implementations.

Stage dependency summary

The numbered stages express a default delivery order and a useful accumulation of evidence. They are not a universal strict dependency chain.

For roadmap reasoning the design distinguishes:

- **semantic prerequisite** – a later claim genuinely depends on an earlier semantic contract;
- **implementation priority** – work should normally be delivered earlier even when another activity could technically begin;
- **exploratory feedback** – bounded work may be done early to expose missing semantics or assumptions;
- **optional early hardening** – low-cost later mechanisms may be pulled forward when they close a concrete current hole;
- **completion dependency** – a named milestone cannot pass until the listed acceptance condition is satisfied.

The default project priority remains:

```
Stages 1-2
    substrate + authoritative model foundation
        |
Stages 3-5
    Choreo + browser + Live executable semantics
        |
Stages 6-8
    whole-application closure + HumanHelp + generic adversarial proof-by-use
        |
Stages 9-15
    compositional certification + typed downstream language + mutation/explanation
        |
Stages 16-17
    independent formalization + production correspondence/release

After sufficient executable/authority foundations exist:
    \--> bounded multi-node prototype may provide exploratory feedback

Stage 18:
    production multi-node realization under its own acceptance requirements
```

This diagram states delivery priority, not that every theorem in Stages 16-17 is a semantic prerequisite for learning from a small real multi-node prototype. Nor does early topology exploration waive any requirement of production multi-node completion.

Low-cost certificate/identity hardening may be added earlier when it helps close Stages 1-8. Full proof-system expansion should not delay making the actual application structurally closed and usable.

If exploratory multi-node work exposes a missing distinction in identity, authority, progression, invalidation, or recovery, the semantic design is corrected at its owning layer and downstream proof assumptions are updated. The prototype does not gain authority merely because it exists earlier in the schedule.

Milestone relationship

Completion scope	Principal stage evidence	Additional requirement
Operational application	Stages 1–8	All Part XI operational gates, including complete required HumanHelp behavior
Compositional certification	Stages 9–15 over the executable foundation	Named certified scope, real Claim pilot, full obligations and explicit assumptions
External formal verification	Stages 16–17 over the selected stable certification scope	Applicable production correspondence, not only a theorem about a separate model
Production multi-node	Stage 18 over sufficient operational foundations	Real topology and fault evidence; external formal completion is not a universal prerequisite

The table is navigational. Part XI owns the completion requirements. A stage's partial implementation or exploratory prototype is not a substitute for those gates. Scheduling can overlap while the evidence needed for a claim remains mandatory.

Avoiding apparent dependency cycles

Early stages may establish integration seams whose full checks are implemented later: the mandatory lifecycle preflight boundary precedes full application closure, and an initial semantic verifier precedes the smaller proof kernel and proof-producing elaborator. The executable semantic core persists through these stages; it is not an interim name for either checker. Reports state exactly what exists at each checkpoint. Placeholder checks cannot be labeled as the final stronger guarantee.

Likewise, explainability, release integrity, and controlled-defect tests begin when their first relevant production boundary exists. Their later dedicated stages expand coverage; they do not permit earlier work to ship silently broken or unreviewable. A new failure class may send work back to its owning semantic layer without creating a second application-specific workaround.

Cross-stage acceptance gates

Several gates apply repeatedly across the roadmap.

Gate A – no silent downgrade

If source declares stronger semantics, missing realization data rejects or renders explicitly unavailable.

It never silently weakens into ordinary transport semantics. Explicit unavailability can be structurally valid while still failing a milestone that requires the capability to be usable.

Gate B – same semantic object in proof, test, and production

The object whose properties are reasoned about is the object production executes or consumes.

No richer test-only semantic machine substitutes for production. Where a formal object is a different representation, the declared checked correspondence or explicit trust premise must connect it to the actual production object.

Gate C – source ownership remains single

Semantic facts have one owner.

Other layers derive or validate.

They do not maintain plausibly synchronized copies.

Gate D – security remains trusted-side

No browser-supplied role, principal, certificate, plan, basis, provenance, or DOM state can increase server authority.

Gate E – exact artifact correspondence

Diagnostic/proof/artifact correspondence is exact where exactness is claimed.

No timestamp/name/best-effort guessing.

Gate F – dynamic facts remain dynamic

Preflight does not pretend to prove current authorization, network availability, or future environment events. It still checks the structural presence and compatibility of required declared handling/recovery paths for supported dynamic conditions that can strand an operation, as specified in #req-preflight-dynamic-handling-coverage. This establishes declared coverage; runtime enforcement and fault tests establish implementation behavior under stated premises. It is not a prediction of dynamic facts or an unconditional recovery/liveness guarantee.

Gate G – real browser remains a release gate

Formal semantic work does not eliminate real Chromium/HTMX integration testing.

Gate H – malformed downstream programs stop early

Once enough information exists to know a structural contradiction, later runtime layers are not allowed to ”try anyway.”

Gate I – representation is not admission

A same-shaped map, nominal-looking value, discriminator key, deserialized record, or browser-supplied label cannot acquire stronger semantic status merely by representation.

Where stronger status matters, the supported path must establish it through the appropriate checker, trusted acquisition, authorization, certificate-validation, or other declared admission boundary. Nominal or opaque types may make that boundary harder to misuse accidentally, but their names are not proofs and they do not create runtime authority.

Gate J – independent capability completeness

A closed graph cannot redefine the product by omission. Required capabilities and their positive outcome tests remain independent of the exposed-operation registry. Unresolved required product choices block the affected full milestone.

Gate K – participant knowledge and authoritative outcome remain distinct

A timeout or retired execution does not prove a submitted command failed. Current-state acquisition can supersede provisional presentation without proving which command produced that state. Recovery and diagnostics preserve the remaining uncertainty.

Gate L – scope and permission survive recovery

Advisory requirements, observations, and adoption are scoped. A newer local result cannot admit unrelated remote Requests or erase their outstanding debt. An authorized explicit Refresh can acquire changes even if no advisory arrived and thus no debt was recorded. Queued successors, reconnect, and stale-plan recovery require their declared acquisition/adoption permission.

Gate M – evidence scope survives composition and release

Every claim retains its subject, assumptions, evidence class, and applicable implementation or admitted assembly generation. Omitted obligations, copied identities, incomplete checks, or tests on a different artifact cannot establish the requested gate. Finite models and conformance vectors retain the limits of their actual evidence.

Gate N – correctness-relevant policy remains visible

The correctness-relevant semantics of an application must be inspectable as named rules, operations, obligations, and explicit premises, with source and realization correspondence. The audit surface should normally be much smaller than the implementation, but a large genuine policy inventory is not a defect and no universal size ratio is an acceptance gate.

A condition that influences an admitted assurance cannot gain the appearance of mechanical coverage merely because it is buried in ordinary Clojure. At the boundaries where Gesso can identify such influence, the condition is represented as admitted semantics, derived from admitted semantics, or exposed as an opaque/trusted premise.

Completeness is claimed only for the stated generated or closed checked subset. Arbitrary host bodies outside that subset carry named implementation-conformance premises and inspection limits; no absence of hidden policy is inferred from an empty escape report. The selected assurance profile decides whether such a premise is admissible or must be discharged by stronger evidence.

The business-rule inventory, operation explanations, dependency views, opaque-premise report, and assurance coverage are derived from the same admitted semantic objects used by production. They are not a second policy database.

Gate O – formal assumptions become production obligations

A formal model may abstract implementation detail, but an assumption essential to its result creates a production obligation.

If the model treats a guard and mutation as one atomic action, the production realization must provide the corresponding atomic/serializable property, or the gap remains an explicit trusted premise. The same rule applies to authority, freshness, command identity, retry, settlement, progression, and other modeled semantics.

A successful model check therefore cannot by itself establish that arbitrary Clojure implements the checked model. Requested assurance requires the declared correspondence from the formal subject and assumptions to the actual admitted production realization.

Gate P – intent review does not impersonate proof

Human or LLM review may identify a well-formed rule as surprising, asymmetric, apparently incomplete, or otherwise suspicious.

Such review is advisory. It cannot alter semantic execution, reject a valid program as a proof result, discharge an obligation, or weaken an admitted rule.

When a reviewer identifies a real business requirement, that requirement is promoted into an explicit semantic rule, invariant, contract, or other checked object. Only the resulting admitted semantics gain mechanical force.

Worked failed release – one green-looking candidate stopped by the final gates

Consider a candidate assembled under deadline pressure. Every individual mistake is plausible precisely because some nearby artifact is green. The final gates prevent those local successes from being promoted into a stronger release claim.

```
(def broken-candidate
  {:operation :request/claim

   ;; Old generated JavaScript survived a semantic-plan change.
   :assembly {:plan :plan/request-claim-P24
              :browser-artifact {:identity "sha256:old-bytes"
                                 :declares-plan :plan/request-claim-P23}}

   ;; A handler silently falls back when the required trusted clock is absent.
   :server {:requires #{:principal/current :clock/current}
            :supplied #{:principal/current}
            :fallback :ordinary-htmx-handler}

   ;; The browser posts authority-shaped values and the handler trusts them.
   :invocation {:principal "helper-17"
                :role :helper
                :basis 42}

   ;; Timeout recovery sees current state and invents command attribution.
   :recovery {:current-request {:status :claimed :progression 43}
              :conclusion :timed-out-command-definitely-succeeded}

   ;; Reconnect automatically adopts every newer Request despite selective policy.
   :continuity {:observation-frontier 47
                :remote-frontier 52
                :action :adopt-all-visible-requests}

   ;; The exposed operation registry accidentally omits Reassign, then calls the
   ;; smaller registry the complete lifecycle.
   :capabilities {:required #{:claim :unclaim :mark-on-the-way
                              :complete :cancel :reassign}
                  :exposed #{:claim :unclaim :mark-on-the-way
                             :complete :cancel}}

   ;; Formal and browser evidence concern neighboring, not admitted, artifacts.
   :evidence {:certificate-subject :plan/request-claim-P23
              :production-plan :plan/request-claim-P24
              :chromium-tested-artifact "sha256:previous-release"
              :release-artifact "sha256:old-bytes"}

   ;; Preflight also overclaims a future fact, with no checked recovery contract.
   :preflight {:network-will-remain-available true
               :network-loss-recovery nil}})
```

The candidate is not repaired by choosing one late gate and calling every defect a release error. Each contradiction stops where enough information first exists:

Defect	Earliest owning rejection	Final gate reinforced
Missing trusted clock with fallback	capability/assembly preflight	A – no silent downgrade
Proof/test subject differs from production	certificate/application composition	B – same semantic object
Browser-provided principal/role increases authority	trusted server admission	D – security trusted-side
P23 artifact presented for P24 plan	browser/application correspondence	E – exact artifact correspondence
Preflight predicts future network availability	preflight contract	F – dynamic facts remain dynamic
Release lacks Chromium evidence for shipped bytes	release admission	G – real browser release gate
Known malformed closure is allowed to run	application preflight	H – stop malformed programs early
Authority-shaped browser map treated as admitted value	trusted admission	I – representation is not admission
Reassign disappears because registry omitted it	independent product completeness check	J – independent capability completeness
Current state is treated as proof of timed-out command outcome	recovery semantics	K – knowledge differs from outcome
Reconnect adopts outside declared policy/scope	acquisition/adoption check	L – scope survives recovery
Certificate/test identities copied onto different artifacts	release composition	M – evidence scope survives release

Gate C is checked throughout as well: fixing the candidate must not introduce a second copy of the Claim policy in the browser, release manifest, or proof layer. The repair points back to the HumanHelp model/rule owner and regenerates or rechecks dependent artifacts from there.

A correct repair therefore rebuilds and re-admits the relevant objects through their owning boundaries; it does not mutate `broken-candidate` until its fields look persuasive.

First the application is reconstructed from the canonical source-owned assembly. The required trusted capabilities must actually resolve, and the browser artifact must be generated from the current normalized assembly rather than renamed to match it. The recovered assembly's source-owned dynamic failure contract must also pass structural preflight before any application listener serves; the product acceptance requirement comes from the independent HumanHelp spec rather than the application's exposed operation list.

```
(def repaired-generation
  (let [assembly
        (humanhelp/application-assembly config)

        prepared-server
        (trusted-server/prepare!
         {:assembly assembly
          :providers
          {:principal/current session-principal-provider
           :clock/current system-clock-provider
           :request/current request-model-provider
           :membership/current membership-model-provider
           :organization/current organization-model-provider}})

        browser-artifact
        (browser-build/generate!
         {:assembly assembly})]

    (application/admit-generation!
     {:canonical-assembly assembly
      :prepared-server-realizations prepared-server
      :generated-browser-artifact browser-artifact
      :required-product-acceptance
      humanhelp/independent-product-acceptance})))
```

That independent acceptance specification covers all six lifecycle operations, creation, details, response flow, search, sorting, visibility, Refresh, and the selected remote lifecycle policy. The sketch is conditional on those product decisions being recorded and evidenced; the current unfinished application cannot be declared complete merely by evaluating this expression.

The request path then treats browser input as an untrusted representation. The server binds the authenticated principal and current dynamic facts itself, checks that route, semantic command, and selected Request correspond, and produces an admitted execution only if the declared operation requirements are satisfied.

```
(defn handle-claim! [system http-request]
  (let [command-envelope
        (protocol/read-command-envelope http-request)

        admitted-execution
        (trusted-server/admit-operation!
         {:assembly-generation repaired-generation
          :operation :request/claim
          :transport-envelope command-envelope
          :authenticated-principal
          (session/current-principal http-request)
          :clock
          (system/current-clock system)
          :route-selection
          {:request/id (routes/request-id http-request)}
          :authoritative-input
          {:request/current
           (request-model/read-current
            (:db system)
            (routes/request-id http-request))
           :membership/current
           (membership-model/read-for-principal
            (:db system)
            (session/current-principal http-request))
           :organization/current
           (organization-model/read-for-request
            (:db system)
            (routes/request-id http-request))}
          :execution-id (execution/new-id!)})]

    (request-model/execute-claim!
     system
     admitted-execution)))
```

Notice what is **not** repaired by this code: a client field named `:principal`, `:role`, `:basis`, `:provenance`, or `:certificate` still grants nothing. A command identity carried by the semantic envelope identifies the command after its protocol/admission checks; it does not authenticate the actor or establish current authoritative state. The model must revalidate its claim guards in the declared atomic transition; these reads are trusted inputs, not an immunity from concurrent changes.

Recovery and continuity are repaired at their own semantic boundaries. Current state can supersede provisional presentation without fabricating knowledge of the lost command outcome, and a reconnect preserves advisory debt until the declared scope/adoption policy permits reacquisition. An advisory can be lost altogether: the permitted explicit board Refresh must independently acquire current authority under the selected HumanHelp policy.

```
(def repaired-recovery-policy
  (application/declared-recovery repaired-generation))
```

The admitted recovery contract must preserve command-outcome uncertainty on lost delivery, record scope-compatible advisory debt on reconnect, and let a permitted explicit board Refresh acquire current authority without an advisory. It does not use reconnect to grant automatic adoption. Its structural coverage and the concrete implementations must be checked during assembly admission and fault tests; writing a similar map after the fact would establish nothing.

For an illustrative later release requesting compositional and external formal assurance, certification/formal/browser evidence must be regenerated or re-admitted for the exact current subjects. Operational application completion under Stages 1–8 does not require completion of the later formal milestone. Matching identifier strings are not enough.

```
(def repaired-assurance
  (let [claim-certification
        (certificate/compose-application!
         {:assembly-generation repaired-generation
          :profile :humanhelp/claim-assurance-v1
          :dependencies
          {:operation admitted-claim-operation-certificate
           :choreography admitted-claim-choreo-certificate
           :projection admitted-helper-projection-certificate
           :plan admitted-claim-plan-certificate}})

        formal-correspondence
        (formal/check-production-correspondence!
         {:certification claim-certification
          :production-checker production/gesso-proof-kernel
          :formal-rules formal/gesso-proof-kernel
          :required-claims
          #{:admission-correspondence
            :claim-execution-refinement}})

        shipped-artifact
        (application/browser-artifact repaired-generation)

        chromium-evidence
        (browser-test/run-real-chromium!
         {:assembly-generation repaired-generation
          :artifact shipped-artifact})]

    {:certification claim-certification
     :formal-correspondence formal-correspondence
     :chromium-evidence chromium-evidence}))
```

Each formal result needs its actual mechanism and scope. An admission/checker correspondence result does not establish execution refinement; any missing execution witness remains a named trusted or tested premise, and a requested formal proof profile must reject it when that profile requires a checked result. The Chromium run must exercise the production served bytes and controls for the same admitted assembly generation, with evidence of that byte binding.

Preflight did not predict that the network would stay available. The following is only a readable excerpt of the dynamic-condition handling declared by the canonical assembly and structurally checked before serving; it is neither a new post-admission override nor an unconditional convergence promise:

```
(def dynamic-failure-contract
  (application/declared-dynamic-handling repaired-generation))

;; For example, this checked declaration requires a transport-loss path that
;; preserves command-outcome uncertainty, a post-commit publication-recovery
;; path, and a scoped reacquisition path permitted even if no advisory arrived.
```

Only after those independently owned checks succeed can the requested later release profile make its scoped claim:

```
(def admitted-release
  (release/admit!
   {:profile :humanhelp/external-formal-release-v1
    :assembly-generation repaired-generation
    :certification (:certification repaired-assurance)
    :formal-correspondence (:formal-correspondence repaired-assurance)
    :browser-evidence (:chromium-evidence repaired-assurance)
    :dynamic-handling-evidence
    (application/dynamic-handling-evidence repaired-generation)
    :product-acceptance-evidence
    humanhelp/independent-product-test-results
    :required-gates
    #{:no-silent-downgrade
      :same-semantic-subject
      :single-source-ownership
      :trusted-side-security
      :exact-artifact-correspondence
      :dynamic-facts-remain-dynamic
      :real-browser
      :early-malformation-rejection
      :representation-is-not-admission
      :capability-completeness
      :knowledge-outcome-distinction
      :scope-preserving-recovery
      :evidence-scope-preserving-composition}}))
```

Even here, the final map literal is only the **request** to the release admission boundary. The release claim comes from that boundary's successful checking of its exact subjects and evidence, not from the presence of the listed gate names. The final gates are therefore not a ceremonial checklist: they are repeated invariants enforced by the construction and admission boundaries that prevent a locally plausible artifact from acquiring a global claim it never earned.

Final unified design principle

The unified design is governed by the following final principle:

> A Gesso choreography is a typed description of how roles acting for principals interact with authoritative and evolving facts.
>
> Projection produces local machines whose behavior is justified by available knowledge. Knowledge may arise through declared inputs, communication, authoritative observation, deterministic derivation, or explicit trusted assertion; communication is one knowledge-producing mechanism rather than the definition of distributed knowledge.
>
> HumanHelp models own semantic truth and atomic authoritative transitions.
>
> Durable cross-user facts normally rendezvous through model authority. Gesso Live transports advisory signals that relevant authority changed, and authoritative rereads advance scoped actor knowledge when acquisition/adoption is permitted, without requiring every observer to receive a dedicated participant message. An invalidation does not itself authorize automatic visible adoption.
>
> Concrete Aleph nodes are interchangeable hosts for logical authority projections. Node-local runtime state may disappear without changing authoritative model truth. Command retry and publication recovery require their explicit contracts; physical host interchangeability does not supply exactly- once effects or knowledge of a lost command outcome.
>
> Optimistic state is typed provisional knowledge derived from a known authoritative basis and semantic command.
>
> Views attach typed operations to affordances and determine presentation.
>
> The browser runtime realizes Gesso semantics through a small pure transition system in the executable semantic core. Its safety/refinement claims carry the actual stated scope of their machine checks; finite exploration alone establishes only the explored bound unless a sound abstraction extends it. HTMX remains the ordinary declarative owner of its request/response/OOB/swap/settle lifecycle and is gated or observed only through documented lifecycle boundaries where Gesso semantics require it; a custom HTMX extension is not an architectural requirement. Gesso-owned timers, provisional DOM work, continuity resources, and other direct browser effects form a deliberately narrow FFI/effect shell. Both boundaries are explicit proof assumptions exercised by real Chromium integration tests.
>
> The browser is adversarial. Client-provided roles, principals, authority labels, projected states, capabilities, basis claims, scopes, or provenance cannot grant server authority. Trusted server projections bind authenticated principals, re-authorize reads and writes, and establish durable application facts through public model operations, normally realized with XTDB.
>
> Gesso may assume XTDB wherever doing so materially simplifies correctness; it does not pay speculative complexity for database portability while still keeping semantic concepts such as authority distinct from accidental XTDB representation details.
>
> Incompatible stale projections retire and follow declared recovery or permitted authoritative reconstruction rather than undefined continuation, while continuity remains a separate local rendering concern. Recovery does not silently waive the application's selective adoption policy. Semantic retirement revokes the declared execution or resource-instance ownership rights immediately; physical browser cleanup is best effort and is not a precondition for new logical ownership. Independent admission of useful command-outcome evidence does not restore those revoked rights.
>
> The proved language grows only when a concrete Gesso failure class or required application behavior justifies the additional proof complexity. Usable HumanHelp capabilities, production enforcement, and assembly closure remain independent acceptance claims; a more sophisticated proof system cannot compensate for an omitted required action or an unchecked production path.
>
> Representations, nominal categories, admitted semantic values, certificates, and proofs remain distinct. Gesso may use nominal or opaque values where they prevent consequential same-shaped substitutions, but ordinary downstream web code should remain declaration-first and should not manually reproduce internal proof plumbing. Stronger semantic status comes from the required construction, checking, trusted acquisition, or authorization boundary rather than from a map shape or type name.
>
> The correctness-relevant semantics of an application form an explicit, source-owned audit surface: named rules, operation requirements and effects, authority and disclosure policy, lifecycle transitions, dependencies, and trusted/opaque premises. The surface is presented as compactly as its actual policy permits, with its inspection scope and enough explanatory context for human or heuristic review. Such review can discover candidate mistakes in intent, but only admitted semantic rules and checked evidence participate in mechanical assurance.

The word **proved** is reserved for properties derived by the formal semantics and its machine-checked proof/refinement machinery under named assumptions. Tests remain essential for the trusted shell, external libraries, real browsers, operational scale, and application contracts, but they are not used as a synonym for proof.